基于物理的深度学习

Physics-based Deep Learning

N. Thuerey 等

v0.3 生成式 AI 版 · 在线出版 · 年份待确认
Model: gpt-6.1-sol（本轮）；gpt-6.1-terra（前轮记录）
审核：曹溪能
Updated: 2026-11-06

译本说明

本文件为全书作者候选稿，依据本地 v0.3 Generative AI Edition PDF（461 页），覆盖前言、第 1–48 章及 44 项书目。独立审校、源码包独立重建和主控最终验收尚未完成；源代码仅静态转写核对，未执行训练或下载数据。本轮恢复与审校使用 gpt-6.1-sol（high）；前轮已核验为 gpt-6.1-terra（high），更早轮次的模型及原版出版年份仍待核实。

目录

第一部分　引言 **1**

第一章　引导示例 **2**
1.1 可微物理 2
1.2 求抛物线的反函数 3
1.3 可微物理方法 5

第二章　概率生成式 AI 方法 **8**
2.1 讨论 12
2.2 下一步 13

第三章　概述 **14**
3.1 动机 14
3.2 分类 16
3.3 展望 17
3.4 实现 17
3.5 模型与方程 18
3.6 使用 phiflow 对 Burgers 方程进行简单正向仿真 20
3.7 Navier–Stokes 正向仿真 25
3.8 优化与收敛 31

第二部分　神经代理模型与算子 **39**

第四章　监督训练 **40**
4.1 问题设置 40
4.2 展望 40

第五章　神经网络架构 **42**
5.1 空间排列 42
5.2 无空间排列的输入 42
5.3 局部与全局 43
5.4 规则、非结构化与逐点数据 43

5.5 层级结构 . . . 45
5.6 谱方法 . . . 46
5.7 注意力与 Transformer . . . 47
5.8 架构总结 . . . 48
5.9 给我看看代码! . . . 48

第六章 翼型周围 RANS 流动的监督训练 49
6.1 概述 . . . 49
6.2 问题表述 . . . 49
6.3 代码来了…… . . . 49
6.4 RANS 训练数据 . . . 50
6.5 网络设置 . . . 51
6.6 训练 . . . 55
6.7 测试评估 . . . 58
6.8 下一步 . . . 61

第七章 监督方法讨论 62
7.1 需要牢记的事项…… . . . 62
7.2 监督训练概要 . . . 64

第三部分 物理损失 65

第八章 物理损失项 66
8.1 使用物理模型 . . . 66
8.2 变体 1：显式表示的残差导数 . . . 67
8.3 变体 2：神经网络表示的导数 . . . 68
8.4 阶段总结 . . . 68

第九章 学习 Helmholtz–Hodge 分解 70
9.1 求解 Navier–Stokes 方程 . . . 70
9.2 建立离散 PDE . . . 71
9.3 神经网络训练 . . . 72
9.4 训练 . . . 77
9.5 使用新输入测试 . . . 77
9.6 更严格的测试：带障碍物的流体仿真 . . . 79

第十章 下一步 83

第十一章 使用 PINN 优化 Burgers 方程 84
11.1 问题表述 . . . 84
11.2 预备工作 . . . 84

11.3 损失函数与训练 . . . 88
11.4 评估 . . . 89
11.5 下一步 . . . 95

第十二章 物理损失讨论 96
12.1 泛化? . . . 96
12.2 总结 . . . 96

第四部分 可微物理 98

第十三章 可微物理简介 99
13.1 可微算子 . . . 99
13.2 Jacobian 矩阵 . . . 100
13.3 通过可微物理算子学习 . . . 100
13.4 一个实际例子 . . . 101
13.5 隐式梯度计算 . . . 104
13.6 可微物理阶段总结 . . . 105

第十四章 使用可微物理梯度优化 Burgers 方程 106
14.1 初始化 . . . 106
14.2 梯度 . . . 107
14.3 优化 . . . 109
14.4 更多优化步骤 . . . 110
14.5 物理信息与可微物理重建比较 . . . 114
14.6 下一步 . . . 116

第十五章 目前进展顺利——初步讨论 117
15.1 与现有数值方法的兼容性 . . . 117
15.2 离散化 . . . 117
15.3 效率 . . . 117
15.4 效率（续） . . . 118
15.5 总结 . . . 118

第十六章 可微流体仿真 119
16.1 物理模型 . . . 119
16.2 问题表述 . . . 119
16.3 开始实现 . . . 120
16.4 批量仿真 . . . 120
16.5 梯度 . . . 122
16.6 优化 . . . 123
16.7 重模拟 . . . 125

16.8 结论 . . . 127
16.9 下一步 . . . 127

第十七章 将可微物理纳入神经网络训练 129
17.1 交换顺序 . . . 130
17.2 循环求值 . . . 130
17.3 神经网络与求解器的组合 . . . 131
17.4 方程形式 . . . 132
17.5 通过求解器步骤反向传播 . . . 132
17.6 替代方案：噪声 . . . 134
17.7 复杂示例 . . . 134

第十八章 用神经算子减少数值误差 135
18.1 问题表述 . . . 135
18.2 开始实现 . . . 137
18.3 仿真设置 . . . 138
18.4 网络与转换函数 . . . 140
18.5 训练设置 . . . 142
18.6 交错执行仿真与神经网络 . . . 144
18.7 测试评估 . . . 147
18.8 下一步 . . . 153

第十九章 用神经网络求解逆问题 154
19.1 问题表述 . . . 154
19.2 不可压流体的控制 . . . 155
19.3 数据生成 . . . 156
19.4 监督初始化 . . . 158
19.5 用可微物理预训练 CFE . . . 159
19.6 用可微物理端到端训练 . . . 159
19.7 下一步 . . . 162

第二十章 可微物理讨论 163
20.1 整合 . . . 163
20.2 通过交互减少数据偏移 . . . 163
20.3 泛化 . . . 163

第五部分 概率学习 165

第二十一章 概率学习导论 166
21.1 不确定性 . . . 166
21.2 正向还是反向? . . . 166

21.3 基于仿真的推断 167

第二十二章 学习概率分布 170

22.1 基础：训练目标 170
22.2 从无条件到有条件 171
22.3 用归一化流学习分布 171
22.4 实际示例：学习 Gaussian 分布 173
22.5 基于仿射耦合的简单归一化流 175
22.6 神经 ODE：使归一化流连续化 182
22.7 归一化流小结 189

第二十三章 得分匹配 190

23.1 具有解析得分的 Gaussian 玩具数据集 190
23.2 学习得分 193
23.3 Langevin 动力学 197
23.4 退火 Langevin 动力学 200
23.5 得分小结 204

第二十四章 去噪 206

24.1 潜变量模型 206
24.2 完整去噪算法 208
24.3 用 DDPM 训练 210

第二十五章 流匹配 218

25.1 用速度学习流 218
25.2 映射与条件化 219
25.3 实现流匹配 220
25.4 小结 226

第二十六章 并排比较去噪与流匹配 227

26.1 导言 227
26.2 问题陈述 228
26.3 实现与设置 228
26.4 用扩散模型去噪 229
26.5 实现 DDPM 230
26.6 流匹配 233
26.7 实现流匹配 234
26.8 测试评价 235
26.9 量化结果 240
26.10 下一步 241

第二十七章 纳入物理约束 **243**
27.1 引导扩散模型 . . . 243
27.2 物理引导的流匹配 . . . 244
27.3 结合可微物理的得分匹配 . . . 249
27.4 基于物理的扩散模型小结 . . . 252

第二十八章 用可微仿真求解概率逆问题 **253**
28.1 玩具问题设置 . . . 253
28.2 实现概览 . . . 253
28.3 物理系统 SDE . . . 254
28.4 SDE 路径的可视化 . . . 255
28.5 生成训练数据 . . . 256
28.6 训练与滑动窗口法 . . . 260
28.7 采样 SDE 与 ODE 轨迹 . . . 265
28.8 下一步 . . . 269

第二十九章 基于扩散的时间预测 **270**
29.1 条件化 . . . 271
29.2 实现 . . . 273
29.3 骨干网络定义 . . . 274
29.4 方差日程 . . . 278
29.5 扩散模型定义 . . . 278
29.6 训练 . . . 282
29.7 测试数据集 . . . 285
29.8 测试推断 . . . 286
29.9 预测精度 . . . 288
29.10 扩散模型时间预测小结 . . . 292

第三十章 无条件稳定性 **293**
30.1 评价的主要考量 . . . 293
30.2 结构比较 . . . 294
30.3 稳定性判据 . . . 295
30.4 批大小与展开 . . . 295
30.5 小结 . . . 297

第三十一章 基于图的扩散模型 **298**
31.1 扩散图网络（DGN） . . . 298
31.2 图上的扩散 . . . 298
31.3 潜空间中的扩散 . . . 300
31.4 三维机翼周围的湍流 . . . 302
31.5 分布精度 . . . 302

31.6 计算性能 303

第三十二章 扩散图网络的分布精度 304
32.1 实现 304
32.2 逐样本精度 308
32.3 评价分布精度 312

第三十三章 概率学习讨论 318

第六部分 强化学习 319

第三十四章 强化学习导言 320
34.1 算法 320
34.2 近端策略优化 321
34.3 用于逆问题 322
34.4 实现 322

第三十五章 用强化学习控制 Burgers 方程 324
35.1 概览 324
35.2 软件安装 324
35.3 生成数据 325
35.4 强化学习训练 326
35.5 RL 评价 328
35.6 可微物理训练 329
35.7 RL 与 DP 比较 333
35.8 训练进展比较 337
35.9 下一步 338

第七部分 改进梯度 339

第三十六章 尺度不变性与求逆 340
36.1 问题的关键 340
36.2 传统优化方法 341
36.3 梯度下降 342
36.4 拟 Newton 方法 342
36.5 逆梯度 343
36.6 逆仿真器 344
36.7 小结 345
36.8 深入逆仿真器 345

第三十七章 比较不同优化器的简单示例 **347**
37.1 问题表述 . . . 347
37.2 三个空间 . . . 347
37.3 实现 . . . 347
37.4 梯度下降 . . . 349
37.5 Newton 方法 . . . 350
37.6 逆仿真器 . . . 352
37.7 物理空间 . . . 354
37.8 结论 . . . 357
37.9 近似求逆 . . . 357
37.10 下一步 . . . 359

第三十八章 尺度不变的物理训练 **360**
38.1 神经网络训练 . . . 360
38.2 损失函数 . . . 361
38.3 迭代与时间依赖 . . . 361
38.4 SIP 训练实例 . . . 362
38.5 SIP 训练的讨论 . . . 363

第三十九章 用尺度不变更新学习热传导求逆 **364**
39.1 问题表述 . . . 364
39.2 实现 . . . 364
39.3 数据生成 . . . 365
39.4 可微物理与梯度下降 . . . 366
39.5 稳定 SIP 梯度 . . . 366
39.6 神经网络与损失函数 . . . 367
39.7 训练 . . . 368
39.8 评估 . . . 369
39.9 下一步 . . . 372

第四十章 半逆梯度 **373**
40.1 推导 . . . 373
40.2 构造 Jacobian . . . 374
40.3 玩具例子展示性质 . . . 374
40.4 良态情形 . . . 375
40.5 病态情形 . . . 375
40.6 半逆梯度小结 . . . 376

第四十一章 使用半逆梯度的耦合振子 **377**
41.1 逆问题设置 . . . 377
41.2 问题表述 . . . 377

41.3 耦合线性振子仿真 . . . 378
41.4 训练设置 . . . 379
41.5 训练 . . . 380
41.6 评估 . . . 385
41.7 下一步 . . . 387

第四十二章 改进梯度的讨论 388
42.1 处理尺度问题 . . . 388
42.2 计算资源 . . . 388
42.3 小结 . . . 388

第八部分 快速概览主题 390

第四十三章 其他主题 391

第四十四章 模型降阶与时间序列 392
44.1 降阶模型 . . . 393
44.2 时间序列 . . . 393
44.3 端到端训练 . . . 394
44.4 源代码 . . . 394

第四十五章 非结构网格与无网格方法 395
45.1 计算网格类型 . . . 395
45.2 非结构网格与图神经网络 . . . 395
45.3 无网格与粒子方法 . . . 396
45.4 连续卷积 . . . 396
45.5 学习液体动力学 . . . 397
45.6 源代码 . . . 397

第四十六章 生成对抗网络 398
46.1 极大似然估计 . . . 398
46.2 对抗训练 . . . 399
46.3 正则化 . . . 399
46.4 条件 GAN . . . 399
46.5 歧义解 . . . 399
46.6 时空超分辨率 . . . 400
46.7 物理生成模型 . . . 400
46.8 讨论 . . . 400
46.9 源代码 . . . 401

第九部分　书末资料 **402**

第四十七章　展望 **403**

47.1 若干具体方向 403

47.2 结束语 403

第四十八章　记号与缩写 **404**

48.1 数学记号 404

48.2 主要缩写汇总 405

参考文献 **406**

目录校核说明：本目录按本译本实际正文自动生成。冻结源文件物理第 3–10 页的目录将“参考资料”列为第 48 章、“记号与缩略语”列为第 49 章，并列出印刷第 455 页的“证明索引”；其实际正文则以第 48 章“记号与缩略语”结束，之后是未编号书目（物理第 459–461 页），没有所列证明索引。本译本依实际可得正文编号，未补造目录中所列但源文件未提供的章节或索引。

欢迎阅读《基于物理的深度学习》（v0.3，生成式 AI 版）

概要：本文是一份实践导向的综合指南，介绍物理仿真领域的深度学习。我们强调实际应用，而不止于理论：每个概念都配有交互式 Jupyter notebook，帮助你快速上手。除了传统的监督学习，我们还将深入讨论物理损失约束、可微仿真、基于扩散的概率生成式 AI 方法、强化学习及先进的神经网络架构。这些基础正在为下一代科学基础模型铺路。我们正处在快速变革的时代，这些方法有望重新定义计算科学的可能性。

注：v0.3 有哪些新内容？ 这一最新版更进一步，新增了重要的生成建模章节，涵盖去噪、流匹配、自回归学习、物理集成约束以及基于扩散的图网络等前沿技术。我们还专门增加了一节，介绍为物理仿真设计的神经网络架构。所有代码示例均已更新，以利用最新的框架。

接下来的内容

先睹为快，后续章节将展示：

- 如何训练神经网络，使用扩散建模预测翼型周围的流动，从而得到能够替代传统模拟器且性能优于它的概率代理模型。
- 如何将模型方程作为残差，训练表示解的网络，以及如何利用可微仿真改进这些残差约束。
- 如何在逆问题中与完整模拟器开展更紧密的交互。例如，我们将展示如何利用训练循环中的模拟器，绕开标准强化学习技术的收敛问题。
- 我们还将讨论选择合适网络架构的重要性：是考虑全局还是局部相互作用，是采用连续还是离散表示，以及使用结构化网格还是非结构化图网格。

全文将介绍把物理模型引入深度学习的不同方法，即基于物理的深度学习（PBDL）方法。我们将按照集成紧密程度由低到高的顺序介绍这些算法变体，并讨论各方法的优缺点。了解各种技术在哪些场景中特别有用十分重要。

可执行代码，即刻运行 我们以 Jupyter notebook 为重点，它的一大优势是所有代码示例都可以直接在浏览器中运行。你可以修改代码并立即看到结果——试着在浏览器中运行这个引导示例。此外，Jupyter notebook 也是一种文学化编程的形式。

意见与建议

本书中的“书”是指数字文本和代码示例的集合，由慕尼黑工业大学（TUM）的基于物理的仿真研究组维护。如有意见，欢迎联系我们，例如通过传统的电子邮件。如果发现错误，也请告诉我们！我们知道这份文档远非完美，并希望不断改进。提前感谢！另外，我们还维护了一个近期研究论文链接集。

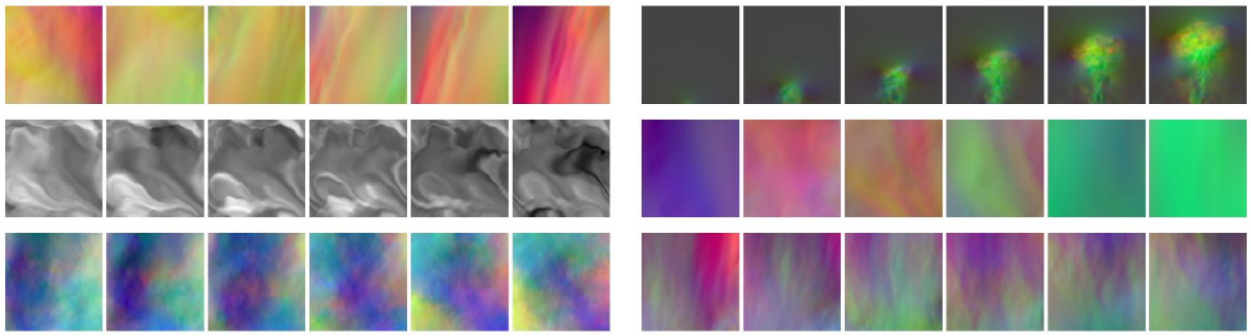


图 1: 数值模拟时间序列的一些可视化示例。本书介绍如何实现将神经网络与数值求解器结合使用的算法。

致谢

没有众多贡献者的帮助，这个项目不可能实现。衷心感谢大家！以下按字母顺序列出：

- Benjamin Holzschuh
- Philipp Holl
- Georg Kohl
- Mario Lino
- Qiang Liu
- Patrick Schnell
- Felix Trost
- Nils Thuerey

特别感谢 Li-Wei Chen、Xin Luo、Maximilian Mueller、Chloe Paillard、Kiwon Um 及所有 GitHub 贡献者！

引用

如果你认为本书有用，请按以下格式引用：

```
@book{thuerey2021pbdl,
  title={Physics-based Deep Learning},
  author={N. Thuerey and B. Holzschuh and P. Holl and G. Kohl
          and M. Lino and Q. Liu and P. Schnell and F. Trost},
  url={https://physicsbaseddeeplearning.org},
  year={2021},
  publisher={WWW}
}
```

开始吧

仿真的未来正在被改写；借助接下来介绍的 AI 和深度学习技术，你将站在这些发展的前沿。让我们开始吧！

第一部分

引言

第一章　引导示例

让我们从一个极度简化的例子开始，它能突出基于物理的学习方法的一些关键能力。假设物理模型是一个非常简单的方程：沿正 x 轴的抛物线。我们也将直接利用这个例子展望概率式“生成式 AI”方法。

尽管问题非常简单，沿 x 轴的每个点都有两个解，即有两个模态，一个位于 x 轴上方，另一个位于下方，如下图左侧所示。如果不加注意，传统学习方法会得到类似中间红线所示的近似，完全偏离目标。采用改进的学习设置，例如使用离散化的数值求解器，我们至少能够准确表示解的一个模态（右侧的绿线）。有趣的是，学习每一点完整分布的方法——下面使用流匹配作为扩散模型的代表——可以捕捉两个模态！

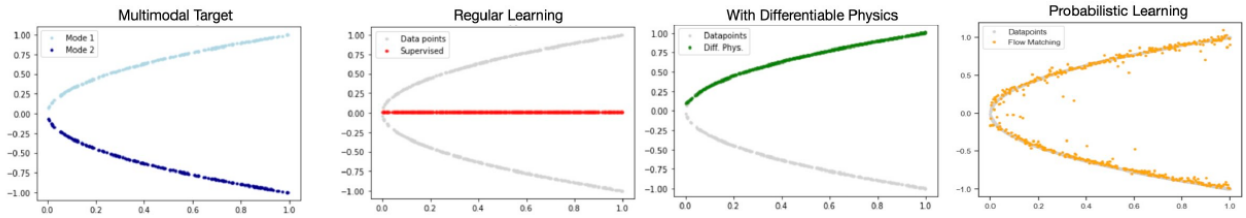


图 1.1: 并列比较：监督训练、可微物理训练与概率训练。

1.1　可微物理

后续章节的关键概念之一是我们所说的可微物理（differentiable physics，DP）。它是指利用模型方程形式的领域知识，并将这些模型的离散版本集成到训练过程中。顾名思义，可微的表述和算子对于这一过程与神经网络训练的结合至关重要。

下面用一个例子说明通过 DP 进行深度学习的性质：我们希望找到一个未知函数 f^*，它以 X 中的元素为输入，生成解空间 Y 中的解，即 $f^*\colon X \to Y$。下文中，我们常用上标 $*$ 表示理想化的未知函数，以区别于不带该上标的离散化、可实现的对应函数。再假设存在一个一般的微分方程 $\mathcal{P}^*\colon Y \to Z$（模型方程），它编码了解的一种性质，例如我们希望匹配的某种现实世界行为。后文中，P^* 常常表示时间演化，但也可以表示守恒定律（例如质量守恒，此时 $\mathcal{P}^*$ 测量散度）。

为了用神经网络 f 学习未知的理想函数 f^*，我们可以收集数据，通过经典的监督训练得到 f。在这一经典设置下，需要从 X 中采样 x，并加入 Y 中相应的解 y，从而构造数据集。例如，可以用经典数值技术获得这些解，然后使用该数据集和经典方法训练神经网络 f。

与监督方法不同，可微物理方法利用了我们通常可以使用物理模型的离散版本 $\mathcal{P}$ 来引导 f 的训练

这一事实。也就是说，我们希望 f 能够感知模拟器 $\mathcal{P}$ 并与之交互。如下面这个非常简单的例子所示，这可以带来根本性的改进；后面还会介绍更复杂的例子。

注意，顾名思义，要使 DP 方法起作用，$\mathcal{P}$ 必须可微。这些以梯度形式出现的微分驱动着学习过程以及与神经网络的集成。

1.2 求抛物线的反函数

为了说明监督方法与 DP 方法的区别，考虑如下简化情形：给定函数 $\mathcal{P} : y \mapsto y^2$，其中 y 位于区间 $[0, 1]$，求未知函数 f，使得对所有 $x \in [0, 1]$ 都有 $\mathcal{P}(f(x)) = x$。例如，当 $x = 0.5$ 时，解为 $\pm\sqrt{0.5}$。[1]注意：为了让问题更有意思，这里用 y^2 表示 $\mathcal{P}$，而不是更常见的 x^2 抛物线。对于这一简单情形，离散化就是在计算机中用浮点数表示 x 和 y。

我们知道 f 的可能解是正平方根或负平方根函数（完整地说，也可以分段组合）。这听起来很容易，因此我们尝试训练神经网络来近似这一逆映射 f。一个显然的起点是以“经典”的监督方式进行，也就是完全基于数据。毕竟，这种方法已被证明是计算机视觉等多种其他应用的有力工具。

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
```

对于监督训练，可以使用问题的求解器 $\mathcal{P}$ 预先计算训练所需的解：我们在正、负平方根之间随机选择。这类似于一般情况，即事先收集所有数据，例如通过优化技术计算解，甚至通过实验获得。此类数据收集通常不会偏向多模态解中的某一个模态。

```
# Generate data
N = 10000
X = np.random.random(N).astype(np.float32).reshape(-1, 1)

# Generation of Y-Data
sign = (- np.ones((N,))).astype(np.float32) ** np.random.randint(2, size=N)
Y = (np.sqrt(X.flatten()) * sign).reshape(-1, 1).astype(np.float32)

# Convert to PyTorch tensors
X_tensor = torch.tensor(X)
Y_tensor = torch.tensor(Y)
```

现在可以定义网络。我们使用简单的全连接架构，包含三个隐藏层和 ReLU 激活函数。[2]

```
# Define the neural network
class SimpleNN(nn.Module):
    def __init__(self, hiddendim=10):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(1, hiddendim)
        self.fc2 = nn.Linear(hiddendim, hiddendim)
        self.fc3 = nn.Linear(hiddendim, 1)
```

[1]原文同时给出 $y \in [0, 1]$ 和负平方根；按照所给定义域，负根并不属于允许范围。此处保留原文，见问题记录。

[2]原文称三个隐藏层，但下列代码中有两个 ReLU 隐藏层和一个线性输出层。

```
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x) # Linear output
        return x
```

接下来实例化模型（隐藏维度为 128），指定损失函数（使用 PyTorch 的 MSELoss() 计算简单的均方误差）和 Adam 优化器。在下面的循环中，网络训练 50 个 epoch：

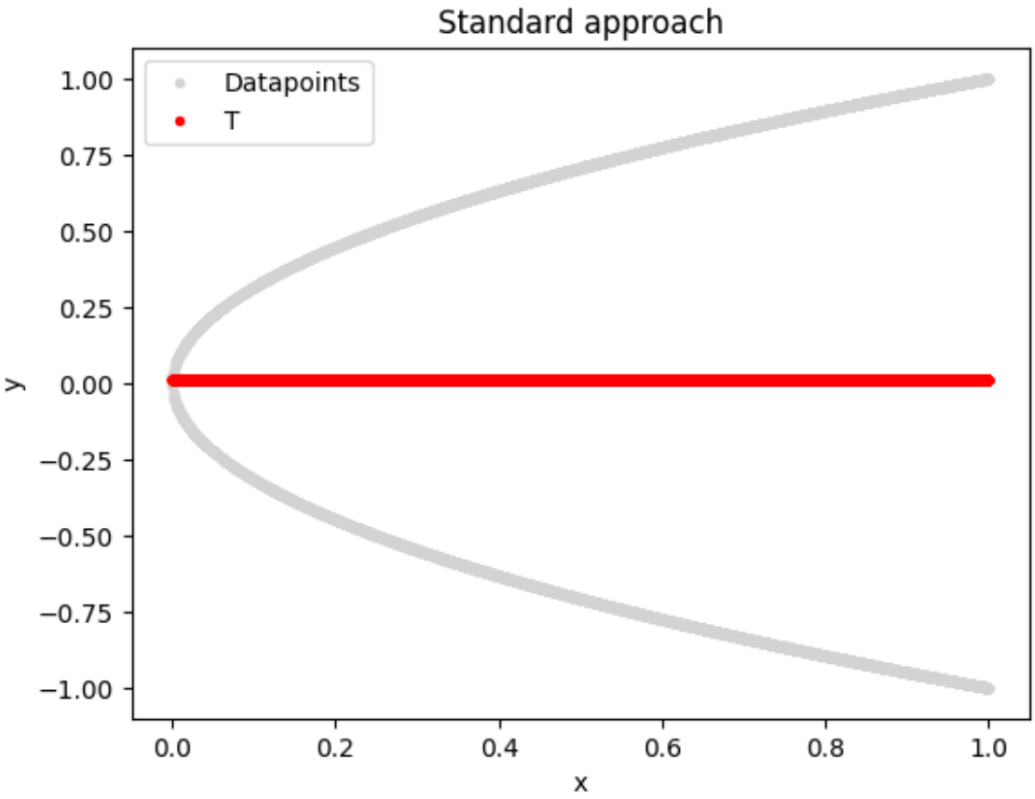
```
nn_sup = SimpleNN(hiddendim=128)
criterion = nn.MSELoss()
optimizer = optim.Adam(nn_sup.parameters(), lr=0.001)

# Training loop
epochs = 50
batch_size = 5

for epoch in range(epochs):
    permutation = torch.randperm(N)
    epoch_loss = 0.0
    for i in range(0, N, batch_size):
        indices = permutation[i:i+batch_size]
        batch_x, batch_y = X_tensor[indices], Y_tensor[indices]
        optimizer.zero_grad()
        outputs = nn_sup(batch_x)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    if(epoch%10==9): print(f"Epoch {epoch+1}/{epochs}, Loss: {epoch_loss/N:.6f}")
```

```
Epoch 10/50, Loss: 0.100193
Epoch 20/50, Loss: 0.100193
Epoch 30/50, Loss: 0.100207
Epoch 40/50, Loss: 0.100202
Epoch 50/50, Loss: 0.100203
```

由于网络和数据集都比较小，训练很快收敛。下面绘出解：数据以浅灰色显示，监督学习得到的解以红色显示。

```
import matplotlib.pyplot as plt

plt.plot(X,Y,'.',label='Datapoints', color="lightgray")
plt.plot(X, nn_sup(torch.tensor(X)).detach(), '.',label='T', color="red")
plt.xlabel('x')
plt.ylabel('y')
```

```
plt.title('Standard approach')
plt.legend()
plt.show()
```



这显然完全不对！红色的解远离灰色所示解的两个模态。训练过程对 x 轴两侧的数据点取了平均，因此未能找到上述问题的满意解。

注意红线往往并不恰好为零；在连续情形下，解的两个模态应该平均为零。原文将此归因于例子中只有 200 个点、采样相对粗糙。[3]

1.3 可微物理方法

现在应用前述可微物理思想求 f：直接把离散模型 $\mathcal{P}$ 纳入训练。注意在这里，$\mathcal{P}^*$ 和 $\mathcal{P}$ 实际上给出回到输入空间 X 的映射，即 $\mathcal{P}^*: Y \to X$。这里没有真正的数据生成步骤，只需从区间 $[0,1]$ 采样。我们沿用前面使用的 x 位置，并创建一个与之前架构相同的神经网络新实例 nn_dp：

```
# X-Data
# X = X , we can directly re-use the X from above, nothing has changed...
```

[3] 此处原文的 200 点与上面代码中的 $N = 10000$ 不一致，保留原说法。

```
# P maps Y back to X, simply by computing a square, as y is a TF tensor input, the
# square operation **2 will be differentiable

def P(y):
    return torch.square(y)

# Define custom loss function using the "physics" operator P
def loss_function(y_true, y_pred):
    return criterion(y_true, P(y_pred))
```

损失函数是训练的关键：我们将要学习的函数 f（称为 nn_dp）直接纳入损失。原文称 Keras 会对 X 中的输入计算 nn_dp，并以第二个参数 y_from_nn_dp 提供输出；我们在这个输出上运行“求解器” P，结果应匹配正确答案 y_true。在此简单例子中，loss_dp 函数只是计算预测值 y_pred 的平方。[4]

后续可以在这里完成更多操作：例如对预测解计算有限差分模板，或者计算求解器的整个隐式时间积分步。这里使用一个简单的均方误差项，形如 $|\mathcal{P}(y_{\text{pred}}) - x_{\text{true}}|^2$，并在训练中最小化它。并非必须如此简化：能够纳入的知识和数值方法越多，就越能引导训练过程。

再次实例化神经网络，使用可微物理损失训练：

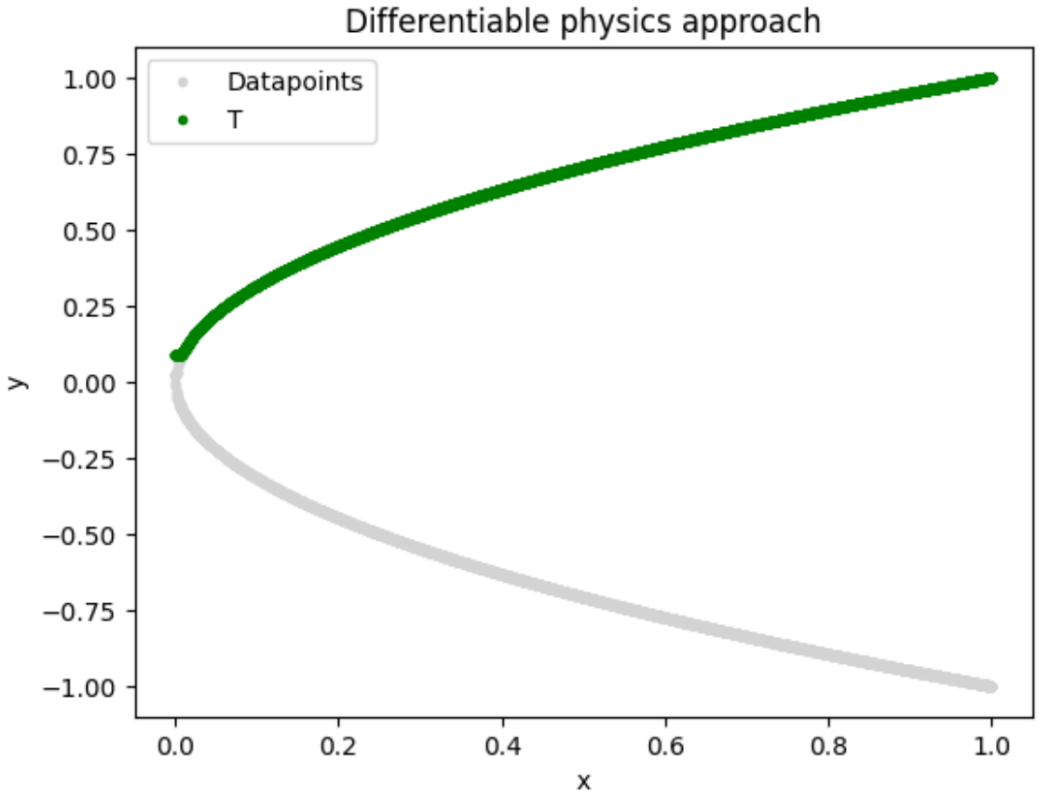
```
nn_dp = SimpleNN(hiddendim=128)
optimizer = optim.Adam(nn_dp.parameters(), lr=0.001)

# Training loop
batch_size = 5
for epoch in range(epochs):
    permutation = torch.randperm(N)
    epoch_loss = 0.0
    for i in range(0, N, batch_size):
        indices = permutation[i:i+batch_size]
        batch_x = X_tensor[indices]
        optimizer.zero_grad()
        outputs = nn_dp(batch_x)
        loss = loss_function(batch_x, outputs)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    if(epoch%10==9): print(f"Epoch {epoch+1}/{epochs}, Loss: {epoch_loss/N:.6f}")
```

```
Epoch 10/50, Loss: 0.000004
Epoch 20/50, Loss: 0.000003
Epoch 30/50, Loss: 0.000003
Epoch 40/50, Loss: 0.000002
Epoch 50/50, Loss: 0.000002
```

现在网络确实学到了抛物线函数的良好逆函数！下图以绿色显示解。

[4]原文此处提及 Keras、TF、y_from_nn_dp 和 loss_dp，而所示实现使用 PyTorch 及 loss_function，后者对平方后的预测和目标计算均方误差。

```
# Results
plt.plot(X,Y,'.',label='Datapoints', color="lightgray")
plt.plot(X, nn_dp(torch.tensor(X)).detach(), '.',label='T', color="green")
plt.xlabel('x')
plt.ylabel('y')
plt.title('Differentiable physics approach')
plt.legend()
plt.show()
```



这看起来好多了，至少在避开原点时如此（这一部分需要额外处理）。发生了什么？

- 我们根据网络当前预测计算离散模型，而非使用预先计算的解，从而避免了对多个解模态进行不期望的平均。这使我们能够找到网络预测附近最好的模态，并避免对解流形上的模态取平均。
- 我们仍然只得到曲线的一侧！这是预料之中的，因为我们用确定性函数表示解，因此只能表示单个模态。有趣的是，上方还是下方模态取决于 f 中权重的随机初始化——多运行几次这个例子即可看到这一效果。为了捕获多个模态，需要扩展神经网络，使其捕获输出的完整分布并以额外维度对其参数化。

第二章　概率生成式 AI 方法

如前所述，利用最先进的 AI 技术，我们还能做得更好：可以学习后验的完整分布，在这里即每个 x 对应的不同答案。下面将使用流匹配，作为生成式、基于扩散的算法中的一种先进方法。

由于这些方法处理带噪声的数据，我们首先需要指定一个新的数据加载器，向数据的 y 值添加不同程度的“噪声”，使网络能够学到通向两个可能模态的正确方向。

```
from torch.utils.data import Dataset, DataLoader
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class FlowMatchingDataset(Dataset):
    def __init__(self, data_x, data_y, n_samples=1000, sigma_min=1e-4):
        super().__init__()
        self.n_samples = n_samples
        self.sigma_min = sigma_min
        self.data_x = data_x
        self.data_y = data_y

    def __len__(self):
        return self.n_samples

    def __getitem__(self, idx):
        x0 = np.random.multivariate_normal([0.0, 0.0], np.eye(2), 1)[0]
        t = np.random.rand() # scalar in [0,1]
        dx = self.data_x[idx] #:idx+1]
        dy_org = self.data_y[idx]# :idx+1]
        x0[0] = dx[0] # keep x value
        x1 = np.concatenate([dx,dy_org],axis=0)
        #print([self.data_x.shape,dx.shape,x1.shape])

        x_t = (1 - ( 1 - self.sigma_min) * t) * x0 + t * x1
        u_t = (x1 - x0)
        x_t = torch.tensor(x_t, dtype=torch.float32)
        t   = torch.tensor([t], dtype=torch.float32)
        u_t = torch.tensor(u_t, dtype=torch.float32)
        return x_t, t, u_t
```

网络本身与之前没有太大区别，只需增加额外的时间输入 t：

```
class VelocityNet(nn.Module):
```

```
    def __init__(self, hidden_dim, in_dim=2, time_dim=1, out_dim=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim + time_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim)
        )
    def forward(self, x, t):
        xt = torch.cat([x, t], dim=1)
        return self.net(xt)
```

训练与之前类似：从数据集中采样带噪声的样本，训练网络将样本移向数据集中的解。

```
batch_size = 128
dataset = FlowMatchingDataset(X, Y, n_samples=N)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
nn_fm = VelocityNet(hidden_dim=128).to(device)
optimizer = optim.Adam(nn_fm.parameters(), lr=0.001)
criterion = nn.MSELoss()

for epoch in range(epochs):
    running_loss = 0.0
    for x_t, t, u_t in dataloader:
        x_t = x_t.to(device)
        t   = t.to(device)
        u_t = u_t.to(device)
        optimizer.zero_grad()
        pred_v = nn_fm(x_t, t)
        loss = criterion(pred_v, u_t)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * x_t.size(0)
    running_loss /= len(dataset)
    if(epoch%10==9): print(f"Epoch {epoch + 1}/{epochs}, Loss: {running_loss:.4f}")
```

```
Epoch 10/50, Loss: 0.3837
Epoch 20/50, Loss: 0.3773
Epoch 30/50, Loss: 0.3735
Epoch 40/50, Loss: 0.3726
Epoch 50/50, Loss: 0.3715
```

评估时，我们反复调用神经网络，改进从简单分布中抽取的初始噪声样本，一步步将其移向“正确”的解。下面的 integrate_flow 函数实现了这个过程。[1]

[1]译注：源路径 $x_t = [1 - (1 - \sigma_{\min})t]x_0 + tx_1$ 的严格导数是 $x_1 - (1 - \sigma_{\min})x_0$，代码目标 $u_t = x_1 - x_0$ 少了 $\sigma_{\min}x_0$，终点也为 $x_1 + \sigma_{\min}x_0$，不能视为严格相同路径。积分中时间张量新建于 CPU，而样本和网络可能在 CUDA；最终 Matplotlib 绘图的张量也未转 CPU，

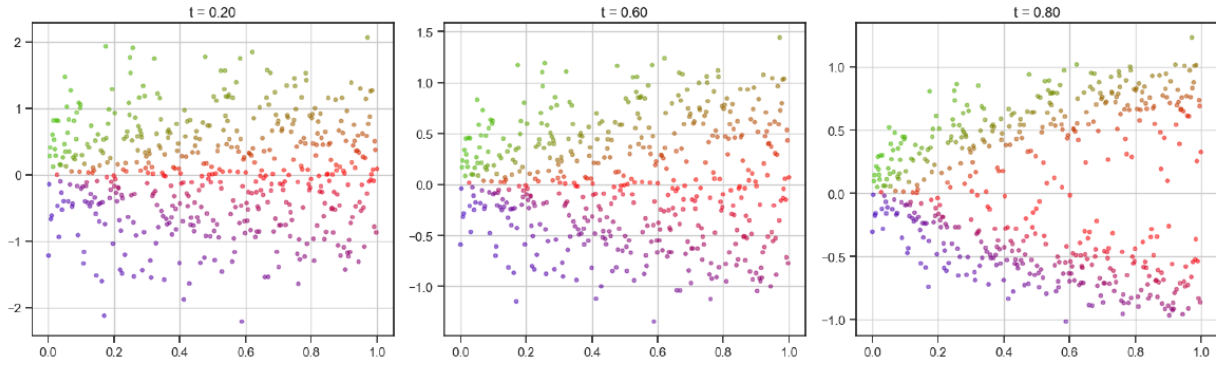
```
def integrate_flow(nn, x0, t_span=(0.0, 1.0), n_steps=100):
    trajectory = []
    t = torch.linspace(t_span[0], t_span[1], n_steps).to(x0.device)
    dt = 1./n_steps
    x_in = x0
    for i in range(n_steps):
        x0 = x0 + dt * nn(x0, torch.tensor([i/n_steps]).expand(x0.shape[0], 1) )
        x0[:,0] = x_in[:,0] # condition on original x position
        trajectory.append(x0)
    return trajectory, t

# Generate samples along x, then randomize along y
n_gen = 500
x_in = torch.linspace(0.,1., n_gen).to(device)
y_in = torch.randn(n_gen).to(device) * 0.95
x0_gen = torch.stack([x_in,y_in],axis=-1)
trajectory, time_points = integrate_flow(nn_fm, x0_gen)
```

为了说明流动过程，下面的代码单元展示流积分在不同时刻的样本。初始随机分布逐渐变为抛物线目标的双模态分布。

```
import seaborn as sns
sns.set_theme(style="ticks", palette="pastel")

def get_angle_colors(positions):
    angles = np.arctan2(positions[:, 1], positions[:, 0])
    angles_deg = (np.degrees(angles) + 360) % 360
    colors = np.zeros((len(positions), 3))
    for i, angle in enumerate(angles_deg):
        segment = int(angle / 120)
        local_angle = angle - segment * 120
        if segment == 0:    # 0 degrees to 120 degrees (R->G)
            colors[i] = [1 - local_angle/120, local_angle/120, 0]
        elif segment == 1: # 120 degrees to 240 degrees (G->B)
            colors[i] = [0, 1 - local_angle/120, local_angle/120]
        else:              # 240 degrees to 360 degrees (B->R)
            colors[i] = [local_angle/120, 0, 1 - local_angle/120]
    return colors

desired_times = [0.2, 0.6, 0.8,]
time_np = time_points.detach().cpu().numpy()
n_steps = len(time_np)
indices = [np.argmin(np.abs(time_np - t_val)) for t_val in desired_times]
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 5))
axes = axes.ravel() # flatten the 2D array for easier indexing
```

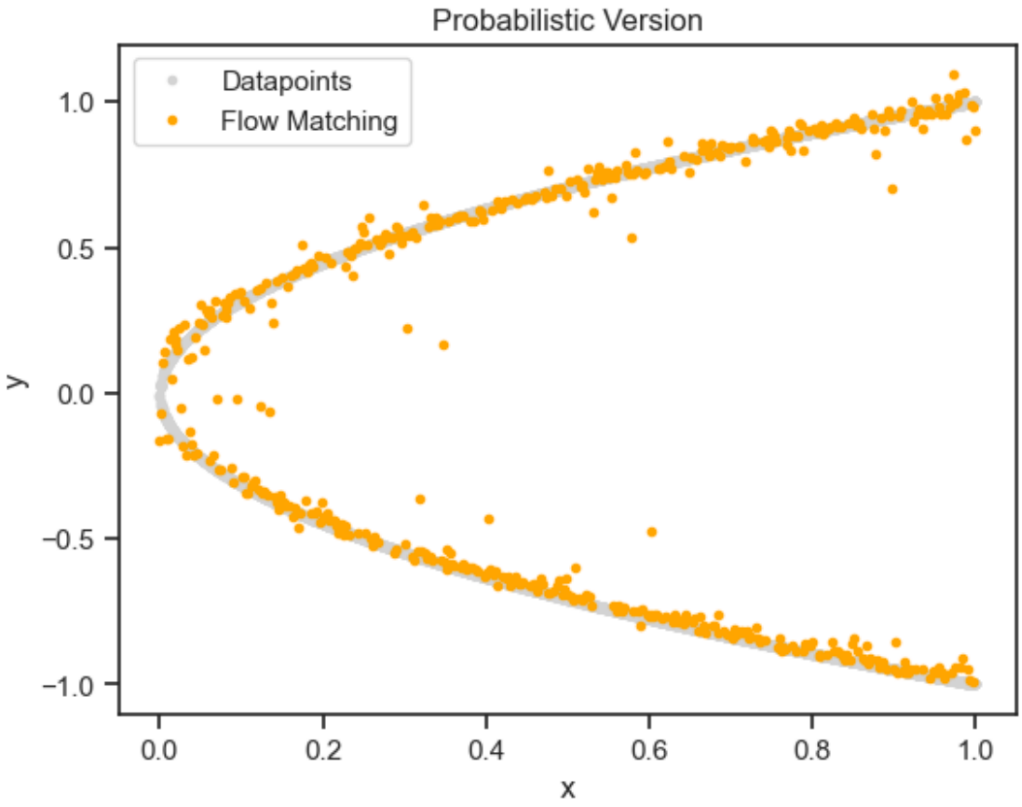
GPU 路径不能直接保证可运行。此外每次更新后才 append，首轨迹已是 dt 时刻，却用 linspace 的 0 标记；`t_span` 仅改变返回标签，不改变固定 $dt = 1/n_steps$ 和网络输入 i/n_steps。图中时间沿源保留，未执行。

```
xx, yy = np.mgrid[0:1:100j, -1:1:100j]
positions = np.vstack([xx.ravel(), yy.ravel()])
for i, idx in enumerate(indices):
    ax = axes[i]
    x_t = trajectory[idx].detach().cpu().numpy()
    if i == 0:
        c = get_angle_colors(x_t)
    ax.scatter(x_t[:, 0], x_t[:, 1], alpha=0.5, s=10, color=c)
    ax.set_title(f"t = {time_np[idx]:.2f}")
    ax.grid(True)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



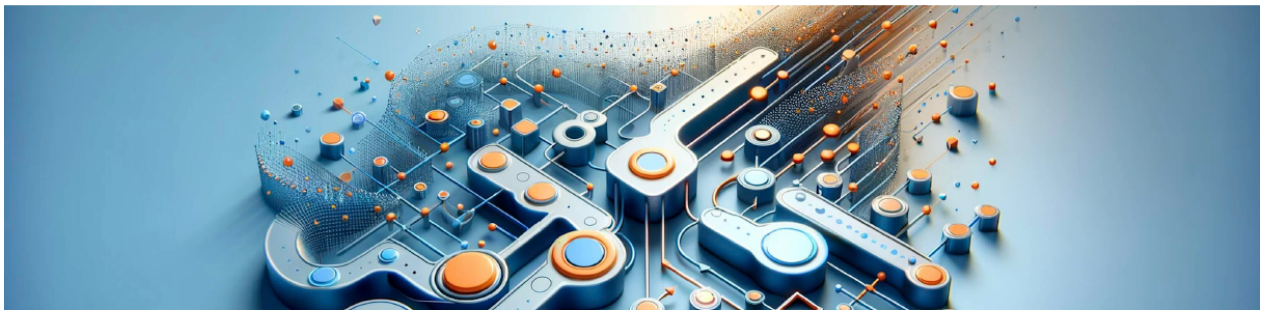
再仿照前面的监督学习和可微物理版本绘出解:

```
# Results
plt.plot(X,Y,'.',label='Datapoints', color="lightgray")
plt.plot(trajectory[-1][:,0].detach(), trajectory[-1][:,1].detach(), '.',
         label='Flow Matching', color="orange")
plt.xlabel('x')
plt.ylabel('y')
plt.title('Probabilistic Version')
plt.legend()
plt.show()
```



正如预期，该方法确实以 x 轴上方和下方的点表示出解的两个“模态”。结果仍有一些噪声，但可以通过改进学习设置来缓解，例如使用更大的网络。

一个显然的问题是：我们又回到了只用数据训练，那么如何集成物理呢？这是一个明显的改进方向，稍后将更详细地讨论带物理约束的扩散方法。展望而言，物理先验尤其有助于引导神经网络略带噪声的输出趋向准确解。



2.1 讨论

这是个非常简单的例子，却清楚展示了监督学习失败的一种情形。尽管乍看之下可能很人为，许多实际偏微分方程都具有各种这样的模态，而我们往往并不清楚感兴趣的解流形中模态的位置和数量。在这种情况下使用监督学习非常危险：我们可能在不知情的情况下得到不同模态的平均值。

流动中的分岔是一个明显而恰当的例子。蜡烛上方升起的烟开始时笔直向上，随后由于运动中的微

小扰动，开始向随机方向振荡。下图通过数值扰动说明了这一情况：完全对称的设置将开始向左或向右偏转，取决于近似误差如何累积。与上面的抛物线例子类似，对两个模态取平均会得到不符合物理的笔直流动。同样，许多数值解都具有不同模态，通常恢复这些模态而非将它们平均掉很重要。因此，后续章节将展示如何在更实际、更复杂的情形下利用可微物理进行训练。

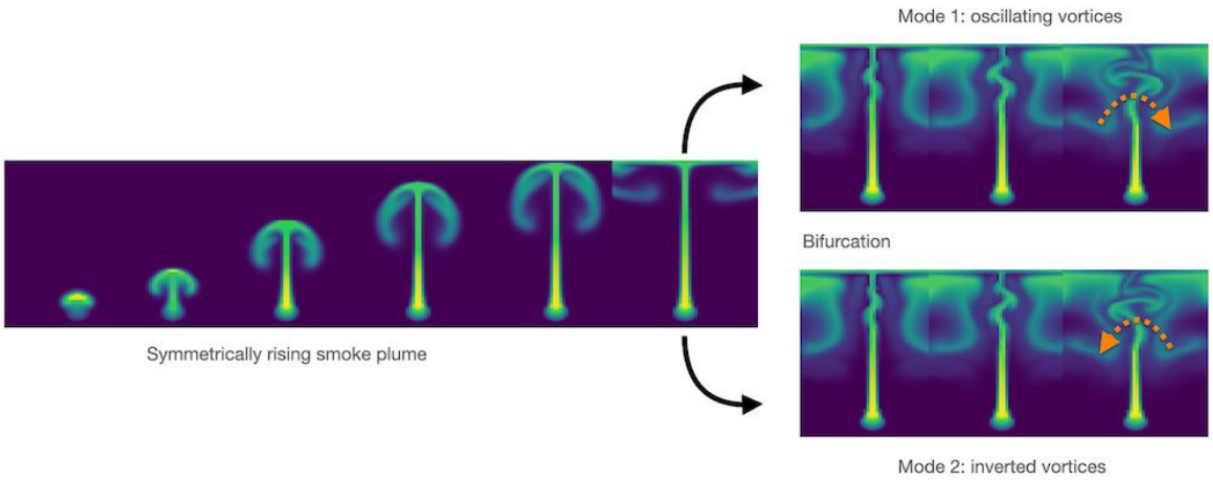


图 2.1: 浮力驱动流中的分岔：绿色显示的“烟”开始时完全笔直地上升，但微小数值误差随时间增长，导致不稳定性，使涡旋交替偏向一侧（右上）或相反方向（右下）。

2.2 下一步

后面的每个 notebook 都有一个类似下面的“下一步”小节，建议从哪里开始修改代码。毕竟，这些 notebook 的目的正是提供可以立即执行的程序，作为自己实验的基础。例子的数据集和神经网络通常相当小，以缩短 notebook 的运行时间，但它们仍是开展可能复杂且大型项目的良好起点。

对于上面的简单 DP 例子：

- 此 notebook 有意采用很简单的设置。可以改变训练设置或网络架构以改善解，例如在零附近提供更多样本来改善原点附近的解。
- 或者尝试将设置扩展到二维情形，即抛物面。给定函数 $\mathcal{P} : (y_1, y_2) \mapsto y_1^2 + y_2^2$，求反函数 f，使所有 $x \in [0, 1]$ 都满足 $\mathcal{P}(f(x)) = x$。
- 如果想在不安装任何软件的情况下实验，也可以在 Colab 中运行这个 notebook。

第三章　概述

本书的名称“基于物理的深度学习”指物理建模和数值仿真与基于人工智能的方法，即神经网络的结合。基于物理的深度学习这一大方向也称为科学机器学习，是一个非常活跃、快速发展的研究领域。本章将更全面地介绍这一主题，并为后续章节奠定基础。

图 3.1: 理解我们的环境并预测其演化是人类面临的关键挑战之一。实现这些目标的重要工具是计算机仿真；下一代仿真很可能从集成人工智能和深度学习组件中大幅受益，从而对环境中的现象作出更准确的预测。

3.1　动机

从天气和气候预报 [Sto14]（见上图），到量子物理 [OMalleyBK+16]，再到等离子体聚变的控制 [MLA+19]，利用数值分析求解物理模型已成为科学不可或缺的一部分。

近年来，以深度神经网络为驱动的人工智能在多个领域取得了令人瞩目的成就：从图像分类 [KSH12] 到自然语言处理 [RWC+19]、蛋白质折叠 [Qur19]，再到各种基础模型。这个领域生机勃勃、发展迅速，展现出广阔的可能性。

3.1.1　取代传统仿真？

深度学习（DL）方法的这些成功引发了人们的担忧：这一技术可能取代以仿真为驱动的传统科学方法。例如，近期工作表明，基于神经网络的代理模型在翼型流动等现实工业应用中达到了所需精度 [CT22]，同时运行时间比传统求解器快几个数量级。

我们是否可以不再依赖从第一性原理精心构造的模型，而是处理足够大的数据集来给出正确答案？正如后续章节将要展示的，这种担忧没有根据。相反，对下一代仿真系统而言，连接这两个世界至关重要：将经典数值技术与人工智能方法结合。此外，后者在处理复杂分布和仿真不确定性等传统方法面临挑战的领域提供了令人振奋的新可能。

与数值方法相结合之所以重要，一个核心原因是深度学习方法虽然强大，却能从物理模型形式的领域知识中大幅受益。深度学习技术和神经网络比较新，有时难以应用；也必须承认，如何将我们对物理过程的理解恰当地融入学习算法，通常并不简单。

过去几十年，人们为求解 Navier–Stokes、Maxwell 或 Schrödinger 方程等基本模型方程，开发了高度专门化且精确的离散格式。离散化中看似微不足道的变化，就可能决定解中能否看见关键现象。本书将展示，在应用深度学习时，尽可能利用数值数学领域已有的强大方法十分有益。

3.1.2 黑箱？

过去，人们常将人工智能和深度学习方法中的训练后神经网络称为黑箱，暗示它们超出了人类理解的范围。不过，这类观点通常源自道听途说，以及对“热门”话题的普遍怀疑。

然而，这在科学中是很常见的情况：我们面对一类新方法，而“所有繁琐细节”尚未完全弄清。各种科学进展中都常有这种情形。数值方法本身就是一个好例子。大约在 1950 年，数值近似和求解器的处境并不容易。例如，H. Goldstine 曾指出，数值不稳定性被认为是“未来持续的忧虑之源”[Gol90]。如今，我们对这些不稳定性已有相当好的理解，数值方法也已广泛应用并得到充分确立。人工智能和神经网络正沿着人类进步的同一道路发展。

因此，重要的是认识到：从某种意义上说，深度学习方法并没有什么特别或超自然之处，它们只是新的一组数值工具。当然，它们确实很新，而且目前无疑是我们处理非线性问题所拥有的最强大工具。尽管所有细节尚未完全弄清并妥善写成文献，这也不应妨碍我们将这些强大方法纳入数值工具箱。

3.1.3 协调人工智能与仿真

退一步看，本书的目标是以我们掌握的强大数值仿真技术为基础，尽可能将其与深度学习结合。因此，一个核心目标是协调人工智能视角与物理仿真。

本文档的目标 下文将讨论的关键方面是：

- 如何利用深度学习技术解决偏微分方程问题；
- 如何将其与现有的物理知识结合；
- 同时不抛弃数值方法。

同时也要指出本文不涵盖的内容：

- 不深入介绍深度学习和数值仿真（已有优秀著作承担了这一任务）；
- 也不以广泛综述该领域研究论文为目标。

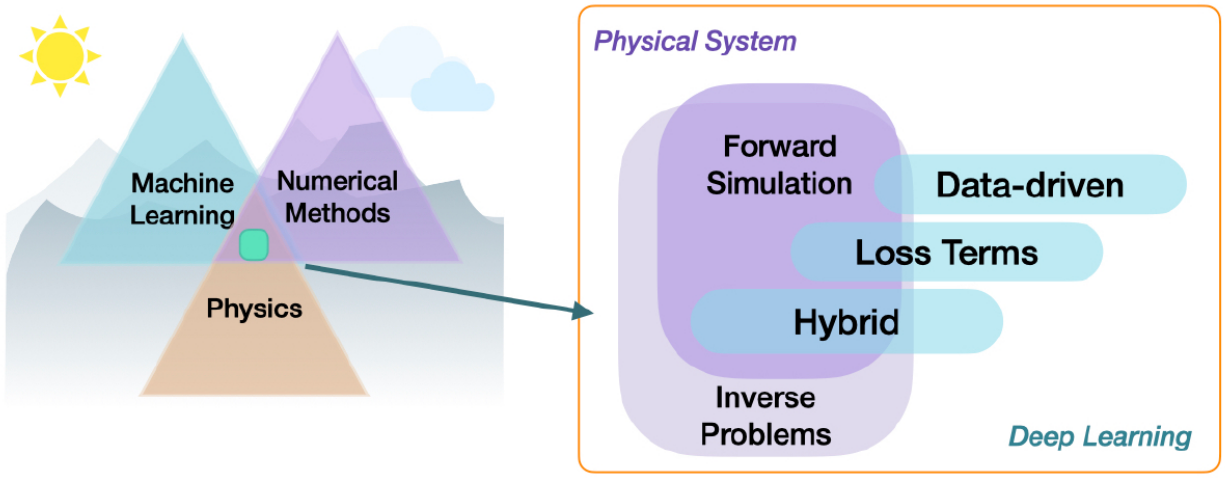
所得方法具有巨大的潜力，可以拓展数值方法的能力：例如，当求解器反复处理某一明确定义的问题领域中的情况时，一次投入大量资源训练一个支持重复求解的神经网络可能很有意义。大型所谓“基

础模型”的发展在这一领域尤其有前景。通过使用较小数据集微调以实现领域专门化，混合求解器可能远远胜过传统的通用求解器。尽管还有许多未解问题，已有早期论文表明这一目标是现实的 [KSA+21, UBH+20]。

另一种看法是，自然界的所有数学模型都是理想化近似，因而包含误差。人们已投入大量努力获得非常好的模型方程；但要向前迈出下一大步，人工智能和深度学习方法为缩小与现实之间剩余的差距提供了强有力的工具 [AAC+19]。

3.2 分类

在基于物理的深度学习领域，我们可以区分多种方法，例如针对约束、组合方法、优化和应用的方法。更具体地说，所有方法所针对的要么是正向仿真（预测状态或时间演化），要么是逆问题（例如从观测中获得物理系统的参数化或状态）。



无论考虑的是正问题还是逆问题，对于后续主题来说，最关键的区别在于深度学习技术与领域知识的集成方式，领域知识通常通过偏微分方程（PDE）形式的模型方程给出。基于物理的深度学习（PBDL）技术大致可分为以下三类：

- **监督方法：**数据由物理系统（真实或仿真的）产生，但除此之外没有进一步交互。这是经典机器学习方法。

- **损失项方法：**物理动力学（或其中一部分）被编码进损失函数，通常采用可微操作的形式。学习过程可以反复计算损失，并通常从基于 PDE 的表述中获得梯度。这些软约束有时也称为“物理信息”训练。

- **混合方法：**完整物理仿真与深度神经网络的输出交错并结合；这通常需要完全可微的模拟器。它代表物理系统与学习过程之间最紧密的耦合，产生结合经典技术和基于 AI 技术的混合求解器。

因此，可以按正向或逆向求解以及物理模型与神经网络集成的紧密程度对方法分类。特别是利用可微物理的混合方法，可以实现深度学习与数值仿真方法的紧密集成。

3.2.1 命名

需要指出的是，下面称作“可微物理”的方法，在其他资料和研究论文中有许多不同名称。可微物理这一名称受到深度学习中可微编程范式的启发。例如，其中也有“可微渲染方法”，用于模拟光如何形成人类看到的图像。相比之下，我们接下来关注物理仿真，因此使用这个名称。

在其他背景下，其他名称更为常见。例如，可微物理方法等价于使用伴随方法并将其与深度学习过程耦合；实际上，也等价于将反向传播或反向模式微分应用于数值仿真。不过，如前所述，基于深度学习的视角，下文将把这些都称为“可微物理”方法。

将深度学习与传统求解器集成所得到的混合求解器，也可视作一个经典主题：在这里，神经网络的任务是修正求解器。这一修正可以针对数值误差，也可以针对方程中未解析的项。这是科学中的基本问题，人们曾以各种名称研究它，例如流体动力学和湍流中的闭合问题、材料科学中的均匀化或粗粒化，以及气候和天气仿真中的参数化。不同领域对这一目标的重新提出表明了底层问题的重要性，本文将说明深度学习为解决该问题提供的新途径。

3.3 展望

物理仿真是一个庞大的领域，我们无法覆盖所有可能的物理模型和仿真类型。

注 本书重点关注：

- 基于稠密场的仿真（不讨论拉格朗日方法）；
- 与深度学习的结合（还有许多其他有趣的机器学习技术，但这里不讨论）；
- 将实验留作展望（即用现实观测代替合成数据）。

还需指出，我们将从一些非常基础的构件开始建立这些方法。下面是关于跳读后续章节的一些建议。

提示：可以跳读的情况

- 如果你十分熟悉数值方法和 PDE 求解器，希望直接开始学习深度学习主题，那么“监督训练”一章是很好的起点。
- 另一方面，如果你已经深入了解神经网络等内容，希望直接进入研究相关主题，我们建议从“物理损失项”一章开始，它为后续章节奠定基础。

不过，无论哪种情况，简要浏览“符号和缩写”一章中的记号都没有坏处！

3.4 实现

本文也介绍深度学习和仿真 API。我们主要使用流行的深度学习 API PyTorch，也会使用少量 TensorFlow，另外还会介绍可微仿真框架 ΦFlow（phiflow）。一些例子也使用 JAX，它提供了一种有趣的替代方案。因此，完成这些示例后，你应该能够大致了解当前可用的 API，从而为新任务选择最合适的工具。

由于大多数 Jupyter notebook 都涉及随机优化，下面许多代码示例每次运行时会产生略有不同的结果。这在神经网络训练中很常见，但执行代码时务必记住。这也意味着，文中讨论的数字可能与重新运行示例时看到的数字不完全一致。

3.5 模型与方程

下面将非常简要地介绍深度学习，主要是为了引入记号。此外还将讨论一些模型方程。注意，与某些其他文本和 API 不同，我们避免用“模型”表示训练后的神经网络，而称之为“神经网络”或“网络”。“模型”通常表示描述某种物理效应的一组模型方程，通常是 PDE。

3.5.1 深度学习与神经网络

深度学习的目标是近似一个未知函数

$$f^*(x) = y^*, \tag{3.1}$$

其中 y^* 表示参考解或“真实值”，而 $f^*(x)$ 应由神经网络 $f(x;\theta)$ 近似。我们通常借助某种损失函数 $L(y, y^*)$ 确定 f，其中 $y = f(x;\theta)$ 为网络输出。这给出一个最小化问题，即寻找使 L 最小的 $f(x;\theta)$。最简单的情况是使用 L^2 误差，得到

$$\arg\min_{\theta} |f(x;\theta) - y^*|_2^2. \tag{3.2}$$

我们通常选择某种随机梯度下降（SGD）优化器，例如 Adam [KB14]，进行优化，也就是训练。我们依靠自动微分计算标量损失 L 对权重的梯度 $\partial L/\partial\theta$。该函数为标量对梯度计算至关重要；损失函数也常称为“误差”“代价”或“目标”函数。

训练时，我们区分从某一分布抽取的训练数据集、验证集（来自同一分布，但数据不同），以及与训练集分布有所不同的测试数据集。后一项区别很重要。测试时，我们通常希望使用分布外（OOD）数据，检查训练模型的泛化能力。注意，测试数据集有极大的选择范围：从肯定有效的微小变化，到几乎肯定失败的完全不同输入。这里没有金标准，但应当谨慎生成测试数据。

如果你觉得上述概述并不显然，我们强烈建议阅读《Deep Learning》一书第 6 至 9 章，尤其是关于多层感知机和“卷积网络”，即 CNN 的部分。

注：分类与回归 经典机器学习中分类和回归问题的区别在这里并不那么重要：下文只处理回归问题。

3.5.2 作为物理模型的偏微分方程

下面简要展望之后深度学习示例中使用的模型方程。我们通常关注记为 $\mathcal{P}^*$ 的连续 PDE 算子，它将输入 $\mathcal{U}$ 映射到 $\mathcal{V}$；在最一般的情况下，$\mathcal{U}$、$\mathcal{V}$ 都是无限维 Banach 空间，即 $\mathcal{P}^*: \mathcal{U} \to \mathcal{V}$。

学习得到的解算子与传统解算子 后文的目标是使用神经网络学习 $\mathcal{P}^*$（或其中一部分）。研究中使用了多种不同的名称，例如学习得到的代理模型、混合模拟器或仿真器、神经算子或求解器、自回归模型（涉及时间步时）等。

实践中，我们感兴趣的解位于 $d \in \{1,2,3\}$ 维空间区域 $\Omega \subset \mathbb{R}^d$ 中。此外，我们经常考虑有限时间区间上的时间演化 $t \in \mathbb{R}^+$。相应的场要么是 d 维矢量场，例如 $\mathbf{u}\colon \mathbb{R}^d \times \mathbb{R}^+ \to \mathbb{R}^d$，要么是标量场 $p\colon \mathbb{R}^d \times \mathbb{R}^+ \to \mathbb{R}$。矢量分量通常以下标 x, y, z 表示，即当 $d = 3$ 时 $\mathbf{v} = (v_x, v_y, v_z)^T$；位置则表示为 $\mathbf{p} \in \Omega$。

为了获得 $\mathcal{P}^*$ 的唯一解，需要指定适当的初始条件，通常是在 $t = 0$ 时对所有感兴趣的量指定初值，以及 Ω 边界上的边界条件，下文将该边界记为 Γ。$\mathcal{P}^*$ 表示连续表述，我们需要对其连续性作温和假设；通常假定一阶和二阶导数存在。

传统上，可以通过离散化，使用数值方法获得 $\mathcal{P}^*$ 这类光滑函数的近似。这不可避免地引入离散化误差，我们希望使其尽可能小。误差可用与精确解析解的偏差衡量；对于 PDE 的离散仿真，通常将其表示为截断误差 $O(\Delta x^k)$ 的函数，其中 Δx 是空间离散步长。同样，通常通过时间步长 Δt 进行时间离散。

符号与缩写 如有疑问，请查阅“符号和缩写”一章对数学记号及所用缩写的汇总。

数值仿真通过执行大小为 Δt 的步来求解离散化 PDE $\mathcal{P}$。解可以表示为 $\mathbf{u}$ 及其导数的函数：$\mathbf{u}(\mathbf{x}, t + \Delta t) = \mathcal{P}(\mathbf{u}_x, \mathbf{u}_{xx}, \dots \mathbf{u}_{xx\dots x})$，其中 $\mathbf{u}_x$ 表示空间导数 $\partial \mathbf{u}(\mathbf{x}, t)/\partial \mathbf{x}$。

对于所有 PDE，我们假设使用如下文所述的无量纲参数化；利用适当的比例因子，它们可以重新缩放为现实世界的物理量。接下来先概述模型方程，然后在下一页开始实际仿真及实现示例。

3.5.3 一些 PDE 示例

以下 PDE 是很好的例子，我们将在后续不同情形中使用它们，展示如何将它们纳入深度学习方法。

Burgers 方程 我们经常从一维或二维 Burgers 方程出发。它是研究充分的 PDE；与 Navier–Stokes 方程不同，它不包含质量守恒等额外约束，因此会产生有趣的激波形成现象。它包含平流项（运动或输运）和扩散项（由热力学第二定律导致的耗散）。二维形式为

$$\begin{aligned} \frac{\partial u_x}{\partial t} + \mathbf{u} \cdot \nabla u_x &= \nu \nabla \cdot \nabla u_x + g_x, \\ \frac{\partial u_y}{\partial t} + \mathbf{u} \cdot \nabla u_y &= \nu \nabla \cdot \nabla u_y + g_y, \end{aligned} \tag{3.3}$$

其中 ν 和 $\mathbf{g}$ 分别表示扩散常数和外力。不含外力的一维 Burgers 方程更简单，将唯一的一维速度分量记为 $u = u_x$，可写为

$$\frac{\partial u}{\partial t} + u \nabla u = \nu \nabla \cdot \nabla u. \tag{3.4}$$

Navier–Stokes 方程 在复杂程度上，一个合适的下一步是 Navier–Stokes 方程，它是成熟的流体模型。除了与 Burgers 方程类似的动量守恒方程外，它还包含质量守恒方程。这阻止了激波形成，但也给数值方法带来了新的挑战：无散度运动的硬约束。

不含外力的二维 Navier–Stokes 方程可写为

$$\begin{aligned}\frac{\partial u_x}{\partial t} + \mathbf{u}\cdot\nabla u_x &= -\frac{1}{\rho}\nabla p + \nu\nabla\cdot\nabla u_x,\\ \frac{\partial u_y}{\partial t} + \mathbf{u}\cdot\nabla u_y &= -\frac{1}{\rho}\nabla p + \nu\nabla\cdot\nabla u_y,\\ \text{满足}\quad \nabla\cdot\mathbf{u} &= 0.\end{aligned} \tag{3.5}$$

其中与前面一样，ν 表示黏性对应的扩散常数。[1]

一个有趣的变体是对变化的密度引入 Boussinesq 近似，例如流体的简单温度变化。用标记场 v 指示高温区域，可得到方程组

$$\begin{aligned}\frac{\partial u_x}{\partial t} + \mathbf{u}\cdot\nabla u_x &= -\frac{1}{\rho}\nabla p,\\ \frac{\partial u_y}{\partial t} + \mathbf{u}\cdot\nabla u_y &= -\frac{1}{\rho}\nabla p + \xi v,\\ \text{满足}\quad \nabla\cdot\mathbf{u} &= 0,\\ \frac{\partial v}{\partial t} + \mathbf{u}\cdot\nabla v &= 0.\end{aligned} \tag{3.6}$$

其中 ξ 表示浮力强度。[2] 最后，三维 Navier–Stokes 模型给出以下方程组：

$$\begin{aligned}\frac{\partial u_x}{\partial t} + \mathbf{u}\cdot\nabla u_x &= -\frac{1}{\rho}\nabla p + \nu\nabla\cdot\nabla u_x,\\ \frac{\partial u_y}{\partial t} + \mathbf{u}\cdot\nabla u_y &= -\frac{1}{\rho}\nabla p + \nu\nabla\cdot\nabla u_y,\\ \frac{\partial u_z}{\partial t} + \mathbf{u}\cdot\nabla u_z &= -\frac{1}{\rho}\nabla p + \nu\nabla\cdot\nabla u_z,\\ \text{满足}\quad \nabla\cdot\mathbf{u} &= 0.\end{aligned} \tag{3.7}$$

此处压力项同样按原文保留，参见前面的注释。

3.5.4 正向仿真

在真正开始学习方法之前，有必要介绍上述模型方程最基本的用法：普通的“正向”仿真。它从一组初始条件出发，利用模型方程的离散化版本随时间演化系统状态。我们将展示如何对一维 Burgers 方程和二维 Navier–Stokes 方程进行正向仿真。

3.6 使用 phiflow 对 Burgers 方程进行简单正向仿真

本节介绍如何运行正向仿真，即从给定初态出发、用数值方法近似之后状态的普通仿真，并介绍 ΦFlow 框架（下文称为 phiflow）。Phiflow 提供一组可微构件，能够直接与深度学习框架对接，因此是本书主题的良好基础。在深入更复杂的集成之前，本 notebook（以及下一个）将展示如何使用 phiflow 进行普通仿真。后面还将看到，这些仿真可以很容易地与神经网络耦合。

[1] 此处两个标量分量方程的压力项均按原文写为 ∇p；其矢量形式与左侧分量的对应关系需要核查。

[2] 该方程组按原 PDF 保留；与上一组相比没有显式黏性项，压力梯度同样采用原文写法。

Phiflow 的主仓库为 `https://github.com/tum-pbs/PhiFlow`，其他 API 文档和示例可在 `https://tum-pbs.github.io/PhiFlow/` 找到。对于本 notebook 和后续所有 notebook，首段末尾都有 “在 Colab 中运行” 链接（也可以使用页面顶部的启动按钮）。这将在 Colab notebook 中加载 PBDL GitHub 仓库的最新版本，供立即执行。具体链接待从原始 notebook 核实。

3.6.1 模型

我们使用 Burgers 方程作为物理模型。这一方程虽然简单，却是非线性且非平凡的模型方程，可以产生有趣的激波，因此是实验的良好起点。其一维形式（来自式 (3.4)）为

$$\frac{\partial u}{\partial t} + u\nabla u = \nu \nabla \cdot \nabla u.$$

3.6.2 导入和加载 phiflow

先完成一些准备工作：导入 phiflow 库，更具体地说是用于流体仿真的 NumPy 算子 `phi.flow`（深度学习框架 X 的可微版本则通过 `phi.X.flow` 加载）。这使不同 API 之间的切换变得容易，例如 phiflow 求解器既可以运行于 PyTorch、TensorFlow，也可以运行于 JAX。

注意：下面第一个以 “!” 为前缀的命令，在取消注释后会通过 pip 从 GitHub 将 phiflow Python 包安装到 Python 环境。我们假定尚未安装 phiflow；如果已经安装，只需注释掉第一行（后续 notebook 也一样）。

```
!pip install --upgrade --quiet phiflow==3.2
from phi.flow import *
print("Using phiflow version: {}".format(phi.__version__))
```

```
Using phiflow version: 3.2.0
```

接下来定义并初始化所需常数（用大写名称表示）：仿真区域采用 N=128 个单元，作为区间 $[-1, 1]$ 上周期区域 Ω 中一维速度 u 的离散点。时间区间为 1，使用 32 个时间步 STEPS，得到 DT=1/32。此外，使用黏度 NU：$\nu = 0.01/\pi$。[3]

我们还将在 NumPy 数组 INITIAL_NUMPY 中定义由 $-\sin(\pi x)$ 给出的初态，在下一个单元用它初始化仿真中的速度 u。这种初始化将在区域中心产生清晰的激波。

Phiflow 采用面向对象设计，以网格形式的场数据为中心（内部由张量对象表示）。也就是说，通过构造若干网格并在时间步中更新它们来组装仿真。Phiflow 内部使用具名维度的张量，这在后续具有额外批次和通道维度的二维仿真中尤其方便；现在只需将一维数组转换为具有单一空间维度`' x'` 的 phiflow 张量。

```
N = 128
DX = 2./N
STEPS = 32
DT = 1./STEPS
NU = 0.01/(N*np.pi)

# initialization of velocities, cell centers of a CenteredGrid have DX/2 offsets
```

[3] 原文文字给出 $0.01/\pi$，但下面代码使用 $0.01/(N\pi)$；这里保留二者，不能视为一致。

```
# for linspace()
INITIAL_NUMPY = np.asarray( [-np.sin(np.pi * x) for x in
    np.linspace(-1+DX/2,1-DX/2,N)] ) # 1D numpy array
INITIAL = math.tensor(INITIAL_NUMPY, spatial('x') ) # convert to phiflow tensor
```

接下来用从 INITIAL NumPy 数组转换而来的张量初始化一维速度网格。区域 Ω 的范围通过 bounds 参数指定为 $[-1, 1]$，网格使用周期边界条件（`extrapolation.PERIODIC`）。这两点是 phiflow 的张量和网格对象的主要区别：后者具有边界条件和物理范围。

作为说明，我们还打印速度对象的一些信息：它是大小为 128 的 phi.math 张量。注意网格实际内容保存在网格的 values 中。下面使用 numpy() 将 phiflow 张量内容转换为 NumPy 数组，并打印其中五项。对更多维度的张量，需要在此指定额外维度，例如二维速度场的`' y,x,vector'`；对于单维张量则可以省略。

```
velocity = CenteredGrid(INITIAL, extrapolation.PERIODIC, x=N, bounds=Box(x=(-1,1)))
vt = advect.semi_lagrangian(velocity, velocity, DT)
#velocity = CenteredGrid(lambda x: -math.sin(np.pi * x),
#    extrapolation.PERIODIC, x=N, bounds=Box(x=(-1,1)))
#velocity = CenteredGrid(Noise(), extrapolation.PERIODIC, x=N,
#    bounds=Box(x=(-1,1))) # random init

print("Velocity tensor shape: "   + format( velocity.shape )) # == velocity.values.shape
print("Velocity tensor type: "    + format( type(velocity.values) ))
print("Velocity tensor entries 10 to 14: " + format( velocity.values.numpy('x')[10:15] ))
```

```
Velocity tensor shape: (x^s=128)
Velocity tensor type: <class 'phi.math._tensors.CollapsedTensor'>
Velocity tensor entries 10 to 14: [0.49289819 0.53499762 0.57580819 0.61523159 0.65317284]
```

3.6.3 运行仿真

现在可以运行仿真了。为了计算模型方程中的扩散和平流分量，只需调用 phiflow 现有的 diffusion 和 semi_lagrangian 算子：`diffuse.explicit(u,...)` 对模型中的 $\nu\nabla\cdot\nabla u$ 项通过中心差分计算一个显式扩散步。接着，`advect.semi_lagrangian(f,u)` 用于稳定地一阶近似速度 u 对任意场 f 的输运。模型中有 $\partial u/\partial t + u\nabla f$，因此在实现中使用 semi_lagrangian 函数以速度输运速度自身：

```
velocities = [velocity]
age = 0.
for i in range(STEPS):
    v1 = diffuse.explicit(velocities[-1], NU, DT)
    v2 = advect.semi_lagrangian(v1, v1, DT)
    age += DT
    velocities.append(v2)
print("New velocity content at t={}: {}".format(
    age, velocities[-1].values.numpy('x,vector')[0:5] ))
```

```
New velocity content at t=1.0: [[0.0057228 ]
 [0.01716715]
```

```
 [0.02861034]
 [0.040052  ]
 [0.05149214]]
```

这里我们实际将所有时间步收集在 velocities 列表中。这通常不是必需的（长时间仿真还可能消耗大量内存），但在这里有助于稍后绘制速度状态的演化。

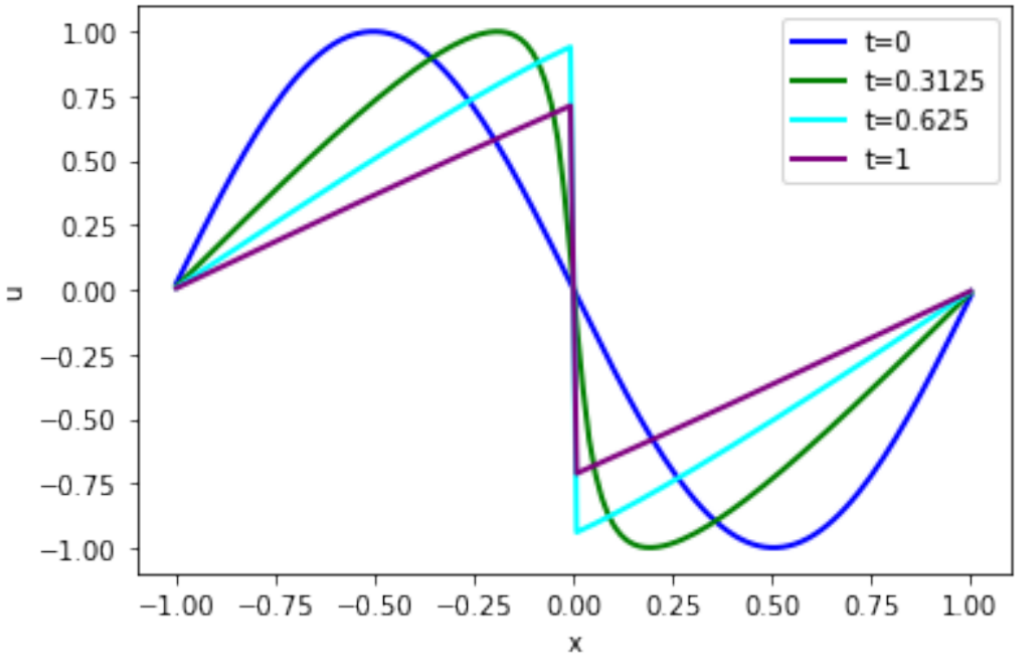
打印结果列出若干速度分量，表明仿真中确实发生了变化；但仅凭这些数字很难直观理解 PDE 的行为。因此，我们将可视化随时间演化的状态，展示发生了什么。

3.6.4 可视化

可以很容易地用图展示这个一维情况：下面的代码以蓝色显示初态，再以绿色、青色和紫色分别显示时间 10/32、20/32 和 1。

```
# get "velocity.values" from each phiflow state with a channel dimension, "vector"
vels = [v.values.numpy('x,vector') for v in velocities] # list of 2D arrays
import pylab
fig = pylab.figure().gca()
fig.plot(np.linspace(-1,1,len(vels[ 0].flatten())), vels[ 0].flatten(),
         lw=2, color='blue',   label="t=0")
fig.plot(np.linspace(-1,1,len(vels[10].flatten())), vels[10].flatten(),
         lw=2, color='green',  label="t=0.3125")
fig.plot(np.linspace(-1,1,len(vels[20].flatten())), vels[20].flatten(),
         lw=2, color='cyan',   label="t=0.625")
fig.plot(np.linspace(-1,1,len(vels[32].flatten())), vels[32].flatten(),
         lw=2, color='purple', label="t=1")
pylab.xlabel('x'); pylab.ylabel('u'); pylab.legend()
```

```
<matplotlib.legend.Legend at 0x7f9fd19fd940>
```

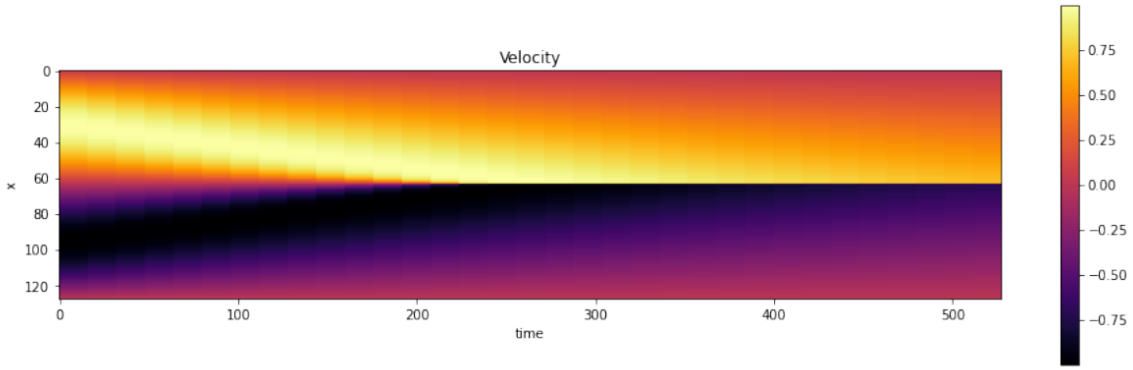


图中清楚地显示了区域中心形成的激波：它来自初始速度的两个“凸起”相碰，左侧正速度向右运动，中心右侧的负速度向左运动。

由于这些曲线可能大量重叠，后续章节还将使用另一种可视化方式，以二维图像显示全部时间步的演化。一维区域沿纵轴显示，横轴上的每一点代表一个时间步。下面代码把收集到的速度状态转换为二维数组，将各时间步重复 8 次以加宽图像。这当然完全是可选的，但有助于看清 Burgers 仿真中发生的过程。[4]

```
def show_state(a, title):
    # we only have 33 time steps, blow up by a factor of 2^4 to make it easier to see
    # (could also be done with more evaluations of network)
    a=np.expand_dims(a, axis=2)
    for i in range(4):
        a = np.concatenate( [a,a] , axis=2)
    a = np.reshape( a, [a.shape[0],a.shape[1]*a.shape[2]] )
    #print("Resulting image size" +format(a.shape))
    fig, axes = pylab.subplots(1, 1, figsize=(16, 5))
    im = axes.imshow(a, origin='upper', cmap='inferno')
    pylab.colorbar(im) ; pylab.xlabel('time'); pylab.ylabel('x'); pylab.title(title)

vels_img = np.asarray( np.concatenate(vels, axis=-1), dtype=np.float32 )
# save for comparison with reconstructions later on
import os; os.makedirs("./temp",exist_ok=True)
np.savez_compressed("./temp/burgers-groundtruth-solution.npz",
                    np.reshape(vels_img,[N,STEPS+1])) # remove batch & channel dimension
show_state(vels_img, "Velocity")
```

[4]原文说重复 8 次，而代码的四次倍增产生 $2^4 = 16$ 倍；此处保留原文及代码。



至此完成第一个 phiflow 仿真。它并不十分复杂，也正因如此，是下一章评估和比较不同基于物理的深度学习方法的良好起点。不过在此之前，下一节将讨论更复杂的仿真类型。

3.6.5 下一步

基于这个仿真设置，可以尝试：

- 自由实验——上面的设置很简单，可以改变仿真参数或初始化。例如，可以通过 `Noise()` 使用噪声场，获得更混沌的结果（参见上面 velocity 代码单元中的注释）。
- 一个稍复杂的尝试是将仿真扩展到二维或更高维。这需要在各处作修改，但上面的所有算子都支持更高维。在尝试之前，可能需要先查看下一个示例，其中介绍了二维 Navier–Stokes 情况。

3.7 Navier–Stokes 正向仿真

现在处理一个稍复杂的例子：基于 Navier–Stokes 方程的流体仿真。使用 ΦFlow（phiflow）仍然非常简单，因为模拟器各个步骤的可微算子已在 phiflow 中提供。不可压缩形式的 Navier–Stokes 方程引入额外的压力场 p 及质量守恒约束，如式 (3.6) 所示。我们还使一个标记场随流运动，这里记为 d。它指示温度较高的区域，并通过浮力因子 ξ 施加力：

$$\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu\nabla\cdot\nabla\mathbf{u} + (0,1)^T\xi d, \qquad \text{满足} \nabla\cdot\mathbf{u} = 0,$$
$$\frac{\partial d}{\partial t} + \mathbf{u}\cdot\nabla d = 0.$$

这里目标是不可压缩流（即 $\rho = \text{const}$），并通过 $(0,1)^T\xi d$ 项使用简单的浮力模型（Boussinesq 近似）。对于不可压缩求解器，这在不显式计算 ρ 的情况下近似密度变化。我们通过矢量 $(0,1)^T$ 假设重力沿 y 方向作用。我们将在物理尺寸为 100×80 个单位的封闭区域 Ω 上求解此 PDE，速度采用 Dirichlet 边界条件 $\mathbf{u} = 0$，压力采用 Neumann 边界条件 $\partial p/\partial x = 0$。在 Colab 中运行的原文链接待核实。

3.7.1 实现

与前一节一样，第一个以 “!” 为前缀的命令通过 pip 从 GitHub 向 Python 环境安装 phiflow 包（必要时跳过或修改此命令）。

```
!pip install --upgrade --quiet phiflow==3.1
#!pip install --upgrade --quiet git+https://github.com/tum-pbs/PhiFlow@develop

from phi.flow import *             # The Dash GUI is not supported on Google colab,
                                   # ignore the warning
import pylab
```

3.7.2 设置仿真

下面的代码设置若干常数，用大写名称表示。我们将使用 40×32 个单元离散区域，通过 ν 引入少量黏性，并令时间步长 $\Delta t = 1.5$。

这里创建第一个 CenteredGrid，并用 Sphere 几何对象初始化。它表示产生热烟的流入区域 INFLOW。

```
DT = 1.5
NU = 0.01
INFLOW = CenteredGrid(Sphere(center=tensor([30,15], channel(vector='x,y')),
    radius=10), extrapolation.BOUNDARY, x=32, y=40,
    bounds=Box(x=(0,80),y=(0,100))) * 0.2
```

流入区域用于向第二个中心网格 smoke 注入烟雾，该网格表示上面的标记场 d。注意，上面定义的 Box 尺寸为 100×80。这是仿真的空间单位所表示的物理尺度，即大小为 1 的速度在一个时间单位内使烟雾密度移动 1 个单位；根据 x、y 的设置，这可能大于或小于离散网格中的一个单元。可以将仿真网格参数化，使其直接对应真实世界单位，也可以记住合适的转换因子。

上面的流入球已经使用“世界”坐标：它位于第一轴 $x = 30$、$y = 15$ 处（在 100×80 的区域盒内）。接下来为待仿真量创建网格。本例需要一个速度场和一个烟雾密度场。

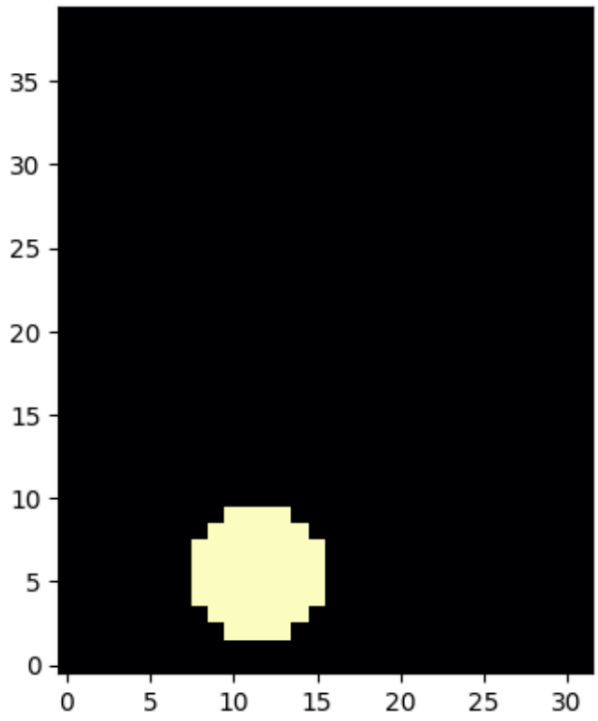
```
smoke = CenteredGrid(0, extrapolation.BOUNDARY, x=32, y=40,
    bounds=Box(x=(0,80),y=(0,100)))  # sampled at cell centers
velocity = StaggeredGrid(0, extrapolation.ZERO, x=32, y=40,
    bounds=Box(x=(0,80),y=(0,100)))  # sampled in staggered form at face centers
```

烟雾场在单元中心采样，速度采用交错形式采样。交错网格内部包含两个具有不同维度的中心网格，可通过 unstack 函数转换为中心网格（或简单的 NumPy 数组），如上文链接所述。

接下来定义仿真更新步，它调用所需函数将流体系统状态推进 dt。下一个代码单元计算一个这样的步，并绘制一帧仿真后的标记密度。

```
def step(velocity, smoke, pressure, dt=1.0, buoyancy_factor=1.0):
    smoke = advect.semi_lagrangian(smoke, velocity, dt) + INFLOW
    buoyancy_force = (smoke * (0, buoyancy_factor)).at(velocity)
    # resamples smoke to velocity sample points
    velocity = advect.semi_lagrangian(velocity, velocity, dt) + dt * buoyancy_force
    velocity = diffuse.explicit(velocity, NU, dt)
    velocity, pressure = fluid.make_incompressible(velocity)
    return velocity, smoke, pressure

velocity, smoke, pressure = step(velocity, smoke, None, dt=DT)
print("Max. velocity and mean marker density: " + format(
```

```
    [ math.max(velocity.values), math.mean(smoke.values) ] ))
pylab.imshow(np.asarray(smoke.values.numpy('y,x')), origin='lower', cmap='magma')

Max. velocity and mean marker density: [0.1558497, 0.008125]
<matplotlib.image.AxesImage at 0x7ebc74337340>
```



在这次 step() 调用中发生了很多事情：我们平流输运烟雾场，通过 Boussinesq 模型添加向上的力，平流输运速度场，最后通过压力求解使速度无散度。

Boussinesq 模型使用与元组 (0, buoyancy_factor) 相乘的方式，将烟雾场变为具有两个分量的交错力场，并通过 at() 函数在速度分量的位置采样。此函数确保各力分量正确插值到交错速度的相应分量位置；注意它也直接确保保留原网格的边界条件。内部还会对所得力网格执行 `StaggeredGrid(..., extrapolation.ZERO,...)`。

make_incompressible 中的压力投影通常是上述序列中计算代价最大的步骤。它根据区域的边界条件求解 Poisson 方程，并使用所得压力的梯度更新速度场。

为了测试，我们还打印了更新后速度的均值和密度的最大值。[5]从结果图中可以看到第一个圆形烟雾区域，并有轻微向上的运动（此处尚未显示出来）。

3.7.3 数据类型与维度

这里为仿真场创建的变量是 Grid 类的实例。与张量一样，网格也有 shape 属性，列出所有批次、空间和通道维度。Phiflow 中的 shape 不仅存储维度大小，还存储名称和类型。

```
print(f"Smoke: {smoke.shape}")
```

[5]原文这句把统计量顺序写反：上面代码和输出实际计算最大速度与标记密度均值。

```
print(f"Velocity: {velocity.shape}")
print(f"Inflow: {INFLOW.shape}, spatial only: {INFLOW.shape.spatial}")

Smoke: (x^s=32, y^s=40)
Velocity: (x^s=32, y^s=40, vector[unreadable]=x,y)
Inflow: (x^s=32, y^s=40), spatial only: (x^s=32, y^s=40)
```

注意这里 phiflow 的输出指示了维度类型，例如 S 表示空间维度，V 表示矢量维度。后面进行学习时还将引入批次维度。[6]

可以通过.sizes 获得 shape 对象的实际内容，也可以通过`.get_size(' dim')` 查询指定维度 dim 的大小。下面是两个例子:

```
print(f"Shape content: {velocity.shape.sizes}")
print(f"Vector dimension: {velocity.shape.get_size('vector')}")

Shape content: (32, 40, 2)
Vector dimension: 2
```

可以通过 values 属性访问网格值。这是与 phiflow 张量对象的重要区别: 后者没有 values，如下面的代码示例所示。

```
print("Statistics of the different simulation grids:")
print(smoke.values)
print(velocity.values)

# in contrast to a simple tensor:
test_tensor = math.tensor(numpy.zeros([3, 5, 2]), spatial('x,y'),
                          channel(vector="x,y"))
print("Reordered test tensor shape: " + format(
    test_tensor.numpy('vector,y,x').shape) ) # reorder to vector,y,x
#print(test_tensor.values.numpy('y,x')) # error! tensors don't return
                                        # their content via ".values"

Statistics of the different simulation grids:
(x^s=32, y^s=40) 0.008 +/- 0.039 (0e+00...2e-01)
(~vector[unreadable]=x,y, x^s=~(x=31, y=32) int64, y^s=~(x=40, y=39) int64)
  -7.23e-09 +/- 2.8e-02 (-1e-01...2e-01)
Reordered test tensor shape: (2, 5, 3)
```

网格还有许多其他属性，原文提供了相应文档链接。还要注意，交错网格具有非均匀形状，因为面数不等于单元数（在这个例子中，x 分量有 31×40 个单元，而 y 分量有 32×39 个）。INFLOW 网格自然具有与 smoke 网格相同的维度。

3.7.4 时间演化

有了上述设置，通过反复调用 step 函数，就可以轻松地将仿真进一步向前推进。

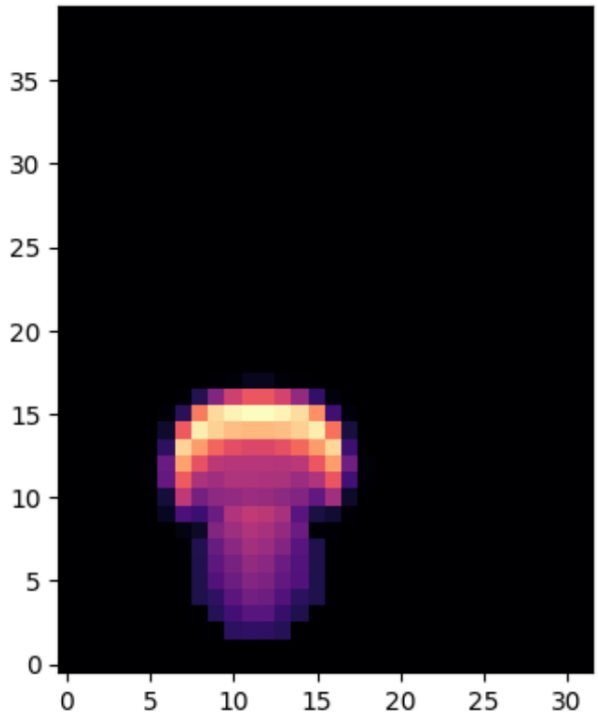
[6]原 PDF 的矢量维度标记显示为缺字，输出中以 [unreadable] 表示，不猜测其字形；空间上标以 `^s`、正负号以 +/- 转写。

```
for time_step in range(10):
    velocity, smoke, pressure = step(velocity, smoke, pressure, dt=DT)
    print('Computed frame {}, max velocity {}'.format(
        time_step , np.asarray(math.max(velocity.values)) ))
```

```
Computed frame 0, max velocity 0.4630011022090912
Computed frame 1, max velocity 0.8966455459594727
Computed frame 2, max velocity 1.4098880290985107
Computed frame 3, max velocity 2.0411267280578613
Computed frame 4, max velocity 2.9279565811157227
Computed frame 5, max velocity 3.839479923248291
Computed frame 6, max velocity 4.526943206787109
Computed frame 7, max velocity 4.867981910705566
Computed frame 8, max velocity 5.131079196929932
Computed frame 9, max velocity 5.483874320983887
```

现在热烟羽开始上升：

```
pylab.imshow(smoke.values.numpy('y,x'), origin='lower', cmap='magma')
<matplotlib.image.AxesImage at 0x7ebc7431dd50>
```

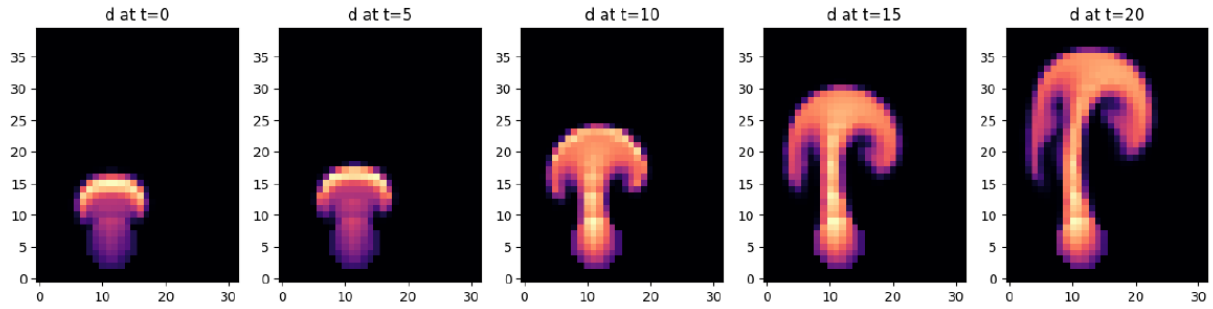


再计算并展示几个仿真步。由于流入区域偏左（x 位置为 30），烟羽碰到区域上壁时将向右弯曲。

```
steps = [[ smoke.values, velocity.vector['x'], velocity.vector['y'] ]]
for time_step in range(20):
  if time_step<3 or time_step%10==0:
    print('Computing time step %d' % time_step)
```

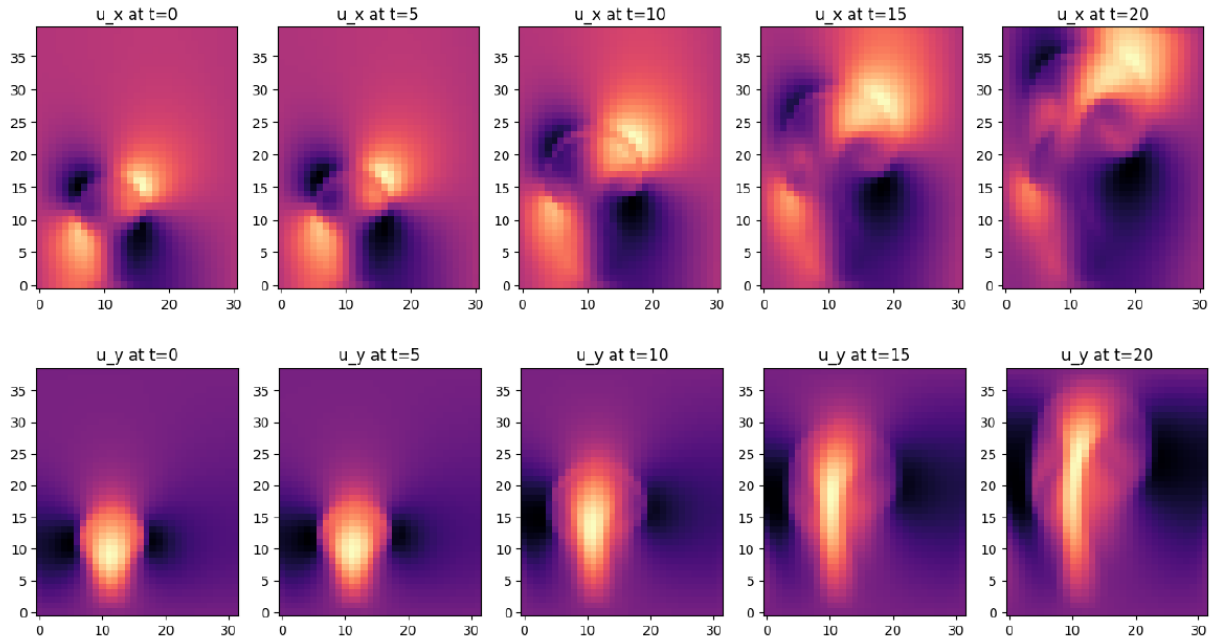
```
  velocity, smoke, pressure = step(velocity, smoke, pressure, dt=DT)
  if time_step%5==0:
    steps.append( [smoke.values, velocity.vector['x'], velocity.vector['y']] )

fig, axes = pylab.subplots(1, len(steps), figsize=(16, 5))
for i in range(len(steps)):
    axes[i].imshow(steps[i][0].numpy('y,x'), origin='lower', cmap='magma')
    axes[i].set_title(f"d at t={i*5}")
```

```
Computing time step 0
Computing time step 1
Computing time step 2
Computing time step 10
```



也可以看看速度。上面的 steps 列表已经以 NumPy 数组保存了速度的 vector[0] 和 vector[1] 分量，接下来可以展示它们。

```
fig, axes = pylab.subplots(1, len(steps), figsize=(16, 5))
for i in range(len(steps)):
    axes[i].imshow(steps[i][1].numpy('y,x'), origin='lower', cmap='magma')
    axes[i].set_title(f"u_x at t={i*5}")

fig, axes = pylab.subplots(1, len(steps), figsize=(16, 5))
for i in range(len(steps)):
    axes[i].imshow(steps[i][2].numpy('y,x'), origin='lower', cmap='magma')
    axes[i].set_title(f"u_y at t={i*5}")
```



尽管这里看起来很简单，这一仿真设置却是强大的工具。它可以轻松扩展到更复杂的情形或三维，而且已经完全兼容深度学习框架中的反向传播流程。

后续章节将展示如何使用这些仿真训练神经网络，以及如何通过训练后的神经网络控制和修改仿真。这将说明，在训练循环中引入求解器，尤其是可微求解器，能在多大程度上改善训练过程。在进入这些更复杂的训练过程之前，下一章将介绍更简单的监督方法。这是非常基础的：即使目标是高级的基于物理的学习设置，能够正常工作的监督训练始终是第一步。

3.7.5 下一步

基于这一设置，可以创建多种有趣的流体仿真。例如，尝试改变空间分辨率、浮力因子及仿真的总运行长度。

3.8 优化与收敛

本节概述不同优化算法的推导。与其他文本不同，我们从最经典的优化算法牛顿法出发，由此推导出几种广泛使用的变体，然后回到深度学习（DL）优化器。主要目标是将深度学习放在这些经典方法的背景中理解。尽管重点是深度学习，本书后面还将回顾经典算法，以改进学习算法。物理仿真会放大神经网络带来的困难，因此下面的主题与基于物理的学习任务特别相关。

注：深入讨论 本节面向对不同优化器理论感兴趣的读者，跳过评估和源代码，专注于理论。强烈建议将其作为“尺度不变性与反演”相关章节的基础，但开始学习可微物理训练等主题时并非“必读”。如果想快速开始实践，可以直接跳到“监督训练”。

3.8.1 记号

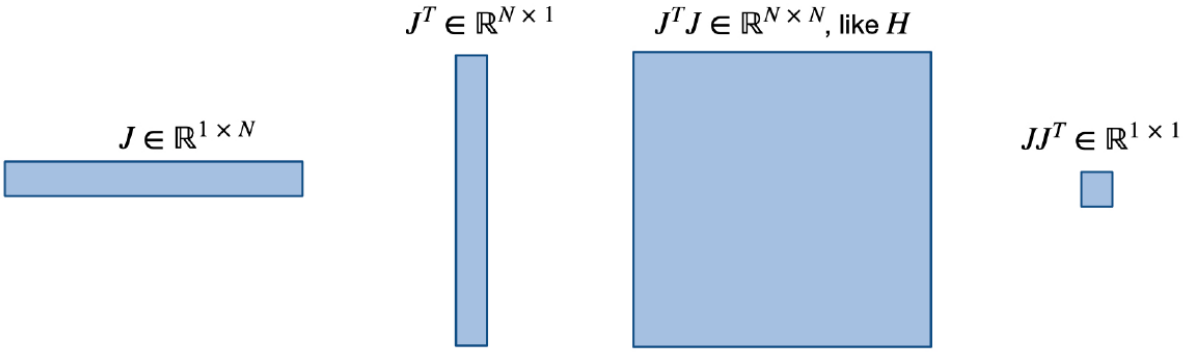
本节使用精心选择的专用记号，以清晰简洁地表示所考虑的全部方法。设有标量损失函数 $L(x)\colon \mathbb{R}^N \to \mathbb{R}$，最优点（$L$ 的最小值）位于 x^*，Δ 表示 x 的一步。x 的不同中间更新步骤用下标表示，例如 x_n 或 x_k。

下面经常需要求逆，即除以某个量。对矩阵 A 和 B，定义 $A/B \equiv B^{-1}A$。当 a 和 b 为矢量时，结果是由下列两种形式之一得到的矩阵；具体使用哪一种将在各处说明：

$$\frac{a}{b} \equiv \frac{aa^T}{a^Tb} \quad \text{或} \quad \frac{a}{b} \equiv \frac{ab^T}{b^Tb}. \tag{3.8}$$

对 L 作用一次 $\partial/\partial x$ 得到 Jacobian $J(x)$。由于 L 为标量，J 是行向量，梯度（列向量）∇L 由 J^T 给出。再作用一次 $\partial/\partial x$ 得到 Hessian 矩阵 $H(x)$，再作用一次得到记为 $K(x)$ 的三阶导数张量。所幸我们无须将 K 作为完整张量计算，但下面某些推导需要它。为简化记号，当函数或导数在位置 x 处求值时，下文通常省略 (x)，例如用 J 表示 $J(x)$。

下图概述一些常用量所得矩阵的形状。后文其实不再需要它；图中 N 表示 x 的维数，即 $x \in \mathbb{R}^N$。



3.8.2 预备知识

下面的推导需要一些工具，现汇总于此供参考。不出意料，我们需要 Taylor 级数展开。按上述记号，它写为

$$L(x+\Delta) = L + J\Delta + \tfrac{1}{2}H\Delta^2 + \cdots.$$

我们还需要 Lagrange 形式：对于区间 $[x, x+\Delta]$ 中的某个 ξ，它给出精确结果

$$L(x+\Delta) = L + J\Delta + \tfrac{1}{2}H(\xi)\Delta^2.$$

我们将在若干地方使用微积分基本定理，为完整起见重复如下：

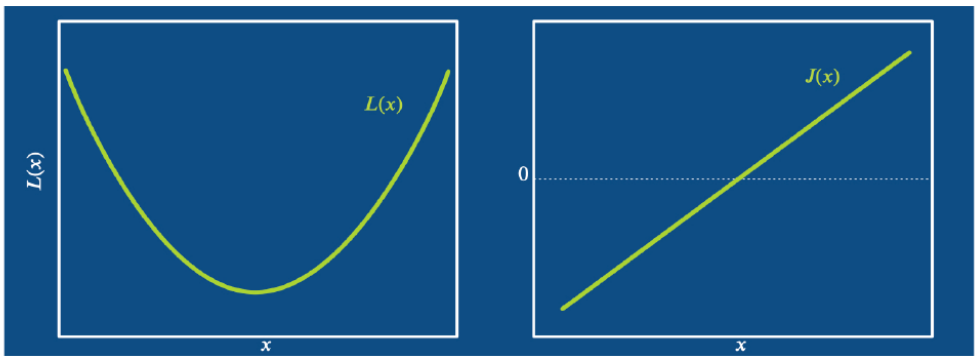
$$f(x+\Delta) = f(x) + \int_0^1 \mathrm{d}s\, f'(x+s\Delta)\Delta.$$

此外还将使用常数为 $\mathcal{L}$ 的 Lipschitz 连续性 $|f(x+\Delta)+f(x)| \le \mathcal{L}\Delta$，以及著名的 Cauchy–Schwarz 不等式 $u^Tv \le |u|\cdot|v|$。[7]

[7]这里 Lipschitz 式的左侧加号照原文保留；一般的 Lipschitz 条件通常涉及函数值之差。此外多维情形中原文的区间、$H\Delta^2$ 与 $\mathcal{L}\Delta$ 的具体含义需独立核查。

3.8.3 牛顿法

现在从可以说是最经典的优化算法——牛顿法——开始。它通过把感兴趣的函数近似为抛物线而导出。这可以由几乎所有极小值在足够近处看起来都像抛物线这一事实来说明。



因此可以在最优点 x^* 附近用抛物线 $L(x) = \frac{1}{2}H(x - x^*)^2 + c$ 表示 L，其中 c 为常数偏移。在位置 x 处观察到 H 和 $J^T = H \cdot (x_k - x^*)$。重新排列即可直接得到计算最小值的方程 $x^* = x_k - J^T/H$。牛顿法默认一步计算出 x^*，因此其在 x 中的更新量为

$$\Delta = -\frac{J^T}{H}. \tag{3.9}$$

看看牛顿法的收敛阶。对 L 的最优点 x^*，令 $\Delta_n^* = x^* - x_n$ 表示从当前 x_n 到最优点的步长，如下图所示。

$$x_n \xrightarrow{\Delta_n^*} x^*$$

假设 J 可微，可以在 x^* 对 J^T 进行 Lagrange 展开：

$$\begin{aligned} 0 = J^T(x^*) &= J^T(x_n) + H(x_n)\Delta_n^* + \tfrac{1}{2}K(\xi_n)\Delta_n^{*2}, \\ \frac{J^T}{H} &= -\frac{K}{2H}\Delta_n^{*2} - \Delta_n^*. \end{aligned}$$

第二行已除以 H，并为简洁起见省略 (x_n) 和 (ξ_n)。将它代入 Δ_n^* 得到：

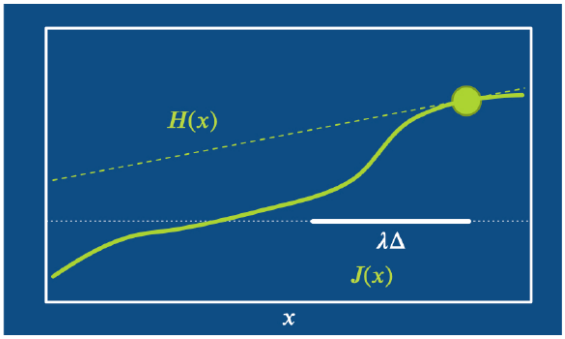
$$\begin{aligned} \Delta_{n+1}^* &= x^* - x_{n+1} \\ &= x^* - \left(x_n - \frac{J^T}{H}\right) \\ &= \Delta_n^* - \frac{K}{2H}\Delta_n^{*2} - \Delta_n^* \\ &= -\frac{K}{2H}\Delta_n^{*2}. \end{aligned}$$

因此到最优点的距离按 Δ_n^{*2} 变化，这意味着一旦足够接近便有二次收敛。这当然很好，但它仍依赖于前因子 $K/(2H)$；原文称该因子大于 1 时会发散。注意，由于使用 Lagrange 展开，这是精确表达式，没有截断。到目前为止我们有二次收敛，但并不保证收敛至最优点，因此需要允许可变步长。[8]

[8] 前因子大于 1 本身不足以判定局部迭代发散；该说法以及多维 Lagrange 余项的适用条件留待独立核查。

3.8.4 自适应步长

因此，作为牛顿法的下一步，我们引入可变步长 λ，得到迭代 $x_{n+1} = x_n + \lambda\Delta = x_n - \lambda J^T/H$。如下图所示，如果 L 并非严格的抛物线，较小的 H 可能导致不期望的过冲，此时可变步长尤其有帮助。原文接着写道：“本例中最左侧：”



为了讨论收敛，需要对损失函数作一些基本假设：凸性和平滑性。然后我们将着重证明损失减小，并沿着损失值更低的一系列较小集合 $\forall x\ L(x) < L(x_n)$ 移动。

首先对 L 应用微积分基本定理：

$$L(x + \lambda\Delta) = L + \int_0^1 \mathrm{d}s\, J(x + s\lambda\Delta)\lambda\Delta,$$

同样用它表示该位置附近的 J：

$$J(x + s\lambda\Delta) = J + s\lambda\Delta^T \int_0^1 \mathrm{d}t\, H(x + st\lambda\Delta).$$

将这个 J^T 代入 L 得到：

$$\begin{aligned}
L(x + \lambda\Delta) - L(x) &= \int_0^1 \mathrm{d}s \left[J + s\lambda\Delta^T \int_0^1 \mathrm{d}t\, H(x + st\lambda\Delta) \right]^T \lambda\Delta \\
&= \int_0^1 \mathrm{d}s \left[-H\Delta + \int_0^1 \mathrm{d}t\, H(x + st\lambda\Delta) s\lambda\Delta \right]^T \lambda\Delta \\
&= \int_0^1 \mathrm{d}s \int_0^1 \mathrm{d}t \left[-H\Delta + H(x + st\lambda\Delta) s\lambda\Delta \right]^T \lambda\Delta \\
&= \int_0^1 \mathrm{d}s \int_0^1 \mathrm{d}t \left[-H\Delta(1 + \lambda s) + [H(x + st\lambda\Delta) - H] s\lambda\Delta \right]^T \lambda\Delta \\
&= -(H\Delta)^T (1 + \tfrac{\lambda}{2})\lambda\Delta + \lambda^2 \int_0^1 \mathrm{d}s\, s \int_0^1 \mathrm{d}t \left[[H(x + st\lambda\Delta) - H]\Delta \right]^T \Delta.
\end{aligned} \tag{3.10}$$

这里首先使用了 $\Delta = -J^T/H$，并在第三行将其与 $s\lambda\Delta$ 项一起移入积分。第四行提出 $H\Delta$，从而能计算上式末行第一项 $-H\Delta(1 + \lambda s)$ 的积分。[9]

接下来的步骤首先使用 $(H\Delta)^T\Delta = \|H^{1/2}\Delta\|^2$。下面第二行将这一项简记为 $\epsilon \equiv \|H^{1/2}\Delta\|^2$。根据 H 的性质，原文称 ϵ “只是”一个延续到最后的小正因子。随后在下面第 3、4 行开始寻找损失变化的

[9] 式 (3.10) 从第三行到第四行的符号及后续结果与直接代数展开存在疑点；第一行转置与原文对 J 为行向量的约定亦需核查。现逐行保留原式，不将其当作已证明推导。

上界：先使用 Cauchy–Schwarz 不等式，再使用针对仿射共轭矩阵的特殊 Lipschitz 条件。对于 H，其形式为

$$\left\|H(x)^{-1/2}(H(y)-H(x))\right\| \le \mathcal{L}\left\|H(x)^{1/2}(y-x)\right\|^2.$$

这要求 H 对称且正定，在实践中并非太不合理。接上面的推导：

$$\begin{aligned}
\cdots = & -\left(\lambda+\frac{\lambda^2}{2}\right)\|H^{1/2}\Delta\|^2+\lambda^2\int_0^1 \mathrm{d}s\, s\int_0^1 \mathrm{d}t\,[[H(x+st\lambda\Delta)-H]\Delta]^T H^{-1/2}H^{1/2}\Delta \\
= & -\lambda\epsilon+\frac{\lambda^2\epsilon}{2}+\lambda^2\int_0^1 \mathrm{d}s\, s\int_0^1 \mathrm{d}t\,[H^{-1/2}[H(x+st\lambda\Delta)-H]\Delta]^T H^{1/2}\Delta \\
\le & -\lambda\epsilon+\frac{\lambda^2\epsilon}{2}+\lambda^2\int_0^1 \mathrm{d}s\, s\int_0^1 \mathrm{d}t\,\|H^{-1/2}[H(x+st\lambda\Delta)-H]\Delta\|\,\|H^{1/2}\Delta\| \\
\le & -\lambda\epsilon+\frac{\lambda^2\epsilon}{2}+\lambda^3\int_0^1 \mathrm{d}s\, s^2\int_0^1 \mathrm{d}t\, t\,\mathcal{L}\|H^{1/2}\Delta\|^2\|H^{1/2}\Delta\| \\
= & -\lambda\epsilon+\frac{\lambda^2\epsilon}{2}+\mathcal{L}\lambda^3\int_0^1 \mathrm{d}s\, s^2\int_0^1 \mathrm{d}t\, t\,\|H^{1/2}\Delta\|^3 \\
= & -\lambda\epsilon+\frac{\lambda^2\epsilon}{2}+\mathcal{L}\lambda^3\int_0^1 \mathrm{d}s\, s^2\int_0^1 \mathrm{d}t\, t\,\epsilon^{3/2} \\
= & -\lambda\epsilon+\frac{\lambda^2\epsilon}{2}+\frac{\lambda^3\mathcal{L}\epsilon^{3/2}}{6}.
\end{aligned} \tag{3.11}$$

由于 $H^T=H$，我们在第二行将 $H^{-1/2}$ 移入积分。第四行在应用特殊 Lipschitz 条件的同时，尽可能将 s,t,λ 从积分中提出。最后三行简化各项，用 ϵ 表示 H 的出现并计算积分。这样得到关于步长 λ 的三次形式。原文强调，第一项线性项为负，因此在较小 λ 下占主导，认为这表明充分小的 λ 会使步长为负：

$$L(x+\lambda\Delta)=L(x)-\lambda\epsilon+\frac{\lambda^2\epsilon}{2}+\frac{\lambda^3\mathcal{L}\epsilon^{3/2}}{6}. \tag{3.12}$$

[10] 不过，这本质上要求能够自由选择 λ，因此证明不适用于前面的固定步长。它的一个优点是“只”要求 H Lipschitz 连续，而不是要求 J 甚至 L 满足该条件。

原文总结称，已经证明自适应步长牛顿法收敛。然而，它需要以 Hessian H 为核心，而实践中很难获得 H。这成为实际障碍，也促使人们发展下列方法：保留牛顿法的基本步，但近似 H。

3.8.5 近似 Hessian

接下来回顾三种从牛顿法导出的常用算法。

3.8.6 Broyden 方法

近似 H 的第一种方法是先作一个非常粗略的猜测，从单位矩阵 $H_0=\mathbf{1}$ 开始，然后使用有限差分近似迭代更新 H_n。Broyden 方法使用向量除法 $a/b\equiv ab^T/(b^Tb)$。

为了简化有限差分，再假设在当前位置 x_n 已达到 $J(x_n)=0$。这当然不一定成立，但它得到下列经过大幅简化的表达式，用于在优化过程中修改 H：

$$H_{n+1}=H_n+\frac{J(x_{n+1})^T}{\Delta}.$$

[10] 式 (3.11) 首、二行的 $\lambda^2\epsilon/2$ 符号不同；由前面的不等式得到式 (3.12) 时原文写成等式。特殊 Lipschitz 条件的幂次与后续代换、以及仅凭下降得出收敛的条件也需独立审查。以上均照录。

与前面一样，对 x 使用步长 $\Delta = -J^T/H$；分母来自有限差分 $[J(x_{n+1})^T - J(x_n)^T]/\Delta$，并假设当前 Jacobian 为零。[11]记住这里的 Δ 是向量（见上面的向量除法），所以有限差分给出一个 $N \times N$ 矩阵，可以加到 H_n。

Broyden 方法的优点是无须计算完整 Hessian，并且 H 的更新可以高效计算。但上述两个假设使它成为非常粗糙的近似，因此通过 J^T/H 中的逆 Hessian 对更新步进行归一化也相应地不可靠。

3.8.7 BFGS

这引出了 BFGS 算法（以 Broyden–Fletcher–Goldfarb–Shanno 命名），它引入了重要改进：不再立刻假设 J 为零，并补偿更新中的冗余部分。这是必要的，因为有限差分 $[J(x_{n+1}) - J(x_n)]/\Delta_n$ 给出 H 的完整近似。我们可以尝试某种平均过程，但这会严重损害 H_n 中已有的内容。因此，我们只减去 H_n 中沿当前步 Δ 的现有分量。这是合理的，因为有限差分近似恰好给出沿 Δ 的估计。结合向量除法 $a/b \equiv aa^T/(a^Tb)$，这些变化给出 H 的更新步：

$$H_{n+1} = H_n + \frac{J(x_{n+1})^T - J(x_n)^T}{\Delta_n} - \frac{H_n\Delta_n}{\Delta_n}.$$

实践中 BFGS 也使用线搜索确定步长 λ。由于 H 很大，常用 BFGS 变体还使用 H 的简化表示以节省内存。尽管如此，沿搜索方向通过有限差分近似更新 Hessian 是 BFGS 的核心，使它至少能部分补偿损失景观的尺度效应。目前 BFGS 变体是求解经典非线性优化问题最广泛使用的算法。

3.8.8 Gauss–Newton 方法

如果将 L 限制为经典的 L^2 损失，还可以导出牛顿法的另一种有吸引力的变体，即 Gauss–Newton（GN）算法。我们仍使用 $\Delta = -J^T/H$，但对任意 $f(x)$ 使用平方损失 $L = |f|^2$。与前面 L 的一般导数 J, H 相区别，f 的导数记为 J_f, H_f。根据链式法则，$J = 2f^TJ_f$。

二阶导数给出下式。对于 GN，我们在下面第二行省略二阶项以作简化：

$$\begin{aligned} H &= 2J_f^TJ_f + 2f^TH_f \\ &\approx 2J_f^TJ_f \\ &= 2\frac{J^TJ}{4|f|^2} \\ &= \frac{J^TJ}{2L}. \end{aligned} \tag{3.13}$$

由于关注 L^2 损失，一阶近似剩余的 $J_f^TJ_f$ 项可以化简：$J_f = J/(2f^T)$ 且 $|f|^2 = L$。式 (3.13) 的最后一行意味着基本上用 J 的平方近似 Hessian。原文称这在许多情形下合理，将它代入更新步得到 Gauss–Newton 更新 $\Delta_{\text{GN}} \approx -J^T/(J^TJ)$。观察这一更新，它本质上使用 $\Delta_{\text{GN}} \approx -1/J$ 形式的步，即基于 L 的 Jacobian 的近似逆。这种逆使所有参数的步长近似相等，因而提供了一个有趣的构件，后续章节将再次讨论。[12]

[11]译注：若真实梯度 $J(x_n) = 0$，再取所写 Newton 步就得到 $\Delta = 0$，向量除法分母也为 0，因此这组简化假设不能按字面共同使用。good-Broyden 完整更新 $B + (y - B\Delta)\Delta^T/(\Delta^T\Delta)$ 可在完整 Newton 关系 $B\Delta = -g_n$ 下化为源式形式，不需假定真实梯度为零；不是一般无条件更新。

[12]式 (3.13) 从 $J_f^TJ_f$ 到 $J^TJ/(4|f|^2)$ 的化简对一般向量值 f 并不自动成立；例如 $f = (x, y)$、$L = x^2 + y^2$ 时，$2J_f^TJ_f = 2I$，而 $J^TJ/(2L)$ 在非零点秩仅为 1，原等号一般不成立。即使标量 f，把该 Hessian 近似代回 Newton 步也仍带 $2L$ 因子，源后文将其省去。原式保留，不作为一般 GN 恒等式。

上述形式仍意味着必须求逆一个大矩阵，代价高昂，而且矩阵本身甚至可能不可逆。

3.8.9 回到深度学习

上面已经说明牛顿法是许多常用非线性优化算法的核心。现在 Gauss–Newton 最终为我们提供了回到深度学习算法、特别是 Adam 优化器的桥梁。

3.8.10 Adam

如往常一样，从牛顿步 $\Delta = -\lambda J^T/H$ 出发，但即便采用 Gauss–Newton 的最简单近似 $H \approx J^T J$，也需要求逆可能极其巨大的矩阵。对于神经网络权重，这是不可行的。因此一个合理的问题是如何进一步简化这一过程。Adam 的答案是使用对角近似。具体来说，Adam 使用

$$H \approx \sqrt{\operatorname{diag}(J^T J)}. \tag{3.14}$$

这是真实 Hessian 的非常粗略的近似。这里只使用了一阶导数的平方，而一般而言，当然有 $(\partial f/\partial x)^2 \neq \partial^2 f/\partial x^2$。原文称这只对 Gauss–Newton 的一阶近似，即式 (3.13) 的第一项成立。现在 Adam 更进一步，只保留 $J^T J$ 的对角线。这一量在深度学习中可以由权重梯度直接获得，使 H 的求逆变得简单。因此，它至少提供了各个权重曲率的某种估计，但忽略了它们之间的相关性。

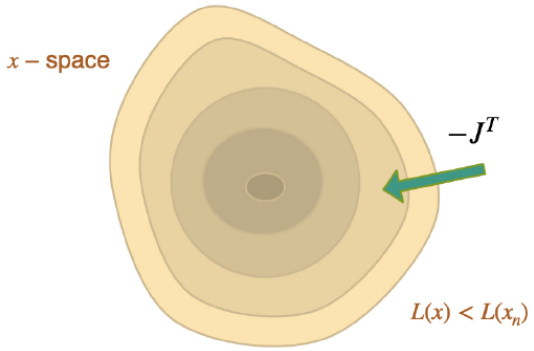
有趣的是，Adam 并不通过 $\operatorname{diag}(J^T J)$ 进行完整求逆，而是逐分量取平方根。这实际上得到 $\sqrt{\operatorname{diag}(J^T J)} \approx \sqrt{\operatorname{diag}(J^2)} \approx \operatorname{diag}(J)$。因此，Adam 沿 $\Delta \approx \operatorname{sign}(-J)$ 移动，近似在所有维度上执行固定大小的步。

实践中，Adam 还引入了一些技巧：使用动量计算梯度 J^T 及梯度平方，也就是对优化迭代过程中的两个量取平均。这使估计更稳健，十分重要：否则，用错误地偏小的梯度分量进行归一化可能导致数值爆炸。Adam 在除法时还加入一个小常数，平方根也有助于缓解过冲。

原文总结：Adam 使用带 Hessian 对角 Gauss–Newton 近似归一化的一阶更新，并额外使用动量来稳定。[13]

3.8.11 梯度下降

为了得到梯度下降（GD）优化，最后假设 $-\lambda J^T/H$ 中 $H = \mathbf{1}$。这样只剩下由缩放梯度组成的更新量 $\Delta = -\lambda J^T$。



[13] 本段保留作者的启发式解释；Adam 的动量、二阶矩和偏差修正并非由上述单步等式严格推出，此外 $\sqrt{\operatorname{diag}(J^T J)}$ 的对角元为 $|J_i|$，不是 J_i；源 Hessian 类比只属启发式，不是一般 Hessian 或 GN 恒等式。

有趣的是，在没有任何形式的求逆时，J^T 本身具有能够加到 x 上的正确形状，但它“存在”于错误的空间 $1/x$，而不是 x 本身。上面所有“经典”方案以某种形式使用的 J^{-1} 与 J^T 本质不同，这一点说明了这一问题。缺乏求逆步骤在涉及物理的问题中尤其突出，因此是本章的关键结论之一。后文将在“尺度不变性与反演”中更详细地讨论这一点。现在建议记住不同算法之间的关系，即使“仅仅”使用 Adam 或 GD 也是如此。

最后来看看 GD 的收敛性。再次假设 L 凸且可微。对步长 Δ 进行 Taylor 展开，并用 Lipschitz 条件 $J(x+\Delta)-J(x)\leq\mathcal{L}\Delta$ 约束二阶项，得到

$$\begin{aligned} L(x+\Delta)-L(x) &\leq J(x)\Delta+\frac{\mathcal{L}}{2}|\Delta|^2 \\ &= J(-\lambda J^T)+\frac{\mathcal{L}}{2}|-\lambda J^T|^2 \\ &= -\lambda JJ^T+\lambda^2\frac{\mathcal{L}}{2}|J|^2 \\ &= -\lambda|J|^2+\frac{\lambda^2\mathcal{L}}{2}|J|^2. \end{aligned}$$

与上面式 (3.11) 中的牛顿法类似，这里有一个负的线性项，在充分小的 λ 下支配损失变化。综合而言，由第一行的 Lipschitz 条件得到上界 $L(x+\Delta)\leq L(x)-\lambda|J|^2+\lambda^2\mathcal{L}|J|^2/2$。选择 $\lambda\leq 1/\mathcal{L}$，可以进一步简化为依赖于 J 的平方的上界 $L(x+\Delta)\leq L(x)-\lambda|J|^2/2$，原文称这确保收敛。[14]

遗憾的是，这个结果在实践中帮助不大，因为深度学习中通常使用 GD 时并不知道 $\mathcal{L}$。但仍值得知道，梯度的 Lipschitz 常数在理论上能为 GD 提供收敛保证。

至此结束对经典优化器及其与深度学习方法关系的概览。为清晰起见，我们关注了非随机算法；随机算法的证明会更加复杂。

有了所有这些背景知识，是时候开始看一些尽可能简单的实践例子：使用完全监督的方法进行训练。

[14]原文此处的向量 Lipschitz 条件未写范数；损失下降本身与收敛到某个最优点所需条件不同。保留其表述，留待独立数学审校。

第二部分

神经代理模型与算子

第四章　监督训练

我们首先讨论与经典机器学习一致的“纯”数据驱动方法。下文称其为监督方法，表示网络完全由数据监督，也以此与使用基于物理的损失相区别。监督方法的核心优势之一是得到一个代理模型（也称“仿真器”或“神经算子”），即一个模仿原始 $\mathcal{P}$ 行为的新函数。

纯数据驱动的监督训练是深度学习背景下所有项目的核心起点。虽然与物理耦合更紧密的方法相比，它可能产生次优结果，但在不存在良好模型方程的某些应用场景中，它可能是唯一选择。本章还将介绍不同神经网络架构的基础知识。除了训练方法之外，架构也是一个重要选择。

4.1　问题设置

对于监督训练，我们面对一个未知函数 $f^*(x) = y^*$，收集大量数据对 $[x_0, y_0^*], \ldots, [x_n, y_n^*]$（训练数据集），直接训练神经网络表示 f^* 的一个近似，记为 f。

以这种方式获得的 f 通常不是精确的，而是通过一个最小化问题得到：调整网络表示 f 的权重 θ，使训练集中所有数据点的误差最小：

$$\arg\min_{\theta} \sum_i \left(f(x_i;\theta) - y_i^*\right)^2 . \tag{4.1}$$

这样得到的 θ 使 $f(x;\theta) = y \approx y^*$ 在所选 f 和训练超参数的条件下尽可能准确。注意上面假设了最简单的 L^2 损失。更一般的形式会在待最小化的损失 L 中使用误差度量 $e(x,y)$，通过 $\arg\min_\theta \sum_i e(f(x_i;\theta), y_i^*)$ 求解。合适度量的选择将在后文讨论。上面的最小化构成实际的“学习”过程；由于 f 通常是非线性函数，这并不简单。

训练数据通常需要相当大的规模，因此利用求解物理模型 $\mathcal{P}$ 的数值仿真产生大量可靠的训练输入输出对很有吸引力。这意味着训练过程使用一组模型方程并对其作数值近似，以拟合神经网络表示 f。这有不少优点，例如不受真实设备测量噪声的影响，也不需要通过人工标注大量样本来获得训练数据。

另一方面，这种方法继承了用仿真替代实验时常见的挑战：首先，必须确保所选模型有足够能力预测我们感兴趣的被模拟现象的行为。此外，数值近似存在数值误差，对于所选应用必须使误差足够小；否则，即使最好的神经网络之后也无法给出有用答案。这些问题在经典仿真中已得到深入研究，因此也可以利用现有知识来设置深度学习训练任务。

4.2　展望

对现实世界现象的 PDE 模型进行数值近似通常计算代价很高。另一方面，训练后的神经网络每次评估的代价固定，而且通常很容易在 GPU 或神经网络计算单元等专用硬件上执行。

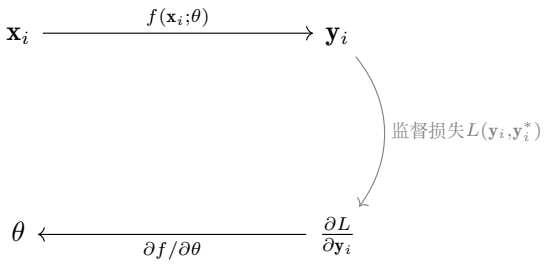


图 4.1: 监督训练的直观概览。与后面将遇到的更复杂变体相比，它简单且是良好的起点。

尽管如此，仍需谨慎：神经网络可能迅速产生大量中间结果。考虑具有 128 个特征的 CNN 层。如果将它应用于 128^2 的输入，即约 1.6 万个单元，就会得到 128^3 个中间值，超过 200 万个。这些值至少需要暂时保存在内存中，并由下一层处理。尽管如此，用快速的学习近似替代复杂昂贵的求解器仍是一个很有吸引力的方向。

在这一阶段，一个重要决定是选择哪种神经网络架构。

第五章　神经网络架构

神经网络中各个“神经元”的连接方式对网络能力有很大影响。典型网络由大量相互连接的“神经元”单元组成。经过多年发展，有几种关键架构已成为特别有用的选择；下面将概述选择架构时的主要考虑因素。我们的重点是介绍如何纳入基于 PDE 的模型（“物理”），而不是神经网络架构的细节。

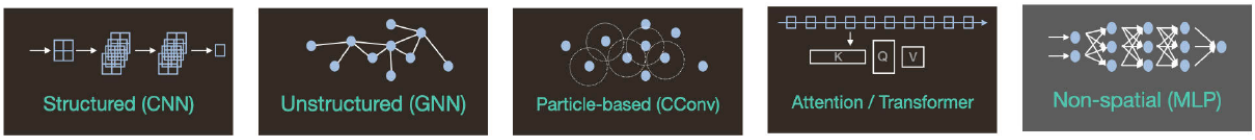


图 5.1: 我们将讨论一系列架构，从规则卷积、基于图和粒子的卷积，到较新的基于注意力的变体。

5.1　空间排列

在物理仿真背景下选择架构（以及排除不合适选项）时，首先要考虑的一个基本方面是数据样本的空间排列。我们可以区分四种主要情况：

1. 没有空间排列；
2. 网格上的规则间距（结构化）；
3. 不规则排列（非结构化）；
4. 没有连接关系的不规则位置（基于粒子或点）。

对于某些问题，不存在空间信息或排列（情况 1）。例如，随时间预测单个测量探头的温度和压力演化不涉及空间维度。与此相对的是将探头放在整齐排列的网格上（情况 2）或任意位置（情况 3）。最后一种（情况 4）是第三种的稍特殊情况，其中样本点之间没有已知的明确或持续的连接。例如，基于粒子的湍流表示可能就是这种情况，因为邻域会随时间快速变化。

5.2　无空间排列的输入

第一种情况比较特殊：没有任何空间排列信息时，只有稠密（“全连接”或 MLP）神经网络适用。

如果选择神经场方法，让网络以位置作为输入，效果也是如此：神经网络无法直接通过架构上的技巧（“归纳偏置”）查询邻居。此时下面的构件并不适用，值得考虑能否通过离散化引入更多结构。

注意，物理信息神经网络（PINN）也属于这一类。后面“可微物理简介”中将进一步讨论；作者一般建议考虑转向利用离散化形式先验知识的方法，认为这通常显著提高推断精度并改善收敛。原文还认为，多年研究之后 PINN 仍无法解决现实世界问题，表明这一方法存在根本问题。[1]

即便集中讨论稠密层，层数、层大小和激活函数仍有若干选择。其他三种情况也面临同样的选择，这些架构超参数通常通过训练运行确定。一般建议采用 ReLU 及其较平滑的变体如 GELU，并使层数和层大小一起扩展。下面关注其余三种有空间信息的情况，因为这些差别可能有深远影响。因此，我们考虑具有“计算区域”的情形，该区域指定样本所在的感兴趣区域。

5.3 局部与全局

不同架构最重要的方面可能是感受野：对于区域中的任一单个样本，其他样本点的哪个邻域能够影响该点的解。这类似于经典 PDE 求解中的考虑：称一个 PDE 为双曲型，表示其局部的波动行为，与具有全局行为的椭圆型方程形成对照。某些神经网络架构，如经典卷积神经网络（CNN），只支持局部影响和感受野；而带池化的层级结构能将感受野扩大为实际上全局的范围。FNO 等谱架构是一种有趣的变体，它们以其他方面为代价提供全局感受野。Transformer（带注意力机制）则提供另一种更复杂但可扩展的选择。

因此，一个基本区分是空间局部架构与全局架构；对于后者，还要区分它如何实现全局感受野。下表先给出概览，随后讨论各变体的优缺点。

	网格	非结构化	点	非空间
局部	CNN、ResNet	GNN	CConv	–
全局				MLP
层级	U-Net、膨胀卷积	多尺度 GNN	多尺度 CConv	–
谱	FNO	谱 GNN	(–)	–
序列	Transformer	图 Transformer	点 Transformer	–

注 应当利用关于数据依赖关系的知识，即依赖关系是局部还是全局。

如果数据主要受局部影响，选择支持全局感受野的架构很可能对精度产生负面影响：网络会“浪费”资源试图捕获全局效应，最坏情况下还会用平滑后的全局模态近似局部效应。反过来，试图用有限感受野近似全局影响将是无法解决的任务，并且很可能引入显著误差。

5.4 规则、非结构化与逐点数据

利用空间排列数据，最自然的起点是使用规则网格。注意它不一定是笛卡尔网格，也可以是变形的自适应网格 [CT22]。唯一要求是样本具有网格状连接，即使其间距不规则也可以。对于非结构化数据，基于图的神经网络（GNN）是不错的选择。虽然常以消息传递操作讨论它们，但它们与结构化网格有许多相似之处：GNN 中一步消息传递的基本操作等价于网格上的卷积 [SGGP+20]。同样，可以通过图粗化构造层级 [LPT25]。因此，虽然下文主要讨论网格，但要记住这些方法也适用于 GNN。由于处理图结构会使实现更复杂，我们到后面才详细讨论。最后，逐点的拉格朗日样本可视为没有连接关系的非结构化网格。不过，显式地按这种方式处理它们可能有助于提高学习和推断性能。尽管如此，卷积和层

[1] 这是原文对 PINN 方法的广泛评价；其适用范围及与现有工作的对应关系尚待核查，不作为译者独立结论。

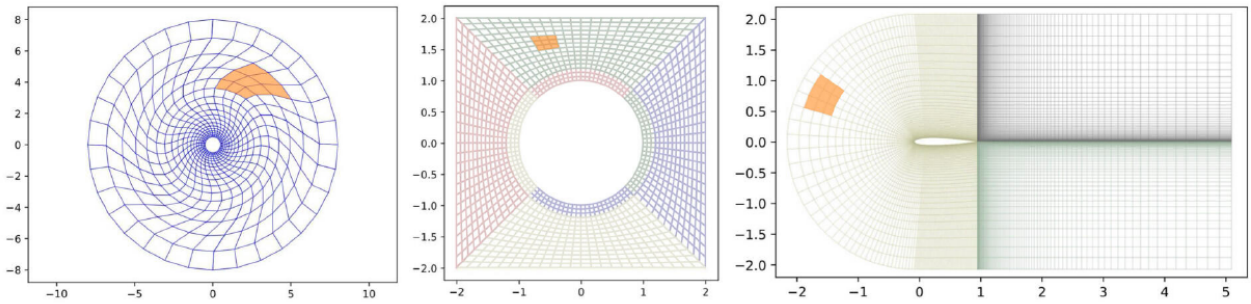


图 5.2: 不同变形的规则多块网格上的 3×3 卷积（橙色）。

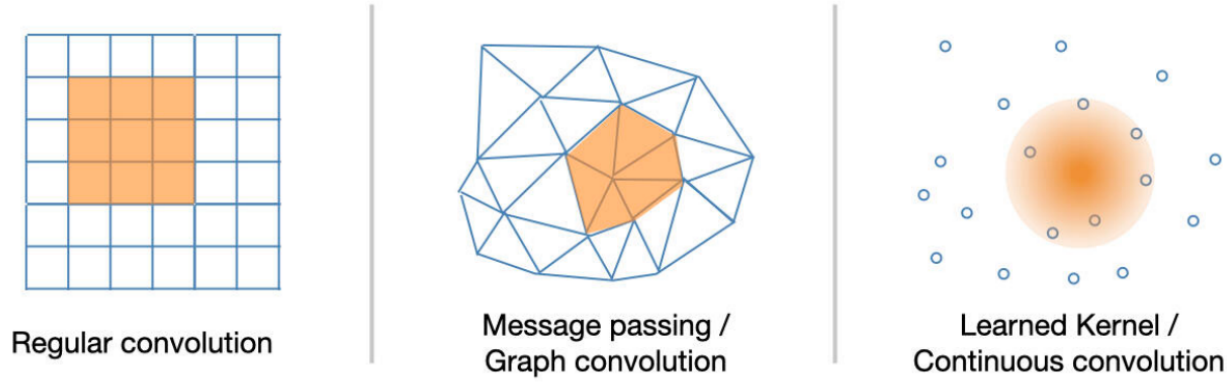


图 5.3: 无论数据结构如何，卷积（以及层级）都以非常相似的方式工作。如图所示，卷积适用于网格、图和点样本。同样，本书中针对网格算法讨论的概念也适用于图和点集。图中依次为规则卷积、消息传递 / 图卷积、学习核 / 连续卷积。

级这两个主要思想仍可用于拉格朗日数据：连续卷积核是合适的工具，而基于邻域的粗化可以产生层级 [PUKT22]。

5.5 层级结构

处理局部依赖关系的一个强大而自然的工具是规则网格上的卷积层。相应的神经网络（CNN）是深度学习的经典构件，已经得到充分研究和广泛支持。它们相对容易训练，通常在 API 中实现得非常高效。它们还与经典数值方法有自然联系：梯度和 Laplace 算子等微分算子的离散化常被视为“模板”，相当于具有一组特定权重的卷积层。例如，一维归一化 Laplace 算子 ∇^2 的经典模板为 $[1, -2, 1]$。它可以直接映射到核大小为 3、只有一个输入通道和一个输出通道的一维卷积，并将核的不可训练权重设为上述模板系数。

使用卷积层相当直接，但如何将全局依赖关系纳入 CNN 是一个有趣的问题。随着时间推移，这一领域形成了两种基本方法：通过池化构造层级网络（U-Net [RFB15]），以及采用间距增大的稀疏逐点样本（膨胀卷积 [YK15]）。两者都实现了建立全局感受野的目标，但内部有一些有趣差异。[2]

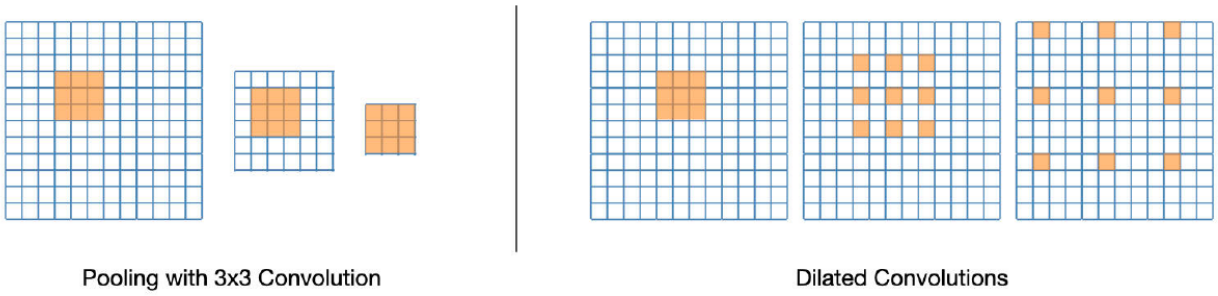


图 5.4: 基于池化的层级结构（左）和膨胀卷积（右）中的 3×3 卷积。两种情况下卷积都覆盖输入数据的较大范围。然而原文称层级结构处理的数据少 $O(\log N)$，而膨胀卷积以更大步幅处理全部输入，因此后者因样本点更多、数据访问不那么规则而代价更高。图中依次标注“带 3×3 卷积的池化”和“膨胀卷积”。

- U-Net 基于池化操作。类似几何多重网格层级，空间样本被下采样到越来越粗的网格，并在网络后半部分上采样。这意味着即使卷积核大小固定，由于之前的下采样，卷积也能在物理空间中“看”得更远。原文称样本点数量对数递减，使层级较低处的卷积非常高效。不同重采样方法（mean、average、逐点等）影响较小，而 U-Net 的关键构件是跳跃连接：它通过特征拼接，将前半部分的早期层直接连接至后半部分。这对避免信息丢失至关重要。最深的“瓶颈”层具有最粗的表示，通常难以存储最细表示的所有细节。通过跳跃连接显式提供这些信息对提高精度至关重要。

- 膨胀卷积将卷积的采样点放得更远。因此，卷积不是例如查看 3×3 邻域，而是考虑 5×5 邻域，计算时却只使用其中 3×3 个样本。所用点之间的其他样本通常被直接忽略。与层级结构不同，样本点数量保持不变。

[2] 原文关于层级结构处理数据量“少 $O(\log N)$”及下文样本数“对数递减”的说法依赖 N 和比较对象的定义；若 d 维每层固定 2 倍池化，样本数 $N_\ell = N_0/2^{d\ell}$ 随层数几何递减；达到固定最粗规模的层数才为 $O(\log N_0)$。不能由此把源措辞直接当成通用总计算复杂度。

两种方法都能实现目标并取得很好的性能，但存在有趣的权衡：U-Net 的实现稍费功夫，却可能快得多。性能提升的原因是膨胀卷积的内存访问并非最优：它以大步幅跳跃访问内存，导致性能降低。另一方面，U-Net 基本上以粗网格形式预先计算了压缩的内存表示。在此粗网格上的卷积计算非常高效。不过，这需要添加合适的池化层，因而实现工作量稍大；膨胀卷积的实现可能只需将普通卷积调用替换为膨胀卷积。需要在应用中部署训练后的网络时，实现 U-Net 的投入长期来看可能非常值得。

如上所述，层级结构对图网络同样重要。但对图网络而言，“是否膨胀”的问题并不存在：这里内存访问总是不规则，在一般图上计算步幅代价高，因此膨胀并不流行。故如果数据中存在全局依赖，强烈建议使用多尺度 GNN 形式的规则层级。

5.6 谱方法

建立全局感受野还有一条根本不同的途径：谱方法通常利用 Fourier 变换将空间数据转移到频域。这类方法中最流行的是 Fourier 神经算子（FNO）[LKA+21]。一个有趣之处是函数表示所承诺的连续表示能力，但对此应保持谨慎：函数空间通常会被截断，因此频率表示能否在超出训练数据分辨率的情况下真正得到合适的解，往往值得怀疑。

不过，下面重点关注感受野及其性能。FNO 类方法有一种有趣的行为：它们用稠密层修改频率信息。由于 Fourier 变换后的频率信号与输入大小相同，稠密层在 M 个最大频率上工作。对于二维输入，这意味着 M^2 个模态，相应稠密层因而需要 M^4 个参数。[3]

频域的一个内在优势及其结果是所有基函数都具有全局支撑。这意味着即使只处理所有频率中的一个子集，FNO 也能处理并修改输入信号的所有部分。这种对全局依赖关系的自然处理是谱方法的内在优势。遗憾的是，原文认为它们不太适合更高维问题：从二维到三维会使待处理频率的规模增加到 M^3；

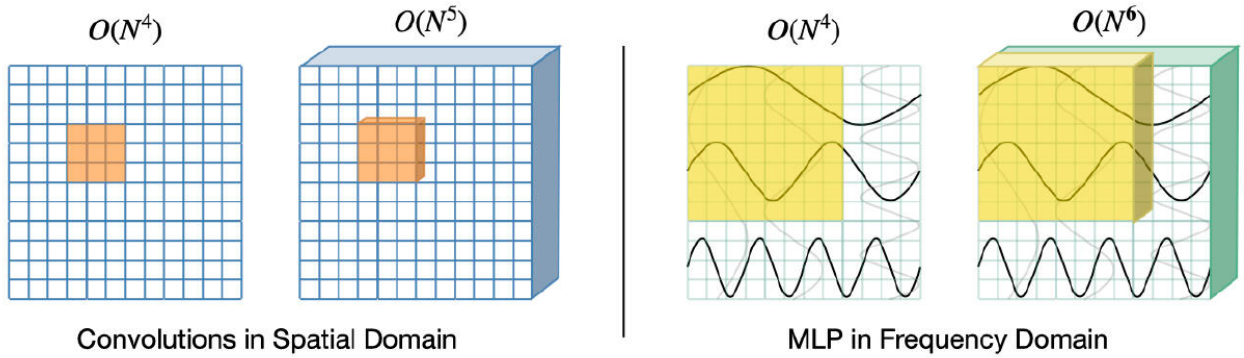
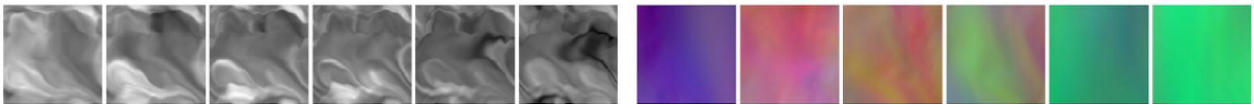


图 5.5: 空间卷积（左，卷积核为橙色）与 FNO 中的频率处理（右，稠密层覆盖范围为黄色）。原文指出，FNO 在三维中的标度较差（六次方而非五次方），其标度常数还与区域大小成正比，因此通常更大。图中字样分别为“空间域卷积”和“频域 MLP”。

对于稠密层，这意味着 M^6 个参数，是三次增长。对于卷积，二维中差异不大：核大小为 K 的规则卷积在二维需要 K^2 个权重，处理特征又引入 $O(K^2)$ 标度，总计 $O(K^4)$。但在三维中，规则卷积的标度好得多：只有核大小增加到 K^3，使三维总体复杂度为 $O(K^5)$，因此指数是 5 而不是 6。

[3]此处是作者对特定实现的复杂度描述；M 的含义、参数共享方式与通道数均会影响实际 FNO 参数量，需对照具体实现核查。

更糟的是，原文认为 FNO 的频率覆盖 M 需要随空间区域大小扩展，因此通常 $M > K$ 且 $M^6 \gg K^5$。由此作者认为 FNO 会需要难以承受的参数量，不推荐用于三维或更高维问题。CNN 等架构所需权重少得多，并且结合层级结构仍能高效处理全局依赖。[4]



5.7 注意力与 Transformer

深度学习领域一个较新且令人兴奋的发展是注意力机制。它们以 Transformer 的形式在处理语言和自然图像方面取得巨大成功，也有望用于物理相关问题。然而，它们是否确实普遍优于更“经典”的架构，仍是开放问题。下面概述主要优缺点。

Transformer 通常分两步工作：首先使用编码器—解码器网络将输入编码为 token。这一步可以有多种形式，通常主要用于减少输入数量，例如处理图像块而不是单个像素。然后注意力机制为一组输入 token 计算权重。这是每个 token 对应的浮点数，传统上解释为指出输入的哪些部分重要、哪些不重要。在现代架构中，注意力浮点权重直接用于修改输入。在自注意力中，从每个输入到所有其他输入 token 计算权重。这是一种处理全局依赖关系的机制，因此直接契合前面的讨论。

实践中通过三个矩阵计算注意力：查询矩阵 Q、键矩阵 K 和值矩阵 V。对于 N 个 token，外积 QK^T 产生 $N \times N$ 矩阵，经过 Softmax 层后与 V（包含输入 token 的线性投影）相乘，产生注意力输出向量。

在 Transformer 架构中，注意力输出是一个构件的一部分：计算注意力并将其作为残差加到输入上，使用层归一化稳定，然后在两层前馈网络（FFN）中处理。后者只是两个稠密层，中间有一个激活函数。如下图所示的 Transformer 块会重复应用多次，最后将输出解码回原始空间。这一 Transformer

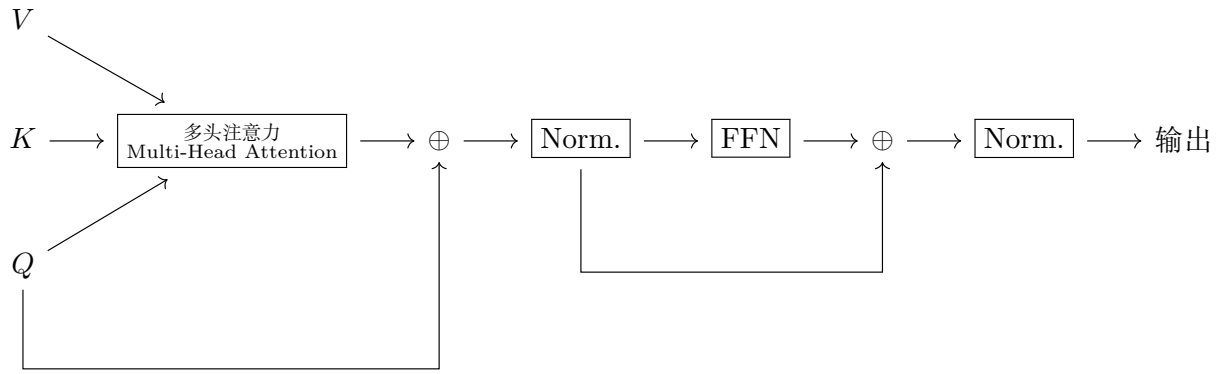


图 5.6: 单个 Transformer 块的可视化概览。完整网络多次重复这一结构以推断结果。图中文字依次为多头注意力、归一化、前馈网络及归一化。

架构已被证明可以非常好地扩展至参数量巨大的网络，这是其关键优势之一。注意，权重的很大一部分通常位于注意力矩阵中，而不只是稠密层中。同时，注意力为处理输入中的全局依赖关系提供了强有力

[4] 上述标度与推荐是源书的模型假设和判断，不能不加条件地推广到所有 FNO 实现；尤其参数量与计算量需要按频率、通道和网络结构分别核对。

的方法，代价是架构更复杂。上述自注意力机制的固有问题是它对 token 数量 N 呈二次复杂度，自然限制了输入的大小与分辨率。

其内部机制也出奇地简单：注意力算法计算一个 $N \times N$ 矩阵，这与应用简单稠密层来处理全局影响并不相距太远（后者同样会有 $N \times N$ 权重矩阵）。

这个瓶颈可以通过线性注意力解决：它改变上述算法，改为将 Q 与 $(K^T V)$ 相乘，并事先对两部分应用非线性函数（例如指数函数）。这样避免了 $N \times N$ 矩阵，对 N 的标度变为线性；不过，改进的代价是注意力向量更加近似。

Transformer 架构另一个有趣方面是既能应用于结构化输入，也能应用于非结构化输入：它们已用于图、点和网格数据。在所有情况下，差别主要在于如何将输入映射到 token；注意力通常在 token 空间中仍是“稠密”的。这是一个明显限制：对于具有已知空间结构的问题，丢弃这一信息不可避免地需要补偿，例如使用更多权重或接受较低推断精度。尽管如此，Transformer 在深度学习中仍是极其活跃的领域，显然是未来神经网络算法的潜在选择。

5.8 架构总结

上文概述了为物理相关问题选择神经网络架构时的若干基本考虑。概括而言，主要考虑因素是关于数据中局部或全局依赖关系的知识，针对这种差异定制架构可能产生重大影响。尽管数据的空间结构似乎决定了选择，但也值得考虑将数据转移到另一种数据结构。例如，将非结构化点投影到（变形的）规则网格上，有可能改善精度和性能。

还应提到，上述典型架构存在混合形式：例如，在具有跳跃连接的经典 U-Net 中加入 Transformer 架构的组件（如注意力和归一化）以提高性能。在“基于扩散的时间预测”中可以找到这种“现代化”U-Net 的实现。

5.9 给我看看代码！

最后来看一个训练神经网络的代码示例：我们将用 [TWPH20] 中的代理模型替代翼型周围湍流的完整求解器，使用具有全局感受野的 U-Net 作为算子。

第六章　翼型周围 RANS 流动的监督训练

6.1　概述

这个监督训练示例针对翼型周围的湍流空气流动：学习得到的算子应针对不同 Reynolds 数和攻角，给出指定翼型几何周围的平均运动及压力分布。因此，神经网络的输入是翼型形状、Reynolds 数和攻角，输出应为翼型周围具有两个分量的时间平均速度场以及压力场。

传统上使用 Reynolds 平均 Navier–Stokes（RANS）模型近似这一问题，这仍然是 Navier–Stokes 求解器在工业中应用最广泛的情形之一。不过，我们现在不依赖求解 RANS 方程的传统数值方法，而是要通过神经网络训练一个完全绕过数值求解器的代理模型。原文提供的 Colab 链接待核实。

6.2　问题表述

使用“监督训练”中的监督表述，学习任务相当直接，可写为

$$\arg\min_{\theta} \sum_{i} \left(f(x_i;\theta) - y_i^* \right)^2 ,$$

其中 x 和 y^* 各由一组物理场组成，下标 i 表示对数据集中所有离散点计算差异。

目标是在以翼型为中心的计算区域 Ω 中推断速度 $\mathbf{u} = (u_x, u_y)$ 和压力场 p。u_x, u_y, p 各自的维度均为 128^2。输入是 Reynolds 数 $\mathrm{Re} \in \mathbb{R}$、攻角 $\alpha \in \mathbb{R}$，以及编码为 128^2 栅格的翼型形状 s。Re 和 α 通过自由来流速度 $\mathbf{f}$ 给出，其 x 和 y 分量表示为同样大小的常值场，翼型区域内为零。因此组合起来，输入和输出的维度相同：$x, y^* \in \mathbb{R}^{3\times128\times128}$。输入包含 $[f_x, f_y, \mathrm{mask}]$，输出保存 $[p, u_x, u_y]$ 三个通道。这正是下面为神经网络指定的输入和输出维度。

这里需要记住，y^* 中感兴趣的量包含三个不同物理场。两个速度分量在性质上相当相似，而压力场的行为通常不同，相对于速度近似呈平方标度（参见 Bernoulli 关系）。这意味着对上面最小化问题中的简单求和应当谨慎，并注意数据归一化。否则某一分量可能占主导，通过均值聚合会使神经网络花更多资源学习数值较大的分量，而不是造成较小误差的其他分量。

6.3　代码来了……

开始实现。注意这里跳过数据生成过程。下面的代码改编自 [TWPH20] 及其代码库，细节可以在那里查看。我们将使用由 OpenFOAM 中的 Spalart–Allmaras RANS 仿真生成的一小组训练数据。首先导入所需模块，并从 Git 安装数据加载器。

```
import os, sys, random
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim

!pip install --upgrade --quiet git+https://github.com/tum-pbs/pbdl-dataset
from pbdl.torch.loader import Dataloader
```

下一个代码单元将从 Hugging Face 下载训练数据，可能需要一些时间。下面的 PBDL 数据加载器调用直接将其分成 320 个训练样本和 80 个验证样本。验证样本与训练样本使用相同的翼型形状，但条件不同（后面将下载新形状用于测试）。

```
BATCH_SIZE = 10

loader_train, loader_val = Dataloader.new_split(
    [320, 80],
    "airfoils",
    batch_size=BATCH_SIZE, normalize_data=None,
)
```

```
Warning: `airfoils` is stored in single-file format. The download might take some time.
Success: Loaded airfoils with 400 simulations and 1 samples each.
```

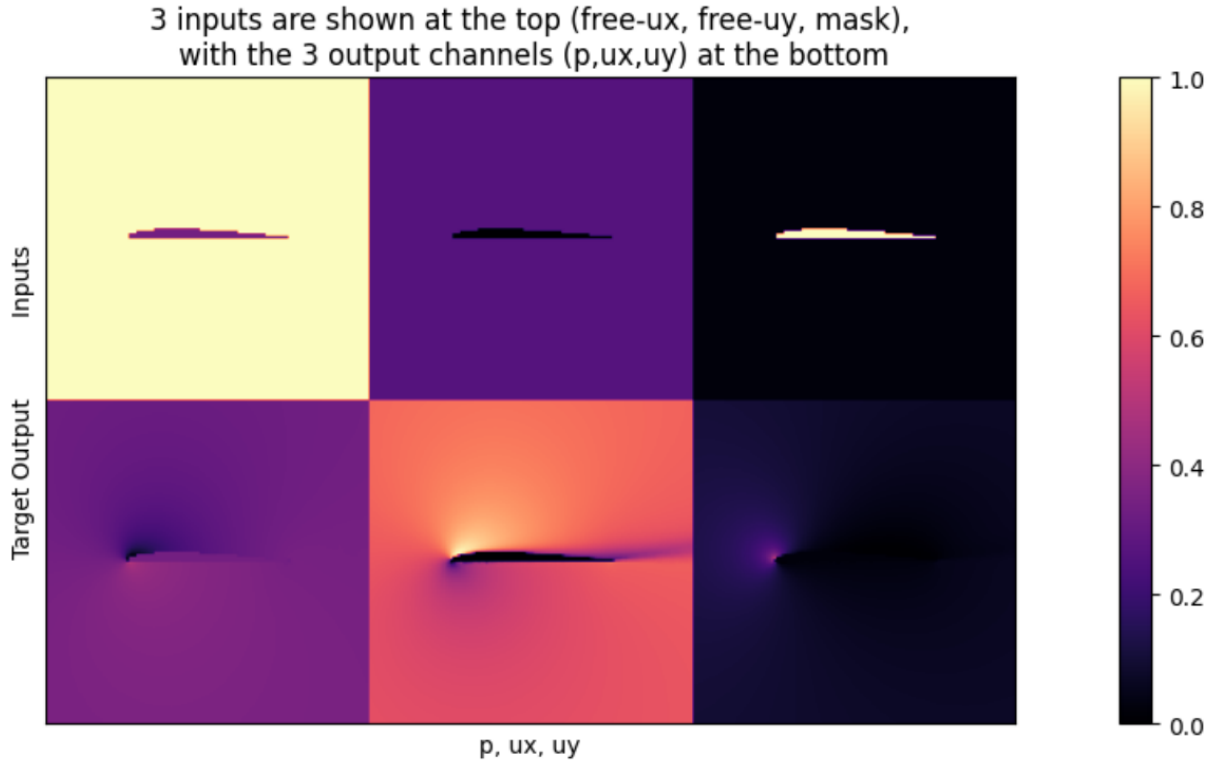
6.4 RANS 训练数据

现在已经有了训练和验证数据。一般而言，尽可能理解所处理的数据非常重要（对任何机器学习任务而言，“垃圾进，垃圾出”原则肯定成立）。至少应该了解数据的维度和大致统计量，理想情况下还应了解其内容，否则将很难解释训练结果。尽管 AI 看起来神奇，如果你无法从数据中看出任何模式，神经网络很可能也找不到有用模式。

因此，来看看一个训练样本。下面只是用于并排显示图像的辅助代码。

```
def plot(a1, a2, mask=None, stats=False, bottom="NN Output", top="Reference",
         title=None):
    c = []
    if mask is not None: mask = np.asarray(mask)
    for i in range(3):
        a2i = np.asarray(a2[i])
        if mask is not None: a2i = a2i - mask*a2i # optionally mask out inner region
        b = np.flipud(np.concatenate((a2i, a1[i]), axis=1).transpose())
        min, mean, max = np.min(b), np.mean(b), np.max(b)
        if stats:
            print("Stats %d: " % i + format([min, mean, max]))
        b -= min
```

```
        b /= max - min
        c.append(b)
    fig, axes = plt.subplots(1, 1, figsize=(16, 5))
    axes.set_xticks([]) ; axes.set_yticks([])
    im = axes.imshow(np.concatenate(c, axis=1), origin="upper", cmap="magma")
    fig.colorbar(im, ax=axes)
    axes.set_xlabel("p, ux, uy")
    axes.set_ylabel("%s           %s" % (bottom, top))
    if title is not None: plt.title(title)
    plt.show()

inputs, targets = next(iter(loader_train))
plot(inputs[0], targets[0], stats=False, bottom="Target Output", top="Inputs",
     title="3 inputs are shown at the top (free-ux, free-uy, mask),\n"
           " with the 3 output channels (p,ux,uy) at the bottom")
```



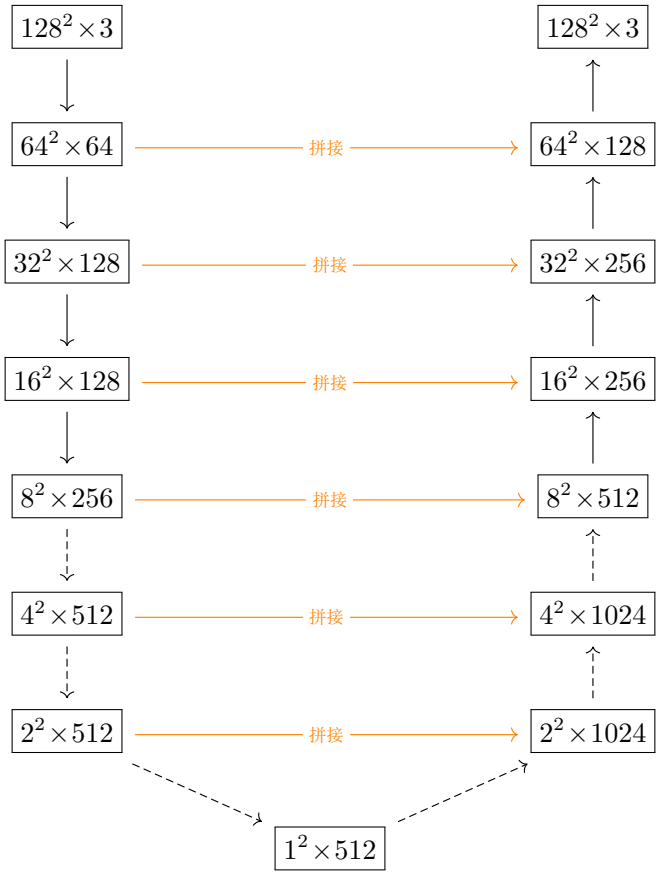
图中上排为三个输入（自由流 u_x、自由流 u_y、掩膜），下排为三个目标输出通道（p, u_x, u_y）；色条表示图中归一化后的数值。

6.5 网络设置

现在可以设置神经网络架构。我们将使用全卷积 U-Net。这是一种广泛使用的架构，在不同空间分辨率上堆叠卷积。与普通卷积网络的主要区别在于层级结构（用于获得全局感受野），以及从编码器到

解码器部分引入跳跃连接。这确保了特征提取过程中不会丢失信息。（注意这仅在网络作为整体使用时有效；例如，希望将解码器作为独立组件使用时就不适用。）下面是架构概览。

编码器–解码器与同尺度特征拼接

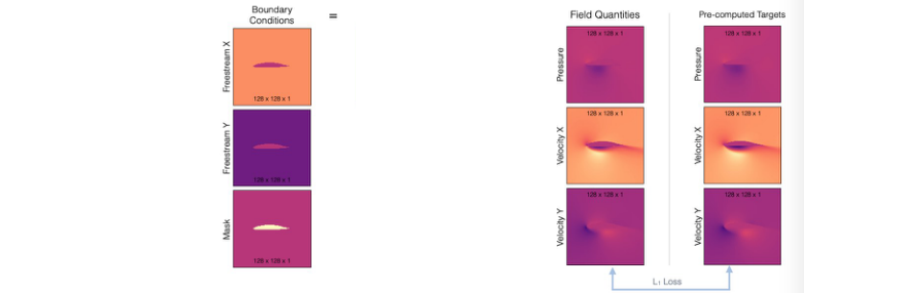


实线向下：4×4 卷积；虚线向下：2×2 卷积。

虚线向上：2×2 resize-convolution；实线向上：4×4 resize-convolution。

橙色横线：逐特征拼接（feature-wise concatenation）。

边界条件＝输入（$128 \times 128 \times 3$）　　输出＝场量与预计算目标



输出：压力、速度 X、速度 Y
$128 \times 128 \times 1$ 各一场

预计算目标：相同三个场
$128 \times 128 \times 1$ 各一场

L1 损失

首先定义一个辅助函数 blockUNet，用于设置网络中的卷积块。注意我们没有使用任何池化！相反，按照最佳实践使用步幅和转置卷积（解码器部分需要对称，即具有奇数核大小）。完整的 PyTorch 神经网络由 DfpNet 类管理。[1]

```
def blockUNet(in_c, out_c, name, size=4, pad=1, transposed=False, bn=True,
              activation=True, relu=True, dropout=0.0):
    block = nn.Sequential()
    if not transposed:
        block.add_module(
            "%s_conv" % name,
            nn.Conv2d(in_c, out_c, kernel_size=size, stride=2, padding=pad, bias=True),
        )
    else:
        block.add_module(
            "%s_upsam" % name, nn.Upsample(scale_factor=2, mode="bilinear")
        )
        # reduce kernel size by one for the upsampling (ie decoder part)
        block.add_module(
            "%s_tconv" % name,
            nn.Conv2d(in_c, out_c, kernel_size=(size - 1), stride=1,
                      padding=pad, bias=True),
        )
    if bn:
        block.add_module("%s_bn" % name, nn.BatchNorm2d(out_c))
    if dropout > 0.0:
        block.add_module("%s_dropout" % name, nn.Dropout2d(dropout, inplace=True))
    if activation:
        if relu:
            block.add_module("%s_relu" % name, nn.ReLU(inplace=True))
        else:
            block.add_module("%s_leakyrelu" % name, nn.LeakyReLU(0.2, inplace=True))
    return block

class DfpNet(nn.Module):
    def __init__(self, channelExponent=6, dropout=0.0):
        super(DfpNet, self).__init__()
        channels = int(2**channelExponent + 0.5)
        self.layer1 = blockUNet(3, channels * 1, "enc_layer1",
            transposed=False, bn=True, relu=False, dropout=dropout, )
        self.layer2 = blockUNet(channels, channels * 2, "enc_layer2",
            transposed=False, bn=True, relu=False, dropout=dropout, )
        self.layer3 = blockUNet(channels * 2, channels * 2, "enc_layer3",
            transposed=False, bn=True, relu=False, dropout=dropout, )
        self.layer4 = blockUNet(channels * 2, channels * 4, "enc_layer4",
            transposed=False, bn=True, relu=False, dropout=dropout, )
```

[1] 原文称使用转置卷积；下面代码的解码分支实际使用双线性上采样再接普通卷积，并未调用转置卷积。

```
        self.layer5 = blockUNet(channels * 4, channels * 8, "enc_layer5",
            transposed=False, bn=True, relu=False, dropout=dropout, )
        self.layer6 = blockUNet(channels * 8, channels * 8, "enc_layer6",
            transposed=False, bn=True, relu=False, dropout=dropout, size=2, pad=0, )
        self.layer7 = blockUNet(channels * 8, channels * 8, "enc_layer7",
            transposed=False, bn=True, relu=False, dropout=dropout, size=2, pad=0, )

        # note, kernel size is internally reduced by one for the decoder part
        self.dlayer7 = blockUNet(channels * 8, channels * 8, "dec_layer7",
            transposed=True, bn=True, relu=True, dropout=dropout, size=2, pad=0, )
        self.dlayer6 = blockUNet(channels * 16, channels * 8, "dec_layer6",
            transposed=True, bn=True, relu=True, dropout=dropout, size=2, pad=0, )
        self.dlayer5 = blockUNet(channels * 16, channels * 4, "dec_layer5",
            transposed=True, bn=True, relu=True, dropout=dropout, )
        self.dlayer4 = blockUNet(channels * 8, channels * 2, "dec_layer4",
            transposed=True, bn=True, relu=True, dropout=dropout, )
        self.dlayer3 = blockUNet(channels * 4, channels * 2, "dec_layer3",
            transposed=True, bn=True, relu=True, dropout=dropout, )
        self.dlayer2 = blockUNet(channels * 4, channels, "dec_layer2",
            transposed=True, bn=True, relu=True, dropout=dropout, )
        self.dlayer1 = blockUNet(channels * 2, 3, "dec_layer1",
            transposed=True, bn=False, activation=False, dropout=dropout, )

    def forward(self, input):
        # note, this Unet stack could be allocated with a loop, of course...
        out1 = self.layer1(input)
        out2 = self.layer2(out1)
        out3 = self.layer3(out2)
        out4 = self.layer4(out3)
        out5 = self.layer5(out4)
        out6 = self.layer6(out5)
        out7 = self.layer7(out6)
        # ... bottleneck ...
        dout6 = self.dlayer7(out7)
        dout6_out6 = torch.cat([dout6, out6], 1)
        dout6 = self.dlayer6(dout6_out6)
        dout6_out5 = torch.cat([dout6, out5], 1)
        dout5 = self.dlayer5(dout6_out5)
        dout5_out4 = torch.cat([dout5, out4], 1)
        dout4 = self.dlayer4(dout5_out4)
        dout4_out3 = torch.cat([dout4, out3], 1)
        dout3 = self.dlayer3(dout4_out3)
        dout3_out2 = torch.cat([dout3, out2], 1)
        dout2 = self.dlayer2(dout3_out2)
        dout2_out1 = torch.cat([dout2, out1], 1)
        dout1 = self.dlayer1(dout2_out1)
```

```
        return dout1

def weights_init(m):
    classname = m.__class__.__name__
    if classname.find("Conv") != -1:
        m.weight.data.normal_(0.0, 0.02)
    elif classname.find("BatchNorm") != -1:
        m.weight.data.normal_(1.0, 0.02)
        m.bias.data.fill_(0)
```

接下来初始化一个 DfpNet 实例。下面的 EXPO 参数控制 U-Net 特征图的指数，直接决定网络大小（指数为 4 时网络约有 58.5 万个参数）。对于具有 $3 \times 128^2 \approx 4.9$ 万个输出的生成式神经网络，这是中等大小，训练仍然较快，因此是一个良好起点。weights_init 函数将卷积网络权重初始化到合理范围，使我们可以直接使用固定学习率训练；否则强烈建议使用学习率调度。

```
# channel exponent to control network size
EXPO = 4

torch.set_default_device("cuda:0")
device = torch.get_default_device()
net = DfpNet(channelExponent=EXPO)
net.apply(weights_init)

# crucial parameter to keep in view: how many parameters do we have?
nn_parameters = filter(lambda p: p.requires_grad, net.parameters())
print("Trainable params: {}   -> crucial! always keep in view... ".format(
    sum([np.prod(p.size()) for p in nn_parameters]) ))

LR = 0.0002                  # learning rate
loss = nn.L1Loss()
optimizer = optim.Adam(net.parameters(), lr=LR, betas=(0.5, 0.999), weight_decay=0.0)

Trainable params: 585027              -> crucial! always keep in view...
```

正如打印语句中的微妙提示所示，参数数量是训练神经网络时必须关注的关键数字。设置很容易改变，使网络拥有数百万参数，结果可能在训练时浪费资源，甚至造成训练不稳定。参数数量必须与训练数据量相匹配，也应随网络深度调整；不过这三者的具体关系取决于问题。

6.6 训练

最后可以训练神经网络。原文称训练将遍历全部 320 个样本 100 次，同时不断评估验证样本以跟踪当前网络状态；这可能需要一些时间。[2]

```
EPOCHS = 200           # number of training epochs
loss_hist = []
loss_hist_val = []
```

[2]原文此处说 100 次，但下面代码设定 `EPOCHS = 200`，输出也显示 200 次。

```
if os.path.isfile("dfpnet"): # NT_DEBUG
    print("Found existing network, loading & skipping training")
    net.load_state_dict(torch.load("dfpnet"))
else:
    print("Training from scratch...")
    pbar = tqdm(initial=0, total=EPOCHS, ncols=96)
    for epoch in range(EPOCHS):
        # training
        net.train()
        loss_acc = 0
        for i, (inputs, targets) in enumerate(loader_train):
            inputs = inputs.float()
            targets = targets.float()
            net.zero_grad()
            outputs = net(inputs)
            lossL1 = loss(outputs, targets)
            lossL1.backward()
            optimizer.step()
            loss_acc += lossL1.item()
        loss_hist.append(loss_acc / len(loader_train))

        # evaluate validation samples
        net.eval()
        loss_acc_v = 0
        with torch.no_grad():
            for i, (inputs, targets) in enumerate(loader_val):
                inputs = inputs.float()
                targets = targets.float()
                outputs = net(inputs)
                loss_acc_v += loss(outputs, targets).item()
        loss_hist_val.append(loss_acc_v / len(loader_val))
        pbar.set_description("loss train: {:7.5f}, loss val: {:7.5f}".format(
            loss_hist[-1], loss_hist_val[-1]), refresh=False); pbar.update(1)
    torch.save(net.state_dict(), "dfpnet")
    print("training done, saved network weights")

loss_hist = np.asarray(loss_hist)
loss_hist_val = np.asarray(loss_hist_val)
```

```
Training from scratch...
loss train: 0.01113, loss val: 0.01996: 100%|...| 200/200 [05:40<00:00, 1.61s/it]
training done, saved network weights
```

上述进度条以省略号简写；原 PDF 显示的完整输出如下。

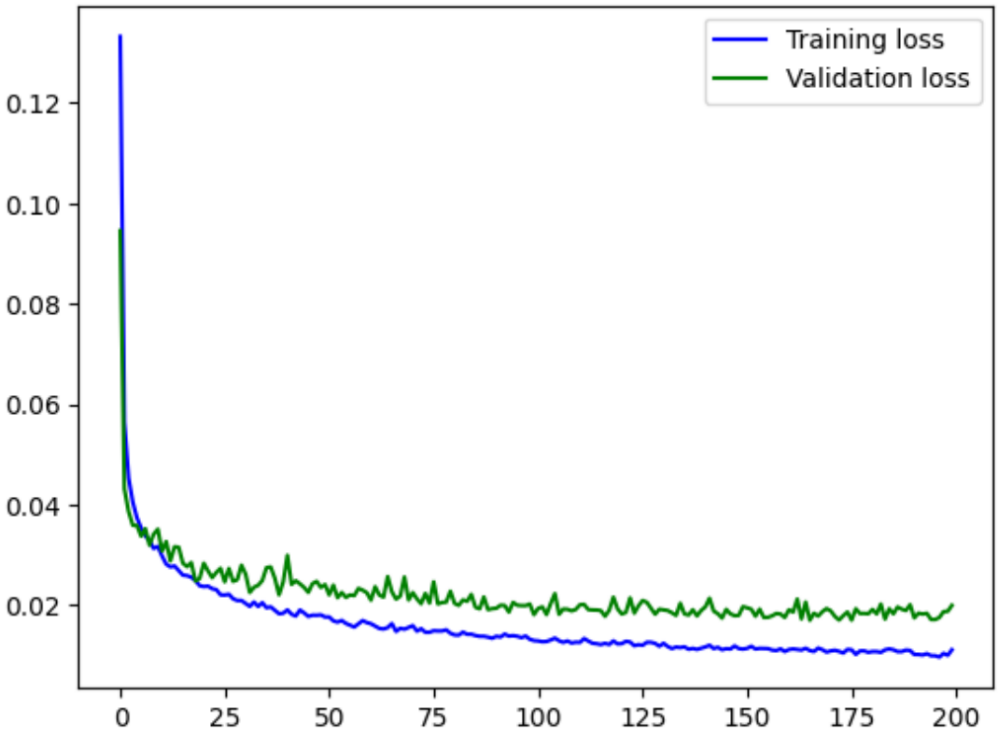
```
Training from scratch...
```

```
loss train: 0.01113, loss val: 0.01996: 100%|████████████████| 200/200 [05:40
↪<00:00,  1.61s/it]
```

```
training done, saved network weights
```

神经网络训练完成后，损失的绝对值应当下降：在标准设置下，验证损失从初始值约 0.2 下降到训练后的约 0.02。来看看曲线，直观了解训练如何随时间进行。这通常对于识别训练中的长期趋势很重要。实践中，很难从命令行日志中大约 100 个带噪数字判断总体趋势是略微上升还是下降；可视化使这一点容易得多。

```
plt.plot(np.arange(loss_hist.shape[0]), loss_hist, "b", label="Training loss")
plt.plot(np.arange(loss_hist_val.shape[0]), loss_hist_val, "g", label="Validation loss")
plt.legend()
plt.show()
```

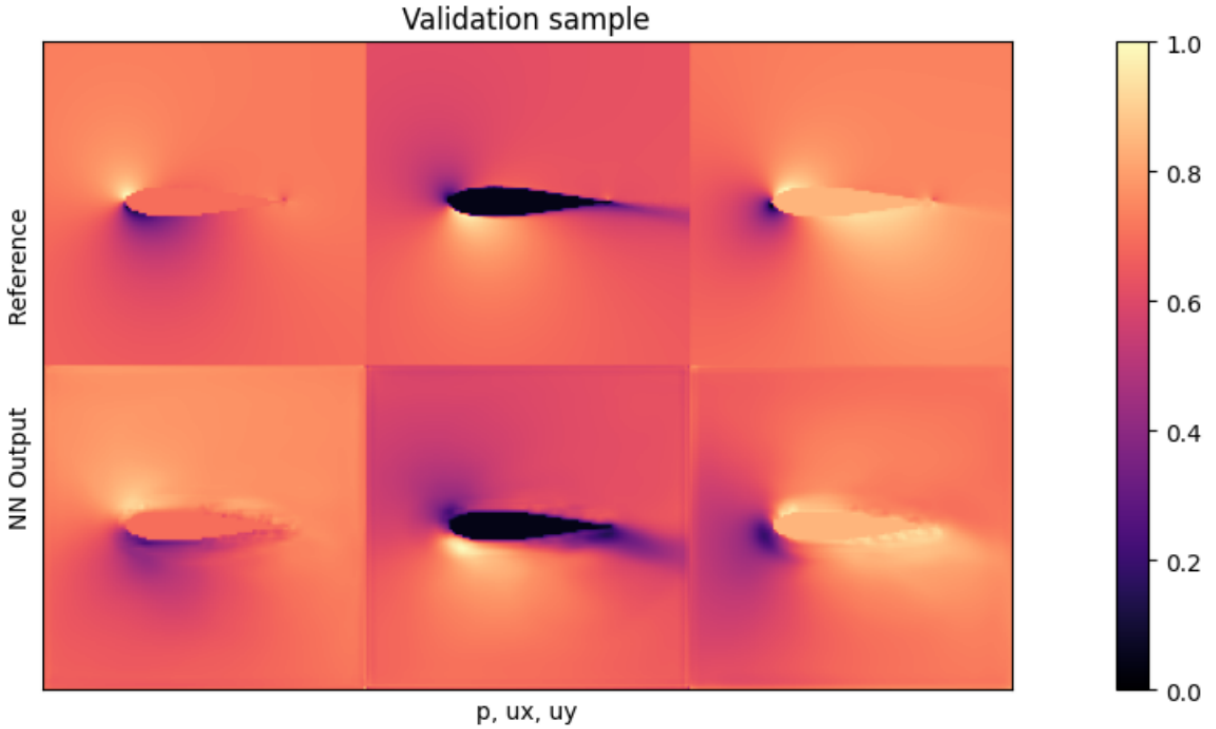


图中蓝线为训练损失，绿线为验证损失。可以看到曲线下降约 40 个 epoch，然后开始趋平。最后一段仍在缓慢下降，而最重要的是验证损失没有上升；验证损失上升会是过拟合的明确信号，是我们应当避免的。（尝试人为减少训练数据量，就应当能够有意造成过拟合。）

注意这里验证损失通常较高，因为数据集相对较小。到某个阶段，网络将无法再从中获得能够迁移到验证样本的新信息。一般而言需要注意：不要用训练数据评估网络，因为过拟合非常常见，这样做所能提供的信息很少。至少使用网络以前没有见过的数据，即验证数据；若结果良好，再尝试差异更大、至少稍微处于分布外的输入，即测试数据。

下一个代码单元在一批验证样本上运行训练后的网络，并使用 plot 函数显示其中一个样本。

```
net.eval()
inputs, targets = next(iter(loader_val))
inputs = inputs.float()
targets = targets.float()
outputs = net(inputs)
outputs = outputs.data.cpu().numpy()
inputs = inputs.cpu()
targets = targets.cpu()
plot(targets[0], outputs[0], mask=inputs[0][2], title="Validation sample")
```



图中上排为参考值，下排为网络输出。这里两者相似度较好。翼型周围区域通常仍有一些噪声（原文认为这由 Dirichlet 边界导致，可以通过修改损失函数和使用更大网络缓解）。压力值通常最难学习；更详细的评估留给测试数据。

6.7　测试评估

现在看看实际测试样本：这里使用新的翼型形状作为分布外（OOD）数据。网络在任何训练样本中都没有见过这些形状，因此它们能在一定程度上说明神经网络对未见输入的泛化能力；仅靠验证数据不足以对泛化作出结论。

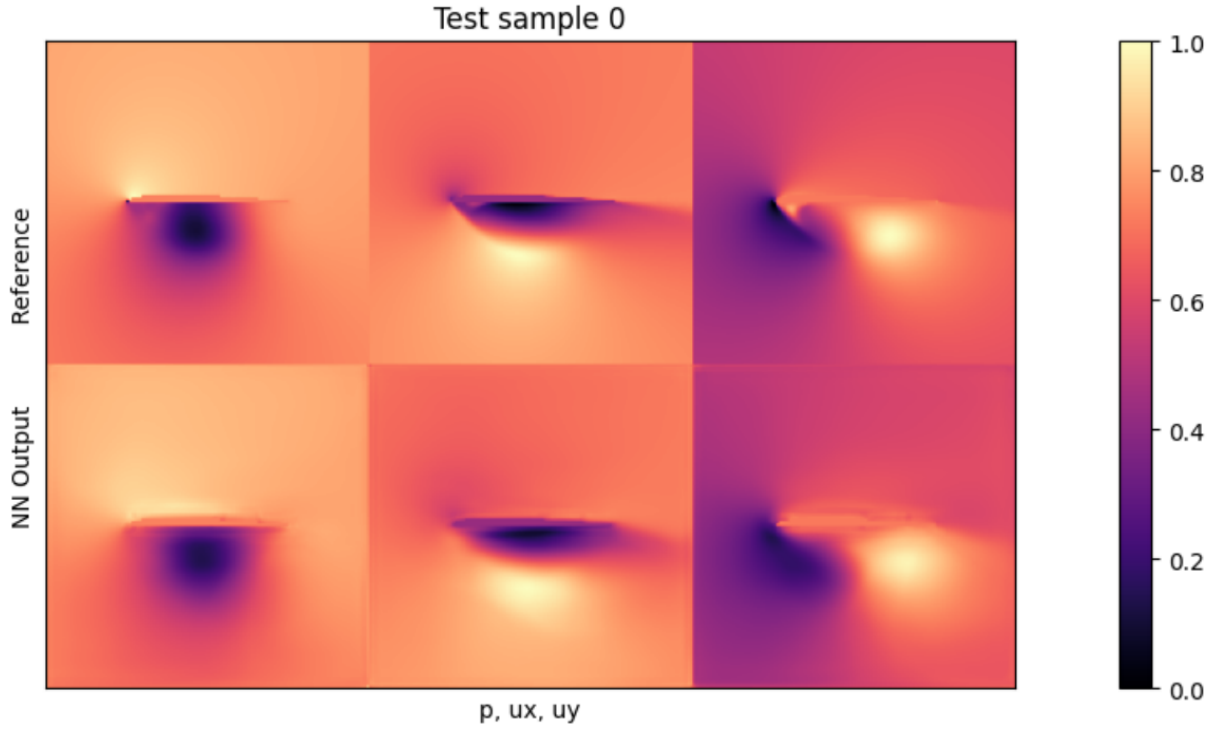
我们沿用之前的可视化。原文称，正如 Bernoulli 方程所提示的，第一列的压力尤其具有挑战性；由于其相对于输入自由流速度呈三次标度，并有局部峰值，它是网络最难推断的量。[3]

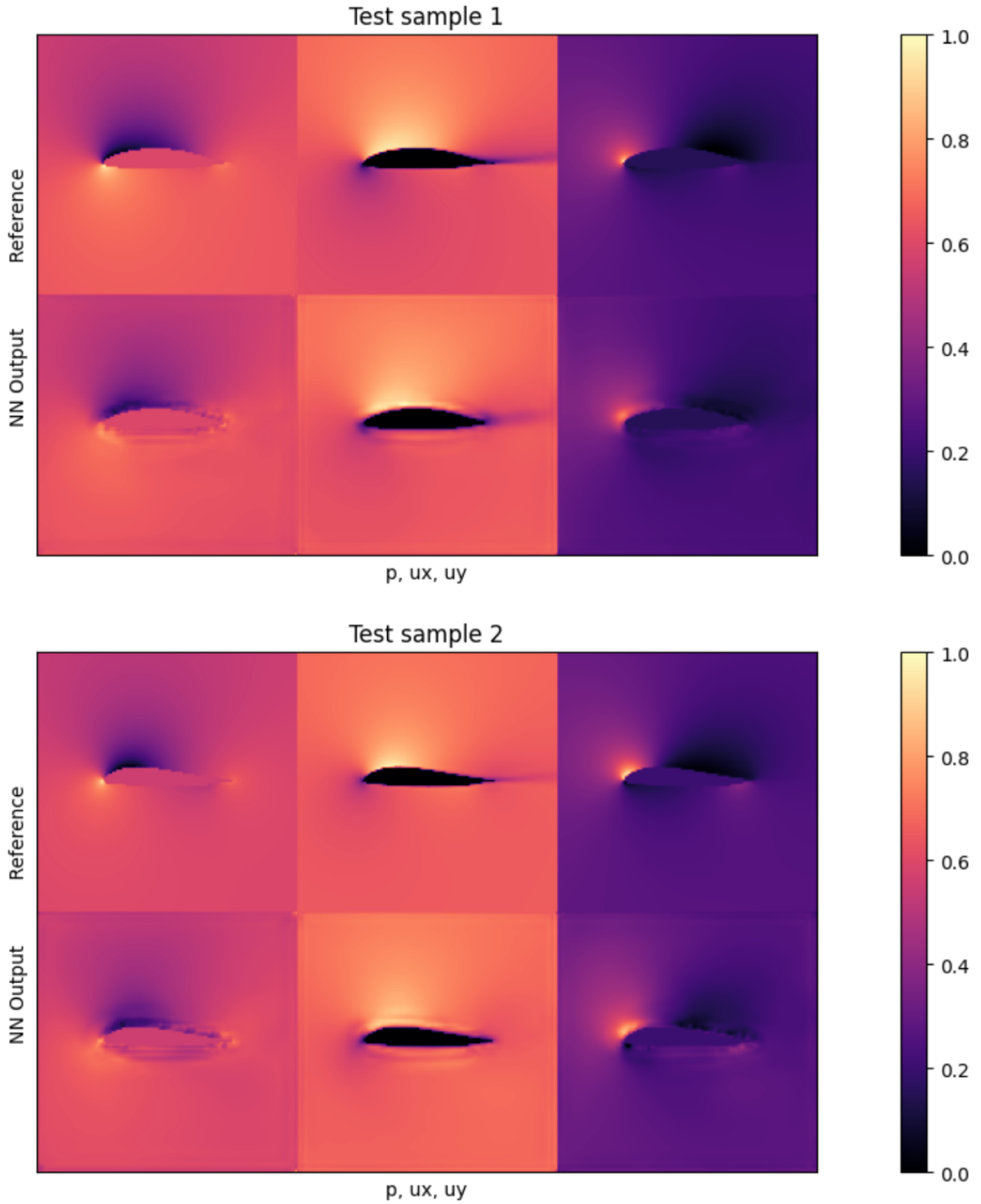
[3]本章第 6.2 节称压力相对速度近似平方标度，此处却称三次；二者不一致，且 Bernoulli 关系通常涉及速度平方，原文说法待核查。

下面的代码单元先下载包含这些测试样本的较小归档，再将它们输入网络。评估循环同时计算累积 L^1 误差，以量化网络在测试样本上的表现。

```
loader_test = Dataloader("airfoils-test", batch_size=1, normalize_data=None,
                         shuffle=False)
loss = nn.L1Loss()
net.eval()
L1t_accum = 0.
for i, testdata in enumerate(loader_test, 0):
    inputs_curr, targets_curr = testdata
    inputs = inputs_curr.float()
    targets = targets_curr.float()
    outputs = net(inputs)
    outputs_curr = outputs.data.cpu().numpy()
    inputs_curr = inputs_curr.cpu()
    targets_curr = targets_curr.cpu()
    L1t_accum += loss(outputs, targets).item()
    if i<3: plot(targets_curr[0], outputs_curr[0], mask=inputs_curr[0][2],
                 title="Test sample %d"%(i))
print("\nAverage relative test error: {}".format(L1t_accum/len(loader_test)))
```

```
Warning: `airfoils-test` is stored in single-file format. The download might take some time.
Success: Loaded airfoils-test with 10 simulations and 1 samples each.
Average relative test error: 0.026288176793605088
```





三幅图依次为测试样本 0、1、2；各图上排为参考值，下排为神经网络输出。默认设置下的平均测试误差应接近 0.025。原文称，由于输入经过归一化，这意味着相对于各量的最大值，三个场的平均相对误差约为 2.5%。对新形状而言这并不差，但显然仍有改进空间。[4]

[4]代码计算的是归一化空间中的 L1 均值，能否解释为相对于每个物理量最大值的误差，取决于数据加载器实际归一化定义，尚未核实。

观察可视化结果会发现，输出中尤其缺少高压峰值和较大 y 方向速度的局部区域。这主要由网络较小、没有足够资源重建细节造成。原文还称 L^2 也具有平均化行为，偏向较大结构（周围区域）而非局部峰值。[5]

尽管如此，我们已经成功用小型快速的神经网络架构替代了相当复杂的 RANS 求解器。它通过 PyTorch 开箱支持 GPU，可微，并且引入的误差只有百分之几。通过进一步改动和更多数据，可以使这一设置达到很高精度 [CT22]。

6.8 下一步

这里有许多显然值得尝试的事情（见下面建议），例如更长时间训练、更大的数据集和网络等。

- 尝试改变学习率、dropout 和网络大小，以降低测试集误差。对于给定训练数据，能将误差降到多低?
- 上面的设置使用归一化数据。可以撤销归一化以恢复原始场，检查网络相对于原始物理量的表现。
- 你会发现这个数据集提供的能力有些有限。可以前往本项目的主要 GitHub 仓库下载更大的数据集，或自行生成数据。

译注：网络示意与代码 原生示意按冻结源保留 $8^2 \times 256 \to 4^2 \times 512$ 的虚线 2×2 卷积，而代码 layer5 采用默认 4×4 卷积，二者并不一致。图例保持 4 种卷积/resize-convolution 与特征拼接共 5 项，6 条同尺度跳接均保留。

[5] 上面实际训练代码使用 `nn.L1Loss()`，与这里的 L^2 描述不一致。

第七章　监督方法讨论

前面的例子说明，监督训练可以作为解决非平凡任务的基础。主要工作量是收集足够大的示例数据集。一旦数据集存在，就可以训练网络近似这些解所表示的解流形，而训练后的网络能非常快速地给出预测。使用监督训练时，有几个重要事项需要牢记。

7.1　需要牢记的事项……

7.1.1　自然的起点

监督训练是任何深度学习项目的自然起点。使用尽可能少的数据进行完全监督测试几乎总是有意义的。这是一个纯粹的过拟合测试，但如果网络不能快速收敛并在单个例子上给出很好的表现，说明代码或数据中存在根本问题。因此没有理由继续进入更复杂的设置，否则只会使发现这些根本问题更加困难。

最佳实践　为总结前面各节分散的评论，下面给出设置深度学习项目的一组“黄金法则”：

- 始终从单样本过拟合测试开始。
- 检查网络的可训练参数数量，并确认数据已经适当归一化。
- 确保神经网络收敛。
- 然后逐渐增加训练数据量（并可能增加网络参数和深度）。
- 调整超参数，尤其是学习率。
- 最后再引入可微求解器或扩散训练等其他组件。

7.1.2　稳定性

监督训练的一个优良性质是它非常稳定。引入更复杂的物理模型或考虑更复杂的神经网络架构后，情况不会更好。

因此再次强调，从简单的过拟合测试开始时，要确保能够看到训练损失良好的指数下降。这是确定学习率这一最核心超参数上界和合理范围的良好设置。之后可能需要将其降低，但至少应该粗略估计 η 的合适数值。

7.1.3 了解数据

所有数据驱动方法都遵循“垃圾进，垃圾出”原则。因此，努力了解正在处理的数据非常重要。虽然没有适合所有情况的最佳方法，但我们强烈建议跟踪数据集的广泛统计量。一个好的起点是每个物理量的均值、标准差、最小值和最大值。如果其中某些值异常，这就是数据集中可能有坏样本的初步信号。

这些数值也可以方便地用直方图可视化，以追踪不需要的异常值。少量此类异常值就可能轻易地使数据集朝不希望的方向偏斜。最后，检查不同物理量之间的关系通常是了解数据集内容的好办法。下图给出这一过程的例子。

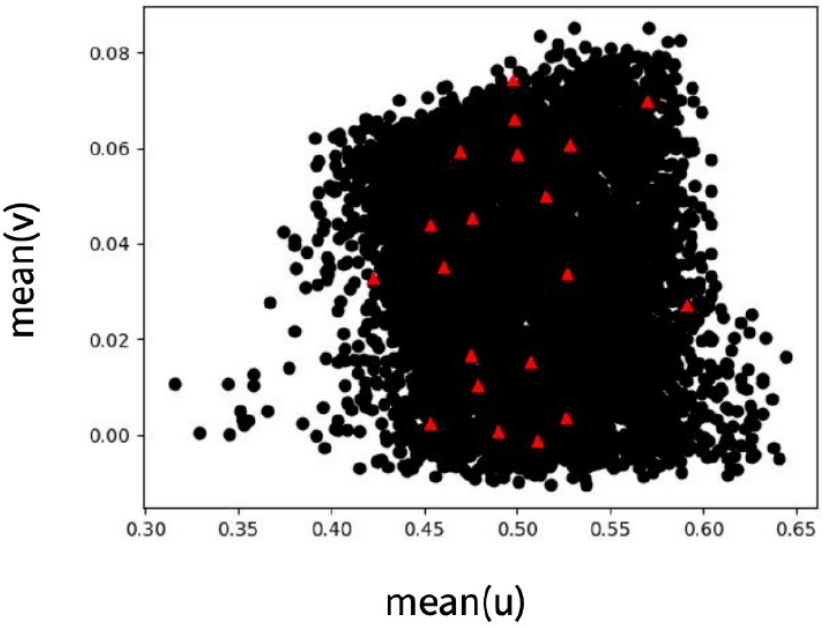


图 7.1: 前一节翼型情形的一个例子：按二维流场的平均 u、v 速度可视化训练数据集。图中清楚显示没有极端异常值，但左侧有少量平均 u 速度相对较低的条目。另以红色三角形叠加一个较小的测试数据集，表明其样本很好地覆盖了平均运动范围。

7.1.4 魔法在哪里？

谈到深度学习方法，特别是使用相对简单的训练方法时，经常会听到一个评论：“它不就是在对数据插值吗?”

没错，正是如此！这也正是神经网络应该做的。从某种意义上说，没有别的事情可做。原文认为所有深度学习方法都是如此：它们给出训练时见到的数据的平滑表示。即使后面在训练时使用复杂的物理模型，神经网络也只是调整权重来表示并再现所接收的信号。

由于宣传热潮和大量成功案例，不熟悉深度学习的人常常认为深度学习像人的思维一样工作，能够从数据集中提取基本且普遍的原理（“来自上帝的信息”之类）。这并非当前最先进方法所发生的情况。尽管如此，它仍是我们近似复杂非线性函数所拥有的最强大工具。它非常有用，但必须记住：正确设置训练后，我们得到的是神经网络被训练去表示的函数的近似，其中没有魔法。

这一点的含义是，不应期望网络在从未见过的数据上工作。从某种意义上说，神经网络之所以表现如此好，恰恰是因为它能准确适应训练时接收的信号；但与其他学习得到的表示相比，它们并不擅长外推。因此不能指望神经网络神奇地适用于新输入，而必须确保能够恰当地塑造输入空间，例如通过归一化和关注不变量。[1]

更具体地说，如果总是在 $[0,1]$ 范围内的输入上训练网络，就不要期望它适用于 $[27,39]$ 的输入。在某些情况下，可以通过减去均值并以标准差或合适分位数归一化输入和输出（确保这不会破坏数据中的重要相关性）。展望而言，快速求解器可能足以处理约 27 的大偏移，使神经网络能够集中处理归一化残差所给出的有限输入范围。

一个经验法则是：确保实际用于训练神经网络的输入与推断时希望使用的输入尽可能相似。在后续章节中应牢记这一点：例如，如果希望网络与某个仿真环境协同工作，就应真正将模拟器纳入训练过程。否则，网络可能专门适应预先计算的数据，而这些数据与神经网络和求解器结合时产生的数据不同，即会受到分布偏移影响。

7.2 监督训练概要

概括而言，监督训练有以下性质。

优点

- 训练非常快速；
- 稳定且简单；
- 是很好的起点。

缺点

- 需要大量数据，加载可能成为瓶颈；
- 精度和泛化性能可能次优；
- 难以与外部“过程”交互，例如嵌入求解器。

后续章节将说明如何缓解监督训练的这些缺点。首先，通过软约束引入模型方程；随后重新讨论结合数值仿真与学习方法面临的挑战；最后，将基本方法扩展到使用扩散模型和流匹配的生成建模。

[1] 这里是原书的概括性经验判断；实际泛化与外推能力依赖任务、模型和数据，须在具体情形中评估。

第三部分

物理损失

第八章　物理损失项

前面几节的监督设置可以通过简单稳定的训练过程快速得到近似解。然而，遗憾的是我们只将物理模型和数值方法当作产生大量数据的“外部”工具。

人类拥有大量关于如何用数学描述物理过程的知识。正如后续章节将展示的，我们可以利用人类的物理知识引导训练，以改善训练过程。

8.1　使用物理模型

给定关于 $\mathbf{u}(\mathbf{x}, t)$ 的时间演化 PDE，通常可以通过 $\mathbf{u}$ 的导数的函数 $\mathcal{F}$ 将其表示为

$$\mathbf{u}_t = \mathcal{F}(\mathbf{u}_x, \mathbf{u}_{xx}, \dots \mathbf{u}_{xx\dots x}),$$

其中 x 下标表示对空间维度的空间导数（当然也可以包括对不同坐标轴的混合导数），$\mathbf{u}_t$ 表示时间变化。给定解 $\mathbf{u}$，可以计算残差 R；对于正确的解，残差自然应等于零：

$$R = \mathbf{u}_t - \mathcal{F}(\mathbf{u}_x, \mathbf{u}_{xx}, \dots \mathbf{u}_{xx\dots x}) = 0.$$

在这个背景下，可以用神经网络近似未知的 $\mathbf{u}$ 本身。如果近似准确，PDE 残差同样应为零。

这与神经网络训练目标可以很好地结合：我们可以训练网络，使该残差与直接损失项共同最小化。除了依赖残差，还可以使用预先计算的 $\mathbf{u}$ 的解 $[x_0, y_0], \dots, [x_n, y_n]$ 作为目标，其中 $\mathbf{u}(\mathbf{x}) = y$。这通常很重要，因为大多数实际 PDE 除非指定初始条件和边界条件，否则没有唯一解。因此，如果只考虑 R，可能得到具有随机偏移或其他不希望有的分量的解。监督样本点因而有助于在某些位置确定解。

现在训练目标变为

$$\arg\min_{\theta} \sum_i \alpha_0 \big(f(x_i; \theta) - y_i^*\big)^2 + \alpha_1 R(x_i), \tag{8.1}$$

其中 $\alpha_{0,1}$ 分别是调整监督项和残差项贡献的超参数。当然也可以在此加入具有合适缩放因子的额外残差项。[1]

理解式 (8.1) 中两项的含义很有启发：第一项是传统的监督 L^2 损失。如果只优化这一项，网络会学会很好地近似训练样本，但可能将解的多个模态平均，在样本点之间的区域表现不佳。相反，如果只优化第二项（物理残差），神经网络可能在局部满足 PDE，却很难找到全局吻合的解。这可能由于解中的“零空间”而发生：不同解都满足残差；局部点可能收敛到不同解，组合后得到非常次优的结果。因

[1] 式 (8.1) 按原文直接写入带符号残差 R，未取模或平方。一般情况下直接最小化带符号残差并不等于使残差趋于零；目标函数的预期形式需要核查，不能由译文擅自改写。

此我们同时优化两个目标，希望在最佳情况下，网络既学会近似训练数据中的特定解，又能捕获关于底层 PDE 的知识。

注意，与用于监督训练的数据样本类似，我们无法保证训练期间残差项 R 真正达到零。训练过程中的非线性优化会尽可能减小监督项和残差项，但没有保证，仍可能残留较大的非零残差。后续代码示例将更详细地讨论；现在需要记住，以这种方式表述的物理约束只是软约束，不保证这些约束被最小化。

前面的概述并未真正说明神经网络如何产生 $\mathbf{u}$。这里可以区分两种方法：通过选定的目标函数显式表示（下文称 v1），或者通过基于全连接神经网络的神经场表示解（v2）。例如，v1 可以设置空间网格、图或一组样本点；第二种情形不存在显式表示，网络接收空间坐标并在查询位置产生解。下面进一步概述这两种变体。

8.2 变体 1：显式表示的残差导数

对于变体 1，我们选择离散化并设置覆盖目标区域的计算网格。不失一般性，假设它是以位置 $\mathbf{p}$ 采样空间的笛卡尔网格。然后训练神经网络产生网格上的解：$\mathbf{u}(\mathbf{p}) = f(\mathbf{x};\theta)$。对于规则网格，CNN 是 f 的良好选择；对于三角网格可以使用图网络，而对于粒子则可以使用带点卷积的网络。现在可以在计算

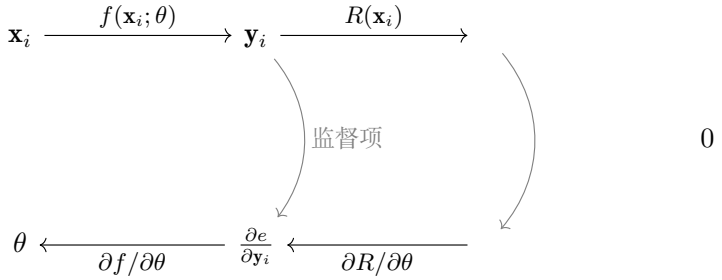


图 8.1: 变体 1：解由选定的计算网格表示，并由神经网络产生。残差在该网格上离散化，可以与监督项结合。图中标注的 supervised component 为监督分量。

网格上离散化 R 的方程，并用所选方法计算导数。唯一需要注意的是：为了将残差纳入训练，必须以深度学习框架能够通过这些计算反向传播的形式进行求值。由于网络 $f()$ 产生解 $\mathbf{u}$，而残差依赖于它，即 $R(\mathbf{u})$，至少需要 $\partial R/\partial\mathbf{u}$，才能为权重 θ 反向传播梯度。幸运的是，如果用深度学习框架的操作构造 R，框架的反向传播功能就会处理这一点。

这一变体在深度学习中已有相当长的“传统”。例如，Tompson 等人 [TSSP17] 早期提出用它学习无散度运动。具体而言，若目标是学习满足 $\nabla\cdot\mathbf{u}=0$ 的速度 $\mathbf{u}(t)$，可以使用这种训练方法，而无须预先计算无散度速度场作为训练数据。为简洁起见，略去空间下标 $(\mathbf{p})$ 并关注 t；还可以进一步简化：无散度条件必须在所有时间成立，因此可以考虑从 $t=0$ 开始、$\Delta t=1$ 的单步，即从有散度的 $\mathbf{u}(0)$ 到无散度的 $\mathbf{u}(1)$ 的归一化步。

对于普通求解器，原文写道必须计算压力 $p=\nabla^{-2}\mathbf{u}(0)$，使得 $\mathbf{u}(1)=\mathbf{u}(0)-\nabla p$。这是著名的向量分析基本定理或 Helmholtz 分解，将向量场分解为无散度部分和无旋部分（压力梯度）。[2]

为了学习这一分解，可以在计算网格上用 CNN 近似 p：$p=f(\mathbf{u}(0);\theta)$。学习目标变为最小化 $\mathbf{u}(0)$ 的散度，即最小化 $\nabla\cdot(\mathbf{u}(0)-\nabla f(\mathbf{u}(0);\theta))$。实现此残差只需提供计算网格上的 $\mathbf{u}$ 的散度算子 $\nabla\cdot$。通常

[2]原文压力式 $p=\nabla^{-2}\mathbf{u}(0)$ 的标量 / 矢量维度及散度项需要核查；常见投影表述会对速度场的散度求逆 Laplace 算子，此处保留原式。

可以通过深度学习框架中包含散度有限差分权重的卷积层轻松实现。好在这种情况下甚至不需要额外监督样本，通常可以完全使用这一残差形式训练。与下面的变体 2 不同，这里还可以直接处理相当大的解空间，而不局限于学习单个解。原文在代码仓库中提供了示例实现。

总体而言，变体 1 与可微物理训练有许多共同之处（基本上是其子集），后者将在 “可微物理简介” 中详细讨论。因此本章将更多关注直接神经网络表示，即变体 2。

8.3 变体 2：神经网络表示的导数

使用物理残差作为软约束的第二种变体，改用全连接神经网络表示 $\mathbf{u}$。这种物理信息方法（PINN）由 Raissi 等人 [RPK19] 推广，其优缺点将在下文概述。目前，该方法可视为神经场表示的一部分；后者还包括 NeRF 和学习得到的有符号距离函数等。

神经场的核心思想是，所寻找的上述一般函数 f 也能用于获得物理场的表示，例如满足 $R = 0$ 的场 $\mathbf{u}$。这意味着 $\mathbf{u}(\mathbf{x})$ 变为 $\mathbf{u}(\mathbf{x}, \theta)$；我们选择网络参数 θ，使所需的 $\mathbf{u}$ 尽可能精确地被表示，而 $\mathbf{u}$ 只需返回空间位置 $\mathbf{x}$ 处的正确值。

这一视角的一个优点是，神经网络表示天然支持计算对输入的导数。如 “概述” 所述，导数 $\partial f / \partial \theta$ 是通过梯度下降学习的关键构件。现在可以使用相同工具计算空间导数，例如 $\partial \mathbf{u} / \partial x = \partial f / \partial x$；上面在 R 中将其简记为 $\mathbf{u}_x$。对于随时间变化的函数，当然也可以添加输入 t，计算 $\partial \mathbf{u} / \partial t$，即上述记号中的 $\mathbf{u}_t$。

因此，对于由 $\mathbf{u}_t$ 和 $\mathbf{u}_x$ 等项组成的一般残差 R，一旦有表示 $\mathbf{u}$ 的神经网络，就可以依靠深度学习框架的反向传播算法计算这些导数。本质上这给出一个函数（神经网络），接收空间和时间坐标并产生 $\mathbf{u}$ 的解。因此输入维度通常相当低，例如随时间变化的三维情形只有 $3 + 1$ 个值，输出往往是标量或空间向量。由于缺少显式空间采样点，这里选择 MLP，即全连接神经网络。

以简单的一维 Burgers 方程 $\partial u / \partial t + u \nabla u = \nu \nabla \cdot \nabla u$ 为例，可以直接构造训练时应尽可能最小化的损失项

$$R = \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2}.$$

对于每一项，例如 $\partial u / \partial x$，只需查询实现 u 的深度学习框架，就能得到相应导数。对于 $\partial^2 u / \partial x^2$ 等高阶导数，可以多次查询框架的求导函数。后面将以 TensorFlow 给出具体例子。

8.4 阶段总结

上述方法提供了将物理方程作为软约束纳入深度学习的方式：残差损失。v1 依赖离散化形式的归纳偏置，而 v2 依赖通过自动微分计算的导数。通常 v2 特别适合逆问题：我们具有某些测量或观测，希望找到对应的 PDE 解。

由于优化和学习问题的不适定性，以及重建的高成本（下文将展示），对这些 PINN 方法来说解流形不宜过于复杂。例如，捕获时间依赖或很宽的解范围通常非常困难，如前面的监督翼型示例所示。接下来将用代码展示这些概念：先展示如何用 v1 方法在实践中学习 Helmholtz 分解，然后通过实际例子说明 v2 的 PINN 方法。

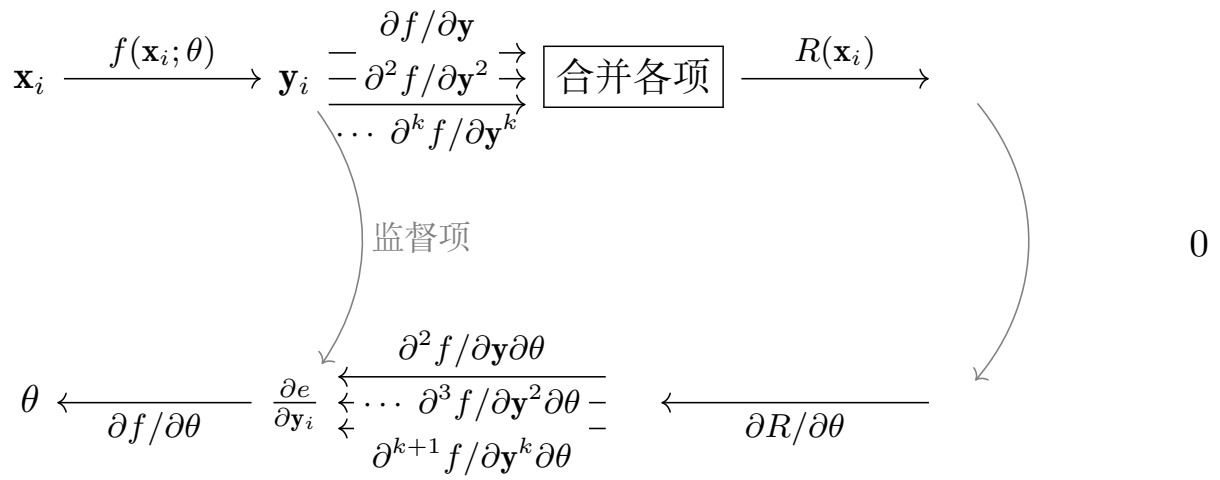


图 8.2: 变体 2：解由全连接网络产生，通常需要将监督损失与神经网络导数组成的残差结合。神经网络导数各有优缺点。图中 combine terms 为组合各项，supervised component 为监督分量。

译注：图中求导变量　两图沿冻结源保留 $R(x_i)$ 标签；神经场图的各阶导数按源写对 y，而正文按空间 x 求导，图节点 y_i 又指输出，存在源变量标签冲突，未静默改成 x。

第九章　学习 Helmholtz–Hodge 分解

在下面的 notebook 中，我们将遵循前面提到的 Tompson 等人论文 [TSSP17]，训练一个执行 Helmholtz–Hodge 分解的神经网络。这是许多数值求解器中非常经典且耗时的一部分，能够将任意向量场分解为无散度部分和无旋部分（压力梯度）。由于传统上它十分耗时，因而是学习方法中一个有趣的目标。作为迈向集成完整求解器的过渡，我们将通过离散化的 PDE 约束构造基于物理的损失（即上一节称为物理损失训练 v1 的方法）。

9.1　求解 Navier–Stokes 方程

为了说明动机，先简要回顾不可压缩 Navier–Stokes 方程通常如何求解：一种基本变体使用算子分裂，分别求解平流和压力修正，得到时间上一阶精度。因此，给定任意流场 $\mathbf{u}$，计算自平流步，求解 $\partial \mathbf{u}/\partial t + \mathbf{u} \cdot \nabla \mathbf{u} = 0$。这一步通常很高效，其复杂度与离散点数量呈线性关系，并给出平流后的流场 $\tilde{\mathbf{u}}$。

使用上述 Helmholtz–Hodge 分解的压力投影（或 Chorin 投影）步骤需要求解 Poisson 问题 $\nabla^2 p = \nabla \cdot \tilde{\mathbf{u}}$，这是一个在一般情形下并不容易求解的椭圆 PDE。在 Navier–Stokes 情形下，它给出瞬时标量压力场 p，其梯度恰好捕获速度场中有散度的部分。因此，一旦减去 ∇p，就得到满足 $\nabla \cdot \mathbf{u} = 0$ 的无散度场。

下面的目标是对新的速度输入 $\mathbf{u}$ 获得 p。这一问题的优点是可以在不使用监督设置的情况下训练：无须预先计算大量 $(\mathbf{u}, p)$ 对，而是依赖离散化的 PDE 约束。这可以称为无监督训练，但原文认为这一名称有误导性，因为所有“无监督”情形中的训练目标都只是即时计算出来的。

这种方法更重要的概念方面是，我们有目标 PDE $\nabla \cdot \mathbf{u} = 0$，可以通过合适的有限差分算子在计算区域上离散化。一旦以可微方式做到这一点，就能直接用有限差分算子训练神经网络：由于目标是最小化 $\nabla \cdot \widetilde{\mathbf{u}}$，实际上得到目标为 0 的 L^2 损失。从这一损失出发，可以通过离散散度算子反向传播，得到对速度的梯度。按照前面的说明，$\widetilde{\mathbf{u}} = \tilde{\mathbf{u}} - \nabla p$，其中 p 是参数为 θ 的神经网络 f 的输出。原文称，既然散度是唯一确定压力场所需的全部信息，就可以将散度传给 f 以简化推断任务。因此，$f(\nabla \cdot \tilde{\mathbf{u}}; \theta)$ 将接收从损失通过所有步骤反向传播的梯度，更新状态 θ，使上述速度场最终无散度。[1]

综合这些步骤，目标是求解最小化问题

$$\arg\min_{\theta} \sum_{i} \left(\nabla \cdot (\tilde{\mathbf{u}} - \nabla f(\nabla \cdot \tilde{\mathbf{u}}; \theta)) \right)^2,$$

其中下标 i 表示任意数量的不同输入，例如训练时一个小批次中的输入。原 PDF 此处有可见排版损坏，文字和公式依据相邻上下文分离，未补造新的物理项。

[1] 在周期或 Neumann 边界条件下，Poisson 压力一般还存在常数规范自由度；源文后续也讨论了常数偏移。压力的唯一性还依赖边界条件及规范选择。

9.2 建立离散 PDE

与前面一样，我们使用 PhiFlow 框架。下面的代码单元通过 pip 安装 PhiFlow，并导入 PyTorch 后端及用于跟踪进度的 tqdm。注意只需修改下面的导入，即可相当容易地将 notebook 切换至 TensorFlow。

```
!pip install --quiet phiflow==3.3 tqdm
from tqdm import tqdm
from phiml import nn
from phi.torch.flow import *
# this notebook largely works identically with tensorflow - try replacing
# the line above with this one
#from phi.tf.flow import *
```

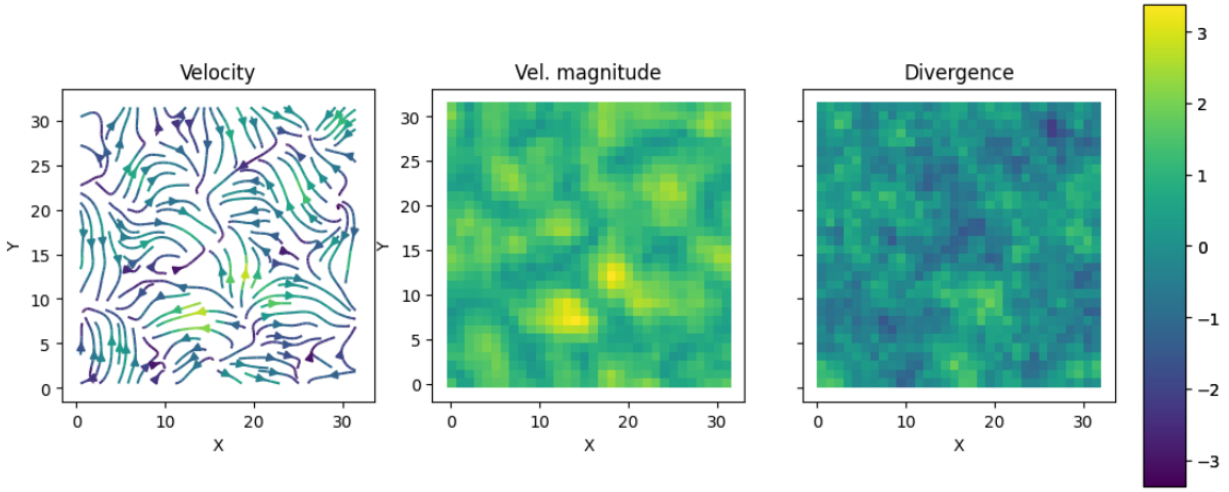
定义一些基本常量：网格分辨率（在 PhiFlow 中也用作物理尺寸）、训练时考虑的不同流场数量以及批次大小。

```
RES_X = 32
RES_Y = 32
NUM_SAMPLES_TRAIN = 100
BATCH_SIZE = 10
```

我们使用 PhiFlow 基于频率的合成方法生成随机周期流场。下一个代码单元预先计算所有这些流场（共 100 个）并可视化其中一个例子。注意，这里只是采用随机的有散度流场；我们不知道其正确的无散度对应场。

```
vel = field.StaggeredGrid(
    values=field.Noise(batch(batch=NUM_SAMPLES_TRAIN)),
    extrapolation=math.extrapolation.PERIODIC,
    bounds=geom.Box(x=RES_X, y=RES_Y),
    resolution=math.spatial(x=RES_X, y=RES_Y),
)

# plot the velocity field
plot = vis.plot({ "velocity": vel.batch[0],
    "vel. magnitude": math.vec_length(vel.batch[0].at_centers().values),
    "divergence": field.divergence(vel.batch[0]), })

# for initial assessments, let's print the typical divergence of an input
print( f"Initial L2 of divergence: {math.l2_loss(field.divergence(vel.batch[0]))}, "
       f"max={field.divergence(vel.batch[0]).values.max}" )
```

```
Initial L2 of divergence: 195.28241, max=1.8969224691390991
```



图中从左至右为流线、速度大小和每个单元的散度。后者正是我们希望消除的。原文称这一可视化表明散度小于速度的实际大小，平均约为 0.4，如图上方的 L^2 输出所示。[2]

接下来定义 Navier–Stokes 仿真步。由于这里是没有外力或障碍物的简化设置，它非常简单：先调用 PhiFlow 中的平流函数，再调用 `fluid.make_incompressible()` 启动 Poisson 求解器。这里需要传入自定义求解器来处理周期求解中的秩亏缺（常数偏移具有歧义）；在压力解唯一的后续情形中则不需要。我们还将直接使用 `@jit_compile` 标记该函数及后续函数，以便即时编译。这对 GPU 上获得良好性能很重要，但会使调试困难得多。因此修改代码时强烈建议移除这些标记；代码仍能正常工作，只是更慢。一旦一切运行正常，再为实际训练重新启用即时编译。

```
@jit_compile
def step(v, dt = 1.0):
    v = advect.mac_cormack(v, v, dt)
    v, p = fluid.make_incompressible(v, [], solve=Solve(rank_deficiency=0))
    return v, p


v,p = step(vel)
```

原书警告输出中的语句（不是另需执行的输入）：

```
warnings.warn("Possible rank deficiency detected. Matrix might be singular which "
              "can lead to convergence problems. ")
```

该单元直接对前面的随机速度场运行求解器。如果感兴趣，可以通过 `vis.plot()` 绘制它。不过，前面章节已经展示过 PhiFlow 的 Navier–Stokes 求解器工作正常，因此这里集中讨论神经网络训练。下面的目标是训练网络，而无须使用上述 `make_incompressible()` 等函数预先计算许多压力样本；相反，我们将使用可微散度算子设置训练。

9.3 神经网络训练

这里面对的是椭圆 PDE 问题，因此具有全局通信能力的神经网络架构很重要，参见“神经网络架构”一章。下面初始化 U-Net，也可以尝试 ResNet 变体；由于局部感受野，它的表现较差。（考虑到这

[2] 从所打印的 `math.l2_loss` 值到平均散度约 0.4 的换算依赖该函数的具体归一化定义，尚未验证。

一性质，其表现仍出奇地好。）为简单起见，我们处理周期区域，网络也需要通过 `periodic=True` 配置为周期处理。输入是单通道的散度，输出是内容完全不同的压力；但对于网络来说，它同样只是一个标量通道。`filters=24` 决定参数总数。可以增大它以提高精度，但会降低网络推断的计算性能。这是神经网络与所有经典数值方法共有的精度与性能权衡。

```
from phiml import nn
network = nn.u_net(in_channels=1, out_channels=1, levels=4, periodic=True,
                   batch_norm=False, in_spatial=(RES_X, RES_Y), filters=24)

# uncomment this for an alternative architecture that could generalize to
# different resolutions
#network = nn.res_net(in_channels=1,out_channels=1,layers=[32,32,32])

# print network and parameter summary
from phiml.backend import BACKENDS
if any([b.name == 'torch' for b in BACKENDS]):
    print(network)
    print("Total number of trainable parameters: "+ str(
        sum(p.numel() for p in network.parameters()) ))
elif any([b.name == 'tf' for b in BACKENDS]):
    network.summary()
```

原 PDF 中网络结构打印输出跨越三页，以下按原样保留其完整可见输出；最后可见可训练参数总数为 83521，约 8.3 万。原书完整网络打印输出（可编辑转写）：

```
UNet(
  (inc): DoubleConv(
    (double_conv): Sequential(
      (0): Conv2d(1, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), padding_mode=circular)
      (1): Identity()
      (2): ReLU()
      (3): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), padding_mode=circular)
      (4): Identity()
      (5): ReLU(inplace=True)
    )
  )
  (down1): Down(
    (maxpool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (conv): ResNetBlock(
      (sample_input): Identity()
      (bn_sample): Identity()
      (bn1): Identity()
      (conv1): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (bn2): Identity()
      (conv2): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    )
  )
  (up1): Up(
```

```
    (up): Upsample(scale_factor=2.0, mode='bilinear')
    (conv): DoubleConv(
      (double_conv): Sequential(
        (0): Conv2d(48, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
        ↪  padding_mode=circular)
        (1): Identity()
        (2): ReLU()
        (3): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
        ↪  padding_mode=circular)
        (4): Identity()
        (5): ReLU(inplace=True)
      )
    )
  )
  (down2): Down(
    (maxpool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (conv): ResNetBlock(
      (sample_input): Identity()
      (bn_sample): Identity()
      (bn1): Identity()
      (conv1): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (bn2): Identity()
      (conv2): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    )
  )
  (up2): Up(
    (up): Upsample(scale_factor=2.0, mode='bilinear')
    (conv): DoubleConv(
      (double_conv): Sequential(
        (0): Conv2d(48, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
        ↪  padding_mode=circular)
        (1): Identity()
        (2): ReLU()
        (3): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
        ↪  padding_mode=circular)
        (4): Identity()
        (5): ReLU(inplace=True)
      )
    )
  )
  (down3): Down(
    (maxpool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (conv): ResNetBlock(
      (sample_input): Identity()
      (bn_sample): Identity()
      (bn1): Identity()
      (conv1): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
```

```
      (bn2): Identity()
      (conv2): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    )
  )
  (up3): Up(
    (up): Upsample(scale_factor=2.0, mode='bilinear')
    (conv): DoubleConv(
      (double_conv): Sequential(
        (0): Conv2d(48, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
  padding_mode=circular)
        (1): Identity()
        (2): ReLU()
        (3): Conv2d(24, 24, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
        ↪  padding_mode=circular)
        (4): Identity()
        (5): ReLU(inplace=True)
      )
    )
  )
  (outc): Conv2d(24, 1, kernel_size=(1, 1), stride=(1, 1))
)
Total number of trainable parameters: 83521
```

接下来可以设置训练过程：下面的 eval_nn() 函数接收一批流场，计算其散度，并以此调用神经网络。所得标量场转换为 PhiFlow 张量并解释为压力场。按照 Helmholtz–Hodge 分解，从输入流场中减去它的梯度。我们的目标是使所得流场 v 不可压缩，即其散度应最小。

为此，定义计算每个单元散度 L^2 的损失函数 loss_div()。为方便起见，还定义供 PhiFlow 使用的辅助函数 loss_func：它对输入计算网络输出及所得损失。PhiFlow 将调用此函数，并确保梯度从单个损失值通过离散散度算子反向传播至网络权重。注意在不可压缩设置中，我们总是移除流中存在的散度，因此时间步长不起作用。

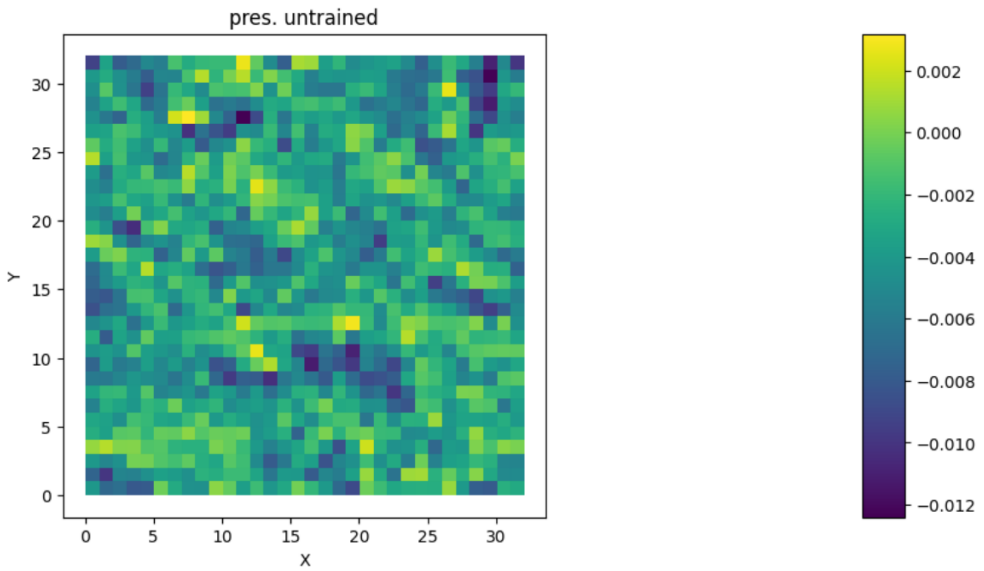
```
@jit_compile
def eval_nn(v):
    nn_input = field.divergence(field=v, order=2)
    p = math.native_call(network, nn_input.values)
    p = field.CenteredGrid(
        values=p,
        extrapolation=math.extrapolation.PERIODIC,
        bounds=geom.Box(x=RES_X, y=RES_Y),
        resolution=spatial(x=RES_X, y=RES_Y),
    )
    grad_pres = field.spatial_gradient(p, boundary=math.extrapolation.PERIODIC,
                                       at=v.sampled_at)
    v = v - grad_pres
    return v, p

# loss functions
```

```
@jit_compile
def loss_div(v):
    div = field.divergence(field=v,order=2)
    div_sum = 2 * math.l2_loss(div)
    return div_sum, div

@jit_compile
def loss_func(v):
    v, p = eval_nn(v)
    loss, div = loss_div(v)
    return math.mean(loss,dim=batch), v, p
```

训练网络之前，先看看随机初始化网络会产生什么。下面对第一个输入流场评估未训练的网络：

```
vel_untrained, pres_untrained = eval_nn(vel.batch[0])

# optionally, visualize the outputs: this doesn't look much different from before
# as the NN is untrained
#plot = vis.plot( {"vel untrained": vel_untrained,
#    "vel len. untrained": math.vec_length(vel_untrained.at_centers().values),
#    "div. untrained": field.divergence(vel_untrained), })

# print the loss and divergence sum of the corrected velocity from untrained NN
loss, div_untrained = loss_div(vel_untrained)
print(f"Loss for untrained network: {loss}")
# also, we visualize the pressure field
plot = vis.plot(pres_untrained,title="pres. untrained")
```

```
Loss for untrained network: 390.21857
```



图中为未训练网络产生的压力场。并不令人意外，压力场看起来相当随机，散度对应的损失也很大。

9.4 训练

有了目前的设置，训练非常简单：选择一些随机流场，计算网络输出，并改变其权重使散度损失减小。下面直接按固定的 epoch 数循环遍历预先计算的流场，不进行随机化。网络收敛得出奇快，几个 epoch 后损失应降低两个以上数量级。

```
optimizer = nn.adam(network, learning_rate=5e-3)
for epoch in tqdm(range(15)):
    for b in range(int(NUM_SAMPLES_TRAIN / BATCH_SIZE)):
        vel_input = vel.batch[b*BATCH_SIZE:b*BATCH_SIZE + BATCH_SIZE]
        loss, pred_v, pred_p = nn.update_weights(
            network, optimizer, loss_func, vel_input
        )

loss, _,_ = loss_func(vel)
print(f"Final loss={loss.numpy()}")
```

```
  0%|          | 0/15 [00:00<?, ?it/s]
100%|##########| 15/15 [00:16<00:00, 1.12s/it]
Final loss=0.6785882711410522
```

上面进度条用 # 表示原文中的实心块。看到损失下降是好事，但现在最重要的问题当然是：只在合成的随机采样流场上训练后，这个网络面对新输入会怎样，即它对不同任意流动的泛化能力如何？

9.5 使用新输入测试

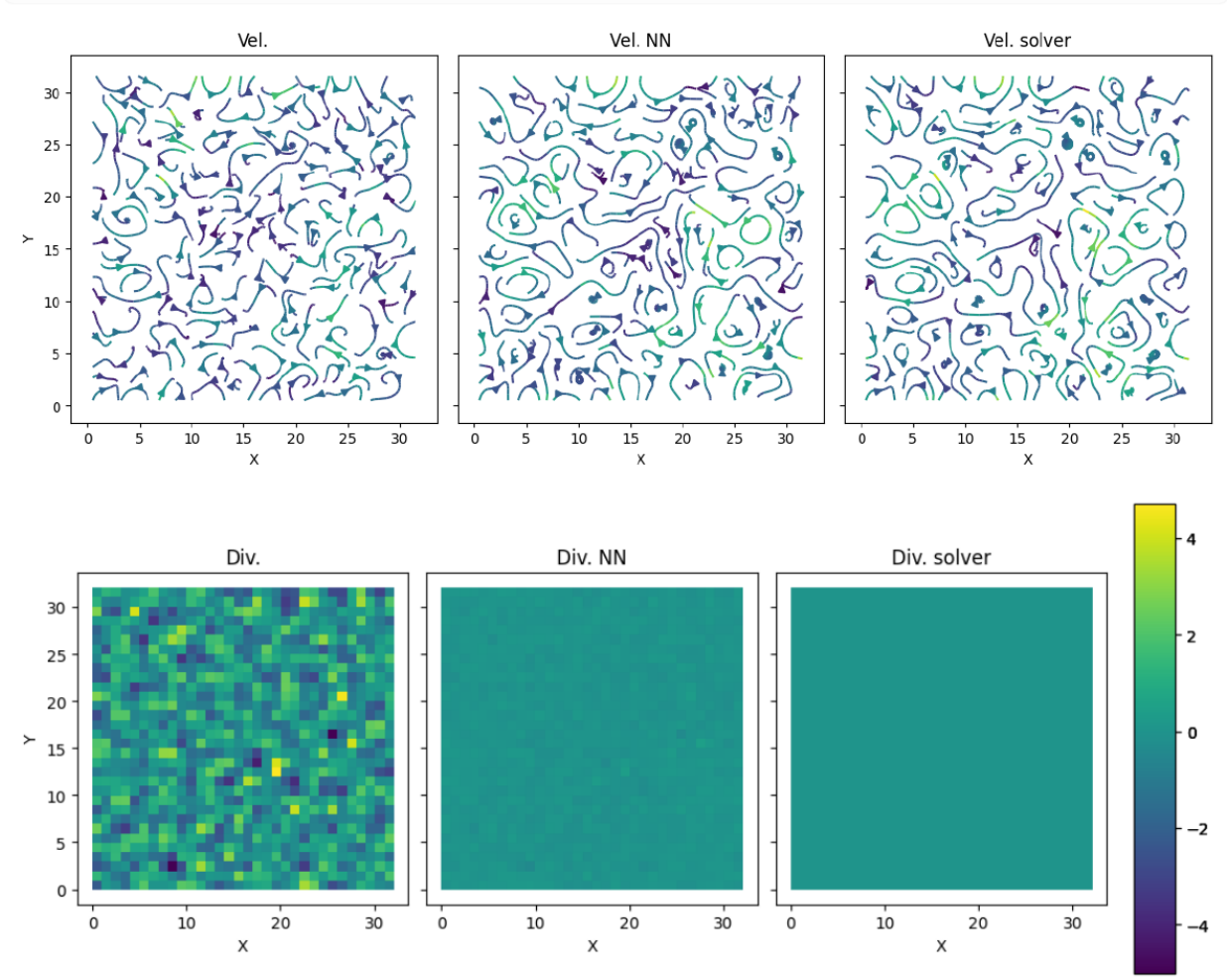
可以通过生成一些新输入来检查这一点。下面同样使用 PhiFlow 的噪声生成获得新场，但为了增加趣味，将尺度增大两倍。因此，网络将接收到此前未见过的散度输入量级，这些样本实际上处于训练输入分布之外。

```
NUM_SAMPLES_TEST = 10
vel_test = field.StaggeredGrid(
    values=field.Noise(math.batch(batch=NUM_SAMPLES_TEST),scale=2.),
    extrapolation=math.extrapolation.PERIODIC,
    bounds=geom.Box(x=RES_X, y=RES_Y),
    resolution=math.spatial(x=RES_X, y=RES_Y),
)
```

现在在这些输入上运行训练后的网络和求解器，以便比较：

```
vel_nn, p_nn = eval_nn(vel_test)
vel_solver, p_solver = fluid.make_incompressible(vel_test, [])

loss, div = loss_div(vel_test)
print(f"Original, mean divergence={loss.mean:.3f}, div. max={math.max(div.values).mean:.3f}")
loss, div = loss_div(vel_nn)
print(f"NN, mean divergence={loss.mean:.3f}, div. max={math.max(div.values).mean:.3f}")
loss, div = loss_div(vel_solver)
```

```
print(f"Solver, mean divergence={loss.mean:.3f}, div. max={math.max(div.values).mean:.3f}")

plot = vis.plot({"vel.": vel_test.batch[0], "vel. NN": vel_nn.batch[0],
                 "vel. solver": vel_solver.batch[0]})
plot = vis.plot({"div.": field.divergence(vel_test.batch[0]),
                 "div. NN": field.divergence(vel_nn.batch[0]),
                 "div. solver": field.divergence(vel_solver.batch[0]),})

anaconda3/envs/torch24/lib/python3.12/site-packages/phiml/math/_optimize.py:631:
UserWarning: Possible rank deficiency detected. Matrix might be singular which can
lead to convergence problems. Please specify using Solve(rank_deficiency=...).
  warnings.warn("Possible rank deficiency detected. Matrix might be singular which
  can lead to convergence problems. Please specify using Solve(rank_deficiency=...).")

Original, mean divergence=2059.143, div. max=4.946
NN, mean divergence=11.510, div. max=0.417
Solver, mean divergence=0.000, div. max=0.000
```

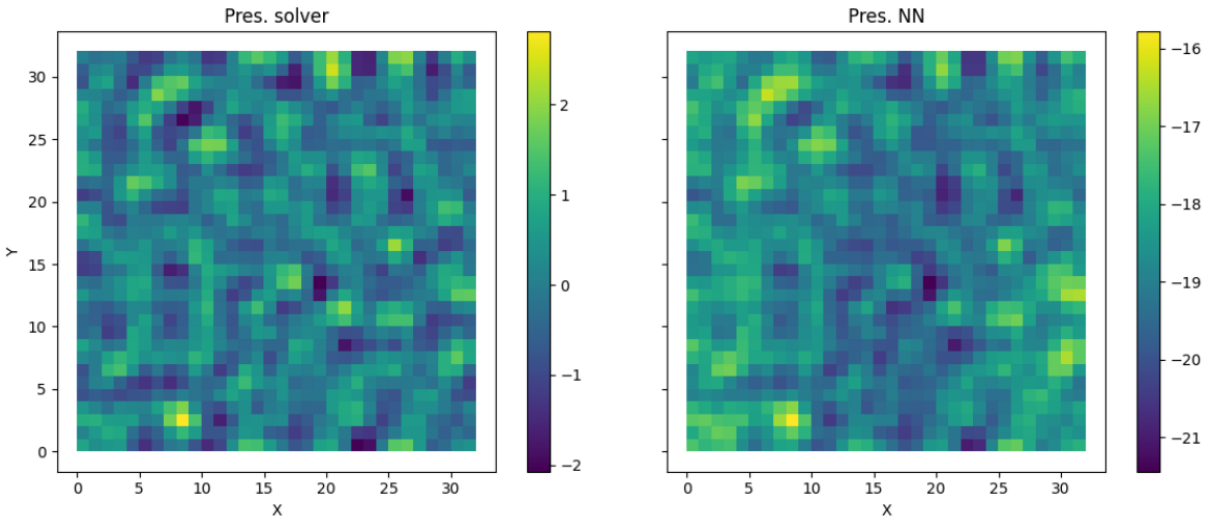


图中上排为速度、下排为散度；各排从左至右是原始输入、网络结果和求解器结果。定量评估确认网络

学会了减小散度。剩余散度大于 PhiFlow 求解器的结果，但仍比原始值小几个数量级。上图清楚地展示了这一点。[3]

接下来检查网络实际学会产生怎样的压力场。

```
# plot the pressure on the training data, compared with the pressure given by
# traditional solver.
plot = vis.plot({"pres. solver": p_solver.batch[0], "pres. NN": p_nn.batch[0]},
                same_scale=False)
```



图中左侧为求解器压力，右侧为网络压力。两个场在结构上有某些相似性，但显然并不相同。它们的数值通常也差异很大（网络训练是随机的，因此差异可以变化）。神经网络经常学会生成大 10 倍的数值，而 PhiFlow 压力求解器被约束为产生均值为零的值。

尽管如此，上面的度量显示网络压力场成功消除了大部分散度。从数学上看，这里的 Poisson 问题有常数偏移形式的零空间：求导后偏移不起作用，因此所有常数偏移都是完全有效的解。用于训练的可微损失对绝对值没有约束，故网络可以自由使用由其随机初始化产生的任意偏移。[4]

9.6 更严格的测试：带障碍物的流体仿真

[5] 最初的目标是 Navier–Stokes 求解器，因此必须检查网络能否实际处理流体求解器产生的流场输入。毕竟，它只在与流体运动相当不同的合成随机场上训练过。为了让问题更有趣，我们直接加入浮力驱动的烟羽和一个障碍物。

下一个代码单元设置一个简单区域，带有烟源和正上方的方形障碍物。

```
smoke = CenteredGrid(0, extrapolation.PERIODIC, x=RES_X, y=RES_Y,
                     bounds=Box(x=RES_X, y=RES_Y))
vel_smk = StaggeredGrid(0, extrapolation.PERIODIC, x=RES_X, y=RES_Y,
```

[3] 输出标签称 mean divergence，但代码实际打印 `loss_div` 的损失均值；其具体归一化仍需核查。

[4] 常数偏移不会改变压力梯度；若场之间还存在尺度或非恒定差异，其影响不能仅以常数规范自由度解释。

[5] 译注：源正文称方形障碍物，但代码 `Box(x=(12,20),y=(18,20))` 是 8×2 矩形。`traj` 先存初态再追加 50 步，因此 `traj[STEPS-1]` 实际显示第 49 步，不是末索引 50；原图与代码保持不变。

```
                        bounds=Box(x=RES_X, y=RES_Y))
RECTANGLE = Obstacle(Box(x=(12, 20), y=(18, 20)))
INFLOW_LOCATION = tensor((16, 5), channel(vector='x,y'))
INFLOW = 0.6 * CenteredGrid(Sphere(center=INFLOW_LOCATION, radius=3),
    extrapolation.PERIODIC, x=RES_X, y=RES_Y, bounds=Box(x=RES_X, y=RES_Y))
CYLINDER = Obstacle(geom.infinite_cylinder(x=16, y=20, radius=3, inf_dim=None))
```

在神经网络支持的求解器中，对障碍物施加零速度边界条件有多种选择。最稳妥的方法是在调用基于网络的压力修正步之前和之后都设置速度。前者确保网络看到固体障碍物，后者确保即使网络在障碍边界上有轻微误差，边界条件仍得到满足。因此，下面的 step_nn() 函数将调用 PhiFlow 的 apply_boundary_conditions() 两次。

同时，我们还定义 step_obs() 函数，运行不使用神经网络的经典流体求解器以供比较。

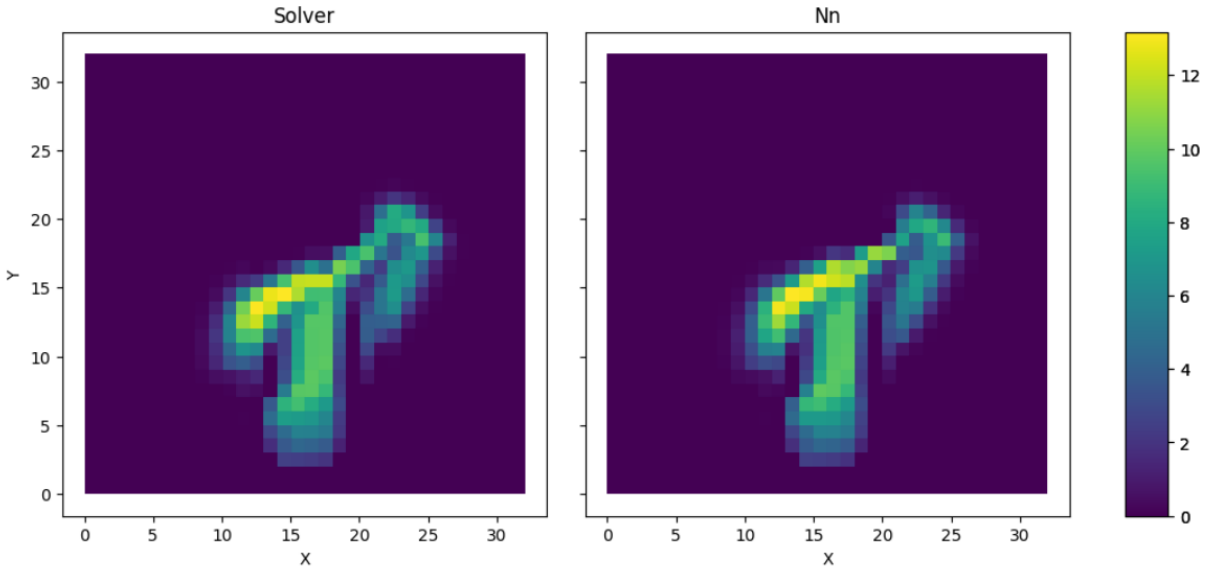
```
dt = 0.1
@jit_compile
def step_nn(v, dt, f, obstacle):
    v = v + f * dt
    v = advect.mac_cormack(v, v, dt)
    v = fluid.apply_boundary_conditions(v, obstacle)
    nn_input = field.divergence(field=v, order=2)
    p = math.native_call(network, nn_input.values)
    p = field.CenteredGrid(values=p,
        extrapolation=math.extrapolation.PERIODIC,
        bounds=geom.Box(x=RES_X, y=RES_Y),
        resolution=spatial(x=RES_X, y=RES_Y),
    )
    grad_pres = field.spatial_gradient(p, at=v.sampled_at)
    v = v - grad_pres
    v = fluid.apply_boundary_conditions(v, obstacle)
    return v, p

@jit_compile
def step_obs(v, dt, f, obstacle):
    v = v + f * dt
    v = advect.mac_cormack(v, v, dt)
    v, p = fluid.make_incompressible(v, obstacle, solve=Solve(rank_deficiency=0))
    return v, p
```

现在可以运行并比较仿真。下面的代码运行 STEPS 个仿真步，依次运行两个求解器。平流输运的标记密度记录在 traj_smk_X 列表中，使我们能够直观、定性地检查结果。标记密度既驱动又跟随流动速度，因此很好地突显了模拟流体运动中的变化。

```
STEPS = 50
smk_solv = smoke
smk_nn = smoke
vel_smk_solv = vel_smk
vel_smk_nn = vel_smk
traj_smk_solv = [smk_solv]
```

```
traj_smk_nn = [smk_nn]

# flexible backends require a small workaround for PyTorch's no_grad context;
# by default (eg for TF) use a null-context
from contextlib import nullcontext
context = nullcontext()
if any([b.name == 'torch' for b in BACKENDS]):
    context = torch.no_grad()

with context:
    for i in tqdm(range(STEPS)):
        smk_solv = advect.mac_cormack(smk_solv, vel_smk_solv, dt=dt) + INFLOW
        smk_nn = advect.mac_cormack(smk_nn, vel_smk_nn, dt=dt) + INFLOW
        buoyancy_force_solv = smk_solv * (0, 1.0) @ vel_smk_solv
        buoyancy_force_nn = smk_nn * (0, 1.0) @ vel_smk_nn
        vel_smk_solv, _ = step_obs(vel_smk_solv, dt, buoyancy_force_solv, [RECTANGLE])
        vel_smk_nn, _ = step_nn(vel_smk_nn, dt, buoyancy_force_nn, [RECTANGLE])
        traj_smk_solv.append(smk_solv)
        traj_smk_nn.append(smk_nn)

traj_smk_solv = field.stack(traj_smk_solv, batch('time'))
traj_smk_nn = field.stack(traj_smk_nn, batch('time'))
plot = vis.plot({"Solver": traj_smk_solv.time[STEPS-1],
                 "NN": traj_smk_nn.time[STEPS-1]}) # show last frame

100%|##########| 50/50 [01:37<00:00, 1.96s/it]
```

进度条中的 # 表示原文实心块。



图中左侧为求解器、右侧为神经网络。上图显示仿真的最后一帧，两者惊人地相似。原文认为这一结果说明，尽管训练时只见过合成噪声输入，网络仍成功学会减小任意流动的散度。

此外，这突出了训练所用可微离散算子的成功：训练收敛很快，训练后的网络以零样本方式泛化到全新输入。训练时网络甚至没有见过任何障碍物，却仍能推断出处理障碍物的（大体正确的）压力场。不过，这仍是该实现中值得改进的方面。可以将障碍物几何作为额外输入提供给网络，也可以通过提高边界区域的损失值，训练网络特别关注该区域。[6]

最后，如果有兴趣观看模拟轨迹的完整演化，可以取消下面代码单元的注释并运行。它通过 PhiFlow 的 `plot(...,animate=X)` 函数生成可在浏览器中观看的视频。

```
#vis.plot({"solver": traj_smk_solv, "nn": traj_smk_nn}, animate='time')
# uncomment to view animation
```

[6] 上述“任意流动”和零样本泛化是原文对所示示例的概括；这张有限测试图本身不能证明对任意流场均成立。

第十章　下一步

- 改进神经网络架构以提升性能。深度和每层特征数量是显然可以改进的方面，其他架构也同样值得尝试。
- 特别值得尝试上面的 res_net 等全卷积架构：评估以这些架构训练的神经网络能够在多大程度上处理新的、更大的输入网格。
- 通过增加一个输入并更新训练数据集，加入对障碍物的支持。

第十一章　使用 PINN 优化 Burgers 方程

为了说明变体 2 的物理信息损失如何工作，考虑一个作为逆问题例子的重建任务。我们使用一维 Burgers 方程 $\partial u/\partial t + u\nabla u = \nu\nabla\cdot\nabla u$ 作为简单但非线性的方程，并在时间 $t = 0.5$ 处获得一系列观测。解应满足 Burgers 方程的残差形式并与观测吻合。此外，在计算区域两侧施加 Dirichlet 边界条件 $u = 0$，并将解定义在时间区间 $t \in [0, 1]$。

注意，与前面的正向仿真示例类似，我们仍以 128 个点（$n = 128$）采样解，但现在通过神经网络进行离散化。因此也可以在点之间采样，而无须显式选择插值基函数。通过神经网络的离散化现在在内部决定如何使用其自由度，将激活函数安排为基函数。因此我们无法直接控制重建。原文提供的 Colab 链接待核实。

11.1　问题表述

按照“模型与方程”及前一节中的 x, y^* 记号，这一重建问题意味着求解

$$\arg\min_{\theta} \sum_i \big(f(x_i;\theta) - y_i^*\big)^2 + R(x_i),$$

其中 x_i 现在表示时空点 $x_i = [p_i, t_i]$，参考解为 $y_i^* = y^*(x_i)$，下标 i 表示数据集中的不同采样点。f 和 y^* 都表示 u 在不同时空位置的解；由于处理的是一维速度，$f, y^*\colon \mathbb{R}^2 \to \mathbb{R}$。在本例中，$y^*$ 表示 $\mathcal{P}$ 为 Burgers 方程时的参考 u，f 应在所有选定时空点 $x_i = [p_i, t_i]$ 处尽可能准确地近似它。

上面的第一项是“监督”数据项，第二项则表示残差函数 R。它汇集对 $f(;\theta)$ 及其导数的额外求值，以构造 $\mathcal{P}$ 的残差。这种使用神经网络导数计算 PDE 残差的方法通常称为物理信息方法，由此得到物理信息神经网络（PINN）[RPK19] 来表示逆重建问题的解。

因此在上述表述中，R 应收敛到零。为简单起见，目标函数中省略了缩放因子。注意这里实际上只处理 $\mathcal{P}$ 的单个解 u 的各个点样本。[1]

11.2　预备工作

这个 notebook 比较旧，因此需要较旧版本的 TensorFlow。下一个代码单元将 TensorFlow 安装或降级至兼容版本。由于 pip 依赖关系，这可能在 Colab 上产生“错误”，原文称可以安全忽略：

```
!pip3 install --upgrade --quiet tensorflow==2.15.0 tensorflow-probability==0.23.0
```

接下来加载 phiflow（使用来自自定义仓库的旧版 1.5.1），以 TensorFlow 后端使用它并初始化随机采样。

[1] 原文目标函数直接加带符号的 R；一般不能据此保证残差趋零，参见第 8 章问题记录，具体实现需与后文代码核对。

```
!pip install --upgrade --quiet git+https://github.com/thunil/PhiFlow.git
from phi.tf.flow import *
import numpy as np
#rnd = TF_BACKEND # for phiflow: sample different points in the domain each iteration
rnd = math.choose_backend(1) # use same random points for all iterations
```

这里虽然导入了 phiflow，但不会像 “使用 phiflow 对 Burgers 方程进行简单正向仿真” 一节那样用它计算 PDE 的解；相反，将如上一节所述，使用神经网络的导数构造训练损失。

接下来设置一个简单神经网络，具有 8 个全连接层，每层 20 个单元，使用 tanh 激活。还将定义 boundary_tx 函数，给出解的约束数组（本例全部位于 $t = 0.5$），以及 open_boundary 函数，保存 $x = \pm 1$ 处值为 0 的约束。

```
def network(x, t):
    """ Dense neural network with 8 hidden layers and 3021 parameters in total.
        Parameters will only be allocated once (auto reuse).
    """
    y = math.stack([x, t], axis=-1)
    for i in range(8):
        y = tf.layers.dense(y, 20, activation=tf.math.tanh, name='layer%d' % i,
                            reuse=tf.AUTO_REUSE)
    return tf.layers.dense(y, 1, activation=None, name='layer_out', reuse=tf.AUTO_REUSE)

def boundary_tx(N):
    x = np.linspace(-1,1,128)
    # precomputed solution from forward simulation:
    u = np.asarray([
        0.008612174447657694, 0.02584669669548606, 0.043136357266407785, 0.060491074685516746,
        0.07793926183951633, 0.0954779141740818, 0.11311894389663882, 0.1308497114054023,
        0.14867023658641343, 0.1665634396808965, 0.18452263429574314, 0.20253084411376132,
        0.22057828799835133, 0.23865132431365316, 0.25673879161339097, 0.27483167307082423,
        0.2929182325574904, 0.3109944766354339, 0.3290477753208284, 0.34707880794585116,
        0.36507311960102307, 0.38303584302507954, 0.40094962955534186, 0.4188235294008765,
        0.4366357052408043, 0.45439856841363885, 0.4720845505219581, 0.4897081943759776,
        0.5072391070000235, 0.5247011051514834, 0.542067187709797, 0.5593576751669057,
        0.5765465453632126, 0.5936507311857876, 0.6106452944663003, 0.6275435911624945,
        0.6443221318186165, 0.6609900633731869, 0.67752574922899, 0.6939334022562877,
        0.7101938106059631, 0.7263049537163667, 0.7422506131457406, 0.7580207366534812,
        0.7736033721649875, 0.7889776974379873, 0.8041371279965555, 0.8190465276590387,
        0.8337064887158392, 0.8480617965162781, 0.8621229412131242, 0.8758057344502199,
        0.8891341984763013, 0.9019806505391214, 0.9143881632159129, 0.9261597966464793,
        0.9373647624856912, 0.9476871303793314, 0.9572273019669029, 0.9654367940878237,
        0.9724097482283165, 0.9767381835635638, 0.9669484658390122, 0.659083299684951,
        -0.659083180712816, -0.9669485121167052, -0.9767382069792288, -0.9724097635533602,
        -0.9654367970450167, -0.9572273263645859, -0.9476871280825523, -0.9373647681120841,
        -0.9261598056102645, -0.9143881718456056, -0.9019807055316369, -0.8891341634240081,
        -0.8758057205293912, -0.8621229450911845, -0.8480618138204272, -0.833706571569058,
        -0.8190466131476127, -0.8041372124868691, -0.7889777195422356, -0.7736033858767385,
```

```
        -0.758020740007683, -0.7422507481169578, -0.7263049162371344, -0.7101938950789042,
        -0.6939334061553678, -0.677525822052029, -0.6609901538934517, -0.6443222327338847,
        -0.6275436932970322, -0.6106454472814152, -0.5936507836778451, -0.5765466491708988,
        -0.5593578078967361, -0.5420672759411125, -0.5247011730988912, -0.5072391580614087,
        -0.4897082914472909, -0.47208460952428394, -0.4543985995006753, -0.4366355580500639,
        -0.41882350871539187, -0.40094955631843376, -0.38303594105786365, -0.36507302109186685,
        -0.3470786936847069, -0.3290476440540586, -0.31099441589505206, -0.2929180880304103,
        -0.27483158663081614, -0.2567388003912687, -0.2386513127155433, -0.22057831776499126,
        -0.20253089403524566, -0.18452269630486776, -0.1665634500729787, -0.14867027528284874,
        -0.13084990929476334, -0.1131191325854089, -0.09547794429803691, -0.07793928430794522,
        -0.06049114408297565, -0.0431364527809777, -0.025846763281087953, -0.00861212501518312
    ])
    t = np.asarray(rnd.ones_like(x)) * 0.5
    perm = np.random.permutation(128)
    return (x[perm])[0:N], (t[perm])[0:N], (u[perm])[0:N]

def _ALT_t0(N): # alternative, impose original initial state at t=0
    x = rnd.random_uniform([N], -1, 1)
    t = rnd.zeros_like(x)
    u = - math.sin(np.pi * x)
    return x, t, u

def open_boundary(N):
    t = rnd.random_uniform([N], 0, 1)
    x = math.concat([math.zeros([N//2]) + 1, math.zeros([N//2]) - 1], axis=0)
    u = math.zeros([N])
    return x, t, u
```

最重要的是，现在还可以构造希望最小化的残差损失函数 f，以引导神经网络恢复模型方程的解。如顶部的方程所示，我们需要对 t、x 的导数及对 x 的二阶导数。下面 f 的前三行正是完成这些计算。

随后只需组合这些导数，构成 Burgers 方程。这里使用 phiflow 的 gradient 函数：

```
def f(u, x, t):
    """ Physics-based loss function with Burgers equation """
    u_t = gradients(u, t)
    u_x = gradients(u, x)
    u_xx = gradients(u_x, x)
    return u_t + u*u_x - (0.01 / np.pi) * u_xx
```

接下来设置内部区域的采样点，以便将解与前面的 phiflow 正向仿真比较。下一个代码单元分配两个张量：grid_x 用 128 个单元覆盖区域，即 -1 到 1；grid_t 用 33 个时间点采样时间区间 $[0, 1]$。最后的 math.expand_dims() 调用添加另一个批次维度，使所得张量与后续示例兼容。

```
N=128
grids_xt = np.meshgrid(np.linspace(-1, 1, N), np.linspace(0, 1, 33), indexing='ij')
grid_x, grid_t = [tf.convert_to_tensor(t, tf.float32) for t in grids_xt]
# create 4D tensor with batch and channel dimensions in addition to space and time
# in this case gives shape=(1, N, 33, 1)
```

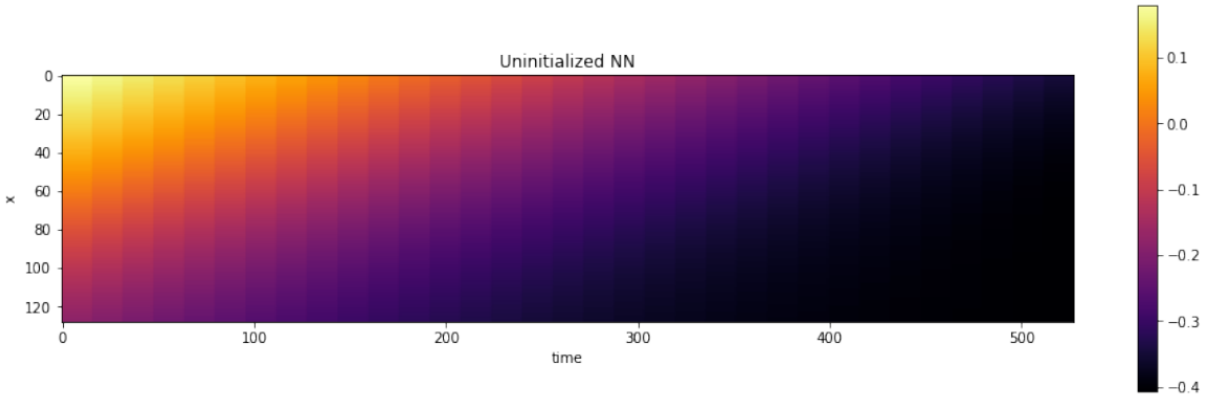
```
grid_u = math.expand_dims(network(grid_x, grid_t))
```

现在 grid_u 包含了在 128×33 个位置评估神经网络的完整计算图，通过 session.run 运行后，以 $[1, 128, 33, 1]$ 数组返回结果。来试试看：可以初始化一个 TensorFlow 会话，计算 grid_u 并将其显示为图像，正如之前计算的 phiflow 解一样。

注意，我们将使用 “使用 phiflow 对 Burgers 方程进行简单正向仿真” 中的 show_state。因此，横轴不显示实际仿真时间，而显示放大 16 倍后的 32 个时间步，使时间变化在图中更容易看清。

```
import pylab as plt
print("Size of grid_u: "+format(grid_u.shape))

session = Session(None)
session.initialize_variables()

def show_state(a, title):
    for i in range(4): a = np.concatenate( [a,a] , axis=3)
    a = np.reshape( a, [a.shape[1],a.shape[2]*a.shape[3]] )
    fig, axes = plt.subplots(1, 1, figsize=(16, 5))
    im = axes.imshow(a, origin='upper', cmap='inferno')
    plt.colorbar(im) ; plt.xlabel('time'); plt.ylabel('x'); plt.title(title)

print("Randomly initialized network state:")
show_state(session.run(grid_u),"Uninitialized NN")
```

```
Size of grid_u: (1, 128, 33, 1)
Randomly initialized network state:
```



图中显示未初始化训练的网络状态。可视化已经呈现空间和时间上的平滑变化，但目前这纯粹是所采样的随机神经网络初始化，与基于 PDE 的模型解尚无关系。

接下来的步骤将实际评估数据约束（来自 boundary 函数）及 f 给出的模型约束，以获得 PDE 的实际解。

11.3 损失函数与训练

现在可以将直接约束——即 $t = 0.5$ 时的解和 Dirichlet 边界条件 $u = 0$——与 PDE 残差损失结合，作为学习过程的目标。下面对两种边界约束各使用 100 个点，再以额外 1000 个点采样内部区域的解。

直接约束通过 `network(x, t)[:, 0] - u` 计算，其中 x、t 是我们希望采样解的时空位置，u 提供对应真实值。对物理损失点，我们没有真实解，只通过网络导数计算 PDE 残差，以检查解是否满足 PDE 模型；如果不满足，这直接给出可通过优化更新步减小的误差。下面相应表达式为 `f(network(x, t)[:, 0], x, t)`。注意，对于数据项和物理项，`network()[:, 0]` 表达式并不会从 L^2 求值中移除任何数据，只是舍弃网络返回的 $(n, 1)$ 张量最后一个大小为 1 的维度。

```
# Boundary loss
N_SAMPLE_POINTS_BND = 100
x_bc, t_bc, u_bc = [math.concat([v_t0, v_x], axis=0) for v_t0, v_x in
    zip(boundary_tx(N_SAMPLE_POINTS_BND), open_boundary(N_SAMPLE_POINTS_BND))]
x_bc, t_bc, u_bc = np.asarray(x_bc,dtype=np.float32), np.asarray(t_bc,dtype=np.float32),\
                   np.asarray(u_bc,dtype=np.float32)
#with app.model_scope():
loss_u = math.l2_loss(network(x_bc, t_bc)[:, 0] - u_bc) # normalizes by first dimension, N_bc

# Physics loss inside of domain
N_SAMPLE_POINTS_INNER = 1000
x_ph, t_ph = tf.convert_to_tensor(rnd.random_uniform([N_SAMPLE_POINTS_INNER], -1, 1)),\
             tf.convert_to_tensor(rnd.random_uniform([N_SAMPLE_POINTS_INNER], 0, 1))
loss_ph = math.l2_loss(f(network(x_ph, t_ph)[:, 0], x_ph, t_ph))
# normalizes by first dimension, N_ph

# Combine
ph_factor = 1.
loss = loss_u + ph_factor * loss_ph # allows us to control the relative influence of loss_ph
optim = tf.train.GradientDescentOptimizer(learning_rate=0.02).minimize(loss)
#optim = tf.train.AdamOptimizer(learning_rate=0.001).minimize(loss)
# alternative, but not much benefit here
```

上面的代码只是初始化损失求值，还没有进行任何优化步骤；但现在终于可以开始了。尽管方程简单，收敛通常非常缓慢。每次迭代计算很快，但此设置需要大量迭代。为了使运行时间保持在合理范围，下面默认只运行 1 万次迭代（ITERS）。可以增加此值以获得更好的结果。

```
session.initialize_variables()
import time
start = time.time()
ITERS = 10000
for optim_step in range(ITERS+1):
    _, loss_value = session.run([optim, loss])
    if optim_step<3 or optim_step%1000==0:
        print('Step %d, loss: %f' % (optim_step,loss_value))
        #show_state(grid_u)
end = time.time()
print("Runtime {:.2f}s".format(end-start))
```

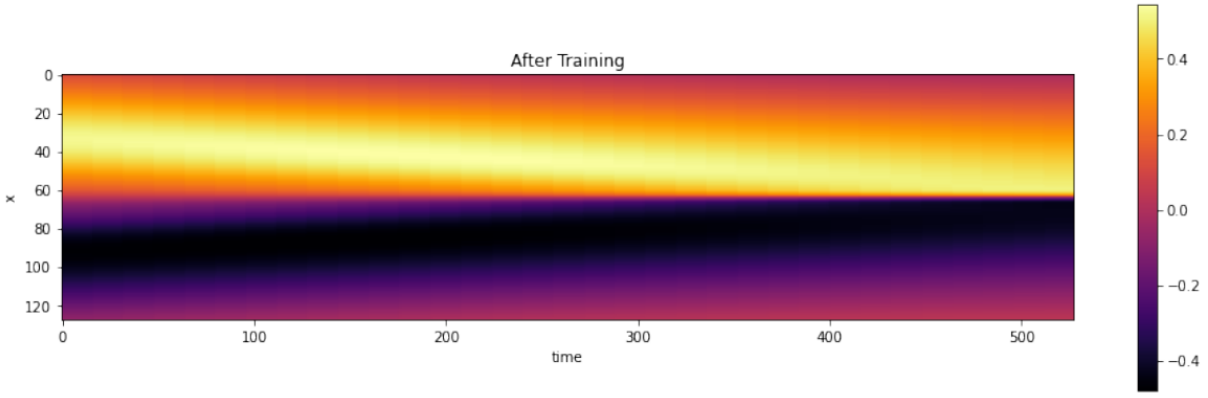
```
Step 0, loss: 0.100037
Step 1, loss: 0.097836
Step 2, loss: 0.096151
Step 1000, loss: 0.054487
Step 2000, loss: 0.049147
Step 3000, loss: 0.045515
Step 4000, loss: 0.042890
Step 5000, loss: 0.039945
Step 6000, loss: 0.037641
Step 7000, loss: 0.035227
Step 8000, loss: 0.033604
Step 9000, loss: 0.031556
Step 10000, loss: 0.029434
Runtime 101.02s
```

由于训练使用成熟的构件（稠密层），在典型笔记本电脑上应只需约两分钟，误差应显著下降（约从 0.2 降至 0.03），网络似乎成功地向一个解收敛。[2]

通过在规则网格中心评估网络，来展示网络的重建结果，从而以图像显示解。注意这实际上相当昂贵，因为必须对网格全部 128×32 个采样点运行包含数千个权重的完整网络。[3]

乍看之下结果相当不错：与上面显示的随机初始化相比发生了明显变化：

```
show_state(session.run(grid_u),"After Training")
```



图中显示训练后的状态。

11.4 评估

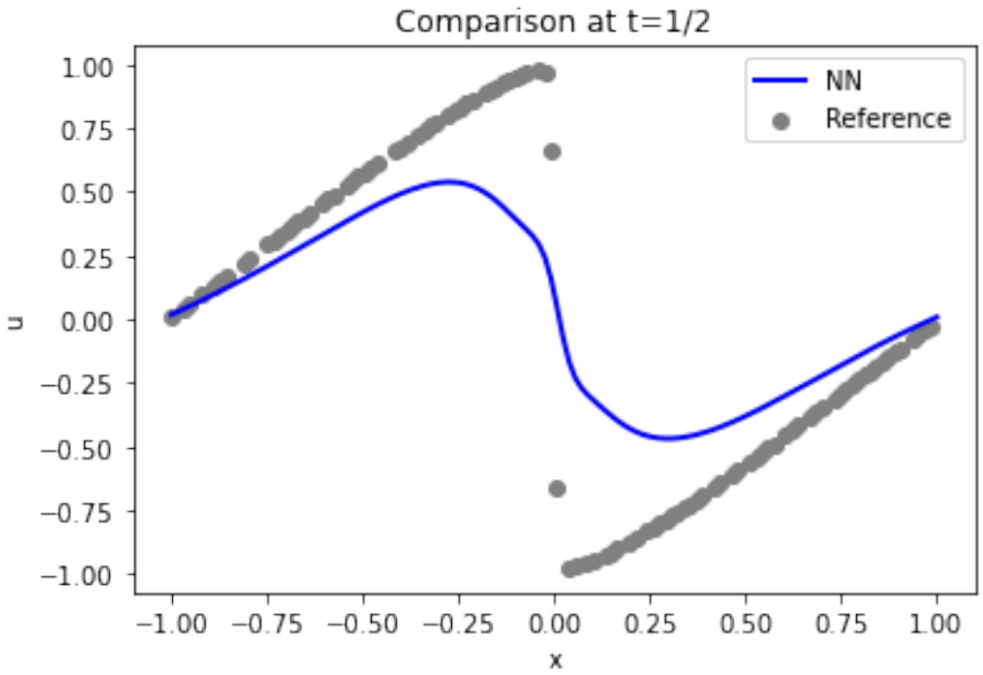
[4] 更详细地比较解。下面以灰色显示用于约束解的实际采样点（时间步 16，即 $t = 1/2$），与蓝色的重建解对比：

[2] 原文给出的示例输出从约 0.100 开始，文字则说约 0.2；此外代码的 `range(ITERS+1)` 执行 10001 次循环。

[3] 前文网格代码为 128×33 个时空采样点，原文此处写 128×32，照录。

[4] 译注：本章 PINN 残差及重模拟采用 $\nu = .01/\pi$，第 3 章冻结代码却为 $.01/(N\pi)$，$N = 128$ 时相差 128 倍。这里含端点的 linspace 网格与第 3 章单元中心不同；重模拟还用 PERIODIC，而 PINN 声明 Dirichlet，不能把跨方法结果视为同 PDE、网格和边界的严格对齐比较。参考数组在端点 $x = -1, 1$ 给出约 $.008612, -.008612$，与同点零边界不一致；随机 100 点是否抽到端点依实际采样。保留源数组和代码，不运行。

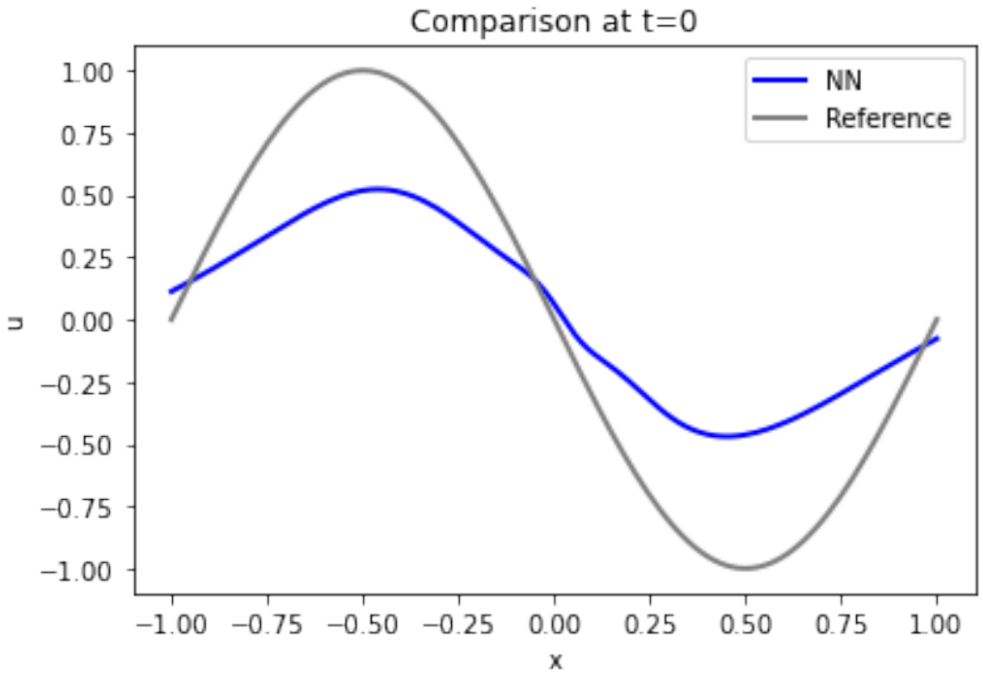
```
u = session.run(grid_u)
# solution is imposed at t=1/2 , which is 16 in the array
BC_TX = 16
uT = u[0,:,BC_TX,0]
fig = plt.figure().gca()
fig.plot(np.linspace(-1,1,len(uT)), uT, lw=2, color='blue', label="NN")
fig.scatter(x_bc[0:100], u_bc[0:100], color='gray', label="Reference")
plt.title("Comparison at t=1/2")
plt.xlabel('x'); plt.ylabel('u'); plt.legend()
<matplotlib.legend.Legend at 0x7f8eca2a7a00>
```



在区域两侧结果还不错（满足 Dirichlet 边界条件 $u = 0$），但中心 $x = 0$ 处的激波表示得不好。

检查 $t = 0$ 的初态重建得如何。这是问题中最有趣也最困难的部分：给定初始状态后，其余部分基本由模型方程和边界条件确定。结果发现初态精度实际上不高：PINN 的蓝色曲线距离参考数据给出的约束（灰色）很远。迭代次数增加后解会变好，但对这个相当简单的情形而言，所需迭代次数多得惊人。

```
# ground truth solution at t0
t0gt = np.asarray( [ [-math.sin(np.pi * x) * 1.] for x in np.linspace(-1,1,N)] )
velP0 = u[0,:,0,0]
fig = plt.figure().gca()
fig.plot(np.linspace(-1,1,len(velP0)), velP0, lw=2, color='blue', label="NN")
fig.plot(np.linspace(-1,1,len(t0gt)), t0gt, lw=2, color='gray', label="Reference")
plt.title("Comparison at t=0")
plt.xlabel('x'); plt.ylabel('u'); plt.legend()
<matplotlib.legend.Legend at 0x7f8eca1b48e0>
```



尤其 $x = \pm 1/2$ 处的最大值 / 最小值偏差很大，且 $x = \pm 1$ 的边界条件不满足：解不为零。

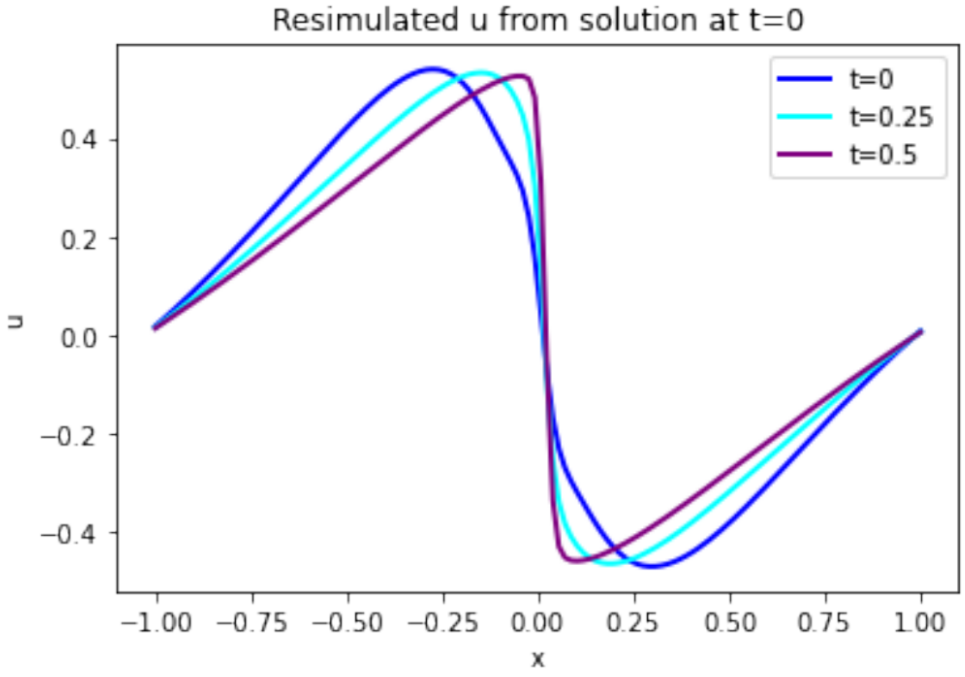
我们有此仿真的正向模拟器，因此可以使用网络的 $t = 0$ 解来评估时间演化重建得如何。这衡量了 PINN 损失的软约束在多大程度上捕获了模型方程的时间演化。下图以蓝色显示初态，以及 $t = 8/32$ 和 $t = 15/32$ 的两个演化状态。注意这些全部来自仿真版本；接下来才显示学习得到的版本。[5]

注意，下列代码段还有一些可选代码来显示 `[STEPS//4]` 处的状态。它们默认被注释掉；如果需要，可以取消注释或添加其他状态来可视化更多时间演化。

```
# re-simulate with phiflow from solution at t=0
DT = 1./32.
STEPS = 32-BC_TX # depends on where BCs were imposed
INITIAL = u[...,BC_TX:(BC_TX+1),0] # np.reshape(u0, [1,len(u0),1])
print(INITIAL.shape)
DOMAIN = Domain([N], boundaries=PERIODIC, box=box[-1:1])
state = [BurgersVelocity(DOMAIN, velocity=INITIAL, viscosity=0.01/np.pi)]
physics = Burgers()
for i in range(STEPS):
    state.append( physics.step(state[-1],dt=DT) )
# we only need "velocity.data" from each phiflow state
vel_resim = [x.velocity.data for x in state]
fig = plt.figure().gca()
pltx = np.linspace(-1,1,len(vel_resim[0].flatten()))
fig.plot(pltx, vel_resim[ 0].flatten(), lw=2, color='blue', label="t=0")
#fig.plot(pltx, vel_resim[STEPS//4].flatten(), lw=2, color='green', label="t=0.125")
fig.plot(pltx, vel_resim[STEPS//2].flatten(), lw=2, color='cyan', label="t=0.25")
```

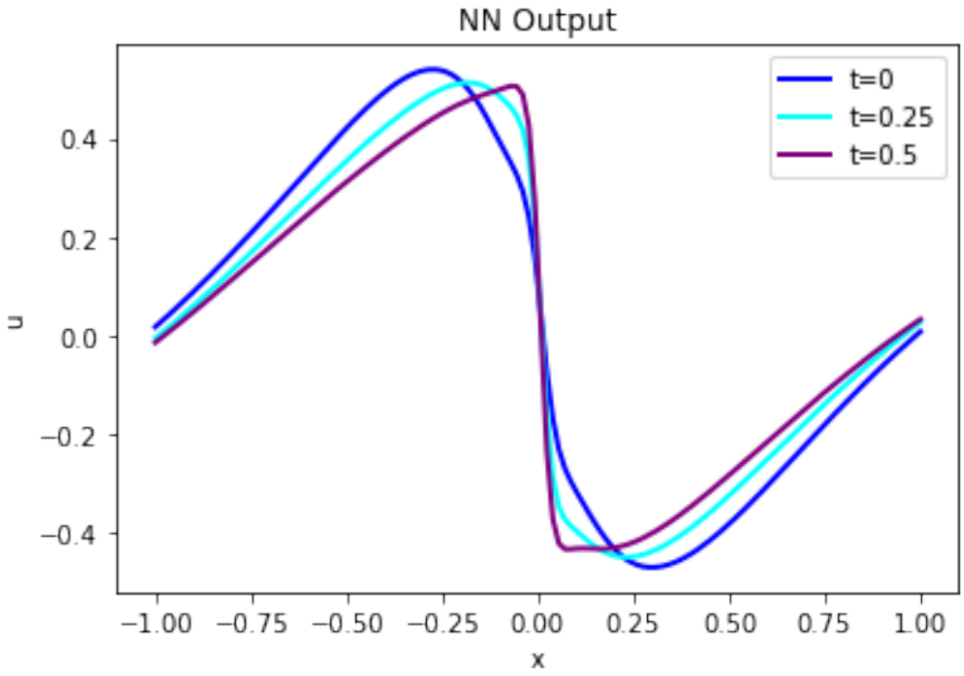
[5] 原文此处称使用 $t = 0$ 的网络解，而下面代码以 `BC_TX=16` 取 $t = 0.5$ 的切片作为重模拟初态；图内时间标签也需结合相对时间理解，照录并留待核查。

```
fig.plot(pltx, vel_resim[STEPS-1].flatten(), lw=2, color='purple',label="t=0.5")
#fig.plot(pltx, t0gt, lw=2, color='gray', label="t=0 Reference") # optionally show GT,
# compare to blue
plt.title("Resimulated u from solution at t=0")
plt.xlabel('x'); plt.ylabel('u'); plt.legend()
(1, 128, 1)
<matplotlib.legend.Legend at 0x7f8eca556820>
```



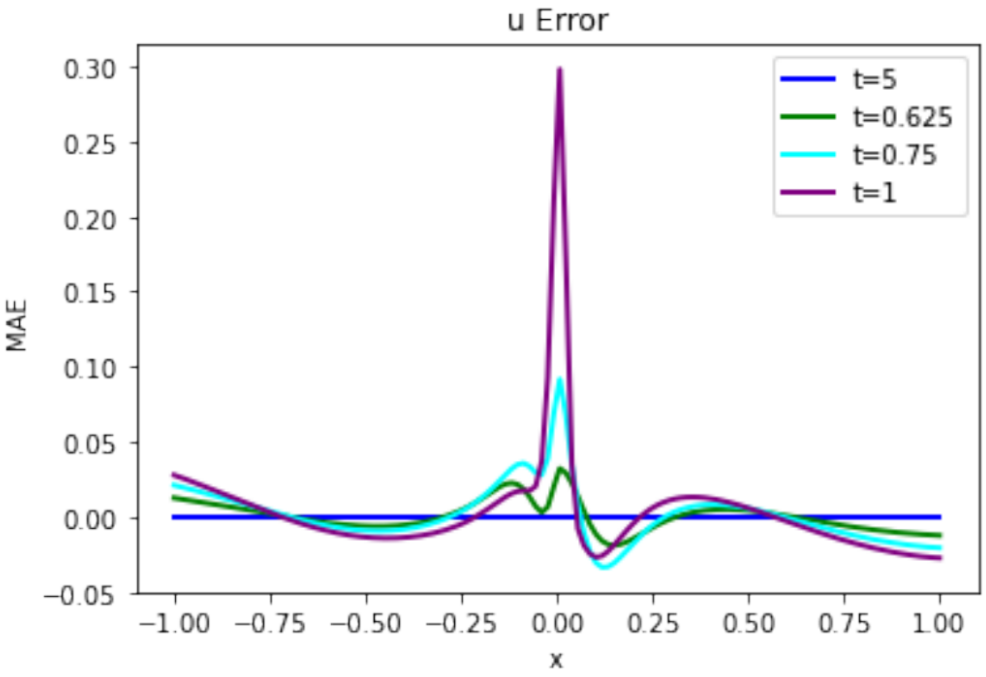
下面是在相同时间步的 PINN 输出 u:

```
velP = [u[0,:,x,0] for x in range(33)]
print(velP[0].shape)
fig = plt.figure().gca()
fig.plot(pltx, velP[BC_TX+ 0].flatten(), lw=2, color='blue', label="t=0")
#fig.plot(pltx, velP[BC_TX+STEPS//4].flatten(), lw=2, color='green', label="t=0.125")
fig.plot(pltx, velP[BC_TX+STEPS//2].flatten(), lw=2, color='cyan', label="t=0.25")
fig.plot(pltx, velP[BC_TX+STEPS-1].flatten(), lw=2, color='purple',label="t=0.5")
plt.title("NN Output")
plt.xlabel('x'); plt.ylabel('u'); plt.legend()
(128,)
<matplotlib.legend.Legend at 0x7f8ec93a4310>
```



凭目测，这两个版本的 u 看起来相当相似，但不出所料，误差随时间增长，并存在显著差异。特别是 $x = 0$ 激波附近解变陡的过程没有被很好地 “捕获”。不过，从这两张图中有些难以看出；下面量化误差并显示实际差值：

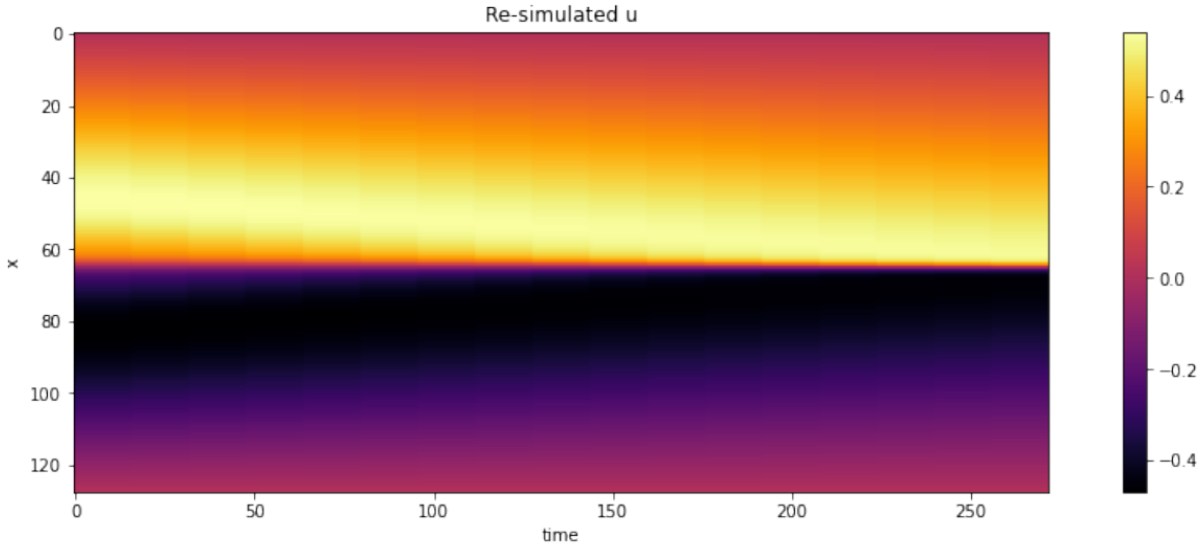
```
error = np.sum(np.abs(np.asarray(vel_resim[0:16]).flatten() -
    np.asarray(velP[BC_TX:BC_TX+STEPS]).flatten())) / (STEPS*N)
print("Mean absolute error for re-simulation across {} steps: {:7.5f}".format(
    STEPS, error))
fig = plt.figure().gca()
fig.plot(pltx, (vel_resim[0].flatten()-velP[BC_TX].flatten()),
         lw=2, color='blue', label="t=5")
fig.plot(pltx, (vel_resim[STEPS//4].flatten()-velP[BC_TX+STEPS//4].flatten()),
         lw=2, color='green', label="t=0.625")
fig.plot(pltx, (vel_resim[STEPS//2].flatten()-velP[BC_TX+STEPS//2].flatten()),
         lw=2, color='cyan', label="t=0.75")
fig.plot(pltx, (vel_resim[STEPS-1].flatten()-velP[BC_TX+STEPS-1].flatten()),
         lw=2, color='purple',label="t=1")
plt.title("u Error")
plt.xlabel('x'); plt.ylabel('MAE'); plt.legend()

Mean absolute error for re-simulation across 16 steps: 0.01136
<matplotlib.legend.Legend at 0x7f8eaa76ed90>
```



图中是重模拟与 PINN 在各空间位置的差。上面的代码计算得到参考重模拟与 PINN 演化之间约 1.5×10^{-2} 的平均绝对误差；对本仿真的数值范围而言，这一误差相当显著。[6]

为了与正向仿真及后面的情形比较，再用色图显示所有时间步：

```
# show re-simulated solution again as full image over time
sn = np.concatenate(vel_resim, axis=-1)
sn = np.reshape(sn, list(sn.shape)+[1] ) # print(sn.shape)
show_state(sn,"Re-simulated u")
```



[6]原文输出为 0.01136，文字则称约 0.015；图例蓝线写作 `t=5`，应结合代码时间索引核查。图中绘制的是带符号差值，纵轴却标为 MAE。

接下来保存 $t = 0 \ldots 1$ 时间区间内的完整解，以便稍后与普通正向求解得到的完整解及可微物理解比较。完整评估和比较将在讨论如何运行可微物理优化之后，于“使用可微物理梯度优化 Burgers 方程”中进行。

```
vels = session.run(grid_u) # special for showing NN results, run through TF
vels = np.reshape( vels, [vels.shape[1],vels.shape[2]] )
# save for comparison with other methods
import os; os.makedirs("./temp",exist_ok=True)
np.savez_compressed("./temp/burgers-pinn-solution.npz",vels)
print("Vels array shape: "+format(vels.shape))
```

```
Vels array shape: (128, 33)
```

11.5 下一步

当然，这一设置只是 PINN 和物理软约束的起点。参数的选择旨在使运行相对快速。正如后续章节将展示的，通过更紧密地集成求解器与学习，可以大幅改善这类逆求解的表现。

不过，上面 PINN 设置的解也可以直接改进。例如尝试：

- 调整训练参数，在不使解发散的情况下进一步降低误差；
- 调整神经网络架构以进一步改进，但要关注权重数量；
- 启用不同优化器并观察行为变化，通常需要调整学习率。注意，在这个相对简单的例子中，更复杂的优化器不一定表现更好；
- 或者修改设置，使测试更有意思：例如将边界条件移至仿真时间中更晚的位置，使需要重建的时间区间更长。

第十二章　物理损失讨论

目前的好消息是，我们有一种深度学习方法，可以通过最小化残差，将物理定律以软约束形式纳入。然而，正如前面非常简单的例子所示，这带来了新的困难，也只是概念上的起点。

积极的一面是，我们可以利用具有反向传播功能的深度学习框架计算模型导数。同时，这使损失景观更复杂，而导数是否可靠取决于学习得到的表示。此外，每个导数都需要通过完整网络反向传播，尤其对于高阶导数可能非常昂贵。

虽然设置相对简单，但通常难以控制。神经网络有灵活性自行细化解；但与此同时，当它没有关注解的正确区域时，还需要一些技巧。

12.1　泛化？

前面 PINN 优化有一个需要注意的方面：测试和约束解的位置就是我们最终感兴趣的位置。因此从经典机器学习角度看，训练集、验证集和测试集之间没有真正区别。对已知给定的一组样本计算解，更类似于经典优化，前面的 Burgers 逆问题就源于这一领域。

在机器学习中，我们通常假设模型的最终性能将在另一组可能未知的输入上评估。测试数据通常应捕获此类分布外（OOD）行为，以便估计模型在应用部署后遇到“现实世界”情形时的泛化能力。使用预设离散化的 v1 版本确实具有这一性质，能够泛化到新输入。

相反，对于这里描述的 PINN 训练，我们在已知给定的时空区域重建单个解。因此，来自该区域的任何样本都服从相同分布，因而不真正代表测试样本或分布外样本。由于神经网络直接编码了解，也难以指望它产生不同的解，或在训练范围外表现良好。如果关心不同的解，就必须从头训练神经网络。[1]

12.2　总结

因此，物理软约束使我们能够用神经网络工具编码 PDE 的解。由于 PINN（v2）使用更广，这里重点讨论它们：一个固有缺点是通常只得到单个解或非常狭窄的解流形，且不容易与传统数值技术结合。与“监督训练”中的神经代理模型或神经算子相比，从某种意义上说我们后退了一步。

例如，学习得到的表示不适合使用共轭梯度法等经典迭代求解器进一步细化。这意味着过去几十年开发的许多强大技术无法在这一背景下使用。将这些数值方法重新引入将是后续章节的核心目标之一。

[1] 本节针对前述单解 PINN 设置；域内样本的训练 / 测试划分及泛化定义仍依采样方案和评估目标而定，原文概括不自动覆盖所有 PINN 变体。

优点

- 使用物理模型；
- 可以方便地通过反向传播计算导数。

缺点

- 收敛存在问题；
- 物理约束只作为软约束施加；
- 与经典数值方法大体不兼容；
- 导数的有效性依赖学习得到的表示。

为解决这些问题，接下来将讨论如何使用可微求解器，利用现有数值方法改进深度学习过程。

第四部分

可微物理

第十三章　可微物理简介

为了更紧密、更普遍地结合深度学习方法与物理仿真，下一步将把可微数值仿真纳入学习过程。下文将这些“物理系统的可微数值仿真”简称为“可微物理”（DP）。

这些方法的核心目标是利用现有数值求解器增强和改进 AI 系统。这需要为求解器提供计算相对于其输入的梯度的功能。一旦为仿真的所有算子实现这一功能，就可以利用深度学习框架的自动微分和反向传播，让梯度信息从模拟器流向神经网络，反之亦然。如下文所述，这带来诸多优点，例如改进学习反馈和泛化。

与上一章的物理信息损失函数不同，它还能处理更复杂的解流形，而不只是单个逆问题。例如，不是像上一章那样使用深度学习求解单个逆问题，而是可以利用可微物理训练神经网络，使之学会非常高效地求解更大类别的逆问题。

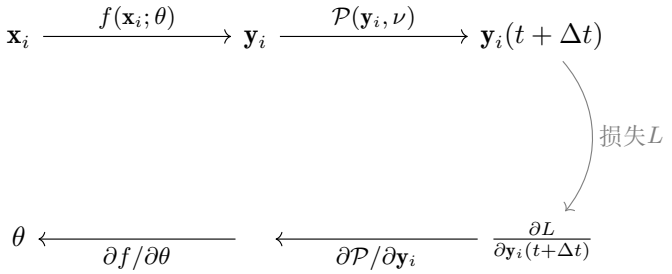


图 13.1: 使用可微物理进行训练，意味着一串可微算子以梯度形式提供方向来引导学习过程。图中 Loss L 为损失。

13.1　可微算子

可微物理以现有数值求解器为基础，也就是说，这一方法高度依赖计算方法领域为世界上各种物理效应开发的算法。首先，我们需要一个连续表述，作为希望仿真的物理效应的模型；如果缺少它就会遇到困难。幸运的是，可以利用现有模型方程集合及离散化连续模型的成熟方法。

假设我们对感兴趣的物理量 $\mathbf{u}(\mathbf{x},t)$: $\mathbb{R}^d \times \mathbb{R}^+ \to \mathbb{R}^d$ 有连续表述 $\mathcal{P}^*(\mathbf{x},\nu)$，模型参数为 ν（例如扩散、黏性或传导常数）。$\mathbf{u}$ 的分量以数字下标表示，即 $\mathbf{u} = (u_1, u_2, \ldots, u_d)^T$。

通常关注的是这种系统的时间演化。离散化得到表述 $\mathcal{P}(\mathbf{x},\nu)$，重新排列后可计算经过时间步 Δt 的未来状态。$t+\Delta t$ 时的状态通过一系列操作 $\mathcal{P}_1, \mathcal{P}_2, \ldots, \mathcal{P}_m$ 计算，使得 $\mathbf{u}(t+\Delta t) = \mathcal{P}_m \circ \cdots \mathcal{P}_2 \circ \mathcal{P}_1(\mathbf{u}(t),\nu)$，其中 $\circ$ 表示函数复合，即 $f(g(x)) = f \circ g(x)$。

注 为了将这一求解器纳入深度学习过程，必须确保每个算子 $\mathcal{P}_i$ 都提供相对于输入的梯度，即上例中的 $\partial\mathcal{P}_i/\partial\mathbf{u}$。

通常不需要 $\mathcal{P}(\mathbf{x},\nu)$ 所有参数的导数。例如，下文略去 ν，假设它是神经网络不应与之交互的给定模型参数。当然，它可以在感兴趣的解流形内变化，但 ν 不会是神经网络表示的输出。此时求解器中可以不提供 $\partial\mathcal{P}_i/\partial\nu$；不过，下面的学习过程自然也可推广到将 ν 作为自由度的情形。

13.2 Jacobian 矩阵

由于 $\mathbf{u}$ 通常是向量值函数，$\partial\mathcal{P}_i/\partial\mathbf{u}$ 表示 Jacobian 矩阵 J，而非单个数值：

$$\frac{\partial\mathcal{P}_i}{\partial\mathbf{u}} = \begin{bmatrix} \partial\mathcal{P}_{i,1}/\partial u_1 & \cdots & \partial\mathcal{P}_{i,1}/\partial u_d \\ \vdots & & \vdots \\ \partial\mathcal{P}_{i,d}/\partial u_1 & \cdots & \partial\mathcal{P}_{i,d}/\partial u_d \end{bmatrix},$$

其中与前面一样，d 表示 $\mathbf{u}$ 的分量数。由于 $\mathcal{P}$ 将一个 $\mathbf{u}$ 值映射为另一个，这里的 Jacobian 为方阵。一般模型方程当然不一定如此，但非方阵 Jacobian 不会给可微仿真造成问题。

实践中，我们依靠现代深度学习框架提供的反向模式微分，重点计算 Jacobian 转置与向量 $\mathbf{a}$ 的矩阵—向量积，即 $(\partial\mathcal{P}_i/\partial\mathbf{u})^T\mathbf{a}$。如果需要构造并存储训练中遇到的所有完整 Jacobian 矩阵，将造成巨大的内存开销并不必要地减慢训练。相反，对于反向传播，由于链的末端始终有标量损失函数，我们可以提供更快的操作来计算与 Jacobian 转置的乘积。

根据上述表述，需要通过链式法则，在某个当前状态 $\mathbf{u}_n$ 处求解 $\mathcal{P}_i$ 函数复合链的导数。例如，对于其中两个算子，

$$\left.\frac{\partial(\mathcal{P}_1\circ\mathcal{P}_2)}{\partial\mathbf{u}}\right|_{\mathbf{u}_n} = \left.\frac{\partial\mathcal{P}_1}{\partial\mathbf{u}}\right|_{\mathcal{P}_2(\mathbf{u}_n)} \left.\frac{\partial\mathcal{P}_2}{\partial\mathbf{u}}\right|_{\mathbf{u}_n},$$

这只是“经典”链式法则 $f(g(x))' = f'(g(x))g'(x)$ 的向量值版本，且可直接扩展到更多复合函数，即 $i>2$。

这里 $\mathcal{P}_1$ 和 $\mathcal{P}_2$ 的导数仍为 Jacobian 矩阵，但已知链“末端”有标量损失（参见“概述”），原文称最右边的 Jacobian 总是只有一列的矩阵，即向量。反向模式从这一向量出发，逐个计算与左侧 Jacobian（上式中的 $\partial\mathcal{P}_1/\partial\mathbf{u}$ 等）的乘积。[1]

有关前向和反向模式微分的细节，请参阅外部材料，例如原文链接的 Baydin 等人综述；链接目标待核实。

13.3 通过可微物理算子学习

因此，一旦模拟器的算子支持 Jacobian—向量积计算，我们就可以像加入普通全连接层或 ReLU 激活一样，将它们纳入深度学习流程。

此时会出现一个很合理的问题：“大多数物理求解器都可以分解为一系列向量和矩阵操作。所有先进深度学习框架都支持这些操作，为什么不直接用这些算子实现物理求解器?”

[1] 此处原文混合了前向 Jacobian 乘积的书写顺序与反向传播时的转置约定；采用列梯度，对 $F=\mathcal{P}_1\circ\mathcal{P}_2$，$DF = J_1(\mathcal{P}_2u)J_2(u)$，反传为 $g_u = J_2(u)^T J_1(\mathcal{P}_2u)^T g_F$。原前向式与省略转置的叙述不能混作同一乘法约定。

理论上确实可以。问题在于 TensorFlow 和 PyTorch 中每个向量及矩阵操作都单独计算，内部需要存储当前前向求值状态以供反向传播（即上面的 $g(x)$）。然而，对典型仿真而言，我们对求解器产生的每一个中间结果并不太感兴趣；通常更关心从 $\mathbf{u}(t)$ 到 $\mathbf{u}(t+\Delta t)$ 这样的重要更新。

因此在实践中，将求解过程分解为一系列有意义但整体实现的算子是很好的做法。这不仅通过避免计算不必要的中间结果节省大量工作，还允许为这些算子的更新和导数选择尽可能好的数值方法。例如，由于这一过程与伴随方法优化非常相似，可以复用该领域开发的许多技术或利用成熟数值方法，如利用多重网格求解器 $O(n)$ 的运行时间进行矩阵求逆。

这种方法的另一面是，它要求一定程度地理解手头问题及数值方法。此外，给定求解器可能不直接提供梯度计算。因此，如果希望对尚未充分理解的模型方程使用深度学习，直接选择通过可微物理学习可能并非好主意。不过，如果还不真正理解模型，或许应先回过头再多研究一下。

实践中还应对导数算子有所取舍，只提供与学习任务相关的算子。例如，如果上述例子中的网络从不输出参数 ν，而 ν 也不出现在损失表述中，就不会在反向传播步骤中遇到 $\partial/\partial\nu$ 导数。

下图总结基于可微物理的学习方法，并说明单次 PDE 求解中通常处理的一系列操作。由于实践中的许多操作都是非线性的，这往往给神经网络带来有挑战性的学习任务：

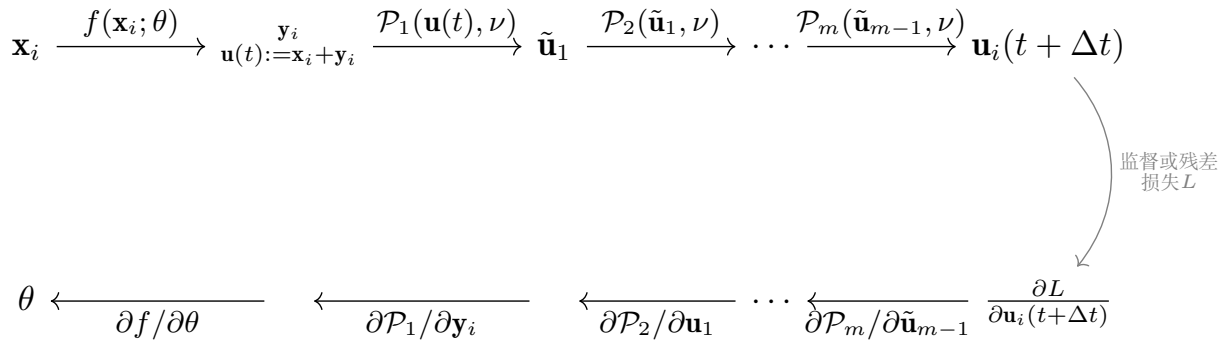


图 13.2: 使用包含 m 个独立算子 $\mathcal{P}_i$ 的 PDE 求解器进行可微物理学习。梯度在影响网络权重 θ 之前向后经过全部 m 个算子。图中 supervised or residual loss 为监督损失或残差损失。

13.4 一个实际例子

作为简单例子，考虑被动标量密度 $d(\mathbf{x},t)$ 在速度场 $\mathbf{u}$ 中的平流，将其作为物理模型 $\mathcal{P}^*$：

$$\frac{\partial d}{\partial t} + \mathbf{u}\cdot\nabla d = 0.$$

不立即将这一表述用作残差方程（如“物理损失项”的 v2），我们可以选择网格和离散化格式将其离散化，得到随时间更新系统状态的表述。这是正向求解的标准过程。为简单起见，这里假设 $\mathbf{u}$ 仅是空间的函数，即随时间恒定；后面会重新引入 $\mathbf{u}$ 的时间演化。

将这一重新表述记为 $\mathcal{P}$。它将状态 $d(t)$ 映射到时间推进后的新状态，即

$$d(t+\Delta t) = \mathcal{P}(d(t), \mathbf{u}, t+\Delta t).$$

作为逆问题与学习任务的简单例子，考虑寻找速度场 $\mathbf{u}$ 的问题。该速度应将时间 t_0 的给定初始标量密度状态 d^0，通过 $\mathcal{P}$ 演化至之后的“结束”时间 t_e，得到具有特定形状或配置的状态 d^{target}。非正式地

说，我们希望找到一个流动，使 d^0 通过 PDE 模型变形为目标状态。表达这一目标的最简单方式是两个状态之间的 L^2 损失，因此要最小化损失函数 $L = |d(t_e) - d^{\text{target}}|^2$。

注意按这里的描述，这一逆问题是纯优化任务：其中没有神经网络，目标是获得 $\mathbf{u}$。我们不打算像真正的学习任务中那样将这个速度应用到其他未见过的测试数据。

标记密度的最终状态 $d(t_e)$ 完全由 $\mathcal{P}$ 通过 $\mathbf{u}$ 的演化决定，因此得到最小化问题

$$\arg\min_{\mathbf{u}} \left|\mathcal{P}(d^0, \mathbf{u}, t_e) - d^{\text{target}}\right|^2.$$

现在希望通过梯度下降（GD）找到目标函数的极小点，梯度由本章前述可微物理方法确定。一旦 GD 能够工作，就可以相对容易地切换到更好的优化器或引入神经网络，因此它始终是良好起点。

为了便于阅读，下文省略 Jacobian 的转置。原文指出 Jacobian 的定义虽然如此，实际上从不需要未转置的形式；请记住实践中处理的是转置 Jacobian $(\partial a/\partial b)^T$，这里“简写”为 $\partial a/\partial b$。

由于离散速度场 $\mathbf{u}$ 包含所有自由度，只需将速度更新量记为 $\Delta\mathbf{u} = \partial L/\partial\mathbf{u}$，并将其分解为 $\Delta\mathbf{u} = (\partial d/\partial\mathbf{u})(\partial L/\partial d)$。[2]

$\partial L/\partial d$ 部分通常足够简单：

$$\frac{\partial L}{\partial d} = \frac{\partial|\mathcal{P}(d^0, \mathbf{u}, t_e) - d^{\text{target}}|^2}{\partial d} = 2(d(t_e) - d^{\text{target}}).$$

如果 d 表示为向量，例如网格每个单元对应一个分量，$\partial L/\partial d$ 同样是大小相同的列向量。这源自 L 始终为标量损失，因此 Jacobian 矩阵沿 L 的维度大小为 1。直观上，这一向量包含结束时间的 d 与目标密度 d^{target} 之间的差。

d 本身的演化由离散物理模型 $\mathcal{P}$ 给出，这里交替使用 $\mathcal{P}$ 和 d。因此更有意思的部分是 Jacobian $\partial d/\partial\mathbf{u} = \partial\mathcal{P}/\partial\mathbf{u}$，用来计算完整的 $\Delta\mathbf{u} = (\partial d/\partial\mathbf{u})(\partial L/\partial d)$。所幸不需要完整的 $\partial d/\partial\mathbf{u}$ 矩阵，只需它与 $\partial L/\partial d$ 的乘积。

那么 d 的实际 Jacobian 是什么？为了计算它，首先需要确定 PDE 模型 $\mathcal{P}$，得到可以求导的表达式。下一节将选择具体的平流格式与离散化，以便更具体地讨论。

13.4.1 引入具体的平流格式

下文使用一维笛卡尔网格上的简单一阶迎风格式，第 i 个单元的标记密度和速度分别为 d_i 和 u_i。为简洁起见，时间 t 的量省略 (t)，即将 $d_i(t)$ 写为 d_i。如上所述，物理模型更新标记密度 $d_i(t + \Delta t) = \mathcal{P}(d_i(t), \mathbf{u}(t), t + \Delta t)$，得到

$$\begin{aligned} d_i(t + \Delta t) &= d_i - \Delta t \left[u_i^+(d_{i+1} - d_i) + u_i^-(d_i - d_{i-1})\right], \\ u_i^+ &= \min(u_i/\Delta x, 0), \\ u_i^- &= \max(u_i/\Delta x, 0). \end{aligned}$$

因此当 u_i 为负时，使用 u_i^+ 沿速度相反方向查看，即相对于运动方向向后；此时 u_i^- 为零。当 u_i 为正时则相反，u_i^+ 为零，并通过 u_i^- 得到后向差分模板。取前一种情况，即 $u_i < 0$，得到

$$\mathcal{P}(d_i(t), \mathbf{u}(t), t + \Delta t) = \left(1 + \frac{u_i\Delta t}{\Delta x}\right) d_i - \frac{u_i\Delta t}{\Delta x} d_{i+1}. \tag{13.1}$$

[2] 原文将更新量写成正梯度；若用于梯度下降，实际更新还需约定负号与步长。此处保留原记号，转置约定也须结合维度核查。

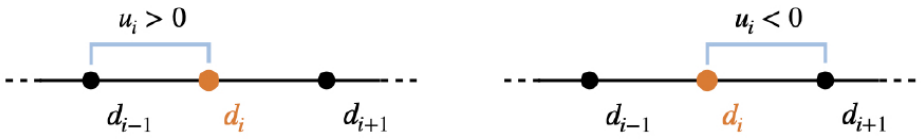


图 13.3: 一阶迎风格式使用简单的单侧有限差分模板，并考虑流动方向。

因此 $\partial\mathcal{P}/\partial u_i$ 给出 $(\Delta t/\Delta x)d_i - (\Delta t/\Delta x)d_{i+1}$。直观而言，速度 u_i 的变化取决于密度的空间导数。由于使用一阶迎风格式，只包含两个相邻值；更高阶方法会依赖 d 的其他分量。

实践中，这一步等价于计算一次转置矩阵乘法。如果将上述计算重写为 $\mathcal{P}(d_i(t), \mathbf{u}(t), t+\Delta t) = A\mathbf{u}$，则 $(\partial\mathcal{P}/\partial\mathbf{u})^T = A^T$。但在许多实际情况下，相比实际构造 A，对这一乘法使用无矩阵实现可能更好。[3]

平流格式另一个可以考虑的导数是相对于前一密度状态 $d_i(t)$（简写为 d_i）的导数。上式对单元 i 的 $\partial\mathcal{P}/\partial d_i$ 给出 $1 + u_i\Delta t/\Delta x$。但完整梯度还需根据相邻单元速度的符号，加入来自单元 $i+1$ 和 $i-1$ 的可能贡献。这个导数将在下一节使用。

13.4.2 时间演化

目前只讨论了 d 从时间 t 到 $t+\Delta t$ 的单步更新，但当然可以有任意多个这样的步。毕竟，上面提出的目标是将初始标记状态 $d(t_0)$ 推进到时间 t_e 的目标状态，这可能跨越很长时间区间。

在上面的 $d_i(t+\Delta t)$ 表达式中，每个 $d_i(t)$ 又依赖时间 $t-\Delta t$ 的速度和密度状态，即 $d_i(t-\Delta t)$。因此，必须一直回溯损失 L 的影响，直到 $\mathbf{u}$ 如何影响初始标记状态。这可能涉及通过 $\mathcal{P}$ 对平流格式进行大量求值。

乍看之下这很有挑战性：例如，可以尝试将时间 $t-\Delta t$ 的式 (13.1) 代入时间 t 的该式，并递归重复，直到得到一个将 d^0 与目标联系起来的单一表达式。但由于 Jacobian 的线性性质，我们将 $\mathcal{P}$ 中每个平流步，即 PDE $\mathcal{P}$ 的每次调用，视作单独的模块化操作，每次调用都遵循上一节的过程。

有了上述机制，回溯实现起来相当简单：对 $\mathcal{P}$ 的每个平流步，计算 Jacobian 与来自损失 L 或前一个平流步的传入导数向量的乘积。重复这一过程，直到将链从带 d^{target} 的损失一路追踪回 d^0。理论上，速度 $\mathbf{u}$ 可以像 d 一样是时间的函数，此时每个时间步 t 都得到一个梯度 $\Delta\mathbf{u}(t)$。但为简化起见，下面假设场随时间恒定，即每次通过 $\mathcal{P}$ 平流时复用相同速度 $\mathbf{u}$。现在每个时间步都会给出对 $\Delta\mathbf{u}$ 的一个贡献，将所有步的贡献累加：

$$\begin{aligned}\Delta\mathbf{u} = &\frac{\partial d(t_e)}{\partial\mathbf{u}}\frac{\partial L}{\partial d(t_e)} \\ &+ \frac{\partial d(t_e-\Delta t)}{\partial\mathbf{u}}\frac{\partial d(t_e)}{\partial d(t_e-\Delta t)}\frac{\partial L}{\partial d(t_e)} \\ &+ \cdots \\ &+ \left(\frac{\partial d(t_0)}{\partial\mathbf{u}}\cdots\frac{\partial d(t_e-\Delta t)}{\partial d(t_e-2\Delta t)}\frac{\partial d(t_e)}{\partial d(t_e-\Delta t)}\frac{\partial L}{\partial d(t_e)}\right).\end{aligned}$$

这里最后一项包含将标记密度完整回溯到时间 t_0 的过程。这些求和项乍看繁琐，但仔细观察，每一行只是在左侧多加一个时间步的 Jacobian。这由链式法则得出，如上面的双算子情形所示。因此求和项包

[3] 原更新包含与速度无关的 d_i 项，一般是关于速度的仿射映射；写为 $A\mathbf{u}$ 是否吸收了该项需要核查。

含许多类似的 Jacobian；实践中可以通过回溯 PDE 前向求值产生的计算步骤序列来高效计算。注意如上所述，这里省略了 Jacobian 的转置。[4]

这种结构也清楚表明，该过程与神经网络的常规训练很相似：对嵌套函数调用计算这些 Jacobian 向量积，正是深度学习框架训练神经网络时所做的事情（那里是权重 θ，而不是速度场）。因此，实践中只需为 $\mathcal{P}$ 提供计算 Jacobian 向量积的自定义函数。

13.5 隐式梯度计算

作为稍复杂的例子，考虑 Poisson 方程 $\nabla^2 a = b$，其中 a 是感兴趣的量，b 为给定值。这是一个非常基础的椭圆 PDE，对从静电学到引力场等各种物理问题都很重要。在流体背景中也会出现：a 是流体中的标量压力场，右侧 b 由流体速度 $\mathbf{u}$ 的散度给出。

对于流体，通常有 $\mathbf{u}_n = \mathbf{u} - \nabla p$，其中 $\nabla^2 p = \nabla \cdot \mathbf{u}$。这里 $\mathbf{u}_n$ 表示新的无散度速度场。这一步通常对施加硬约束 $\nabla \cdot \mathbf{u} = 0$ 至关重要，也称为 Chorin 投影或 Helmholtz 分解，是向量分析基本定理的直接结果。

如果现在引入一个在求解器中修改 $\mathbf{u}$ 的神经网络，就不可避免地需要通过 Poisson 求解反向传播。也就是说，需要 $\mathbf{u}_n$ 的梯度，按此处记号为 $\partial \mathbf{u}_n / \partial \mathbf{u}$。

综合起来，我们希望计算 $\mathbf{u}_n = \mathbf{u} - \nabla((\nabla^2)^{-1}\nabla \cdot \mathbf{u})$。外侧梯度（来自 ∇p）和内侧散度（$\nabla \cdot \mathbf{u}$）都是线性算子，其梯度易于计算。主要困难是获得 Poisson 方程的矩阵逆 $(\nabla^2)^{-1}$。这里保持简单，但它通常依赖时间且为非线性。[5]

实践中，$(\nabla^2)^{-1} b$（其中 $b = \nabla \cdot \mathbf{u}$）的矩阵—向量积并不通过显式矩阵运算计算，而是用可能不显式构造矩阵的迭代求解器近似。例如，共轭梯度（CG）法是很常用的选择。因此，理论上可以将该迭代求解器视为函数 $\mathcal{S}$，使 $p = \mathcal{S}(\nabla \cdot \mathbf{u})$。原文指出，矩阵求逆是非线性过程，尽管矩阵本身是线性的。

由于 CG 等求解器也基于矩阵和向量运算，可以按求解器所有迭代将 $\mathcal{S}$ 分解为一系列较简单操作，$\mathcal{S}(x) = \mathcal{S}_n(\mathcal{S}_{n-1}(\dots \mathcal{S}_1(x)))$，并通过每一步反向传播。这当然可行，但原文认为不是好主意：它可能引入数值问题，速度也会很慢。如前所述，深度学习框架默认存储本例中 $\mathcal{S}_i()$ 等每个可微算子的内部状态，因而需要在内存中组织和保存可能数量庞大的中间状态。但这些状态对原始 PDE 完全不重要，只是 CG 求解器的中间状态。

退一步看 $p = (\nabla^2)^{-1} b$，其梯度 $\partial p / \partial b$ 是 $((\nabla^2)^{-1})^T$。在此情况下，∇^2 是对称矩阵，因此 $((\nabla^2)^{-1})^T = (\nabla^2)^{-1}$。这与原方程中遇到的逆矩阵相同，因而可以复用未修改的迭代求解器计算梯度，无须拆解求解过程并存储中间状态而降低速度。

不过，迭代求解器计算的是 $(\nabla^2)^{-1} b$ 的矩阵—向量积，那么反向传播时 b 是什么？优化设置中前向链末端总有损失函数 L。反向传播将给出输出梯度，这里假设是 $\partial L / \partial p$，它需要传到前向过程的较早操作。因此，只需在反向过程中调用迭代求解器，计算 $\partial p / \partial b = \mathcal{S}(\partial L / \partial p)$。假设前向过程选择了良好的求解器 $\mathcal{S}$，原文称反向过程得到完全相同的性能和精度。[6]

---

[4] 末项中的 $d(t_0)$ 是否依赖 $\mathbf{u}$ 取决于初态设定；本节此前将 d^0 视为给定。对 $d_{k+1} = \mathcal{P}_k(d_k, u)$，须区分固定 d_k 的局部 $B_k = \partial \mathcal{P}_k / \partial u$ 与 $A_k = \partial \mathcal{P}_k / \partial d_k$，列梯度反传为 $g_u = \sum_{k=0}^{K-1} B_k^T A_{k+1}^T \cdots A_{K-1}^T g_K$。给定初态有 $\partial d_0 / \partial u = 0$，源末项不能当作首步局部 B_0；固定正负速度分支的导数也不自动适用于 $u = 0$ 不可微点。原式保留。

[5] 逆算子是否存在及是否对称取决于边界条件和离散化；周期或 Neumann 问题还需处理常数零空间。源文关于“时间依赖、非线性”的表述不应直接用于固定系数线性 Poisson 算子。

[6] 此处右侧是将逆算子作用于上游梯度的向量，通常对应损失对 b 的梯度，而不是 Jacobian $\partial p / \partial b$ 本身；有限容差下前后误差也需分别评估。按原文保留。

如果对代码示例感兴趣，phiflow 的 differentiate-pressure 例子正是使用这一过程，通过压力投影步骤进行优化：流场右侧受到约束，优化左侧内容，使其在压力投影后与右侧目标匹配。

主要结论是：重要的是不要盲目地通过前向计算反向传播，而要考虑对前向过程解析方程的哪些步骤计算梯度。在上述情形中，通常可以找到改进的梯度解析表达式，然后对其作数值近似。

隐函数定理与时间　隐函数定理：上述过程本质上产生一个隐式导数。我们没有显式地推导全部前向步骤，而是依靠隐函数定理计算导数。

时间：实际上可以将迭代求解器的步骤视为虚拟“时间”，并通过这些步骤反向传播。与其他可微物理方法一致，这使神经网络能与迭代求解器交互。一个例子是 [UBH+20] 中学习 CG 求解器的初始猜测。原文在链接中提供了细节和代码。

13.6　可微物理阶段总结

总结而言，使用可微物理仿真为我们提供了一个工具，可以将具有选定离散化的物理方程纳入深度学习。与上一章的残差约束不同，它使神经网络能够与物理求解器无缝交互。

以前一旦神经网络训练完成，就会完全抛弃物理模型和求解器：在“使用 PINN 优化 Burgers 方程”的例子中，神经网络直接给出解，绕过所有求解器和模型方程。可微物理方法与“物理损失项”中的物理信息神经网络（v2）有显著区别，与受控离散化（v1）有更多共同点；后者本质上是可微物理训练的子集或部分应用。

但与两种残差方法不同，可微物理可以在数值求解器旁训练神经网络，因此之后在推断时也能使用由求解器表示的物理模型。这使我们能够超越求解单个逆问题，并得到对新输入具有相当稳健泛化能力的神经网络。现在重新在可微物理背景下考察“使用 PINN 优化 Burgers 方程”中的示例问题。

译注：算子链图的记号　图中第二算子前向输入写 $\tilde{u}_1$，其反向标签按源写 $\partial \mathcal{P}_2/\partial u_1$，波浪号不一致保留，不能据图省略的转置直接确定梯度乘法顺序。

第十四章　使用可微物理梯度优化 Burgers 方程

为了说明如何在可微物理（DP）设置中计算梯度，我们研究“使用 PINN 优化 Burgers 方程”中 PINN 示例所用的同一个逆问题，即重建任务。选择 DP 方法有一些直接影响：我们从离散化 PDE 出发，系统演化现在完全由所得数值求解器决定。因此唯一真正的未知量是初态。仍需多次重新计算初态与目标状态之间的所有状态，只是现在此步骤不需要神经网络，而是依赖模型方程的离散化。

此外，由于为 DP 方法选择了初始离散化，未知初态由相关物理场的采样点组成，可以直接将这些未知量表示为浮点变量。因此即使对于初态也无需设置神经网络。故使用 DP 求解时，Burgers 重建问题化为一个完全不含神经网络、基于梯度的优化问题。尽管如此，它仍是说明这一过程的良好起点。

首先设置离散仿真。这里使用 phiflow，如“概述”中 Burgers 正向仿真一节所示。原文提供的 Colab 链接待核实。

14.1　初始化

只要不使用 NumPy 后端，phiflow 就直接提供一系列可微操作。重要步骤是导入 `phi.tf.flow` 而不是 `phi.flow`（对于 PyTorch，可以使用 `phi.torch.flow`）。因此，第一步是设置一些常数，并用零初始化速度场，以及将 $t = 0.5$（第 16 步）的约束初始化为 phiflow 中的 CenteredGrid。两者都使用周期边界条件（通过 `extrapolation.PERIODIC`）和空间离散步长 $\Delta x = 1/128$。[1]

```
!pip install --upgrade --quiet phiflow==3.1
from phi.tf.flow import *
N = 128
DX = 2/N
STEPS = 32
DT = 1/STEPS
NU = 0.01/(N*np.pi)
# allocate velocity grid
velocity = CenteredGrid(0, extrapolation.PERIODIC, x=N, bounds=Box(x=(-1,1)))
# and a grid with the reference solution
REFERENCE_DATA = math.tensor([
    0.008612174447657694, 0.02584669669548606, 0.043136357266407785, 0.060491074685516746,
    0.07793926183951633, 0.0954779141740818, 0.11311894389663882, 0.1308497114054023,
    0.14867023658641343, 0.1665634396808965, 0.18452263429574314, 0.20253084411376132,
```

[1]原文文字写 $\Delta x = 1/128$，但代码中区域为 $[-1, 1]$ 且 `DX=2/N`，因此 $N = 128$ 时为 $2/128$；保留两者。

```
    0.22057828799835133, 0.23865132431365316, 0.25673879161339097, 0.27483167307082423,
    0.2929182325574904, 0.3109944766354339, 0.3290477753208284, 0.34707880794585116,
    0.36507311960102307, 0.38303584302507954, 0.40094962955534186, 0.4188235294008765,
    0.4366357052408043, 0.45439856841363885, 0.4720845505219581, 0.4897081943759776,
    0.5072391070000235, 0.5247011051514834, 0.542067187709797, 0.5593576751669057,
    0.5765465453632126, 0.5936507311857876, 0.6106452944663003, 0.6275435911624945,
    0.6443221318186165, 0.6609900633731869, 0.67752574922899, 0.6939334022562877,
    0.7101938106059631, 0.7263049537163667, 0.7422506131457406, 0.7580207366534812,
    0.7736033721649875, 0.7889776974379873, 0.8041371279965555, 0.8190465276590387,
    0.8337064887158392, 0.8480617965162781, 0.8621229412131242, 0.8758057344502199,
    0.8891341984763013, 0.9019806505391214, 0.9143881632159129, 0.9261597966464793,
    0.9373647624856912, 0.9476871303793314, 0.9572273019669029, 0.9654367940878237,
    0.9724097482283165, 0.9767381835635638, 0.9669484658390122, 0.659083299684951,
    -0.659083180712816, -0.9669485121167052, -0.9767382069792288, -0.9724097635533602,
    -0.9654367970450167, -0.9572273263645859, -0.9476871280825523, -0.9373647681120841,
    -0.9261598056102645, -0.9143881718456056, -0.9019807055316369, -0.8891341634240081,
    -0.8758057205293912, -0.8621229450911845, -0.8480618138204272, -0.833706571569058,
    -0.8190466131476127, -0.8041372124868691, -0.7889777195422356, -0.7736033858767385,
    -0.758020740007683, -0.7422507481169578, -0.7263049162371344, -0.7101938950789042,
    -0.6939334061553678, -0.677525822052029, -0.6609901538934517, -0.6443222327338847,
    -0.6275436932970322, -0.6106454472814152, -0.5936507836778451, -0.5765466491708988,
    -0.5593578078967361, -0.5420672759411125, -0.5247011730988912, -0.5072391580614087,
    -0.4897082914472909, -0.47208460952428394, -0.4543985995006753, -0.4366355580500639,
    -0.41882350871539187, -0.40094955631843376, -0.38303594105786365, -0.36507302109186685,
    -0.3470786936847069, -0.3290476440540586, -0.31099441589505206, -0.2929180880304103,
    -0.27483158663081614, -0.2567388003912687, -0.2386513127155433, -0.22057831776499126,
    -0.20253089403524566, -0.18452269630486776, -0.1665634500729787, -0.14867027528284874,
    -0.13084990929476334, -0.1131191325854089, -0.09547794429803691, -0.07793928430794522,
    -0.06049114408297565, -0.0431364527809777, -0.025846763281087953, -0.00861212501518312
], math.spatial('x'))
SOLUTION_T16 = CenteredGrid(REFERENCE_DATA, extrapolation.PERIODIC, x=N,
                            bounds=Box(x=(-1,1)))
```

下面验证仿真的场现在由 TensorFlow 支持。

```
type(velocity.values.native())
tensorflow.python.framework.ops.EagerTensor
```

14.2 梯度

phiflow 的 math.gradient 操作为标量损失生成梯度函数。下面使用它计算具有所选 32 个时间步的完整仿真的梯度。

对于 Burgers 情形，需要计算适当损失：希望 $t = 0.5$ 的解与参考数据吻合。因此，只需计算第 16 步与约束数组之间的 L^2 差作为损失，然后计算初始速度状态 velocity 相对于这一损失的梯度函数。Phiflow 的 math.gradient 生成一个为每个参数返回梯度的函数；这里只有一个参数，即速度，因此 grad[0] 表示初始速度的梯度。

```
def loss_function(velocity):
    velocities = [velocity]
    for time_step in range(STEPS):
        v1 = diffuse.explicit(1.0*velocities[-1], NU, DT, substeps=1)
        v2 = advect.semi_lagrangian(v1, v1, DT)
        velocities.append(v2)
    loss = field.l2_loss(velocities[16] - SOLUTION_T16)*2./N # MSE
    return loss, velocities

gradient_function = math.gradient(loss_function)
(loss,velocities), grad = gradient_function(velocity)
print('Loss: %f' % (loss))
```

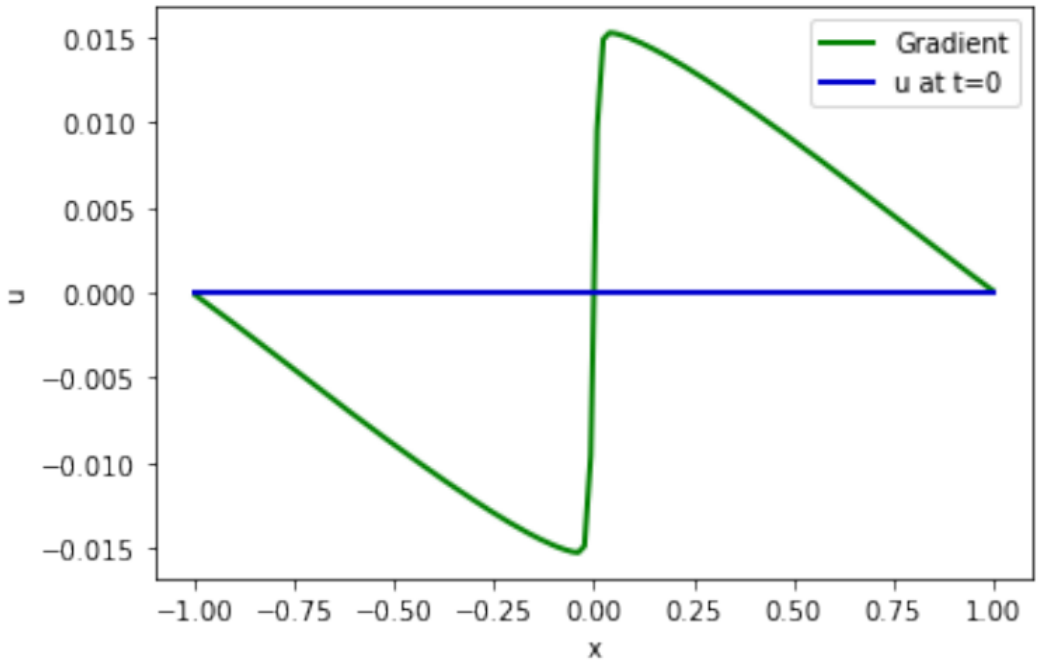
```
Loss: 0.382915
```

由于只约束时间步 16，在这一设置中实际上可以省略第 17 至 31 步。它们没有任何自由度，也不受任何约束。不过，为公平地与前面的情形比较，仍将它们包括在内。

注意这里进行了大量计算：首先是仿真的 32 步，然后从损失向后进行另外 16 步。这些步骤由梯度带记录，并用于将损失反向传播至仿真初态。由于从零出发，第 16 个仿真步存在约 0.38 的显著初始误差也不足为奇。

这里得到怎样的梯度？它与速度维度相同，可以方便地可视化：从零速度状态（蓝色）出发，下面以绿色显示第一个梯度。与解相比，它如预期般指向相反方向。解的量级大得多，因此这里省略（见下一张图）。

```
import pylab as plt
fig = plt.figure().gca()
pltx = np.linspace(-1,1,N)
# first gradient
fig.plot(pltx, grad[0].values.numpy('x'), lw=2, color='green', label="Gradient")
fig.plot(pltx, velocity.values.numpy('x'), lw=2, color='mediumblue', label="u at t=0")
plt.xlabel('x'); plt.ylabel('u'); plt.legend();
# some (optional) other fields to plot:
# fig.plot(pltx, (velocities[16]).values.numpy('x'), lw=2, color='cyan', label="u at t=0.5")
# fig.plot(pltx, (SOLUTION_T16).values.numpy('x'), lw=2, color='red', label="solution at t=0.5")
# fig.plot(pltx, (velocities[16] - SOLUTION_T16).values.numpy('x'), lw=2,
#          color='blue', label="difference at t=0.5")
```



图中绿线为梯度，蓝线为 $t = 0$ 的 u。这为每个速度变量给出一个“搜索方向”。根据线性近似，梯度告诉我们如何改变各变量来增大损失函数（梯度总是指向“上坡”）。因此可以使用梯度运行优化，找到使损失最小的初始状态速度。

14.3 优化

有了梯度，现在运行梯度下降优化。下面使用学习率 LR=5，并对更新后的状态重新计算损失，以跟踪收敛。

在下列代码块中，还将梯度保存在名为 grads 的列表中，以便稍后可视化。对于常规优化，当然可以在更新速度后丢弃梯度。

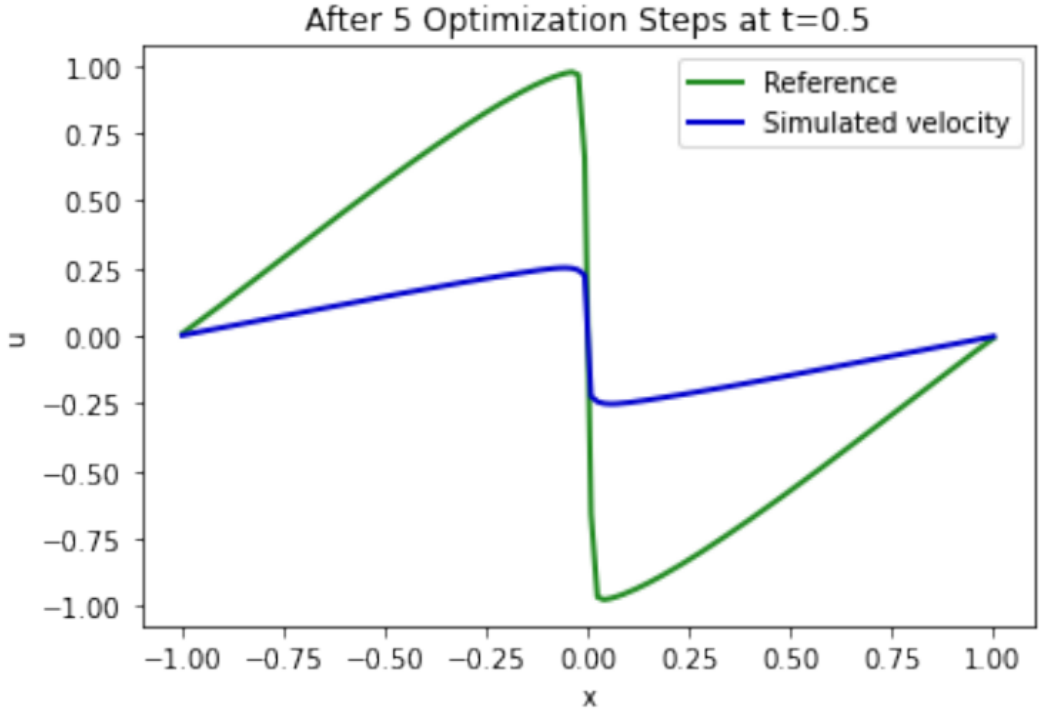
```
LR = 5.
grads=[]
for optim_step in range(5):
    (loss,velocities), grad = gradient_function(velocity)
    print('Optimization step %d, loss: %f' % (optim_step,loss))
    grads.append( grad[0] )
    velocity = velocity - LR * grads[-1]
```

```
Optimization step 0, loss: 0.382915
Optimization step 1, loss: 0.326882
Optimization step 2, loss: 0.281032
Optimization step 3, loss: 0.242804
Optimization step 4, loss: 0.210666
```

现在检查经过 5 次更新后，仿真第 16 个状态与目标实际上吻合得如何；毕竟损失衡量的正是这一点。下一张图以绿色显示约束（即希望获得的解），并显示初始速度经梯度更新五次、再由求解器更新 16 次

之后的重建状态。[2]

```
fig = plt.figure().gca()
# target constraint at t=0.5
fig.plot(pltx, SOLUTION_T16.values.numpy('x'), lw=2, color='forestgreen',
         label="Reference")
# optimized state of our simulation after 16 steps
fig.plot(pltx, velocities[16].values.numpy('x'), lw=2, color='mediumblue',
         label="Simulated velocity")
plt.xlabel('x'); plt.ylabel('u'); plt.legend()
plt.title("After 5 Optimization Steps at t=0.5");
```



图中绿线为参考，蓝线为模拟速度。结果看起来正朝正确方向发展！当然并不完美，但目前只计算了 5 次梯度下降更新。激波左侧正速度峰值与右侧负峰值开始显现。

这很好地表明，通过全部 16 个仿真步反向传播梯度的过程工作正常，并将解引向正确方向。上图只是暗示了这种设置的强大之处：从每个仿真步及其中各项操作获得的梯度，都可以轻松串联成更复杂的序列。在上例中，反向传播经过全部 16 个仿真步，稍微修改代码就能扩大优化的这一“前瞻”范围。

14.4 更多优化步骤

在转向更复杂的物理仿真或引入神经网络之前，先完成当前优化任务，运行更多步骤以获得更好的解。

[2]循环中的 `velocities` 在每次更新 `velocity` 之前计算；循环结束后直接绘制缓存的 `velocities[16]`，因此五次循环结束后的缓存来自第 4 次更新状态，变量本身已更新 5 次。同理后续 `range(5,50)` 结束时缓存来自第 49 次优化更新，`velocity` 已更新 50 次；After50 图及保存的 NPZ 沿用 49 步缓存，而 initial50 图用 50 步变量。此为静态时序差异，未运行或改代码。

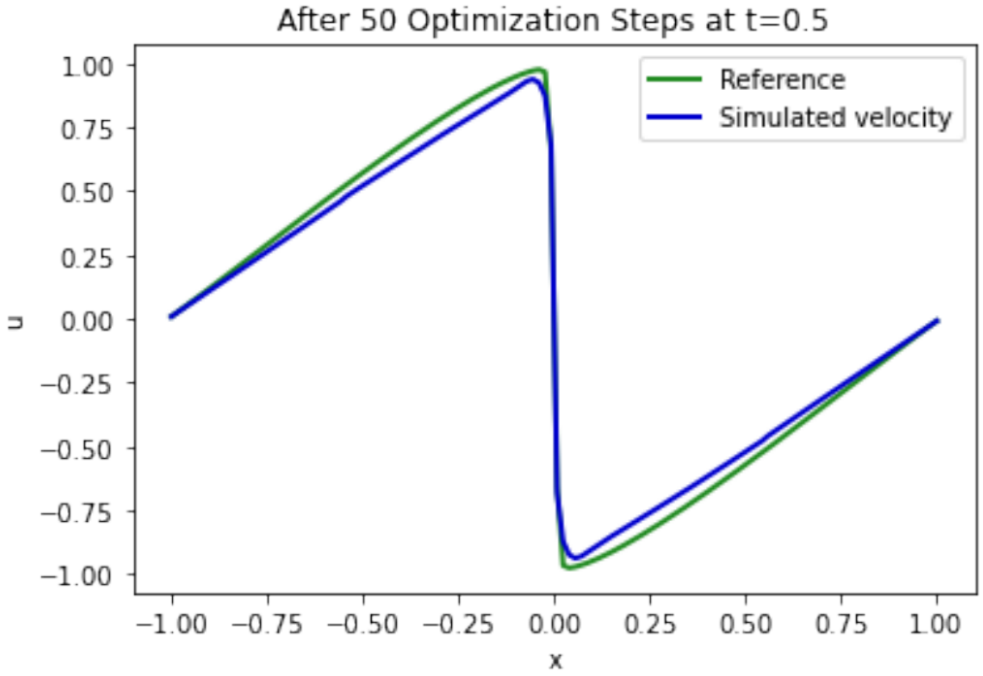
```
import time
start = time.time()
for optim_step in range(5,50):
    (loss,velocities), grad = gradient_function(velocity)
    velocity = velocity - LR * grad[0]
    if optim_step%5==0:
        print('Optimization step %d, loss: %f' % (optim_step,loss))
end = time.time()
print("Runtime {:.2f}s".format(end-start))
```

```
Optimization step 5, loss: 0.183476
Optimization step 10, loss: 0.096224
Optimization step 15, loss: 0.054792
Optimization step 20, loss: 0.032819
Optimization step 25, loss: 0.020334
Optimization step 30, loss: 0.012852
Optimization step 35, loss: 0.008185
Optimization step 40, loss: 0.005186
Optimization step 45, loss: 0.003263
Runtime 132.33s
```

回想前面的 PINN 版本，在相近运行时间内，这里的误差下降明显更多，约为两个数量级。原文认为这一行为源于 DP 为完整解及其全部离散点和时间步提供梯度，而不是局部更新。

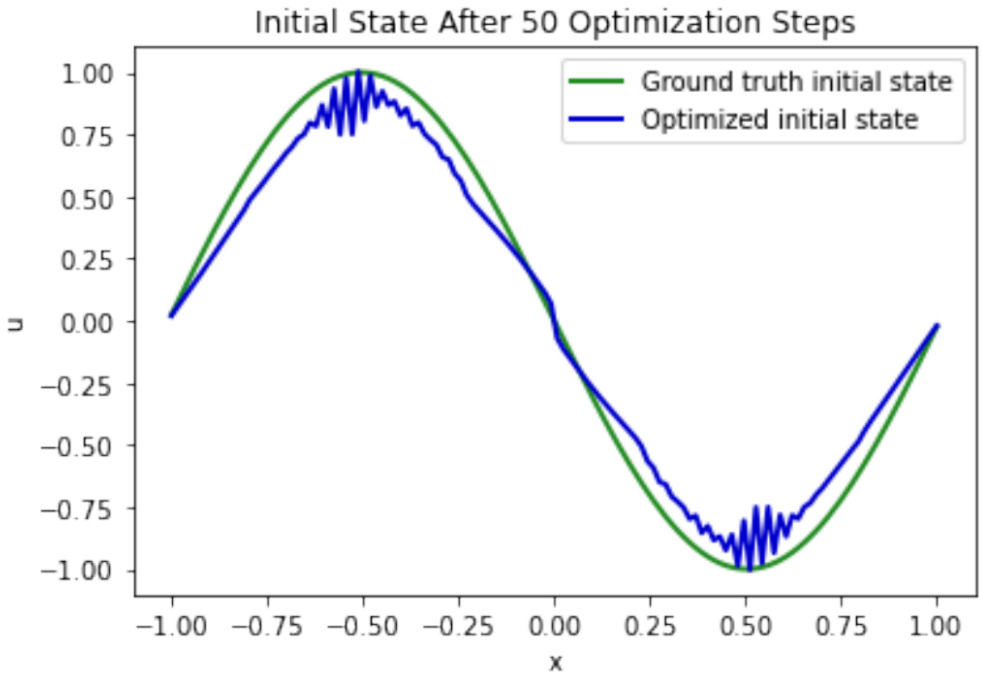
再次绘制现在 $t = 0.5$ 时的解（蓝色）与约束（绿色）的吻合程度：

```
fig = plt.figure().gca()
fig.plot(pltx, SOLUTION_T16.values.numpy('x'), lw=2, color='forestgreen',
         label="Reference")
fig.plot(pltx, velocities[16].values.numpy('x'), lw=2, color='mediumblue',
         label="Simulated velocity")
plt.xlabel('x'); plt.ylabel('u'); plt.legend()
plt.title("After 50 Optimization Steps at t=0.5");
```



还不错。但通过 16 个仿真步反向传播，初态恢复得如何？这正是我们改变的量，而且由于它只通过稍后时间的观测受到间接约束，因此有更大空间偏离所需或预期的解。下一张图显示了这一点：

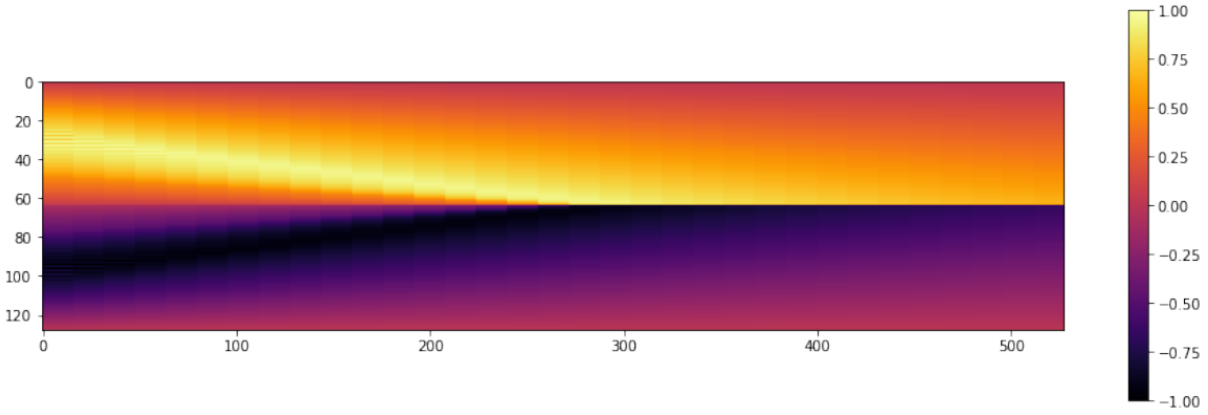
```
fig = plt.figure().gca()
pltx = np.linspace(-1,1,N)
# ground truth state at time=0 , move down
INITIAL_GT = np.asarray([-np.sin(np.pi * x) for x in
                         np.linspace(-1+DX/2,1-DX/2,N)]) # 1D numpy array
fig.plot(pltx, INITIAL_GT.flatten(), lw=2, color='forestgreen',
         label="Ground truth initial state") # ground truth initial state of sim
fig.plot(pltx, velocity.values.numpy('x'), lw=2, color='mediumblue',
         label="Optimized initial state") # manual
plt.xlabel('x'); plt.ylabel('u'); plt.legend()
plt.title("Initial State After 50 Optimization Steps");
```



绿线为真实初态，蓝线为优化初态。这自然是更困难的任务：优化过程直接收到关于 $t = 0.5$ 状态应有形态的反馈，但由于模型方程非线性，通常有大量解能够精确或在数值上非常接近地满足约束。因此极小化结果不一定找到出发时的精确状态（默认设置下这里可观察到来自扩散算子的一些数值振荡）。不过在这个 Burgers 情形中，解仍然相当接近。

在衡量重建的总体误差之前，先可视化系统随时间的完整演化，这也以 NumPy 数组形式给出解，便于与其他版本比较：

```
import pylab
def show_state(a):
    a=np.expand_dims(a, axis=2)
    for i in range(4):
        a = np.concatenate( [a,a] , axis=2)
    a = np.reshape( a, [a.shape[0],a.shape[1]*a.shape[2]] )
    fig, axes = pylab.subplots(1, 1, figsize=(16, 5))
    im = axes.imshow(a, origin='upper', cmap='inferno')
    pylab.colorbar(im)

# get numpy versions of all states
vels = [ x.values.numpy('x,vector') for x in velocities]
# concatenate along vector/features dimension
vels = np.concatenate(vels, axis=-1)
# save for comparison with other methods
import os; os.makedirs("./temp",exist_ok=True)
np.savez_compressed("./temp/burgers-diffphys-solution.npz",
                    np.reshape(vels,[N,STEPS+1])) # remove batch & channel dimension
show_state(vels)
```



14.5 物理信息与可微物理重建比较

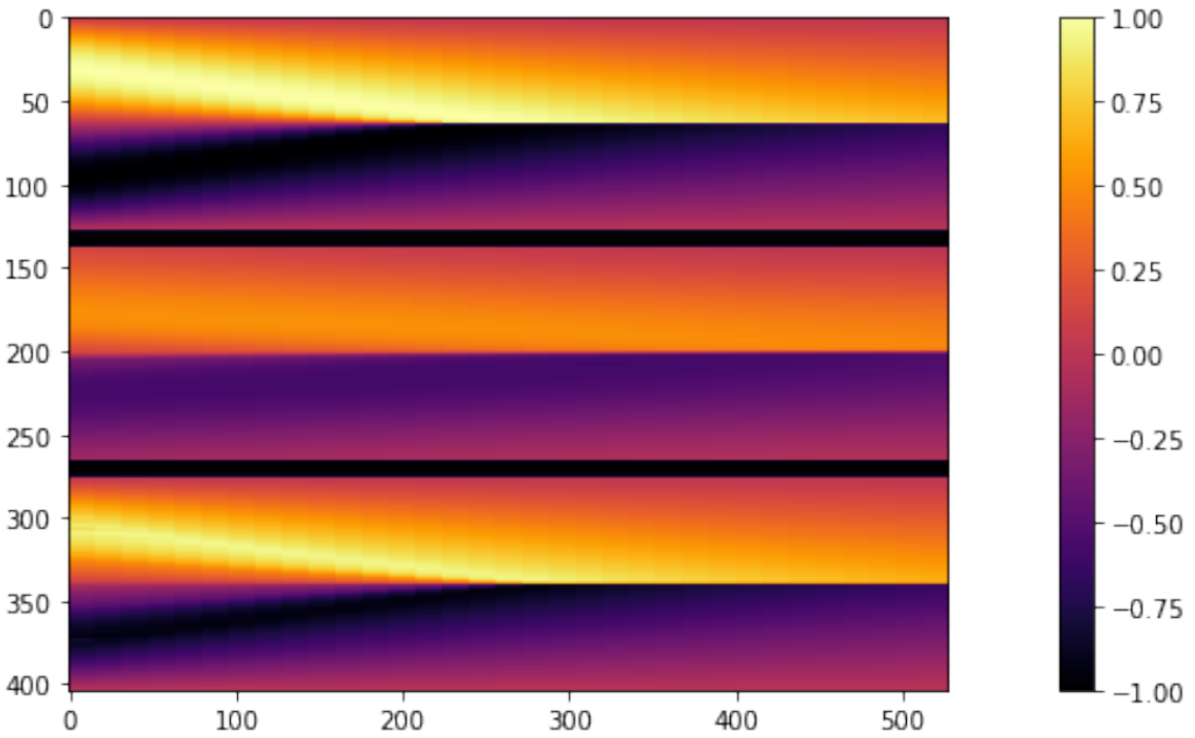
[3] 现在已有 PINN 和 DP 两个版本，可以更详细地比较两种重建。注意，下面的代码单元要求先在同一环境中执行 Burgers 正向仿真及 PINN notebook，使`./temp` 目录中的`.npz` 文件可用。

先并排查看解。下面代码生成包含三个版本的图像，从上到下依次为普通正向仿真给出的“真实值”（GT）解、中间的 PINN 重建、底部的可微物理版本。

```
# note, this requires previous runs of the forward-sim & PINN notebooks
# in the same environment
sol_gt=npfile=np.load("./temp/burgers-groundtruth-solution.npz")["arr_0"]
sol_pi=npfile=np.load("./temp/burgers-pinn-solution.npz")["arr_0"]
sol_dp=npfile=np.load("./temp/burgers-diffphys-solution.npz")["arr_0"]

divider = np.ones([10,33])*-1. # we'll sneak in a block of -1s to show a black
                               # divider in the image
sbs = np.concatenate( [sol_gt, divider, sol_pi, divider, sol_dp], axis=0)
print("\nSolutions Ground Truth (top), PINN (middle) , DiffPhys (bottom):")
show_state(np.reshape(sbs,[N*3+20,33,1]))
```

```
Solutions Ground Truth (top), PINN (middle) , DiffPhys (bottom):
```

[3] 译注：DP/GT 冻结代码的黏度为 $.01/(N\pi)$，PINN 残差为 $.01/\pi$；网格端点和边界设定也不同，源 MAE 与运行时间不能解释为严格同设定的控制比较。保存的 `burgers-diffphys-solution.npz` 还使用前述第 49 步缓存；这些范围限定同样适用于以下比较与保存。



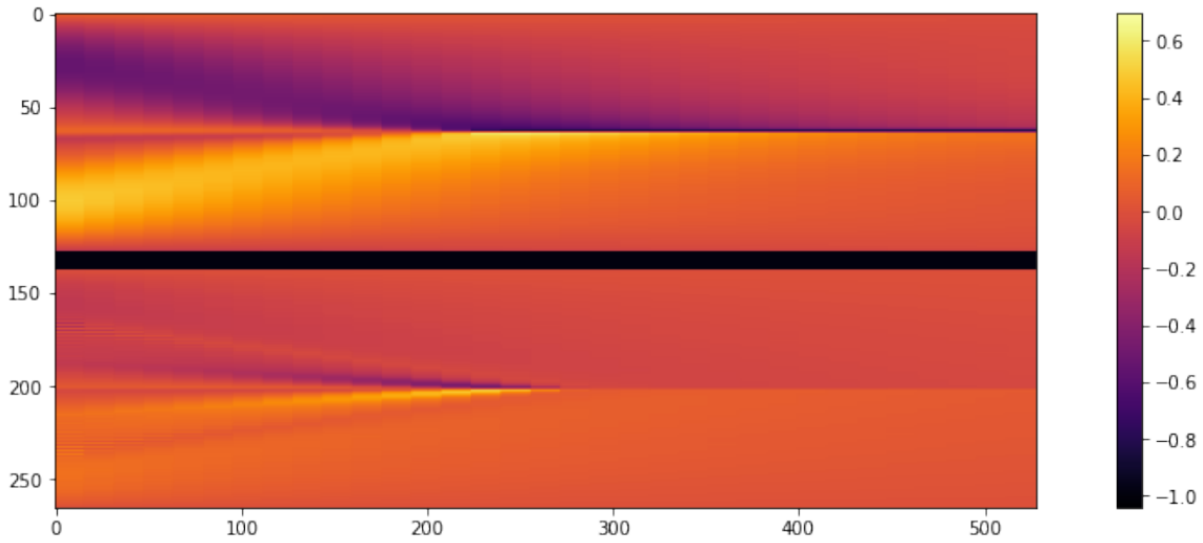
图中清楚可见，中间的 PINN 解恢复了解的整体形状，因此时间约束至少部分得到满足；但它没有很好地捕获 GT 解的振幅。

通过可微求解器优化得到的底部重建更接近真实值，原文将此归因于梯度在整个序列中更好地流动。此外，它可以在前向和反向过程中利用基于网格的离散化，以此为未知初态提供更准确的信号。不过仍可看到重建缺少 GT 版本的某些更“尖锐”特征，例如解图的左下角。

量化整个序列上的误差：

```
err_pi = np.sum( np.abs(sol_pi-sol_gt)) / (STEPS*N)
err_dp = np.sum( np.abs(sol_dp-sol_gt)) / (STEPS*N)
print("MAE PINN: {:7.5f} \nMAE DP:   {:7.5f}".format(err_pi,err_dp))
print("\nError GT to PINN (top) , GT to DiffPhys (bottom):")
show_state(np.reshape(np.concatenate([sol_pi-sol_gt, divider, sol_dp-sol_gt],
                                     axis=0),[N*2+10,33,1]))
```

```
MAE PINN: 0.19298
MAE DP:   0.06382
Error GT to PINN (top) , GT to DiffPhys (bottom):
```



图中上部为 PINN 与 GT 之差，下部为 DP 与 GT 之差。结果相当清楚：PINN 误差比可微物理重建误差大三倍以上。原文称这一差异在共同显示的图像中也很明显：DP 重建误差的量级更接近零，如图中紫色所示。[4]

这种简单直接的重建问题始终是 DP 求解器良好的初始测试。在进入与神经网络耦合等更复杂设置前，可以独立测试它。如果直接优化不收敛，可能仍存在根本问题，此时没有必要引入神经网络。

现在已有第一个例子展示两种方法的异同。下一节将讨论目前的发现，再在后续章节转向更复杂的情形。

14.6 下一步

与之前一样，上述代码可以通过多种方式改进和实验：

- 尝试调整训练参数，进一步改善重建；
- 启用不同优化器，观察收敛行为的变化（不一定改善）；
- 改变仿真和重建的步数或分辨率；
- 尝试在 loss_function 前一行加入 `@jit_compile`。这会产生一次性编译开销，但会显著加快优化。

[4]颜色与零的对应关系应以图中色条为准；原图零值附近颜色并非明显的紫色。此处保留原文并列为待核查。公式中数组含 33 个时间点而分母是 `STEPS*N`，是否按预期定义平均也需核对。

第十五章　目前进展顺利——初步讨论

上一节展示了两种重建示例：一种以变体 2（PINN）形式使用物理残差作为软约束，另一种使用可微物理（DP）求解器。虽然两种方法都能为类似逆问题找到极小点，但所得解以及各自表述所得到的非线性优化问题的行为存在显著差异。下面更详细地讨论这些差异，并将第 11 章和第 14 章 Burgers 示例行为得出的结论，与外部研究论文 [HKT19] 的观察结合起来。

15.1　与现有数值方法的兼容性

显然 PINN 实现相当简单，这是积极的一面；但同时，它与通常用于求解 Burgers 方程等 PDE 的“典型”离散化和求解方法有很大区别。导数通过神经网络计算，因此依赖相当准确的解表示，才能为优化问题提供良好方向。

另一方面，DP 版本本质上依赖与学习过程相结合的数值求解器。因此它需要对手头问题进行离散化，并可通过这一离散化使用现有的、可能强大的数值技术。这意味着可以以已知且可控制的精度高效计算解及其导数。

15.2　离散化

对合适离散化的依赖要求一定程度理解并了解所考虑的问题。次优离散化可能妨碍学习过程，最坏情况下可能导致训练发散。然而，考虑到针对多种物理问题的稳定求解器已有大量理论与实践实现，这通常并非不可克服的障碍。

另一方面，PINN 方法不要求预先选择离散化，因此似乎“没有离散化”。但原文认为这只是表面上的优势：研究者正尝试将离散化“重新纳入”PINN 训练。一般而言，PINN 不可避免地在计算机中产生解，因此必须离散化问题。它们在训练过程中构造这一离散化，方式受底层非线性优化支配，不容易从外部控制。因此，所得精度取决于训练对实际使用情形中问题复杂度的估计，以及训练数据对解中未知区域的近似程度。

例如，如 Burgers 例子所示，PINN 解通常在向后传播时间信息方面有显著困难，这与方法效率密切相关。

15.3　效率

原文认为 PINN 方法还产生根本上更困难的训练任务，导致收敛问题。PINN 通常对解进行局部采样和修正，意味着以权重更新形式给出的修正通常也是局部的。空间与时间边界条件的满足相应地可能

缓慢，实践中导致长时间训练。[1]

DP 方法中选择良好的离散化可以缓解这种行为，并改善梯度信息流。同时，对计算网格的依赖意味着可以很快获得解。给定插值格式或一组基函数，仅需计算网格上的非常局部的邻域，就可以在任意时空点采样解。最坏情况下这可能导致轻微的内存开销，例如反复存储解中大多恒定的值。

另一方面，对于使用全连接网络的 PINN 表示，要获得单个点的解样本，需要完整经过整个网络中可能数量庞大的值。网络实际上必须编码完整的高维解，其大小同样决定导数计算的效率。

15.4 效率（续）

话虽如此，由于 DP 方法能够覆盖大得多的解流形，这些流形的结构通常也难以学习。例如，使用更多迭代训练网络，即对未来进行较长的前瞻时，这通常比短前瞻代表更难学习的信号。

因此，这些训练不仅每次神经网络迭代消耗更多计算资源，也往往需要更长时间收敛。就资源而言，每次计算前瞻可能需要大量仿真步，反向传播步骤通常也需要相近数量的资源。因此，尽管它们有时看起来代价高昂且收敛缓慢，原因通常是需要学习更复杂的信号。

15.5 总结

下表总结物理信息（PI）与可微物理（DP）方法的优缺点：

方法	优点	缺点
PI	通过反向传播计算解析导数；实现容易。	神经网络求值昂贵，导数计算非常昂贵；与现有数值方法不兼容；无法控制离散化。
DP	利用现有数值方法；高效计算仿真及导数。	实现更复杂；需要理解问题以选择合适的离散化。

总的来说，两种方法都很有意思，也有很大潜力。存在许多更复杂的扩展和算法修改，可以改变并改善上述两种方法的各种不利方面。

不过，作者认为在写作时，PINN 方法在性能和与现有数值方法的兼容性方面有明显限制。因此，当我们具有手头问题的知识时——而当我们选择合适的 PDE 模型来约束学习过程时通常如此——使用可微物理求解器训练神经算子，可以显著改善训练过程及所得解的质量。下文将重点关注 DP 变体，并通过更复杂的情形说明其能力。首先考虑一个高效计算瞬态流体仿真时空梯度的情形。[2]

[1] PINN 权重更新是否在空间上局部，以及这些经验判断的适用条件，依架构和训练设置而定；此处忠实保留源书论述，待独立审校。

[2] 本章比较和表格表达的是原书作者的归纳；不同 PINN、神经算子及数值耦合方案的能力和效率需按具体方法、问题与实验条件判断。

第十六章　可微流体仿真

现在以 Navier–Stokes 方程作为物理模型，研究一个更复杂的例子。与 “Navier–Stokes 正向仿真” 一致，我们考虑二维情形。

作为优化目标，考虑前面 Burgers 例子的一个更困难变体：观测密度 s 的状态应在 $n = 20$ 个仿真步后与给定目标吻合。与以前不同，作为被观测量的标记场 s 不能以任何方式改变，只能修改 $t = 0$ 时的初始速度 $\mathbf{u}^0$。这使损失表述中的可观测量与优化期间（或稍后通过神经网络）可以交互的量分离。原文提供的 Colab 链接待核实。

16.1　物理模型

我们使用速度为 $\mathbf{u}$ 的无黏 Navier–Stokes 模型，不含显式黏性项，以及驱动简单 Boussinesq 浮力项 ηd 的烟雾标记密度 s，该项沿 y 方向施加力。由于没有显式黏性，这些方程等价于 Euler 方程。得到

$$\begin{aligned}
\frac{\partial u_x}{\partial t} + \mathbf{u}\cdot\nabla u_x &= -\frac{1}{\rho}\nabla p, \\
\frac{\partial u_y}{\partial t} + \mathbf{u}\cdot\nabla u_y &= -\frac{1}{\rho}\nabla p + \eta d, \\
\text{满足}\quad \nabla\cdot\mathbf{u} &= 0,
\end{aligned}$$

并有被动平流标记密度 s 的附加输运方程

$$\frac{\partial s}{\partial t} + \mathbf{u}\cdot\nabla s = 0.$$

[1]

16.2　问题表述

使用 “模型与方程” 中的记号，上述逆问题可表述为最小化问题

$$\arg\min_{\mathbf{u}^0} \sum_i \left(f(x_{t_e,i}; \mathbf{u}^0) - y^*_{t_e,i} \right)^2,$$

其中 $y^*_{t_e,i}$ 是目标时间 t_e 的参考解样本，而 $x_{t_e,i}$ 表示模拟器在相同采样位置和时间的估计。下标 i 遍历流体求解器中所有离散空间自由度（下面为 32×40）。

[1] 原文将标记密度记为 s，浮力项却写 ηd；分量方程中压力项亦写作矢量梯度 ∇p。均按源保留，符号对应及分量形式待核查。此外离散 step 在平流后持续加 `INFLOW`，不是首行无源被动输运式的纯实现；`smoke*(0,1)` 对应代码中的单位浮力系数，需与源 ηd 区分。

与之前不同，我们不再处理预先计算的量，现在 $x_{t_e,i}$ 本身是复杂的非线性函数。更具体地说，模拟器以初始速度 $\mathbf{u}^0$ 和密度 s^0 开始，通过对离散 PDE $\mathcal{P}$ 求值 n 次计算 $x_{t_e,i}$。这给出模拟的最终状态 $y_{t_e,i} = s_{t_e} = \mathcal{P}^n(\mathbf{u}^0, s^0)$。下文固定 s^0，并以 $\mathbf{u}^0$ 为自由度，因此优化只能改变 $\mathbf{u}^0$，使 $y_{t_e,i}$ 尽可能与参考 $y^*_{t_e,i}$ 一致。[2]

16.3 开始实现

先完成 Python 模块加载。通过导入 `phi.torch.flow`，可以获得在 PyTorch 计算图内工作并提供梯度的流体仿真函数；`phi.tf.flow` 则是 TensorFlow 的替代选择。

```
!pip install --upgrade --quiet phiflow==3.1
from phi.torch.flow import *
import pylab # for visualizations later on
```

16.4 批量仿真

现在可以设置仿真，它将与“Navier–Stokes 正向仿真”中的普通仿真示例一致。不过现在将直接加入一个额外维度，类似神经网络训练中的小批次。为此引入名为 inflow_loc 的具名维度。该维度位于之前空间维度 y、x 之上；后者声明为矢量通道的维度。正如 inflow_loc 名称所示，此维度上的主要差异是不同流入位置，从而得到不同流动仿真。phiflow 的具名维度使在不同张量的匹配维度之间广播信息非常方便。

```
# closed domain
INFLOW_LOCATION = tensor([(12, 4), (13, 6), (14, 5), (16, 5)],
                         batch('inflow_loc'), channel(vector="x,y"))
INFLOW = (1./3.) * CenteredGrid(Sphere(center=INFLOW_LOCATION, radius=3),
    extrapolation.BOUNDARY, x=32, y=40, bounds=Box(x=(0,32), y=(0,40)))
BND = extrapolation.ZERO # closed, boundary conditions for velocity grid below

# uncomment this for a slightly different open domain case
#INFLOW_LOCATION = tensor([(11, 6), (12, 4), (14, 5), (16, 5)],
#                         batch('inflow_loc'), channel(vector="x,y"))
#INFLOW = (1./4.) * CenteredGrid(Sphere(center=INFLOW_LOCATION, radius=3),
#    extrapolation.BOUNDARY, x=32, y=40, bounds=Box(x=(0,32), y=(0,40)))
#BND = extrapolation.BOUNDARY # open boundaries
INFLOW.shape
```

```
(inflow_loc^b=4, x^s=32, y^s=40)
```

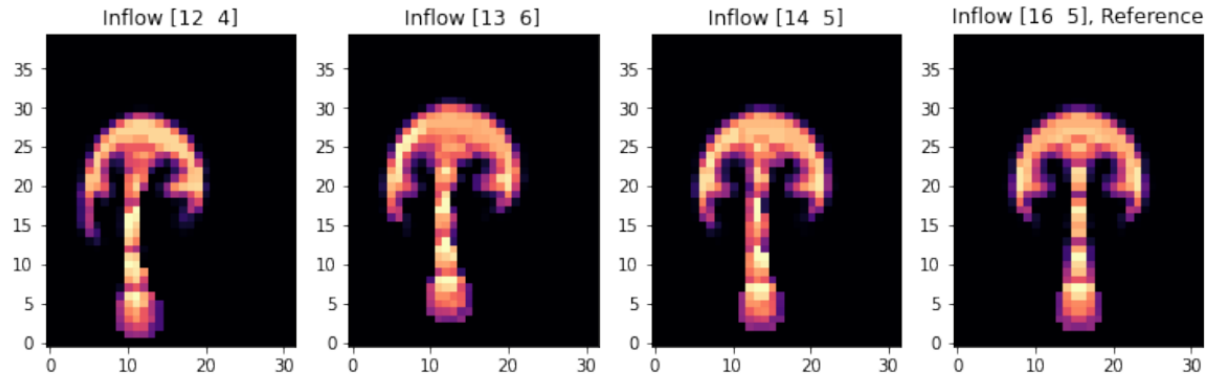
最后一条语句验证，INFLOW 网格除了空间维度 x 和 y 外也有 inflow_loc 维度。可以用.exists 布尔属性测试 phiflow 张量维度是否存在，它可对任意维度名求值。例如，上面的 `INFLOW.inflow_loc.exists` 给出 True，而 `INFLOW.some_unknown_dim.exists` 给出 False。上标 b 表示 inflow_loc 是批次维度。上面的输出为避免字体缺字，以 `^b` 和 `^s` 转写原 PDF 的上标。

[2] 源文对 $x_{t_e,i}$ 和模拟输出 $y_{t_e,i}$ 的角色表述需结合后续代码核查，译文照录。

Phiflow 张量通过名称自动广播到新维度，因此通常不需要重塑操作。例如，可以轻松地对具有不同维度的张量进行加法或乘法。下面将一个交错网格与沿 inflow_loc 维度的全一张量相乘，通过 `StaggeredGrid(...) * math.ones(batch(inflow_loc=4))` 得到具有 x、y、inflow_loc 维度的交错速度。

现在可以从这些不同初始条件轻松仿真若干步。得益于广播，在概述章节中用于单次正向仿真的完全相同代码将产生四个烟雾流入位置不同的仿真。

```
smoke = CenteredGrid(0, extrapolation.BOUNDARY, x=32, y=40,
                     bounds=Box(x=(0,32), y=(0,40))) # sampled at cell centers
velocity = StaggeredGrid(0, BND, x=32, y=40,
                         bounds=Box(x=(0,32), y=(0,40)))
# sampled in staggered form at face centers

def step(smoke, velocity):
    smoke = advect.mac_cormack(smoke, velocity, dt=1) + INFLOW
    buoyancy_force = (smoke * (0, 1)).at(velocity)
    velocity = advect.semi_lagrangian(velocity, velocity, dt=1) + buoyancy_force
    velocity, _ = fluid.make_incompressible(velocity)
    return smoke, velocity

for _ in range(20):
    smoke,velocity = step(smoke,velocity)

# store and show final states (before optimization)
smoke_final = smoke
fig, axes = pylab.subplots(1, 4, figsize=(10, 6))
for i in range(INFLOW.shape.get_size('inflow_loc')):
    axes[i].imshow(smoke_final.values.numpy('inflow_loc,y,x')[i,...],
                   origin='lower', cmap='magma')
    axes[i].set_title(f"Inflow {INFLOW_LOCATION.numpy('inflow_loc,vector')[i]}" +
                      (", Reference" if i==3 else ""))
pylab.tight_layout()
```



图中从左至右给出四个流入位置，最右侧为参考。最后一幅图显示 20 个仿真步后平流烟雾场的状态。

流入位于 $(16, 5)$ 的仿真 [3] 位于最右侧，烟羽笔直，其最终形状将作为下面的参考状态。下面的优化过程将修改其他三个仿真的初始速度，以匹配这一参考。

顺带一提：phiflow 张量会使用创建时的后端跟踪其操作链。例如，用 NumPy 创建的张量将继续使用 NumPy/SciPy 操作，除非同一操作也接收 PyTorch 或 TensorFlow 张量。因此，不时通过例如 `GRID.values.default_backend` 验证张量使用的后端是个好办法。

16.5 梯度

来看看如何从仿真中获得梯度。上面已经完成的第一个简单步骤，是导入 `phi.torch.flow`，以导入用于构建模拟器的可微算子。

现在希望优化初始速度，使所有仿真的最终状态都接近右侧流入位于 $(16, 5)$、即沿 x 居中的参考仿真。为此，在仿真期间记录梯度并定义简单的 L^2 损失函数。使用的损失为 $L = |s_{t_e} - s^*_{t_e}|^2$，其中 s_{t_e} 表示烟雾密度，$s^*_{t_e}$ 是批次中第四个仿真的参考状态，两者均在最后时间步 t_e 求值。计算损失时，通过 `field.stop_gradient()` 将参考状态视为外部常数。如开头所述，s 通过平流方程依赖 $\mathbf{u}$，后者又由 Navier–Stokes 方程给出。因此，经由多个时间步的链，s 依赖初始速度状态 $\mathbf{u}^0$。

在记录梯度之前，初始速度必须具有 inflow_loc 维度，以便拥有四个速度版本的完整“小批次”；稍后优化中其中三个将通过梯度更新。为了获得适当的速度张量，用沿 inflow_loc 批次维度的零张量初始化一个 StaggeredGrid。由于交错网格已有 y、x 和 vector 维度，这给出所需四个维度，如下面的打印语句所验证。

Phiflow 通过需要返回损失值并可选返回状态值的函数，在不同平台上提供统一梯度 API。它使用基于损失函数的接口，为此下面定义 simulate 函数。simulate 计算上述 L^2 误差，并返回 20 个仿真步后演化得到的烟雾和速度状态。

```
initial_smoke = CenteredGrid(0, extrapolation.BOUNDARY, x=32, y=40,
                             bounds=Box(x=(0,32), y=(0,40)))
initial_velocity = StaggeredGrid(math.zeros(batch(inflow_loc=4)), BND, x=32, y=40,
                                 bounds=Box(x=(0,32), y=(0,40)))
print("Velocity dimensions: "+format(initial_velocity.shape))

def simulate(smoke: CenteredGrid, velocity: StaggeredGrid):
    for _ in range(20):
        smoke,velocity = step(smoke,velocity)
    loss = field.l2_loss(smoke - field.stop_gradient(smoke.inflow_loc[-1]) )
    # optionally, use smoother loss with diffusion steps - no difference here,
    # but can be useful for more complex cases
    #loss = field.l2_loss(diffuse.explicit(smoke -
    #    field.stop_gradient(smoke.inflow_loc[-1]), 1, 1, 10))
    return loss, smoke, velocity

Velocity dimensions: (inflow_loc^b=4, x^s=32, y^s=40, vector[unreadable]=x,y)
```

输出中的上标以 ASCII 转写，原 PDF 中矢量维度标记显示为缺字，故标为 `[unreadable]`。

Phiflow 的 field.gradient() 是计算梯度的核心函数。接下来用它获得相对于初始速度的梯度。由于速度是 simulate() 的第二个参数，传入 `wrt=[1]`。Phiflow 也有 field.spatial_gradient 函数，但它计算

张量沿 x、y 等空间维度的导数。

gradient 生成一个梯度函数。作为演示，下一个代码单元用烟雾和速度初态计算一次梯度，最后一条语句打印所得梯度张量一部分的摘要。

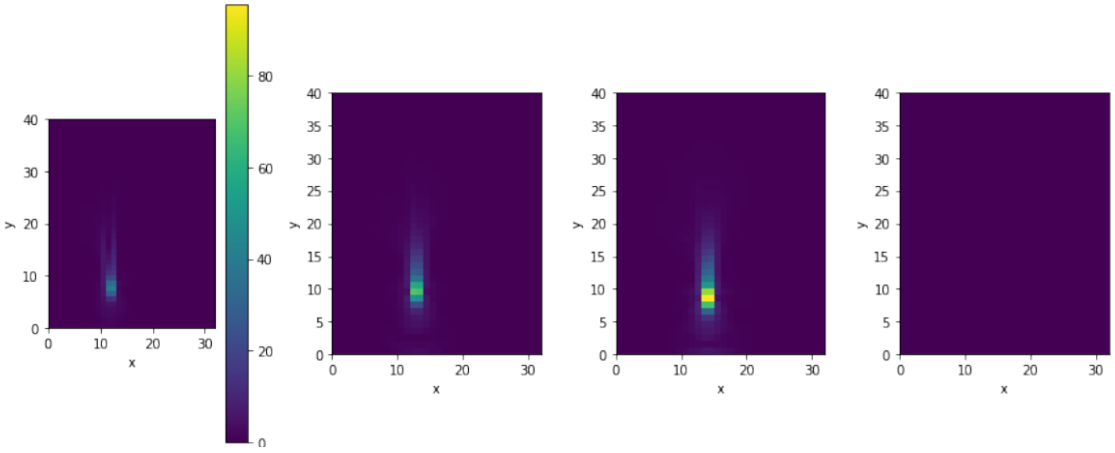
```
sim_grad = field.gradient(simulate, wrt=[1], get_output=False)
(velocity_grad,) = sim_grad(initial_smoke, initial_velocity)
print("Some gradient info: " + format(velocity_grad))
print(format(velocity_grad.values.inflow_loc[0].vector[0])) # one example, location 0,
# x component, automatically prints size & content range
```

```
Some gradient info: StaggeredGrid[(inflow_loc^b=4, x^s=32, y^s=40,
 vector[unreadable]=x,y), size=(x=32, y=40) int64, extrapolation=0]
(x^s=31, y^s=40) 2.61e-08 +/- 8.5e-01 (-2e+01...1e+01)
```

最后两行只打印所得梯度场的一些信息。它自然与速度本身形状相同：是包含四个流入位置的交错网格。最后一行展示如何访问其中一个梯度的 x 分量。

可以使用常规绘图函数查看计算出的梯度内容，例如通过 `velocity_grad.values.inflow_loc[0].vector[0].numpy('y,x')` 将其中一个仿真的 x 分量转换为 NumPy 数组。交互式替代方案是 phiflow 的 view() 函数，它自动分析网格内容并提供界面按钮选择不同查看模式。可以显示箭头、二维速度矢量的单个分量或其大小。但由于交互性质，相应图像不会显示在 Jupyter 之外，因此下面改用 plot() 显示向量长度。

```
# neat phiflow helper function:
v = vis.plot(field.vec_length(velocity_grad)) # show magnitude
<Figure size 864x360 with 5 Axes>
```



原文称左侧第四个梯度为零，因为它已匹配参考。其他三个梯度检测到了初始圆形流入位置的变化，表现为流入圆形周围的正负区域；左侧距离较大的情况也明显更大。[3]

16.6 优化

上面可视化的梯度只是指向损失增大方向的线性化变化。现在可以沿相反方向更新初始速度以最小化损失，并迭代寻找极小点。

[3] 图中为向量长度，非带符号分量；原文“正负区域”和“左侧第四个”的描述需结合实际绘图与索引核查。

这是一项困难任务：由于初始空间密度配置不同，仿真产生不同动力学。优化现在应找到一个初始速度状态，使仿真在 $t = 20$ 时得到与参考仿真相同的状态。因此，经过 20 个非线性更新步，仿真应再现所需的标记密度状态。直接改变标记流入位置更容易达到目标，但为使问题更困难、更有趣，流入位置不是自由度；优化器只能改变初始速度 $\mathbf{u}^0$。

下面的代码单元实现简单的最速梯度下降优化：重新计算梯度函数，并以学习率（步长）LR 多次迭代优化 $\mathbf{u}^0$。field.gradient 的 get_output 参数决定是否返回函数原始结果（这里是 simulate() 的结果），还是只返回梯度。由于跟踪损失随迭代的演化很有意义，我们以 `get_output=True` 重新定义梯度函数。

```
sim_grad_wloss = field.gradient(simulate, wrt=[1], get_output=True) # if we need outputs...
LR = 1e-03
for optim_step in range(80):
    (loss, _smoke, _velocity), (velocity_grad,) = sim_grad_wloss(
        initial_smoke, initial_velocity)
    initial_velocity = initial_velocity - LR * velocity_grad
    if optim_step<3 or optim_step%10==9:
        print('Optimization step %d, loss: %f' % (optim_step, np.sum(loss.numpy()) ))
```

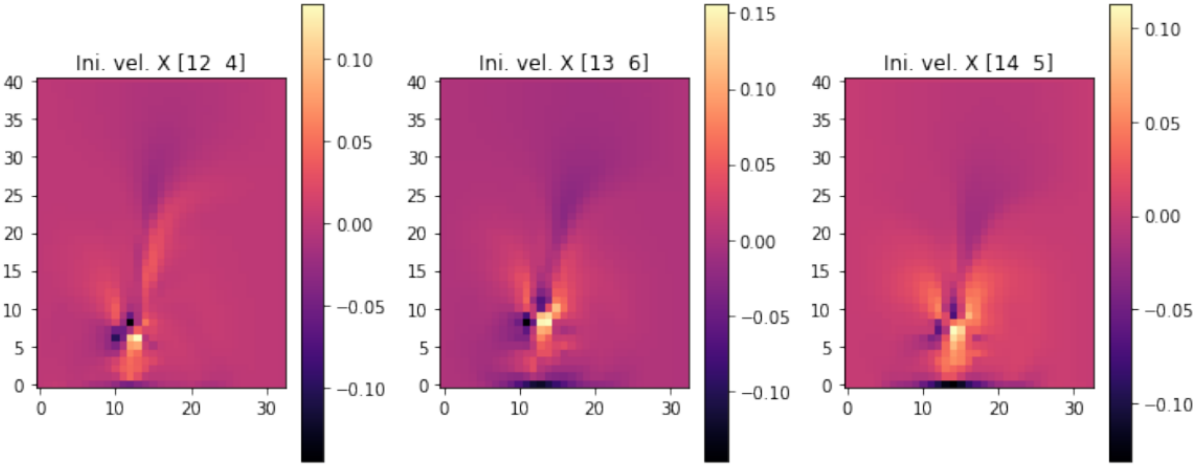
```
Optimization step 0, loss: 298.286163
Optimization step 1, loss: 291.454376
Optimization step 2, loss: 276.057831
Optimization step 9, loss: 233.706482
Optimization step 19, loss: 232.652145
Optimization step 29, loss: 178.186951
Optimization step 39, loss: 176.523254
Optimization step 49, loss: 169.360931
Optimization step 59, loss: 167.578674
Optimization step 69, loss: 175.005310
Optimization step 79, loss: 169.943680
```

损失应显著下降，从接近 300 降到 170 以下。现在也可以可视化优化得到的初始速度。

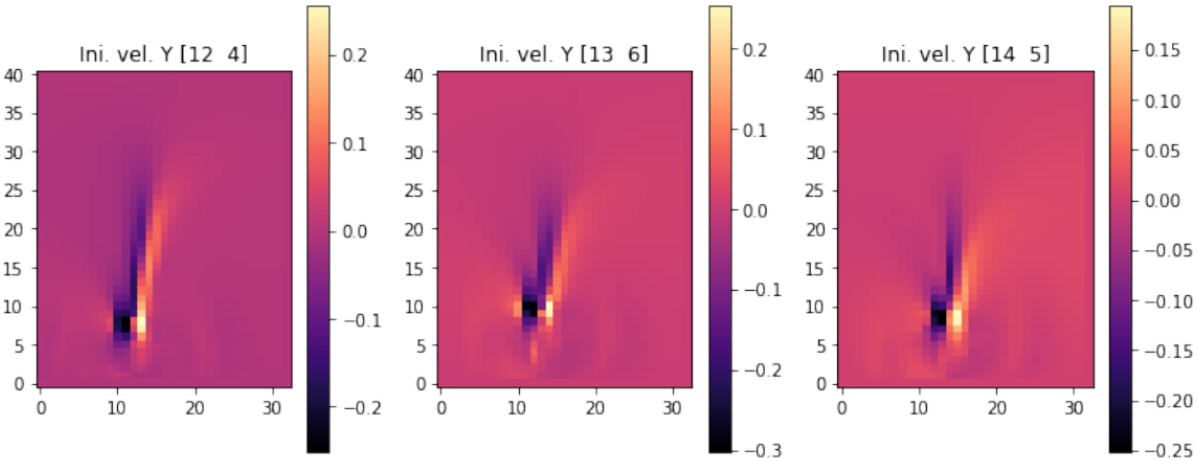
下列图像按 x 分量（第一组）和 y 分量（第二组）显示所得三个初始速度。原文称跳过第四组 inflow_loc[0]，因为它只含零。[4]

```
fig, axes = pylab.subplots(1, 3, figsize=(10, 4))
for i in range(INFLOW.shape.get_size('inflow_loc')-1):
    im = axes[i].imshow(initial_velocity.staggered_tensor().numpy(
        'inflow_loc,y,x,vector')[i,...,0], origin='lower', cmap='magma')
    axes[i].set_title(f"Ini. vel. X {INFLOW_LOCATION.numpy('inflow_loc,vector')[i]}")
    pylab.colorbar(im,ax=axes[i])
pylab.tight_layout()
```

[4] 代码循环实际绘制索引 0、1、2，跳过第四个索引 3；原文所写 inflow_loc[0] 与此不一致。




```
fig, axes = pylab.subplots(1, 3, figsize=(10, 4))
for i in range(INFLOW.shape.get_size('inflow_loc')-1):
    im = axes[i].imshow(initial_velocity.staggered_tensor().numpy(
        'inflow_loc,y,x,vector')[i,...,1], origin='lower', cmap='magma')
    axes[i].set_title(f"Ini. vel. Y {INFLOW_LOCATION.numpy('inflow_loc,vector')[i]}")
    pylab.colorbar(im,ax=axes[i])
pylab.tight_layout()
```



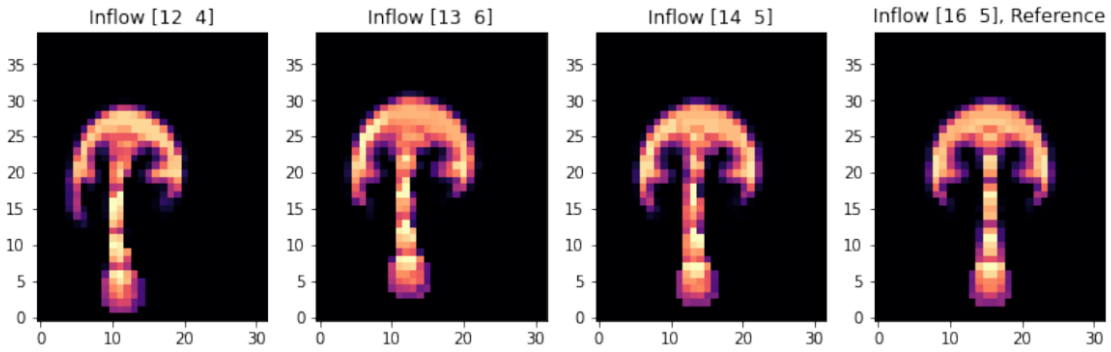
16.7 重模拟

给定每个流入位置的新初始速度条件，还可以可视化完整仿真在 20 步中的表现。这正是优化时每次梯度计算内部发生的过程，也是损失函数所衡量的内容。因此，直观了解优化找到了怎样的解很有意义。

下面使用 initial_velocity 中的新初始条件重新运行正向仿真：

```
smoke = initial_smoke
velocity = initial_velocity
```

```
for _ in range(20):
    smoke,velocity = step(smoke,velocity)

fig, axes = pylab.subplots(1, 4, figsize=(10, 6))
for i in range(INFLOW.shape.get_size('inflow_loc')):
    axes[i].imshow(smoke_final.values.numpy('inflow_loc,y,x')[i,...],
                   origin='lower', cmap='magma')
    axes[i].set_title(f"Inflow {INFLOW_LOCATION.numpy('inflow_loc,vector')[i]}" +
                      (", Reference" if i==3 else ""))
pylab.tight_layout()
```

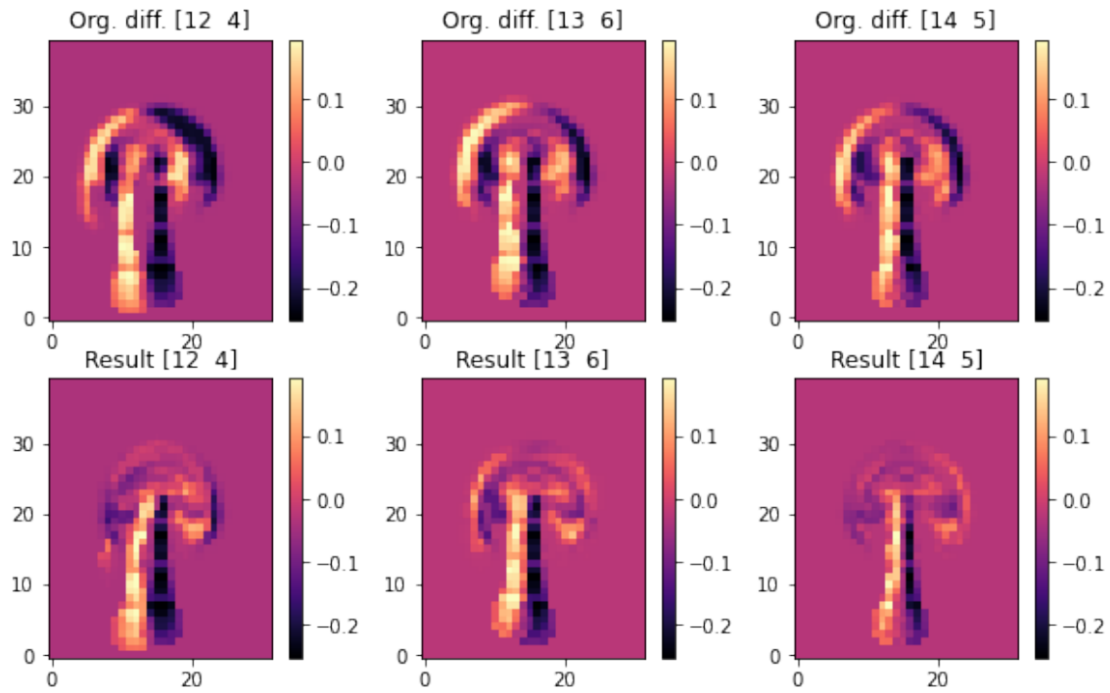


原文称最右图自然相同，因为它是参考；其他三个仿真现在都表现出向右的位移。[5]

由于差异有些细微，下面可视化目标配置与各最终状态的差。下图包含演化后的模拟密度与目标密度之差，因此暗区表示目标应该出现但尚未出现的位置。上排显示初始速度为零时的原始状态，下排显示优化调整初始速度后的版本，故每列可比较优化前（上）和优化后（下）：

```
fig, axes = pylab.subplots(2, 3, figsize=(10, 6))
for i in range(INFLOW.shape.get_size('inflow_loc')-1):
    axes[0,i].imshow(smoke_final.values.numpy('inflow_loc,y,x')[i,...] -
                     smoke_final.values.numpy('inflow_loc,y,x')[3,...],
                     origin='lower', cmap='magma')
    axes[0,i].set_title(f"Org. diff. {INFLOW_LOCATION.numpy('inflow_loc,vector')[i]}")
    pylab.colorbar(im,ax=axes[0,i])
for i in range(INFLOW.shape.get_size('inflow_loc')-1):
    axes[1,i].imshow(smoke.values.numpy('inflow_loc,y,x')[i,...] -
                     smoke_final.values.numpy('inflow_loc,y,x')[3,...],
                     origin='lower', cmap='magma')
    axes[1,i].set_title(f"Result {INFLOW_LOCATION.numpy('inflow_loc,vector')[i]}")
    pylab.colorbar(im,ax=axes[1,i])
```

[5] 上述绘图代码使用的是优化前保存的 `smoke_final`，而非刚刚重模拟得到的 `smoke`；因此确实绘出优化前的缓存状态，不能由该图认证刚重模拟后的结果；代码按源保留。



这些差值图清楚显示，优化使烟羽上部区域对齐得很好。每幅原图（上排）都以黑色光晕显示明显不对齐，而优化后的状态与参考目标烟雾配置大体重合，各烟云前部的差更接近零。[6]

注意，三个仿真都必须在固定流入条件下“工作”，因此不能凭空“产生”标记密度来匹配目标。每个仿真还需考虑非线性模型方程如何在 20 个时间步内改变系统状态。因此优化目标相当困难，在此情形下不可能精确满足约束以匹配参考仿真。例如，烟羽的茎部在优化后仍显示黑色光晕。优化无法移动流入位置，因而需要集中对齐烟羽上部。

16.8 结论

本例说明可微物理方法如何轻松扩展到复杂得多的 PDE。上面我们对完整 Navier–Stokes 求解器 20 步的小批次进行了优化。

这是引入神经网络的强大基础。上面的自由度仍是规则网格，我们共同求解了单个逆问题。虽然确有三个情形作为小批次求解，但设置仍然表示直接优化。因此，与“使用 PINN 优化 Burgers 方程”中的 PINN 示例一样，这里并没有真正处理机器学习任务。不过，DP 训练允许与神经网络进行多种灵活组合，这将是后续章节的主题。

16.9 下一步

基于上述代码示例，建议尝试：

[6] 原代码调用 `imshow` 后未将返回值赋给 `im`，却用已有的 `im` 创建色条；该旧变量明确沿用上一速度 Y 图最后一次绘图的 norm/cmap，不对应当前差值图，不能据此色条读取差值量级。

- 修改仿真设置，使四个实例之间差异更大、运行更久，或采用更细的空间离散化，即更大的网格。注意这会使优化问题更困难，因此用这一简单设置可能不会直接收敛。
- 作为更大的改动，加入多分辨率优化以处理差异更大的情况，即先用粗离散化求解，再以其解作为更细离散化的初始猜测。

第十七章　将可微物理纳入神经网络训练

现在研究将可微物理（DP）设置纳入神经网络。将 DP 方法用于学习应用时，DP 和神经网络构件的组合具有很大灵活性。由于某些差别比较细微，下文将进一步详细讨论。我们尤其关注多次重复计算 PDE 和神经网络的求解器，例如计算物理系统随时间的多个状态。在经典数值方法中，这称为迭代时间步进方法；在 AI 背景下，则是自回归方法。

提示：修正与预测　最适合用 DP 方法处理的问题非常基础。不完善的物理模型与改进项的组合，在经典领域有许多不同名称：流体动力学和湍流中的闭合问题、材料科学中的均匀化或粗粒化，以及气候和天气领域中的参数化。

下文将所有包含“神经网络＋求解器”的任务统称为修正任务，与推断时不涉及求解器的纯预测任务相对。

回顾一下前面关于结合神经网络与 DP 算子的图。这些算子在图中看起来像损失项：它们通常没有权重，只提供影响神经网络权重优化的梯度。

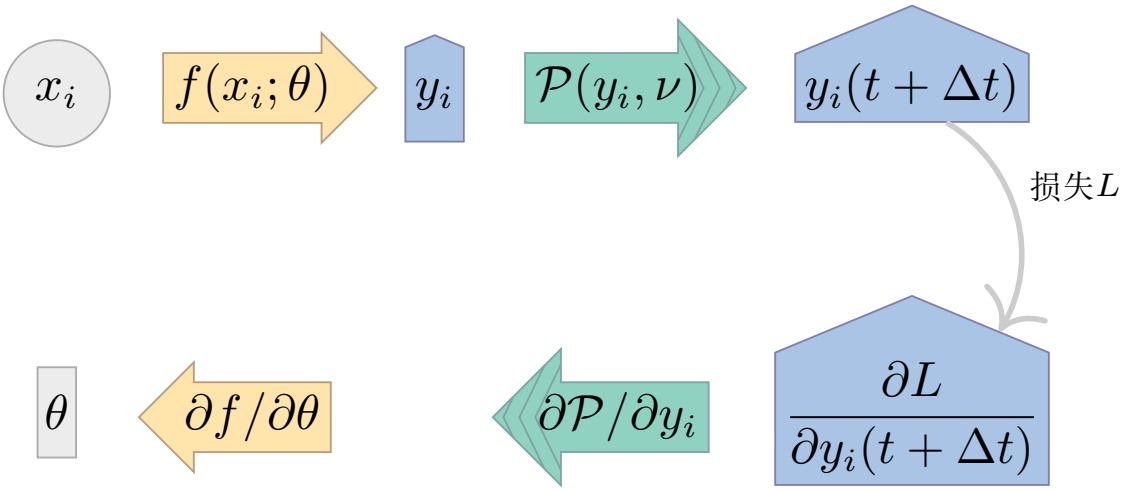


图 17.1: 前几章描述的 DP 方法。网络产生 PDE 求解器 $\mathcal{P}$ 的输入，后者在反向传播步骤中为训练提供梯度。译注：按源图重建为可编辑数学图；黄色箭头为网络及其导数，绿色箭头为物理求解器及其导数。

可以将这一设置看作网络接收关于其输出如何影响 PDE 求解器结果的信息。也就是说，梯度将提供如何产生使损失最小的网络输出的信息。与前述“物理损失项”类似，例如这可能意味着在时间演化中维持守恒定律或更一般的基于 PDE 的约束。

17.1 交换顺序

然而使用 DP 时，没有真正的理由局限于这一设置。例如可以设想交换神经网络和 DP 组件，得到下列结构：

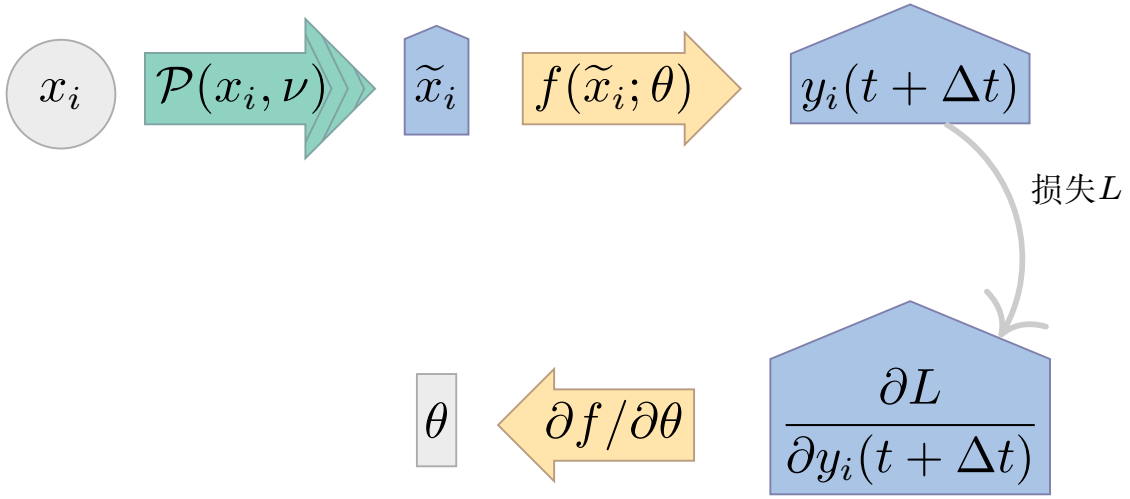


图 17.2: PDE 求解器产生输出，再由神经网络处理。译注：图中 Loss L 为损失；按源图重建。

在这种情况下，PDE 求解器本质上是即时数据生成器。这并不一定总是有用：由于 PDE 求解器不受网络影响，可以预先计算相同输入来替代这一设置。因此不需要通过 $\mathcal{P}$ 反向传播，可以用简单“加载”函数替代它。

另一方面，训练期间随机采样输入参数并运行 PDE 求解器，可以很好地采样输入数据分布。如果掌握输入如何变化的真实范围，这可以改善网络训练。若实现正确，求解器还可以减少存储和加载大量数据的需求，而在训练期间更快地产生数据，例如直接在 GPU 上生成。近期方法在主动学习背景下探索这一方向。

然而，这个版本没有利用可微求解器提供的梯度信息，因此下面的变体更有意思。

17.2 循环求值

一种特别合理的组合是展开模拟器时间步进过程的迭代，并让系统状态受神经网络影响。一般而言，只要维度兼容，神经网络层和 DP 算子的组合没有被禁止的形式。

在展开的情形中，前向过程中计算可能很长的一系列 PDE 求解器步骤。在求解器步骤之间，神经网络修改系统状态，然后用该状态计算下一 PDE 求解器步。反向传播时，向后经过所有步骤，计算对损失函数的贡献；损失可在执行链中任意一个或多个位置计算。同时，梯度信息通过 DP 和神经网络算子反向传播。这种求解器迭代的展开，实质上向网络反馈其“动作”如何影响物理系统状态及所得损失。下图概览了这种组合形式。

由于这一过程具有迭代性质，误差开始时很小，随后对于 Jacobian 特征值大于 1 的模态，会在迭代过程中逐渐指数增长。因此，在单次求值中极难检测，例如较简单的监督训练设置便是如此。相反，训练时向网络反馈误差如何随迭代演化至关重要。此外，这类迭代情形无法预先计算状态，因为迭代依赖网络状态，而网络状态在训练前未知并在训练期间变化。这是经典机器学习中的数据偏移问题。因此，

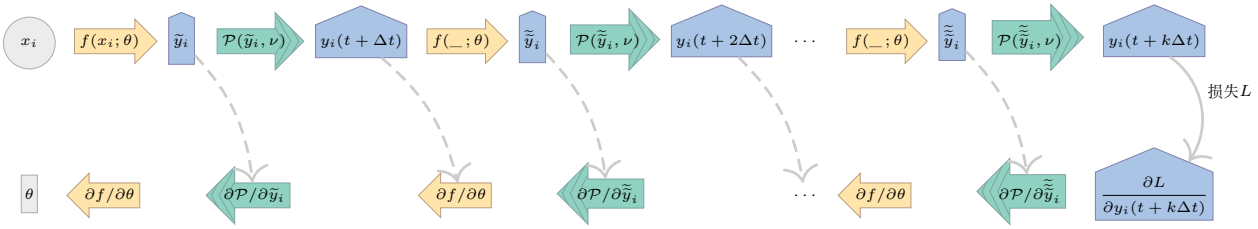
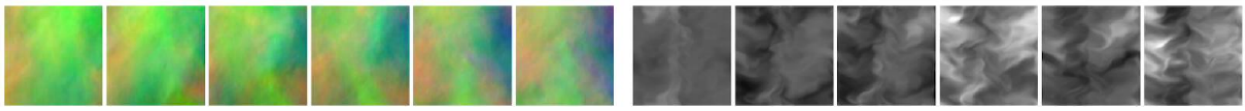


图 17.3: 对 k 次求解器迭代，以交错的 DP 和神经网络操作进行时间步进。灰色虚线箭头表示可选的中间损失项求值（与最后一步 k 的灰色实线箭头相似）；神经网络的中间输出用波浪号表示。译注：按源图重建，图中使用 y_i 及逐层增加的波浪号，正文则沿原文使用 x 记状态。

在循环设置中，基于 DP 的训练对于向网络提供关于当前状态如何影响求解器迭代、以及相应地应如何改变权重以更好实现学习目标的梯度至关重要。

具有许多时间步的 DP 设置可能难以训练：梯度需要反向传播经过全部 PDE 求解器和神经网络求值链，而每一步通常都代表复杂非线性函数。因此步数较多时，梯度消失和爆炸问题会使训练困难。关于缓解这一问题的实际考虑将在“使用神经算子减少数值误差”中讨论。[1]



17.3 神经网络与求解器的组合

此前忽略了一个问题：如何将网络输出合并到迭代求解过程中？在上图中，网络 f 看起来产生了物理系统的完整状态，作为 $\mathcal{P}$ 的输入。也就是说，对于第 j 步状态 $x(t+j\Delta t)$，网络给出中间状态 $\tilde{x}(t+j\Delta t)=f(x(t+j\Delta t);\theta)$，求解器据此产生下一步的新状态 $x(t+(j+1)\Delta t)=\mathcal{P}(\tilde{x}(t+j\Delta t))$。

虽然这种方法可行，但并非在所有情形下都是最佳选择。特别是当网络只需产生当前状态的修正时，可以复用当前状态的一部分，避免将网络资源，即 θ 的一部分，分配给推断已经正确的部分。类似 U-Net 中的跳跃连接和 ResNet 的残差，这时最好使用合并 x 与 $\tilde{x}$ 的算子 $\circ$，即 $x(t+(j+1)\Delta t)=\mathcal{P}(x(t+j\Delta t)\circ\tilde{x}(t+j\Delta t))$。最简单情况下可定义 $\circ$ 为加法，此时 $\tilde{x}$ 表示对 x 的加性修正。简言之，通过计算 $\mathcal{P}(x+\tilde{x})$ 得到下一状态，网络只需更新 x 中尚不满足学习目标的部分。

一般而言，$\circ$ 可以是任意可微算子，例如乘法或积分格式。与损失函数一样，这一选择取决于问题，但加法通常是一个好的起点。

[1] 译注：误差是否指数增长依赖具体局部 Jacobian 的谱及时间变化；此处保留源文的模态描述，不能仅凭单步特征值推断任意非定常系统的长期行为。

17.4 方程形式

接下来将前述描述形式化，具体回答：以 Jacobian 表示时，θ 的更新步是什么样的？给定以下标 i 表示的小批次和损失函数 L，用 k 表示一次迭代展开的总步数。为简化记号，$x_{i,j} = x_i(t + j\Delta t)$ 表示批次 i 在时间步 j 的状态 x。于是网络权重梯度可写为

$$\frac{\partial L}{\partial \theta} = \sum_i \sum_{m=1}^{k} \left[\frac{\partial L}{\partial x_{i,k}} \left(\prod_{n=k}^{m+1} \frac{\partial x_{i,n}}{\partial x_{i,n-1}} \right) \frac{\partial x_{i,m}}{\partial \tilde{x}_{i,m-1}} \frac{\partial \tilde{x}_{i,m-1}}{\partial \theta} \right]. \tag{17.1}$$

乍看之下并不直观，但表达式结构相当简单：对 i 的第一重求和累积小批次所有条目；外层对 m 的求和（方括号）考虑从 1 到 k 的所有时间步。对于每个 m，沿途将所有 Jacobian 相乘（括号内下标 n），从最终状态 k 回溯至每个 m。沿途每一步由对应时间步相对于 x 的 Jacobian 组成，后者又依赖网络修正 $\tilde{x}$（未展开写出）。

在神经网络的每个最后步骤 m 处，我们“分支”，确定第 m 个时间步中网络输出 $\tilde{x}$ 及其权重 θ 方面的变化。不同 m 的贡献相加，得到训练优化器使用的最终更新 $\Delta\theta$。

必须记住，当 m 较大时，循环应用的 $\mathcal{P}$ 和 f 的 Jacobian 会强烈影响后续时间步的贡献，因此稳定训练尤其关键，以防止梯度爆炸。这一主题后面将多次讨论。[2]

实现方面，所有深度学习框架都会复用不同 m 的重复重叠部分。这由反向传播求值自动处理；实践中求和将按从大到小的 m 计算，使我们在转向较小 m 时可以“忘掉”后续步骤。因此反向传播确实增加计算成本，但只要有合适算子计算 $\mathcal{P}$ 的导数，其成本通常与前向过程同一量级。

17.5 通过求解器步骤反向传播

现在搭建了所有这些机制，一个值得问的问题是：“使用可微物理模拟器训练究竟能带来多大改进？难道不能像标准循环训练那样，直接展开监督设置而不使用可微求解器吗？”或者换一种说法，通过求解器的多个步骤反向传播到底能获得多少收益？

原文的简短回答是：很多！下面介绍 List 等人 [LCT22] 的湍流混合层评估，以说明这一差异。在深入细节前，需要指出这一比较使用了可微的二阶半隐式流体求解器，以及一组自定义湍流损失项。因此它不是玩具问题，而是展示了可微性对复杂现实情形的影响。

这个例子的优点是可以用湍流问题成熟的统计测量进行评估，并据此量化差异。流动的能谱通常是起点，但这里跳过它，读者可参见原论文 [LCT22]；我们转而关注两个信息更丰富的指标。下图显示 Reynolds 应力和湍动能（TKE），两者均以流场某一截面上可分辨尺度的量表示。参考解以橙色圆点显示。

特别是在彩色箭头指出的区域，“展开的监督训练”对应的红线明显更偏离参考解。两个测量都取自流体求解器结合训练后网络运行 1024 个时间步的结果。因此两种解都相当稳定，并明显优于图中蓝色的未修改求解器输出。

定性比较涡量场的可视化结果时，差异也十分明显：

[2]译注：保留原式的倒序乘积记号；应按从 $n = k$ 到 $n = m + 1$ 的有序矩阵乘积理解，$m = k$ 时为空乘积。该式对应损失只依赖末态、网络权重在步间共享，且末项对 θ 的导数只取固定输入时的显式依赖；多处中间损失须另累积相应项。原文把梯度的累积称作最终更新，实际更新还依赖优化器和步长。乘积长度由 $k - m$ 决定，不能只据原文“m 较大”的一句话判断放大程度。

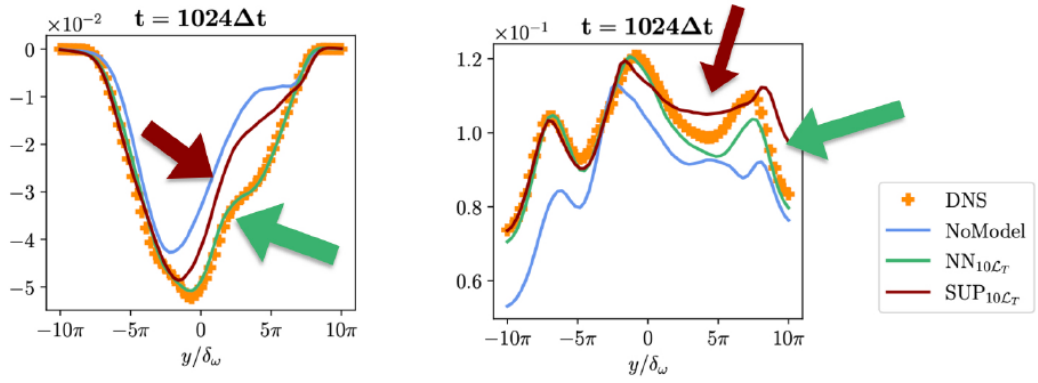


图 17.4: 使用湍流指标进行定量评估：左为 Reynolds 应力，右为湍动能。未使用可微求解器训练的红线比 DP 训练的绿线更偏离真实值（橙色圆点）。图例译解：DNS 为直接数值仿真，NoModel 为无模型结果。

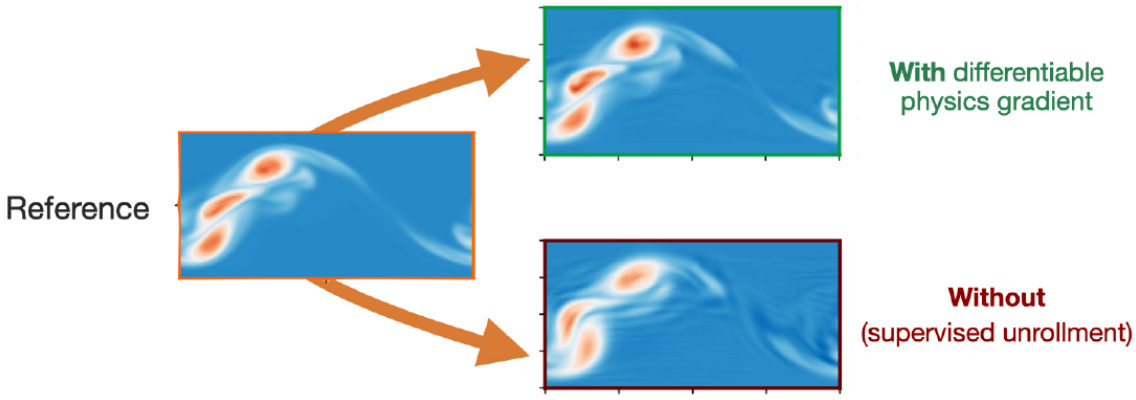


图 17.5: 以涡量进行定性视觉比较。使用可微物理求解器训练（上）的结果，更好地保留了直接数值仿真所得参考解的结构。图中英文译解：字样分别为参考、带可微物理梯度和不带可微物理梯度（监督展开）。

无论是否使用求解器梯度，两种版本都从展开中显著受益；这一比较展开了 10 步。但没有 DP 的监督变体在训练时无法使用关于网络影响的较长期信息，因此能力受到限制。使用可微求解器训练的版本接收完整 10 个展开步骤的反馈，从而能够推断修正，提高最终网络增强求解器的精度。

作为展望，这一例子还突出了将神经网络纳入求解器的实践优势：可以衡量普通仿真达到某一湍流统计精度需要多长时间。在此例中，普通仿真所需时间超过带神经网络求解器的 14 倍 [LCT22]。虽然这只是第一个数据点，但训练完网络后，性能方面的实际改进能够或多或少地直接实现，仍是值得看到的结果。[3]

17.6 替代方案：噪声

其他工作提出在训练时用噪声扰动输入及迭代过程 [SGGP+20]，与 dropout 等正则化方法有些相似。这有助于防止对训练状态过拟合，从而有助于稳定迭代求解器的训练。

但噪声在本质上非常不同：它通常没有方向，因此不如使用仿真实际演化进行训练那样准确。所以，噪声可以作为容易过拟合的训练设置的良好起点。不过，如有可能，最好通过 DP 方法将实际求解器纳入训练循环，向网络提供有关系统时间演化的反馈。

原文认为，在当前进展下，生成建模方法（去噪扩散或流匹配）为纳入噪声提供了基础更充分的方法。我们将在“无条件稳定性”中更详细地讨论这一主题。

17.7 复杂示例

下面几节将给出更复杂情形的代码示例，展示可微物理训练能够实现什么。

首先，按照 Um 等人 [UBH+20]，展示一个使用深度学习表示数值仿真误差的场景。这是一项非常基础的任务，需要学习得到的模型与数值求解器紧密交互，因此是必须将数值求解器引入深度学习循环的典型例子。

接着，按照 Holl 等人 [HKT19]，展示如何让神经网络求解困难的逆问题，即 Navier–Stokes 仿真的长期控制。这项任务需要长期规划，因此需要两个网络：一个预测演化，另一个采取动作达到预期目标。后面将在“使用强化学习控制 Burgers 方程”中，将此方法与使用强化学习的另一深度学习变体比较。

两个例子所需资源都明显多于之前的例子，因此这些 notebook 预计运行更久；处理这些示例时使用检查点是个好主意。

[3] 译注：这些精度和速度比较属于原文所述 [LCT22] 的特定设置；本译稿未独立复现，不能据此推广到所有求解器或硬件。

第十八章　用神经算子减少数值误差

本例以连续 PDE $\mathcal{P}^*$ 离散化时产生的数值误差为目标，也就是构造 $\mathcal{P}$ 时产生的误差。这一方法将说明：尽管没有闭式描述，离散化误差通常仍是具有规则、重复结构的函数，因此可由离散化的神经算子学习。训练完成后，可以局部计算神经网络（NN），改进 PDE 求解器的解，即减少其数值误差。所得方法是一种混合方法：它始终先进行一次（粗分辨率的）PDE 求解，再用 NN 推断的修正在运行时改进结果。

几乎所有数值方法都含有某种迭代过程：显式求解器随时间反复更新，隐式或稳态求解器则在一次更新步内部迭代。原文在 “此处” 提供了后一种情形的例子，但下面以第一种情形，即随时间迭代为目标。原文另标有 “在 Colab 中运行”。[1]

18.1　问题表述

在减少误差的问题中，可微物理求解器至关重要，因为学习过程需要将求解器的更新考虑在内。基于监督学习或 PINN 的训练无法实现这种交互。监督 NN 即使只有很小的推断误差，也会随时间累积，导致数据分布不同于预先计算的数据分布。这种分布偏移会使结果次优，甚至使求解器发散。

为了学习误差函数，考虑同一 PDE $\mathcal{P}^*$ 的两种不同离散化。第一种为参考版本，假定其准确且保真度高：离散化为 $\mathcal{P}_r$，解为 $\mathbf{r} \in \mathcal{R}$，其中 $\mathcal{R}$ 表示 $\mathcal{P}_r$ 的解流形。与之并行的是同一 PDE 的低保真度求解器，称为源版本，因为它正是以后要与 NN 交互的求解器。类似地，记为 $\mathcal{P}_s$，解为 $\mathbf{s} \in \mathcal{S}$。训练后，源求解器 $\mathcal{P}_s$ 与训练好的算子结合，构成混合求解器，产生改进的解，即更接近 $\mathcal{P}_r$ 所产生的解。

假定 $\mathcal{P}$ 将解推进一个时间步 Δt，并用上标表示连续 n 步：

$$\mathcal{P}_s^n(\mathcal{T}\mathbf{r}_t) = \mathcal{P}_s\big(\mathcal{P}_s(\cdots \mathcal{P}_s(\mathcal{T}\mathbf{r}_t)\cdots)\big).$$

相应的仿真状态为 $\mathbf{s}_{t+n} = \mathcal{P}^n(\mathcal{T}\mathbf{r}_t)$。这里假定存在映射算子 $\mathcal{T}$，将参考解转移到源流形，例如简单的下采样操作。特别是对较长序列，即较大的 n，源状态 $\mathbf{s}_{t+n}$ 会偏离相应参考状态 $\mathbf{r}_{t+n}$。以下将用基于 NN 的算子处理这个问题。

与之前一样，用 L^2 范数量化偏差，即误差函数

$$e(\mathbf{s}_t, \mathcal{T}\mathbf{r}_t) = \|\mathbf{s}_t - \mathcal{T}\mathbf{r}_t\|_2.$$

学习目标是训练修正算子 $\mathcal{C}(\mathbf{s})$，使应用修正后的解比原来未修改的（源）解误差更低：

$$e\big(\mathcal{P}_s(\mathcal{C}(\mathcal{T}\mathbf{r}_t)), \mathcal{T}\mathbf{r}_{t+1}\big) < e\big(\mathcal{P}_s(\mathcal{T}\mathbf{r}_t), \mathcal{T}\mathbf{r}_{t+1}\big).$$

[1] 译注：冻结 PDF 这些文字虽有链接颜色，但对应页没有 PDF 链接注释，无法从该文件恢复 “此处” 和 Colab 的目标地址；本译稿不猜测地址。下面全部代码、注释、输出和图像均依据冻结源转录，未安装示例依赖、下载数据或权重，也未运行训练和仿真。

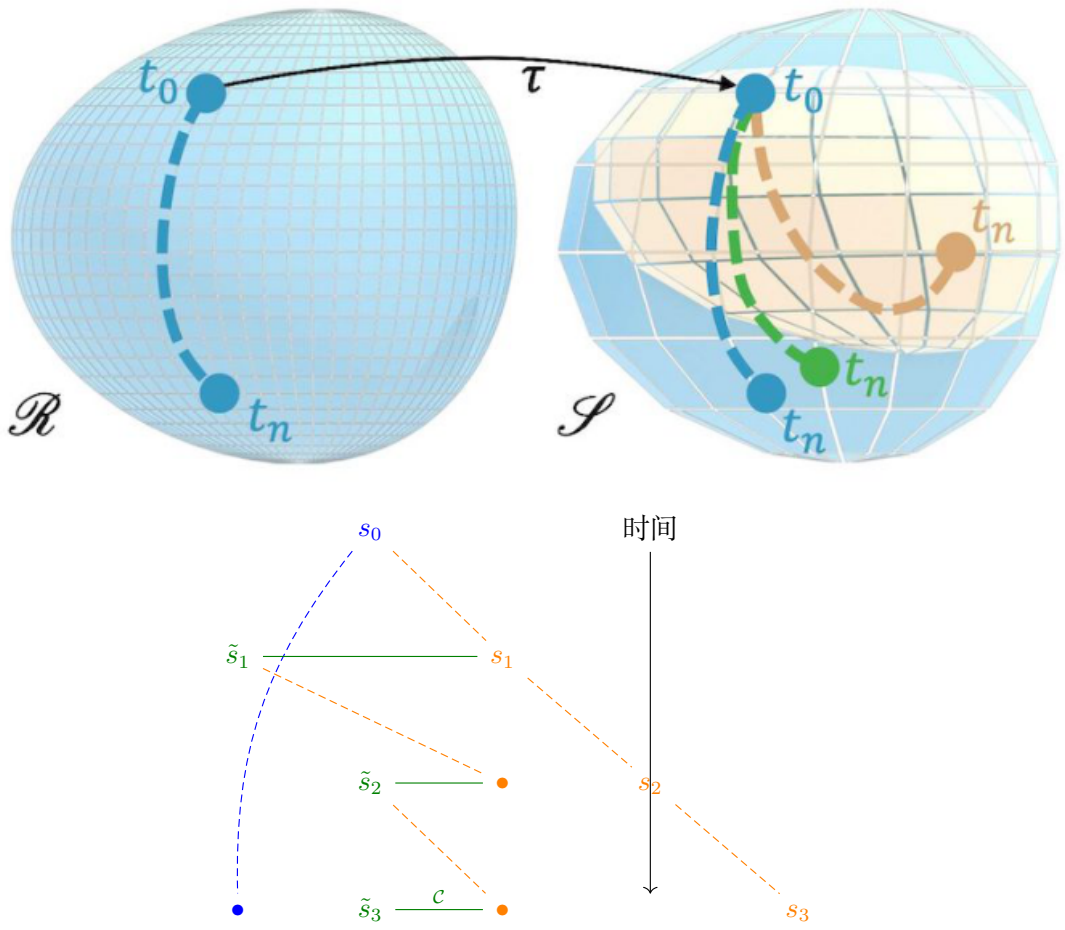


图 18.1: 粗分辨率流形与参考流形的示意。图中 Time 表示时间；蓝色表示参考轨迹及其映射，橙色表示源轨迹，绿色表示修正后的轨迹。原图包含曲面几何、轨迹和映射，按冻结源裁图保留。

修正算子 $\mathcal{C}(\mathbf{s} \mid \theta)$ 由权重为 θ 的深度神经网络表示，接收状态 $\mathbf{s}$，推断一个维度相同的加性修正场。为区分原始状态 $\mathbf{s}$ 和修正后的状态，用加波浪号的 $\tilde{\mathbf{s}}$ 表示后者。整体学习目标于是成为

$$\arg\min_{\theta} \left((\mathcal{P}_s \mathcal{C})^n (\mathcal{T}\mathbf{r}_t) - \mathcal{T}\mathbf{r}_{t+n} \right)^2 .$$

为简化记号，这里省略了对不同样本的求和，即之前写法中的 i。上式中一个关键而容易忽略的细节是：修正依赖已经修改的状态，即它是 $\tilde{\mathbf{s}}$ 的函数，故写为 $\mathcal{C}(\tilde{\mathbf{s}} \mid \theta)$。这些状态实际上在训练期间随时间演化，事先并不存在。[2]

简要概括 我们将训练神经算子 $\mathcal{C}$，减少模拟器相对于更准确参考解的数值误差。关键是把源求解器实现为可微物理算子，从而为改进 $\mathcal{C}$ 的训练提供梯度。

18.2 开始实现

以下示例复现论文 *Solver-in-the-loop: learning from differentiable physics to interact with iterative pde-solvers* [UBH+20] 的一个实验，更多细节见该论文附录 B.1 节。[3]

首先导入所需库，主要是 phiflow 和 PyTorch，并先处理好设备设置，以便专注于有意思的部分……

```
try:
    import google.colab  # 确保当前位于 Colab 中
    !pip install --upgrade --quiet phiflow==3.3
    #!pip install --upgrade --quiet git+https://github.com/tum-pbs/PhiFlow@develop

    # 用于 pbdl-dataset:
    !pip install --upgrade --quiet git+https://github.com/tum-pbs/pbdl-dataset

except ImportError:
    print("This notebook is running locally, please make sure the necessary pip packages are
    ↪  installed.")
    pass
```

原书输出如下，意思是“此 notebook 在本地运行，请确保已安装所需 pip 包”；输出中的拼写 `isntalled` 按源保留。

```
This notebook is running locally, please make sure the necessary pip packages are isntalled.
```

```
import os, sys, logging, argparse, pickle, glob, random, pylab, time
from tqdm import tqdm
from phi.torch.flow import *

random.seed(42)
np.random.seed(42)
```

[2]译注：以上按源保留记号：一处 $\mathcal{P}^n$ 未写下标 s，目标式用向量差的平方而未显式写范数；同时 $\mathcal{C}$ 既被称为修正算子，又被描述为输出加性修正场。若网络只输出增量，完整状态修正还需加回输入。理论式按通常复合顺序先修正、后求解，而后面的代码明确先求解、后加网络修正。两种写法没有在源文中统一，不能直接当作逐步完全相同的实现。

[3]译注：“复现”是源书对其示例的描述，本译稿没有执行该实验。[UBH+20] 沿用源文标签，完整文献接入仍待全书阶段完成。

```
math.seed(42) # phiflow 随机种子
math.set_global_precision(32) # 单精度

USE_CPU = 0
TORCH.set_default_device("GPU")
device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
if USE_CPU > 0:
    device = 'cpu'
device = torch.device(device)
print("Using device: "+str(device))
```

原书输出表明使用的设备为 `cuda:0`：

```
Using device: cuda:0
```

顺便也设定随机种子——显然，42 是这里的终极选择。[4]

18.3 仿真设置

现在设置源仿真 $\mathcal{P}_s$。本 notebook 不处理 $\mathcal{P}_r$：来自高保真度求解的下采样参考数据已包含在训练数据集中。生成参考数据时，空间与时间离散化都比源仿真细四倍。下面专注于源求解器与 NN 的交互。

下面的 `KarmanFlow` 求解器模拟一个相对标准的 Navier–Stokes 尾流问题：矩形区域中有球形障碍物，并用显式黏性求解来获得不同 Reynolds 数。其几何设置如图 18.2。

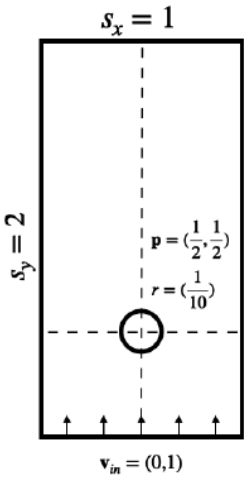


图 18.2: 尾流问题的区域设置；实现中的尺寸额外乘以 100。图中 $s_x = 1$、$s_y = 2$；障碍物中心 $\mathbf{p} = (\frac{1}{2}, \frac{1}{2})$，半径 $r = \frac{1}{10}$，底部流入速度 $\mathbf{v}_{\text{in}} = (0, 1)$。

求解器通过预乘掩膜 `vel_BcMask` 对 y 方向速度施加流入边界条件，在仿真步中设定区域底部的 y

[4]译注：原 PDF 该句末和本章测试结论末的装饰性符号显示异常，无法可靠辨认；没有据文本提取中的乱码猜测其含义。

分量。该掩膜由 phiflow 的 `HardGeometryMask` 创建，可正确初始化交错网格各分量在空间上错开的条目。仿真步很直接：先计算黏性、流入和平流的贡献，最后通过隐式压力求解使所得运动无散。

```
# 补偿原 solver-in-the-loop 论文中原始标度的因子
RE_FAC_SOL = 10/(128*128)

class KarmanFlow():
    def __init__(self, domain):
        self.domain = domain
        self.vel_BcMask = self.domain.staggered_grid(
            HardGeometryMask(Box(y=(None, 5), x=None)))

        # 必要时随区域大小缩放!
        self.inflow = self.domain.scalar_grid(Box(y=(5,10), x=(25,75)))
        self.obstacles = [Obstacle(Sphere(
            center=tensor([50, 50], channel(vector="y,x")), radius=10))]

    def step(self, marker_in, velocity_in, Re, res, buoyancy_factor=0, dt=1.0):
        velocity = velocity_in
        marker = marker_in
        Re_phiflow = Re / RE_FAC_SOL # 为 phiflow 重新缩放

        # 黏性
        velocity = phi.flow.diffuse.explicit(
            u=velocity, diffusivity=1.0/Re_phiflow*dt*res*res, dt=dt)

        # 流入边界条件
        velocity = velocity*(1.0 - self.vel_BcMask) + self.vel_BcMask * (1,0)

        # 平流
        marker = advect.semi_lagrangian(marker+ 1. * self.inflow, velocity, dt=dt)
        velocity = advected_velocity = advect.semi_lagrangian(
            velocity, velocity, dt=dt)

        # 质量守恒 (压力求解)
        pressure = None
        velocity, pressure = fluid.make_incompressible(velocity, self.obstacles)
        self.solve_info = {'pressure': pressure, 'advected_velocity': advected_velocity}

        return [marker, velocity]
```

这里的标记密度表示被动平流的标记场，并非流体的密度。下面修正任务只关注速度，跟踪标记密度仅用于可视化。[5]

[5]译注：源文称“球形”，本例实际为二维，图示为圆形截面。黏性调用在 `diffusivity` 中含 `dt`，同时又传入 `dt=dt`，其一般时间步标度需另核，默认 `dt=1.0` 时不暴露该差异；参数 `buoyancy_factor` 在此函数中未使用。均照录而未改变实现。

18.4 网络与转换函数

另定义一个简单神经网络表示算子 $\mathcal{C}$。使用全卷积网络，即不含任何全连接（MLP）层的网络。用 phiflow 的网络工具设置 `conv_net`，层数由 `layers` 列表指定。网络输入是三个场：

- 表示 x, y 速度的两个场；
- 作为常数通道的 Reynolds 数。

输出为：

- 一个包含 x, y 速度的二分量场。

在卷积网络中，输入维度由输入张量确定，后者有三个通道：u、v 和 Re。随后数据经过一系列卷积层和激活，每层有 32（及 48）个特征，最后在输出处减少到两个通道。下面代码还用增益为 0.1 的均匀 Xavier 初始化重新初始化卷积权重。这样可避免一开始数值过大，从而简化训练，并能直接启用多步展开。否则就需要加入课程安排，先训练一段时间，找到修正的合适范围，再通过展开将网络用于更长序列。

同时，当然也要检查网络参数数量；它是估计 NN 计算成本的重要指标。

```
layers = [32,32,32] # 小网络
#layers = [32,48,48,48,32] # 取消注释，使用稍大且更深的网络
#network = conv_net(in_channels=3,out_channels=2,layers=layers) # 更简单的变体
network = res_net(in_channels=3,out_channels=2,layers=layers)
print(network)

# 重新初始化
import torch.nn as nn
for m in network.modules():
    if isinstance(m, nn.Conv2d):
        nn.init.xavier_uniform_(m.weight, gain=0.1)

print("Total number of trainable parameters: "+ str(sum(p.numel() for p in network.parameters())))
```

原书打印的完整网络结构及可训练参数总数如下。[6]`Identity` 表示恒等映射；`Conv2d` 为二维卷积。

```
ResNet(
  (Res_in): ResNetBlock(
    (sample_input): Conv2d(3, 32, kernel_size=(1, 1), stride=(1, 1))
    (bn_sample): Identity()
    (bn1): Identity()
    (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (bn2): Identity()
    (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  )
  (Res1): ResNetBlock(
```

[6]译注：正文以 `conv_net` 说明网络，但实际启用的是 `res_net`；二者均按源保留。打印参数数目 47330 与所列卷积层逐层计数一致；这只是结构核数，并非运行模型所得结果。

```
    (sample_input): Identity()
    (bn_sample): Identity()
    (bn1): Identity()
    (conv1): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (bn2): Identity()
    (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  )
  (Res2): ResNetBlock(
    (sample_input): Identity()
    (bn_sample): Identity()
    (bn1): Identity()
    (conv1): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (bn2): Identity()
    (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  )
  (Res_out): Conv2d(32, 2, kernel_size=(1, 1), stride=(1, 1))
)
Total number of trainable parameters: 47330
```

接下来定义非常重要的转换函数。它们不修改内容，而是把仿真状态转换为各软件包适用的数据结构：求解器使用交错网格，NN 使用 PyTorch 张量，另一个辅助函数则将数据加载器的 NumPy 输出转换为 NS 求解器可方便使用的 phiflow 网格和值。

`to_phiflow` 接收一批 PyTorch 张量，将第二维中与 x, y 相关的两个通道（1 和 2）转换为 phiflow 交错网格。要注意数组大小：由于封闭边界、开放边界及交错网格，phiflow 对 x 和 y 分量分别要求大小为 `[64, 31]` 和 `[65, 32]` 的数组。调用 `math.tensor` 时，还须告诉 phiflow 哪些是批次和空间维度。

`to_pytorch` 接收包含 `[marker,velocity]` 的短列表，并通过 `vector['x']` 和 `vector['y']` 使用交错网格速度的两个矢量分量，舍去速度场网格的最外层。这样得到两个相同大小的张量，并沿 `channels` 维堆叠。还用 `math.ones` 添加一个常数通道，将其乘以 `ext_const_channel` 中所需的 Reynolds 数，加入堆叠。最后通过 `native(order='batch,channels,y,x')` 将这些网格转为一个 PyTorch 张量，作为神经网络输入。[7]

最后一个辅助函数将数据加载器输出转为 phiflow 数据结构。数据加载器产生 NumPy 数组，通过上面的 `to_phiflow` 转为速度网格。同时，为 NN 算子归一化 Reynolds 数；为 phiflow 求解器则保留原始标量，因为后者与网络不同，使用“物理单位”。训练时每个批次包含多个 Reynolds 数相同的样本，由数据加载器负责安排。

```
def to_phiflow(batch):
    vx = batch[:,1,:-1,:-1]
    vy = batch[:,2,:,:] # 无需修改

    #print("v_dims "+str([vx.shape,vy.shape])) # 调试示例
    # v_dims 应为 vx [torch.Size([B, 64, 31]), vy torch.Size([B, 65, 32])]

    vel = domain.staggered_grid(math.stack([
        math.tensor(vy, math.batch('batch'), math.spatial('y, x')),
```

[7] 译注：代码对 x 分量零填充、对 y 分量截去末层。正文的 `ext_const_channel` 在函数中实际为 `Re`。

```
        math.tensor(vx, math.batch('batch'), math.spatial('y, x')),
    ], math.dual(vector="y,x")))
    return vel

def to_pytorch(marker_vel, Re):
    # 对齐交错速度网格的边，使其大小与中心网格相同
    grid = math.stack(
        [
            math.pad(marker_vel[1].vector['x'].values, {'x':(0,1)}, math.extrapolation.ZERO), # x
            ↪ 分量
            marker_vel[1].vector['y'].y[:-1].values, # y 分量
            math.ones(marker_vel[0].shape)*Re # 常数 Re
        ],
        math.channel('channels')
    ).native(order='batch,channels,y,x')
    return grid

def to_solver(inputs):
    marker_in = inputs[:,0,:-1,:]
    marker_in = domain.scalar_grid(math.tensor(
        marker_in, math.batch('batch'), math.spatial('y, x')))
    v_in = to_phiflow(inputs)
    Re = math.tensor(inputs[0,3, 0,0].detach()) # 标量，取首个索引 0,0

    Re_norm = (Re - math.tensor(DATA_RE_MEAN)) / math.tensor(DATA_RE_STD)
    Re_norm = float(Re_norm.native().detach()) # 只需要一个数
    return marker_in, v_in, Re, Re_norm
```

18.5 训练设置

现在还需设置训练，即初始化数据集和优化器。后者较简单：使用给定学习率的 Adam 优化器，同时定义批次大小，此处 3 是一个合适的默认值。

```
LEARNING_RATE = 1e-3
optimizer = adam(network, LEARNING_RATE)

# 最关键的参数之一：每个训练步向未来看多少个仿真步
MSTEPS = 4

BATCH_SIZE = 3
```

最重要也最有意思的参数是 MSTEPS，它定义每次训练迭代展开多少个仿真步，直接影响每个训练步的运行时间：先正向模拟全部步骤，再将梯度反向传播经过与 NN 求值交错的全部 MSTEPS 个仿真步。不过，也正是在这里，我们从梯度中获得关于推断修正如何实际影响运行中仿真的重要反馈。因此，较大的 msteps 通常更好。理想情况下，训练还会采用课程安排，随时间增加 MSTEPS；为简化代码，这里略去。

训练数据使用 PBDL 的数据加载器类。它会在必要时自动从 HuggingFace 下载数据，并返回一个容易迭代的类，每次获得新训练批次。这里需指定 `BATCH_SIZE`，选取数据集中的仿真 0 至 5，保留 6 至 9 用于之后测试。前六个仿真的 Reynolds 数在一个中间范围内，因而可保留该范围之外的新 Reynolds 数，之后测试泛化。数据加载器还需提供足够多步的真实参考数据，用于计算整条展开轨迹的损失；源文说明通过 `time_steps=MSTEPS+1` 和 `intermediate_time_steps=True` 保证这一点。[8]

```
import pbdl
import pbdl.torch.loader

dataloader = pbdl.torch.loader.Dataloader(
    "solver-in-the-loop-wake-flow", MSTEPS, shuffle=True, sel_sims=[0,1,2,3,4,5],
    batch_size=BATCH_SIZE, normalize_const="std", normalize_data="std",
    intermediate_time_steps=True)

# 在同一脚本内同时使用未归一化和归一化值的变通办法：
# 保存 Reynolds 数条件变量的归一化常数，再关闭该归一化 (norm=None)；
# 随后手动归一化 Re 值
DATA_RE_MEAN = dataloader.dataset.norm_strat_const.const_mean[0][0]
DATA_RE_STD = dataloader.dataset.norm_strat_const.const_std[0][0]
print([DATA_RE_MEAN,DATA_RE_STD])
dataloader.dataset.norm_strat_const = None
dataloader.dataset.norm_strat_data = None
```

原书输出如下：数据下载完成，十个仿真中选中六个，每个有 496 个样本；未发现完整的预计算归一化数据，因此进行计算，并打印两个归一化常数。[9]

```
download completed [progress bar] 100%6m
Success: Loaded solver-in-the-loop-wake-flow with 10 simulations (6 selected) and 496 samples each.
Info: No precomputed normalization data found (or not complete). Calculating data...
[np.float64(1237.79296875), np.float64(1453.7359614526729)]
```

另外，在下一个代码单元中定义一些控制训练与仿真的全局变量。

下面把流体求解器对象称为 `simulator`。为便于在 phiflow 中创建网格，它使用 phiflow 的 `Domain` 对象。此对象主要为方便而设：保存区域的分辨率、物理尺寸和边界条件。每个网格都需要这些信息，故集中存于 `Domain` 中很方便。对于上面的设置，x, y 方向需要不同边界条件：分别为封闭壁面和允许自由流入、流出。

```
# phiflow 实际采用的网格单元分辨率 (并非特别关键)
SOURCE_RES = [64,32]

# 区域包围盒以抽象单位表示的物理大小
#  (对计算单位和物理单位之间的转换很重要)
LENGTH = 100.
```

[8]译注：下方调用实际把 `MSTEPS` 作为第二个位置参数，没有显式写 `time_steps=MSTEPS+1`。两处均照录，不能未经数据加载器版本和接口核查就视为完全等价。

[9]译注：以下输出保留全部信息、路径、数值和警告；仅重排长行，并将无法稳定排印的进度条填充改写为 `[progress bar]`。源下载进度行的 `100%6m` 含渲染异常，按可见字符保留；原始完整页面在本章源页证据中保存。

```
# 为便于阅读
from phi.physics._boundaries import Domain, OPEN, STICKY as CLOSED

BNDS = {
    'y':(phi.physics._boundaries.OPEN, phi.physics._boundaries.OPEN),
    'x':(phi.physics._boundaries.STICKY,phi.physics._boundaries.STICKY)
}

domain = phi.physics._boundaries.Domain(
    y=SOURCE_RES[0], x=SOURCE_RES[1], boundaries=BNDS,
    bounds=Box(y=2*LENGTH, x=LENGTH))

simulator = KarmanFlow(domain=domain)
```

原书出现以下弃用警告：Domain 将在未来版本移除，建议用字典直接创建网格；HardGeometryMask 和 SoftGeometryMask 已弃用，建议使用 field.mask 或 field.resample。重复出现的警告仍完整保留。

```
/tmp/ipykernel_774262/1641432920.py:8: FutureWarning: Domain (phi.physics._boundaries) is
↪  deprecated and will be removed in a future release.
Please create grids directly, replacing the domain with a dict, e.g.
    domain = dict(x=64, y=128, bounds=Box(x=1, y=1))
    grid = CenteredGrid(0, **domain)
  from phi.physics._boundaries import Domain, OPEN, STICKY as CLOSED
/tmp/ipykernel_774262/1641432920.py:15: DeprecationWarning: Domain is deprecated and will be
↪  removed in a future release. Use a dict instead, e.g. CenteredGrid(values, extrapolation,
↪  **domain_dict)
  domain = phi.physics._boundaries.Domain(y=SOURCE_RES[0], x=SOURCE_RES[1], boundaries=BNDS,
  ↪  bounds=Box(y=2*LENGTH, x=LENGTH))
/tmp/ipykernel_774262/1641432920.py:15: FutureWarning: Domain is deprecated and will be removed in
↪  a future release. Use a dict instead, e.g. CenteredGrid(values, extrapolation, **domain_dict)
  domain = phi.physics._boundaries.Domain(y=SOURCE_RES[0], x=SOURCE_RES[1], boundaries=BNDS,
  ↪  bounds=Box(y=2*LENGTH, x=LENGTH))
/tmp/ipykernel_774262/1529251970.py:7: DeprecationWarning: HardGeometryMask and SoftGeometryMask
↪  are deprecated. Use field.mask or field.resample instead.
  self.vel_BcMask = self.domain.staggered_grid(HardGeometryMask(Box(y=(None, 5), x=None)))
```

18.6 交错执行仿真与神经网络

为了高效训练包含复杂仿真的模型，应关注运行时间。高效运行尤其需要使用参与训练的 GPU，并尽可能把数据留在 GPU 上。对于 phiflow，这在很大程度上可通过对核心步骤进行 JIT 编译实现。下个代码单元对 Navier–Stokes 仿真步这样处理。该步包含隐式压力求解等操作，每次前向和反向传播都可能调用多次，因此很适合 JIT 编译。（原文建议移去 @jit_compile，亲自比较变慢的程度。）

```
@jit_compile
def simulation_step(marker,velocity,Re,resolution):
    m,v = simulator.step(
```

```
        marker_in=marker,
        velocity_in=velocity,
        Re=Re, res=resolution
    )
    return m,v
```

接下来是整个设置中最关键的步骤：定义一个函数，封装每个训练步中的仿真步与网络求值链。有了前面的辅助函数，这其实很简单：循环 MSTEPS 次，用 simulator_step 对输入状态调用模拟器，随后通过 network(to_pytorch(...)) 计算修正算子。然后把 NN 修正加到 prediction 状态列表中最后的仿真状态上。该列表保留每个时间步的标记密度和速度。[10]

除 Reynolds 数外，不归一化状态本身，因为它们已经处于 -1 至 1 范围。其他仿真中，最好在调用网络前归一化，在随后的物理求解步前再反归一化。[11]

```
def training_step(inputs_targets):
    [inputs, targets] = inputs_targets
    marker_in, v_in, Re, Re_norm = to_solver(inputs)
    prediction = [[marker_in,v_in]]
    loss = 0

    for i in range(MSTEPS):
        m2,v2 = simulation_step(
            marker=prediction[-1][0],
            velocity=prediction[-1][1],
            Re=Re, resolution=SOURCE_RES[1]
        )

        net_in = to_pytorch([m2,v2],Re_norm)
        net_out = network(net_in)

        cy = net_out[:,1,:,:] # 对 y 填充
        cy = torch.nn.functional.pad(input=cy, pad=(0,0, 0,1), mode='constant', value=0)
        cx = net_out[:,0,:,:-1]

        v_corr = domain.staggered_grid(math.stack([
            math.tensor(cy, math.batch('batch'), math.spatial('y, x')),
            math.tensor(cx, math.batch('batch'), math.spatial('y, x')),
        ], math.dual(vector="y,x")))

        prediction += [[domain.scalar_grid(m2), v2 + v_corr]]
        vdiff = prediction[-1][1] - to_phiflow(targets[:,i,...])
        loss += field.l2_loss(vdiff)

    return loss, prediction
```

[10]译注：本段原文写 simulator_step，代码实际定义和调用 simulation_step，这里保留该差异，代码不据正文改名。训练与 run_sim 均在 make_incompressible 之后直接相加 $v_{\text{sim}} + v_{\text{corr}}$，未再投影；修正后状态因此不自动严格无散，也不自动满足障碍约束。

[11]译注：该范围是源书对其设置的说明，不是任意流动的界；本章后面的速度图色条还显示超过 1 的值，故不应把该句当作经过验证的严格范围。

上面的 `training_step` 也直接计算并返回损失。这里简单使用整条序列，即全部展开的 `msteps` 上的网格 L^2 损失，网格表示为 phiflow 场。在 `vdiff` 中计算目标与当前预测的差，然后计算其 `l2_loss`。

定义训练步后，训练就很简单：只需让优化器计算使损失最小的梯度。Phiflow 提供辅助函数 `update_weights`，向它传入神经网络、优化器和计算损失的函数，损失为 `training_step` 的第一个返回值。我们重复 `EPOCHS` 次，每次遍历 `dataloader` 中整个数据集。进度条 `pbar` 只是便于观察训练进展。第一次迭代触发模拟器的 JIT 编译，因此耗时稍长，随后应明显加快。代码还在每个训练轮次 N 将网络状态保存为 `net-N.pickle`。

```
EPOCHS = 5

pbar = tqdm(initial=0, total=EPOCHS*len(dataloader), ncols=96)

for epoch in range(EPOCHS):
    for b, (input_cpu, targets_cpu) in enumerate(dataloader):
        input = torch.tensor(input_cpu, dtype=torch.float32).to(device);
        targets = torch.tensor(targets_cpu, dtype=torch.float32).to(device)

        loss, prediction = update_weights(network, optimizer, training_step, [input, targets])

        pbar.set_description("loss "+str(np.sum(loss.numpy("batch"))), refresh=False);
        pbar.update(1)

    torch.save(network.state_dict(), "net-"+str(epoch)+".pickle")

pbar.close()
```

原书输出如下。复制张量的警告建议用 `clone().detach()`（或再启用梯度）代替 `torch.tensor`；秩亏警告提示矩阵可能奇异、导致收敛问题，并建议指定 `Solve(rank_deficiency=...)`；稀疏 CSR 张量支持仍处于测试阶段。以下是原书日志，而非本译稿的运行记录。

```
0%| [progress bar] | 0/4960 [00:00<?, ?it/s]
/tmp/ipykernel_774262/4154370188.py:7: UserWarning: To copy construct from a tensor, it is
↪  recommended to use sourceTensor.clone().detach() or
↪  sourceTensor.clone().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  input = torch.tensor(input_cpu, dtype=torch.float32).to(device);
/tmp/ipykernel_774262/4154370188.py:8: UserWarning: To copy construct from a tensor, it is
↪  recommended to use sourceTensor.clone().detach() or
↪  sourceTensor.clone().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  targets = torch.tensor(targets_cpu, dtype=torch.float32).to(device)
anaconda3/envs/torch24/lib/python3.12/site-packages/phiml/math/_optimize.py:631: UserWarning:
↪  Possible rank deficiency detected. Matrix might be singular which can lead to convergence
↪  problems. Please specify using Solve(rank_deficiency=...).
  warnings.warn("Possible rank deficiency detected. Matrix might be singular which can lead to
  ↪  convergence problems. Please specify using Solve(rank_deficiency=...).")
```

```
anaconda3/envs/torch24/lib/python3.12/site-packages/phiml/backend/torch/_torch_backend.py:803:
↪  UserWarning: Sparse CSR tensor support is in beta state. If you miss a functionality in the
↪  sparse tensor support, please submit a feature request to
↪  https://github.com/pytorch/pytorch/issues. (Triggered internally at
↪  ../aten/src/ATen/SparseCsrTensorImpl.cpp:53.)
  return torch.sparse_csr_tensor(row_pointers, column_indices, values, shape, device=values.device)
anaconda3/envs/torch24/lib/python3.12/site-packages/phiml/math/_optimize.py:631: UserWarning:
↪  Possible rank deficiency detected. Matrix might be singular which can lead to convergence
↪  problems. Please specify using Solve(rank_deficiency=...).
  warnings.warn("Possible rank deficiency detected. Matrix might be singular which can lead to
  ↪  convergence problems. Please specify using Solve(rank_deficiency=...).")
loss 15.62617: 0%| [progress bar] | 1/4960 [00:12<16:36:58, 12.06s/it]
/tmp/ipykernel_774262/4154370188.py:7: UserWarning: To copy construct from a tensor, it is
↪  recommended to use sourceTensor.clone().detach() or
↪  sourceTensor.clone().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  input = torch.tensor(input_cpu, dtype=torch.float32).to(device);
/tmp/ipykernel_774262/4154370188.py:8: UserWarning: To copy construct from a tensor, it is
↪  recommended to use sourceTensor.clone().detach() or
↪  sourceTensor.clone().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  targets = torch.tensor(targets_cpu, dtype=torch.float32).to(device)
loss 0.23079132: 100%|[progress bar]| 4960/4960 [3:10:57<00:00, 2.31s/it]
```

注意，每次调用 `update_weights`，内部都会执行全部四次交错的仿真步和神经网络调用，并将梯度反向传播经过整条链。由于顺序是先求解器、后网络，第一步不经过求解器反向传播。但接下来三步会给第一次网络调用提供反馈，说明其推断的修正是否把之后三个状态推向正确方向。

使用默认参数，损失应从初始的 10 以上下降到约 1.0；增加训练轮次，使用更大、更深的网络，还会进一步降低。损失下降是好迹象，但更重要的当然是 NN 与求解器的组合在新输入上的表现。这种训练方法实现了一个混合求解器，由普通源模拟器和网络算子组成；网络算子专门训练为在选定的仿真情形范围内与该模拟器交互。[12]

18.7 测试评估

为了评估 “AI 驱动的求解器” 的性能，基本上只需把每次训练迭代的内层循环重复更多步。训练时只限于 `msteps` 次求值，现在则可以让求解器运行任意长度。这能很好地测试求解器是否学会把数据保持在所需分布内，也是在测试向更长展开轨迹的泛化。

还需要一组新数据。下面使用此前下载的数据集中尚未使用的最后四个仿真。

用 `sel_sims=[6,7,8,9]` 创建新的数据加载器。

```
del dataloader # 关闭 hdf5 文件句柄

# 获取新的、从未见过的测试数据
dataloader_test = pbdl.torch.loader.Dataloader(
    "solver-in-the-loop-wake-flow", 200, batch_size=1, shuffle=True,
```

[12]译注：上面原书实际示例日志末值为 0.23079132，与正文所说 “约 1.0” 并非同一数值，均保留。更长展开涉及运行成本和梯度稳定性，原文 “通常更好” 是经验性说法，不是单调改进保证；本译稿未运行这些比较。

```
    sel_sims=[6,7,8,9], normalize=False, intermediate_time_steps=True)
```

原书输出表明，十个仿真中选中四个，每个有 300 个样本。

```
download completed [progress bar] 100%6m
Success: Loaded solver-in-the-loop-wake-flow with 10 simulations (4 selected) and 300 samples each.
```

若已有预训练网络，此时适合加载模型。默认假定上面的训练已完成，故无需加载。

```
# 可选：加载模型
if False:
    fn = "net-"+str(EPOCHS-1)+".pickle" # 加载最后一个
    network.load_state_dict(torch.load(fn, map_location=device, weights_only=True))
    print("Loaded "+fn)
```

可以复用上面很多求解器代码，但接下来考虑两个模拟版本：为比较，在源流形上运行一个对照仿真，即基于 $\mathcal{P}_s$ 而不施加任何修正；第二个版本为实际结果，反复计算源求解器并加上学到的修正。

下方 `run_sim` 根据 `network` 是否提供神经算子，在这两个版本间切换。没有网络时，只运行源求解器并追加状态；有网络时则运行完整混合求解器。两种情形都沿途计算相对参考解的误差。为便于后续分析与可视化，除相对误差及求解器实际计算的状态外，函数还返回修正和参考解。[13]

```
def run_sim(inputs, targets, steps, network=None):
    marker_in, v_in, Re, Re_norm = to_solver(inputs)

    simtype = "With corr."
    if (network==None): simtype = "Sim. only"
    print("Running test with Re="+str(Re)+", "+simtype)

    prediction = [[marker_in,v_in]]
    correction = [[marker_in,v_in]]
    refs = [v_in]
    errors = []

    for i in tqdm(range(steps), desc=simtype, ncols = 64):
        marker_sim,v_sim = simulation_step(
            marker=prediction[-1][0],
            velocity=prediction[-1][1],
            Re=Re, resolution=SOURCE_RES[1] # 从常数场中取 Re
        )

        if network: # 用训练好的神经算子运行混合求解器
            net_in = to_pytorch([marker_sim,v_sim],Re_norm)
            net_out = network(net_in)

            cy = net_out[:,1,:,:] # 对 y 填充
            cy = torch.nn.functional.pad(input=cy, pad=(0,0, 0,1), mode='constant', value=0)
```

[13]译注：`prediction` 已含初态，循环中又追加新态，但误差行仍使用 `prediction[i][1]`，与刚追加的 `refs[-1]` 看来相差一个时间步；训练函数则使用 `prediction[-1]`。此处保留源代码索引而不静默修正。下方所报误差需在时刻对齐和指标实现核查后再解释。

```
            cx = net_out[:,0,:,:-1]

            v_corr = domain.staggered_grid(math.stack([
                math.tensor(cy, math.batch('batch'), math.spatial('y, x')),
                math.tensor(cx, math.batch('batch'), math.spatial('y, x')),
            ], math.dual(vector="y,x")))

            prediction += [[domain.scalar_grid(marker_sim), v_sim + v_corr]]
            correction += [[domain.scalar_grid(marker_sim), v_corr]]

        else: # 只运行低保真度求解器
            prediction += [[domain.scalar_grid(marker_sim), v_sim]]

        refs += [to_phiflow(targets[:,i,...])]
        vdiff = prediction[i][1] - refs[-1]

        error_phi = field.l1_loss(vdiff)
        errors += [float(error_phi.native("batch")[0] /
        ↪  field.l1_loss(refs[-1]).native("batch")[0])]

    return errors, prediction, refs, correction
```

用 `next(iter(dataloader_test))`，可从数据加载器获得 Reynolds 数此前未见的新状态。随后从相同初态出发，将源求解器和混合求解器各运行 `ROLLOUT_STEPS` 次迭代。与训练相似，先从数据加载器获得 `input` 中的初态和 `targets` 中的参考状态。（由于随机采样，可能需多次运行，才能例如取到较高 Reynolds 数的情形。）

```
# 获取一个样本
(input_cpu, targets_cpu) = next(iter(dataloader_test))
input = torch.tensor(input_cpu, dtype=torch.float32).to(device)
targets = torch.tensor(targets_cpu, dtype=torch.float32).to(device)
print("Re ",math.tensor(input[0,3, 0,0].detach()))
```

原书打印 Reynolds 数，并再次出现复制张量的两条警告。

```
Re  73.24219

/tmp/ipykernel_774262/1514360431.py:3: UserWarning: To copy construct from a tensor, it is
↪  recommended to use sourceTensor.clone().detach() or
↪  sourceTensor.clone().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  input = torch.tensor(input_cpu, dtype=torch.float32).to(device)
/tmp/ipykernel_774262/1514360431.py:4: UserWarning: To copy construct from a tensor, it is
↪  recommended to use sourceTensor.clone().detach() or
↪  sourceTensor.clone().detach().requires_grad_(True), rather than torch.tensor(sourceTensor).
  targets = torch.tensor(targets_cpu, dtype=torch.float32).to(device)
```

有意思的问题当然是：NN 算子究竟在多大程度上改进了低保真度源求解器的精度？原文称，这由 `run_sim` 每次计算的相对 L^2 误差刻画，衡量相对于参考速度平方幅值的平方距离，并称上面的代码

单元直接绘制聚合误差。[14]

下面用训练好的神经修正算子运行混合求解器，并单独运行低保真度求解器作比较。ROLLOUT_STEPS 决定两种版本计算多少时间步；代码单元还直接输出平均相对误差。由于随机训练和数值舍入误差，不同运行结果可能略有变化。不过，一般而言，带神经修正算子的混合求解器应显著降低数值误差，通常为低保真度求解器的 1/5 至 1/6。

这表明该训练设置的核心目标已经实现：混合求解器显著降低数值误差，并泛化到新的 Reynolds 数。[15]

```
ROLLOUT_STEPS = 100
err_lowfid_only, prediction_lowfid_only, refs, _ = run_sim(input, targets, ROLLOUT_STEPS)
err_corrected, prediction_corrected, _, corrs = run_sim(input, targets, ROLLOUT_STEPS, network)

print("\n Rel. L2 errors: low-fidelity:", float(np.mean(err_lowfid_only))," corrected:",
↪  float(np.mean(err_corrected)))
```

原书输出如下；Sim. only 表示仅仿真，With corr. 表示加入修正。再次出现的两条秩亏警告按源保留。

```
Running test with Re=73.24219, Sim. only

Sim. only: 0%| [progress bar] | 0/100 [00:00<?, ?it/s]
anaconda3/envs/torch24/lib/python3.12/site-packages/phiml/math/_optimize.py:631: UserWarning:
↪  Possible rank deficiency detected. Matrix might be singular which can lead to convergence
↪  problems. Please specify using Solve(rank_deficiency=...).
  warnings.warn("Possible rank deficiency detected. Matrix might be singular which can lead to
  ↪  convergence problems. Please specify using Solve(rank_deficiency=...).")
anaconda3/envs/torch24/lib/python3.12/site-packages/phiml/math/_optimize.py:631: UserWarning:
↪  Possible rank deficiency detected. Matrix might be singular which can lead to convergence
↪  problems. Please specify using Solve(rank_deficiency=...).
  warnings.warn("Possible rank deficiency detected. Matrix might be singular which can lead to
  ↪  convergence problems. Please specify using Solve(rank_deficiency=...).")
Sim. only: 100%|[progress bar]| 100/100 [00:31<00:00, 3.14it/s]

Running test with Re=73.24219, With corr.

With corr.: 100%|[progress bar]| 100/100 [00:52<00:00, 1.89it/s]

Rel. L2 errors: low-fidelity: 0.10863638424314559 corrected: 0.031211425131186844
```

现在更详细地绘图比较两个版本。下一幅图显示误差随时间的演化。低保真度求解器的相对误差随时间线性上升，混合求解器的增长则慢得多。使用更大的网络并成功训练时，几乎可以抑制所有增长，将误差保持在很低水平。这说明它能成功引导低保真度求解器保持目标的精度。若没有基础求解器，这项任务会困难得多，因为 NN 就需要承担全部工作。这里可以依靠耦合求解器的预测，通常小幅修正就已足够。

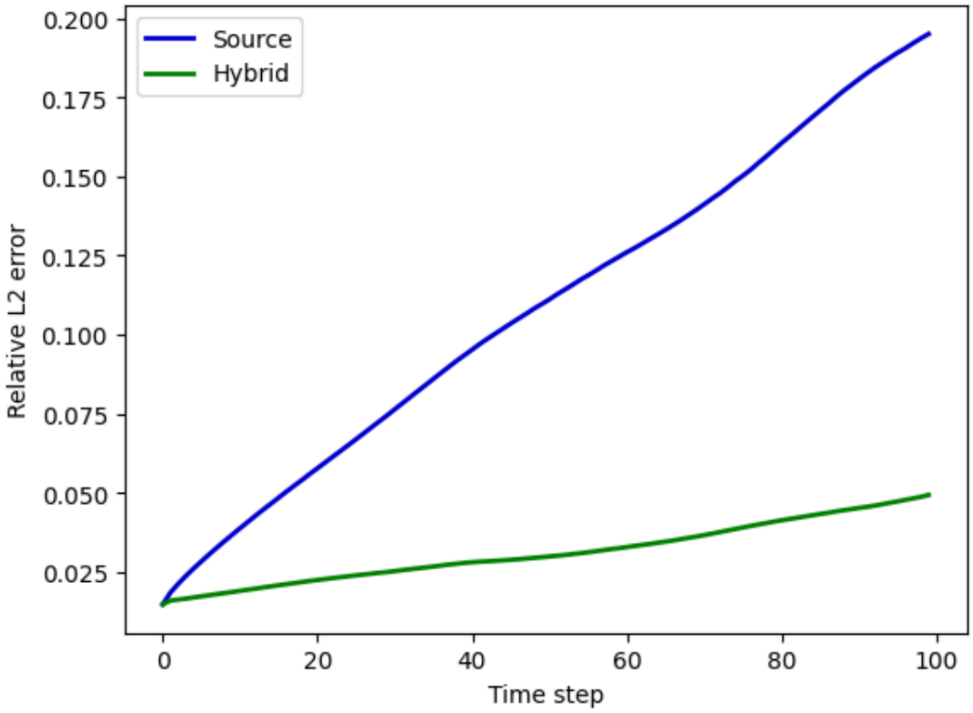
[14]译注：源代码实际使用 field.l1_loss，没有在测试误差处调用 l2_loss；该段的 L^2、平方距离描述及后续打印标签/纵轴均与代码不一致。紧前的取样单元也未绘图，绘图代码在后面。均保留源文信息，不能把这些输出认证为正确的相对 L^2 误差。

[15]译注：这句话保留原书主张力度，但不是本译稿验证的结论。下方冻结输出比值约为 3.48，并非正文 “5–6 倍”；还存在上述 L1/L2 与时间索引疑点。本章只是源书的特定尾流测试，不能据此推论任意 Reynolds 数、几何或展开长度的泛化、稳定性和误差保证。

```
fig = pylab.figure().gca()
pltx = np.linspace(0,ROLLOUT_STEPS-1,ROLLOUT_STEPS)
fig.plot(pltx, err_lowfid_only, lw=2, color='mediumblue', label='Source')
fig.plot(pltx, err_corrected, lw=2, color='green', label='Hybrid')
pylab.xlabel('Time step'); pylab.ylabel('Relative L2 error'); fig.legend()
```

原书还打印了 Matplotlib 图例对象：

```
<matplotlib.legend.Legend at 0x7de77dc7b770>
```

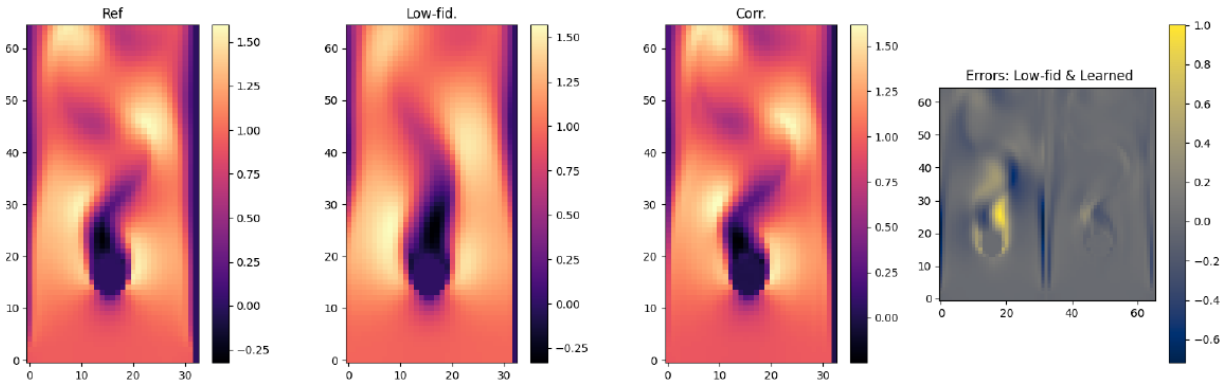


原书绘图输出：误差随时间步的变化。Source 为源求解器（蓝），Hybrid 为混合求解器（绿）；横轴 Time step 为时间步，纵轴原标为相对 L2 误差。此标签与所列 L1 误差代码不一致，见正文译注。

尽管量化结果给出了神经算子性能的重要概括，仍需进行合理性检查，确保 NN 按预期工作。下个代码单元把参考解、低保真度求解器和混合求解器的状态并排绘出，最右侧还绘出两种求解器的误差。

```
# 显示哪个批次的哪一步；默认显示第一个样本的最后一步
STEP = ROLLOUT_STEPS
BATCH = 0
NUM_SHOW = 4
PRINT_STATS = False # 可选：打印统计量

fig, axes = pylab.subplots(1, 4, figsize=(16, 5))
i = 0

v = refs[STEP].staggered_tensor().numpy('batch,y,x,vector')[BATCH,:,:,0]
if PRINT_STATS: print(["reference ", BATCH, i, np.mean(v), np.min(v), np.max(v)])
axes[i].set_title(f"Ref")
im=axes[i].imshow(v, origin='lower', cmap='magma');
```

```
pylab.colorbar(im); i=i+1; vy_ref=v

v = prediction_lowfid_only[STEP][1].staggered_tensor().numpy('batch,y,x,vector')[BATCH,:,:,0]
if PRINT_STATS: print(["low-fid. ", BATCH, i, np.mean(v), np.min(v), np.max(v)])
axes[i].set_title(f"Low-fid.")
im=axes[i].imshow(v, origin='lower', cmap='magma');
pylab.colorbar(im); i=i+1; vy_lowfid=v

v = prediction_corrected[STEP][1].staggered_tensor().numpy('batch,y,x,vector')[BATCH,:,:,0]
if PRINT_STATS: print(["corrected", BATCH, i, np.mean(v), np.min(v), np.max(v)])
axes[i].set_title(f"Corr.")
im=axes[i].imshow(v, origin='lower', cmap='magma');
pylab.colorbar(im); i=i+1; vy_corr=v

# 并排显示误差
err_lf = vy_ref - vy_lowfid
err_corr = vy_ref - vy_corr
v = np.concatenate([err_lf,err_corr], axis=1)
axes[i].set_title(f" Errors: Low-fid & Learned")
im=axes[i].imshow(v, origin='lower', cmap='cividis');
pylab.colorbar(im); i=i+1

pylab.tight_layout()
```



原书绘图输出。Ref：参考解；Low-fid.：低保真度结果；Corr.：修正后结果；Errors: Low-fid & Learned：低保真度与学习结果的误差，左、右半幅分别对应两者。最后的误差色条有正负值，表示有符号差，而非逐点绝对误差。

这清楚显示出中间的纯源仿真如何偏离左侧参考解；学习后的版本与参考解更接近。

右侧两幅逐单元误差图也说明了这一点：源版本的误差大得多，即颜色更亮，显示它系统性地低估了应形成的涡旋。学习后版本的误差分布均匀得多，幅值也显著更小。[16]

评估到此结束。注意，AI 驱动的混合求解器行为的改进，可能难以用 MAE 或 L^2 范数等简单向量范数可靠衡量。要改进这一点，需要采用领域特有的其他指标；此例可用基于涡量和流动湍流性质的流体指标。不过，本书更关注深度学习相关主题，下一章将用可微物理求解器处理另一个逆问题。

[16]译注：“颜色更亮”沿原文保留，但代码绘制的是有符号差，色条跨过零；亮度不能直接等同于误差绝对值。涡旋与速度图的定性解释也属于源书描述，尚未独立复现实验。

18.8 下一步

- 通过设置 `msteps=1` 关闭可微物理训练，并与展开版本比较。这得到监督训练，因为不再需要梯度流过求解器；相对误差会明显增大。[17]
- 类似地，尝试使用更大的 `msteps`，例如 8 或 16 训练网络。由于训练的循环性质，可能需要加载预训练状态，稳定最初几次迭代；这实际上加入了“课程学习”。
- 使用外部 GitHub 代码生成更困难的测试数据，并在这些情形上运行训练好的 NN。会看到，训练误差降低并不总与测试性能改善直接相关。[18]

[17]译注：源文以小写 `msteps` 叙述，示例实际配置变量为大写 `MSTEPS`。这里的“监督训练”指单步求解后只对网络修正反传，求解器仍在前向过程中。误差增大的说法是待做比较的原文预期，不是本译稿的新实验结果。

[18]译注：冻结源仅写“外部 GitHub 代码”，该页没有链接注释可恢复目标地址，故不补造；本译稿未执行本节任何建议实验。

第十九章　用神经网络求解逆问题

逆问题涵盖科学中一大类实际应用。一般而言，目标不是直接计算未来某时刻的速度等物理场——那是正向求解的典型情形——而是更一般地计算模型方程中的一个或多个参数，使某些约束得到满足。一个很常见的目标是在给定约束下找到单个参数的最优设置。例如，为平流—扩散模型确定一个全局扩散常数，使模型尽可能准确地拟合测量数据。通过观测调整任何模型参数，或者重建初始条件，例如粒子图像测速（PIV），都会遇到逆问题。更复杂的情形旨在根据最优条件计算边界几何，例如求出流体中阻力最小的形状。

下面的关键在于，我们不只求解逆问题的单个实例，而希望用深度学习求解更大的一组逆问题。因此，与“用 PINN 优化 Burgers 方程”的物理信息示例，或“可微流体仿真”中的可微物理（DP）优化不同——它们针对特定逆问题实例求解优化问题——现在要训练一个 NN，学习求解更大一类逆问题，即整个解流形。当然，这些问题仍需具有一定相似性，否则就没有可学习的内容，隐含的解流形连续性假设也会失效。

下面用一个极具挑战性的测试问题代表这类逆问题：计算一个高维控制函数，在不可压流体仿真的整个过程中施加力，使流体中被动平流的标记达到期望的目标状态。这意味着约束非常间接：仅为序列末尾的一个状态；而自由度很多：控制力函数是时空函数，其自由度与流场本身相同。

控制的长期性质使其成为困难的逆问题：物理系统状态的任何变化，都可能导致后续时刻的巨大变化，因此控制器必须预判受影响后系统将如何表现。这意味着 NN 也要学习底层物理如何演化和变化，而 DP 训练的梯度正是在这里将学习引向能够达到目标的解。

原文警告　这段代码如今已非常“经典”，需要不少 Colab 不再支持的旧 API，主要是 Python 3.6、TF 1.14 和 phiflow 1.4.1。因此，原文建议在本地 conda 环境运行此例。[1]

19.1　问题表述

用“模型与方程”中的记号，得到最小化问题

$$\arg\min_{\theta} \sum_{m} \sum_{i} \big(f(x_{m,i}; \theta) - y^*_{m,i} \big)^2,$$

其中 $y^*_{m,i}$ 表示标记场目标状态的样本，$x_{m,i}$ 表示标记密度的仿真状态。与之前一样，下标 i 在不同空间位置采样解，通常覆盖全部网格单元；这里的 m 则遍历一大组不同目标状态。

[1]译注：本章忠实转录冻结版示例，未安装旧依赖、克隆仓库、下载预训练网络、生成数据或执行训练。文中的运行与性能描述均来自原书。

目标是训练两个网络 OP 和 CFE，权重分别为 θ_{OP} 和 θ_{CFE}，使序列

$$\begin{aligned}\mathbf{u}_n, d_n &= \mathcal{P}\big(\text{CFE}(\mathcal{P}(\text{CFE}(\cdots\mathcal{P}(\text{CFE}(\mathbf{u}_0, d_0, d_{OP}))\cdots)))\big)\\ &= (\mathcal{P}\,\text{CFE})^n(\mathbf{u}_0, d_0, d_{OP})\end{aligned}$$

最小化上述损失。OP 是预测器，确定 CFE 的动作应瞄准的状态 d_{OP}，也就是进行较长期规划，再据此确定动作。给定目标 d^*，它计算 $d_{OP} = \text{OP}(d, d^*) = f_{\text{OP}}(d, d^*; \theta_{\text{OP}})$。CFE 通过计算 $\mathbf{u} + f_{\text{CFE}}(\mathbf{u}, d, f_{\text{OP}}(d, d^*; \theta_{\text{OP}}); \theta_{\text{CFE}})$，以加性方式作用于速度场。这里用 f_{OP} 和 f_{CFE} 分别表示 OP 和 CFE 的 NN 表示，d^* 表示目标密度状态，θ_{OP} 和 θ_{CFE} 为相应网络权重。

本问题的模型 PDE $\mathcal{P}$ 包含速度 $\mathbf{u}$ 的二维不可压 Navier–Stokes 方程的离散化版本：

$$\begin{aligned}&\frac{\partial u_x}{\partial t} + \mathbf{u}\cdot\nabla u_x = -\frac{1}{\rho}\nabla p,\\ &\frac{\partial u_y}{\partial t} + \mathbf{u}\cdot\nabla u_y = -\frac{1}{\rho}\nabla p,\\ &\text{满足}\qquad \nabla\cdot\mathbf{u} = 0.\end{aligned}$$

其中没有显式黏性，并附加标记密度 d 的输运方程 $\partial d/\partial t + \mathbf{u}\cdot\nabla d = 0$。[2]

概括而言，预测器 OP 给出方向，动作网络 CFE 对物理模型 $\mathcal{P}$ 施加力。三者须协同工作，才能在 n 次仿真迭代后到达给定目标。由这一表述可见，尤其因为三个函数均非线性，这并不是简单的逆问题，正因如此，DP 方法的梯度才十分重要。（上述观点也提示，强化学习可能是另一选择。在“用强化学习控制 Burgers 方程”中将比较 DP 与这些替代方案。）

19.2 不可压流体的控制

下面几节用 ΦFlow 介绍从数据生成到网络训练的全部必要步骤。由于控制问题复杂，先用监督学习初始化网络，再切换到更准确的 DP 端到端训练。注意，此例使用较旧的 ΦFlow 1.4.1。

下方代码复现 *Learning to Control PDEs with Differentiable Physics* [HKT19] 中的逆问题示例，即形状转换实验；更多细节见该论文附录 D.2 节。

首先加载 phiflow，并检出 PDE-Control Git 仓库；其中还含有以 NumPy 数组保存的初始形状。

```
!pip install --quiet phiflow==1.4.1

import matplotlib.pyplot as plt
from phi.flow import *

if not os.path.isdir('PDE-Control'):
    print("Cloning, PDE-Control repo, this can take a moment")
    os.system("git clone --recursive https://github.com/holl-/PDE-Control.git")

# 现在加载所需 phiflow 库和辅助函数
import sys; sys.path.append('PDE-Control/src')
from shape_utils import load_shapes, distribute_random_shape
```

[2]译注：分量动量方程右侧仍按源写作矢量梯度 ∇p，其分量形式待核；序列的紧凑复合记号也按源保留，OP 在不同时间尺度的具体调用由后面的执行方案处理，不能仅据该式推断为全程固定一次目标预测。

```
from control.pde.incompressible_flow import IncompressibleFluidPDE
from control.control_training import ControlTraining
from control.sequences import StaggeredSequence, RefinedSequence
```

19.3 数据生成

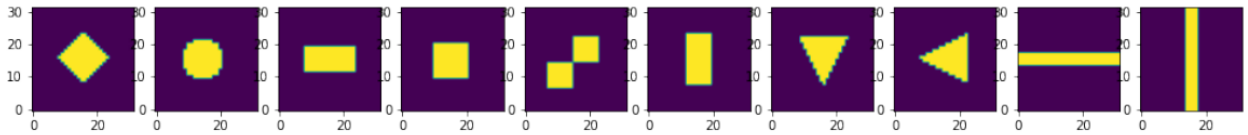
开始训练前，必须生成训练数据集，即一组真实时间序列 u^*。由于后面的训练较复杂，采用分阶段方法：先预训练监督网络作为粗略初始化，再逐步改进，使其学会更长时间范围的控制。这将通过训练处理越来越长序列的专用 NN 实现。

首先设置区域及数据生成步骤的基本参数。

```
domain = Domain([64, 64]) # 一维网格分辨率与物理大小
step_count = 16 # 执行多少个求解器步
dt = 1.0 # 每个求解器步的时间增量
example_count = 1000
batch_size = 100
data_path = 'shape-transitions'
pretrain_data_path = 'moving-squares'
shape_library = load_shapes('PDE-Control/notebooks/shapes')
```

最后一行的 `shape_library` 包含十种不同形状，用于在随机位置初始化标记密度。这些形状如下：[3]

```
import pylab
pylab.subplots(1, len(shape_library), figsize=(17, 5))
for t in range(len(shape_library)):
    pylab.subplot(1, len(shape_library), t+1)
    pylab.imshow(shape_library[t], origin='lower')
```



下面的单元用这些形状创建训练数据集。每个样本包含起始帧和目标（结束）帧；从 `shape_library` 中随机选择一个形状，放在区域内某处，分别生成两帧。

```
for scene in Scene.list(data_path): scene.remove()

for _ in range(example_count // batch_size):
    scene = Scene.create(data_path, count=batch_size, copy_calling_script=False)
    print(scene)
    start = distribute_random_shape(domain.resolution, batch_size, shape_library)
    end__ = distribute_random_shape(domain.resolution, batch_size, shape_library)
    [scene.write_sim_frame([start], ['density'], frame=f) for f in range(step_count)]
    scene.write_sim_frame([end__], ['density'], frame=step_count)
```

原书打印以下场景目录：

[3]译注：原代码注释写“一维网格”，但 `Domain([64,64])` 和本章问题均为二维，照录原注释并在此说明。

```
shape-transitions/sim_000000
shape-transitions/sim_000100
shape-transitions/sim_000200
shape-transitions/sim_000300
shape-transitions/sim_000400
shape-transitions/sim_000500
shape-transitions/sim_000600
shape-transitions/sim_000700
shape-transitions/sim_000800
shape-transitions/sim_000900
```

由于此数据集没有中间帧，无法进行监督预训练：预训练 CFE 网络需要两个连续帧，而预训练 OP_n 网络需要三个相距 $n/2$ 的帧。

因此另建包含这些中间帧的第二个数据集。由于它只用于通过预训练初始化网络，不需要与实际数据集很相近。这里让矩形在区域中线性移动。

```
for scene in Scene.list(pretrain_data_path): scene.remove()

for scene_index in range(example_count // batch_size):
    scene = Scene.create(pretrain_data_path, count=batch_size, copy_calling_script=False)
    print(scene)
    pos0 = np.random.randint(10, 56, (batch_size, 2)) # 起始位置
    pose = np.random.randint(10, 56, (batch_size, 2)) # 结束位置
    size = np.random.randint(6, 10, (batch_size, 2))
    for frame in range(step_count+1):
        time = frame / float(step_count + 1)
        pos = np.round(pos0 * (1 - time) + pose * time).astype(np.int)
        density = AABox(lower=pos-size//2, upper=pos-size//2+size).value_at(domain.center_points())
        scene.write_sim_frame([density], ['density'], frame=frame)
```

原书输出：

```
moving-squares/sim_000000
moving-squares/sim_000100
moving-squares/sim_000200
moving-squares/sim_000300
moving-squares/sim_000400
moving-squares/sim_000500
moving-squares/sim_000600
moving-squares/sim_000700
moving-squares/sim_000800
moving-squares/sim_000900
```

译注 第一数据集的代码实际上把同一起始状态写入 0 至 15 帧，末帧写目标；这些重复帧不是真实的物理中间轨迹。第二数据集最后的 `time` 为 16/17，不会精确到达 `pose`。`np.int` 属于旧 NumPy API；所有代码和 `scene.remove()` 清理语句均原样保留且未执行。

19.4 监督初始化

先将 1000 个样本划分为 100 个测试样本、100 个验证样本与 800 个训练样本。

```
test_range = range(100)
val_range = range(100, 200)
train_range = range(200, 1000)
```

下个单元训练所有 OP_n，$n \in \{2,4,8,16\}$。这里 n 表示网络预测目标所涉及的时间步数。为了覆盖更长时间范围，以二的幂次分层划分物理系统需要受控的时间区间。

用 `ControlTraining` 类设置相应优化问题。监督初始化的损失定义为中间帧速度的观测损失：

$$L_o^{\mathrm{sup}} = \left| \mathrm{OP}(d_{t_i}, d_{t_j}) - d^*_{(t_i+t_j)/2} \right|^2 .$$

因此，无需模拟序列，取 `sequence_class=None`。

只需在第 $n/2$ 帧计算观测损失，即设定 `obs_loss_frames=[n // 2]`。

预训练网络检查点保存在 `supervised_checkpoints` 中。[4]

原文提示 下个单元运行需要一些时间。PDE-Control Git 仓库附带预训练网络；若只关注评估，可注释训练单元，并取消下面加载单元的注释，跳过训练、直接加载预训练网络。

```
supervised_checkpoints = {}

for n in [2, 4, 8, 16]:
    app = ControlTraining(n, IncompressibleFluidPDE(domain, dt),
        datapath=pretrain_data_path, val_range=val_range, train_range=train_range,
        trace_to_channel=lambda _: 'density',
        obs_loss_frames=[n//2], trainable_networks=['OP%d' % n],
        sequence_class=None).prepare()
    for i in range(1000):
        app.progress() # 对一个批次执行优化
    supervised_checkpoints['OP%d' % n] = app.save_model()

supervised_checkpoints # 重新训练后的检查点保存于此
```

原书输出为：

```
{'OP16': '/root/phi/model/control-training/sim_000003/checkpoint_00001000',
 'OP2': '/root/phi/model/control-training/sim_000000/checkpoint_00001000',
 'OP4': '/root/phi/model/control-training/sim_000001/checkpoint_00001000',
 'OP8': '/root/phi/model/control-training/sim_000002/checkpoint_00001000'}
```

```
# supervised_checkpoints = {'OP%d' % n: 'PDE-Control/networks/shapes/supervised/OP%d_1000' % n for
↪  n in [2, 4, 8, 16]}
```

OP 网络预训练到此结束。它们至少能粗略规划运动，随后通过端到端训练改进。不过，在此之前还要初始化 CFE 网络，使其能够采取动作，即对仿真施加力；这一步与 OP 网络完全解耦。

[4] 译注：源文称“速度”，但公式与实际 `trace_to_channel` 取的是密度 `density`，两者照录并标明。

19.5 用可微物理预训练 CFE

为预训练 CFE 网络，设置一个仅运行一步可微求解器的仿真。

下个单元从头训练 CFE 网络。若已有预训练网络，可以跳过训练，通过执行随后单元加载检查点。

```
app = ControlTraining(1, IncompressibleFluidPDE(domain, dt),
    datapath=pretrain_data_path, val_range=val_range, train_range=train_range,
    trace_to_channel=lambda _: 'density',
    obs_loss_frames=[1], trainable_networks=['CFE']).prepare()
for i in range(1000):
    app.progress() # 对一个批次执行优化
supervised_checkpoints['CFE'] = app.save_model()

# supervised_checkpoints['CFE'] = 'PDE-Control/networks/shapes/CFE/CFE_2000'
```

注意，实际上并未为训练设置仿真，因为 CFE 网络只推断成对状态之间的力。[5]

19.6 用可微物理端到端训练

现在两种网络都有了初始版本，可以开始最关键的步骤：通过可微流体求解器，对两个网络进行耦合的端到端训练。尽管预训练阶段依赖监督训练，下一步会显著提高控制质量。

为用 phiflow 中的可微物理损失启动 CFE 和全部 OP_n 网络的端到端训练，采用交错执行方案，并新建 `ControlTraining` 实例。

下个单元构建含 `step_count` 个求解器步的计算图，不初始化网络权重。

```
staggered_app = ControlTraining(step_count, IncompressibleFluidPDE(domain, dt),
    datapath=data_path, val_range=val_range, train_range=train_range,
    trace_to_channel=lambda _: 'density',
    obs_loss_frames=[step_count], trainable_networks=['CFE', 'OP2', 'OP4', 'OP8', 'OP16'],
    sequence_class=StaggeredSequence, learning_rate=5e-4).prepare()
```

原书完整日志如下。`Partition` 表示在指定帧划分相应长度的序列，`Execute` 表示执行到指定步；其余依次显示目标损失、力损失、监督损失、损失设置、数据准备、变量初始化与验证值。末尾的字母 `n` 按冻结输出保留。

```
App created. Scene directory is /root/phi/model/control-training/sim_000005 (INFO), 2021-04-09
↪  00:41:17,299n
Sequence class: <class 'control.sequences.StaggeredSequence'> (INFO), 2021-04-09 00:41:17,305n
Partition length 16 sequence (from 0 to 16) at frame 8
Partition length 8 sequence (from 0 to 8) at frame 4
Partition length 4 sequence (from 0 to 4) at frame 2
Partition length 2 sequence (from 0 to 2) at frame 1
Execute -> 1
Execute -> 2
```

[5]译注：本句与本节开头“设置单步可微求解器仿真”冲突，代码也传入 `IncompressibleFluidPDE`。两处原陈述均保留，未经旧 API 实现核查不能消解其差异。

```
Partition length 2 sequence (from 2 to 4) at frame 3
Execute -> 3
Execute -> 4
Partition length 4 sequence (from 4 to 8) at frame 6
Partition length 2 sequence (from 4 to 6) at frame 5
Execute -> 5
Execute -> 6
Partition length 2 sequence (from 6 to 8) at frame 7
Execute -> 7
Execute -> 8
Partition length 8 sequence (from 8 to 16) at frame 12
Partition length 4 sequence (from 8 to 12) at frame 10
Partition length 2 sequence (from 8 to 10) at frame 9
Execute -> 9
Execute -> 10
Partition length 2 sequence (from 10 to 12) at frame 11
Execute -> 11
Execute -> 12
Partition length 4 sequence (from 12 to 16) at frame 14
Partition length 2 sequence (from 12 to 14) at frame 13
Execute -> 13
Execute -> 14
Partition length 2 sequence (from 14 to 16) at frame 15
Execute -> 15
Execute -> 16
Target loss: Tensor("truediv_16:0", shape=(), dtype=float32) (INFO), 2021-04-09 00:41:44,654n
Force loss: Tensor("truediv_107:0", shape=(), dtype=float32) (INFO), 2021-04-09 00:41:51,312n
Supervised loss at frame 16: Tensor("truediv_108:0", shape=(), dtype=float32) (INFO), 2021-04-09
↪  00:41:51,332n
Setting up loss (INFO), 2021-04-09 00:41:51,338n
Preparing data (INFO), 2021-04-09 00:42:32,417n
Initializing variables (INFO), 2021-04-09 00:42:32,443n
Model variables contain 0 total parameters. (INFO), 2021-04-09 00:42:36,418n
Validation (000000): Learning_Rate: 0.0005, GT_obs_16: 399498.75, Loss_reg_unscaled: 1.2424506,
↪  Loss_reg_scale: 1.0, Loss: 798997.5 (INFO), 2021-04-09 00:42:59,618n
```

下个单元用监督检查点初始化网络，再联合训练全部网络。可增加优化步数，或多次执行该单元，以进一步提高性能。

原文提示 下个单元运行需要一些时间。也可跳过，使用随后单元的代码加载预训练网络。

```
staggered_app.load_checkpoints(supervised_checkpoints)
for i in range(1000):
    staggered_app.progress() # 对一个批次执行交错优化
staggered_checkpoint = staggered_app.save_model()

# staggered_checkpoint = {net: 'PDE-Control/networks/shapes/staggered/all_53750' for net in ['CFE',
↪  'OP2', 'OP4', 'OP8', 'OP16']}
```

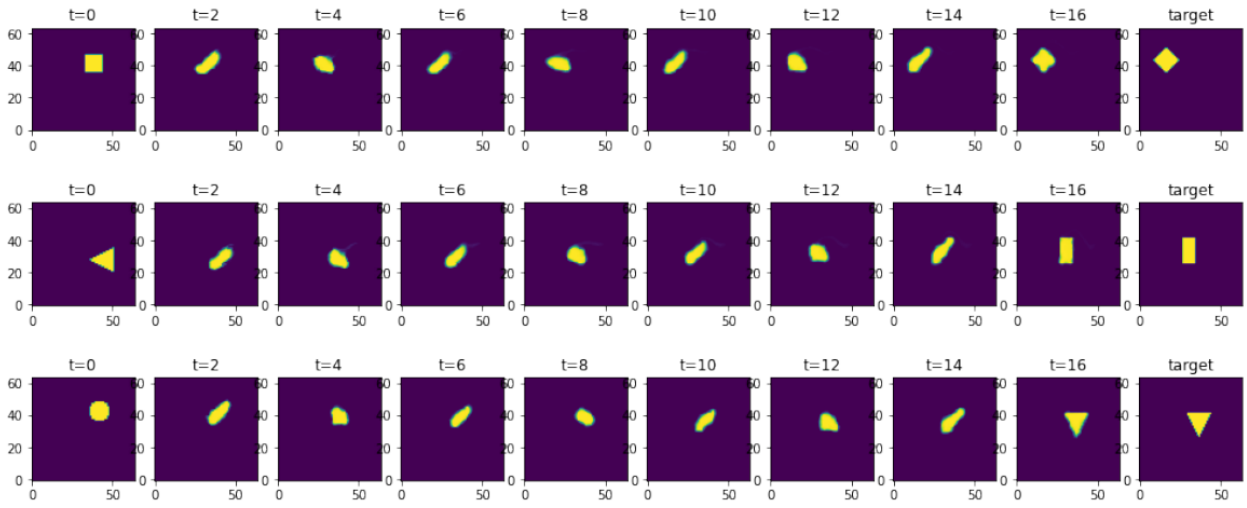
```
# staggered_app.load_checkpoints(staggered_checkpoint)
```

网络训练完成后，可以推断测试集中的一些轨迹；这对应原论文的图 5b 和图 18b。下个单元选取前一百个配置，即 `test_range` 定义的测试集，让网络推断相应逆问题的解。

```
states = staggered_app.infer_all_frames(test_range)
```

通过下面的索引列表 `batches`，可以选择显示部分解。每一行显示一个时间序列：从初始条件开始，在 NN 控制力作用下演化仿真 16 个时间步。最后一步 $t = 16$ 应与最右侧图像中的目标匹配。

```
batches = [0,1,2]

pylab.subplots(len(batches), 10, sharey='row', sharex='col', figsize=(14, 6))
pylab.tight_layout(w_pad=0)

# 解
for i, batch in enumerate(batches):
    for t in range(9):
        pylab.subplot(len(batches), 10, t + 1 + i * 10)
        pylab.title('t=%d' % (t * 2))
        pylab.imshow(states[t * 2].density.data[batch, ..., 0], origin='lower')

# 添加目标
testset = BatchReader(Dataset.load(staggered_app.data_path,test_range),
↪   staggered_app._channel_struct)[test_range]
for i, batch in enumerate(batches):
    pylab.subplot(len(batches), 10, i * 10 + 10)
    pylab.title('target')
    pylab.imshow(testset[1][i,...,0], origin='lower')
```



原书输出：三组控制轨迹，每行从 $t = 0$ 到 $t = 16$，最右列 target 为目标。

从最右侧两列可以看到，网络很好地求解了这些逆问题：流体标记被推向正确位置，并发生合适的变形，与目标吻合。

看起来简单的任务，对神经网络其实颇为棘手：它必须在 16 个时间积分步中引导完整二维 Navier–Stokes 仿真。只要施加的力稍有偏差或不协调，流体就可能开始旋转并作混沌运动。不过，网络已经学会保持运动的整体性，引导标记密度到达目标位置。

接下来，通过比较最终密度配置相对于初始密度的平均绝对误差，量化所得误差率。按上面标准训练设置，下个单元应给出 5%–6% 的相对残余误差。反过来说，这意味着约 94% 以上的标记密度到达了正确位置！[6]

```
errors = []
for batch in enumerate(test_range):
    initial = np.mean(np.abs(states[0].density.data[batch, ..., 0] - testset[1][batch,...,0]))
    solution = np.mean(np.abs(states[16].density.data[batch, ..., 0] - testset[1][batch,...,0]))
    errors.append(solution/initial)
print("Relative MAE: "+format(np.mean(errors)))
```

原书打印的相对 MAE 为：

```
Relative MAE: 0.05450168251991272
```

19.7　下一步

进一步实验可例如：

- 改变 `test_range` 索引，查看不同样本；或者用新形状作为目标，测试已训练控制器网络的泛化。
- 用 `RefinedSequence` 替代 `StaggeredSequence`，按预测细化方案训练。这会进一步改进控制、降低密度误差。

以上改进预期属于原书建议，未在本译稿执行验证。[HKT19] 的完整书目信息待全书文献阶段接入；冻结源物理 188 页仅有运行页眉页脚，没有正文内容。

[6] 译注：这是原书的解释；所列量是“末态与目标的 MAE”除以“初态与目标的 MAE”，不是直接测量落入目标区域的质量比例，因此不能由该比值严格推出 94% 的标记质量到位。下方循环还把 `enumerate(test_range)` 返回的二元组当作 `batch` 索引，不能直接用于任意测试索引。当前 `test_range=range(100)` 给出的二元组为 (j, j)；若底层为 NumPy 数组，它作为首轴高级索引会重复选择同一样本，随后求 mean 仍可能与单索引相同，因此不能只凭 tuple 断言当前 MAE 数值错误。绘制目标时使用 `i` 而非 `batch`，仅在当前 `batches=[0,1,2]` 下对应。全部照录而不修复。

第二十章　可微物理讨论

[1] 前面各节介绍了用于深度学习的可微物理方法，并给出一系列例子：从最基本的梯度计算，到由先进仿真支撑的复杂学习设置。现在适合退后一步作个评价：这些方法中的可微物理组件，归根结底并不太复杂。它们主要基于现有数值方法，重点在于高效地利用这些方法，不仅完成正向仿真，还计算梯度信息。这里最令人兴奋的，是这些数值方法与深度学习结合所带来的影响。

20.1　整合

最重要的是，通过可微物理训练，可以无缝结合这两个领域：得到混合方法，将现有最好的数值方法同时用于仿真本身和训练过程。然后，用训练后的模型改进正向或反向求解。因此，最终得到的求解器结合了传统求解器与学习组件，两者共同提高数值方法的能力。

20.2　通过交互减少数据偏移

要使这些混合方法有效，一个关键因素是让 NN 在训练时与 PDE 求解器交互。可微仿真使待训练模型能够“探索并体验”物理环境，并在整个求解器迭代过程中，获得针对其交互行为的定向反馈。

这可以应对机器学习中经典的数据偏移问题：训练网络不再依赖事先指定的分布，而是在训练过程中通过展开，实时生成新轨迹，并由这些轨迹计算训练信号。这可看作一种后验方法，使训练后的 NN 对未见输入有更强的适应能力。在“无条件稳定性”中将作更详细的评价：其他方法实际上很难胜过良好的展开设置。

注意，可微物理也自然属于更广泛的“将机器学习视作可微编程”这一背景。

20.3　泛化

对仿真器而言，混合方法还特别有前景：让 PDE 求解器处理数据分布的大尺度变化，能够改善训练后模型的泛化能力。于是，学习模型可以专注于离散化未捕捉到的局部结构。物理模型通常具有很好的泛化能力，而学习模型往往专门适应训练时见过的数据分布。因此，这一点受益于前述数据偏移的减少，并进一步容许输入分布存在更大的差异。若 NN 设置得当，混合方法中的经典求解器就能处理这些差异。

[1] 译注：（独立审校补充）硬约束只对实际投影后的状态、在数值容差内成立；若之后再加未经约束的神经修正，例如第 18 章的 $v_{\mathrm{sim}} + v_{\mathrm{corr}}$，并不自动保持约束。

例如，“用神经算子减少数值误差”中的模型展示了这些优点：训练后的模型可以应对物理行为显著变化的解流形，而更简单的训练变体会随递归时间步推进逐渐退化。

概括而言，通过 DP 训练 NN 的优缺点如下。

优点

- 使用物理模型和数值方法进行离散化。
- 所选方法的效率与精度可延续到训练中。
- 物理模型与 NN 可以非常紧密地耦合。
- 改善适应能力和泛化能力。

缺点

- 并非与所有仿真器兼容：需要提供梯度。
- 相比此前讨论的方法，需要更复杂的框架支持。

展望 最后这一项有关框架复杂性的缺点，目前正在明显改善。许多现有仿真器，例如流行的开源框架 OpenFoma，以及许多商业仿真器，都在推进与 NN 的紧密整合。不过，仍有很大改进空间。本书着重采用 phiflow 的例子，因为它从设计之初就考虑了与深度学习框架对接。[2]

通过可微物理（DP）训练，可以将完整的数值仿真纳入深度神经网络训练。这实际上提供了硬约束，因为耦合求解器可以像经典求解器一样进行投影并施加约束。这是一种十分通用的方法，适用于基于 PDE 的模型与深度学习的广泛组合。[3]

接下来的各章先扩展学习任务的范围，纳入不确定性，即处理完整分布，而非单个确定性状态与轨迹。随后，还将比较 DP 训练与强化学习，并着眼于底层学习过程，以获得更好的 NN 状态。

[2]译注：框架名按冻结源写作 OpenFoma，疑为专名误排；本段的“目前”指原书写作时，未作为 2026 年的软件现状重新核验。

[3]译注：“硬约束”按作者的概括保留；具体满足程度仍取决于离散模型、数值求解精度，以及所耦合求解器实际施加的约束。

第五部分

概率学习

第二十一章　概率学习导论

此前将目标函数 $f(x) = y$ 视为确定性的：每个输入都有唯一解 y。这当然是极大的简化。实际中，解可能有歧义，学习模型也可能混淆不同情形，两种效应还可能同时出现。因此，需要转向概率设定，本章就来讨论这一点。前面各节的方法会派上用场，因为概率视角实质上为问题引入了另一个维度。不再只有单个 y，而是有从分布 Y 抽取的多个解，每个解对应概率 $p_Y(y)$，常简写为 $p(y)$。从分布抽取的样本 $y \sim p(y)$ 应服从这一概率，从而区分罕见与常见的情形。

概括而言，现在面对的是大量样本 $y \sim p(y)$，而非单独的解 y。

21.1　不确定性

我们处理的一切测量、模型和离散化都存在不确定性。测量与观测中的不确定性，通常表现为测量误差。另一方面，模型方程一般只包含所关心系统的一部分，其余部分留下不确定性；数值仿真则不可避免地引入离散化误差。在机器学习背景下，还会有训练后模型引入的误差。所有这些误差与不明确因素，共同组成预测结果的不确定性，即**预测不确定性**。在实际应用中，必须能够量化这种不确定性。这是采用概率模型以及“不确定性量化”（UQ）等相邻领域的核心动机。

注：偶然不确定性与认知不确定性　许多情况下，预测不确定性可区分为两类：

- 偶然不确定性（aleatoric uncertainty）指数据自身的不确定性，例如测量噪声。
- 认知不确定性（epistemic uncertainty）则描述模型内部的不确定性，例如训练后的神经网络中的不确定性。

这里需要提醒：尽管这一划分看似明确，两种效应会叠加，可能难以区分。例如，面对离散化误差，不确定的结果可能来自数据中未知的歧义，也可能来自不理想的离散表示。在实践中，可能很难分离这些因素。

所谓**基于仿真的推断**（simulation-based inference，SBI）方法与之密切相关，但视角略有不同。这里的主要动机是估计计算机仿真中的似然，从而获得可靠的解概率分布。SBI 视角为处理计算机仿真与不确定性提供了方法论，也将成为后续各节的线索。

21.2　正向还是反向？

此时，需要重新审视正向问题与逆问题（“反向”问题）的核心区别：大多数经典数值方法面向正向问题，计算系统的稳态或未来状态。正向问题出现在许多场景中；但总体而言，逆问题至少同样常见。

在逆问题中，正向仿真仍起核心作用，不过主要问题不是仿真生成的状态，而是为解释某项测量或观测，仿真器参数应取什么值。

将其形式化：仿真器 f 由一组输入参数 ν（例如黏度）参数化，并接受状态 x，产生改变后的状态 y。已有观测 $\widetilde{y}$，希望求出能够产生该观测的 ν。最简单时，可将逆问题写成最小化问题

$$\arg\min_{\nu} |f(x;\nu) - \widetilde{y}|_2^2 .$$

求解后，就能得知被观测材料的黏度。从材料科学到宇宙学，几乎所有领域都有类似问题。为简化记号，下文将 ν 合并进 x，相应地对 x 最小化；但必须记住，x 可以包含逆问题中希望求解的任意一组参数或状态样本。

下文将着重讨论逆问题，因为它们最能展示概率建模的能力；不过，讨论的算法并非只适用于逆问题，后面会给出一个例子。

21.3 基于仿真的推断

在逆问题中，实践里只匹配单次观测 $\widetilde{y}$ 并不足够。更希望所得参数能够解释广泛的观测，也可能关心是否有多个参数值都能解释观测。同样，实际场景中还必须量化估计的不确定性：是只有很窄范围的参数能够解释观测，还是参数可以跨越多个数量级而几乎不影响观测？这些问题需要统计分析，通常称为推断，以便根据逆问题求解结果得出结论。联系前面对认知与偶然不确定性的划分，这里主要处理后者：在由仿真器给定科学假设后，观测中还有哪些不确定性？

为形式化这些逆问题，考虑向量值输入 x，其中可包含状态，以及前面提到的参数（如 ν），或两者兼有。另有潜变量分布 $z \sim p(z \mid x)$，描述系统中的未知部分。z 的例子包括不可观测的随机变量、中间仿真步骤或仿真器的控制流。

注：Bayes 定理 Bayes 定理是以下讨论的基础。为完整起见，列出

$$\begin{aligned} p(x \mid y)\, p(y) &= p(y \mid x)\, p(x) \\ &= p(x, y) = p(y, x). \end{aligned}$$

应记住两边都等于联合概率。

输入 x 有先验分布 X，概率密度为 $p(x)$；仿真器生成观测或输出 $y \sim p(y \mid x, z)$。因此，x 可以取不同值，可能含有噪声；z 不受我们控制，也同样可以影响所生成的 y。

条件概率函数 $p(y \mid x)$ 称为**似然函数**，是以下讨论中的关键量。注意，它不依赖 z，因为这些潜在状态不受我们控制。因而，实际需要对所有可能的 z 积分，计算边缘似然

$$p(y \mid x) = \int p(y, z \mid x)\, \mathrm{d}z.$$

这样做是必要的，因为似然函数不应依赖 z，否则就必须先知道 z 的确切值，才能计算似然。遗憾的是，这往往难以处理：z 可能很难采样，有时甚至无法合理控制。已有一些计算似然的算法，其中流行的一种是近似 Bayes 计算（ABC），但这些方法都非常昂贵，设置时需要大量专业知识。它们受到维数灾难的影响，即自由度数量增多时，计算会非常昂贵。因此，下文还会回到如何得到良好的似然近似这一主题。[1]

[1] 译注：本段关于各类算法代价及 ABC 的表述照原书保留；ABC 通常基于模拟与接受准则近似后验，不宜据本段概括理解成普遍显式求出似然。

有了似然函数，就可按以下方式计算主要目标量——后验分布：

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{\int p(y \mid x')p(x')\,\mathrm{d}x'}.$$

分母 $\int p(y \mid x')p(x')\,\mathrm{d}x'$ 称为**证据**。证据就是 $p(y)$，因此后验公式直接来自 Bayes 定理 $p(x \mid y) = p(y \mid x)p(x)/p(y)$。

证据可以用 Markov 链 Monte Carlo（MCMC）等随机方法计算。它主要“归一化”后验分布，通常比似然更容易获得，但仍是一个困难的量。[2]

利用深度学习 深度学习在这里极为有用：可以为后验 $p(x \mid y)$ 训练条件密度估计器 $q_\theta(x \mid y)$，使其支持采样，并且仅由仿真 $y \sim p(y \mid x)$ 训练。

对于准确估计后验分布这一经典难题，深度学习提供了新的处理途径，这正是本章重点。此前将神经网络记为 f_θ；下文改记为 $q_\theta = f_\theta$，以表明处理的是学习到的概率。具体而言，目标是学习概率密度的神经网络，即

$$\int q_\theta(x)\,\mathrm{d}x = 1.$$

通常先讨论无条件密度，再说明如何修改为学习条件版本 $q_\theta(x \mid y)$。

展望后文，学习型 SBI 方法，即计算后验分布的方法，具有如下性质。

优点

- 训练完成后，推断快速。
- 受维数灾难的影响较小。
- 可以表示任意先验。

缺点

- 前期训练代价高。
- 缺乏严格的理论保证。

以下将解释并推导一类非常流行而强大的方法，统称为**扩散模型**。当然可以直接给出最终算法，它会出奇地简单；但了解这些方法的来龙去脉也十分有趣。这里着重介绍基础，将基于物理的扩展，即加入可微仿真器，留待后节。通向扩散模型的道路，还会引出若干非常有趣的机器学习概念，为讨论近年的奠基性论文提供线索。下面开始。

[2]译注：此处为源书概括。普通 MCMC 产生归一化常数未知的目标分布样本，并不自动给出证据；估计证据通常需要附加方法与条件。

注：历史上的替代方法——Bayes 神经网络 还应提及一个经典变体："Bayes 神经网络"（BNN）。它更直接遵循 Bayes 思路，在神经网络参数上规定先验分布，以学习后验分布。假定 NN 的每个权重与偏置都为 Gaussian 分布，分别具有自己的均值与方差，并在训练时调整。推断时，可以"采样"一个网络，再像普通 NN 那样使用。尽管这一想法在理论上很好，但在学习复杂分布时遇到了问题，而且需要仔细调节涉及的超参数。因此，原书强烈建议如今采用流匹配，或至少采用扩散模型。若希望了解细节，可在 PBDL 的 v0.3 中找到 BNN 及代码示例：`https://arxiv.org/abs/2109.05237v3`。[3]

[3]译注：本段是作者的历史比较与建议，非本译稿对所有 BNN 的一般结论；链接指向的 arXiv v3 与本书冻结封面同有"v0.3"标记，不能据同名认定内容相同。本次不下载或替换源书。

第二十二章　学习概率分布

首先讨论一个普遍且十分基本的问题：如何学习概率分布？若对问题有一定了解，就可以选择某种分布，例如最简单的 Gaussian 分布；有了从目标分布抽取的样本，就可计算目标分布与待学习的参数化分布之间的差异来训练。

22.1　基础：训练目标

这里一个特别简单实用的度量，是概率分布 P 与 Q 之间的 Kullback–Leibler（KL）散度，二者密度分别为 p 与 q。它被广泛用作损失项，读者很可能已经见过。定义为

$$\mathrm{KL}(p\|q) = \int p(x) \log\left(\frac{p(x)}{q(x)}\right) \mathrm{d}x.$$

它“严格大于零”，$\mathrm{KL}(p\|q) \geq 0$，且具有一个良好性质：当且仅当 P 与 Q 相同时，$\mathrm{KL}(p\|q) = 0$。[1]

设有由 θ 参数化的密度族 $\{q_\theta\}_\theta$。必须强调，每个 q_θ 都需要是有效密度：非负，且在整个概率空间上的积分等于 1，即 $\int q_\theta(x)\,\mathrm{d}x = 1$。希望在这个密度族中，找到尽可能接近 p 的参数化密度。为此，可以最小化 $\mathrm{KL}(p\|q_\theta)$。

由 KL 散度得到的训练目标可改写为

$$\begin{aligned}\mathrm{KL}(p\|q_\theta) &= \int p(x) \log\left(\frac{p(x)}{q(x)}\right) \mathrm{d}x \\ &= \mathbb{E}_{x\sim p(x)}\left[\log\left(\frac{p(x)}{q_\theta(x)}\right)\right] \\ &= \mathbb{E}_{x\sim p(x)}[\log p(x)] - \mathbb{E}_{x\sim p(x)}[\log q_\theta(x)].\end{aligned}$$

第一项甚至不依赖 θ。因此，对 θ 的训练目标可简化为只最小化第二项 $\mathbb{E}_{x\sim p(x)}[-\log q_\theta(x)]$。也就是说，只需从 p 采样，并最小化 $q_\theta(x)$ 的负对数似然，就能训练 $q_\theta(x)$。[2]

[1] 译注：源文用语 “strictly larger than zero” 与随后的非负不等式和等号条件冲突；照录并说明。对密度作等号判断应理解为几乎处处相同。

[2] 译注：推导首行分母按源写 $q(x)$，后两行写 $q_\theta(x)$；隐含 p 固定、相关期望与密度支持满足使目标有意义的条件。

22.2 从无条件到有条件

这个十分简单的设定是无条件的：没有额外信息或输入，待学习的分布 P 固定。但许多情况下，希望以观测或附加输入 y 为条件来确定分布 P。当学习后验分布 $P \mid Y$ 时就是如此，其密度 $p(x \mid y)$ 依赖 y。此时，考虑条件密度 $p(x \mid y)$，而非无条件密度 $p(x)$。还需在候选密度族 $\{p(x \mid y)_\theta\}_\theta$ 中纳入信息 y。更新后的目标增加了对 y 采样的期望：

$$\mathbb{E}_{y\sim p(y)}\left[\mathbb{E}_{x\sim p(x|y)}[-\log q_\theta(x \mid y)]\right].$$

由 Bayes 定理，可以直接改写为

$$\begin{aligned}
&\mathbb{E}_{y\sim p(y)}\left[\mathbb{E}_{x\sim p(x|y)}[-\log q_\theta(x \mid y)]\right] \\
&\quad = -\iint p(y)p(x \mid y)\log p(x \mid y)\,\mathrm{d}x\,\mathrm{d}y \\
&\quad = -\iint p(y,x)\log p(x \mid y)\,\mathrm{d}x\,\mathrm{d}y \\
&\quad = -\iint p(x)p(y \mid x)\log p(x \mid y)\,\mathrm{d}y\,\mathrm{d}x \\
&\quad = \mathbb{E}_{x\sim p(x),\,y\sim p(y|x)}[-\log q_\theta(x \mid y)].
\end{aligned}$$

译注 上述等式链的三个积分中，冻结源都写 $\log p(x \mid y)$，两端却写 $\log q_\theta(x \mid y)$；一般并不相等。按源保留，不能将中间各式当作已经正确给出的推导。候选族的 p_θ 与损失中的 q_θ 记号差异亦照录。

在基于仿真的推断与科学机器学习中，$p(x)$ 表示先验，y 表示观测。给定先验样本 $x \sim p(x)$，可以例如通过数值仿真，采样观测 $y \sim p(y \mid x)$。对科学工作者最有意义、也最难获得的量，是后验 $p(x \mid y)$。

上面的公式告诉我们，可以在候选密度族 $\{p(x \mid y)_\theta\}_\theta$ 中寻找最优 θ，学习后验 $p(x \mid y)$。为此，需要最小化

$$\mathbb{E}_{x\sim p(x),\,y\sim p(y|x)}[-\log q_\theta(x \mid y)].$$

这只要求从先验抽取 x，再抽取观测 $y \sim p(y \mid x)$，而这两件事我们都知道如何做。

因此，即便对于逆问题的后验这类条件概率，也能采用极其简单的训练目标——当且仅当可以确保 q_θ 是概率密度。如何施加这一要求，就是下一节的主题。值得指出，对 Gaussian 密度作负对数似然训练，实际上等价于最小化 L^2 误差。这与之前“监督训练”等主题有良好的概念联系；但以下希望摆脱简单 Gaussian 分布的限制。[3]

22.3 用归一化流学习分布

深度学习领域已经提出多种学习分布与概率的方法，其中**归一化流**（normalizing flow）尤其有力 [KPB20]。一般而言，它将简单基分布 p_Z 变换为可能很复杂的目标分布 p_Y。核心思想是使用一串可逆且可微的映射作为神经网络层，名称中的“归一化”由此而来。这里的 p_Z 类似前述 SBI 中的潜在状态，但受到一个限制：必须是易于采样的简单分布。

[3]译注：Gaussian 负对数似然与未加权平方误差的这种等价，需固定且适当选取协方差；本句按源概括保留。

对单个可逆映射 $g:\mathbb{R}^D\to\mathbb{R}^D$ 及其逆函数 $f=g^{-1}$，有 $y=g(y)$，反之 $z=f(y)$。概率密度为

$$p_Y(y)=p_Z(f(y))\left|\det\frac{\partial f}{\partial y}\right|.$$

这里 $\partial f/\partial y$ 是 f 的 Jacobian，行列式的绝对值给出 f 对 y 的缩放。P_Z 通常是标准 Gaussian 分布，因此 $p_Z(f(y))$ 容易计算。[4]

若 p_Y 含内部参数，对其参数的对数似然也有类似关系：

$$\log p_Y(y)=\log p_Z(f(y))+\log\left|\det\frac{\partial f}{\partial y}\right|.$$

因此，随后进行最大似然训练的一种方便方式，是使用前述无条件或有条件的 KL 散度项。

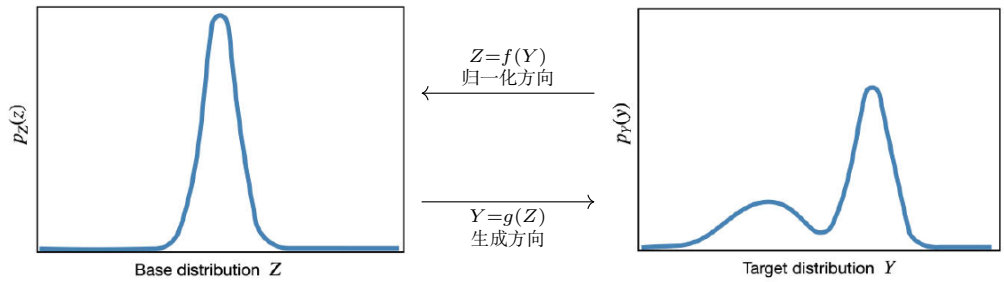


图 22.1: 直观示例：简单 Gaussian 分布（左）变换为非平凡的目标分布（右）。左横轴为基分布 Z、纵轴 $p_Z(z)$；右横轴为目标分布 Y、纵轴 $p_Y(y)$。原图 22.1；两个变换箭头与标签用可编辑 `tikzcd` 恢复。

注意，最大化概率密度等价于最大化似然，原书称这来自微积分基本定理，因此许多工作及以下说明会交替使用这两个说法。[5]

当然，并不限于单个函数 g 与 f。同样可以使用映射序列

$$g=g_1\circ g_2\circ\cdots\circ g_n,\qquad f=f_n\circ f_{n-1}\circ\cdots\circ f_1,$$

其中每个 g_i 都是 f_i 的逆。概率密度仍可写成

$$p_Y(y)=p_Z(f(Z))\prod_{i=1}^{n}\left|\det\frac{\partial f_i}{\partial y_i}\right|.$$

译注 末式的 $f(Z)$ 中大写 Z、各 y_i 的记号均按源保留；其与前述变量方向不一致，各 Jacobian 应在相应中间状态求值，源文没有在该式旁明确这些状态。

这种设定下，采样也很方便：从 $p_Z(z)$ 抽取随机向量，该分布通常为标准 Gaussian。通过 $y=g(z)$ 得到目标分布样本，再按上述方式将 $p_Z(z)$ 变换为 p_Y，计算其概率。y 由全部 g_i 构成的“正向”序列得到；“反向”序列 f_i 及其 Jacobian 则给出正确的概率密度。这非常方便，配合简单的负对数似然目标，既能无条件采样，也能有条件采样。

[4]译注：此处 $y=g(y)$ 按冻结源保留，与逆变换关系不相容；本节后文和图 22.1 写为 $y=g(z)$。

[5]译注：似然是在给定观测后将同一密度表达式视为参数函数；这种用语对应关系本身不需要用微积分基本定理证明。此处保留作者说明。

22.4 实际示例：学习 Gaussian 分布

用这些理论来学习概率分布。先定义由多个 Gaussian 组成的概率分布 P，使其支持采样和似然计算。[6]

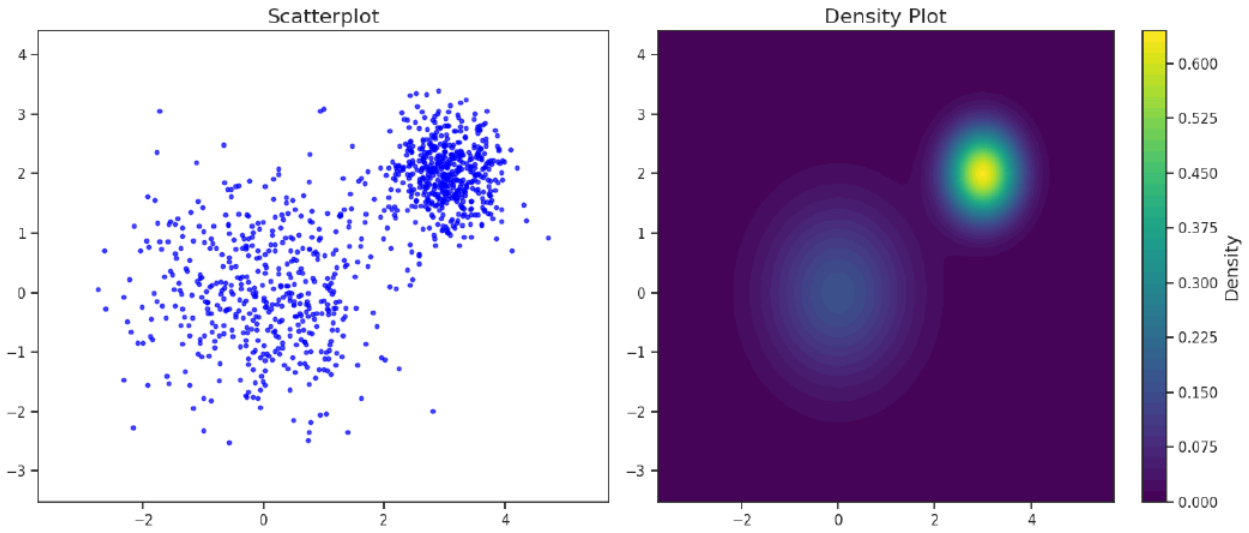
```
import numpy as np
class GaussianMixture:
    def __init__(self, parameters):
        self.parameters = parameters
        self.distributions = [
            {
                'mean': np.array(dist['mean']),
                'std': np.array(dist['std']),
                'cov': np.diag(np.array(dist['std']) ** 2)
            }
            for dist in parameters
        ]
    def sample(self, num_samples):
        samples = []
        num_distributions = len(self.distributions)
        for _ in range(num_samples):
            idx = np.random.randint(num_distributions) # Choose a random Gaussian
            dist = self.distributions[idx]
            sample = np.random.multivariate_normal(mean=dist['mean'], cov=dist['cov'])
            samples.append(sample)
        return np.array(samples)
    def likelihood(self, points):
        likelihoods = np.zeros(points.shape[0])
        for dist in self.distributions:
            mean = dist['mean']
            cov = dist['cov']
            inv_cov = np.linalg.inv(cov)
            det_cov = np.linalg.det(cov)
            # Multivariate Gaussian PDF
            factor = 1 / (2 * np.pi * np.sqrt(det_cov))
            diff = points - mean
            exponents = -0.5 * np.sum(diff @ inv_cov * diff, axis=1)
            likelihoods += factor * np.exp(exponents)
        return likelihoods
```

上面的 `GaussianMixture`（GM）类可以从多个二维 Gaussian 分布采样，也可计算不同位置的似然。现在看看样本如何分布，以及似然是什么样。先定义绘图函数，再画出由两个均值与方差不同的 Gaussian 组成的模型。

```
import matplotlib.pyplot as plt
import numpy as np
```

[6]译注：本章全部代码保留为可编辑文本，仅作 token/AST 核对，未执行训练、数据生成、安装命令或下载；图与日志全部来自冻结源。

```
import seaborn as sns
sns.set_theme(style="ticks")
def plot_gaussian_mixture(gm, samples, grid_size=100):
    x_min, x_max = np.min(samples[:, 0]) - 1, np.max(samples[:, 0]) + 1
    y_min, y_max = np.min(samples[:, 1]) - 1, np.max(samples[:, 1]) + 1
    x = np.linspace(x_min, x_max, grid_size)
    y = np.linspace(y_min, y_max, grid_size)
    X, Y = np.meshgrid(x, y)
    points = np.column_stack([X.ravel(), Y.ravel()])
    densities = gm.likelihood(points).reshape(grid_size, grid_size)
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    ax1 = axes[0]
    ax1.scatter(samples[:, 0], samples[:, 1], s=10, alpha=0.7, color="blue")
    ax1.set_title("Scatterplot", fontsize=16)
    ax1.set_xlim(x_min, x_max)
    ax1.set_ylim(y_min, y_max)
    ax2 = axes[1]
    contour = ax2.contourf(X, Y, densities, cmap="viridis", levels=50)
    cbar = fig.colorbar(contour, ax=ax2)
    cbar.set_label("Density", fontsize=14)
    ax2.set_title("Density Plot", fontsize=16)
    ax2.set_xlim(x_min, x_max)
    ax2.set_ylim(y_min, y_max)
    plt.tight_layout()
    plt.show()
parameters = [
    {"mean": [0, 0], "std": [1, 1]},
    {"mean": [3, 2], "std": [0.5, 0.5]}
]
gm = GaussianMixture(parameters)
samples = gm.sample(1000)
plot_gaussian_mixture(gm, samples)
```



原书输出：左为样本散点，右为密度图，色条为密度。

后续代码示例都以这个分布为起点。[7]

22.5 基于仿射耦合的简单归一化流

构建一个运用上述思想的简单网络。以三层、ReLU 激活的全连接 NN（以下称 FCNN）为构件，将其转成前述可逆层。下面单元给出基类 `RealNVP2D`，连接多个这样的构件，本例为 6 个，形成可训练的 NN，学习上面的玩具 GM 分布。

```
import torch.nn as nn
class FCNN(nn.Module):
    def __init__(self, input_dim, output_dim, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )
    def forward(self, x):
        return self.net(x)
class NVPBlock2D(nn.Module):
    def __init__(self, dim_flow, hidden_dim=256, flip=False):
        super().__init__()
        self.dim_flow = dim_flow
        self.hidden_dim = hidden_dim
        self.flip = flip
```

[7]译注：`sample` 等概率选择各分量，但 `likelihood` 将各分量密度直接相加，没有除以分量个数；因此返回值积分为分量个数，而非 1。本例两分量时，所画“密度/似然”是相应等权混合密度的两倍。未改代码或源图。

```
        self.f = FCNN((dim_flow // 2), dim_flow, hidden_dim)
    def shift_and_log_scale_fn(self, x1):
        s = self.f(x1)
        shift, log_scale = torch.chunk(s, 2, dim=1)
        return shift, log_scale
    def forward(self, x, ldj=None):
        d = self.dim_flow // 2
        x1, x2 = x[:, :d], x[:, d:]
        if self.flip:
            x1, x2 = x2, x1
        fcnn_input = x1
        shift, log_scale = self.shift_and_log_scale_fn(fcnn_input)
        y2 = x2 * torch.exp(log_scale) + shift
        if self.flip:
            x1, y2 = y2, x1
        z = torch.cat([x1, y2], dim=-1)
        if ldj is not None:
            ldj = ldj + log_scale.sum(dim=-1)
        return z, ldj
    def inverse(self, z, ldj=None):
        d = self.dim_flow // 2
        y1, y2 = z[:, :d], z[:, d:]
        if self.flip:
            y1, y2 = y2, y1
        fcnn_input = y1
        shift, log_scale = self.shift_and_log_scale_fn(fcnn_input)
        x2 = (y2 - shift) * torch.exp(-log_scale) # Apply inverse affine transformation
        if self.flip:
            y1, x2 = x2, y1
        x = torch.cat([y1, x2], dim=-1)
        if ldj is not None:
            ldj = ldj - log_scale.sum(dim=-1)
        return x, ldj
class RealNVP2D(nn.Module):
    def __init__(self, dim_flow, steps=6, hidden_dim=256):
        super().__init__()
        self.flows = nn.ModuleList()
        flip = False
        for _ in range(steps):
            self.flows.append(NVPBlock2D(dim_flow, hidden_dim, flip=flip))
            flip = not flip
    def forward(self, x, num_layers=None):
        if num_layers is None:
            num_layers = len(self.flows)
        ldj = torch.zeros(x.shape[0], device=x.device)
        for flow in self.flows[:num_layers]:
            x, ldj = flow(x, ldj)
```

```
        return x, ldj
    def inverse(self, z, num_layers=None):
        if num_layers is None:
            num_layers = len(self.flows)
        ldj = torch.zeros(z.shape[0], device=z.device)
        for flow in list(reversed(self.flows[:num_layers])):
            z, ldj = flow.inverse(z, ldj)
        return z, ldj
```

22.5.1 设置数据集并训练归一化流

直接从 GM 采样作为数据集，创建 RealNVP 模型，并按选定轮数训练，下面取 50 轮。

```
import torch
from torch.utils.data import DataLoader, TensorDataset
def generate_2d_gaussian_mixture(num_samples, gm):
    samples = gm.sample(num_samples)
    return torch.tensor(samples, dtype=torch.float32)
def train_model(model, dataloader, optimizer, num_epochs=50, device="cuda"):
    model.train()
    model.to(device)
    losses = []
    for epoch in range(num_epochs):
        epoch_loss = 0.0
        for x in dataloader:
            x = x[0].to(device)
            optimizer.zero_grad()
            z, ldj = model(x)
            prior = (-0.5 * z ** 2).sum(-1) - 0.5 * torch.log(torch.tensor(2.0 * torch.pi))
            loss = (-prior - ldj).mean()
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
        avg_loss = epoch_loss / len(dataloader)
        losses.append(avg_loss)
        print(f"Epoch {epoch + 1}/{num_epochs}, Loss: {avg_loss:.4f}")
    return losses
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
from sklearn.utils import shuffle
samples = generate_2d_gaussian_mixture(50000, gm)
samples = shuffle(samples.numpy())
dataset = TensorDataset(torch.tensor(samples, dtype=torch.float32))
dataloader = DataLoader(dataset, batch_size=128, shuffle=True)
dim_flow = 2
steps = 6
hidden_dim = 256
realnvp_model = RealNVP2D(dim_flow, steps, hidden_dim).to(device)
```

```
optimizer = torch.optim.Adam(realnvp_model.parameters(), lr=2e-4)
# Step 3: Train the model
num_epochs = 50
losses = train_model(realnvp_model, dataloader, optimizer, num_epochs=num_epochs, device=device)
```

原书训练输出如下，省略号为源书已有：

```
Epoch 1/50, Loss: 2.6399
Epoch 2/50, Loss: 1.9512
Epoch 3/50, Loss: 1.9392
Epoch 4/50, Loss: 1.9355
Epoch 5/50, Loss: 1.9310
Epoch 6/50, Loss: 1.9323
Epoch 7/50, Loss: 1.9339
Epoch 8/50, Loss: 1.9270
Epoch 9/50, Loss: 1.9281
Epoch 10/50, Loss: 1.9275
...
Epoch 40/50, Loss: 1.9234
Epoch 41/50, Loss: 1.9278
Epoch 42/50, Loss: 1.9236
Epoch 43/50, Loss: 1.9236
Epoch 44/50, Loss: 1.9216
Epoch 45/50, Loss: 1.9222
Epoch 46/50, Loss: 1.9236
Epoch 47/50, Loss: 1.9215
Epoch 48/50, Loss: 1.9226
Epoch 49/50, Loss: 1.9202
Epoch 50/50, Loss: 1.9213
```

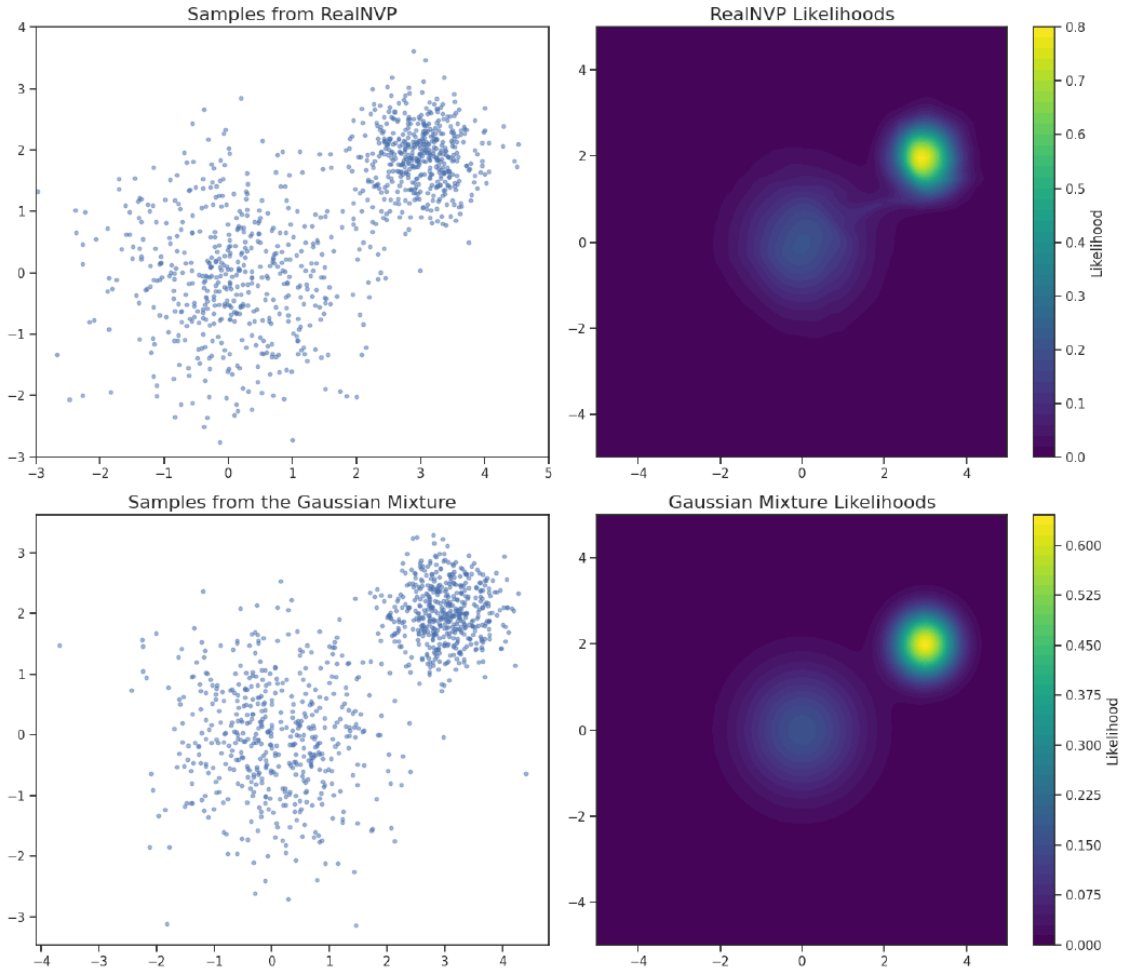
译注 训练中的 `prior` 为 $-\|z\|^2/2 - \log(2\pi)/2$，但二维标准 Gaussian 的常数项应为 $-\log(2\pi)$。原代码照录；该常数不改变此处关于模型参数的梯度，却会整体平移报告的负对数似然，并使随后直接指数化的密度多出常数因子。

22.5.2 可视化训练后归一化流的似然

选择简单 Gaussian 混合分布作学习目标，一个主要动机是可通过可视化轻松检验学习效果。因此，下面单元画出原分布与学习分布的样本，定性检查归一化流是否学到了目标分布的近似。还可在稠密网格上采样，显示似然；相应图像在右侧。

```
def visualize_training_results(model, gm, grid_size=100, dim=2, model_desc='Model'):
    model.eval()
    with torch.no_grad():
        z = torch.randn(1000, dim).to(device)
        samples, _ = model.inverse(z)
    samples = samples.cpu().numpy()
    gm_samples = gm.sample(1000)
```

```
    x = np.linspace(-5, 5, grid_size)
    y = np.linspace(-5, 5, grid_size)
    X, Y = np.meshgrid(x, y)
    points = np.column_stack([X.ravel(), Y.ravel()])
    with torch.no_grad():
        points_tensor = torch.tensor(points, device=device, dtype=torch.float32)
        z, ldj = model(points_tensor)
        prior = (-0.5 * z ** 2).sum(-1) - 0.5 * torch.log(torch.tensor(2.0 * torch.
  pi))
        model_likelihoods = torch.exp(prior + ldj).cpu().numpy().reshape(grid_size,
  grid_size)
        gm_likelihoods = gm.likelihood(points).reshape(grid_size, grid_size)
    fig, axes = plt.subplots(2, 2, figsize=(14, 12))
    axes[0, 0].scatter(samples[:, 0], samples[:, 1], s=10, alpha=0.5, label="")
    axes[0, 0].set_title(f"Samples from {model_desc}", fontsize=16)
    axes[0, 0].set_xlabel('')
    axes[0, 0].set_ylabel('')
    contour = axes[0, 1].contourf(X, Y, model_likelihoods, levels=50, cmap="viridis")
    fig.colorbar(contour, ax=axes[0, 1], label="Likelihood")
    axes[0, 1].set_title(f"{model_desc} Likelihoods", fontsize=16)
    axes[0, 1].set_xlabel('')
    axes[0, 1].set_ylabel('')
    axes[1, 0].scatter(gm_samples[:, 0], gm_samples[:, 1], s=10, alpha=0.5, label="")
    axes[1, 0].set_title("Samples from the Gaussian Mixture", fontsize=16)
    axes[1, 0].set_xlabel('')
    axes[1, 0].set_ylabel('')
    contour = axes[1, 1].contourf(X, Y, gm_likelihoods, levels=50, cmap="viridis")
    fig.colorbar(contour, ax=axes[1, 1], label="Likelihood")
    axes[1, 1].set_title("Gaussian Mixture Likelihoods", fontsize=16)
    axes[1, 1].set_xlabel('')
    axes[1, 1].set_ylabel('')
    for i in range(1):
        for j in range(1):
            axes[i, j].set_xlim([-3,5])
            axes[i, j].set_ylim([-3,4])
    plt.tight_layout()
    plt.show()
visualize_training_results(realnvp_model, gm, model_desc='RealNVP')
```



上行：RealNVP 样本与似然；下行：Gaussian 混合样本与似然。右列色条均表示似然，源代码的归一化差异见译注。

正如预期，网络学到的似然很好地近似了参考似然。[8]

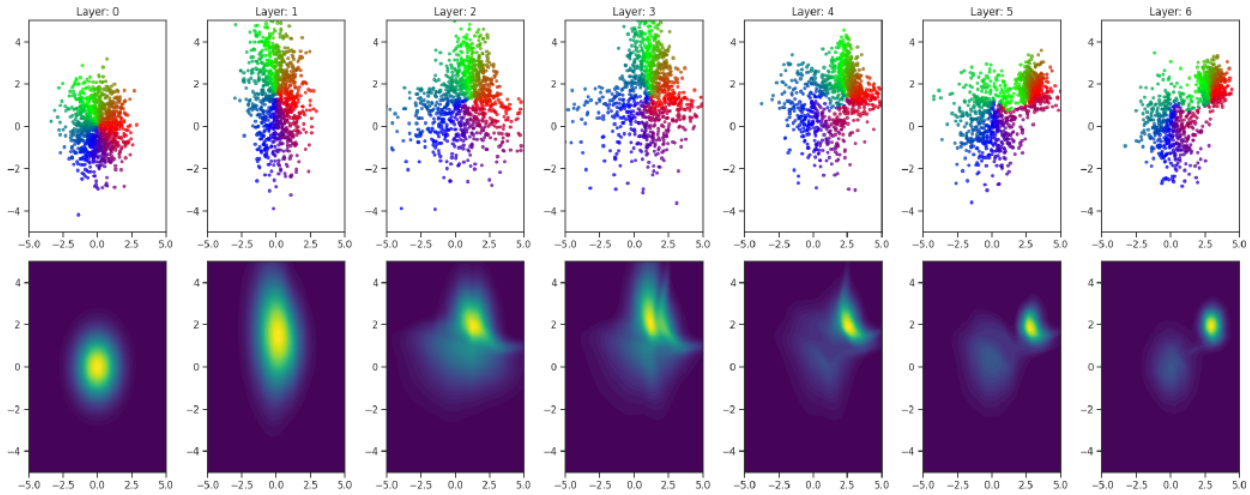
22.5.3 可视化不同网络层

可逆 RealNVP 网络有六层，逐步将先验分布变换为后验。每层映射都保持密度所对应的质量，因此可以逐步检查发生了什么。下面的单元展示这一点：

```
import matplotlib.pyplot as plt
def get_angle_colors(positions):
    angles = np.arctan2(positions[:, 1], positions[:, 0])
    angles_deg = (np.degrees(angles) + 360) % 360
    colors = np.zeros((len(positions), 3))
    for i, angle in enumerate(angles_deg):
        segment = int(angle / 120)
        local_angle = angle - segment * 120 # angle within segment [0, 120]
        if segment == 0:
```

[8]译注：这是原书的定性判断；前述两处归一化因子不同，不能仅凭形状相似和各自色条认定数值密度一致。代码的 `range(1)` 也仅给一个子图显式设置了该段坐标范围。

```
            colors[i] = [1 - local_angle/120, local_angle/120, 0]
        elif segment == 1:
            colors[i] = [0, 1 - local_angle/120, local_angle/120]
        else:
            colors[i] = [local_angle/120, 0, 1 - local_angle/120]
    return colors
def visualize_progression_with_layers_and_likelihoods(model, grid_size=100, num_layers_max=6,
↪  num_samples=1000):
    model.eval()
    fig, axes = plt.subplots(2, num_layers_max + 1, figsize=(20, 8))
    x = np.linspace(-5, 5, grid_size)
    y = np.linspace(-5, 5, grid_size)
    X, Y = np.meshgrid(x, y)
    points = np.column_stack([X.ravel(), Y.ravel()])
    points_tensor = torch.tensor(points, dtype=torch.float32).to(device)
    for num_layers in range(num_layers_max + 1):
        z = torch.randn(num_samples, model.flows[0].dim_flow).to(device)
        c = get_angle_colors(z.detach().cpu().numpy())
        with torch.no_grad():
            samples, _ = model.inverse(z, num_layers=num_layers)
        samples = samples.cpu().numpy()
        scatter_ax = axes[0, num_layers]
        scatter_ax.scatter(samples[:, 0], samples[:, 1], s=10, alpha=0.7, c=c)
        scatter_ax.set_title(f"Layer: {num_layers}")
        scatter_ax.set_xlim(-5, 5)
        scatter_ax.set_ylim(-5, 5)
        scatter_ax.set_xlabel("")
        scatter_ax.set_ylabel("")
        with torch.no_grad():
            z, ldj = model(points_tensor, num_layers=num_layers)
            prior = (-0.5 * z ** 2).sum(-1) - 0.5 * torch.log(torch.tensor(2.0 *
  torch.pi))
            likelihoods = torch.exp(prior + ldj).cpu().numpy().reshape(grid_size, grid_size)
        likelihood_ax = axes[1, num_layers]
        contour = likelihood_ax.contourf(X, Y, likelihoods, levels=50, cmap="viridis")
        likelihood_ax.set_xlim(-5, 5)
        likelihood_ax.set_ylim(-5, 5)
        likelihood_ax.set_xlabel("")
        likelihood_ax.set_ylabel("")
    plt.tight_layout()
    plt.show()
visualize_progression_with_layers_and_likelihoods(realnvp_model, grid_size=100, num_layers_max=6,
↪  num_samples=1000)
```



上行：Layer 0–6 各层对应的彩色样本；下行：相应密度形状。

有趣的是，网络到达目标所走的“路线”并不直观：先验分布以看似任意的方式扭曲，但逐步接近目标分布。中间分布没有约束，因此强烈依赖随机初始化。源文此处写网络 “only receives no gradients about the final distribution”，字面是“只接收不到关于最终分布的梯度”。于是，按原文接下来的解释，只要最终状态匹配，中间分布就可以保留其任意形状。[9]

22.6 神经 ODE：使归一化流连续化

前节基于仿射耦合层的归一化流共有六层，可以观察初始 Gaussian 分布如何逐步变换为目标分布。这类流的一项重大限制是架构必须可逆，而深度学习中多数高效架构并不可逆。

下一个目标是摆脱这一限制，以便采用任意架构。同时还要处理另一不足：当前层数固定，能否让层数可变？若把采样分布到目标分布的变换设想成连续过程，而非一串可逆映射，就可以引入人为的“时间” t。归一化流应从 $t=0$ 的采样分布，平滑地变换为 $t=1$ 的目标分布。

神经 ODE [CRBD19] 可以实现这一点，在物理仿真背景下尤其有趣。数学上，它以学习到的速度预测器 g_θ 替代映射 g_i，即

$$\frac{\partial z(t)}{\partial t} = g_\theta(z(t), t).$$

于是，构成 g 的各个 g_i 步骤，可由对速度积分替代，成为简单的 ODE 积分。引入连续时间轴，$t=0$ 对应标准 Gaussian 分布，$t=1$ 对应目标分布。沿这一时间轴求解 ODE，例如在沿途各时刻查询 g 给出的速度，将基分布 p_Z 变换为目标分布 p_Y。更好的是，ODE 求解的反向传播路径，其梯度具有解析表述。这是可微物理求解器，即 ODE 求解器，的一个巧妙例子，能高效计算梯度，也与“尺度不变性与反演”中的主题呼应。

概率随时间的变化，也可通过学习函数的迹方便地计算：

$$\frac{\partial \log p(z(t))}{\partial t} = -\operatorname{Tr}\left(\frac{\partial g_\theta}{\partial z(t)}\right).$$

[9] 译注：源句中 `only` 与 `no` 的组合疑有误排，与按最终分布损失训练的代码不协调；保留原句并标明，未静默删去 `no`。本例为无条件密度学习，此处原文仍称“先验/后验”。

与前面的归一化流相比，神经 ODE 视角下的一个重要区别是：现在只有一个函数 $g_\theta(\cdot, t)$，在时间区间 $t \in [0, 1]$ 的不同位置反复计算。乍看只是微小变化，却是走向扩散模型等更强概率模型的关键一步：可以复用一个学习函数，而无需手工构建许多含大量可训练参数的层，这一点很重要。

22.6.1 构建连续归一化流

下面用可逆动力学的自由形式 Jacobian（Free-form Jacobian of Reversible Dynamics，FFJORD）架构，实现连续归一化流。源文此处附有 “from here” 链接，其目标待原 PDF 链接注释核对后接入。

```
import torch.nn as nn
def kernel_init_fn():
    return nn.init.xavier_uniform_
def bias_init_fn():
    return nn.init.zeros_
class ConcatSquash(nn.Module):
    def __init__(self, in_size, out_size):
        super().__init__()
        self.out_size = out_size
        self.lin1 = nn.Linear(in_size, out_size)
        self.lin2 = nn.Linear(1, out_size)
        self.lin3 = nn.Linear(1, out_size, bias=False)
        kernel_init = kernel_init_fn()
        kernel_init(self.lin1.weight)
        kernel_init(self.lin2.weight)
        kernel_init(self.lin3.weight)
        bias_init = bias_init_fn()
        bias_init(self.lin1.bias)
        bias_init(self.lin2.bias)
    def forward(self, t, y):
        if t.dim() == 0:
            t = t.view(1, 1)
        elif t.dim() == 1:
            t = t.view(-1, 1)
        return self.lin1(y) * torch.sigmoid(self.lin2(t)) + self.lin3(t)
class FFJORD(nn.Module):
    def __init__(self, data_size, width_size, depth):
        super().__init__()
        self.data_size = data_size
        self.width_size = width_size
        self.depth = depth
        layers = []
        if self.depth == 0:
            layers.append(ConcatSquash(in_size=data_size, out_size=self.data_size))
        else:
            layers.append(ConcatSquash(in_size=data_size, out_size=self.width_size))
            for _ in range(self.depth - 1):
                layers.append(ConcatSquash(in_size=width_size, out_size=self.width_size))
```

```
            layers.append(ConcatSquash(in_size=width_size, out_size=self.data_size))
        self.layers = nn.ModuleList(layers)
    def forward(self, t, y):
        for layer in self.layers[:-1]:
            y = layer(t, y)
            y = torch.tanh(y)
        y = self.layers[-1](t, y)
        return y
```

22.6.2 连续归一化流网络

ODE 积分采用 torchdiffeq 包中的可微 ODE 求解器。

```
try:
    import google.colab # only to ensure that we are inside colab
    %pip install --quiet torchdiffeq
except ImportError:
    print("This notebook is running locally, please install torchdiffeq manually.")
```

ContinuousNormalizingFlow 类实现了通过 odeint 对神经速度估计器可微积分的基本功能。可微性十分重要，使损失及积分输出的梯度能够反向传播到 FFJORD 网络权重。

```
import torch
import torch.nn as nn
from torchdiffeq import odeint
class CNFVelocityFn(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers=8):
        super().__init__()
        self.net = FFJORD(data_size=input_dim, width_size=hidden_dim, depth=num_layers)
    def forward(self, t, combined_state):
        y, ldj = combined_state
        with torch.set_grad_enabled(True):
            y.requires_grad_(True)
            t.requires_grad_(True)
            t = torch.unsqueeze(t.repeat(y.shape[0]), 1)
            velocity = self.net(t, y)
            divergence = 0.0
            for i in range(y.shape[1]):
                divergence += torch.autograd.grad(velocity[:, i].sum(), y, create_graph=True)[0][:,
                ↪  i]
        return velocity, divergence.view(velocity.shape[0], 1)
class ContinuousNormalizingFlow(nn.Module):
    def __init__(self, input_dim, hidden_dim,
                 time_0=0.0, time_T=1.0):
        super().__init__()
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.time_0 = time_0
```

```
        self.time_T = time_T
        self.velocity_fn = CNFVelocityFn(input_dim=input_dim, hidden_dim=hidden_dim)
    def solveODE(self, x, t):
        batch_size, dim = x.shape
        assert dim == self.input_dim, "Input dimension mismatch!"
        y0 = x
        ldj0 = torch.zeros(batch_size, device=x.device)
        combined_state = (y0, ldj0)
        result = odeint(self.velocity_fn, combined_state, t,
                        method='dopri5',
                                    atol=[1e-5, 1e-5],
                                    rtol=[1e-5, 1e-5],
                        )
        final_y, final_ldj = result
        return final_y[-1], final_ldj[-1]
    def forward(self, x):
        t = torch.tensor([self.time_0, self.time_T], device=x.device)
        return self.solveODE(x, t)
    def inverse(self, x):
        t = torch.tensor([self.time_T, self.time_0], device=x.device)
        return self.solveODE(x, t)
```

译注 源代码中初始 `ldj0` 的形状为批大小，而返回的散度被重塑为 “批大小乘 1”，二者形状不一致，留待独立代码审校。这里只确认转写 token 和 Python 语法，不把 AST 成功当作运行正确性。代码的 `forward` 用 0 到 1 将训练数据映到基分布、`inverse` 用 1 到 0 生成样本；这与本节叙述中以 0 为 Gaussian 起点的时间约定相反，两种约定都按源保留。

22.6.3 训练

训练步骤可以复用上面的 `train_model`，因为数据格式、损失等基本形式都不变。只是将逐层的 “离散” 变换替换为连续积分版本。

```
samples = generate_2d_gaussian_mixture(5000, gm) # use fewer samples because training takes longer
samples = shuffle(samples.numpy())
dataset = TensorDataset(torch.tensor(samples, dtype=torch.float32))
dataloader = DataLoader(dataset, batch_size=128, shuffle=True)
input_dim = 2
hidden_dim = 128
time_0 = 0.0
time_T = 1.0
cnf_model = ContinuousNormalizingFlow(input_dim=input_dim, hidden_dim=hidden_dim, time_0=time_0,
↪  time_T=time_T)
learning_rate = 2e-4
optimizer = torch.optim.Adam(cnf_model.parameters(), lr=learning_rate)
num_epochs = 50
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
losses = train_model(cnf_model, dataloader, optimizer, num_epochs=num_epochs, device=device)
```

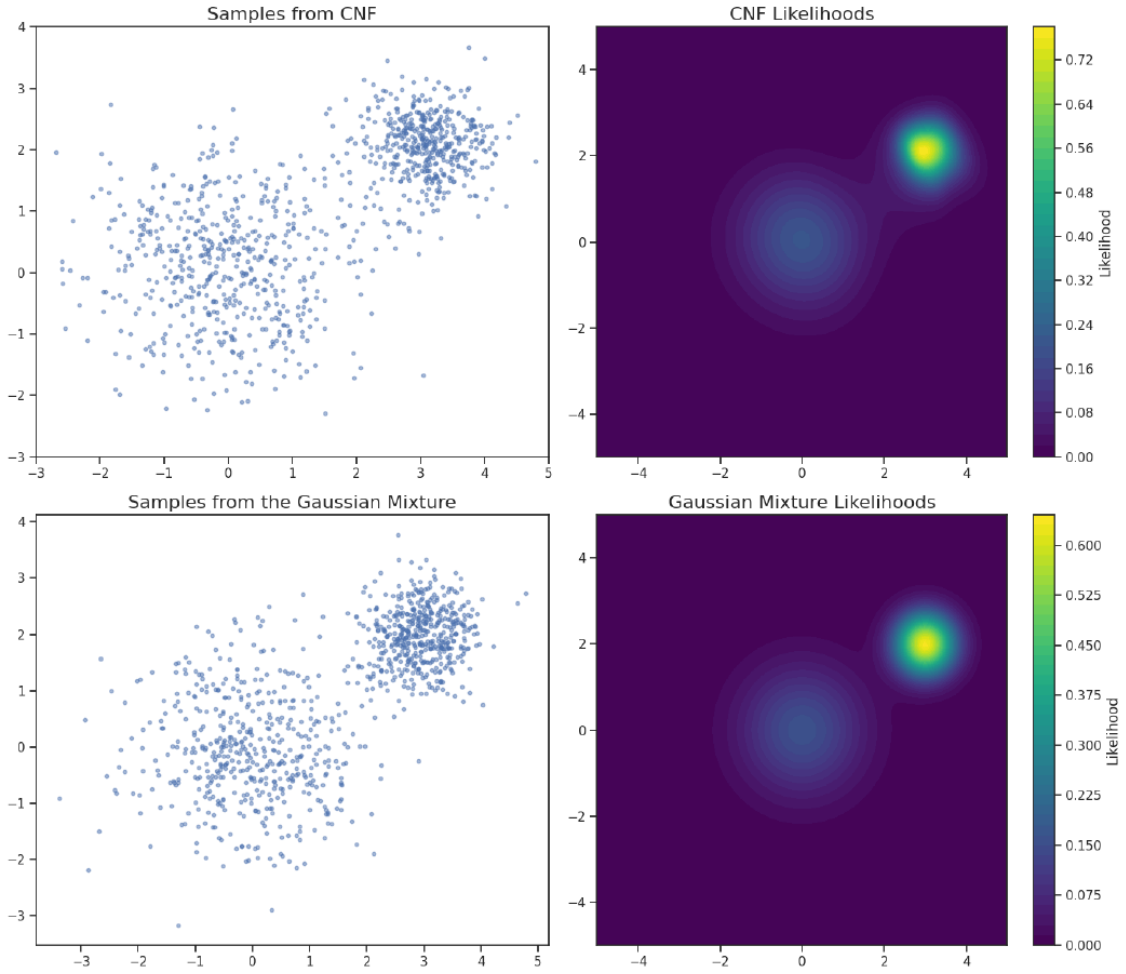
原书输出：

```
Epoch 1/50, Loss: 4.8563
Epoch 2/50, Loss: 2.7663
Epoch 3/50, Loss: 2.4124
Epoch 4/50, Loss: 2.3910
Epoch 5/50, Loss: 2.3755
Epoch 6/50, Loss: 2.3799
Epoch 7/50, Loss: 2.3673
Epoch 8/50, Loss: 2.3654
Epoch 9/50, Loss: 2.3439
Epoch 10/50, Loss: 2.3458
...
Epoch 40/50, Loss: 1.9560
Epoch 41/50, Loss: 1.9537
Epoch 42/50, Loss: 1.9505
Epoch 43/50, Loss: 1.9367
Epoch 44/50, Loss: 1.9524
Epoch 45/50, Loss: 1.9343
Epoch 46/50, Loss: 1.9524
Epoch 47/50, Loss: 1.9574
Epoch 48/50, Loss: 1.9440
Epoch 49/50, Loss: 1.9543
Epoch 50/50, Loss: 1.9453
```

22.6.4 可视化连续归一化流

现在重复此前步骤，看看第二个网络学到了什么。

```
visualize_training_results(cnf_model.to(device), gm, 100, model_desc='CNF')
```

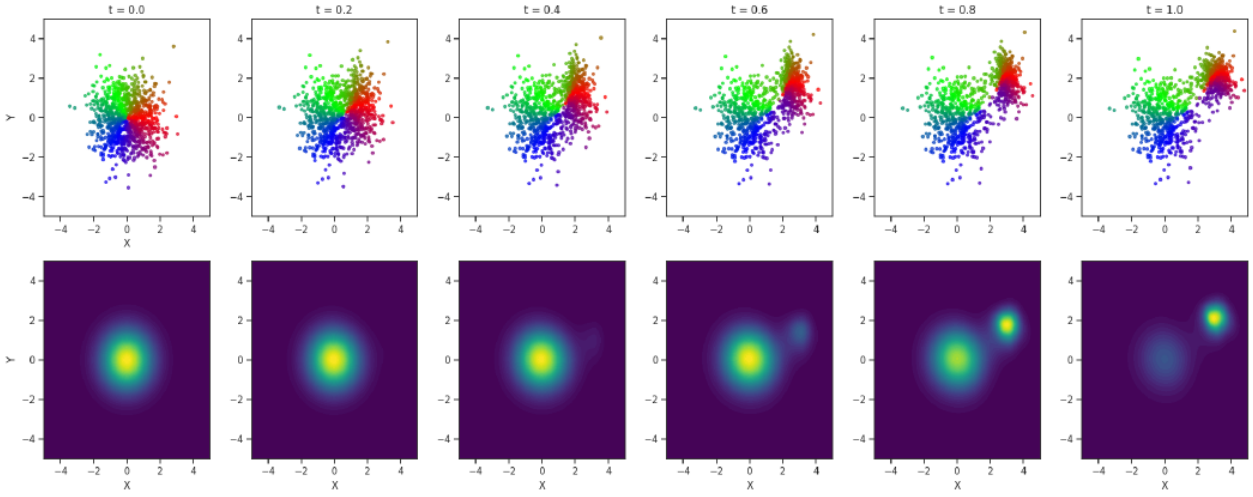


上行：CNF 样本与似然；下行：Gaussian 混合样本与似然。色条按源保留。

结果看起来不错。连续版本如何处理中间分布？下面进行可视化。

```
def visualize_progression_with_time_and_likelihoods(model, grid_size=100, num_timepoints=6,
↪  num_samples=1000):
    model.eval()
    timepoints = torch.linspace(0.0, 1.0, num_timepoints)
    fig, axes = plt.subplots(2, num_timepoints, figsize=(20, 8))
    x = np.linspace(-5, 5, grid_size)
    y = np.linspace(-5, 5, grid_size)
    X, Y = np.meshgrid(x, y)
    points = np.column_stack([X.ravel(), Y.ravel()])
    points_tensor = torch.tensor(points, dtype=torch.float32).to(device)
    z = torch.randn(num_samples, 2).to(device)
    c = get_angle_colors(z.cpu().numpy())
    eps = 1e-5
    for i, t in enumerate(timepoints):
        t_tensor = torch.tensor([0.0, t+eps]).to(device)
        t_tensor_inv = torch.tensor([1.0, 1.0-t-eps]).to(device)
        with torch.no_grad():
```

```
            samples, _ = model.solveODE(z, t_tensor_inv)
            samples = samples.cpu().numpy()
            z_t, ldj = model.solveODE(points_tensor, t_tensor)
            prior = (-0.5 * z_t ** 2).sum(-1) - 0.5 * torch.log(torch.tensor(2.0 *
  torch.pi))
            likelihoods = torch.exp(prior + ldj).cpu().numpy().reshape(grid_size,
  grid_size)
        scatter_ax = axes[0, i]
        scatter_ax.scatter(samples[:, 0], samples[:, 1], s=10, alpha=0.7, c=c)
        scatter_ax.set_title(f"t = {t:.1f}")
        scatter_ax.set_xlim(-5, 5)
        scatter_ax.set_ylim(-5, 5)
        scatter_ax.set_xlabel("X" if i == 0 else "")
        scatter_ax.set_ylabel("Y" if i == 0 else "")
        likelihood_ax = axes[1, i]
        contour = likelihood_ax.contourf(X, Y, likelihoods, levels=50, cmap="viridis")
        likelihood_ax.set_xlim(-5, 5)
        likelihood_ax.set_ylim(-5, 5)
        likelihood_ax.set_xlabel("X")
        likelihood_ax.set_ylabel("Y" if i == 0 else "")
    plt.tight_layout()
    plt.show()
visualize_progression_with_time_and_likelihoods(
    cnf_model,
    grid_size=100,
    num_timepoints=6,
    num_samples=1000
)
```



上行从 $t = 0.0$ 到 1.0 的彩色样本，下行为相应密度。

图中显示，与离散层实现相比，连续时间归一化流能将 Gaussian 采样分布更平滑地变换为目标分布。

22.7 归一化流小结

这是一个很好的结果。利用 ODE 求解知识，速度模型基本上可采用任意神经网络架构。根据现有计算预算，可以选择不同求解器与步长，在 ODE 求解速度和精度之间权衡。

但这种方法也有缺点。训练连续归一化流时，为了高精度计算似然，必须求解整个 ODE，将目标分布样本从 $t=1$ 输运到 $t=0$ 的 Gaussian 分布。这需要大量网络计算，代价高昂。因此，神经 ODE 很难扩展到高维数据与大型神经网络。[10]

下一节将着重介绍更容易扩展的方法，它们可以结合极大的网络与高维数据。[KPB20]、[CRBD19] 的完整书目待全书文献单元接入；冻结源物理 216 页仅有运行页眉页脚，没有正文。

译注：源实现与结论的适用范围 神经 ODE 可逆与密度迹公式需要解存在唯一、足够光滑（或适当几乎处处条件）及有限时间流；$\log p(z(t))$ 的导数是沿轨线全导数。源 `ldj0` 为 (B) 而其导数为 (B,1)，不满足 ODE 状态/导数同形；若保持 (B,1) 再加 (B) 先验会广播为 (B,B)，不是逐样本相加。最后可视化散点用 $1 \to 1-t-\epsilon$ 的流段，密度用 $0 \to t+\epsilon$，非自治网络一般不是同一中间分布。反例 $g(z,t)=t$、基分布均值 0、忽略 epsilon，二者中间均值分别为 $-t+t^2/2$ 与 $-t^2/2$，在 $t=1/2$ 为 $-3/8, -1/8$，虽端点相同仍不一致。冻结 PDF 相关页无可恢复链接注释，不猜补外部地址。

[10] 译注：这里的时间方向遵从原书文字；上方实现所用的方向差异已在代码后注明。

第二十三章　得分匹配

首先有一个重要认识：概率密度有一个方便的替代量，使我们能够处理未归一化函数，即所谓**得分**（score）。它是对数似然函数梯度的通用名称：$\nabla_x \log p(x)$。用神经网络学习这个函数，记作 $s_\theta(x)$，以表明它是学习到的得分。好处在于：不必再担心归一化。积分时需要正确的常数偏移，而对于 x 处的“局部”梯度，只需要该点的导数。

23.1　具有解析得分的 Gaussian 玩具数据集

继续使用前面相同的 Gaussian 混合例子。不过，为了实验得分函数，在基类中增加 `score()` 函数，以便检查学习近似的效果。幸运的是，可直接从 Gaussian 混合模型计算参考得分。[1]

```
import numpy as np
class GaussianMixture:
    def __init__(self, parameters):
        self.parameters = parameters
        self.distributions = [
            {
                'mean': np.array(dist['mean']),
                'std': np.array(dist['std']),
                'cov': np.diag(np.array(dist['std']) ** 2)
            }
            for dist in parameters
        ]
    def sample(self, num_samples):
        samples = []
        num_distributions = len(self.distributions)
        for _ in range(num_samples):
            idx = np.random.randint(num_distributions)
            dist = self.distributions[idx]
            sample = np.random.multivariate_normal(mean=dist['mean'], cov=dist['cov'])
            samples.append(sample)
        return np.array(samples)
    def likelihood(self, points):
        likelihoods = np.zeros(points.shape[0])
        for dist in self.distributions:
```

[1]译注：本章代码全部按冻结 PDF 转写，做程序 token 与 AST 检查而不执行；训练、数据生成、依赖安装均未运行，图和日志均来自原书。

```
            mean = dist['mean']
            cov = dist['cov']
            inv_cov = np.linalg.inv(cov)
            det_cov = np.linalg.det(cov)
            factor = 1 / (2 * np.pi * np.sqrt(det_cov))
            diff = points - mean
            exponents = -0.5 * np.sum(diff @ inv_cov * diff, axis=1)
            likelihoods += factor * np.exp(exponents)
        return likelihoods
    def score(self, points):
        scores = np.zeros_like(points)
        likelihoods = np.zeros(points.shape[0])
        component_likelihoods = []
        for dist in self.distributions:
            mean = dist['mean']
            cov = dist['cov']
            inv_cov = np.linalg.inv(cov)
            det_cov = np.linalg.det(cov)
            factor = 1 / (2 * np.pi * np.sqrt(det_cov))
            diff = points - mean
            exponents = -0.5 * np.sum(diff @ inv_cov * diff, axis=1)
            comp_likelihood = factor * np.exp(exponents)
            component_likelihoods.append(comp_likelihood)
            likelihoods += comp_likelihood
        for dist, comp_likelihood in zip(self.distributions, component_likelihoods):
            mean = dist['mean']
            inv_cov = np.linalg.inv(dist['cov'])
            weights = comp_likelihood / (likelihoods + 1e-8)
            diff = points - mean
            component_score = -(diff @ inv_cov)
            scores += np.multiply(weights[:, np.newaxis], component_score)
        return scores
```

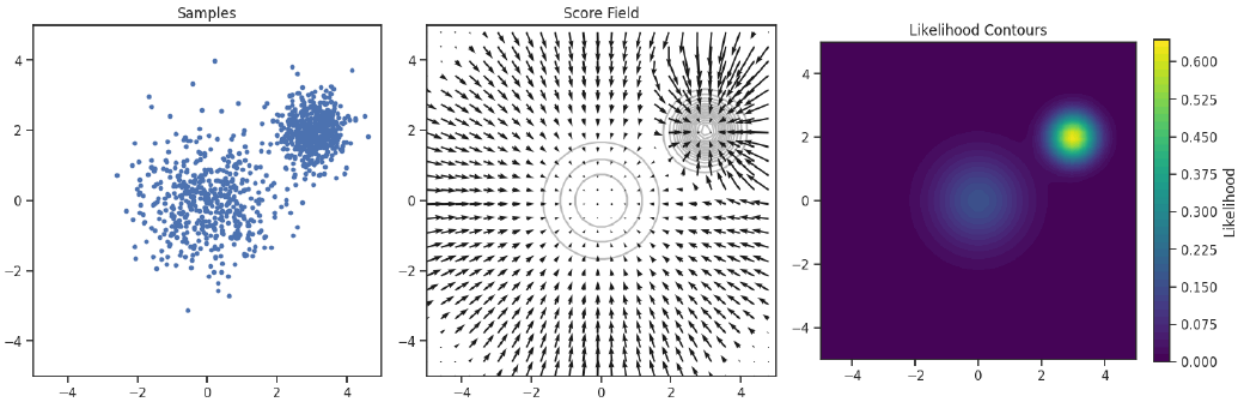
译注 与前章一样，`likelihood` 未除以分量个数，返回的是未归一化密度之和。归一化常数在对数梯度中会消去，但 `score` 分母另加 10^{-8}；低密度尾部会因此受到明显影响，不能将这个数值实现处处等同于精确解析得分。

23.1.1 可视化样本、似然与得分

直接定义一个辅助函数，在一张图中显示样本、得分场和似然。

```
from matplotlib import pyplot as plt
import seaborn as sns
sns.set_theme(style="ticks")
def visualize_gaussian_mixture_with_score(mixture, num_samples=1000, grid_size=50):
    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 6))
    # Create grid for visualization
```

```
    x = np.linspace(-5, 5, grid_size)
    y = np.linspace(-5, 5, grid_size)
    X, Y = np.meshgrid(x, y)
    points = np.column_stack([X.ravel(), Y.ravel()])
    # Compute likelihoods and scores
    likelihoods = mixture.likelihood(points)
    scores = mixture.score(points)
    samples = mixture.sample(num_samples)
    # Reshape for plotting
    likelihoods = likelihoods.reshape(grid_size, grid_size)
    scores_x = scores[:, 0].reshape(grid_size, grid_size)
    scores_y = scores[:, 1].reshape(grid_size, grid_size)
    ax1.scatter(samples[:, 0], samples[:, 1], s=10, alpha=1)
    ax1.set_title("Samples")
    skip = 2
    ax2.quiver(X[::skip, ::skip], Y[::skip, ::skip],
               scores_x[::skip, ::skip], scores_y[::skip, ::skip],
               scale=100, width=0.005)
    # Add likelihood contours to score plot for reference
    ax2.contour(X, Y, likelihoods, levels=20, colors='k', alpha=0.3)
    ax2.set_title("Score Field")
    # Plot likelihood contours
    contour = ax3.contourf(X, Y, likelihoods, levels=50, cmap='viridis')
    ax3.set_title("Likelihood Contours")
    plt.colorbar(contour, ax=ax3, label='Likelihood', fraction=0.046)
    # Plot score field
    # Subsample the grid for clearer arrows
    # Set consistent limits and labels
    for ax in [ax1, ax2, ax3]:
        ax.set_xlim(-5, 5)
        ax.set_ylim(-5, 5)
        ax.set_aspect('equal')
        ax.set_xlabel("")
        ax.set_ylabel("")
    plt.tight_layout()
    plt.show()
parameters = [
    {"mean": [0, 0], "std": [1, 1]},
    {"mean": [3, 2], "std": [0.5, 0.5]}
]
mixture = GaussianMixture(parameters)
visualize_gaussian_mixture_with_score(mixture)
```



从左到右为样本、得分向量场和似然等高图；右侧色条为似然，均按源保留。

必须记住，通常只知道来自目标分布的样本，并不知道其似然。事实证明，这种情况下，获取得分比获取似然容易得多。这是为什么？

为估计某个单独样本的似然，需要目标分布中的大量独立样本进行比较。主要困难是：若用候选密度 q_θ 近似目标密度 p，就必须保证在整个概率空间上积分为 1，即密度的标准要求之一 $\int q_\theta(x)\,\mathrm{d}x = 1$。为了得到单个样本 x 的正确密度值，仍需满足密度积分为 1 这一归一化要求。

归一化流通过模型与网络的构造，始终满足这一条件。但如前节所见，归一化流难以扩展到高维数据，计算也昂贵。

对于通用而灵活的方法，确保 q_θ 的积分为 1 极其困难。得分之所以有趣，是因为它只依赖局部信息：它考虑对数似然的梯度，因而只依赖似然的局部值。这使神经网络更容易学习得分。

先探索如何通过神经网络，从目标分布样本学习得分；随后介绍第一种利用得分从目标分布采样的方法。

23.2 学习得分

可以用 L^2 项直接学习对数似然梯度 $\nabla_x \log p(x)$。在这里，L^2 也称为 “Fisher 散度”：

$$\mathbb{E}_{x\sim p(x)}\left[\left\|\nabla_x \log p(x) - s_\theta(x)\right\|^2\right].$$

但这要求知道真实梯度 $\nabla_x \log p(x)$，在有意义的实际设定中，通常并不知道。机器学习中一个已确立的技巧，是用 Gaussian 噪声轻微扰动数据集，从而计算学习目标 [Vin11]。这将逐点样本集合转成能够计算梯度的连续函数。

具体地，对数据集 $\{x_1, \ldots, x_n\}$ 添加 Gaussian 噪声，得到扰动数据集 $\{\widetilde{x}_1, \ldots, \widetilde{x}_n\}$，其中 $\widetilde{x} = x + \sigma z$，$z \sim \mathcal{N}(0, I)$。暂时将噪声水平 $\sigma > 0$ 固定在某个值，但它是一个关键参数，稍后会再次讨论。

扰动样本的分布 p_σ，可写成对条件进行边缘化的条件分布：

$$\begin{aligned} p_\sigma(\widetilde{x}) &= \int p_\sigma(\widetilde{x} \mid x) p(x)\,\mathrm{d}x \\ &= \mathbb{E}_{x\sim p(x)}[p_\sigma(\widetilde{x} \mid x)]. \end{aligned}$$

噪声水平 σ 越小，扰动密度 p_σ 与原密度 p 越接近，即 $\lim_{\sigma\to 0}\mathrm{KL}(p_\sigma\|p)=0$。直观而言，需要噪声来计算梯度，但噪声又不宜大到扭曲目标分布。[2]

利用扰动密度由 Gaussian 构造这一点，就能计算得分梯度。条件密度的解析形式为

$$p_\sigma(\widetilde{x}\mid x)=\frac{1}{\sqrt{(2\pi)^D}\,\sigma^D}\exp\left[-\frac{1}{2\sigma^2}(\widetilde{x}-x)^T(\widetilde{x}-x)\right], \tag{23.1}$$

其得分非常简单：

$$\nabla_{\widetilde{x}}\log p_\sigma(\widetilde{x}\mid x)=-\frac{\widetilde{x}-x}{\sigma^2}. \tag{23.2}$$

利用以下恒等式，就可以用条件密度训练 s_θ，近似扰动数据集的得分：

$$\begin{aligned}&\arg\min_\theta \mathbb{E}_{\widetilde{x}\sim p_\sigma(\widetilde{x})}\left[\|s_\theta(\widetilde{x})-\nabla_{\widetilde{x}}\log p_\sigma(\widetilde{x})\|^2\right]\\&=\arg\min_\theta \mathbb{E}_{x\sim p(x),\,\widetilde{x}\sim p_\sigma(\widetilde{x}|x)}\left[\|s_\theta(\widetilde{x})-\nabla_{\widetilde{x}}\log p_\sigma(\widetilde{x}\mid x)\|^2\right].\end{aligned}$$

23.2.1 定义得分网络与数据集

为在 Gaussian 混合问题中实现这些想法，先定义具有四个隐藏层的简单得分网络，以及按上述方式扰动样本的数据集类。`DenoisingScoreMatchingDataset` 的参数 `sigma` 控制扰动大小。[3]

```
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
class ScoreNetwork(nn.Module):
    def __init__(self, input_dim=2, hidden_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, input_dim)
        )
    def forward(self, x):
        return self.net(x)
class DenoisingScoreMatchingDataset(Dataset):
    def __init__(self, gaussian_mixture, num_samples=10000, sigma=0.1):
        self.samples = torch.FloatTensor(gaussian_mixture.sample(num_samples))
        self.sigma = sigma
    def __len__(self):
        return len(self.samples)
    def __getitem__(self, idx):
```

[2]译注：KL 极限按源陈述；它并非对所有目标分布无条件成立，须有支持、绝对连续性及适当的可积/正则性条件。例如离散目标不能直接套用该密度 KL 极限。

[3]译注：“四个隐藏层”按原文保留；实际 `Sequential` 包含四个 `Linear`，其中前三个后接 `SiLU`，第四个为输出层，即通常所称的三个隐藏层。

```
        x = self.samples[idx]
        noise = torch.randn_like(x) * self.sigma
        x_noisy = x + noise
        return x_noisy, noise, x
```

23.2.2 训练

现在训练得分网络，估计扰动数据集的梯度。目标由下方数据集中的扰动 `noise` 除以 σ^2 得到。

```
import torch
from torch import optim
def train_score_network(model, train_loader, num_epochs=100, sigma=0.1, device='cpu'):
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
    model.train()
    losses = []
    for epoch in range(num_epochs):
        epoch_loss = 0
        for batch_idx, (x_noisy, noise, x_clean) in enumerate(train_loader):
            x_noisy, noise = x_noisy.to(device), noise.to(device)
            optimizer.zero_grad()
            pred_score = model(x_noisy)
            target_score = -noise / (sigma**2)
            loss = torch.mean(torch.sum((pred_score - target_score)**2, dim=1))
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
        avg_loss = epoch_loss / len(train_loader)
        losses.append(avg_loss)
        if (epoch + 1) % 10 == 0:
            print(f'Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss:.4f}')
    return losses
# Create dataset and dataloader
dataset = DenoisingScoreMatchingDataset(mixture, num_samples=10000, sigma=0.1)
dataloader = DataLoader(dataset, batch_size=128, shuffle=True)
# Initialize and train the model
device = 'cuda' if torch.cuda.is_available() else 'cpu'
score_net = ScoreNetwork().to(device)
losses = train_score_network(score_net, dataloader, num_epochs=100, device=device)
```

原书输出：

```
Epoch 10/100, Loss: 192.8589
Epoch 20/100, Loss: 197.8407
Epoch 30/100, Loss: 196.8402
Epoch 40/100, Loss: 198.7428
Epoch 50/100, Loss: 194.3992
Epoch 60/100, Loss: 198.6634
Epoch 70/100, Loss: 194.6108
```

```
Epoch 80/100, Loss: 197.0947
Epoch 90/100, Loss: 191.6466
Epoch 100/100, Loss: 197.7943
```

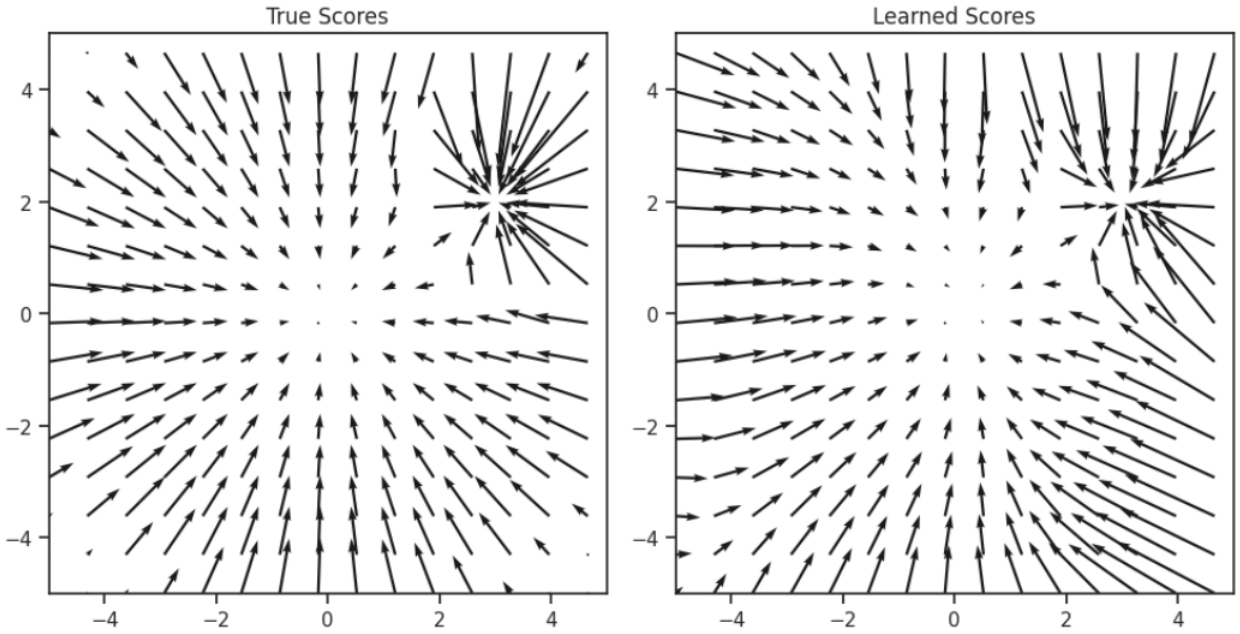
译注 上一句原文只说 “除以 σ^2”，完整得分目标还包含负号；式 (23.2) 和代码中的得分目标都保留负号。

23.2.3 比较学习得分与参考得分

网络训练完成后，定性评价其预测得分是否准确。上面已经得到真实得分向量场，下个单元将它与训练后网络预测的向量场并排绘制。

```
# Visualization function for learned scores
def visualize_learned_scores(mixture, score_net, grid_size=30, device='cpu'):
    """
    Visualize true and learned scores side by side
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))
    # Create grid for visualization
    x = np.linspace(-5, 5, grid_size)
    y = np.linspace(-5, 5, grid_size)
    X, Y = np.meshgrid(x, y)
    points = np.column_stack([X.ravel(), Y.ravel()])
    # Compute true scores
    true_scores = mixture.score(points)
    true_scores_x = true_scores[:, 0].reshape(grid_size, grid_size)
    true_scores_y = true_scores[:, 1].reshape(grid_size, grid_size)
    # Compute learned scores
    score_net.eval()
    with torch.no_grad():
        learned_scores = score_net(torch.FloatTensor(points).to(device)).cpu().numpy()
    learned_scores_x = learned_scores[:, 0].reshape(grid_size, grid_size)
    learned_scores_y = learned_scores[:, 1].reshape(grid_size, grid_size)
    # Plot true scores
    skip = 2
    ax1.quiver(X[::skip, ::skip], Y[::skip, ::skip],
               true_scores_x[::skip, ::skip], true_scores_y[::skip, ::skip],
               scale=50, width=0.005)
    ax1.set_title("True Scores")
    # Plot learned scores
    ax2.quiver(X[::skip, ::skip], Y[::skip, ::skip],
               learned_scores_x[::skip, ::skip], learned_scores_y[::skip, ::skip],
               scale=50, width=0.005)
    ax2.set_title("Learned Scores")
    # Set consistent limits and labels
    for ax in [ax1, ax2]:
        ax.set_xlim(-5, 5)
```

```
        ax.set_ylim(-5, 5)
        ax.set_aspect('equal')
        ax.set_xlabel("")
        ax.set_ylabel("")
    plt.tight_layout()
    plt.show()
# Visualize the results
visualize_learned_scores(mixture, score_net, device=device)
```



左：真实得分；右：学习得分。向量方向与长度按原图保留。

这说明了为何要通过扰动后的条件密度，作这一并不简单的构造：第二个优化表达式中的所有步骤都能高效计算。因此，只要知道适当的 σ，就有了学习得分函数的实用方法。

接下来，假设网络已经成功训练，如何用 $\nabla_{\widetilde{x}} \log p_\sigma(\widetilde{x})$ 构造 $p(x)$ 的生成模型？也就是说，如何由得分生成实际样本 x？

23.3 Langevin 动力学

为了得到实用方法，转向 Langevin 动力学。传统上，它用于具有确定性与随机力的分子系统。相关研究表明，沿得分迭代，并在每步加入恰当强度的额外扰动，实际上会收敛到来自 $p(x)$ 的样本。

具体而言，更新为

$$x_{i+1} \leftarrow x_i + \epsilon \nabla_x \log p(x) + \sqrt{2\epsilon}\, z_i, \tag{23.3}$$

其中 $i = 0, 1, \ldots, K$，$z_i \sim \mathcal{N}(0, I)$。在一组正则性条件下，当 $K \to \infty$ 且 $\epsilon \to 0$ 时，迭代量 x_K 收敛为 $p(x)$ 的样本。理论上还需考虑初始化 x_0 从哪个分布采样，该分布通常记为 $\pi(x)$。当然，更新 x_i 时，可以用训练后的网络 s_θ 在每步计算得分 $\nabla_x \log p(x)$。[4]

[4]译注：源式没有在梯度后显式写 x_i 处求值；代码在当前 x 计算网络。收敛陈述包含源文的正则性与极限条件，不等于固定有限步长/步数、

23.3.1 Langevin 动力学算法

用得分函数试一试。下面的 `langevin_dynamics` 针对不同步数，从 0 到 500，展示积分如何收敛。初始点位于稠密规则网格上，以展示区域中各处的点，是否以及如何沿得分梯度向底层分布的高密度区移动。

```
def langevin_dynamics(score_net, n_steps=1000, n_samples=35, step_size=0.01):
    device = next(score_net.parameters()).device
    x = torch.linspace(-5, 5, n_samples)
    y = torch.linspace(-5, 5, n_samples)
    X, Y = torch.meshgrid(x, y)
    x = torch.stack([X.flatten(), Y.flatten()], dim=1)
    x = x.to(device)
    trajectory = [x.cpu().detach().numpy()]
    score_net.eval()
    with torch.no_grad():
        for step in range(n_steps):
            score = score_net(x)
            noise = torch.randn_like(x)
            x = x + step_size * score + np.sqrt(2 * step_size) * noise
            if step in [0, 100, 200, 300, 400, 500]:
                trajectory.append(x.cpu().detach().numpy())
    return trajectory
trajectory = langevin_dynamics(
    score_net,
    n_steps=501, # run for 501 steps to include step 500
    n_samples=35,
    step_size=0.01,
)
samples = trajectory[-1]
```

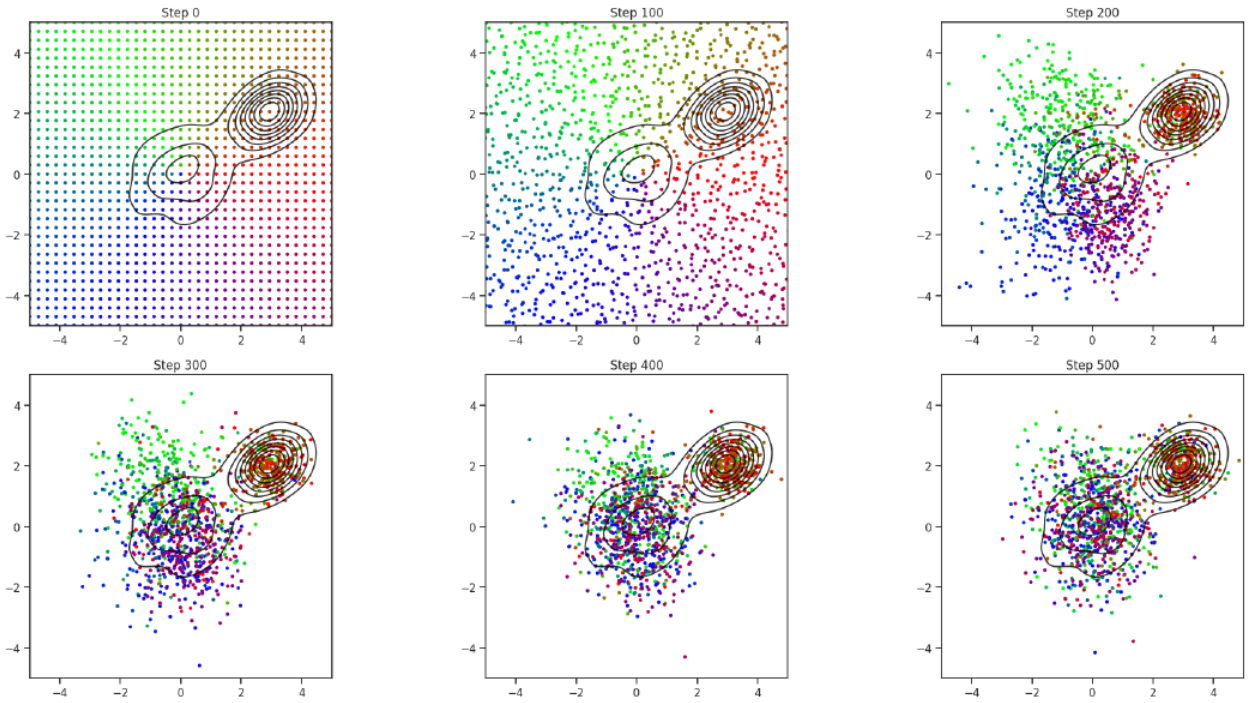
23.3.2 可视化轨迹

样本点已经保存在 `samples` 数组中，现在将它们作为彩色点，叠加在得分等高线上。初始网格用平滑 RGB 渐变着色，后续点的颜色即可显示初始位置最终到达目标密度地形的何处。

```
def get_angle_colors(positions):
    angles = np.arctan2(positions[:, 1], positions[:, 0])
    angles_deg = (np.degrees(angles) + 360) % 360
    colors = np.zeros((len(positions), 3))
    for i, angle in enumerate(angles_deg):
        segment = int(angle / 120)
        local_angle = angle - segment * 120
        if segment == 0: # 0 degrees to 120 degrees (R->G)
            colors[i] = [1 - local_angle/120, local_angle/120, 0]
        elif segment == 1: # 120 degrees to 240 degrees (G->B)
```

近似得分下的精确采样保证。

```
            colors[i] = [0, 1 - local_angle/120, local_angle/120]
        else: # 240 degrees to 360° (B->R)
            colors[i] = [local_angle/120, 0, 1 - local_angle/120]
    return colors
def visualize_langevin_trajectory(trajectory, mixture, figsize=(20, 10)):
    n_plots = len(trajectory)
    fig, axes = plt.subplots(2, 3, figsize=figsize)
    axes = axes.ravel()
    real_samples = mixture.sample(1000)
    from scipy.stats import gaussian_kde
    xx, yy = np.mgrid[-5:5:100j, -5:5:100j]
    positions = np.vstack([xx.ravel(), yy.ravel()])
    kernel = gaussian_kde(real_samples.T)
    density = np.reshape(kernel(positions).T, xx.shape)
    steps = [0, 100, 200, 300, 400, 500]
    for idx, (samples, step) in enumerate(zip(trajectory, steps)):
        ax = axes[idx]
        if idx == 0:
            c = get_angle_colors(samples)
        ax.contour(xx, yy, density, levels=10, alpha=0.8, colors='black')
        ax.scatter(samples[:, 0], samples[:, 1], alpha=1, s=7, color=c)
        ax.set_xlim(-5, 5)
        ax.set_ylim(-5, 5)
        ax.set_aspect('equal')
        ax.set_title(f'Step {step}')
    plt.tight_layout()
    plt.show()
visualize_langevin_trajectory(trajectory, mixture)
```



原图标为 Step 0、100、200、300、400、500；黑线为参考样本的核密度等高线。

看起来效果不错。规则排列的点正确地向高密度区移动；右侧偏红的点主要到达右侧高密度簇，其他蓝、绿、红混杂的点，则到达中央较低的密度峰。[5]

23.4 退火 Langevin 动力学

尽管上述方法可证明在极限下收敛，实践中仍有困难：需要相当大的噪声 σ，才能确保梯度真正将样本“引向”正确目标。若 σ 太小，就“没有得分”，即梯度不给出方向；若太大，彼此重叠的 Gaussian 又会降低数据样本质量，使重要细节淹没在噪声中。[6]

考虑从 $\pi(x)$ 得到的所有 x 所在空间，提高扰动数据集的噪声 σ，会覆盖扰动数据空间的更大区域，从而无论从何处出发，都能得到梯度。这在实践中很重要：不应为了找到良好初始点而作大量假设，否则只是将生成输出的问题，转移为为算法生成合适输入的问题。这也是许多经典工作必须仔细指定先验分布，以确保算法收敛的原因。这里需要微妙平衡：较大 σ 对实际应用很重要，但扰动若过大，就有 $p(x) \not\approx p_\theta(x)$，最坏时所生成样本毫无用处。[7]

这就引出问题：为何只能用单个 σ？从大噪声逐步减小，可以化解“没有得分”与“质量差”的矛盾。考虑多个噪声尺度 $0 < \sigma_1 < \sigma_2 < \cdots < \sigma_L$，得到所谓**退火 Langevin 动力学**。这也更接近扩散模型领域的一个核心主题：不同且变化的噪声水平。

从最大噪声 σ_L 到最小噪声 σ_1，对每个尺度反复应用 Langevin 动力学；训练网络 $s_\theta(x, \sigma_j)$，把噪

[5]译注：`trajectory` 含初始状态及更新 step=0、100、200、300、400、500 后的六个状态，共 7 个；绘图与 6 个标签 zip，只显示前 6 个，末态未显示。因而除 Initial 外，所画状态与标题步数错位；保留源代码/源图而不修复。

[6]译注：“没有得分”是源文的实践性概括，不是正密度 Gaussian 混合在小正 σ 下数学上不存在得分；低密度区的学习覆盖及数值实现同样影响结果。

[7]译注：此处比较量按源写 p_θ，前文扰动密度记为 p_σ；两种记号不作静默统一。

声尺度 σ_j 作为附加输入。这样，迭代更新 x_i 时，也逐步减小 σ_j，以确保收敛到准确样本 $x \sim p_\theta(\widetilde{x})$。[8]

23.4.1 含噪声水平的网络与数据集定义

来实现这个想法。先修改得分网络，使其感知噪声水平，为此增加输入参数 σ。相应地，扩展数据集以覆盖不同噪声强度。

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
class ScoreNetworkWithSigma(nn.Module):
    def __init__(self, input_dim=2, hidden_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim + 1, hidden_dim), # +1 for sigma
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, input_dim)
        )
    def forward(self, x, sigma):
        sigma = sigma.view(-1, 1)
        x_sigma = torch.cat([x, sigma], dim=1)
        return self.net(x_sigma)
class AnnealedDenoisingScoreMatchingDataset(Dataset):
    def __init__(self, gaussian_mixture, num_samples=10000, sigmas=[4.0, 2.0, 1.0, 0.5,
    ↪  0.2, 0.01]):
        self.samples = torch.FloatTensor(gaussian_mixture.sample(num_samples))
        self.sigmas = sigmas
    def __len__(self):
        return len(self.samples) * len(self.sigmas)
    def __getitem__(self, idx):
        sample_idx = idx // len(self.sigmas)
        sigma_idx = idx % len(self.sigmas)
        x = self.samples[sample_idx]
        sigma = self.sigmas[sigma_idx]
        # Add noise to the sample
        noise = torch.randn_like(x) * sigma
        x_noisy = x + noise
        return x_noisy, noise, x, torch.tensor(sigma).float()
```

[8]译注：末尾样本变量 x 与密度自变量 $\widetilde{x}$ 的差异按源保留；有限噪声表及近似网络并不自动提供精确目标采样保证。

23.4.2 训练

有了这两个组成部分，训练与此前一样进行。

```
def train_annealed_score_network(model, train_loader, num_epochs=100, device='cpu'):
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
    model.train()
    losses = []
    for epoch in range(num_epochs):
        epoch_loss = 0
        for batch_idx, (x_noisy, noise, x_clean, sigma) in enumerate(train_loader):
            x_noisy, noise = x_noisy.to(device), noise.to(device)
            sigma = sigma.to(device)
            optimizer.zero_grad()
            pred_score = model(x_noisy, sigma)
            target_score = -noise / (sigma.view(-1, 1) ** 2)
            loss = torch.mean(torch.sum((pred_score - target_score) ** 2, dim=1))
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
        avg_loss = epoch_loss / len(train_loader)
        losses.append(avg_loss)
        if (epoch + 1) % 10 == 0:
            print(f'Epoch {epoch + 1}/{num_epochs}, Loss: {avg_loss:.4f}')
    return losses
sigmas = [4.0, 2.0, 1.0, 0.5, 0.2, 0.01]
dataset = AnnealedDenoisingScoreMatchingDataset(mixture, num_samples=10000,
↪  sigmas=sigmas)
dataloader = DataLoader(dataset, batch_size=128, shuffle=True)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
score_net = ScoreNetworkWithSigma().to(device)
losses = train_annealed_score_network(score_net, dataloader, num_epochs=100,
↪  device=device)
```

原书输出：

```
Epoch 10/100, Loss: 3306.6889
Epoch 20/100, Loss: 3374.6170
Epoch 30/100, Loss: 3296.4916
Epoch 40/100, Loss: 3328.9254
Epoch 50/100, Loss: 3302.5448
Epoch 60/100, Loss: 3362.3191
Epoch 70/100, Loss: 3349.0398
Epoch 80/100, Loss: 3337.2041
Epoch 90/100, Loss: 3354.9199
Epoch 100/100, Loss: 3328.9864
```

为在变化噪声下进行 Langevin 采样，修改采样过程，逐步降低 σ，并用感知噪声的得分网络提供相应梯度。

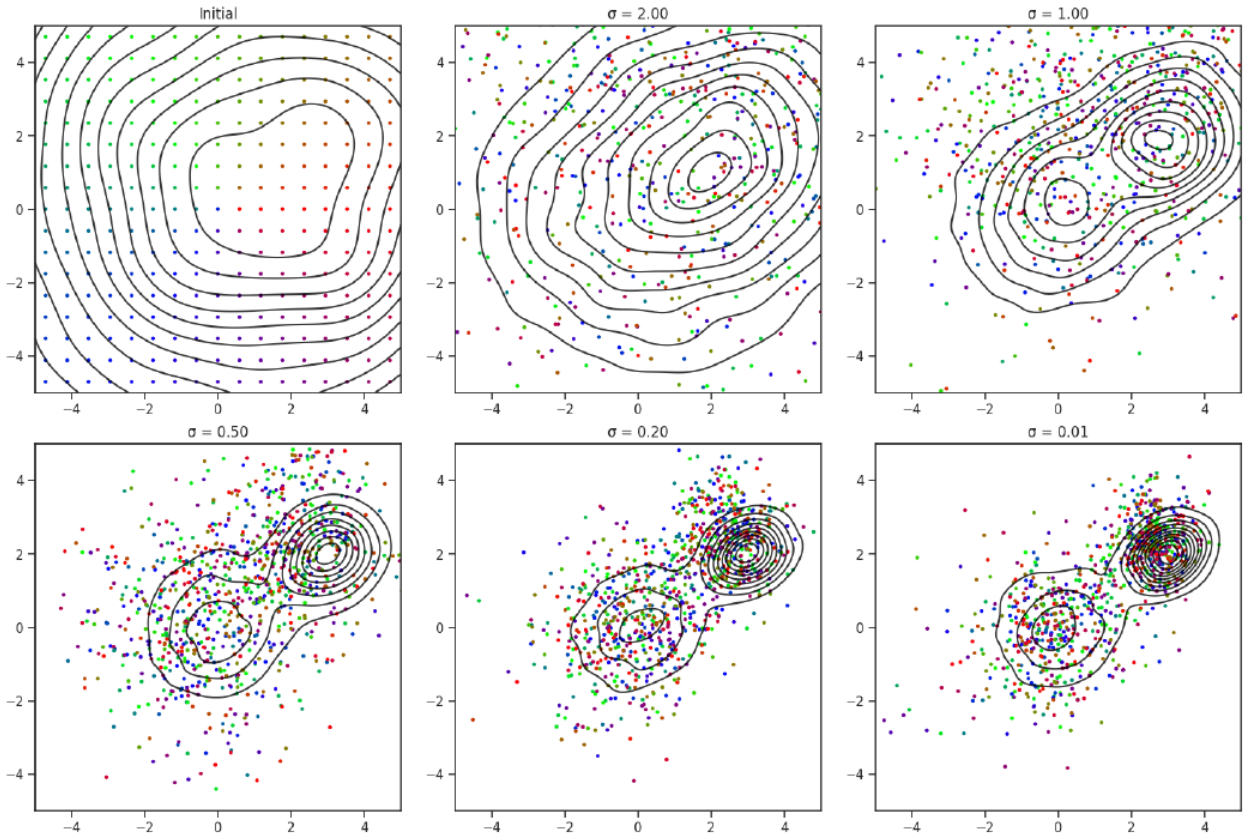
```
def annealed_langevin_dynamics(score_net, sigmas,
                               n_steps_each=100, n_samples=1000,
                               step_size=0.001):
    device = next(score_net.parameters()).device
    x = torch.linspace(-10, 10, n_samples)
    x = torch.stack(torch.meshgrid(x, x), dim=-1).reshape(-1, 2)
    x = x.to(device)
    trajectory = [x.cpu().detach().numpy()]
    score_net.eval()
    with torch.no_grad():
        for sigma in sigmas:
            alpha = step_size * (sigma / sigmas[-1]) ** 2
            for step in range(n_steps_each):
                sigma_tensor = torch.full((n_samples*n_samples,), sigma, device=device)
                score = score_net(x, sigma_tensor)
                noise = torch.randn_like(x)
                x = x + alpha * score + np.sqrt(2 * alpha) * noise
            trajectory.append(x.cpu().detach().numpy())
    return trajectory
trajectory = annealed_langevin_dynamics(
        score_net,
        sigmas=sigmas,
        n_steps_each=150,
        n_samples=35,
        step_size=0.00001
        )
```

23.4.3 可视化轨迹

终于可以画出轨迹。不仅样本位置在移动，底层得分函数的等高线也随扰动 σ 相应变化。

```
def visualize_annealed_langevin_trajectory(trajectory, mixture, sigmas, figsize=(15,
↪  10)):
    fig, axes = plt.subplots(2, 3, figsize=figsize)
    axes = axes.ravel()
    from scipy.stats import gaussian_kde
    xx, yy = np.mgrid[-5:5:100j, -5:5:100j]
    positions = np.vstack([xx.ravel(), yy.ravel()])
    for idx, (samples, sigma) in enumerate(zip(trajectory, sigmas)):
        ax = axes[idx]
        if idx == 0:
            c = get_angle_colors(samples)
        real_samples = torch.tensor(mixture.sample(5000))
        real_samples += torch.randn_like(real_samples) * sigma
        kernel = gaussian_kde(real_samples.T)
        density = np.reshape(kernel(positions).T, xx.shape)
        ax.contour(xx, yy, density, levels=10, alpha=0.8, colors='black')
```

```
        ax.scatter(samples[:, 0], samples[:, 1], alpha=1.0, s=5, color=c)
        ax.set_xlim(-5, 5)
        ax.set_ylim(-5, 5)
        ax.set_aspect('equal')
        ax.set_title(f'σ = {sigma:.2f}' if idx > 0 else 'Initial')
    plt.tight_layout()
    plt.show()
visualize_annealed_langevin_trajectory(trajectory, mixture, sigmas)
```



原图依次显示 Initial、$\sigma = 2.00, 1.00, 0.50, 0.20, 0.01$，等高线按各次扰动样本估计。

这些图清楚显示：从变化很平缓的得分函数出发，等高线间距很大，逐步走向目标分布中更尖锐的峰。退火成功消除了手工指定扰动的需要。[9]

23.5 得分小结

现在已接近目标。在迈出最后一步、推导著名的“去噪”任务之前，先概括迄今的进展。

- 去噪得分匹配对图像等高维数据也很有效。
- 无需将梯度反向传播过许多步骤，因此比 CNF 或神经 ODE 更易扩展。

[9]译注：末句按源保留，但代码仍需手工指定完整 `sigmas` 序列。轨迹也是初始状态加 6 个尺度末态，而绘图 zip 只取 6 个，因此最小噪声末态被丢弃，其余标签与实际完成尺度错位一格。等高线代码是对扰动参考样本的核密度估计，未直接绘制得分的标量势。

- 已有办法从 $p(x)$ 生成样本，无需以非平凡先验分布的形式作出假设。

尽管有这些重要进展，仍留下几个难点：

- 选定良好的噪声尺度序列至关重要，但目前还不清楚如何获得。
- 可以从 $p(x)$ 采样，但不能直接计算似然。
- 不再适用 “方便的” 最大似然训练。

还有实际性能问题：推断需要多次计算 $s_\theta(x, \sigma_j)$，生成样本可能因此昂贵。先确保获得准确样本，稍后还会回到计算效率问题。[Vin11] 完整书目在全书文献单元接入。

译注：源实现与结论的适用范围 仅 $K \to \infty, \epsilon \to 0$ 不足以保证 Langevin 采样收敛，还需有效总积分时间和混合条件。例如 $\epsilon = K^{-2}$ 时 $K\epsilon \to 0$，累计噪声方差 $2K\epsilon \to 0$，迭代仍停在初分布附近。固定 $\sigma = .1$ 的 DSM 理想目标为 $\nabla \log p_\sigma$，并非未扰动 p；左参考图却用原混合得分（另含 10^{-8} 正则）。一般向量网络输出也未必是某个可归一化密度的保守得分场，有限步、近似场不自动精确采样目标。

第二十四章　去噪

从退火 Langevin 动力学的视角，考虑一个极端情形：从纯噪声出发，并将退火步骤视为连续时间维度。定义纯噪声起始时刻为 $t = T$，然后按照连续加噪日程减少噪声，在 $t = 0$ 时降至零。若只考虑 Gaussian 噪声 $\mathcal{N}$，多个 Gaussian 噪声叠加后仍为 Gaussian。因此，这实际上给出一条 Markov 链，其正向链以高度可控的方式加噪。将正向过程记为 q，标准差函数记为 β，得到

$$q(x_{1:T} \mid x_0) := \prod_{t=1}^{T} q(x_t \mid x_{t-1}),$$
$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\Big).$$

前面考虑的过程，即 “Langevin 动力学”，对应反向过程。将它直接记为 p_θ，因为这就是之后要学习的对象。沿扩散时间 t 的反向过程为

$$p_\theta(x_{0:T}) := p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t),$$
$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \beta_t I).$$

乍看很简单，其实所有困难都藏在学习到的分布均值 μ_θ 中，它是 p_θ 的关键构件。

由于采用标准差为 β_t 的 Gaussian，每一步都有解析形式：给定数据点 x_0，可从正向 Markov 链抽取含噪潜状态 x_t：

$$q(x_t \mid x_0) = \mathcal{N}(x_t, \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I)).$$

其中反向权重 $\alpha_t = 1 - \beta_t$，截至时刻 t 累积的 alpha 记为 $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$。[1]

24.1　潜变量模型

概念上，这一表述就是机器学习中所谓的潜变量模型。不再像前面的退火 Langevin 动力学那样采用略显任意的中间状态，而是沿扩散时间 t 显式建模潜状态。目标 $x_0 \sim q(x_0)$ 的分布具有形式

$$p_\theta(x_0) = \int p_\theta(x_{0:T})\, \mathrm{d}x_{1:T},$$

[1] 译注：源文两次称 β_t 为标准差，但在 $\mathcal{N}(\mu, \beta_t I)$ 中它是协方差系数。末式按源保留额外的首参数 x_t 和末尾右括号；若采用通常 Gaussian 分布的 “均值、协方差” 记法，这种写法不一致。

其中 $x_1, \dots, x_T$ 是与 x_0 维数相同的潜变量。

此前直接用扰动数据的梯度训练模型，得到得分。在不断变化的潜变量背景下，如何训练？扩散过程的精确边缘似然包含难以处理的积分，但逐步累积 Gaussian 噪声给出可用的变分下界。对对数边缘似然应用 Jensen 不等式，得到证据下界（ELBO）：

$$\log p_\theta(x_0) \geq \mathbb{E}_{q(x_1,\dots,x_T|x_0)}\left[\log \frac{p_\theta(x_0, x_1, \dots, x_T)}{q(x_1, \dots, x_T \mid x_0)}\right].$$

展开并简化，得到最大似然目标的下界：

$$\mathbb{E}[-\log p_\theta(x_0)] \leq \mathbb{E}\left[-\log p(x_T) - \sum_{t\geq 1} \log \frac{p_\theta(x_{t-1} \mid x_t)}{q(x_t \mid x_{t-1})}\right].$$

注意，这里还使用了前面 Bayes 条件化技巧。该训练目标可用 KL 散度改写为三项：

$$\begin{aligned}
&\mathbb{E}[L_T + L_{t-1} + L_1] \\
&\quad = \mathbb{E}\Bigg[\mathrm{KL}(q(x_T \mid x_0) \| p(x_T)) \\
&\qquad + \sum_{t>1} \mathrm{KL}\big(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t)\big) - \log p_\theta(x_0 \mid x_1)\Bigg].
\end{aligned}$$

译注 原文称前一不等式为“下界”，但其展示的是负对数似然的上界；对数似然的下界则是此前 ELBO 式。原 PDF 在不等式段还有字面的 `$` 排版残留，此处保留数学内容。三项记号中的左侧 L_{t-1} 未显式求和，右侧有求和，亦按源保留。

看似复杂，但三项各有清晰含义：

- 第一项 L_T 不依赖 θ；
- 最后一项 L_1 容易用于训练；
- 中间项 L_{t-1} 只是两个已知 Gaussian 分布之间的 KL 散度。

因此，关键是中间项 L_{t-1}，而上述 α 与 β 已使 Gaussian 受控。正向过程对应的表达式稍长，但可以写出。条件 Gaussian 为

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\widetilde{\mu}_t(x_t, x_0), \widetilde{\beta}_t I),$$

均值与标准差为

$$\begin{aligned}
\widetilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t, \\
\widetilde{\beta}_t &= \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t.
\end{aligned}$$

这已经给出可行的训练目标，并被许多论文使用；但不必止步于此。进一步观察，是向均值添加噪声 ϵ，即 $x + \epsilon$。预测叠加噪声 ϵ 比预测均值，即信号 x 本身，更容易；减去噪声，同样能得到 x。用上述术语写为

$$x_t(x_0, \epsilon) = \sqrt{\bar{a}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{24.1}$$

不直接预测均值 $\mu_\theta(x_t,t)$，改为预测噪声 $\epsilon_\theta(x_t,t)$，再由它计算均值：

$$\mu_\theta(x_t,t)=\frac{1}{\sqrt{\alpha_t}}\left(x_t-\frac{1-\alpha_t}{\sqrt{1-\bar\alpha_t}}\epsilon_\theta(x_t,t)\right).$$

于是，上述 L_{t-1} 简化为

$$\mathbb{E}_{x_0,\epsilon}\left[\frac{\beta_t^2}{2\beta_t\alpha_t(1-\bar\alpha_t)}\left\|\epsilon-\epsilon_\theta\big(\sqrt{\bar\alpha_t}\,x_0+\sqrt{1-\bar\alpha_t}\,\epsilon,t\big)\right\|^2\right]+C.$$

更好的是，可以去掉开头的 β^2 因子。人们发现，该因子的缩放并不理想，去掉后会增加反向链末端、β 较小样本的权重。得到简化去噪损失

$$L_{\text{DM}}(\theta)=\mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon-\epsilon_\theta(x_t(x_0,\epsilon),t)\|^2\right].$$

按式 (24.1)，将 $\sqrt{\bar\alpha_t}\,x_0$ 与适量噪声 $\sqrt{1-\bar\alpha_t}\,\epsilon$ 混合，容易得到正确的 x_t。

译注 源文又把 $\widetilde{\beta}_t$ 称作标准差，但其在协方差位置。式 (24.1) 及紧随的解释采用 Latin $\bar a_t$，此前定义的是 $\bar\alpha_t$，这里不默改。带权损失分母 $2\beta_t\alpha_t(1-\bar\alpha_t)$ 按可见源公式保留；从它到无权损失移除了整个时间权重，不仅是分子 β_t^2，因此属于目标重加权，不能当作逐项相等的代数化简。

24.2 完整去噪算法

这个极其简单的 L^2 损失，正是该方法获得突破性成功的原因：容易计算且非常稳定。这种训练变体通常称为**去噪扩散概率模型**（DDPM）。例如 GAN 的训练需要脆弱地平衡两个网络，而 DDPM 只需考虑完全监督的损失。当然，代价是计算资源增加。在讨论计算挑战之前——原文称目前这些挑战也大多得到解决——先回顾去噪目标对应的训练与推断算法。

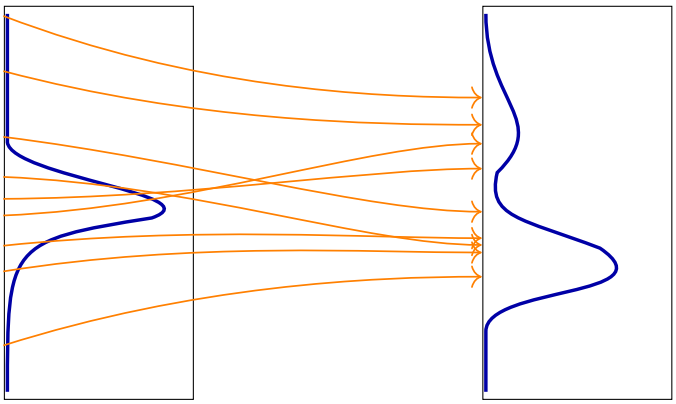


图 24.1: DDPM 过程，与 CNF 对照：通过大量去噪步骤，将简单基分布（左，x_T）去噪，最终得到目标分布（右，x_0）的样本；橙色轨迹标注 $\epsilon_\theta(x_t,t)$。原图 24.1。

训练过程可用以下伪代码概括。

算法 1（DDPM 训练） **输入：**目标分布 q、噪声日程 $\bar{\alpha}$。 **输出：**训练后的噪声估计 NN ϵ_θ。

1. 重复直到收敛：
   2. $x_0 \sim q(x_0)$；
   3. $t \sim \mathrm{Uniform}(1, \dots, T)$；
   4. $\epsilon \sim \mathcal{N}(0, I)$；
   5. $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$；
   6. optimizer_step$(\nabla_\theta \|\epsilon - \epsilon_\theta(x_t, t)\|^2)$。
2. 返回 ϵ_θ。

均匀抽取整数时间 t，计算含噪样本 x_t。这很简单：按权重 $\bar{\alpha}_t$，将正向过程第 0 步的样本 x_0 与均匀噪声混合。让网络根据 x_t 与时间 t 估计噪声，再用选定的优化器，通常是 Adam，沿 L^2 损失梯度前进一步。给定步数 T 后，可以预先计算噪声日程 $\bar{\alpha}_t$ 和相应系数，训练时只需查表。正如前面解释，省去噪声日程的权重，使损失尤其简单。[2]

推断时必须正确计算权重，过程稍复杂。权重也可预先计算，但昂贵之处在于，必须经过全部 T 步，才能完全去噪。

算法 2（DDPM 推断） **输入：**噪声估计器 ϵ_θ，噪声日程 $\alpha, \bar{\alpha}, \sigma$。 **输出：**分布 p 的样本。

1. $x \sim \mathcal{N}(0, I)$。
2. 对 $t = T, \dots, 1$：
   3. 若 $t > 1$，$z \sim \mathcal{N}(0, I)$；
   4. 否则，$z = 0$；
   5. $x = \dfrac{1}{\sqrt{\alpha_t}} \left(x - \dfrac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x, t) \right) + \sigma_t z$。
3. 返回 x。

也就是说，必须顺序计算训练后的网络 ϵ_θ 共 T 次。原始 DDPM 论文实际上采用 $T = 1000$，以获得极高质量样本。实际中可将 T 显著减小约一个数量级，仍得到很好结果，但这仍意味着约 100 次网络计算。

显然，这比前几章一次前向传播输出均值的确定性网络昂贵。不过，强大而根本的变化是：DDPM 网络能够学习并可靠地再现分布。例如，若同一个流动输入的数据中，20% 的解向左旋转，80% 向右，训练后 DDPM 就会以正确概率再现这些解。使用不同初始噪声 x 反复推断，就会以 0.2 的概率观察到左旋解，以 0.8 的概率观察到右旋解。网络本身当然仍是确定性的；原文还指出，只要推断算法第 1 行的初始 x 固定，总会生成相同输出。[3]

另一项强大能力是，可通过条件可靠地引导网络生成样本，后面还会讨论。这对最初的 SBI 目标十分重要：学习后验 $p(x \mid y)$，为给定观测 y 产生正确输出。展望后文，仍需处理三个重要扩展：

[2]译注：本段 “均匀噪声” 按源写法，算法实际抽取 Gaussian 噪声；均匀抽样的是时间 t。

[3]译注：概率比例是原书理想化说明，实际训练不自动保证精确 20/80。最后一句与算法中每步重新抽取 z 冲突：网络本身确定，不意味着含随机反向步的整个采样器在仅固定初始 x 时确定；还需固定全部随机噪声或采用确定性采样方案。

- 提高推断速度，主要是减少网络调用。
- 以外部参数和观测为条件。
- 在物理仿真中，通过基于 PDE 的约束重新引入先验知识。

尽管如此，DDPM 已经完成最重要的第一步：将确定性网络转成概率推断工具，稳健地学习并再现复杂分布。在开始代码示例前，先看看计算开销：DDPM 需要大量去噪步骤，模型运行代价可能比相应确定性模型高几个数量级。核心问题是，如何在不损失后验精度的情况下，减少所需步数?

24.3 用 DDPM 训练

看看这些想法在 Gaussian 混合（GM）例子中的效果。为构成独立 notebook，先初始化归一化流和得分测试中同样的双峰 GM。[4]

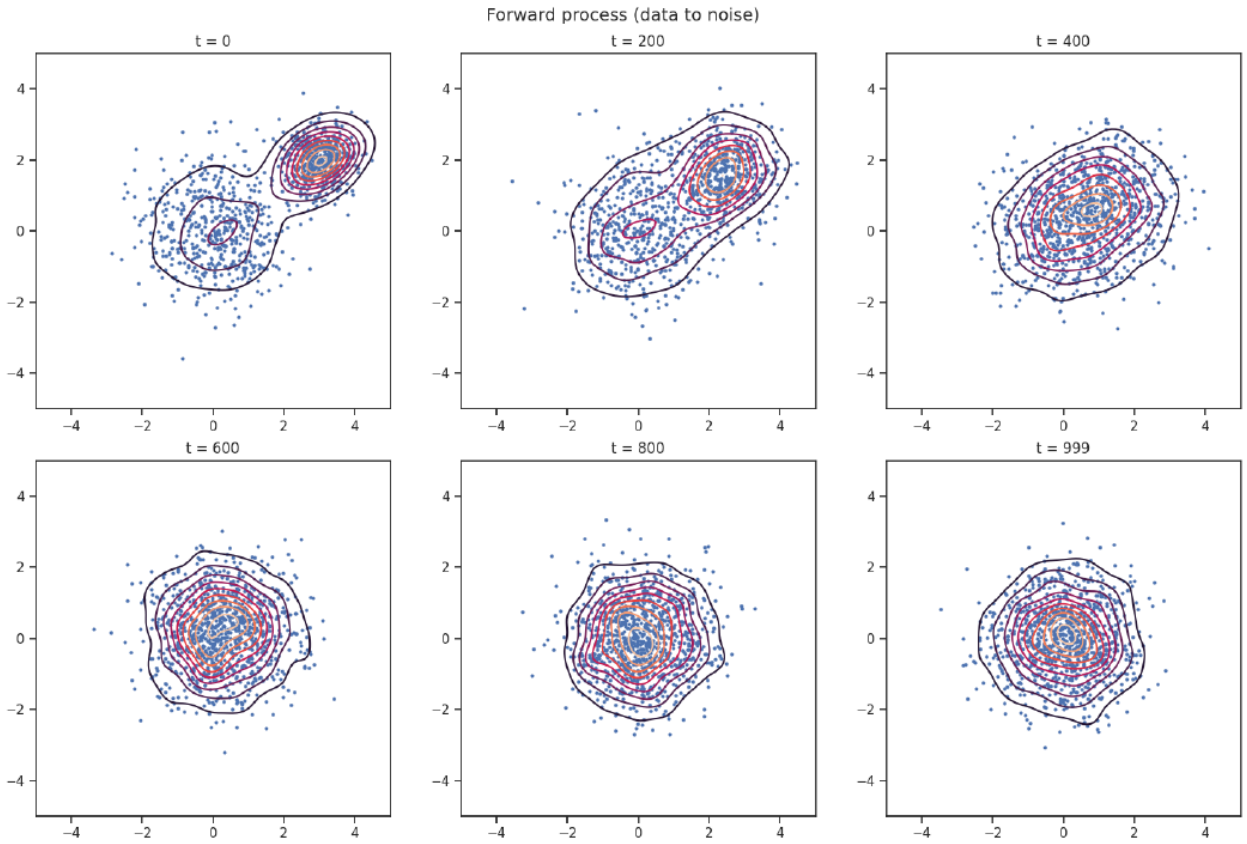
```
import numpy as np
class GaussianMixture:
    def __init__(self, parameters):
        self.parameters = parameters
        self.distributions = [
            {
                'mean': np.array(dist['mean']),
                'std': np.array(dist['std']),
                'cov': np.diag(np.array(dist['std']) ** 2)
            }
            for dist in parameters
        ]
    def sample(self, num_samples):
        samples = []
        num_distributions = len(self.distributions)
        for _ in range(num_samples):
            idx = np.random.randint(num_distributions) # Choose a random Gaussian
            dist = self.distributions[idx]
            sample = np.random.multivariate_normal(mean=dist['mean'], cov=dist['cov'])
            samples.append(sample)
        return np.array(samples)
parameters = [
    {"mean": [0, 0], "std": [1, 1]},
    {"mean": [3, 2], "std": [0.5, 0.5]}
]
mixture = GaussianMixture(parameters)
```

24.3.1 定义正向过程

接着实现 DDPM 正向过程：对 GM 分布逐步加大噪声，直到样本中残留的信息痕迹都被移除。下面展示这一破坏过程。

[4]译注：本章代码仅转写及 token/AST 核查，不训练，不生成数据或运行采样；输出与图均按原书保存。

```
import torch
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="ticks")
class DDPMForwardProcess:
    def __init__(self, num_timesteps=1000):
        self.num_timesteps = num_timesteps
        # Standard beta schedule from DDPM paper
        scale = 1000 / num_timesteps
        beta_start = scale * 0.0001
        beta_end = scale * 0.02
        self.betas = torch.linspace(beta_start, beta_end, num_timesteps)
        self.alphas = 1 - self.betas
        self.alphas_cumprod = torch.cumprod(self.alphas, dim=0)
        self.sqrt_betas = torch.sqrt(self.betas)
        self.sqrt_alphas_cumprod = torch.sqrt(self.alphas_cumprod)
        self.sqrt_one_minus_alphas_cumprod = torch.sqrt(1 - self.alphas_cumprod)
    def forward_process(self, x_0, t):
        epsilon = torch.randn_like(x_0)
        mean = self.sqrt_alphas_cumprod[t] * x_0
        std = self.sqrt_one_minus_alphas_cumprod[t]
        return mean + std * epsilon, epsilon
    def visualize_forward_process(self, mixture, num_samples=1000):
        timesteps_to_show=[0, 200, 400, 600, 800, 999]
        x_0 = torch.FloatTensor(mixture.sample(num_samples))
        fig, axes = plt.subplots(2, 3, figsize=(15, 10))
        axes = axes.ravel()
        xx, yy = np.mgrid[-5:5:100j, -5:5:100j]
        positions = np.vstack([xx.ravel(), yy.ravel()])
        for idx, t in enumerate(timesteps_to_show):
            x_t, _ = self.forward_process(x_0, t)
            samples = x_t.numpy()
            from scipy.stats import gaussian_kde
            kernel = gaussian_kde(samples.T)
            density = np.reshape(kernel(positions).T, xx.shape)
            ax = axes[idx]
            ax.contour(xx, yy, density, levels=10, alpha=1.0)
            ax.scatter(samples[:, 0], samples[:, 1], alpha=1.0, s=4)
            ax.set_xlim(-5, 5)
            ax.set_ylim(-5, 5)
            ax.set_aspect('equal')
            ax.set_title(f't = {t}')
        plt.suptitle("Forward process (data to noise)")
        plt.tight_layout()
        plt.show()
ddpm = DDPMForwardProcess(num_timesteps=1000)
ddpm.visualize_forward_process(mixture)
```



正向过程：数据到噪声。原图时刻为 $t = 0, 200, 400, 600, 800, 999$。

可以看到，$t = 0$ 时的两个峰，在加入约 200 步噪声后开始消失，视觉上形成以零为中心的单个 Gaussian 峰。

24.3.2 数据集与数据加载器

下个单元定义用于训练 DDPM 的小数据集类。它预先计算每个 t 对应的 alpha 与 beta，以便训练时快速取得具有不同噪声强度的样本。

```
from torch import nn
from torch.optim import Adam
from torch.utils.data import Dataset, DataLoader
import numpy as np
class ForwardDiffusionDataset(Dataset):
    def __init__(self, num_samples, input_dim,
                 T, mixture, beta_value=0.02):
        super().__init__()
        self.num_samples = num_samples
        self.input_dim = input_dim
        self.T = T
        self.beta_value = beta_value
        self.gm = mixture
        self.betas = beta_value * torch.ones(T)
```

```
        self.alphas = 1 - self.betas
        self.alpha_bar = torch.cumprod(self.alphas, dim=0)
        self.x0 = torch.FloatTensor(self.gm.sample(num_samples))
    def __len__(self):
        return self.num_samples
    def __getitem__(self, idx):
        x0_sample = self.x0[idx]
        t = torch.randint(low=1, high=self.T + 1, size=(1,)).item()
        eps = torch.randn(self.input_dim)
        sqrt_alpha_bar = torch.sqrt(self.alpha_bar[t - 1])
        sqrt_one_minus_alpha_bar = torch.sqrt(1 - self.alpha_bar[t - 1])
        x_t = sqrt_alpha_bar * x0_sample + sqrt_one_minus_alpha_bar * eps
        return x_t, torch.tensor(t, dtype=torch.float32), eps
```

接着定义与此前得分网络很相似的简单网络。原文称，三个隐藏层就足以学习去噪任务。主要区别是：网络输入去噪时间 t，而非此前的扰动大小。

```
class DdpmNet(nn.Module):
    def __init__(self, in_dim=2, time_dim=1, hidden_dim=128, out_dim=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim + time_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim)
        )
    def forward(self, x, t):
        t = t.unsqueeze(1)
        xt = torch.cat([x, t], dim=1)
        return self.net(xt)
```

译注 实际网络包含三个 Linear 层，其中两个为隐藏层、一个为输出层。前面的 `DDPMForwardProcess` 采用线性 beta 日程，训练数据集则采用恒定 `beta_value=0.02`，后面推断也用常量；两种日程按源分别保留。正向演示数组的索引 0 已经含第一步噪声，并非严格未扰动的 x_0。

24.3.3 初始化与训练

```
num_samples = 10000
input_dim = 2
T = 1000
dataset = ForwardDiffusionDataset(num_samples=num_samples, input_dim=input_dim, T=T,
↪  mixture=mixture, beta_value=0.02)
batch_size = 64
data_loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
# %% Training the DDPM model
time_dim = 1
```

```
hidden_dim = 128
model = DdpmNet(in_dim=input_dim, time_dim=time_dim, hidden_dim=hidden_dim, out_dim=input_dim)
optimizer = Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()
epochs = 100
model.train()
for epoch in range(1, epochs + 1):
    epoch_loss = 0
    for x_t_batch, t_batch, eps_batch in data_loader:
        optimizer.zero_grad()
        # Normalize time t to range [0,1]
        t_norm = t_batch / T
        eps_pred = model(x_t_batch, t_norm)
        loss = loss_fn(eps_pred, eps_batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    avg_loss = epoch_loss / len(data_loader)
    print(f"Epoch {epoch}/{epochs}, Loss: {avg_loss:.4f}")
```

原书输出如下，省略号为原有：

```
Epoch 1/100, Loss: 0.1529
Epoch 2/100, Loss: 0.0871
Epoch 3/100, Loss: 0.0863
Epoch 4/100, Loss: 0.0747
Epoch 5/100, Loss: 0.0759
Epoch 6/100, Loss: 0.0737
Epoch 7/100, Loss: 0.0715
Epoch 8/100, Loss: 0.0742
Epoch 9/100, Loss: 0.0706
Epoch 10/100, Loss: 0.0704
...
Epoch 90/100, Loss: 0.0631
Epoch 91/100, Loss: 0.0592
Epoch 92/100, Loss: 0.0597
Epoch 93/100, Loss: 0.0595
Epoch 94/100, Loss: 0.0570
Epoch 95/100, Loss: 0.0632
Epoch 96/100, Loss: 0.0609
Epoch 97/100, Loss: 0.0570
Epoch 98/100, Loss: 0.0580
Epoch 99/100, Loss: 0.0606
Epoch 100/100, Loss: 0.0624
```

24.3.4 利用反向过程推断

训练网络时，单个去噪步骤就已足够。推断时则需完成全部去噪过程：从初始纯噪声样本出发，用 1000 次迭代更新并去噪，最后得到目标分布样本。下方代码实现这一过程，并记录不同快照供可视化。

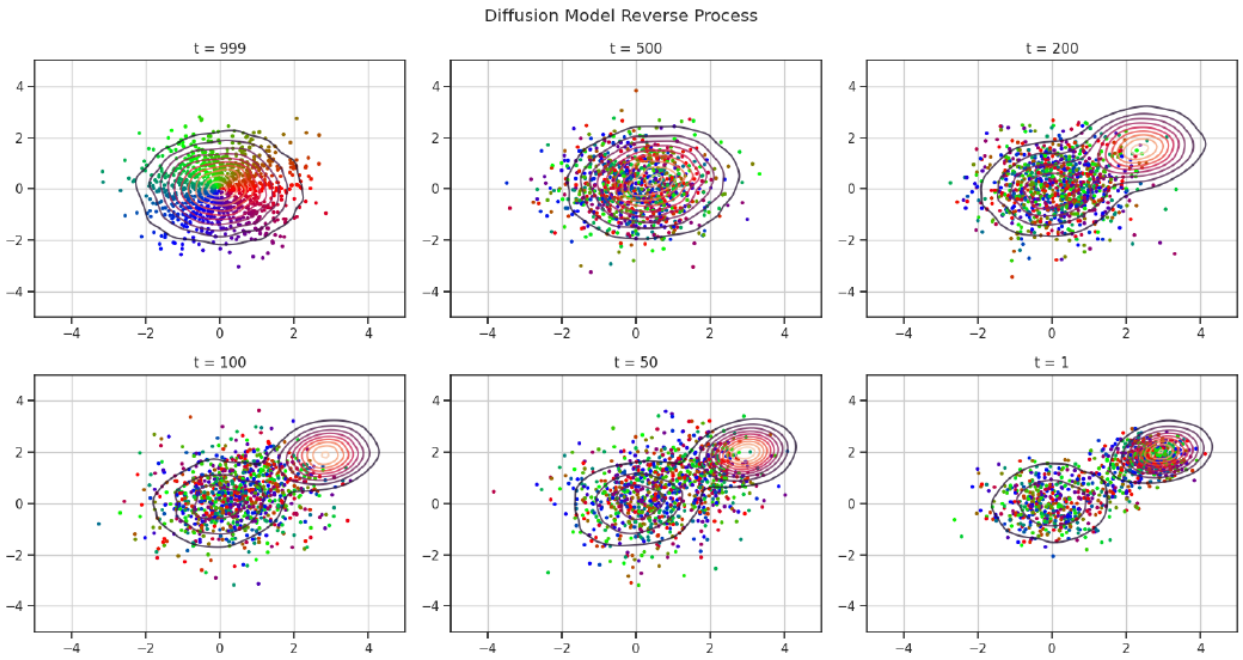
```
n_infer = 1000
betas = 0.02 * torch.ones(T)
alphas = 1 - betas
alpha_bar = torch.cumprod(alphas, dim=0)
with torch.no_grad():
    x = torch.randn(n_infer, input_dim)
    record_steps = {999, 500, 200, 100, 50, 1}
    snapshots = {}
    for t in range(T, 0, -1):
        t_tensor = (torch.ones(n_infer) * t / T).float()
        eps_theta = model(x, t_tensor)
        beta_t = betas[t - 1]
        alpha_t = alphas[t - 1]
        alpha_bar_t = alpha_bar[t - 1]
        coef = beta_t / torch.sqrt(1 - alpha_bar_t)
        x = (1 / torch.sqrt(alpha_t)) * (x - coef * eps_theta)
        if t > 1:
            sigma_t = torch.sqrt(beta_t)
            x = x + sigma_t * torch.randn_like(x)
        if t in record_steps:
            snapshots[t] = x.clone().cpu().numpy()
```

24.3.5 可视化采样轨迹

接着，观察初始分布如何缓慢变成 GM 例子的目标分布。

```
import matplotlib.pyplot as plt
def get_angle_colors(positions):
    angles = np.arctan2(positions[:, 1], positions[:, 0])
    angles_deg = (np.degrees(angles) + 360) % 360
    colors = np.zeros((len(positions), 3))
    for i, angle in enumerate(angles_deg):
        segment = int(angle / 120)
        local_angle = angle - segment * 120
        if segment == 0: # 0 degrees to 120 degrees (R->G)
            colors[i] = [1 - local_angle/120, local_angle/120, 0]
        elif segment == 1: # 120 degrees to 240 degrees (G->B)
            colors[i] = [0, 1 - local_angle/120, local_angle/120]
        else: # 240 degrees to 360° (B->R)
            colors[i] = [local_angle/120, 0, 1 - local_angle/120]
    return colors
def visualize_snapshots(snapshots):
    sorted_steps = sorted(snapshots.keys(), reverse=True)
```

```
    n_plots = len(sorted_steps)
    fig, axes = plt.subplots(2, 3, figsize=(5 * 3, 8))
    axes = axes.ravel()
    if n_plots == 1:
        axes = [axes]
    # Create grid for density visualization
    xx, yy = np.mgrid[-5:5:100j, -5:5:100j]
    positions = np.vstack([xx.ravel(), yy.ravel()])
    idx = 0
    for ax, t in zip(axes, sorted_steps):
        samples = snapshots[t]
        if idx == 0:
            c = get_angle_colors(samples)
            idx += 1
        x_0 = torch.FloatTensor(mixture.sample(num_samples))
        x_t, _ = ddpm.forward_process(x_0, t)
        samples_ = x_t.numpy()
        from scipy.stats import gaussian_kde
        kernel = gaussian_kde(samples_.T)
        density = np.reshape(kernel(positions).T, xx.shape)
        ax.contour(xx, yy, density, levels=10, alpha=0.8)
        ax.scatter(samples[:, 0], samples[:, 1], alpha=1.0, s=5, color=c)
        ax.set_title(f"t = {t}")
        ax.set_xlabel("")
        ax.set_ylabel("")
        ax.grid(True)
    plt.suptitle("Diffusion Model Reverse Process")
    plt.tight_layout()
    plt.show()
visualize_snapshots(snapshots)
```



扩散模型反向过程，原图依次为 $t = 999, 500, 200, 100, 50, 1$；点为生成样本，线为相应参考加噪样本的核密度等高线。

译注 参考等高线调用线性日程的 `ddpm.forward_process`，生成点却来自恒定 beta 日程，不能据每幅点线的错开直接判定模型误差。快照按反向更新后的 t 保存，参考正向数组索引也与理论从 1 计步约定存在差异，均按原代码保留。

定性来看，DDPM 网络准确捕捉了分布中的两个峰；更好的是，训练非常简单且稳定。只需要初始样本，以及按加噪日程施加的受控扰动。

展望后文，DDPM 还有三个重要扩展需要处理：

- 提高推断速度，主要是减少网络调用。
- 以外部参数和观测为条件。
- 在物理仿真中，通过基于 PDE 的约束重新引入先验知识。

尽管如此，DDPM 已完成最重要的第一步：将确定性网络变成概率推断工具，稳健学习并再现复杂分布。由此得到从数据学习分布的强大、稳定方法，剩下的问题是计算代价：概率 DDPM 网络需要多次顺序计算才能生成样本。这些神经函数求值（NFE）当然昂贵。许多实际场景可少于上面的 1000 次，但即使只做 10 次，也意味着网络比“普通”确定性版本慢 10 倍。这引出生成式 AI 旅程最后还需处理的问题：怎样加快推断，同时不损害目标，即所推断后验分布，的精度？[5]

译注：源实现与结论的适用范围 有限 T 且 $0 < \beta_t < 1$ 时，$q(x_T \mid x_0)$ 仍含 $\sqrt{\bar{\alpha}_T}x_0$，通常只接近而非严格等于标准正态。终端纯噪声是近似，L_T 通常非零但与模型参数 θ 无关；日程很长可使残留很小，不能把近似说成严格所有信息消失。

[5]译注：NFE 数目的 10 倍是源书的工作量类比，并非在所有硬件、批次或模型设置下经实测成立的墙钟时间比；本译稿未运行性能测试。

第二十五章　流匹配

为减少 DDPM 的大量函数求值，转向流匹配 [LCBH+22]。从去噪转向流匹配的动机，可借助式 (23.2) 的得分表述说明：沿连续的虚拟时间轴，对对数似然梯度 ∇_x 积分，将噪声变换为目标分布。简要回顾：用 Langevin 动力学

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \epsilon \nabla_x \log p(x) + \sqrt{2\epsilon}\, z_i$$

沿时间积分 $\nabla_{\widetilde{x}} \log p_\sigma(\widetilde{x})$，训练目标为

$$\arg\min_\theta \mathbb{E}\Big[\|s_\theta(\widetilde{x}) - \nabla_{\widetilde{x}} \log p_\sigma(\widetilde{x})\|^2\Big],$$

由此学习得分 $s_\theta(\widetilde{x})$。得分建模利用了扩散过程的概率流与常微分方程（ODE）形式的概率密度轨迹之间的数学等价性。[1]

25.1　用速度学习流

流匹配的目标是在速度场中构建流。得分匹配的梯度实际上可以直接解释为速度：忽略前面 Langevin 时间演化的随机项，剩下的速度 $\mathrm{d}x/\mathrm{d}t = \nabla_x \log p(x)$ 就是得分。得分匹配中一直面对的主要挑战，是如何得到良好的参考速度来训练神经网络。

现在有了去噪这一强大工具，可以重新审视流与速度：目标是学习连续时间流，为其指定尽可能简单的参考速度。与此前一样，速度视角使网络架构不受限制，而简单速度将使训练与推断比之前显著加快。

形式化地，流匹配等方法通常归为“连续时间流模型”，把采样分布 p_0 的样本 x，变换为目标或后验分布 p_1 的样本。映射用 ODE 表示：

$$\frac{\mathrm{d}x}{\mathrm{d}t} = v_\theta(x, t),$$

其中 $v_\theta(x,t)$ 为参数是 θ 的神经网络。流匹配通过回归一个向量场来训练 v_θ，该向量场生成从 p_0 映到 p_1 的概率路径。

若称为速度的光滑向量场 $u : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$ 满足连续性方程

$$\frac{\partial p}{\partial t} = -\nabla \cdot (p_t u_t),$$

[1]译注：源文把离散噪声 z_i 直接写在时间导数式右侧，照录；严格连续随机过程须用相应 SDE/Wiener 增量表述。概率流 ODE 与 SDE 的关系也不能仅通过把随机项直接删去来一般地推出，以下仍按源思路说明。

就称它生成概率路径 p_t。非正式地说，从 p_t 采样，可以先抽取 $x_0 \sim p_0$，再以 x_0 为初值求解 $\mathrm{d}x = u(x,t)\,\mathrm{d}t$。下文将 $u(x,t)$ 记为 $u_t(x)$。为回归速度场，定义流匹配目标

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\, x\sim p_t(x)} \|v_\theta(x,t) - u_t(x)\|^2. \tag{25.1}$$

要计算这一损失，必须能从 $p_t(x)$ 采样，并知道速度 $u_t(x)$。与得分匹配中采用扰动数据不同，流匹配换一种解决方式：引入服从 $q(z)$ 的潜变量 z，定义依赖它的条件似然 $p_t(x \mid z)$，使

$$p_t(x) = \int p_t(x \mid z) q(z)\,\mathrm{d}z.$$

有趣的是，若条件似然由速度 $u_t(x \mid z)$ 生成，则 $u_t(x)$ 可通过 $u_t(x \mid z)$ 与 $p_t(x \mid z)$ 表成期望：

$$u_t(x) = \mathbb{E}_{q(z)}\left[u_t(x \mid z)\frac{p_t(x \mid z)}{p_t(x)}\right].$$

可以自由选择易于采样、且生成速度 $u_t(x \mid z)$ 已知的路径 $p_t(x \mid z)$。因此，只要能生成足够多以 z 为条件的速度，就可在生成全部样本的过程中求得期望，获得训练网络所需的正确生成速度。

随后，用条件流匹配损失训练流网络：

$$\mathcal{L}_{\mathrm{CFM}}(\phi) = \mathbb{E}_{q(z,t),\, p_t(x|z)} \|v_\phi(x,t) - u_t(x \mid z)\|^2. \tag{25.2}$$

相比式 (25.1) 的无条件目标，这个版本可以实际计算并用于训练。也就是说，可训练 $v_\theta(x,t)$ 回归 $u_t(x)$，生成从 p_0 到目标分布 p_1 的映射。[2]

25.2 映射与条件化

特别重要的是：通过条件变量 z 与条件似然 p_t 指定 p_0 到 p_1 的映射时，有很大的自由度。如何充分利用它？事实表明，最简单的选择——直且不相交的路径——非常好。希望在时间 t，速度从当前状态 x 直接朝向目标 x_1，即 $(x_1 - x)/(1-t)$。唯一的小麻烦是，在 $t = 1$ 时仍需最小噪声 $\sigma_{\min} > 0$，使离散样本依然对应连续分布。这与此前得分匹配中的扰动样本一致。因此，$\sigma_{\min}$ 影响指定路径的终点及其概率分布。

代入公式，考虑耦合 $q(z) = p_1(x)$，相应条件概率与生成速度为

$$\begin{aligned} p_t(x \mid x_1) &= \mathcal{N}\big(tx_1, (1 - (1-\sigma_{\min})t)I\big), \\ u_t(x \mid x_1) &= \frac{x_1 - (1-\sigma_{\min})x}{1 - (1-\sigma_{\min})t}. \end{aligned} \tag{25.3}$$

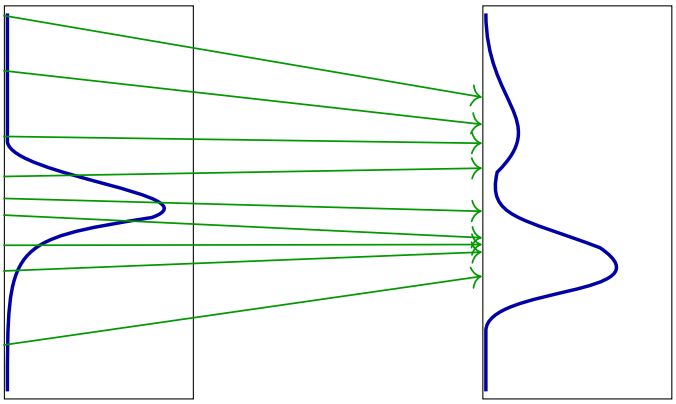
以 x_1 为条件，这一耦合将采样分布中的点 $x_0 \sim \mathcal{N}(0, I)$，沿线性轨迹 tx_1 输运到终点 x_1 所在的后验分布。同时，将标准差从 1 减小到平滑常数 $\sigma_{\min}$。此时，输运路径与两个 Gaussian 分布间的最优输运一致。[3]

[2] 译注：条件损失公式用 ϕ，周围文字用 θ；参数记号按源保留。

[3] 译注：Gaussian 式把 $1 - (1-\sigma_{\min})t$ 写在协方差位置，文字却将它作为标准差；若按后者解释，协方差应含该量平方。代码路径终点为 $x_1 + \sigma_{\min} x_0$，并非每个样本严格到 x_1；均值轨迹才是 tx_1。原式与文字各自保留。

乍看好像退了一大步：又回到变换分布，而前面归一化流一节曾指出这一想法存在问题。但借助得分匹配与去噪的方法论，现在得到的是根本不同且更强的分布变换方式。例如，不再受保持密度的约束，而且学习目标容易计算、使用方便。与神经 ODE 不同，可以用单步训练，不必沿完整链从末端到起点反向传播。[4]

此外，流匹配相对去噪扩散模型的一项根本优势，来自巧妙选择生成速度路径：沿朝向目标的线性速度，就会从原样本到目标点走直线。这里的线性意味着，可以从任意起点，用单个 Euler 步计算结果。图 25.1 展示了这种对照：沿弯曲、无约束的去噪路径，可能需要数百步。实际中并非总得到完美直线，因此单步可能不理想，或需进一步微调、蒸馏流。尽管如此，与去噪相比，流匹配通常所需迭代显著减少。推断时间正比于步数，因此计算后验样本时相应加快。最重要的是，没有牺牲最初目标：方便采样，以及仅凭数据准确覆盖后验。[5]



基分布 p_0 $v_\theta(x_t, t)$ 目标分布 p_1

图 25.1: 流匹配，与 CNF 及 DDPM 对照：简单基分布（左，p_0）通过对推断速度的 ODE 积分，得到目标分布（右，p_1）样本。线性路径只需很少积分步，绿色轨迹标为 $v_\theta(x_t, t)$。原图 25.1。

25.3 实现流匹配

与 DDPM 一样，首先初始化标准 Gaussian 混合（GM）例子。[6]

```
import numpy as np
class GaussianMixture:
    def __init__(self, parameters):
        self.parameters = parameters
        self.distributions = [
            {
                'mean': np.array(dist['mean']),
                'std': np.array(dist['std']),
                'cov': np.diag(np.array(dist['std']) ** 2)
```

[4]译注：原文“不再受保持密度约束”按其措辞保留；由连续性方程生成的流仍保持概率质量。此处便利在于训练无需显式计算完整密度及其归一化/Jacobian 项。

[5]译注：原文的单 Euler 步与时间正比陈述是理想路径/固定每步成本下的说明，并非对任意边缘学习速度、数值误差、网络架构或自适应积分器的统一保证。

[6]译注：本章全部代码仅转写并作 token/AST 核查，未训练、采样、安装依赖或下载数据。图像及日志为冻结源输出。

```
            }
            for dist in parameters
        ]
    def sample(self, num_samples):
        samples = []
        num_distributions = len(self.distributions)
        for _ in range(num_samples):
            idx = np.random.randint(num_distributions) # Choose a random Gaussian
            dist = self.distributions[idx]
            sample = np.random.multivariate_normal(mean=dist['mean'], cov=dist['cov'])
            samples.append(sample)
        return np.array(samples)
parameters = [
    {"mean": [0, 0], "std": [1, 1]},
    {"mean": [3, 2], "std": [0.5, 0.5]}
]
mixture = GaussianMixture(parameters)
```

设置训练数据集，比去噪任务更简单。不必加入噪声日程，只需为每次请求的样本随机选取时间 t，然后计算直线路径的速度 `u_t`。

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
class FlowMatchingDataset(Dataset):
    def __init__(self, mixture, n_samples=1000, sigma_min=1e-4):
        super().__init__()
        self.n_samples = n_samples
        self.mixture = mixture
        self.sigma_min = sigma_min
        self.x1 = mixture.sample(n_samples)
    def __len__(self):
        return self.n_samples
    def __getitem__(self, idx):
        x0 = np.random.multivariate_normal([0.0, 0.0], np.eye(2), 1)[0]
        t = np.random.rand() # scalar in [0,1]
        x1 = self.x1[idx]
        x_t = (1 - ( 1 - self.sigma_min) * t) * x0 + t * x1
        u_t = (x1 - x0)
        x_t = torch.tensor(x_t, dtype=torch.float32)
        t = torch.tensor([t], dtype=torch.float32)
        u_t = torch.tensor(u_t, dtype=torch.float32)
        return x_t, t, u_t
```

译注 源代码对 x_t 使用 $[1-(1-\sigma_{\min})t]x_0 + tx_1$，但监督速度写 $x_1 - x_0$；前者对 t 的精确导数为 $x_1 - (1-\sigma_{\min})x_0$。默认 $\sigma_{\min} = 10^{-4}$ 时差异虽小，仍不完全相同，故照录并说明，未改代码。

与此前类似，流匹配使用一个简单的三层神经网络作为骨干。

```
class VelocityNet(nn.Module):
    def __init__(self, in_dim=2, time_dim=1, hidden_dim=128, out_dim=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim + time_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim)
        )
    def forward(self, x, t):
        xt = torch.cat([x, t], dim=1)
        return self.net(xt)
```

25.3.1 训练速度模型

现在可以训练速度模型：直接让网络预测数据集类给出的“监督”真实速度。与去噪一样，这是目标明确且稳定的学习任务。下面单元训练 50 轮。

```
n_samples = 10000
batch_size = 128
n_epochs = 50
learning_rate = 1e-3
dataset = FlowMatchingDataset(mixture, n_samples=n_samples)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
model = VelocityNet().to(device)
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
criterion = nn.MSELoss()
for epoch in range(n_epochs):
    running_loss = 0.0
    for x_t, t, u_t in dataloader:
        x_t = x_t.to(device)
        t = t.to(device)
        u_t = u_t.to(device)
        optimizer.zero_grad()
        pred_v = model(x_t, t)
        loss = criterion(pred_v, u_t)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * x_t.size(0)
    running_loss /= len(dataset)
    print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {running_loss:.4f}")
```

原书输出如下，省略号为源已有：

```
Epoch 1/50, Loss: 2.8302
Epoch 2/50, Loss: 2.2797
```

```
Epoch 3/50, Loss: 2.2231
Epoch 4/50, Loss: 2.1215
Epoch 5/50, Loss: 2.1258
Epoch 6/50, Loss: 2.0737
Epoch 7/50, Loss: 2.1223
Epoch 8/50, Loss: 2.0625
Epoch 9/50, Loss: 2.0656
Epoch 10/50, Loss: 2.0527
...
Epoch 40/50, Loss: 2.0522
Epoch 41/50, Loss: 2.0709
Epoch 42/50, Loss: 2.0460
Epoch 43/50, Loss: 2.0688
Epoch 44/50, Loss: 2.0273
Epoch 45/50, Loss: 2.0711
Epoch 46/50, Loss: 2.0120
Epoch 47/50, Loss: 2.0344
Epoch 48/50, Loss: 2.0347
Epoch 49/50, Loss: 2.0537
Epoch 50/50, Loss: 2.0440
```

25.3.2 通过求解 ODE 推断

推断时，只需从随机样本出发，积分网络给出的速度。可以用简单 Euler 步；这里仍用 `torchdiffeq` 的 ODE 求解器，以增加灵活性。也可尝试其他积分格式。

```
try:
    import google.colab # only to ensure that we are inside colab
    %pip install --quiet torchdiffeq
except ImportError:
    print("This notebook is running locally, please install torchdiffeq manually.")
```

积分本身由 `odeint` 执行，通过 `ode_func` 包装调用训练后的网络。默认沿轨迹积分 100 步，实际中更少步骤也应有良好结果。即便如此，仍比前面 DDPM 版本少一个数量级。

```
import torch
from torchdiffeq import odeint
def integrate_flow(model, x0, t_span=(0.0, 1.0), n_steps=100, method='dopri5'):
    t = torch.linspace(t_span[0], t_span[1], n_steps).to(x0.device)
    def ode_func(t, x):
        t_tensor = t.expand(x.shape[0], 1)
        return model(x, t_tensor)
    trajectory = odeint(ode_func, x0, t, method=method,
                        atol=1e-5, rtol=1e-5)
    return trajectory, t
n_gen = 500
x0_gen = torch.randn(n_gen, 2).to(device)
trajectory, time_points = integrate_flow(model, x0_gen)
```

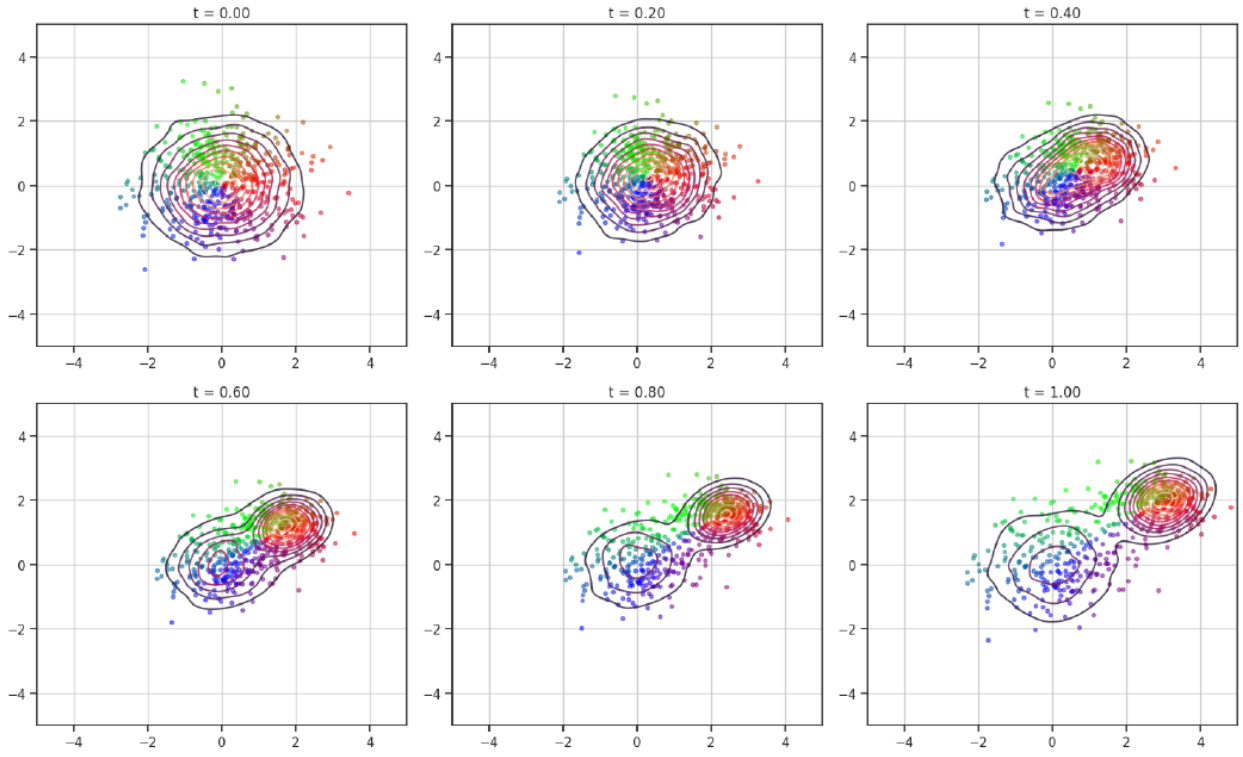
译注 默认 dopri5 是自适应求解器；此处 n_steps=100 设置请求输出的时间点数量，不能直接等同于内部积分步数或神经函数求值 NFE。因此原文的 100 步与 DDPM1000 次调用之比，不是由这段代码已测得的 NFE 或运行时间比。

25.3.3 可视化采样轨迹

从标准 Gaussian 分布的点开始，按初始位置着色。下面仍显示轨迹中六个时刻的状态。

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="ticks", palette="pastel")
def get_angle_colors(positions):
    angles = np.arctan2(positions[:, 1], positions[:, 0])
    angles_deg = (np.degrees(angles) + 360) % 360
    colors = np.zeros((len(positions), 3))
    for i, angle in enumerate(angles_deg):
        segment = int(angle / 120)
        local_angle = angle - segment * 120
        if segment == 0: # 0 degrees to 120 degrees (R->G)
            colors[i] = [1 - local_angle/120, local_angle/120, 0]
        elif segment == 1: # 120 degrees to 240 degrees (G->B)
            colors[i] = [0, 1 - local_angle/120, local_angle/120]
        else: # 240 degrees to 360° (B->R)
            colors[i] = [local_angle/120, 0, 1 - local_angle/120]
    return colors
# Desired time points to visualize
desired_times = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]
# Determine indices in the trajectory corresponding to these times.
# We assume time_points is a 1D tensor of size n_steps.
time_np = time_points.detach().cpu().numpy()
n_steps = len(time_np)
# Get the index for each desired time: here we choose the index with minimum absolute difference.
indices = [np.argmin(np.abs(time_np - t_val)) for t_val in desired_times]
# Create subplots: We'll use 2 rows and 3 columns, one subplot per time.
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(15, 10))
axes = axes.ravel() # flatten the 2D array for easier indexing
# Create grid for density visualization
xx, yy = np.mgrid[-5:5:100j, -5:5:100j]
positions = np.vstack([xx.ravel(), yy.ravel()])
for i, idx in enumerate(indices):
    ax = axes[i]
    # Get the samples at this time point: shape (batch, 2)
    x_t = trajectory[idx].detach().cpu().numpy()
    if i == 0:
        c = get_angle_colors(x_t)
    x_0 = mixture.sample(5000)
```

```
    t = time_np[idx]
    eps = np.random.randn(5000, 2)
    x_t_forward = t * x_0 + (1-t) * eps
    samples_ = x_t_forward
    # Compute KDE for visualization
    from scipy.stats import gaussian_kde
    kernel = gaussian_kde(samples_.T)
    density = np.reshape(kernel(positions).T, xx.shape)
    # Plot density contours
    ax.contour(xx, yy, density, levels=10, alpha=0.8)
    # Scatter plot: plot each sample as a point
    ax.scatter(x_t[:, 0], x_t[:, 1], alpha=0.5, s=10, color=c)
    ax.set_title(f"t = {time_np[idx]:.2f}")
    ax.set_xlabel("")
    ax.set_ylabel("")
    ax.grid(True)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



原图从 $t = 0.00$ 到 1.00 展示六个时刻；彩色点为 ODE 生成轨迹，线为参考样本核密度等高线。

结果正如预期：两个密度峰都有很好的表示。现在，流匹配提供了一种从后验生成准确样本的方法，相比确定性模型，其计算开销相对温和。[7]

[7]译注：源段末有装饰符号渲染异常，文本提取为 “2/7”，没有据此猜译，原页 PNG 保留。绘图参考路径使用 $tx_0 + (1-t)\epsilon$，没有训练中的 $\sigma_{\min}$ 项，亦按原代码保留。此处玩具例子实际为无条件 GM，原文沿用 “后验” 称呼。

25.4 小结

流匹配算法是扩散模型领域的重要里程碑，也结束了这次深度学习生成建模历史之旅。有趣的是，在提出之后，人们发现去噪、流匹配及其他变体，比从各自推导所想象的更相近。不过，从它们各自看待问题的视角理解，通常比从更一般的数学框架出发更容易。这仍是非常活跃的领域，原书建议上网浏览近期进展。

现在，是展示这些方法在比 GM 玩具问题更具体、更复杂例子中能力的时候了。下一个 notebook 就会这样做：先在一个带不确定性的 RANS 翼型任务上，比较去噪扩散模型与流匹配模型。[LCBH+22] 完整引文待全书书目单元接入。

译注：独立核对的适用条件 直线是以目标 x_1 条件化的参考轨迹；网络回归的边缘场是 $u_t(x) = \mathbb{E}[u_t(x \mid x_1) \mid x_t = x]$，即使精确学到，它的一般积分轨迹也不等于每条条件直线。因此参考线性不能直接推出单 Euler 步精确生成，也不能单独证明全局最优输运。

第二十六章　并排比较去噪与流匹配

为展示去噪扩散与流匹配的能力，选用一个能够可靠生成任意数量真实数据的学习任务，从而量化目标分布学得如何。具体考虑翼型周围的 Reynolds 平均 Navier–Stokes（RANS）仿真。一个有趣特征是：在较高 Reynolds 数下，OpenFoam 等典型求解器会由稳态解转为振荡解。下面把这一转变作为扩散模型的学习任务。细节见作者的 diffusion-based flow prediction 仓库。为使 notebook 自成一体，还会回顾前节最重要的概念。源文另标有 “在 Colab 中运行”。[1]

注：正向问题的记号切换　若从上一章接着读，需要注意一个重要区别：为简化问题，这里暂离基于仿真的推断视角，将去噪与流匹配用于正向问题。不是从观测 y 恢复 x，而是假定已有初始条件 x，希望计算解 y。因此，下文 x、y 的角色切换并非误排。

26.1　导言

过去几年，扩散模型成为深度学习的热门方法，使我们能够用出奇简单、稳健的训练设置，训练强大生成模型。在这个子领域中，一个很有前景的新进展是基于流的方法，通常称作流匹配 [LCBH+22] 或整流流（rectified flow）[LGL22]。为简单起见，这里采用前一种称呼，将这类模型记作 FM。

在最初的扩散模型中，去噪任务尤其成功：神经网络学会从纯噪声恢复信号。得分函数提供另一视角，但最终也归结到去噪任务。基于流的方法则旨在变换分布：将 Gaussian 噪声等已知分布，变换为所关心的信号或目标函数的分布。虽然视角看似不同，实际上都是从噪声出发，逐步得到目标信号的样本。有趣的是，FM 视角不仅使训练更稳定，还因路径更直而使推断加快数个数量级。更好的是，若已有可用 DM 设置，转换成 FM 出奇简单。[2]

以下结合前述 RANS 流动，突出两种方法的相同与不同，并进行评价。

[1]译注：仓库 URL 在下方代码中保留；其他仅有链接文字但无可恢复目标的地址不猜补。本章不运行安装、克隆、数据解包/下载、训练或推断，图与日志全部来自冻结源。

[2]译注：稳定性与速度比较均为原书在其背景下的陈述，非本译稿重新测试的普遍性能保证。

26.2　问题陈述

与此前监督学习任务不同，现在需要考虑分布。“经典” 监督任务给出唯一输入输出对 (x, y)，训练模型根据内部参数 θ，由 x 给出 y，即 $y = f(x; \theta)$。

相反，假定有某个隐藏状态 ψ，对同一个 x 也会变化。例如测量噪声、逆问题优化的起点，或本例中不收敛的 RANS 求解器解。于是，可用随机变量 Y 表述问题，解从分布 $y \sim P_Y(Y)$ 抽取。通常以同一 x 下、不同 ψ 的样本给出边缘化分布。从概率视角，重要的是捕捉解的条件概率 $p(y \mid x)$，并对 ψ 边缘化；实践中无需知道 ψ 的具体细节。

生成模型应学习的，就是这个条件分布 $p(y \mid x)$，即后验：反复计算模型时，应以正确概率产生后验样本，并且高效，不浪费计算。[3]

26.3　实现与设置

先安装所需包并克隆仓库：

```
try:
    import google.colab # to ensure that we are inside colab
    !pip install --upgrade --quiet einops bayesian_torch
except ImportError:
    print('This notebook is running locally, please make sure the necessary pip
    ↪  packages are installed.')
    pass
!git clone https://github.com/tum-pbs/Diffusion-based-Flow-Prediction.git
%cd Diffusion-based-Flow-Prediction/
```

原书输出警告：

```
site-packages/IPython/core/magics/osm.py:417: UserWarning: This is now an optional
↪  IPython functionality, setting dhist requires you to install the `pickleshare`
↪  library.
  self.shell.db['dhist'] = compress_dhist(dhist)[-100:]
```

还需准备训练数据集。下面的数据可由 OpenFoam 生成，为方便起见在此下载。组织数据的数据结构与 `DataFiles` 类来自上述仓库；源文另给出详情链接。

```
import zipfile
from airfoil_diffusion.airfoil_datasets import *
from airfoil_diffusion.networks import *
from airfoil_diffusion.trainer import *
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print("Using device: "+str(device))
if not os.path.exists("./datasets/1_parameter/data/"):
    files=[file for file in os.listdir("./datasets/1_parameter/") if
    ↪  file.endswith(".zip")]
    for file in tqdm(files):
```

[3]译注：此处为正向条件生成，源文仍称条件分布 “后验”，照录；$P_Y(Y)$ 的源记号也保留，不自行改写成密度定义。

```
        f=zipfile.ZipFile("./datasets/1_parameter/"+file,'r')
        for file in f.namelist():
            f.extract(file,"./datasets/1_parameter/data/")
        f.close()
df_train=FileDataFiles("./datasets/1_parameter/train_cases.txt",base_path="./datasets」
↪  /1_parameter/data/")
train_dataset = AirfoilDataset(df_train,data_size=32)
```

原书输出：

```
Using device: cuda:0
Loading data: 100%|██████████| 125/125 [00:00<00:00, 341.19it/s]
```

接着实现去噪扩散模型，以便与流匹配比较。

26.4 用扩散模型去噪

核心去噪任务如名称所示：通过最小化 $\|\epsilon - f_\theta(x,t)\|^2$，学习估计噪声 ϵ。关键是仔细控制噪声，使其容易学习。

本节所有公式省去此前 “模型与方程” 中表示批次或数据集中不同样本的下标 i，例如将 y_i 简记为 y，并省略对 i 的求和。

先定义正向加噪过程，使 $t=1$ 时得到完美 Gaussian 分布 $\mathcal{N}(0,I)$。在 $t=0$，从数据样本 y，即 $\mathcal{N}(y,0)$ 出发，随着 t 增大，逐渐转成噪声。加噪/去噪时间 t 是虚拟时间，没有物理意义。均值与标准差的变化由噪声日程 $\beta^t \in (0,1)$ 控制，从 0 逐渐增加，在链末端 $t=1$ 达到 $\beta^t = 1$。

选择 Gaussian 分布，就可将各步解耦为分布 q 的 Markov 链：

$$q(y_{0:T}) = q(y_0)\prod_{t=1}^{T} q(y_t \mid y_{t-1}), \qquad q(y_t \mid y_{t-1}) = \mathcal{N}(\sqrt{1-\beta^t}\, y_{t-1}, \beta^t I).$$

累积越来越多噪声，显然可以破坏任意输入信号 y。更有趣的是移除噪声的反向过程。类似地，分布 p_θ 的反向 Markov 链为

$$p_\theta(y_{0:T}) = p(y_T)\prod_{t=1}^{T} p_\theta(y_{t-1} \mid y_t),$$

下标表明转移将由参数 θ 学习，并且

$$y_t = \sqrt{\bar{\alpha}_t}\, y_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon. \tag{26.1}$$

系数由正向链噪声日程给出：$\alpha_t = 1-\beta^t$、$\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$。每个反向步骤具有具体形式

$$p_\theta(y_{t-1} \mid y_t) = \mathcal{N}(\mu(f_\theta), \sigma_\theta). \tag{26.2}$$

这里用网络 f_θ 预测噪声；也可称为 ϵ_θ，但为一致起见仍记作 f_θ。网络推断的噪声参数化均值：

$$\mu(\epsilon) = \frac{1}{\sqrt{\alpha_t}}\left(y_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right).$$

有趣的是，标准差不依赖噪声或网络，而按时间演化：

$$\sigma = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta^t I.$$

因此，给定 (x, y)，可直接计算 $t-1$ 与 t 的适当噪声强度，生成 y_t 与 y_{t-1}。

实践中，不只要任意 y，还希望它符合选定 Reynolds 数等全局参数。这些条件与翼型形状一起，正是前面 $f(x) = y$ 中的 x。因此，还要让 f 以 x 为条件，写成 $f_\theta(y_t, x, t)$。

具体做法是选取时间 t、计算适量噪声 ϵ，并按式 (26.1) 通过 $\bar{\alpha}_t$ 线性插值得到 y_t，让网络预测噪声。损失为

$$\mathcal{L}_{\mathrm{DM}}(\theta) = \mathbb{E}_{x,\epsilon\sim\mathcal{N}(0,I),t}\left[\|\epsilon - f_\theta(y_t, x, t)\|^2\right].$$

译注 源文连续/离散 t、β^t/β_t 并用；$\beta^t \in (0,1)$ 与末端等于 1 的措辞、将协方差写作 “标准差”、式 (26.2) 的 σ_θ 却随后说不依赖网络，均照录。式 (26.1) 实际是正向边缘加噪采样式，位于源反向链讨论中；原文本 “ref(eq-yt)” 未正确渲染，这里绑定其实际对应源式 26.1。损失中的期望下标按源保留，不能据此把条件数据 x 解释为必然服从标准 Gaussian。

26.5 实现 DDPM

实现分为两部分：计算去噪日程系数的辅助类，以及管理训练循环的训练器类。

辅助类称为 `MyDiffuser`，先计算所谓余弦 beta 日程，其形式为

$$\beta = \cos^2\left(\frac{t/T + s}{1+s}\frac{\pi}{2}\right).$$

偏移 s 的选取，使 $\sqrt{\beta}$ 的标准差小于典型 RGB 图像的颜色步长 $1/256$。函数 `generate_parameters_from_beta()` 将预计算的 beta 列表转成 PyTorch 张量，同时计算相应 alpha 与 alpha_bar。类还实现 `forward_diffusion()`，给定输入 y, t，通过预计算 alpha_bar 进行正向扩散，得到上述 y_t。

```
class MyDiffuser():
    def __init__(self, steps, device):
        self.device = device
        self.steps = steps
        self.name = "Cos2ParamsDiffuser"
        s = 0.008
        tlist = torch.arange(1, self.steps+1, 1)
        temp1 = torch.cos((tlist/self.steps+s)/(1+s)*np.pi/2)
        temp1 = temp1*temp1
        temp2 = np.cos(((tlist-1)/self.steps+s)/(1+s)*np.pi/2)
        temp2 = temp2*temp2
        self.beta_source = 1-(temp1/temp2)
        self.beta_source[self.beta_source > 0.999] = 0.999
        self.generate_parameters_from_beta()
    def generate_parameters_from_beta(self):
        self.betas = torch.cat( (torch.tensor([0]), self.beta_source), dim=0)
        self.betas = self.betas.view(self.steps+1, 1, 1, 1)
```

```
        self.betas = self.betas.to(self.device)
        self.alphas = 1-self.betas
        self.alphas_bar = torch.cumprod(self.alphas, 0)
        self.one_minus_alphas_bar = 1 - self.alphas_bar
        self.sqrt_alphas = torch.sqrt(self.alphas)
        self.sqrt_alphas_bar = torch.sqrt(self.alphas_bar)
        self.sqrt_one_minus_alphas_bar = torch.sqrt(self.one_minus_alphas_bar)
    def forward_diffusion(self, y0, t, noise):
        yt = self.sqrt_alphas_bar[t]*y0+self.sqrt_one_minus_alphas_bar[t]*noise
        return yt
```

译注 源文将余弦平方本身称为 beta；实际代码取相邻余弦平方之比的补数，并截断到 0.999，二者不同。原公式、解释与代码均保留，未替作者统一日程定义。

现在可以训练。`MyDiffusionTrainer` 继承仓库中的训练器基类，主要负责参数、记账及其他常规工作，确保获得训练批次，再调用最重要的 `train_step()`。

它实现上述过程：给定 y，随机选 t，计算噪声和正向一步得到的 y_t；让网络从 y_t、条件 x 与 t 预测噪声 ϵ。最后计算真实与预测噪声的 MSE，再由 PyTorch 反向传播，更新网络权重。

```
class MyDiffusionTrainer(TrainerStepLr):
    def __init__(self) -> None:
        super().__init__()
    def set_configs_type(self):
        super().set_configs_type()
        self.configs_handler.add_config_item("diffusion_step",value_type=int,default_v」
        ↪  alue=200,description="The number of diffusion steps.")
    def event_before_training(self,network):
        self.diffuser =
        ↪  MyDiffuser(steps=self.configs.diffusion_step,device=self.configs.device)
    def train_step(self, network: torch.nn.Module, batched_data, idx_batch: int,
    ↪  num_batches: int, idx_epoch: int, num_epoch: int):
        condition = batched_data[0].to(device=self.configs.device)
        targets = batched_data[1].to(device=self.configs.device)
        t = torch.randint(1, self.diffuser.steps+1,
                          size=(targets.shape[0],), device=self.configs.device)
        noise = torch.randn_like(targets, device=self.configs.device)
        yt = self.diffuser.forward_diffusion(targets, t, noise)
        predicted_noise = network(yt, t, condition)
        loss=torch.nn.functional.mse_loss(predicted_noise, noise)
        return loss
diffusion_trainer = MyDiffusionTrainer()
dif_network =
↪  AifNet("./pre_trained/single_parameter/32/diffusion/network_configs.yaml")
```

该单元末尾直接实例化训练器与神经网络。`AifNet` 实现带有扩散建模新技巧的先进 U-net，但归根结底“只是 U-net”，这里省略细节。

更重要的是，终于可以用训练器的 `train_from_scratch()` 训练 DDPM。默认 10000 步可能需要

一些时间，原书预计在当时 GPU 上不超过约半小时。不愿等待时，也可跳过训练，用末尾注释代码加载 DBFP 仓库中的预训练模型。

```
diffusion_trainer.train_from_scratch(name="diffusion", #device="cpu",
                                     network=dif_network,
                                     train_dataset=train_dataset,
                                   path_config_file="./pre_trained/train_configs.yaml",
                                     save_path="./training/single_parameter/32/",
                                     ↪  epochs=10000)
# alternative load
#dif_network.load_state_dict(torch.load("./pre_trained/single_parameter/32/diffusion/」
↪  weights_0.pt"))
```

原书完整输出如下：

```
Trainer created at 2024-11-05-03_25_26
Working path:./training/single_parameter/32/diffusion/2024-11-05-03_25_26/
Random seed: 1730773526
Training configurations saved to
↪  ./training/single_parameter/32/diffusion/2024-11-05-03_25_26/configs.yaml
Network has 1185218 trainable parameters
There are 5 training batches in each epoch
Batch size for training:25
Training epochs:10000
Total training iterations:50000
learning rate:0.0001
Optimizer:AdamW
Learning rate scheduler:step
Training start!
  0%| | 0/10000 [00:00<?, ?it/s]/tmp/ipykernel_65435/3287631965.py:11:
  ↪  DeprecationWarning: __array_wrap__ must accept context and return_scalar
  ↪  arguments (positionally) in the future. (Deprecated NumPy 2.0)
  temp2 = np.cos(((tlist-1)/self.steps+s)/(1+s)*np.pi/2)
lr:1.000e-05 train loss:0.00100: 100%|██████████| 10000/10000 [30:40<00:00, 5.43it/s]
Training finished!
```

译注　文字称 10000 步，而调用参数为 10000 个 epoch，日志显示每轮 5 批，共 50000 次迭代。1,185,218 参数、原始种子、路径、警告、训练损失与耗时均是源记录，不是本次远端实测。

先直接训练流匹配版本，再并排评价两者能力。

26.6 流匹配

流匹配变换概率分布，而非加噪、去噪。考虑随时间变化的可微映射 $\phi:[0,1]\times\mathbb{R}^d\to\mathbb{R}^d$，将初始分布 $p(x_0)$ 中的样本 x_0，变成分布 $p(x_t)$ 中的样本 x_t，简记 $x_t=\phi(x_0)$。

之后，x_0 对应 Gaussian 等简单分布样本，类似之前扩散模型的初始噪声；x_1 则对应目标分布，即训练数据中的样本，也是前述记号中的 y。从 Gaussian 噪声走向目标，外观上类似去噪：从很嘈杂到没有噪声，虽然流匹配原始表述未必以此为目标。变换分布时，采用随 t 连续变化的 $p(x_t)$ 很方便。只需记住，在 $t=1$，x_1 与 y 相同，即 $p(x_t)|_{t=1}=p(x_1)=p(y)$。

流匹配学习变换的时间导数，即流，表示为随时间变化的向量场 $u:[0,1]\times\mathbb{R}^d\to\mathbb{R}^d$，其中 $u_t(x_t)=\mathrm{d}x_t/\mathrm{d}t$。网络 $f_\theta(x,t)$ 的损失，就是预测速度与目标速度的 L^2：

$$\mathcal{L}_{\mathrm{FM}}(\theta)=\mathbb{E}_{t\sim[0,1],\,x_t\sim p(x_t)}\|f_\theta(x_t,t)-u_t(x_t)\|^2,$$

这里 p_t 表示时间 $t\sim[0,1]$ 的中间分布。

看起来出奇简单，也很像上面噪声估计的损失。但没有额外技巧，这一损失难以计算，因为中间分布 $p(x_t)$ 与正确速度 u_t 都未知。

幸运的是，已有工作表明，可利用已知样本 x_1 驱动过程，而不扭曲分布。基于 x_1 构造条件向量场 $u_t(x_t\mid x_1)$，再积分消去 x_1，得到边缘中间密度与向量场：

$$\begin{aligned}p(x_t)&=\int p(x_t\mid x_1)p(x_1)\,\mathrm{d}x_1,\\ u_t(x_t)&=\int u_t(x_t\mid x_1)\frac{p(x_t\mid x_1)p(x_1)}{p(x_t)}\,\mathrm{d}x_1.\end{aligned}$$

通过这种边缘化，已有研究证明，学习条件流在数学上等价于学习原始流。

于是有了指定流的自由；直的整流运动尤其有趣。它们可由最优输运导出，在 $p(x_0)$ 与 $p(x_1)$ 样本间定义线性映射。从 $t=0$ 的标准 Gaussian 开始，希望标准差 σ_t 按 $1-t$ 线性下降，直到 $t=1$ 时不再有随机性；同时均值由零变化到 x_1，即 $\mu_t(x_1)=tx_1$。映射为

$$\phi_t(x_0)=\sigma_t(x_t)x_0+\mu_t(x_t),$$

其时间导数为速度：

$$u_t(x_t\mid x_1)=\frac{\mathrm{d}}{\mathrm{d}t}\phi_t(x_0)=\sigma_t'(x_1)x_0+\mu_t'(x_1).$$

实际中引入阈值 $\sigma_{\min}$，确保标准差大于零，且选得足够小，使 $p(x_1\mid x_1)$ 表示以 x_1 为中心、非常集中的 Gaussian。标准差日程为 $\sigma_t(x_1)=1-(1-\sigma_{\min})t$。

这一设置的巨大优势是，确实给出“直”的运动，向量场为常数，与时间 t 无关。若一切按预期进行，就能从 x_0 到 x_1 用单个 Euler 步完成变换。相比之下，去噪网络可以学习任意路径，因此需更多步才能到达目标。实际中单步推断还需一些技巧，但下面会看到，即使采用这个简单基本表述，流匹配也比去噪所需步数少得多。

现在有了目标速度的全部组成部分：

$$u_t(x_t\mid x_1)=x_1-(1-\sigma_{\min})x_0.$$

条件流匹配损失为

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t\sim[0,1],\, x_1\sim p(x_1),\, x_t\sim p(x_t|x_1)} \left\| f_\theta(x_t, t, x_1) - u_t(x_t \mid x_1) \right\|^2 .$$

训练后，查询网络给出的向量场，再沿时间积分，从 x_0 生成 x_1：

$$x_1 = x_0 + \int_0^1 f_\theta(x_t, t)\,\mathrm{d}t.$$

任意 ODE 求解器均可用。为简单起见，以下用 Euler 步；DBFP 代码还提供多种高阶方法供尝试。

译注 映射式的 $\sigma_t(x_t), \mu_t(x_t)$ 与后续导数的 x_1 自变量按源各自保留。恒定速度指给定端点样本对的条件直线路径，不意味着边缘网络向量场处处、全时均为常数。源条件损失将目标端点 x_1 写进网络输入，随后积分式没有此输入；实际实现由物理条件 `condition` 输入网络，不在推断时提供未知目标。源推导、记号与代码差异均不静默统一。

26.7 实现流匹配

仍将核心功能与训练代码分开。辅助类 `MyFlowMatcher` 更简单：`phi_t()` 对两个样本 x_0, x_1 线性插值；`u_t()` 计算时间导数。由于目标是直线运动，条件速度在时间上为常数，不再依赖 t。最后的 `cfm_loss()` 计算目标速度与单步条件流匹配损失。

```
class MyFlowMatcher():
    def __init__(self):
        self._unifrom_sampler = torch.distributions.uniform.Uniform(0.,1.)
        self.sig_min = 0.001
    def phi_t(self, x_0, x_1, t):
        return (1-(1-self.sig_min)*t)*x_0 + t*x_1
    def u_t(self, x_0, x_1): # note - linear flow, does not depend on t anymore!
        return x_1-(1-self.sig_min)*x_0
    def cfm_loss(self, network, x_1, x_0=None, *args,**kwargs):
        x_0 = torch.randn_like(x_1) if x_0 is None else x_0
      t = self._unifrom_sampler.sample([x_1.shape[0]]+[1]*(x_1.dim()-1)).to(x_1.device)
        x_t = self.phi_t(x_0,x_1,t)
        v_t = self.u_t(x_0,x_1)
        return torch.mean( (network(x_t,t.view(t.shape[0]),*args,**kwargs) - v_t) **2)
```

流匹配训练器也使用基类，主要把目标和条件数据转到 PyTorch 设备，再调用 `cfm_loss()`，几乎没有其他工作。

```
class MyFMTrainer(TrainerStepLr):
    def __init__(self) -> None:
        super().__init__()
    def event_before_training(self,network):
        self.flow_matcher = MyFlowMatcher()
    def train_step(self, network: torch.nn.Module, batched_data, idx_batch: int,
    ↪  num_batches: int, idx_epoch: int, num_epoch: int):
```

```
        condition = batched_data[0].to(device=self.configs.device)
        targets = batched_data[1].to(device=self.configs.device)
       loss=self.flow_matcher.cfm_loss(network=network,x_1=targets,condition=condition)
        return loss
```

接着实例化训练器与网络。为公平比较两种训练方法，使用与去噪完全相同的 U-net。

```
fmatching_trainer=MyFMTrainer()
network = AifNet("./pre_trained/single_parameter/32/diffusion/network_configs.yaml")
```

现在可开始训练。与之前类似，原书预计 10000 轮约半小时；若希望跳过，可用下面代码加载 GitHub 仓库中的模型。

```
fmatching_trainer.train_from_scratch(name="flowmatching",
                                     network=network,
                                     train_dataset=train_dataset,
                                   path_config_file="./pre_trained/train_configs.yaml",
                                     save_path="./training/single_parameter/32/",
                                     ↪  epochs=10000)
# uncomment to load the checked in model
#network.load_state_dict(torch.load("./pre_trained/single_parameter/32/flow_matching/」
↪  weight.pt"))
```

原书完整输出：

```
Trainer created at 2024-11-05-03_56_06
Working path:./training/single_parameter/32/flowmatching/2024-11-05-03_56_06/
Random seed: 1730775366
Training configurations saved to
↪  ./training/single_parameter/32/flowmatching/2024-11-05-03_56_06/configs.yaml
Network has 1185218 trainable parameters
There are 5 training batches in each epoch
Batch size for training:25
Training epochs:10000
Total training iterations:50000
learning rate:0.0001
Optimizer:AdamW
Learning rate scheduler:step
Training start!
lr:1.000e-05 train loss:0.00430: 100%|██████████| 10000/10000 [30:44<00:00, 5.42it/s]
Training finished!
```

终于有了可并排评价的两个训练后模型。

26.8 测试评价

为评价模型对训练时未见输入的表现，首先还需下载一些数据，下个单元负责这一点。`read_single_file()` 的 `scale_factor=0.25` 确保得到与训练集一样的 32×32 场。不过，测试集的 Reynolds

数未在训练中出现，可检查泛化。加载数据时同时计算真实均值与标准差，保存在 `mean_field_test_gd` 与 `std_field_test_gd`，供之后量化差异。

```
df_test=FileDataFiles("./datasets/1_parameter/test_cases.txt",base_path="./datasets/1」
↪  _parameter/data/")
df_test.sort()
std_field_test_gd=[]
mean_field_test_gd=[]
inputs_test=[]
samples_gd=[]
for case in df_test.get_simulation_cases():
    datas=[]
    selected_cases=df_test.select_simulation_cases([case])
    for case in selected_cases:
        raw_data=read_single_file(case['path']+case['file_name'],model="dimless",scale」
        ↪  _factor=0.25)
        datas.append(
            raw_data[3:]
            )
    # scale factor is 0.25 to get 32x32 data
    inputs_test.append(read_single_file(case['path']+case['file_name'],model="normaliz」
    ↪  ed",scale_factor=0.25)[0:3])
    samples_gd.append(np.stack(datas,axis=0))
    std_field_test_gd.append(samples_gd[-1].std(axis=0))
    mean_field_test_gd.append(samples_gd[-1].mean(axis=0))
std_field_test_gd=np.stack(std_field_test_gd,axis=0)
mean_field_test_gd=np.stack(mean_field_test_gd,axis=0)
df_all=DataFiles(df_train.case_list+df_test.case_list)
df_all.sort()
std_value_gd=[]
for case in df_all.get_simulation_cases():
    datas=[]
    selected_cases=df_all.select_simulation_cases([case])
    for case in selected_cases:
        datas.append(
            read_single_file(case['path']+case['file_name'],model="dimless",scale_fact」
            ↪  or=0.25)[3:]
            )
    std_value_gd.append(np.stack(datas,axis=0).std(axis=0).mean())
```

下个单元定义辅助函数，对训练后的 FM 模型按选定步数进行 Euler 积分，计算

$$x_1 = x_0 + \int_0^1 v_\theta(x,t)\,\mathrm{d}t.$$

每批 25 个样本，主函数 `sample_flowmatching()` 将较大输入拆成 25 个一组。当然，也可尝试更先进的 ODE 方法，配套 GitHub 仓库提供多种积分器。

需明确一个不易从公式看出的差别：去噪过程中会反复重新加噪；流匹配只使用初始噪声，随后沿学习向量场指定的轨迹前进。因此，流匹配推断的各步骤不再加入新噪声。

```
def integrate_euler( f, x_0, t_0, t_1, dt):
    t_0 = torch.as_tensor(t_0, dtype=x_0.dtype, device=x_0.device)
    t_1 = torch.as_tensor(t_1, dtype=x_0.dtype, device=x_0.device)
    dt = torch.as_tensor(dt, dtype=x_0.dtype, device=x_0.device)
    with torch.no_grad():
        t=t_0
        x=x_0
        while (t_1 - t) > 0:
            dt = torch.min(abs(dt), abs(t_1 - t))
            x, t = x + dt * f(t,x), t + dt
    return x
def fm_sample( network, x_0, dt, condition):
    with torch.no_grad():
        def wrapper(t,x):
            return network(x,
                        t*torch.ones((x.shape[0],)).to(x_0.device),
                        condition=condition)
        return integrate_euler( f=wrapper, x_0=x_0, t_0=0., t_1=1., dt=dt)
def sample_flowmatching(network, input_field, dt, num_sample=100):
    network.eval();network.to(device);predictions=[]
    batch_size=25;N_all=num_sample
    while N_all>0:
        batch_size_now=min(batch_size,N_all)
        N_all-=batch_size
        condition=input_field.to(device).repeat(batch_size_now,1,1,1)
        noise=torch.randn_like(condition)
        prediction_batch=normalized2dimless(
            fm_sample(x_0=noise,
                      network=network, dt=dt,
                      condition=condition)
            )
        predictions.append(prediction_batch.detach().cpu().numpy())
    predictions=np.concatenate(predictions,axis=0)
    return np.mean(predictions,axis=0), np.std(predictions,axis=0), predictions
```

直接改变积分步数测试 FM。正如前述，FM 可用很少步骤生成结果，因此这是值得改变的超参数。下面通过 `sample_flowmatching()` 收集 1 到 100 步的结果。为定性比较，只考虑一个测试输入，将结果与真实值并排显示。每个步数用不同初始噪声生成 100 个样本，统计均值与标准差。

```
index=3
input_field=inputs_test[index].unsqueeze(0)
titles=[]
result_fms=[]
for step in tqdm([1,5,20,100]):
    mean_fm,std_fm,samples_fm=sample_flowmatching(network,input_field,dt=1/step)
    titles.append("Flow Matching {}".format(step))
    result_fms.append(np.concatenate([mean_fm,std_fm],axis=0))
```

原书进度输出：

```
100%|██████████| 4/4 [00:05<00:00, 1.36s/it]
```

随后重复这一过程，定义扩散模型的采样辅助函数。`DDPM_sample_step()` 会按当前日程重新加入适当噪声。

```
def diffusion_sample_from_noise(diffuser, model, condition):
    with torch.no_grad():
        x_t=torch.randn_like(condition)
        t_now = torch.tensor([diffuser.steps],
        ↪  device=diffuser.device).repeat(x_t.shape[0])
        t_pre = t_now-1
        for t in range(diffuser.steps):
            predicted_noise=model(x_t,t_now,condition)
            x_t=DDPM_sample_step(diffuser, x_t,t_now,t_pre,predicted_noise)
            t_now=t_pre
            t_pre=t_pre-1
        return x_t
def DDPM_sample_step(d, x_t, t, t_pre, noise):
    coef1 = 1/d.sqrt_alphas[t]
    coef2 = d.betas[t]/d.sqrt_one_minus_alphas_bar[t]
    sig = torch.sqrt(d.betas[t]) *d.sqrt_one_minus_alphas_bar[t_pre]
    ↪  /d.sqrt_one_minus_alphas_bar[t]
    return coef1*(x_t-coef2*noise) + sig*torch.randn_like(x_t)
```

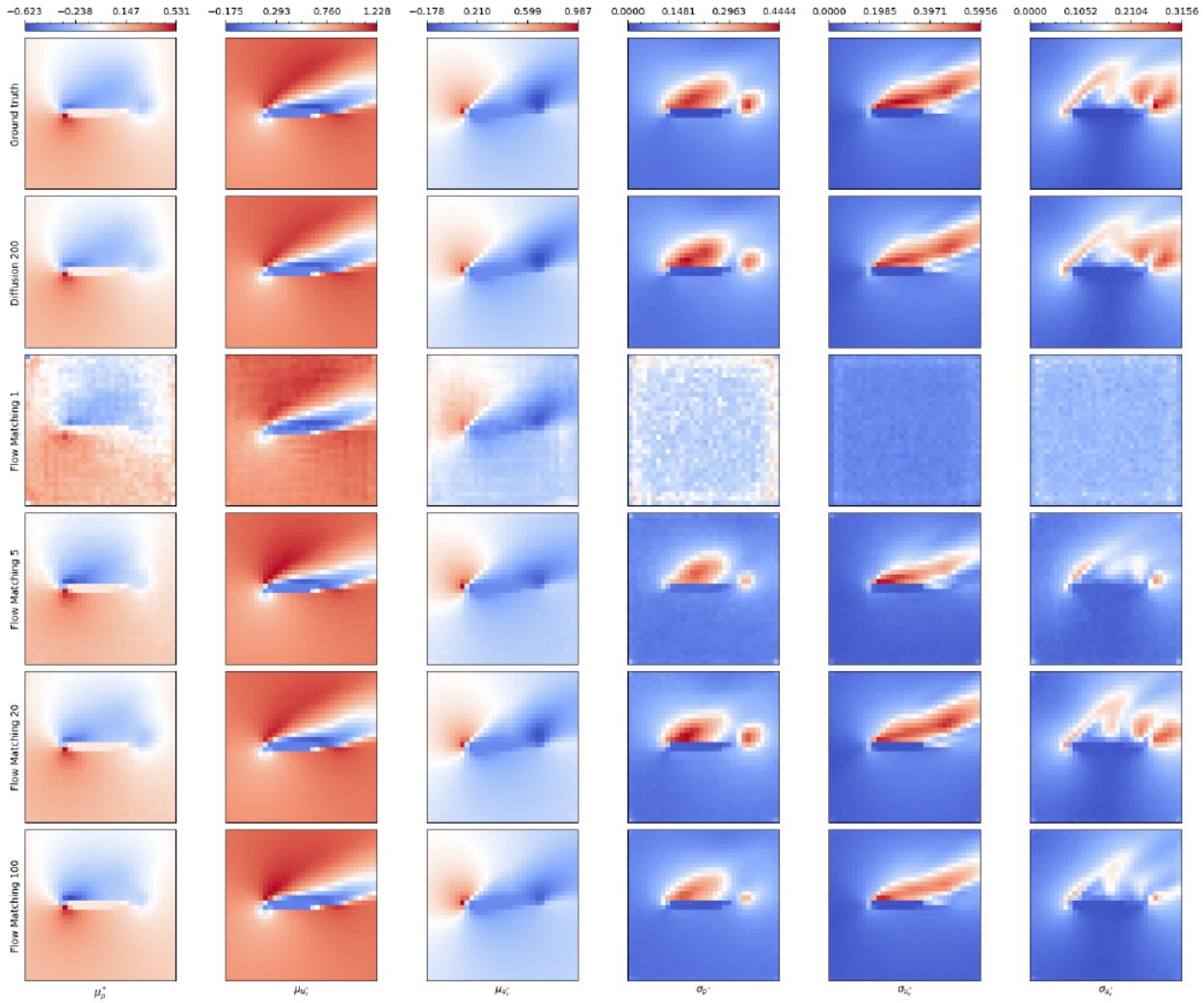
这些片段紧随原始翼型论文的采样，例如 `sample.ipynb`。下方计算单批扩散输出，保存在 `result_diffusion`。

```
def sample_diffusion(diffuser, network,input_field,num_diffusion_sample=100):
    network.eval();network.to(device);predictions=[]
    batch_size=25;N_all=num_diffusion_sample
    while N_all>0:
        batch_size_now=min(batch_size,N_all)
        N_all-=batch_size
        prediction_batch=normalized2dimless(
            diffusion_sample_from_noise(diffuser, network,
                                        input_field.to(device).repeat(batch_size_now,1,1,1)
                                        ↪  ))
        predictions.append(prediction_batch.detach().cpu().numpy())
    predictions=np.concatenate(predictions,axis=0)
    return np.mean(predictions,axis=0),np.std(predictions,axis=0),predictions
mean,std,samples_diffusion=sample_diffusion(diffusion_trainer.diffuser,dif_network,in」
↪  put_field)
result_diffusion=np.concatenate([mean,std],axis=0)
```

先画出单个测试例子的均值与标准差场，作定性比较。

```
result_ground_truth=np.concatenate([mean_field_test_gd[index],std_field_test_gd[index」
↪  ]],axis=0)
```

```
CHANNEL_NAME_MEAN=[r"$\mu_p^*$",r"$\mu_{u_x^*}$",r"$\mu_{u_y^*}$"]
CHANNEL_NAME_STD=[r"$\sigma_{p^*}$",r"$\sigma_{u_x^*}$",r"$\sigma_{u_y^*}$"]
from airfoil_diffusion.plotter import *
show_each_channel([result_ground_truth,result_diffusion]+result_fms,
                  channel_names=CHANNEL_NAME_MEAN+CHANNEL_NAME_STD,
                  case_names=["Ground truth","Diffusion
                  ↪  200"]+titles,transpose=True,inverse_y=True)
```



原书输出：行依次为真实值、200 步扩散、1/5/20/100 步 FM；列依次为无量纲压力、两速度分量的均值及相应标准差。各列色条与全部数值按源保留。

上面每种方法生成 100 个样本，图中显示规则网格每个空间位置的均值与标准差。所有方法都较容易得到正确均值：相应场变化不大，即使 1 步 FM 也基本正确，只带一点噪声。

跨样本标准差更困难：1 步 FM 在此完全失败，20 步 FM 却已很好。它的步数仅为 200 步 DDPM 的十分之一，因此源文称 FM 实际上快 10 倍，同时精度相当。[4]

[4]译注：这里直接确定的是网络步数比；并非本次硬件独立测得 10 倍墙钟加速。图是原书有限样本比较，未在本译稿复现实验。

26.9 量化结果

此前只看一个测试例子，可能恰好有利于 FM。因此，以下在不同 Reynolds 数的多个例子上重复评价。测试集共六个 Reynolds 数，中间四个是训练参数间的插值，首尾两个是外推。

辅助函数 `do_test()` 计算给定网络在完整测试集上的统计量。

```
def do_test(sample_func):
    mean_predictions=[]
    std_predictions=[]
    std_a_predictions=[]
    for input_field in tqdm(inputs_test):
        mean_fields,std_fields,_=sample_func(input_field.unsqueeze(0))
        mean_predictions.append(mean_fields)
        std_predictions.append(std_fields)
        std_a_predictions.append(np.mean(std_fields))
    return mean_predictions,std_predictions,std_a_predictions
```

由于测试例子较多，以下单元可能耗时较长，尤其 200 步扩散模型。

```
std_a_fms=[]
labels=[]
for step in [1,5,20,100]:
    _,_,std_a_fm_i = do_test(lambda x:sample_flowmatching( network, x, dt=1/step,
    ↪  num_sample=500))
    std_a_fms.append(std_a_fm_i)
    labels.append("Flow Matching {}".format(step))
```

原书四种 FM 步数的进度输出：

```
100%|██████████| 6/6 [00:01<00:00, 4.35it/s]
100%|██████████| 6/6 [00:06<00:00, 1.08s/it]
100%|██████████| 6/6 [00:25<00:00, 4.28s/it]
100%|██████████| 6/6 [02:09<00:00, 21.59s/it]
```

```
_,_,std_a_predictions_dif = do_test(lambda
↪  x:sample_diffusion(diffusion_trainer.diffuser, dif_network, x,
↪  num_diffusion_sample=500))
```

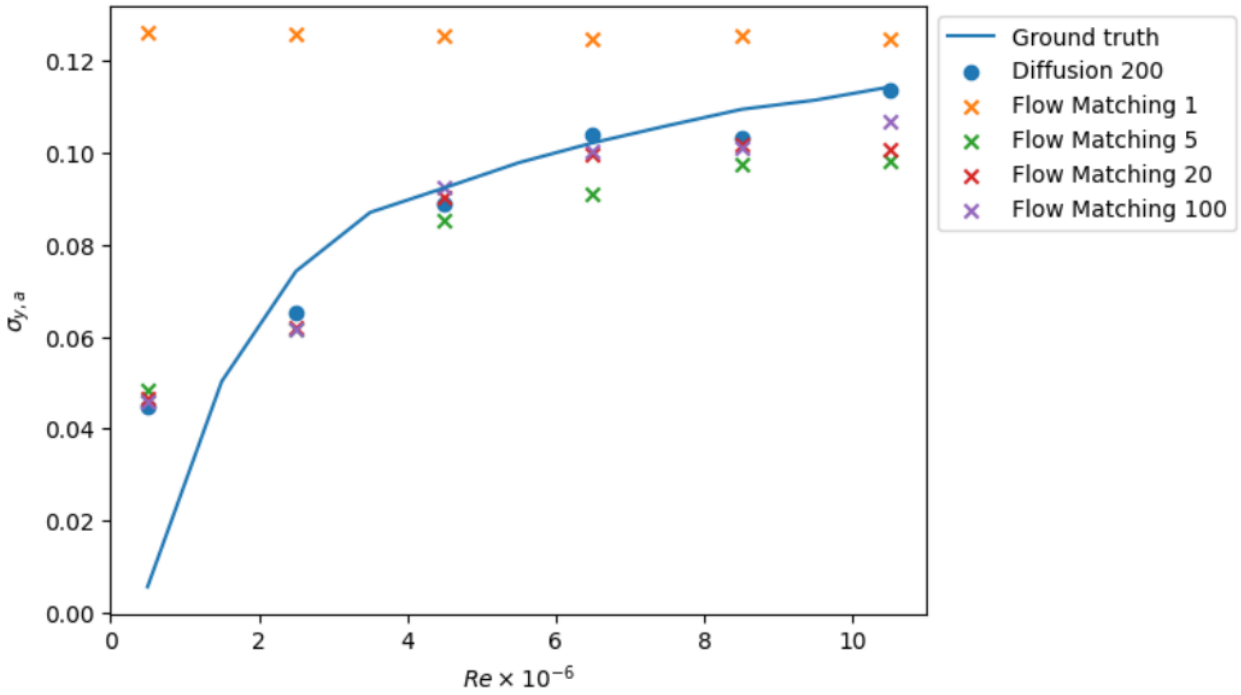
原书扩散模型进度输出：

```
100%|██████████| 6/6 [03:40<00:00, 36.68s/it]
```

现在得到全部数值。下个单元将每个 Reynolds 数的真实标准差（蓝线），与各个训练后 NN 的估计并排绘图。

```
x=[0.5 +2*i for i in range(6)]
plt.plot([0.5 +i for i in range(11)],std_value_gd,label="Ground truth")
plt.scatter(x,std_a_predictions_dif,label="Diffusion 200",marker="o")
for i in range(len(std_a_fms)):
    plt.scatter(x,std_a_fms[i],label=labels[i],marker="x")
```

```
plt.legend(bbox_to_anchor=(1.0,1.0))
plt.xlabel(r"$Re \times 10^{-6}$")
plt.ylabel(r"$\sigma_{y,a}$")
plt.show()
```



横轴 $Re \times 10^{-6}$，纵轴 $\sigma_{y,a}$；蓝线真实值，蓝点 200 步扩散，橙/绿/红/紫叉依次为 1/5/20/100 步 FM。

这幅图表明，上面的单例并非异常：1 步 FM 不好，但 5 步或 20 步已很好，大体与 DDPM 相当。总体 NN 精度很高，首尾外推点也合理捕捉。低 Re 时，网络高估方差，因为训练数据中几乎没见过静态例子；右侧可能更困难的高 Re 情形则处理得很好。[5]

总体而言，这不是简单任务：网络需要学习从几乎不变到强烈变化、跨不同 Reynolds 数的后验分布。流匹配 NN 表现突出，只有很少积分步，就能给出良好估计与后验样本。这对实际应用很重要：这些代理模型必须与经典数值仿真竞争，而许多领域已有高度优化的仿真代码。因此，训练后 NN 必须在合理硬件要求下快速提供估计，不能依赖过大的参数数量。这些结果表明，流匹配是概率仿真的有力候选。

26.10 下一步

- 在此设置中尝试高阶积分方法。相对 Euler，是否有改进？务必统计积分器的所有 NFE，例如 RK4 每步的四次 NN 调用。

- 增加训练轮数及 U-net 特征数，提高整体精度。

[5]译注：图和代码统计的是标准差，而本句原文称方差，按原措辞保留并注明；关于未见静态情形的因果解释属于原书说明，非由本译稿另作控制实验确认。

- 上面采用基本去噪与流匹配，可试改进方案，例如 Liu 等论文中的额外整流步骤，可能进一步减少所需步数。

这些均为原书后续实验建议，未在本译稿执行。[LCBH+22]、[LGL22] 完整书目待整书文献接入；冻结源物理 268 页为空白正文页。

译注：独立核对的适用条件 这里主要验证有限 RANS 样本的均值和逐格标准差，最终还对空间/通道平均；相符不足以证明完整条件分布、尾部或空间相关准确。cosine 日程代码将 β 截至 0.999，有限终点非严格标准 Gaussian；FM 的 $\sigma_{\min} = .001$ 给终点 $x_1 + .001x_0$，仍含随机残留。

第二十七章　纳入物理约束

尽管前面讨论的扩散与流网络具有强大的生成建模能力，训练时网络、观测与样本之间仍没有直接反馈回路。因此，没有直接机制引入 PDE 先验等物理约束。仅靠学习很难生成高精度样本，而科学应用往往希望将误差降到任意选定阈值。

本章概述弥补这一不足的策略。承接前章，两种方法的核心目标都是将可微仿真重新纳入训练与推断回路。前面已展示其能力，主要问题是在扩散建模中如何最有效地使用。

注：回到逆问题　以下聚焦 “概率学习导论” 中的逆问题。系统为 $y = f(x)$，数值仿真器为 $y = \mathcal{P}(x)$。给定观测 y，希望得到逆问题分布解 $x \sim p(x \mid y)$ 的后验。

27.1　引导扩散模型

有了可微物理仿真 $\mathcal{P}(x) = y$，就能获得穿过仿真器的梯度 ∇_x。仍以逆问题为目标：给定输出 y，从条件后验 $p(x \mid y)$ 采样，得到能够解释 y 的 x。前章展示了扩散模型学习这类分布；给定物理先验 $\mathcal{P}$ 后，首先要作一个根本选择：训练时使用梯度，改善学习分布 p_θ；还是推断时使用梯度，改善采样 $x \sim p_\theta(x \mid y)$?

用物理先验训练　训练时将基于物理的信号作为梯度引入，希望改善训练后的 p_θ。例如可能弥补训练数据稀疏，但要显著提高学习分布精度，希望不大。扩散与流匹配训练通常已能得到非常有力的网络，善于生成后验近似样本。精度往往受模型与数据规模限制，在此阶段很难从根本上提高模型能力。因此，更值得关注的是推断时提高精度。

用物理先验推断　经典仿真通常提供精度控制。例如线性方程组的迭代求解器有迭代次数与残差阈值；若解不够准，就降低残差阈值。NN 通常没有这样的控制，即使去噪或流匹配速度积分的迭代次数，也受到最终精度限制：多做几步通常减少噪声和误差，但最终会停在训练模型能力决定的平台。物理求解器梯度恰在此有前景：外部过程可以引导并改善扩散输出。下面会展示如何突破纯学习的精度，得到真正优于传统求解器的逆问题方法。[1]

回顾去噪：训练噪声估计器 ϵ_θ，推断时迭代

$$x_{\text{new}} = x - \widehat{\alpha}_t \epsilon_\theta(x, t) + \widehat{\sigma}_t \mathcal{N}(0, I),$$

[1] 译注：本节有关训练作用与求解器比较的判断均为作者的方法动机，不作对所有模型、数据或离散化的普遍保证。经典残差阈值本身也不消除建模与离散化误差。

其中 $\widehat{\alpha},\widehat{\sigma}$ 汇总两项的缩放因子。加入梯度最直接的方式，是再沿 $\nabla_x\|\mathcal{P}(x)-y\|^2$ 作一步；这里为简化，使用到观测 y 的 L^2 距离。即使后验不是条件分布，只能访问 $x\sim p_\theta(x)$，这种方法也能引导采样，称为扩散后验采样（DPS）[CKM+23]。

尽管纳入了 $\mathcal{P}$，仍有两个困难：x 通常嘈杂，梯度步可能扭曲去噪采样的分布。第一点可用后述外推步骤较容易处理，第二点更难：通过 $\nabla_x\mathcal{P}$ 作梯度下降，类似经典逆问题优化，可能强烈扭曲扩散输出。最坏时，会把后验中不同点拉向仿真器偏爱的单个情形。因此，下文介绍合并仿真与学习、同时保持后验分布的策略。先讨论先进的流匹配，再讨论将扩散步骤本身视为物理过程的变体。

27.2 物理引导的流匹配

遵循 [HT23]，将仿真器控制信号重新引入流匹配。目标是用灵活控制信号改造已预训练的流网络，将学习流与控制信号合成为受控流。由第二个网络，即控制网络，完成这一任务，避免仿真信号损害后验分布。控制网络比预训练流网络小，冻结预训练权重效果很好，因此只需较少附加参数与计算进行控制微调。

若仿真器可微，控制可来自代价函数及梯度；也可直接从仿真输出学习。下文将展示，仿真反馈带来的性能改进很大，不能只靠更大训练集获得。具体地，在引力透镜问题中，带仿真反馈的 FM 在精度上可与 MCMC 基线竞争，推断时间则显著更优，是实际应用中有吸引力的工具。

受控流 v_θ^C　先预训练不带控制信号的普通条件流网络 $v_\theta(x,y,t)$，确保达到纯学习可实现的最佳性能。第二阶段引入控制网络 $v_\theta^C(v,c,t)$，输入预训练流 v 与控制 c，例如利用 PDE 梯度，输出改善的 FM 速度。推断时仍积分 $\mathrm{d}x/\mathrm{d}t=v_\theta^C(v,c,t)$，只是每步要先计算 $v_\theta(x,y,t)$ 和 c。

控制网络仅约为普通流网络权重的 10%。可冻结 v_θ，用式 (25.2) 的条件流损失再训练少量步骤；因无需穿过 $v_\theta(x,y,t)$ 反向传播，时间与计算减少。冻结通常不损害性能，但联合端到端训练可能进一步改善。

译注　原图 27.1 使用 $v_\phi,\widehat{\theta}_1,x_o$ 表示流、预测参数、观测，正文用 $v_\theta,\widehat{x}_1,y$。概览保留源图符号；后续控制类型图为与公式对照，明确标明观测 $x_o=y$。

一步预测　条件 FM 网络 $v_\theta(x,y,t)$ 通过逐步积分 $\mathrm{d}x_t/\mathrm{d}t=v_\theta(x_t,y,t)$，将 p_0 样本变到 p_1。当前轨迹点 x_t、观测 y 与物理模型之间尚无直接反馈。第一个问题是：x_t 常常远非良好后验样本 x_1 的估计，尤其推断初期 x_0 来自通常为 Gaussian 的源分布，噪声很大。多数仿真器不适应极嘈杂输入，再在其上计算梯度显然不妥。

将 x_t 沿时间前向外推，得到估计

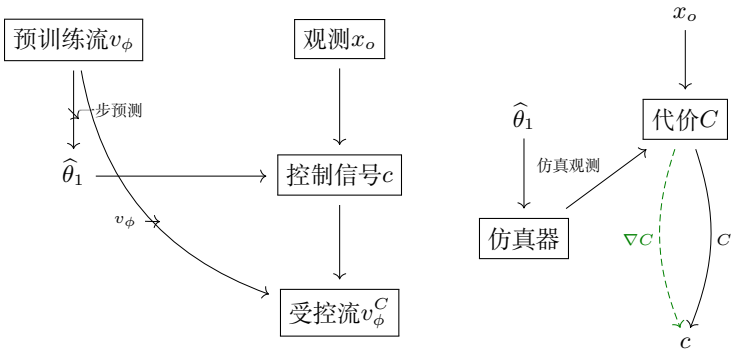
$$\widehat{x}_1=x_t+(1-t)v_\theta(x_t,y,t). \tag{27.1}$$

然后在 $\widehat{x}_1$ 上控制和引导，替代当前可能嘈杂的 x_1。[2]

一步预测与扩散采样中的似然引导也有概念联系。扩散模型基于条件得分 $\nabla_{x_t}\log p(x_t\mid y)$，可分解为

$$\nabla_{x_t}\log p(x_t\mid y)=\nabla_{x_t}\log p(x_t)+\nabla_{x_t}\log p(y\mid x_t). \tag{27.1}$$

[2]译注：本句末尾源文写 x_1，上下文的当前轨迹点则为 x_t，两者差异保留说明。



虚线：梯度控制信息；斜杠：停止梯度。

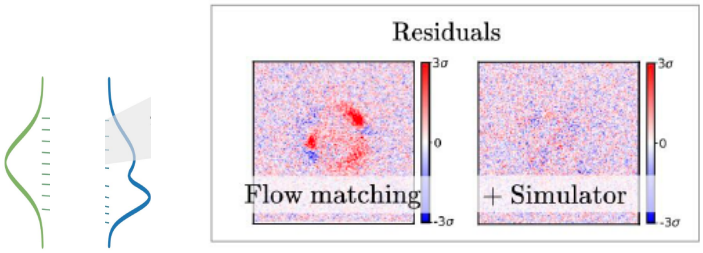


图 27.1: 控制框架概览。预训练流预测轨迹点 x_t 的流，估计 $\widehat{x}_1$；可微仿真器与代价 C 给出控制，再由附加网络合成新受控流。通过合适控制，作者报告可避开局部最优，以低推断时间取得高精度样本。原图 27.1；节点、箭头、停止梯度与梯度标记可编辑，残差小图按源保存。

第一项可由预训练扩散网络估计；后一项通常难算，但可近似为

$$p(y \mid x_t) \approx p_{y|x_0}(y \mid \widehat{x}(x_t)), \qquad \widehat{x}(x_t) = \mathbb{E}_q[x_0 \mid x_t].$$

去噪估计通常通过 Tweedie 公式 $(\mathbb{E}_q[x_0 \mid x_t] - x_t)/(t\sigma^2)$ 获得。实践中，x_t 仍嘈杂时，$\widehat{x}(x_t)$ 很差，使早期推断困难。相比之下，基于线性条件输运路径的流，在经验上曲率比去噪网络更小，因而用更少步骤推断，并提供更好的 $\widehat{x}_1$。

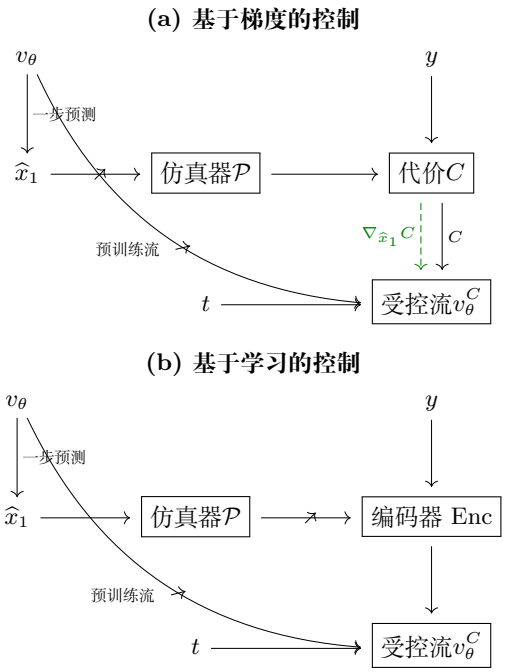
译注 源书将上述两个不同公式都编号为 27.1，重复编号照录，并用不同内部 label 定位。所写 Tweedie 分式是带特定噪声标度的得分关系形式，不是 $\widehat{x}$ 本身的一般直接公式；源文省去等式另一侧及具体噪声条件，不能作为完整恒等式独立使用。

27.2.1 基于物理的控制

现在讨论控制信号 c 的内容：用物理信号扩展自条件化，使网络输出与底层物理先验之间增加反馈。区分两类控制：可微仿真器的梯度控制，以及学习估计网络的控制。

基于梯度的控制信号 第一种情况，使用可微仿真器 $\mathcal{P}$ 构建可微代价函数 C，从而计算预测解的梯度。依靠扩散/FM 的随机性，仿真器本身可以是确定性的。

给定观测 y 与一步预测 $\widehat{x}_1$，控制由代价 C 判断该预测能多好地解释 y。例如可用 L^2 损失或似然



源图观测记为 x_o，此处用正文 y；编码器前斜杠保留停止梯度语义。

图 27.2: 两类控制信号。(a) 来自可微仿真器；(b) 来自学习编码器。原图 27.2；数据依赖、附加时间输入、预训练流分支、梯度及停止梯度均可编辑。

$p(y \mid \widehat{x}_1)$。定义控制的两个分量：代价本身，以及代价梯度，

$$c(\widehat{x}_1, y) := [C(\mathcal{P}(\widehat{x}_1), y); \nabla_{\widehat{x}_1} C(\mathcal{P}(\widehat{x}_1), y)]. \tag{27.2}$$

信息传给网络后，网络可自由利用当前到目标的距离，即 C 的值，以及梯度提供的降低代价方向信息。

基于学习的控制信号 仿真器不可微时，第二种学习估计器派上用场。引入参数为 θ_E 的编码器 Enc，结合仿真输出与观测 y，判断两者相似性。编码器输出很小，为 $O(\dim(x))$，控制定义为

$$c(\widehat{x}_1, y) := \mathrm{Enc}(\mathcal{P}(\widehat{x}_1), y). \tag{27.3}$$

如图 27.2，在仿真器 $\mathcal{P}$ 输出处停止梯度反向传播。展示两类控制效果前，先讨论其性质。

27.2.2 其他考虑

随机仿真器 许多 Bayes 推断问题的仿真器带随机性。为简化，假定所有随机性都能由附加输入 $z \sim \mathcal{N}(0, I)$ 控制。利用期望与梯度交换的等价性

$$\nabla_{\widehat{x}_1} \mathbb{E}_{z\sim\mathcal{N}(0,1)}[C(\mathcal{P}_z(\widehat{x}_1), y)] = \mathbb{E}_{z\sim\mathcal{N}(0,1)}[\nabla_{\widehat{x}_1} C(\mathcal{P}_z(\widehat{x}_1), y)], \tag{27.4}$$

调用仿真器时抽取 z。训练中每个样本、每一步都重新随机抽取；推断时，每条轨迹的 z 保持固定。[3]

时间依赖 若 $\widehat{x}_1$ 很差，代价 $C(\widehat{x}_1, y)$ 很高，梯度与控制可能不可靠。FM 较晚时刻的估计会更可靠。实践中 $t \geq 0.8$ 是良好阈值，因此仅在该区间训练控制网络 v_θ^C，集中利用更有用的控制，例如用可微仿真的准确梯度微调解。$t < 0.8$ 时，直接输出预训练流 $v_\theta(t, x, y)$。

理论正确性 上式中的近似 $\widehat{x}_1$ 只影响控制信号，而控制是受控流网络 v_θ^C 的输入。若仿真器确定，控制就是 x_t 的函数。受控网络与普通 FM 采用同一损失训练，因此理论性质得以保持。相比之下，基于似然的引导，在推断时将近似 $\nabla_{x_t} \log p(y \mid x_t)$ 加为引导项，不在原始 FM 理论覆盖范围。[4]

27.2.3 天体物理例子

为展示物理求解器引导如何影响样本和后验精度，考虑强引力透镜：这是需精确后验以准确建模观测的困难天体物理逆问题。在星系尺度强透镜中，源星系光线被源与观测者之间星系的引力势偏折，形成源的多重像。传统方法建模单个系统需数分钟到数小时、数天，迫切需要学习方法减少计算与推断。该实验展示，FM 结合仿真反馈控制，可以获得与 MCMC 相竞争的透镜建模后验，推断则更快。

图 27.3 显示一次重建及残差。FM 与物理版本都很准，肉眼很难区分，但纯 FM 只与经典逆求解器相当；引入仿真后，精度显著改善，即使经典方法也很难做到。右侧量化表中，最佳经典基线 AIES 的平均 χ^2 为 1.74，FM 加仿真器为 1.48。考虑到带噪观测在本场景下最佳可能结果为 1.17，后者精度很高。

建模时间方面，FM 明显快于经典求解器。仿真器将纯 NN 的 10 秒增至 19 秒，而经典基线重建需 50 倍以上时间。DPS 这类 “简单” 物理组合则未能在精度上赶上经典方法。总体上，FM 加仿真器不仅更准，还比这里最佳经典求解器 AIES 快约 35 倍。源文称代码 “即将在此仓库提供”。[5]

物理 FM 的优缺点如下。

优点

- 比纯学习扩散模型更精确。
- 可以控制残差精度。
- 相比传统逆求解器，运行时间减少。

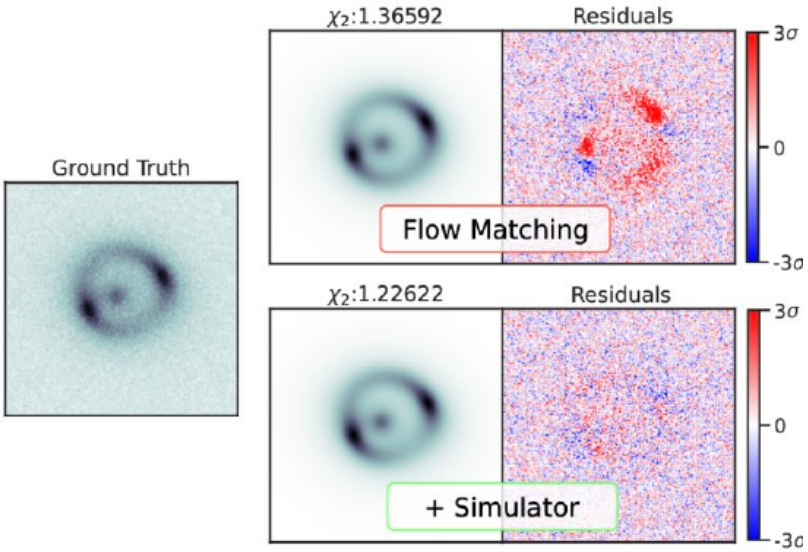
缺点

- 需要可微物理过程。
- 增加计算资源。

[3]译注：期望与导数交换需适当正则性/可积条件，不对任意随机仿真器无条件成立。源公式用 $\mathcal{N}(0,1)$，前文向量输入用 $\mathcal{N}(0, I)$，均保留。

[4]译注：此处保留作者针对所述构造的理论论断；有限数据、近似训练、阈值与数值积分下的误差，仍需单独验证，不能由同名损失自动宣布后验精确。

[5]译注：时间与精度为原书记录，未重新测试。表中 564/10 与 672/10 均超过 50，而 564/19 约 29.7、672/19 约 35.4；因此 “超过 50 倍” 比较的是纯 FM 的 10 秒，不能解释成全部相对物理版 19 秒。代码 “即将” 仅指源书写作时，未猜测没有明示的仓库地址。



方法	平均 $\chi_2 \downarrow$	建模时间 $\downarrow$
NUTS	1.83	$\sim 56\times$（564 秒）
AIES	1.74	$\sim 67\times$（672 秒）
DPS	9.98	$\sim 42\times$（427 秒）
FM	1.83	$1\times$（10 秒）
FM+ 仿真器	1.48	$\sim 2\times$（19 秒）

图 27.3: 引力透镜重建结果。图中左为真实观测，右上为 FM 及残差，右下为 FM 加可微仿真及残差；源单例标注 $\chi_2 = 1.36592, 1.22622$。表为原图右侧整体比较，平均值与单例值不可混同。原图 27.3，表完整可编辑。

译注 第一条缺点按源列表保留，但前面已描述非可微仿真器的学习编码控制，不能据此断言本章所有变体都需可微仿真器。

27.3 结合可微物理的得分匹配

此前，去噪/FM 的扩散时间完全是虚拟过程，独立于正向与逆问题所代表的物理时间。这是最通用的视角，如上所示效果很好。不过，另一选择也值得思考：合并两过程，将扩散视为物理系统的内在组成。

以下遵循论文 “Solving Inverse Physics Problems with Score Matching”[HVT23]，介绍这种组合方法。源文另有代码仓库链接。

该方法利用得分匹配求解物理逆问题，结合近似逆物理仿真器与学习修正函数，逐步将当前状态向后推进。核心认识是：以单步损失训练修正，等价于得分匹配目标；训练时递归预测更长轨迹，则与相应概率流的最大似然训练相关。所得逆求解器精度和时间稳定性良好；与扩散建模一致，区别于经典学习求解器，它能采样解的后验。下文简称 SMDP，即结合可微物理的得分匹配。

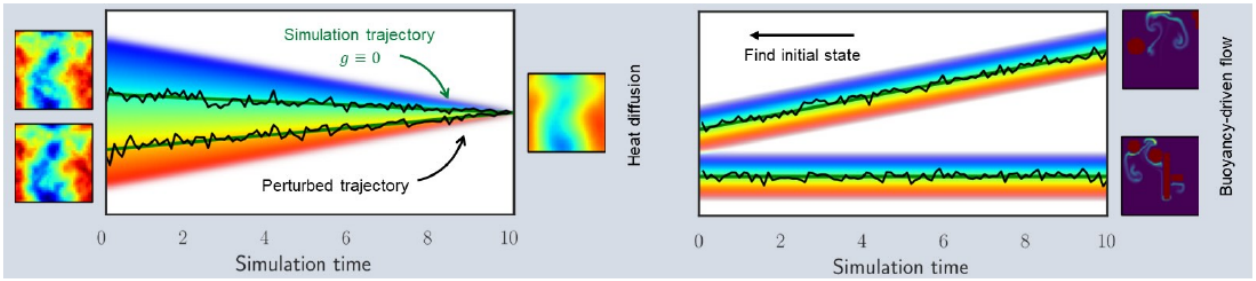
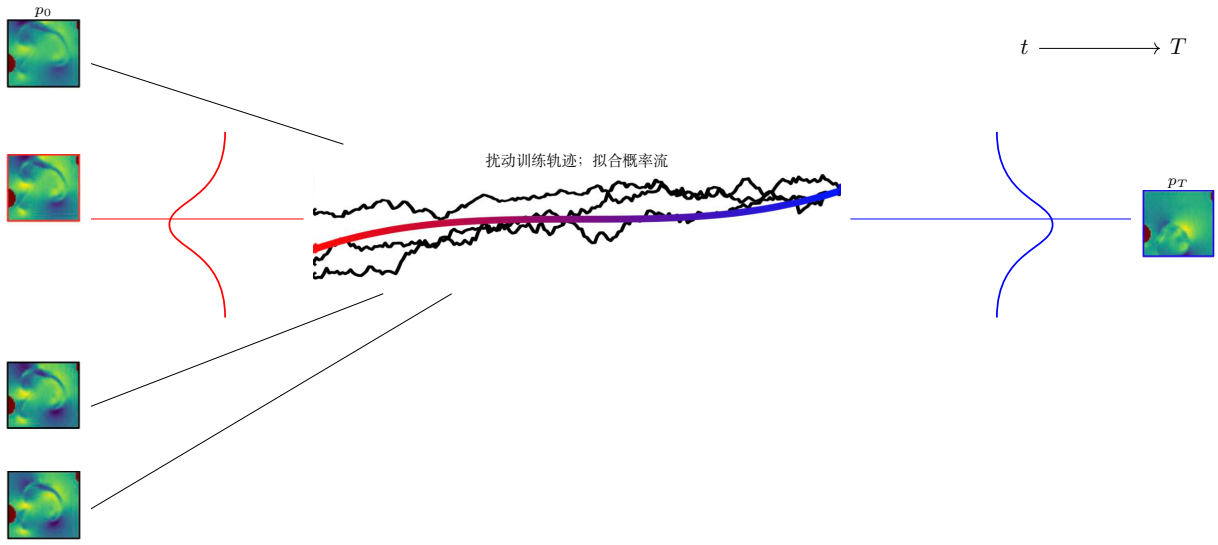


图 27.4: 物理过程（左，以热扩散为例）扰动并 “破坏” 初态；左图绿色为 $g = 0$ 仿真轨迹，黑线为扰动轨迹。推断时（右，以浮力流为例），结合得分步骤与求解器梯度，计算逆步骤并找初态；横轴均为仿真时间。原图 27.4。

27.3.1 SMDP 的训练与推断

训练时，SMDP 将神经 ODE，即概率流，拟合到扰动训练轨迹。概率流由近似逆物理仿真器 $\widetilde{\mathcal{P}}^{-1}$ 和修正函数 s_θ 组成。推断时，将逆仿真器、训练后的 s_θ 及每步 Gaussian 噪声结合，从 $\mathbf{x}_T$ 向后模拟到 $\mathbf{x}_0$。优化 s_θ 时，沿训练轨迹移动长度为 S 的滑动窗口，重建当前窗口，累积关于 θ 的梯度，并穿过所有预测步反向传播。图 27.5 展示这一过程。

译注 原图注末尾的数学标记未渲染，显示字面 `$\matcal{P}^{-1}.`，这里不据错误宏生成新对象。图内概率流式显示 $\mathrm{d}\mathbf{x} = [-\widetilde{\mathcal{P}}^{-1}(\mathbf{x}) - s_\theta(\mathbf{x}, t)]\mathrm{d}t$，与下面训练更新及推断式的符号不同，全部保留；这些显示式不足以验证采用了同一概率流约定，详见本章末适用条件注。



$$\longleftrightarrow \mathrm{d}\mathbf{x} = \big[-\boxed{\widetilde{\mathcal{P}}^{-1}(\mathbf{x})} - \boxed{s_\theta(\mathbf{x},t)}\big]\,\mathrm{d}t$$

$\widetilde{\mathcal{P}}^{-1}$：逆物理步；$s_\theta$：修正函数。

图 27.5: 纳入物理求解器及其近似逆求解器的得分匹配训练。扰动训练轨迹从边缘密度 p_0 到 p_T；彩色曲线拟合概率流，底部标明逆物理步与修正函数。原图 27.5，科学场图与连续轨迹取自源图，关系层和可见源式原生重建；保留源负号冲突。

近似逆步可用可微求解器或学习代理模型。参数为 θ 的网络 $s_\theta(\mathbf{x},t)$ 训练为满足

$$\mathbf{x}_m \approx \mathbf{x}_{m+1} + \Delta t[\widetilde{\mathcal{P}}^{-1}(\mathbf{x}_{m+1}) + s_\theta(\mathbf{x}_{m+1}, t_{m+1})].$$

修正项处理近似误差与随机外力 $F_{t_m}(z_m)$ 的不确定性。训练时可展开多步，改善精度与稳定性。推断时，用 Euler–Maruyama 方法积分 SDE

$$\mathrm{d}\mathbf{x} = [-\widetilde{\mathcal{P}}^{-1}(\mathbf{x}) + C\, s_\theta(\mathbf{x},t)]\,\mathrm{d}t + g(t)\,\mathrm{d}W,$$

得到逆问题解。取 $C=1$ 并去掉噪声，即得到概率流 ODE：唯一的确定性解。原文指出，这个确定性变体不再具有概率性，但有其他有趣性质。[6]

27.3.2 SMDP 实例

考虑随机热方程实验。热方程 $\partial u/\partial t = \alpha\Delta u$ 在许多物理系统中起基本作用；这里轻微扰动热扩散，加入 $g(t)\xi$，其中 ξ 为空间时间白噪声。取扩散率 $\alpha=1$，在 $t=0$ 从 Gaussian 随机场采样初态，参数 $n=4$，分辨率 32×32。用固定步长、$g\equiv0.1$ 的谱求解器 $\mathcal{P}_h$ 和 Euler–Maruyama 方法，从 $t=0$ 模拟到 0.2。给定末态 $\mathbf{x}_T$，希望恢复可能的初态 $\mathbf{x}_0$。

本例不能直接用正向求解器推断初态，因为噪声中的高频会被放大，产生不合理结果。近似逆步改用正向求解器步实现，即 $\widetilde{\mathcal{P}}^{-1}(\mathbf{x}) \approx -\mathcal{P}_h(\mathbf{x})$。

[6]译注：确定性指给定末态/初值的轨迹演化；若输入本身随机，ODE 推送的样本整体仍可形成分布。源文“不再具有概率性”按原判断保留，不能解释为概率流 ODE 不能输运密度。

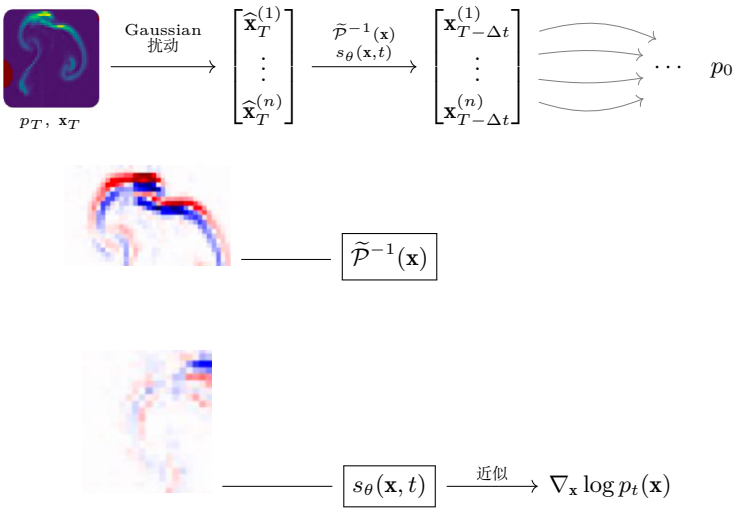


图 27.6: SMDP 推断概览。末态经 Gaussian 扰动产生多份状态，用近似逆物理步及得分修正逐步回溯，最终形成初态分布 p_0。原图 27.6；批状态、运算与连接均可编辑，原示例场的无文字局部作为节点图像保留。

得分函数 $s_\theta(\mathbf{x}, t)$ 采用带编码器/解码器的小型 ResNet 类架构；谱求解器用 JAX 可微编程实现。基线包括同样架构的监督训练、Bayes 神经网络（BNN）和 FNO。全部采用 L^2 损失，数据为初态、末态对 $(\mathbf{x}_0, \mathbf{x}_T)$。另有去掉近似逆步的 SMDP 变体，动力学逆映射完全由 s_θ 学习，记作 “s_θ only”。

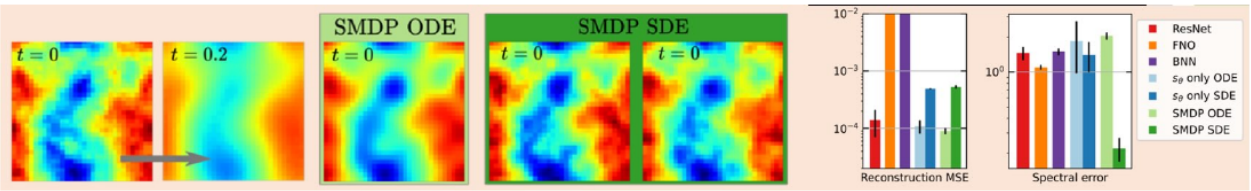


图 27.7: ODE 轨迹给出平滑解、重建 MSE 最低；SDE 解合成高频，显著改善谱误差。无逆物理步的 “s_θ only” 谱误差明显更大。右侧指标平均三次运行，保留全部模型图例、误差棒与对数纵轴；左侧为初态/末态及 ODE、SDE 重建。原图 27.7。

在 500 组初态与末态测试集上，以重建 MSE 评价 SMDP 和基线：先将网络预测初态用 $\mathcal{P}_h$ 正向模拟到末态，再与真实末态计算 L^2 距离。缺点是，这不衡量预测是否符合训练数据流形，即是否具有初始 Gaussian 随机场的性质。因此，再用状态的功率谱密度作为谱损失。图 27.7 显示，ODE 推断的重建 MSE 最好，但解太平滑，缺少必要小尺度结构，谱误差较高；SDE 的谱误差很好，视觉可信，重建 MSE 只略增。[7]

这突出噪声在扩散推断中作为熵源的作用，例如 SMDP 的 SDE，合成小尺度结构需要它。两个指标自然存在权衡：相同权重下，ODE 和 SDE 分别在两个指标上最好。热扩散例子显示了把物理过程纳入扩散过程的优点与性质，也可推广到其他物理。SMDP 仓库还展示了逆 Navier–Stokes 求解。

[7]译注：原文此处引用显示为未解析的 `\ref` 加标签 `fig:stochastic_heat_eq_overview`，已据紧邻原图 27.7 的实际对应关系绑定；不据损坏标签补造新图。

27.4 基于物理的扩散模型小结

上面介绍了两种把 PDE 形式的物理约束与模型纳入扩散的方法。有趣的是，其引入方式与 “可微物理导论” 大体一致：物理求解器梯度居核心地位，展开等概念也重要。另一方面，概率建模增加训练与推断复杂性，带来强大工具与解分布，甚至尚未讨论不确定性量化等后续应用，但有代价。

作为经验准则，只有当解的分布不能由均值良好表示时，才应采用扩散建模。若均值可接受，“普通”NN 在训练与推断复杂性上有很大优势。

但若解确实需要分布，扩散模型就是处理复杂、多样解的强大工具。考虑其能力，额外复杂性可能出奇地少。例如 FM 训练稳健，可在确定性训练基础上构建，只增加温和计算开销。[8]

下一节给出 SMDP 用例代码，展示物理求解器与扩散模型如何组合。[CKM+23]、[HT23]、[HVT23] 完整书目待全书文献接入。

译注：独立核对的适用条件 c 是 x_t 的函数且采用同一 CFM 损失，并不单独保证受限输入的 $v^C(v,c,t)$ 等于 $\mathbb{E}[u \mid x_t, y, t]$；该回归的理想最优是 $\mathbb{E}[u \mid v, c, t]$，还需 (v,c,t) 保留所需信息、模型可表示目标速度。另同一时间约定下，SDE 漂移 b 的同边缘概率流 ODE 漂移含 $-g^2 s_\theta/2$ 修正，不能只删除噪声。写 $C=1$ 还需明确 SDE 与得分的缩放；原图 27.5、训练更新与推断符号并不一致，目前这些式子不足以验证为同一约定，原式和符号冲突注均保留。

[8] 译注：源文经验准则两段各有装饰符号渲染异常，提取为 “2/7”，未据乱码猜译。各项建议仍是作者针对任务的判断，未作新的性能比较。

第二十八章　用可微仿真求解概率逆问题

本 notebook 以清晰的简化问题，展示概率学习导言介绍的一些概念，例如通过对数似然训练得分函数，以及所得函数的形态。同时，设置中集成了一个简单的可微仿真器，用于说明结合物理约束一章的 SMDP 方法所采用的基于物理的扩散建模（源文另链完整论文）。该方法沿合并后的时间维度结合物理与得分匹配，以求解逆问题。源文附 “在 Colab 中运行” 链接。[1]

28.1　玩具问题设置

考虑轨迹按二次项衰减的玩具问题。轨迹从 1 或 -1 出发，随 t 增加趋近于 0。对应 SDE 为

$$dx = -\big[\lambda_1 \operatorname{sign}(x) x^2\big]\,dt + \lambda_2\,dw. \tag{28.1}$$

其中 $\lambda_1 = 7$，$\lambda_2 = 0.03$。对应的逆时间 SDE 在原文写作

$$dx = -\big[\lambda_1 \operatorname{sign}(x) x^2 - \lambda_2^2 \nabla_x \log p_t(x)\big]\,dt + \lambda_2\,dw. \tag{28.1}$$

[2] 所有实验均令 p_0 为类别分布，以相同概率抽取 1 或 -1。

28.2　实现概览

下面的实现包括数据生成、网络定义、用滑动窗口法训练、所学得分的可视化，以及用概率流 ODE 与逆时间 SDE 推断。训练和推断的核心算法很容易修改，以实现论文讨论的不同变体。该实现使用较多包，先导入它们。神经网络采用 JAX 及其 haiku、optax 库；数据生成和推断采用 diffrax：

```
try:
    import google.colab # only to ensure that we are inside colab
    %pip install diffrax jax jaxlib scipy optax dm-haiku
except ImportError:
    print("This notebook is running locally, please make sure the packages above are
    ↪  installed")
    pass
```

[1] 译注：本章代码、日志与图片均来自冻结 PDF，未安装依赖、生成数据、训练或推断。链接文字未携可恢复目标时不猜补地址。

[2] 译注：源文重复编号 (28.1)，照录。若 $dt < 0$ 表示沿原时间反向积分，标准逆时间漂移为正向漂移减 λ_2^2 得分；源式展开后却为正向漂移加得分项，与后文 `physics_operator-forward_fn` 的代码不一致，不能视为同一符号约定下已验证的等价式。有噪声时轨迹也并非逐条单调收敛到零。

```
import warnings
warnings.filterwarnings('ignore')
from diffrax import diffeqsolve, ControlTerm, Euler, MultiTerm, ODETerm, SaveAt,
↪  VirtualBrownianTree, WeaklyDiagonalControlTerm
import diffrax as dfx
import jax
import jax.random as jr
import jax.numpy as jnp
import optax
import haiku as hk
```

此外还使用常见的 numpy 与 matplotlib 库：

```
import math
import numpy as np
from tqdm import tqdm
from scipy.interpolate import griddata
from typing import Any, Callable, Iterable, List, Optional, Tuple, Union
from matplotlib import pyplot as plt
from mpl_toolkits.axes_grid1 import make_axes_locatable
import matplotlib.gridspec as gridspec
```

28.3 物理系统 SDE

用 $\lambda = 7$、$g = 0.03$ 设置上述 SDE 的 lambda 函数：

```
g = 0.03
lambda_ = 7
physics_operator = lambda x: - jnp.sign(x) * x * x * lambda_
```

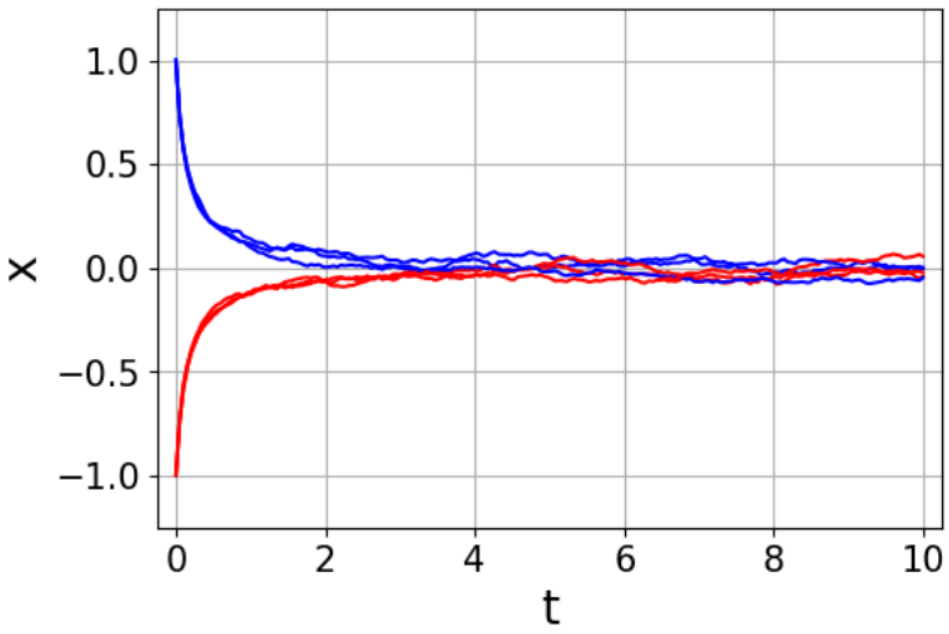
用 diffrax 和 Euler 步求解 SDE：

```
def r_process(initial_value, noise_scaling, seed):
    initial_shape = (1,)
    y0 = jnp.ones(shape=initial_shape) * initial_value
    t0, t1 = 0.0, 10.0
    drift = lambda t, y, args: physics_operator(y)
    diffusion = lambda t, y, args: noise_scaling * jnp.ones(initial_shape)
    brownian_motion = VirtualBrownianTree(t0, t1, tol=1e-3, shape=initial_shape,
  key=jr.PRNGKey(seed))
    terms = MultiTerm(ODETerm(drift), WeaklyDiagonalControlTerm(diffusion,
    ↪  brownian_motion))
    solver = Euler()
    saveat = SaveAt(dense=True)
    sol = diffeqsolve(terms, solver, t0, t1, dt0=0.01, y0=y0, saveat=saveat)
    return sol
```

28.4 SDE 路径的可视化

先为 SDE 路径设置种子，绘制若干示例路径。

```
seed = 2022
if 1:
    # paths starting in 1.0
    value_one = [r_process(1.0, g, seed+n) for n in range(0, 3)]
    # paths starting in -1.0
    value_minus_one = [r_process(-1.0, g, seed+n) for n in range(3, 6)]
    fig, ax = plt.subplots(figsize=(6,4))
    x = jnp.linspace(0,10,200)
    for i in range(3):
        sol = value_one[i]
        ax.plot(x, jnp.diag(sol.evaluate(x)), color='blue')
        sol = value_minus_one[i]
        ax.plot(x, jnp.diag(sol.evaluate(x)), color='red')
    ax.tick_params(axis='x', colors='black')
    ax.tick_params(axis='y', colors='black')
    ax.set_xlabel('t', size = 20)
    ax.set_ylabel('x', size = 20)
    ax.grid(True)
    ax.tick_params(labelsize=15)
    plt.xlim([-0.25, 10.25])
    plt.ylim([-1.25,1.25])
    plt.show()
```



如预期，轨迹从两侧出发（蓝色和红色），以随机方式趋近零线。

28.5 生成训练数据

生成包含 250 条路径的数据集，时间从 $t = 0$ 至 $t = 10$，原文称 $\Delta t = 0.02$。

```
import os.path
import pickle
if os.path.isfile('toy_example_data.p'):
    with open('toy_example_data.p', 'rb') as file:
        dataset = pickle.load(file)
else:
    dataset = []
    x = jnp.linspace(0,10,500)
    for n in tqdm(range(250)):
        sol = r_process((-1) ** n, g, seed+n)
        dataset.append(jnp.diag(sol.evaluate(x)))
data = jnp.array(dataset)
```

原书输出：

```
100%|██████████| 250/250 [04:39<00:00, 1.12s/it]
```

```
import pickle
with open('toy_example_data.p', 'wb') as file:
    pickle.dump(dataset, file)
```

[3] 接着实现数据集的迭代器函数：

```
def _prepare_batch(batch):
    batch_x = batch[:,::-1]
    batch_t = jnp.linspace(0,10, batch_x.shape[1])[::-1]
    return (batch_x, batch_t)
def iterbatches(X, batch_size, shuffle=False):
    def iterate(X, batch_size, shuffle=False):
        n_samples = X.shape[0]
        ids = np.arange(n_samples)
        sample_perm = np.arange(n_samples)
        if batch_size is None:
            batch_size = n_samples
        if shuffle:
            sample_perm = np.random.permutation(n_samples)
        batch_idx = 0
        num_batches = math.ceil(n_samples / batch_size)
        while batch_idx < num_batches:
            start = batch_idx * batch_size
            end = min(n_samples, (batch_idx + 1) * batch_size)
            indices = range(start, end)
            perm_indices = sample_perm[indices]
            X_batch = X[perm_indices]
```

[3]译注：`linspace(0,10,500)` 包含两端点，实际间隔是 $10/499$，并非恰为 0.02。代码按源保留。

```
            ids_batch = ids[perm_indices]
            batch_idx += 1
            yield X_batch
    return iterate(X, batch_size, shuffle)
```

还需实现得分场可视化所需的迭代器：

```
def _prepare_batch_grid(batch):
    inputs = [np.split(x, x.shape[1], 1) for x in batch]
    return inputs
def iterbatches_grid(X, batch_size, shuffle=False):
    def iterate(X, batch_size, shuffle=False):
        n_samples = X.shape[0]
        sample_perm = np.arange(n_samples)
        if batch_size is None:
            batch_size = n_samples
        if shuffle:
            sample_perm = np.random.permutation(n_samples)
        batch_idx = 0
        num_batches = math.ceil(n_samples / batch_size)
        while batch_idx < num_batches:
            start = batch_idx * batch_size
            end = min(n_samples, (batch_idx + 1) * batch_size)
            indices = range(start, end)
            perm_indices = sample_perm[indices]
            X_batch = X[perm_indices]
            batch_idx += 1
            yield ([X_batch])
    return iterate(X, batch_size, shuffle)
```

28.5.1 神经网络设置

本例用参数为 θ 的神经网络 $s_\theta(x,t)$ 近似得分。它是采用 elu 激活的简单多层感知机，有五个隐藏层，神经元数依次为 30、30、25、20、10。网络结构用 haiku 中的 MLP 实现：

```
EPSILON = 1e-5
def f(x, t):
    t = jnp.log(t + EPSILON)
    x = jnp.hstack([x, t])
    net = hk.nets.MLP(output_sizes = [30, 30, 25, 20, 10, 1],
                      activation = jax.nn.elu)
    return net(x)
```

下一单元初始化参数。之后主要通过 `forward_fn` 计算网络 f。

```
init_params, forward_fn = hk.transform(f)
rng = jax.random.PRNGKey(0)
x_init = jnp.ones((10,1))
t_init = x_init
params = init_params(rng, x_init, t_init)
```

再定义模型求值函数，以及给定优化器和模型损失后执行反向传播与参数更新的函数。

```
def create_eval_fn(forward_fn, params):
    @jax.jit
    def eval_model(t, x, rng=None):
        res = forward_fn(params, rng, x, t)
        return res
    return eval_model
def create_default_update_fn(optimizer: optax.GradientTransformation,
                             model_loss: Callable):
    @jax.jit
    def update(params, opt_state, batch, rng) -> Tuple[hk.Params, optax.OptState,
    ↪  jnp.ndarray]:
        batch_loss, grads = jax.value_and_grad(model_loss)(params, rng, *batch)
        updates, opt_state = optimizer.update(grads, opt_state)
        new_params = optax.apply_updates(params, updates)
        return new_params, opt_state, batch_loss
    return update
```

最后用 Euler 步实现概率流 ODE 求解器。给定初始状态、真值 `x_train` 和时间离散 `t_train`，计算 L2 损失：

```
def gradient_fn(forward_fn, physics_operator, g):
    @jax.jit
    def model_loss(model_weights, rng, x_train, t_train):
        x = x_train[:,0]
        i = 1
        loss = 0.0
        for t1, t0 in zip(t_train, t_train[1:]):
            delta_t = t1-t0
            physics_update = physics_operator(x)
            # note that we absorb g**2 (constant) in the definition of forward_fn here
            score_update = - 0.5 * forward_fn(model_weights, rng, jnp.expand_dims(x,
  axis=1),
                                              jnp.repeat(jnp.tile(t1, 1)[None], x.
  shape[0], axis=0))[:,0]
            x = x - delta_t * (physics_update + score_update)
            x_true = x_train[:,i]
            loss += jnp.mean(jnp.square(x - x_true))
            i += 1
        return loss
    return model_loss
```

4

[4]译注：代码把常数 g^2 吸收到 `forward_fn` 中，所以网络输出近似 $g^2 s_\theta$；可视化时再除以 g^2。代码累加逐步均方误差，不另加时间步权重，也未在此直接计算显式对数似然。

28.5.2 得分场的可视化

问题本身虽清晰，其背后的得分函数形态却不那么直观。下面展示得分函数的不同版本。下一单元实现辅助函数，把规则网格上的得分采样绘成图像，先设置相应网格：

```
x_ = np.linspace(0., 10., 400)
y_ = np.linspace(-1.25, 1.25, 200)
X, Y = np.meshgrid(x_,y_)
full_domain = np.hstack((X.flatten()[:,None], Y.flatten()[:,None]))
meshgrid = (X,Y)
```

以下 `save_snapshot_score` 将得分函数画为图像。

```
dpi = 200
height = 6
width = 4
scaling_factor = 1.
def save_snapshot_score(params, forward_fn, meshgrid, step=None, savename=None):
    eval_fn = create_eval_fn(forward_fn, params)
    results = None
    X, Y = meshgrid
    full_domain = np.hstack((X.flatten()[:,None], Y.flatten()[:,None]))
    full_domain_scaled = full_domain * scaling_factor
    generator = iterbatches_grid(full_domain_scaled, 1000, shuffle=False)
    for batch in generator:
        inputs = _prepare_batch_grid(batch)
        output_values = eval_fn(*inputs[0], rng)
        if isinstance(output_values, jnp.ndarray):
            output_values = [output_values]
        output_values = [jax.device_get(t) for t in output_values]
        if results is None:
            results = [[] for i in range(len(output_values))]
        for i, t in enumerate(output_values):
            results[i].append(t)
    final_results = []
    if results is not None:
        for r in results:
            final_results.append(np.concatenate(r, axis=0))
    x_scaled = x_ * scaling_factor
    y_scaled = y_ * scaling_factor
    meshgrid_scaled = (meshgrid[0] * scaling_factor, meshgrid[1] * scaling_factor)
    fig, axes = plt.subplots(nrows = 1, ncols=1, figsize=(height,width))
    fig.set_dpi(dpi)
    ax = axes
    u_pred = final_results[0][:].flatten()
    U_pred = griddata(full_domain_scaled, u_pred.flatten(), meshgrid_scaled,
    ↪  method='cubic')
    vmax = 75
    h = ax.imshow(jnp.flip(U_pred / (g**2), axis=0), cmap='jet',
```

```
                extent=[ x_scaled.min(), x_scaled.max(), y_scaled.min(),
                ↪  y_scaled.max()],
                aspect='auto', vmin = -vmax, vmax = vmax)
    cbar = fig.colorbar(h)
    ax.set_xlabel("t")
    ax.set_ylabel("x")
    plt.figure(figsize=(10, 6)) # NT_DEBUG , test
    if savename:
        plt.savefig(f'{savename}.svg', transparent=True)
    plt.show()
```

观察尚未训练的得分函数 `forward_fn`：

```
save_snapshot_score(params, forward_fn, meshgrid)
```

不出所料，它包含随机而平滑的过渡。第一次训练后再来看它。

28.6　训练与滑动窗口法

接着实现网络训练函数，包括滑动窗口方法及相应训练算法。滑动窗口从 `ROLLOUT_start` 开始，每经过 `steps` 个 epoch 增加 `ROLLOUT_add`，共增加 `ROLLOUT_increases` 次。还用 `subsample` 控制子采样，以减少轨迹上的点数。

```
def update_network(params_, forward_fn, dataset, rng, steps = 20, ROLLOUT_increases =
↪  18, max_training_time=None,
                   ROLLOUT_start = 4, ROLLOUT_add = 2, lr=5e-4, bidirectional=False,
                   ↪  subsample=5):
    scheduler = optax.piecewise_constant_schedule( init_value=lr)
    # Optax otimizer using Adam
    opt = optax.chain(
        optax.scale_by_adam(b1=0.9, b2=0.99),
        optax.scale_by_schedule(scheduler),
        optax.scale(-1.0))
    opt_state = opt.init(params_)
    # Define model loss and update for network parameters
    model_loss_fn = gradient_fn(forward_fn, physics_operator, g)
    grad_update = create_default_update_fn(opt, model_loss_fn)
    avg_loss = 0
    grad_updates = 0
    ROLLOUT = ROLLOUT_start
    history = []
    # Iterate through all sliding window sizes
    for _ in range(ROLLOUT_increases+1):
        print('Window size: ', ROLLOUT)
        pbar = tqdm(range(steps))
        # Iterate through all epochs
        for n in pbar:
```

```
            # Iterate through data set
            generator = iterbatches(dataset, 256, shuffle=False)
            for batch in generator:
                x_train, t_train = _prepare_batch(batch)
                # Concentrate on specific parts of trajectory
                if max_training_time is not None:
                    x_train = x_train[:, -max_training_time:]
                    t_train = t_train[-max_training_time:]
                # Use subsampling to reduce number of points on trajectory
                x_train = x_train[:, ::subsample]
                t_train = t_train[::subsample]
                # Iterate through trajectory
                for t in range(x_train.shape[1]):
                    # Select values based on position and window size
                    x_train_sub = x_train[:, t:t+ROLLOUT]
                    t_train_sub = t_train[t:t+ROLLOUT]
                    # Compute loss for current part of trajectory
                    params_, opt_state, batch_loss_backward = grad_update(params_,
                    ↪  opt_state, [x_train_sub, t_train_sub], rng=rng)
                    # Reverse values and time discretization for the forward time
                    ↪  direction
                    if bidirectional:
                        params_, opt_state, batch_loss_forward = grad_update(params_,
  opt_state,
                                                             [x_train_sub[:,::-1], t_train_su」
                                                                                  ↪  b[::-
                                                                                  ↪  1]],
                                                                                  ↪  rng=」
                                                                                  ↪  rng)
                    grad_updates += 1
                    rng, _ = jax.random.split(rng)
                    avg_loss += jax.device_get(batch_loss_backward)
                    if bidirectional:
                        avg_loss += jax.device_get(batch_loss_forward)
                    history.append(batch_loss_backward+batch_loss_forward)
            pbar.set_description(f'loss: {avg_loss/(n+1):.5f} grad updates:
            ↪  {grad_updates}')
        ROLLOUT += ROLLOUT_add
    return params_, history
```

5

[5]译注：源代码仅在 `bidirectional=True` 时赋值 `batch_loss_forward`，随后却无条件把它加到 `history`；默认 False 分支存在未赋值风险。展示的两次调用均显式为 True。窗口在轨迹末端还可能缩短至单点，原代码未跳过；保留源实现，不声称已运行验证。

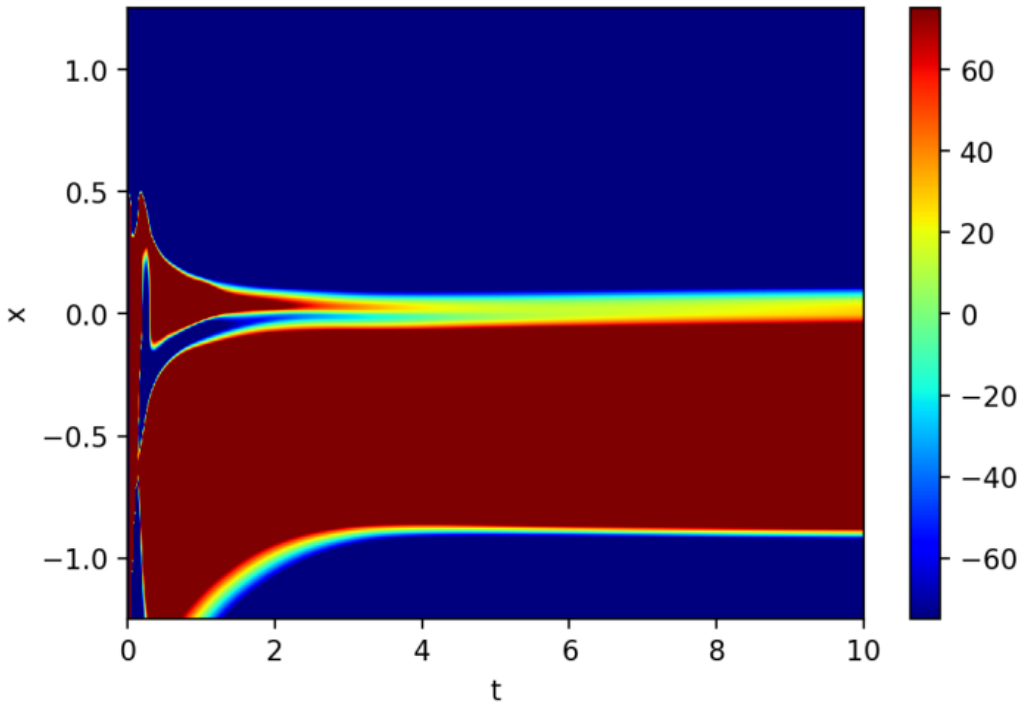
28.6.1 单步损失

用概率流 ODE 的单步损失训练得分网络 2500 个 epoch（参见源文 “Single steps, Section 3”）。单步由大小为 2 的滑动窗口实现，即每个样本向前走一个时间步。下面默认 `ROLLOUT_increases=0` 禁用展开长度的增加，以加快默认训练；后续测试可启用。还使用因子 5 的子采样减少轨迹点。训练后再次绘制得分函数。

```
ROLLOUT_increases=0
key = jax.random.PRNGKey(seed)
params_single_step, history = update_network(params, forward_fn, data, key, steps =
↪  2500,
                                             ROLLOUT_start=2,
                                             ↪  ROLLOUT_increases=ROLLOUT_increases,
                                             ↪  bidirectional=True, lr=1e-3,
                                             subsample=5)
save_snapshot_score(params_single_step, forward_fn, meshgrid)
```

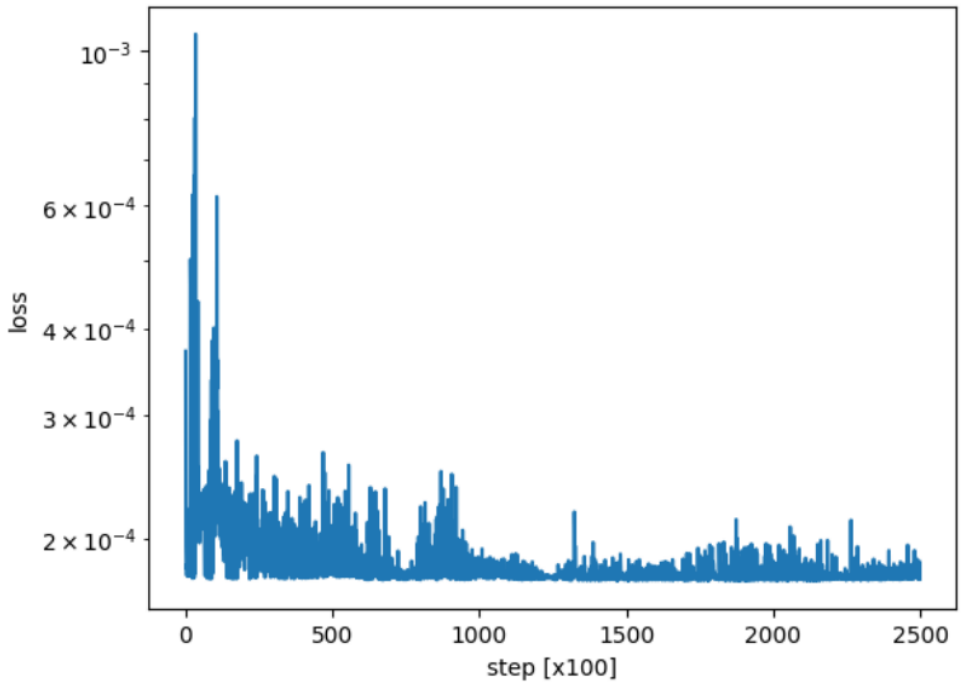
原书输出：

```
Window size: 2
loss: 0.02322 grad updates: 250000: 100%|██████████| 2500/2500 [10:23<00:00, 4.01it/s]
```



这应比初始随机状态更有意义。应能辨认出从 -1 与 1 出发的两条轨迹带。例如顶部的蓝色区域表示较大的负梯度，会把落入这里的样本向下移动较大距离，推向正值区域中的正确轨迹（见上图）。某些部分，通常是靠近终止时间处（右侧 $t = 10$），仍可能出现符号变化和近乎随机的内容。这是因为网络主要在训练样本附近受训；远离这些样本时，它 “没有学会” 如何把样本正确变换为容许的样本。还绘制训练损失，展示第一阶段的进展。

```
plt.plot(np.array(history[::100]))
plt.yscale('log')
plt.xlabel('step [x100]')
plt.ylabel('loss')
plt.show()
```

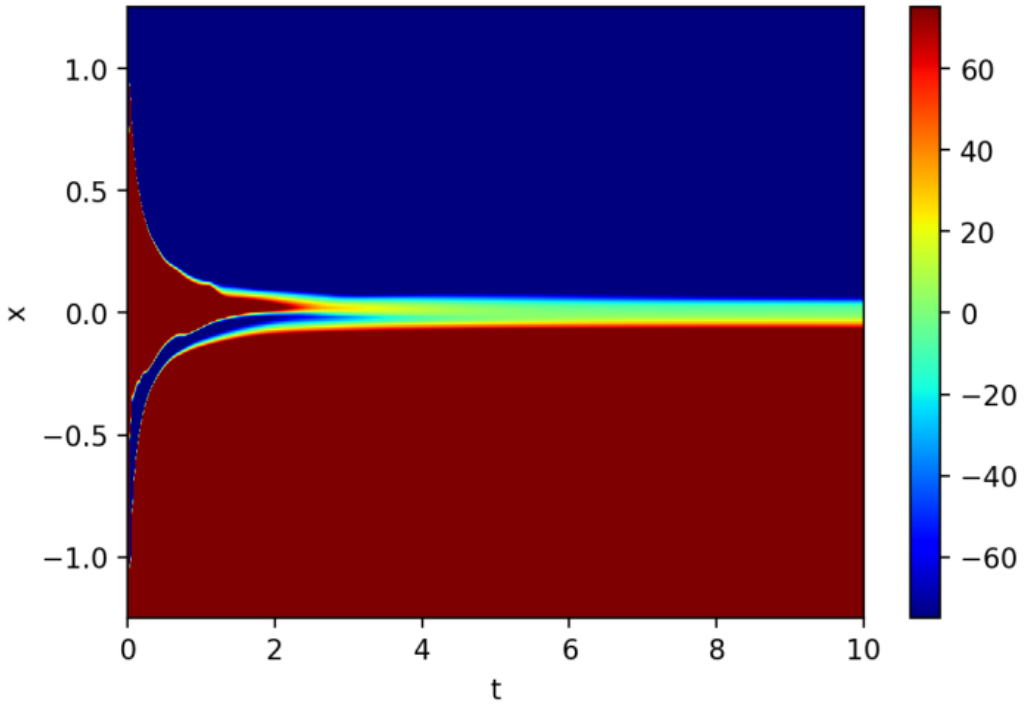


下一训练阶段旨在改进所学得分的质量：降低学习率，不再通过“子采样”减少轨迹点，让网络见到训练数据中的全部样本。

```
params_single_step, history = update_network(params_single_step, forward_fn, data, key,
↪  steps = 1000,
                                             ROLLOUT_start=2,
                                             ↪  ROLLOUT_increases=ROLLOUT_increases,
                                             ↪  bidirectional=True, lr=1e-4,
                                             subsample=1)
save_snapshot_score(params_single_step, forward_fn, meshgrid)
```

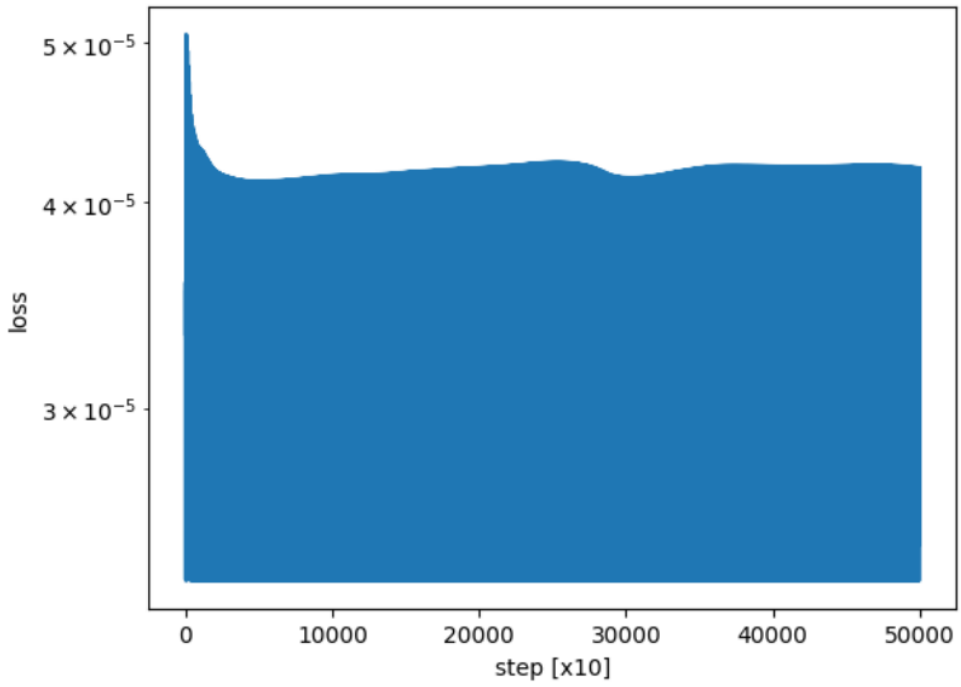
原书输出：

```
Window size: 2
loss: 0.01599 grad updates: 500000: 100%|██████████| 1000/1000 [20:41<00:00, 1.24s/it]
```



通常这会得到更干净的得分函数，在两条真值轨迹带之外的符号变化更少。上述运行时间通过 `ROLLOUT_increases=0` 缩短；启用如 `ROLLOUT_increases=10`，并可增加迭代次数后，应能在可视化中看到得分场进一步改善。这一阶段的训练也更稳定，如下图所示。

```
plt.plot(np.array(history[::10]))
plt.yscale('log')
plt.xlabel('step [x10]')
plt.ylabel('loss')
plt.show()
```



28.7 采样 SDE 与 ODE 轨迹

上面只查看了所学得分函数，现在评价积分两类微分方程所生成的轨迹。用训练好的得分和 diffrax 求解概率流 ODE，并模拟逆时间 SDE 的路径。

28.7.1 逆时间 SDE

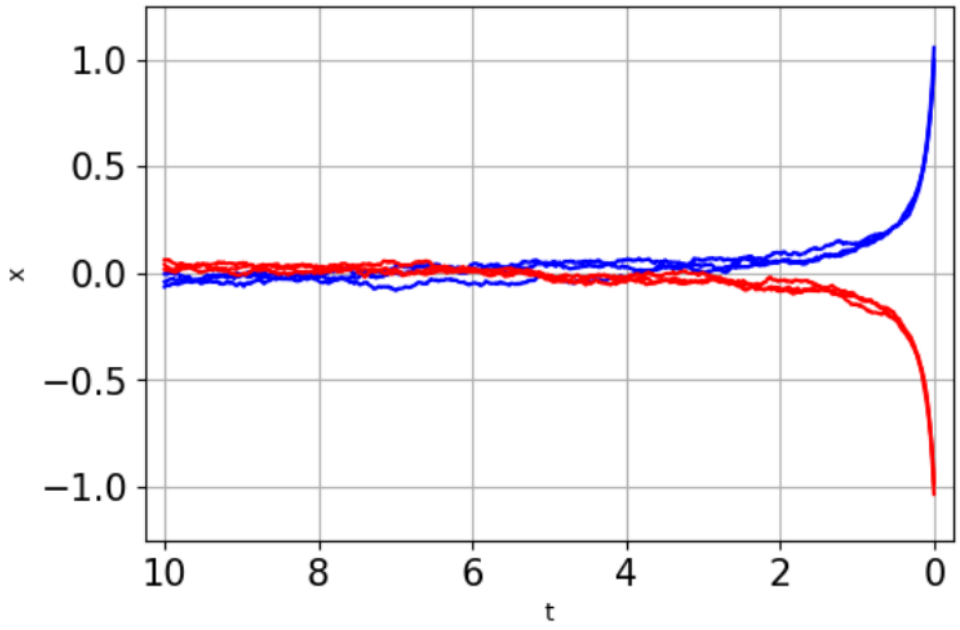
定义用 diffrax 模拟逆时间 SDE 路径的函数，其中 `WeaklyDiagonalControlTerm(diffusion, brownian_motion)` 处理噪声：

```
def r_process_reverse(initial_value, params, noise_scaling, seed):
    key = jr.PRNGKey(seed)
    initial_shape = (1,)
    y1 = jnp.ones(shape=initial_shape) * initial_value
    t0, t1 = 0.0, 10.0
    dt0 = 0.01
    def drift(t, y, args):
        return physics_operator(y) - forward_fn(params, key, y, t)
    diffusion = lambda t, y, args: noise_scaling * jnp.ones(initial_shape)
    brownian_motion = VirtualBrownianTree(t0, t1, tol=1e-3, shape=initial_shape,
  key=key)
    terms = MultiTerm(ODETerm(drift), WeaklyDiagonalControlTerm(diffusion,
    ↪  brownian_motion))
    solver = dfx.Euler()
    t0 = jnp.array(0.0)
    args = None
```

```
    tprev = jnp.array(t1)
    tnext = jnp.array(t1 - dt0)
    y = y1
    state = solver.init(terms, tprev, tnext, y1, args)
    y_list = []
    for i in range(((t1-t0) / dt0).astype(int)):
        y, _, _, state, _ = solver.step(terms, tprev, tnext, y, args, state,
        ↪  made_jump=False)
        tprev = tnext
        tnext = jnp.array(jnp.maximum(tprev - dt0, t0))
        y_list.append(y)
    return y_list
```

接着绘制从逆时间 SDE 采样的十条不同路径（原文如此）：

```
if 1:
    combinations = zip(np.linspace(-0.065,0.065, 6), range(6))
    fig, ax = plt.subplots(figsize=(6,4))
    for init, seed_ in combinations:
        sol = r_process_reverse(init, params_single_step, g, seed_)
        if sol[-1] > 0:
            col = 'blue'
        else:
            col = 'red'
        ax.plot(np.linspace(10.0, 0.0, 1000), sol, color=col)
    ax.tick_params(axis='x')
    ax.tick_params(axis='y')
    ax.set_xlabel('t')
    ax.set_ylabel('x')
    ax.grid(True)
    ax.tick_params(labelsize=15)
    plt.xlim([10.25, -0.25])
    plt.ylim([-1.25,1.25])
    plt.show()
```



[6] 轨迹按逆时间 SDE 的积分方向绘制，因此相对上面的真值轨迹，看起来关于纵轴镜像。默认设置下，轨迹应表现较好：初始阶段呈噪声运动，并可靠趋向 1 和 -1。若降低精度或缩短训练时间，得分会出现错误区域，使粒子在到达 $t = 0$ 前就飞离。

28.7.2 概率流 ODE

概率流 ODE 是 SDE 的确定性对应物，是去掉 `brownian_motion` 扩散项得到的有趣变体。类似地，定义仅含漂移项的概率流 ODE 求解函数：

```
def r_process_reverse_ode(initial_value, params, noise_scaling, seed):
    key = jr.PRNGKey(seed)
    initial_shape = (1,)
    y1 = jnp.ones(shape=initial_shape) * initial_value
    t0, t1 = 0.0, 10.0
    dt0 = 0.01
    def drift(t, y, args):
        return physics_operator(y) - 0.5 * forward_fn(params, key, y, t)
    terms = ODETerm(drift)
    solver = dfx.Euler()
    t0 = jnp.array(0.0)
    args = None
    tprev = jnp.array(t1)
    tnext = jnp.array(t1 - dt0)
    y = y1
    state = solver.init(terms, tprev, tnext, y1, args)
    y_list = []
    for i in range(((t1-t0) / dt0).astype(int)):
```

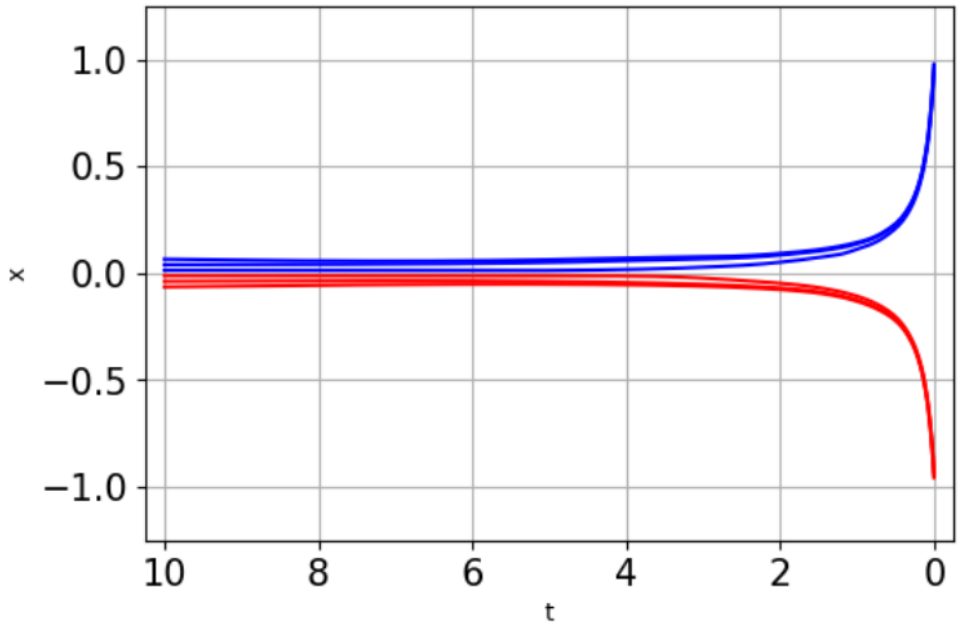
[6] 译注：绘图代码实际用 6 组初值和种子，而非正文所称的 10 条；源图照录。

```
        y, _, _, state, _ = solver.step(terms, tprev, tnext, y, args, state,
        ↪  made_jump=False)
        tprev = tnext
        tnext = jnp.array(jnp.maximum(tprev - dt0, t0))
        y_list.append(y)
    return y_list
```

[7] 最后画出概率流 ODE 的解：

```
#with plt.style.context("seaborn-white"):
if 1:
    combinations = zip(np.linspace(-0.065,0.065, 6), range(6))
    fig, ax = plt.subplots(figsize=(6,4))
    for init, seed_ in combinations:
        sol = r_process_reverse_ode(init, params_single_step, g, seed_)
        if sol[-1] > 0:
            col = 'blue'
        else:
            col = 'red'
        ax.plot(np.linspace(10.0, 0.0, 1000), sol, color=col)
    ax.tick_params(axis='x')
    ax.tick_params(axis='y')
    ax.set_xlabel('t')
    ax.set_ylabel('x')
    ax.grid(True)
    ax.tick_params(labelsize=15)
    plt.xlim([10.25, -0.25])
    plt.ylim([-1.25,1.25])
    plt.show()
```

[7]译注：除了移除噪声，代码还把得分漂移系数从 1 改为 1/2；这是概率流 ODE 与逆时间 SDE 之间的关键区别，不能仅删除 Brownian 项而保持原漂移。



它们展示了无随机扰动时得分的影响；本例中轨迹大多水平延伸，随后分叉向两个端点。

28.8 下一步

- 可在此 notebook 中改变展开步数（见上面的 `ROLLOUT_increases`），评价它们对得分学习的影响。
- 也可故意用较短训练降低得分质量，这应产生混乱且错误的轨迹。

译注：独立核对的适用条件 双向训练每个窗口调用优化器两次，但 grad_updates 只加 1，日志 250000/500000 是窗口计数，实际调用 500000/1000000 次（含末端单点零损失窗口）。avg_loss 全程累加再除 n+1，是此阶段累计窗口损失的每 epoch 平均，不是当前 epoch 或逐窗口平均；多窗口阶段不清零且 n 重置还会改变含义。逆采样只存步后态，名义时间为 9.99,9.98,…,0，绘图 linspace 却从 10 起，间隔为 10/999。初分布是 $\frac{1}{2}\delta_{-1}+\frac{1}{2}\delta_{1}$，$t=0$ 无普通 Lebesgue 密度得分，相关得分表述应限 $t>0$ 或终点极限；时间加 EPS 仅作数值处理。六个等距起点也非从 p_{10} 独立采样，轨迹图不能独立验证 50/50 端点概率或精确后验。

第二十九章　基于扩散的时间预测

模拟偏微分方程（PDE），例如湍流流动，通常需要解析随时间演化的解。也就是说，关心的不是时间平均或长期平衡态，而是系统随时间的实际变化。这需要按自回归方式逐步调用迭代求解器，产生随时间变化的解。尽管这一领域不断进步，如何在较长时间范围内获得稳定而准确的预测，仍是关键挑战。许多动力系统本身复杂且混沌，长期忠实捕捉细致的物理现象并不容易。不确定性也影响时间序列预测：仿真所用空间平均状态中很小的模糊性，也可能随时间产生迥异结果。大多数传统求解器和学习方法还以确定性方式处理仿真轨迹，视其为一阶 Markov 过程（一个状态完全决定下一个）。Mori–Zwanzig 形式则为许多系统提供了更现实的视角：我们只观测系统的一部分，而状态中“未观测”（或未模拟）的部分仍会影响其演化。确定性仿真器只产生一个解，没有计及底层过程可能具有的概率性。因此，与前章一样，应把时间序列各步看成随时间的概率分布，而不是确定性状态序列。概率仿真器可学习未观测状态的影响，并由其变化推断不同的解。最坏情况下，若未观测状态的影响可忽略，应得到方差几乎为零的平均状态，所以作者认为并无损失。[1]

以下 notebook（源文附“在 Colab 中运行”链接）介绍有效的基于分布的时间预测方法：

- 用条件扩散模型进行自回归展开，得到“概率仿真器”；
- 将这种扩散预测器与前章的确定性基线和神经算子比较，当然很有意义；
- 为评价精度，采用可由参考仿真计算统计量的跨声速流动。

问题表述　此前常训练网络执行 $f(x) = y$，现在关注 $f(x^t) = x^{t+1}$，表明任意后续步（例如 $f(x^{t+1}) = x^{t+2}$）与第一步同样重要。仍有真值 x^*_{t+1}，例如由昂贵的高保真仿真得到，目标是最小化

$$\arg\min_{\theta} |f(x_t;\theta) - x^*_{t+1}|_2^2. \tag{29.1}$$

注：无条件稳定性　扩散时间预测器最有趣的性质之一是时间稳定性。扩散过程似乎促使网络学会处理状态中的扰动和累积误差，不易偏离轨道。这对无条件稳定性至关重要，即网络可以自回归调用任意多次而不发散。作者称下述训练以出奇简单的方法产生无条件稳定网络（这里用 DDPM，流匹配也可用）。源文另链长期稳定性的详细评价。[2]

[1] 译注：这里是建模动机。投影状态一般可带记忆或随机项；下面实际网络以两个已预测时间步为条件。概率模型的有限数据拟合、计算开销与零方差极限并无自动保证。

[2] 译注：这是源文术语与实验解释，不是对所有初值、步长和任意无限时间的数学证明；章末仅报告有限但很长的测试范围。

29.1 条件化

此前在逆问题设置中，只简要说明生成后验分布的推断依赖超参数，如选定的边界条件。令 $x = (c, d)$，即数据点由条件成分 c 与目标数据 d 组成。时间预测还强依赖当前时间步，因此 c 除 Reynolds 数等参数外还含 x_t。借此说明实现条件化的一些细节。核心要点是：所有条件输入都应与扩散过程的输出同样处理。这初看有悖直觉，因为条件更像输入。已有工作表明，训练时“强迫”网络在去噪目标的同时也去噪条件，会迫使它充分考虑条件变量，使输出特征与条件紧密耦合。在高噪声水平移除条件信息，还可减少误差累积：逆过程 p_θ 的最初几步因条件噪声很大而近于无条件，故 c 中由此前去噪步累积的误差不会立即进入预测 d。所以训练时同样处理 c、d。图左为训练，右为推断；去噪时间记作 r，区别于物理时间 t。模型训练后，利用已知 c 的精确值这一事实。噪声 ϵ 同样是完全可控的输入，故可

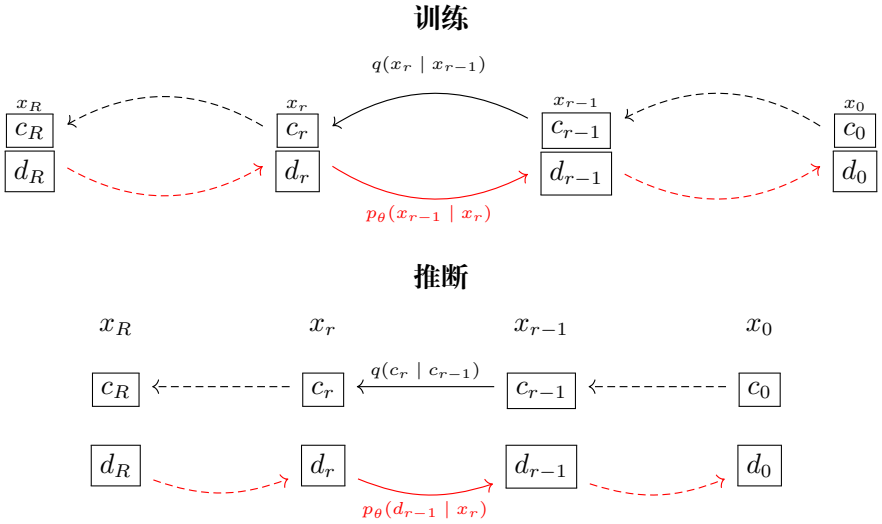
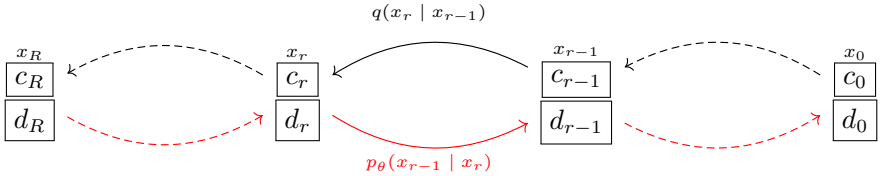


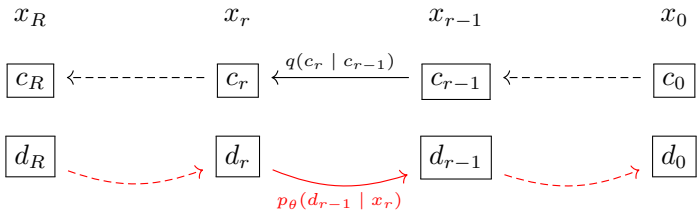
图 29.1: 带条件 c 与数据 d 的正向加噪和逆向去噪链。虚线表示省略的链段；红箭头为逆过程。

使去噪时间 r 的 c_r 恰含所选噪声场和加噪日程要求的内容。调用 $p_\theta(x_{r-1} \mid x_r)$ 得到 c_{r-1} 与 d_{r-1}，二者都在 x_{r-1} 中。若训练良好，预测的 c_{r-1} 应较好；为确保没有漂移，直接由已知真值 c 和正确噪声量 ϵ_{r-1} 重算它。再调用 $p_\theta(x_{r-2} \mid x_{r-1})$，此处 x_{r-1} 包含重算的条件以及上一去噪步推断的 d_{r-1}。时间预测并不更复杂，但记号略易混淆：条件就是物理时间的前一步 x^t。（c 还可包含全局参数，简明起见忽略；只需把它们附到 c。）每条去噪链的 d 成分给出下一个物理状态 x^{t+1}。网络因此同时去噪两个时间步；推断时规定已知状态的正确加噪值，即 $c = x^t$，$d = x^{t+1}$。由此得到下图。两种情况下，物理时间 t 都与 r 完全正交：除要求输出服从系统动力学外，去噪过程不以其他方式使用 t。一个有趣变体是把可变步长 Δt 放入条件 c。以下固定 Δt，为简单起见归一化为 1。

训练： 全部 c_r 行对应 x^t，全部 d_r 行对应 x^{t+1}。



推断： 每次都重算 c_r：x^t + 噪声。



最终 $d_0 = x^{t+1}$。

图 29.2: 时间预测的正、逆向链。训练时联合处理 $c = x^t$ 与 $d = x^{t+1}$；推断时每次迭代均以真值条件覆盖 c。

29.2 实现

具体介绍自回归条件扩散模型（ACDM），遵循已有基准与论文，目标是构造基于扩散的结构，以概率方式准确预测湍流仿真的下一步。首先下载预训练模型检查点以缩短后续训练，可能需要几分钟；成功下载的文件不会重复下载。

```
!mkdir -p models/
!wget -nc "https://dataserv.ub.tum.de/s/m1734798.001/download?path=/&files=checkpoint」
↪  s_acdm_tra.zip" -O models/checkpoints_acdm_tra.zip --no-check-certificate
!unzip -nq models/checkpoints_acdm_tra.zip -d models
```

原书输出：

```
File ‘models/checkpoints_acdm_tra.zip’ already there; not retrieving.
```

Colab 已预装许多包，只需另装 einops 与 PBDL-Dataloader。若在本地而非 Colab 运行，应遵循相应安装说明，并确保安装 `pipinstallgit+https://github.com/tum-pbs/pbdl-dataset@main`。

```
try:
    import google.colab # only to ensure that we are inside colab
    %pip install einops
    %pip install git+https://github.com/tum-pbs/pbdl-dataset@main
except ImportError:
    print("This notebook is running locally, please follow the ACDM installation
    ↪  instructions and install the PBDL loader")
    pass
```

原书输出：

```
This notebook is running locally, please follow the ACDM installation instructions and
↪  install the PBDL loader
```

[3] 导入下面所需的包。

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, SequentialSampler, RandomSampler
from einops import rearrange
from functools import partial
import matplotlib.pyplot as plt
import numpy as np
import math
import os, json
from typing import List,Tuple,Dict
from pbdl.torch.loader import Dataloader
```

[3]译注：以上是原书操作说明；本翻译未下载模型、数据或执行安装，后续所有日志及图也均来自冻结源。

29.3　骨干网络定义

当然还需神经网络结构。这里采用 “现代”U-Net。相较翼型 RANS 监督训练章的经典 U-Net，源文概括几处差异：用注意力机制取代跳接、用 GELU 取代 ReLU，并在每层使用组归一化。这是许多流行扩散模型的骨干，通常相对简单结构至少提高几个百分点，有时更多。[4] 先定义残差块和 U-Net 的上下采样操作。还用 Transformer 的正弦位置嵌入，通过时间嵌入 MLP 把扩散步融入 U-Net 各层。

```
class Residual(nn.Module):
    def __init__(self, fn):
        super().__init__()
        self.fn = fn
    def forward(self, x, *args, **kwargs):
        return self.fn(x, *args, **kwargs) + x
def Upsample(dim):
    return nn.ConvTranspose2d(dim, dim, 4, 2, 1)
def Downsample(dim):
    return nn.Conv2d(dim, dim, 4, 2, 1)
class SinusoidalPositionEmbeddings(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.dim = dim
    def forward(self, time):
        device = time.device
        half_dim = self.dim // 2
        embeddings = math.log(10000) / (half_dim - 1)
        embeddings = torch.exp(torch.arange(half_dim, device=device) * -embeddings)
        embeddings = time[:, None] * embeddings[None, :]
        embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
        return embeddings
```

下一类是各级的核心卷积块，基于 ConvNeXtBlock。

```
class ConvNextBlock(nn.Module):
    def __init__(self, dim, dim_out, *, time_emb_dim=None, mult=2, norm=True):
        super().__init__()
        self.mlp = (
            nn.Sequential(nn.GELU(), nn.Linear(time_emb_dim, dim))
            if time_emb_dim is not None else None
        )
        self.ds_conv = nn.Conv2d(dim, dim, 7, padding=3, groups=dim)
        self.net = nn.Sequential(
            nn.GroupNorm(1, dim) if norm else nn.Identity(),
            nn.Conv2d(dim, dim_out * mult, 3, padding=1),
            nn.GELU(),
            nn.GroupNorm(1, dim_out * mult),
            nn.Conv2d(dim_out * mult, dim_out, 3, padding=1),
```

[4]译注：性能是作者经验概括；以下实际 U-Net 仍用 `h.append`、`h.pop` 和拼接保留跨尺度跳接，注意力并未消除所有跳接。

```
        )
        self.res_conv = nn.Conv2d(dim, dim_out, 1) if dim != dim_out else nn.Identity()
    def forward(self, x, time_emb=None):
        h = self.ds_conv(x)
        if self.mlp is not None and time_emb is not None:
            assert time_emb is not None, "time embedding must be passed in"
            condition = self.mlp(time_emb)
            h = h + rearrange(condition, "b c -> b c 1 1")
        h = self.net(h)
        return h + self.res_conv(x)
```

再定义注意力机制。普通注意力（亦称 softmax 注意力）的开销对输入规模呈二次增长，故只用于 U-Net 瓶颈。此处潜空间最小，稠密连接不至于导致资源开销剧增。跳接则用线性注意力。它对输入规模为线性，但对内部核维数仍是二次；后者可自由选择，而 U-Net 输入规模是给定且可能很大。

```
class Attention(nn.Module):
    def __init__(self, dim, heads=4, dim_head=32):
        super().__init__()
        self.scale = dim_head**-0.5
        self.heads = heads
        hidden_dim = dim_head * heads
        self.to_qkv = nn.Conv2d(dim, hidden_dim * 3, 1, bias=False)
        self.to_out = nn.Conv2d(hidden_dim, dim, 1)
    def forward(self, x):
        b, c, h, w = x.shape
        qkv = self.to_qkv(x).chunk(3, dim=1)
        q, k, v = map(
            lambda t: rearrange(t, "b (h c) x y -> b h c (x y)", h=self.heads), qkv
        )
        q = q * self.scale
        sim = torch.einsum("b h d i, b h d j -> b h i j", q, k)
        sim = sim - sim.amax(dim=-1, keepdim=True).detach()
        attn = sim.softmax(dim=-1)
        out = torch.einsum("b h i j, b h d j -> b h i d", attn, v)
        out = rearrange(out, "b h (x y) d -> b (h d) x y", x=h, y=w)
        return self.to_out(out)

class LinearAttention(nn.Module):
    def __init__(self, dim, heads=4, dim_head=32):
        super().__init__()
        self.scale = dim_head**-0.5
        self.heads = heads
        hidden_dim = dim_head * heads
        self.to_qkv = nn.Conv2d(dim, hidden_dim * 3, 1, bias=False)
        self.to_out = nn.Sequential(nn.Conv2d(hidden_dim, dim, 1),
                                    nn.GroupNorm(1, dim))
    def forward(self, x):
        b, c, h, w = x.shape
```

```
        qkv = self.to_qkv(x).chunk(3, dim=1)
        q, k, v = map(
            lambda t: rearrange(t, "b (h c) x y -> b h c (x y)", h=self.heads), qkv
        )
        q = q.softmax(dim=-2)
        k = k.softmax(dim=-1)
        q = q * self.scale
        context = torch.einsum("b h d n, b h e n -> b h d e", k, v)
        out = torch.einsum("b h d e, b h d n -> b h e n", context, q)
        out = rearrange(out, "b h c (x y) -> b (h c) x y", h=self.heads, x=h, y=w)
        return self.to_out(out)
```

下一个辅助类便于在中间状态上下采样前添加组归一化。

```
class PreNorm(nn.Module):
    def __init__(self, dim, fn):
        super().__init__()
        self.fn = fn
        self.norm = nn.GroupNorm(1, dim)
    def forward(self, x):
        x = self.norm(x)
        return self.fn(x)
```

组合以上各层，定义完整的注意力 U-Net：

```
class Unet(nn.Module):
    def __init__(
        self,
        dim,
        init_dim=None,
        out_dim=None,
        dim_mults=(1, 2, 4, 8),
        channels=3,
        with_time_emb=True,
        resnet_block_groups=8,
        use_convnext=True,
        convnext_mult=2,
    ):
        super().__init__()
        # determine dimensions
        self.channels = channels
        init_dim = init_dim if init_dim is not None else dim // 3 * 2
        self.init_conv = nn.Conv2d(channels, init_dim, 7, padding=3)
        dims = [init_dim, *map(lambda m: dim * m, dim_mults)]
        in_out = list(zip(dims[:-1], dims[1:]))
        if use_convnext:
            block_klass = partial(ConvNextBlock, mult=convnext_mult)
        else:
            raise NotImplementedError()
```

```
        # time embeddings
        if with_time_emb:
            time_dim = dim * 4
            self.time_mlp = nn.Sequential(
                SinusoidalPositionEmbeddings(dim),
                nn.Linear(dim, time_dim),
                nn.GELU(),
                nn.Linear(time_dim, time_dim),
            )
        else:
            time_dim = None
            self.time_mlp = None
            self.cond_mlp = None
            self.sim_mlp = None
        # layers
        self.downs = nn.ModuleList([])
        self.ups = nn.ModuleList([])
        num_resolutions = len(in_out)
        for ind, (dim_in, dim_out) in enumerate(in_out):
            is_last = ind >= (num_resolutions - 1)
            self.downs.append(
                nn.ModuleList(
                    [
                        block_klass(dim_in, dim_out, time_emb_dim=time_dim),
                        block_klass(dim_out, dim_out, time_emb_dim=time_dim),
                        Residual(PreNorm(dim_out, LinearAttention(dim_out))),
                        Downsample(dim_out) if not is_last else nn.Identity(),
                    ]
                )
            )
        mid_dim = dims[-1]
        self.mid_block1 = block_klass(mid_dim, mid_dim, time_emb_dim=time_dim)
        self.mid_attn = Residual(PreNorm(mid_dim, Attention(mid_dim)))
        self.mid_block2 = block_klass(mid_dim, mid_dim, time_emb_dim=time_dim)
        for ind, (dim_in, dim_out) in enumerate(reversed(in_out[1:])):
            is_last = ind >= (num_resolutions - 1)
            self.ups.append(
                nn.ModuleList(
                    [
                        block_klass(dim_out * 2, dim_in, time_emb_dim=time_dim),
                        block_klass(dim_in, dim_in, time_emb_dim=time_dim),
                        Residual(PreNorm(dim_in, LinearAttention(dim_in))),
                        Upsample(dim_in) if not is_last else nn.Identity(),
                    ]
                )
            )
        out_dim = out_dim if out_dim is not None else channels
```

```
        self.final_conv = nn.Sequential(
            block_klass(dim, dim), nn.Conv2d(dim, out_dim, 1)
        )
    def forward(self, x, time):
        x = self.init_conv(x)
        t = self.time_mlp(time) if self.time_mlp is not None else None
        h = []
        # downsample
        for block1, block2, attn, downsample in self.downs:
            x = block1(x, t)
            x = block2(x, t)
            x = attn(x)
            h.append(x)
            x = downsample(x)
        # bottleneck
        x = self.mid_block1(x, t)
        x = self.mid_attn(x)
        x = self.mid_block2(x, t)
        # upsample
        for block1, block2, attn, upsample in self.ups:
            x = torch.cat((x, h.pop()), dim=1)
            x = block1(x, t)
            x = block2(x, t)
            x = attn(x)
            x = upsample(x)
        return self.final_conv(x)
```

29.4 方差日程

为确定扩散过程中的噪声水平，需要噪声日程。这里采用 Ho 等提出的简单线性日程，并调整方差参数，使大于 10 的扩散步数均应能良好工作：

```
def linear_beta_schedule(timesteps):
    if timesteps < 10:
        raise ValueError("Warning: Less than 10 timesteps require adjustments to this
        ↪  schedule!")
    beta_start = 0.0001 * (500/timesteps) # adjust reference values determined for 500
    ↪  steps
    beta_end = 0.02 * (500/timesteps)
    betas = torch.linspace(beta_start, beta_end, timesteps)
    return torch.clip(betas, 0.0001, 0.9999)
```

29.5 扩散模型定义

现在已集齐定义实际扩散模型类的各个部分。先计算 DDPM 加噪日程，用 PyTorch 的 `register_buffer` 声明超参数，确保学习中不把它们当作变量。

```
class DiffusionModel(nn.Module):
    def __init__(self, diffusionSteps:int, condChannels:int, dataChannels:int):
        super(DiffusionModel, self).__init__()
        self.timesteps = diffusionSteps
        betas = linear_beta_schedule(timesteps=self.timesteps)
        betas = betas.unsqueeze(1).unsqueeze(2).unsqueeze(3)
        alphas = 1.0 - betas
        alphasCumprod = torch.cumprod(alphas, axis=0)
        alphasCumprodPrev = F.pad(alphasCumprod[:-1], (0,0,0,0,0,0,1,0), value=1.0)
        sqrtRecipAlphas = torch.sqrt(1.0 / alphas)
        # calculations for diffusion q(x_t | x_{t-1}) and others
        sqrtAlphasCumprod = torch.sqrt(alphasCumprod)
        sqrtOneMinusAlphasCumprod = torch.sqrt(1. - alphasCumprod)
        # calculations for posterior q(x_{t-1} | x_t, x_0)
        posteriorVariance = betas * (1. - alphasCumprodPrev) / (1. - alphasCumprod)
        sqrtPosteriorVariance = torch.sqrt(posteriorVariance)
        self.register_buffer("betas", betas)
        self.register_buffer("sqrtRecipAlphas", sqrtRecipAlphas)
        self.register_buffer("sqrtAlphasCumprod", sqrtAlphasCumprod)
        self.register_buffer("sqrtOneMinusAlphasCumprod", sqrtOneMinusAlphasCumprod)
        self.register_buffer("sqrtPosteriorVariance", sqrtPosteriorVariance)
        # backbone model
        self.unet = Unet(
            dim=128,
            channels= condChannels + dataChannels,
            dim_mults=(1,1,1),
            use_convnext=True,
            convnext_mult=1,
        )
    # input shape (both inputs): B S C W H (D) -> output shape (both outputs): B S nC W
    ↪  H (D)
    def forward(self, conditioning:torch.Tensor, data:torch.Tensor) -> torch.Tensor:
        device = "cuda" if data.is_cuda else "cpu"
        seqLen = data.shape[1]
        # combine batch and sequence dimension for decoder processing
        d = torch.reshape(data, (-1, data.shape[2], data.shape[3], data.shape[4]))
        cond = torch.reshape(conditioning, (-1, conditioning.shape[2],
        ↪  conditioning.shape[3], conditioning.shape[4]))
        # TRAINING
        if self.training:
            # forward diffusion process that adds noise to data
            d = torch.concat((cond, d), dim=1)
            noise = torch.randn_like(d, device=device)
            t = torch.randint(0, self.timesteps, (d.shape[0],), device=device).long()
            dNoisy = self.sqrtAlphasCumprod[t] * d + self.sqrtOneMinusAlphasCumprod[t]
            ↪  * noise
            # noise prediction with network
```

```
            predictedNoise = self.unet(dNoisy, t)
            # unstack batch and sequence dimension again
            noise = torch.reshape(noise, (-1, seqLen, conditioning.shape[2] +
            ↪  data.shape[2], data.shape[3], data.shape[4]))
            predictedNoise = torch.reshape(predictedNoise, (-1, seqLen,
            ↪  conditioning.shape[2] + data.shape[2], data.shape[3], data.shape[4]))
            return noise, predictedNoise
        # INFERENCE
        else:
            # conditioned reverse diffusion process
            dNoise = torch.randn_like(d, device=device)
            cNoise = torch.randn_like(cond, device=device)
            for i in reversed(range(0, self.timesteps)):
                t = i * torch.ones(cond.shape[0], device=device).long()
                # compute conditioned part with normal forward diffusion
                condNoisy = self.sqrtAlphasCumprod[t] * cond +
                ↪  self.sqrtOneMinusAlphasCumprod[t] * cNoise
                dNoiseCond = torch.concat((condNoisy, dNoise), dim=1)
                # backward diffusion process that removes noise to create data
                predictedNoiseCond = self.unet(dNoiseCond, t)
                # use model (noise predictor) to predict mean
                modelMean = self.sqrtRecipAlphas[t] * (dNoiseCond - self.betas[t] *
                ↪  predictedNoiseCond / self.sqrtOneMinusAlphasCumprod[t])
                dNoise = modelMean[:, cond.shape[1]:modelMean.shape[1]] # discard
                ↪  prediction of conditioning
                if i != 0:
                   # sample randomly (only for non-final prediction), use mean directly
                    ↪  for final prediction
                    dNoise = dNoise + self.sqrtPosteriorVariance[t] *
                    ↪  torch.randn_like(dNoise)
            # unstack batch and sequence dimension again
            dNoise = torch.reshape(dNoise, (-1, seqLen, data.shape[2], data.shape[3],
            ↪  data.shape[4]))
            return dNoise
```

最重要的是 `DiffusionModel` 的 `forward`。它用 `self.training` 切换训练与推断：训练时评价 Markov 链中的单步以反向传播，推断时运行整条链以得到一个样本。变量前缀 c 和 d 区分 x 的条件和数据成分。例如推断代码的 `dNoiseCond=concat(condNoisy,dNoise)` 将二者拼成 x。两部分联合去噪，网络运行后通过 `modelMean[:,cond.shape[1]:modelMean.shape[1]]` 移除条件预测。随后用真值条件覆盖，使模型专注于生成准确的 d 部分。下一单元用 PBDL-Dataloader 获取训练和测试数据。先删除可能残留的旧 loader；Jupyter 中这样可避免 HDF5 文件锁错误。小麻烦是这里只用完整数据集的一小部分：仿真 ID20、21 用于训练，22 用于测试。为使用全数据的正确归一化常数，又避免下载全部 5GB，下面用完整数据的常数手动初始化 `norm_X_mean` 与 `norm_X_std` 对应字段。

```
# delete loaders if existing to ensure the dataset file is acessible
try:
    del trainLoader
```

```
except:
    pass
try:
    del testLoader
except:
    pass
# adjust dataset normalization buffers to match data normalization of pretrained model
download = Dataloader("transonic-cylinder-flow", sel_sims=[20,21,22])
del download
import h5py
dataPath = "datasets/transonic-cylinder-flow.hdf5"
with h5py.File(dataPath, "r+") as f:
    if "norm_fields_sca_mean" in f:
        f.__delitem__("norm_fields_sca_mean")
    if "norm_fields_sca_std" in f:
        f.__delitem__("norm_fields_sca_std")
    if "norm_const_mean" in f:
        f.__delitem__("norm_const_mean")
    if "norm_const_std" in f:
        f.__delitem__("norm_const_std")
    f["norm_const_mean"] = np.array([0.70])
    f["norm_const_std"] = np.array([0.118322])
    f["norm_fields_sca_mean"] = np.array([[[0.560642]], [[-0.000129]], [[0.903352]],
    ↪  [[0.637941]]])
    f["norm_fields_sca_std"] = np.array([[[0.216987]], [[0.216987]], [[0.145391]],
    ↪  [[0.119944]]])
    f.close()
batch = 32 # training batch size
# training set configuration
trainLoader = Dataloader(
    "transonic-cylinder-flow",
    time_steps=5, # leads to six timesteps that are later strided by a factor of 2 (for
    ↪  dt compatibility with pre-trained model with two input steps and one target
    ↪  step)
    intermediate_time_steps=True,
    step_size=6, # discard some datasamples close together
    sel_sims=[20, 21], # select simulations
    trim_start=250, # discard some initial timesteps from the simulation
    normalize_data="mean-std", # data normalization
    normalize_const="mean-std", # constants normalization
    batch_size=batch,
    shuffle=True,
    drop_last=True,
    num_workers=2,
)
```

原书输出：

```
download completed ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100%6m
Success: Loaded transonic-cylinder-flow with 41 simulations (3 selected) and 1 samples
↪  each.
download completed ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100%6m
Success: Loaded transonic-cylinder-flow with 41 simulations (2 selected) and 124
↪  samples each.
```

29.6 训练

有了以上组件和数据，就可训练扩散模型。可以从给定检查点继续训练少量 epoch（原书称 10 个 epoch 不到一分钟），也可从头在小示例集上训练（需几小时）。因为数据有限，从头训练的预测质量和多样性会明显较差。若不想训练，也可完全跳过本步，直接用预训练检查点继续采样。

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Training device: %s" % device)
startFromCheckpoint = True
if startFromCheckpoint:
    epochs = 10 # finetune only for a small number of epochs
    lr = 0.00001 # since the model is already trained, a conservatively low learning
    ↪  rate
else:
    epochs = 100000 # train from scratch for large number of epochs, this will take a
    ↪  while
    lr = 0.0001 # larger learning rate for training from scratch
diffusionSteps = 20 # the provided model checkpoint was pretrained on 20 diffusion steps
# model definition
condChannels = 2 * 5 # two timesteps with 5 channels each (vel_x, vel_y, dens, pres,
↪  mach)
dataChannels = 5 # one timestep
model = DiffusionModel(diffusionSteps, condChannels, dataChannels)
if startFromCheckpoint:
    # load weights from checkpoint
    loaded = torch.load("models/models_tra/128_acdm-r20_02/Model.pth",
    ↪  map_location=torch.device('cpu'))
    model.load_state_dict(loaded["stateDictDecoder"])
model.train()
model.to(device)
# print model info and trainable weigths
paramsTrainable = sum([np.prod(p.size()) for p in filter(lambda p: p.requires_grad,
↪  model.parameters())])
params = sum([np.prod(p.size()) for p in model.parameters()])
#print(model)
print("Trainable Weights (All Weights): %d (%d)" % (paramsTrainable, params))
# training loop
print("\nStarting training...")
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
for epoch in range(epochs):
```

```
    losses = []
    for s, sample in enumerate(trainLoader, 0):
        optimizer.zero_grad()
        input, target = sample
        input = input.unsqueeze(1) # add timestep dimension
        mach = input[:,:,-1:].expand(-1, target.shape[1], -1, -1, -1) # extract mach
        ↪  number from input
        target = torch.concat([target, mach], dim=2) # add mach number to target
        d = torch.concat([input,target], dim=1).to(device).float() # combined [batch,
        ↪  timesteps, channels, x, y] tensor
        d = d[:,::2] # temporal stride of 2 for dt compatiblity with pre-trained model
        inputSteps = 2
        cond = []
        for i in range(inputSteps):
            cond += [d[:,i:i+1]] # collect input steps
        conditioning = torch.concat(cond, dim=2) # combine along channel dimension
        data = d[:, inputSteps:inputSteps+1]
        noise, predictedNoise = model(conditioning=conditioning, data=data)
        loss = F.smooth_l1_loss(noise, predictedNoise)
       print(" [Epoch %2d, Batch %4d]: %1.7f" % (epoch, s, loss.detach().cpu().item()))
        loss.backward()
        losses += [loss.detach().cpu().item()]
        optimizer.step()
    print("[Epoch %2d, FULL]: %1.7f" % (epoch, sum(losses)/len(losses)))
del trainLoader # delete to ensure file access
print("Training complete!")
```

原书输出：

```
Training device: cuda
/tmp/ipykernel_917142/3219208168.py:22: FutureWarning: You are using `torch.load` with
↪  `weights_only=False` (the current default value), which uses the default pickle
↪  module implicitly. It is possible to construct malicious pickle data which will
↪  execute arbitrary code during unpickling (See
↪  https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
↪  details). In a future release, the default value for `weights_only` will be flipped
↪  to `True`. This limits the functions that could be executed during unpickling.
↪  Arbitrary objects will no longer be allowed to be loaded via this mode unless they
↪  are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`.
↪  We recommend you start setting `weights_only=True` for any use case where you don't
↪  have full control of the loaded file. Please open an issue on GitHub for any issues
↪  related to this experimental feature.
  loaded = torch.load("models/models_tra/128_acdm-r20_02/Model.pth",
  ↪  map_location=torch.device('cpu'))
Trainable Weights (All Weights): 6994035 (6994035)
Starting training...
    [Epoch 0, Batch 0]: 0.0019515
    [Epoch 0, Batch 1]: 0.0019654
```

```
    [Epoch 0, Batch 2]: 0.0008050
    [Epoch 0, Batch 3]: 0.0006933
    [Epoch 0, Batch 4]: 0.0009360
    [Epoch 0, Batch 5]: 0.0018654
    [Epoch 0, Batch 6]: 0.0016832
[Epoch 0, FULL]: 0.0014143
    [Epoch 1, Batch 0]: 0.0011901
    [Epoch 1, Batch 1]: 0.0018340
    [Epoch 1, Batch 2]: 0.0007823
    [Epoch 1, Batch 3]: 0.0014465
    [Epoch 1, Batch 4]: 0.0011019
    [Epoch 1, Batch 5]: 0.0012040
    [Epoch 1, Batch 6]: 0.0007916
[Epoch 1, FULL]: 0.0011929
    [Epoch 2, Batch 0]: 0.0008890
    [Epoch 2, Batch 1]: 0.0013503
    [Epoch 2, Batch 2]: 0.0011220
    [Epoch 2, Batch 3]: 0.0009489
    [Epoch 2, Batch 4]: 0.0010360
    [Epoch 2, Batch 5]: 0.0012546
    [Epoch 2, Batch 6]: 0.0009530
[Epoch 2, FULL]: 0.0010791
    [Epoch 3, Batch 0]: 0.0009957
    [Epoch 3, Batch 1]: 0.0013200
    [Epoch 3, Batch 2]: 0.0025289
    [Epoch 3, Batch 3]: 0.0009580
    [Epoch 3, Batch 4]: 0.0010404
    [Epoch 3, Batch 5]: 0.0010360
    [Epoch 3, Batch 6]: 0.0009525
[Epoch 3, FULL]: 0.0012616
    [Epoch 4, Batch 0]: 0.0024588
    [Epoch 4, Batch 1]: 0.0013688
    [Epoch 4, Batch 2]: 0.0008885
    [Epoch 4, Batch 3]: 0.0007770
    [Epoch 4, Batch 4]: 0.0008546
    [Epoch 4, Batch 5]: 0.0012591
    [Epoch 4, Batch 6]: 0.0013123
[Epoch 4, FULL]: 0.0012742
    [Epoch 5, Batch 0]: 0.0013485
    [Epoch 5, Batch 1]: 0.0012143
    [Epoch 5, Batch 2]: 0.0013597
    [Epoch 5, Batch 3]: 0.0007690
    [Epoch 5, Batch 4]: 0.0014855
    [Epoch 5, Batch 5]: 0.0011670
    [Epoch 5, Batch 6]: 0.0015456
[Epoch 5, FULL]: 0.0012699
    [Epoch 6, Batch 0]: 0.0010924
```

```
    [Epoch 6, Batch 1]: 0.0023275
    [Epoch 6, Batch 2]: 0.0008677
    [Epoch 6, Batch 3]: 0.0008471
    [Epoch 6, Batch 4]: 0.0014519
    [Epoch 6, Batch 5]: 0.0011380
    [Epoch 6, Batch 6]: 0.0017657
[Epoch 6, FULL]: 0.0013558
    [Epoch 7, Batch 0]: 0.0013877
    [Epoch 7, Batch 1]: 0.0011317
    [Epoch 7, Batch 2]: 0.0010489
    [Epoch 7, Batch 3]: 0.0018891
    [Epoch 7, Batch 4]: 0.0020820
    [Epoch 7, Batch 5]: 0.0007572
    [Epoch 7, Batch 6]: 0.0007788
[Epoch 7, FULL]: 0.0012965
    [Epoch 8, Batch 0]: 0.0017368
    [Epoch 8, Batch 1]: 0.0009061
    [Epoch 8, Batch 2]: 0.0009467
    [Epoch 8, Batch 3]: 0.0009662
    [Epoch 8, Batch 4]: 0.0013592
    [Epoch 8, Batch 5]: 0.0014016
    [Epoch 8, Batch 6]: 0.0010049
[Epoch 8, FULL]: 0.0011888
    [Epoch 9, Batch 0]: 0.0010400
    [Epoch 9, Batch 1]: 0.0014242
    [Epoch 9, Batch 2]: 0.0011576
    [Epoch 9, Batch 3]: 0.0008646
    [Epoch 9, Batch 4]: 0.0011597
    [Epoch 9, Batch 5]: 0.0011647
    [Epoch 9, Batch 6]: 0.0012795
[Epoch 9, FULL]: 0.0011557
Training complete!
```

29.7 测试数据集

接着下载测试集，确认能在新数据上评价训练后的网络。这里只用与训练具有不同 Mach 数和物理时间的序列：仿真 ID22，其 Mach 数应大于训练采用的 $\mathrm{Ma} = 0.72$。

```
# delete loaders if existing to ensure the dataset file is acessible
try:
    del trainLoader
except:
    pass
try:
    del testLoader
except:
    pass
```

```
# test set configuration
testLoader = Dataloader(
    "transonic-cylinder-flow",
    time_steps=119, # leads to 120 timesteps that are later strided by a factor of 2
    ↪  (for dt compatibility with pre-trained model)
    intermediate_time_steps=True,
    step_size=120,
    sel_sims=[22], # select simulations
    trim_start=600, # discard some initial timesteps from the simulation
    normalize_data="mean-std", # data normalization
    normalize_const="mean-std", # constants normalization
    batch_size=1,
    shuffle=False,
    drop_last=False,
    num_workers=0,
)
```

原书输出：

```
download completed ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100%6m
Success: Loaded transonic-cylinder-flow with 41 simulations (1 selected) and 2 samples
↪  each.
```

29.8　测试推断

现在可以采样训练后的扩散模型，概率预测测试流动轨迹。将预测与真值存成形状为

$$\text{samples} \times \text{sequences} \times \text{sequenceLength} \times \text{channels} \times \text{sizeX} \times \text{sizeY}$$

的张量，用于以下评价和可视化。

```
device = "cuda" if torch.cuda.is_available() else "cpu"
numSamples = 5
diffusionSteps = 20
try: # load model if not trained/finetuned above
    model
except NameError:
    condChannels = 2 * 5 # two timesteps with 5 channels each (vel_x, vel_y, dens,
    ↪  pres, mach)
    dataChannels = 5 # one timestep
    model = DiffusionModel(diffusionSteps, condChannels, dataChannels)
    # load weights from checkpoint
    loaded = torch.load("models/models_tra/128_acdm-r20_02/Model.pth",
    ↪  map_location=torch.device('cpu'))
    model.load_state_dict(loaded["stateDictDecoder"])
model.eval()
model.to(device)
# sampling loop
```

```
print("\nStarting sampling...")
gt = []
pred = []
with torch.no_grad():
    for s, sample in enumerate(testLoader, 0):
        input, target = sample
        input = input.unsqueeze(1) # add timestep dimension
        mach = input[:,:,-1:].expand(-1, target.shape[1], -1, -1, -1) # extract mach
        ↪  number from input
        target = torch.concat([target, mach], dim=2) # add mach number to target
        data = torch.concat([input,target], dim=1).to(device).float() # combined
        ↪  [batch, timesteps, channels, x, y] tensor
        data = data[:,::2] # temporal stride of 2 for dt compatiblity with pre-trained
        ↪  model
        gt += [data.unsqueeze(0).cpu().numpy()]
        d = data.to(device).repeat(numSamples,1,1,1,1) # reuse batch dim for samples
        prediction = torch.zeros_like(d, device=device)
        inputSteps = 2
        for i in range(inputSteps): # no prediction of first steps
            prediction[:,i] = d[:,i]
        for i in range(inputSteps, d.shape[1]):
            cond = []
            for j in range(inputSteps,0,-1):
                cond += [prediction[:, i-j : i-(j-1)]] # collect input steps
            cond = torch.concat(cond, dim=2) # combine along channel dimension
            result = model(conditioning=cond, data=d[:,i-1:i]) # auto-regressive
            ↪  inference
            result[:,:,-1:] = d[:,i:i+1,-1:] # replace mach number prediction with true
            ↪  values
            prediction[:,i:i+1] = result
        prediction = torch.reshape(prediction, (numSamples, -1, d.shape[1], d.shape[2],
        ↪  d.shape[3], d.shape[4]))
        pred += [prediction.cpu().numpy()]
        print(" Sequence %d finished" % s)
print("Sampling complete!\n")
gt = np.concatenate(gt, axis=1)
pred = np.concatenate(pred, axis=1)
print("Ground truth and prediction tensor with shape:")
print("(samples, sequences, sequenceLength, channels, sizeX, sizeY)")
print("GT: %s" % str(gt.shape))
print("Prediction: %s" % str(pred.shape))
```

原书输出：

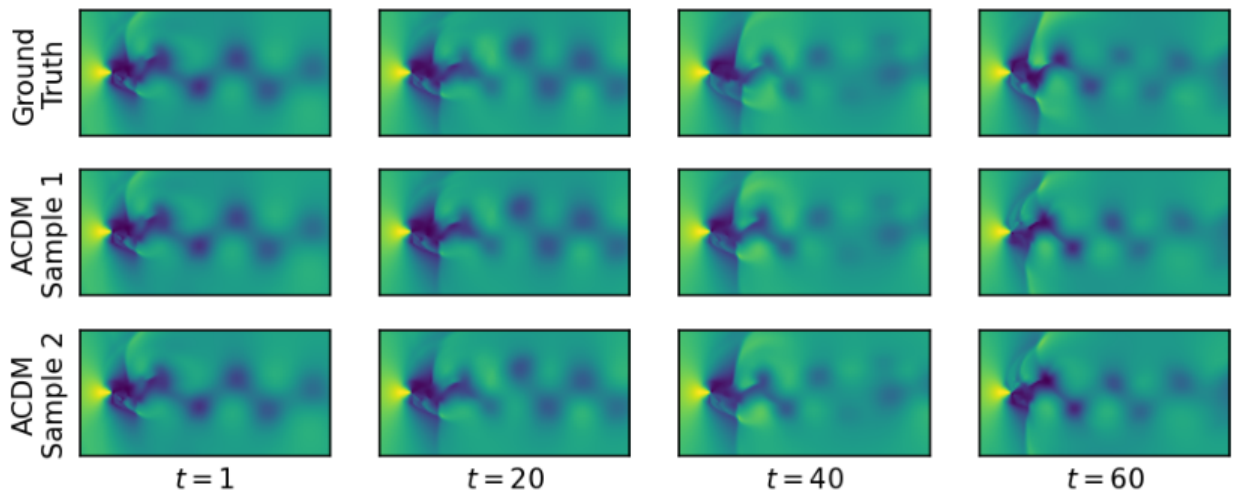
```
Starting sampling...
  Sequence 0 finished
  Sequence 1 finished
Sampling complete!
```

```
Ground truth and prediction tensor with shape:
(samples, sequences, sequenceLength, channels, sizeX, sizeY)
GT: (1, 2, 60, 5, 128, 64)
Prediction: (5, 2, 60, 5, 128, 64)
```

29.9 预测精度

采样后分析真值轨迹和 ACDM 样本。先直接绘图作定性检查。

```
sequence = 0
samples = [0,4]
timeSteps = [0,19,39,59]
field = 3 # velocity_x (0), velocity_y (1), density (2), or pressure (3)
predPart = pred[samples]
gtPred = np.concatenate([gt[:,sequence,timeSteps,field],
↪  predPart[:,sequence,timeSteps,field]])
fig, axs = plt.subplots(nrows=gtPred.shape[0], ncols=gtPred.shape[1],
↪  figsize=(gtPred.shape[1]*1.9, gtPred.shape[0]), dpi=150, squeeze=False)
for i in range(gtPred.shape[0]):
    for j in range(gtPred.shape[1]):
        if i == gtPred.shape[0]-1:
            axs[i,j].set_xlabel("$t=%s$" % (timeSteps[j]+1), fontsize=10)
        if j == 0:
            if i == 0:
              axs[i,j].set_ylabel("Ground\nTruth", fontsize=10)
            else:
              axs[i,j].set_ylabel("ACDM\nSample %d" % i, fontsize=10)
        axs[i,j].set_xticks([])
        axs[i,j].set_yticks([])
        im = axs[i,j].imshow(gtPred[i][j].transpose(), interpolation="catrom",
        ↪  cmap="viridis")
plt.show()
```



训练后的时间算子与早期状态很接近；末态 $t = 60$ 应可看出变化。圆柱激波很不稳定，使网络能生成现实但各异的预测。用不同噪声重新推断可产生更多变化，更长展开还会进一步放大差异。

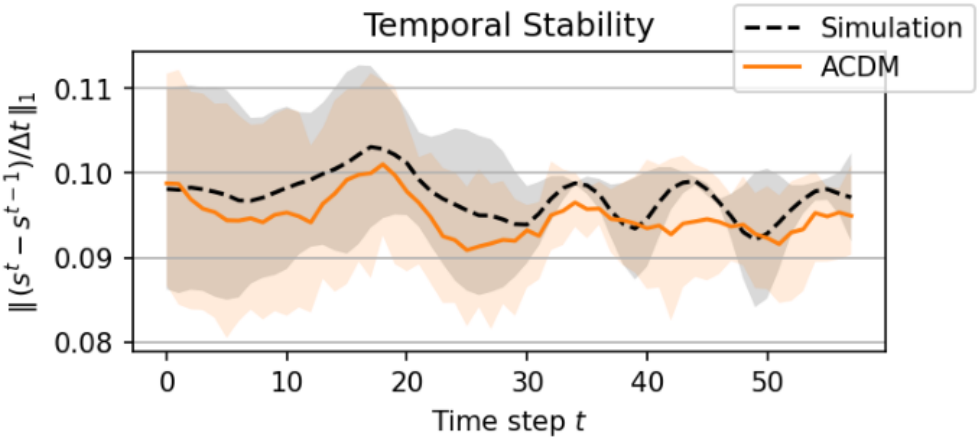
29.9.1 时间稳定性

再计算时间导数并与仿真比较，以研究样本的时间稳定性。序列和样本越多，结果越平滑；增加训练数据也可改善结果。ACDM 后验样本自然应比单条仿真轨迹表现出更大方差。

```
gtTemp = gt[:,:,:,0:4] # ignore scalar Mach number here
predTemp = pred[:,:,:,0:4]
diffGt = np.abs( gtTemp[:,:,1:gtTemp.shape[2]-1] - gtTemp[:,:,2:gtTemp.shape[2]])
diffGt = np.mean(diffGt, axis=(3,4,5)) # channel-wise and spatial mean
minGt = np.min(diffGt, axis=(0,1)) # lower bound over sequences
maxGt = np.max(diffGt, axis=(0,1)) # upper bound over sequences
meanGt = np.mean(diffGt, axis=(0,1)) # sample- and sequence mean
diffPred = np.abs( predTemp[:,:,1:predTemp.shape[2]-1] -
↪  predTemp[:,:,2:predTemp.shape[2]])
diffPred = np.mean(diffPred, axis=(3,4,5)) # channel-wise and spatial mean
minPred = np.min(diffPred, axis=(0,1)) # lower bound over samples and sequences
maxPred = np.max(diffPred, axis=(0,1)) # upper bound over samples and sequences
meanPred = np.mean(diffPred, axis=(0,1)) # sample- and sequence mean
fig, ax = plt.subplots(1, figsize=(5,2), dpi=150)
ax.set_title("Temporal Stability")
ax.set_ylabel("$\Vert \, (s^{t} - s^{t-1}) / \Delta t \, \Vert_1$")
ax.yaxis.grid(True)
ax.set_xlabel("Time step $t$")
ax.plot(np.arange(meanGt.shape[0]), meanGt, color="k", label="Simulation",
↪  linestyle="dashed")
ax.fill_between(np.arange(meanGt.shape[0]), minGt, maxGt, facecolor="k", alpha=0.15)
ax.plot(np.arange(meanPred.shape[0]), meanPred, color="tab:orange", label="ACDM")
ax.fill_between(np.arange(meanPred.shape[0]), minPred, maxPred,
↪  facecolor="tab:orange", alpha=0.15)
fig.legend()
plt.show()
```

原书输出：

```
<>:19: SyntaxWarning: invalid escape sequence '\V'
<>:19: SyntaxWarning: invalid escape sequence '\V'
/tmp/ipykernel_917142/1134808229.py:19: SyntaxWarning: invalid escape sequence '\V'
  ax.set_ylabel("$\Vert \, (s^{t} - s^{t-1}) / \Delta t \, \Vert_1$")
```



[5]

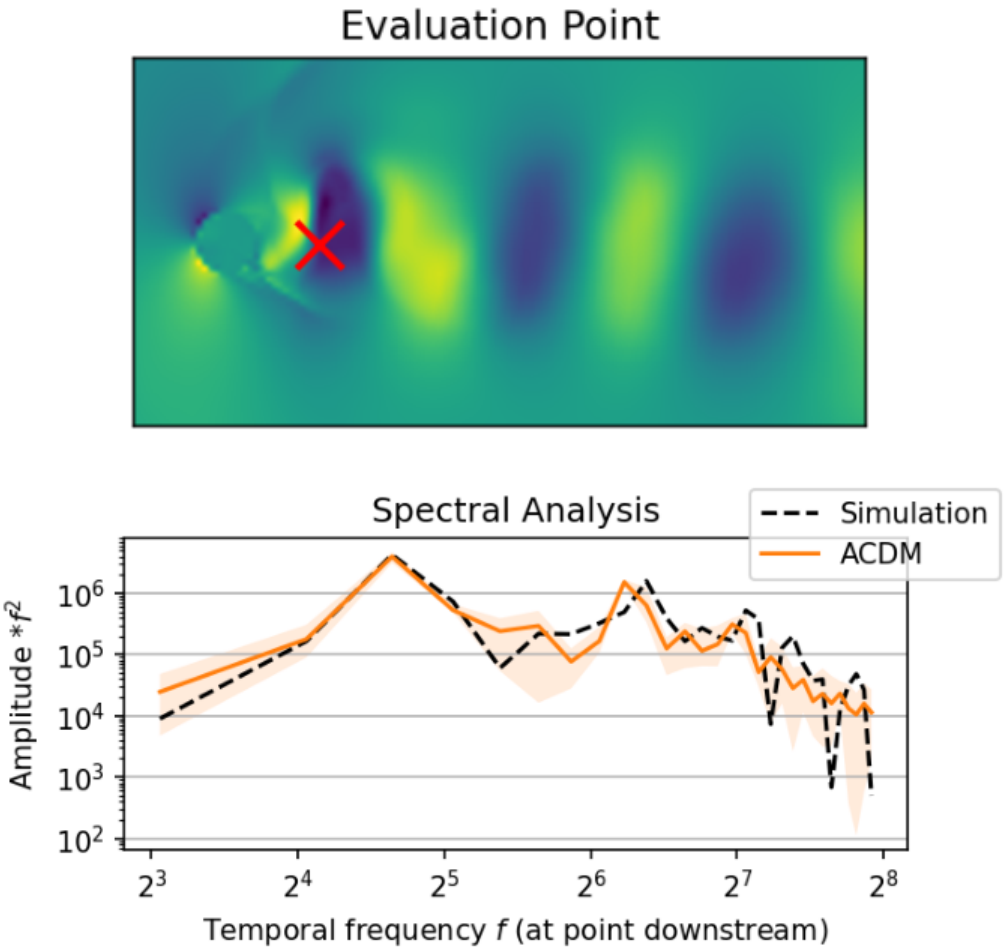
29.9.2　谱分析

最后在圆柱下游一点做频率分析，比较真值与 ACDM 预测的谱。

```
sequence = 0
fracX = 0.25 # closely behing the cylinder
fracY = 0.5 # vertically centered
field = 1 # velocity_x (0), velocity_y (1), density (2), or pressure (3)
posX = int(fracX * gt.shape[4])
posY = int(fracY * gt.shape[5])
gtPred = np.concatenate([gt[:,sequence,:,field, posX, posY], pred[:,sequence,:,field,
↪  posX, posY]])
fft = np.fft.fft(gtPred, axis=1)
fft = np.real(fft * np.conj(fft))
n = fft.shape[1]
gridSpacing = 0.002 # delta t between frames from simulation
freq = np.fft.fftfreq(n, d=gridSpacing)[1:int(n/2)]
fft = fft[:,1:int(n/2)] # only use positive fourier frequencies
gtFFT = fft[0]
minPredFFT = np.min(fft[1:], axis=0) # lower bound over samples
maxPredFFT = np.max(fft[1:], axis=0) # upper bound over samples
meanPredFFT = np.mean(fft[1:], axis=0) # sample mean
# plot eval point
fig, ax = plt.subplots(1, figsize=(5,2), dpi=150)
ax.set_title("Evaluation Point")
ax.imshow(gt[0,sequence,0,field].transpose(), interpolation="catrom", cmap="viridis")
ax.scatter(posX, posY, s=200, color="red", marker="x", linewidth=2)
ax.set_xticks([])
ax.set_yticks([])
```

[5]译注：此图计算空间、通道平均的相邻帧绝对差；代码没有显式除以物理 Δt，并跳过首对帧。此前 $\Delta t = 1$ 的归一化可解释该量，但不能据此证明任意长时间稳定性。所画最小/最大值为有限样本包络，不是置信区间。

```
plt.show()
# plot spectral analysis

fig, ax = plt.subplots(1, figsize=(5,2), dpi=150)
ax.set_title("Spectral Analysis")
ax.set_xlabel("Temporal frequency $f$ (at point downstream)")
ax.set_ylabel("Amplitude $*f^2$")
ax.set_xscale("log", base=2)
ax.set_yscale("log", base=10) # NOTE: y-axis values are not physical as data
↪  normalization is not reversed
ax.yaxis.grid(True)
ax.plot(freq, gtFFT * (freq**2), color="k", label="Simulation", linestyle="dashed")
ax.plot(freq, meanPredFFT * (freq**2), color="tab:orange", label="ACDM")
ax.fill_between(freq, minPredFFT * (freq**2), maxPredFFT * (freq**2),
↪  facecolor="tab:orange", alpha=0.15)
fig.legend()
plt.show()
```



谱图显示训练后的 ACDM 在低频和高频都与真值仿真（黑虚线）匹配。仿真在末尾有几个向下尖峰，

大多被学习的预测器平滑，但预测器仍很好地捕捉了真值统计。[6]

29.10 扩散模型时间预测小结

上述代码得到一个 PDE 时间预测概率模型。其优点是估计数据集中的变化与不确定性，在推断时再现它们；因而能随时间采样后验，多次运行推断不同可能的解。另一方面，扩散模型预测平均解通常并不优于经典方法（详细评价见 ACDM 基准）。所以若数据中的输入输出关系唯一，扩散模型不会带来收益，只会增加推断计算。以上网络也是如此：网络更贵，且需反复运行才能产生一个样本。还可加速，例如用流匹配（这里用去噪），但仍会保留一定的小倍率开销。作者认为，对多数非平凡数据集，扩散模型仍值得使用：数据中的模糊性不会被平均掉，而会作为分布来处理和再现。此外，前文提示的无条件稳定性非常有趣。对于合适的学习任务，训练模型不会随时间发散，也不会把输入变为平凡稳态；这两种情况在其他训练方法中都常见。相反，扩散网络可以在任意长展开中保持参考数据的统计。要证明其永不发散很难，但作者测试中稳定网络在数十万展开步内没有发散。这种吸引人的现象表明扩散模型具有根本不同的行为。下一章将进一步说明，并与时间展开比较。[7]

译注：独立核对的适用条件 一阶 Markov 只要求给定当前状态后不再依赖更早历史，转移仍可随机，并不等同于确定性。式 (29.1) 是状态预测 L2 示意；实际训练用噪声/预测噪声间的 smooth_l1 (Huber) 损失，并非同一字面目标，Huber 的最优也不能无条件等同噪声 MSE 的条件均值。

[6] 译注：源代码未逆归一化，纵轴不是物理单位；画的是未归一化 FFT 模方乘 f^2，不是直接的振幅。仅用单个位置、一个真值序列及有限样本，不能由此推定完整时空分布相同。

[7] 译注：这里保留作者的经验性评价及“任意长”措辞；已报告的证据仍为数十万步的有限测试，未构成无限时间定理。本译稿未重复这些实验。

第三十章　无条件稳定性

上一章扩散模型时间预测的结果，以及此前可微物理讨论的结果，都清楚表明无条件稳定的网络确实可以实现，其他多项工作也有同样报告。但仍有不少方法似乎难以保持长期稳定。于是一个有趣问题是：获得无条件稳定性需要哪些要素？这里指训练后的网络在任意长展开中保持稳定。是否需要归纳偏置或特殊训练法，还是只要尝试足够多种初始化？我们的设置为澄清此问题提供了良好起点。前章的一些成功案例相当简单，说明对神经网络预测器而言，无条件稳定性并非“特殊”性质。除适当的学习设置（合适超参数、足够大的模型及足够数据）外，并不要求特殊损失或技巧。误差会累积，故网络规模与训练更新总步数应很重要。有趣的是，网络结构似乎并不关键：只要足够大，几乎“任何”结构都能得到稳定展开。以下仅考虑固定长度的时间步。“无条件稳定性”指任意多次迭代调用的稳定性。网络也可能训练为支持可变步长，但这里聚焦反复调用这一稳定性“维度”。[1]

30.1　评价的主要考量

如上一章所示，扩散模型表现极好，可归因于以纯噪声为输入的任务（如去噪或流匹配）。网络结构同样影响较小，只需足够大，以提供收敛的迭代。对于监督或展开训练，可用离散和连续神经算子，如 CNN、U-Net、FNO 和 Transformer。FNO 的结构将解投影到离散频率的子空间，内在地去掉主要驱动不稳定的高频，因此受展开的影响较小（源文另链细节）。卷积等较能保留小尺度细节的算子，则可从展开显著获益，以下将重点消融研究。批大小与展开时域长度起着关键却相互冲突的作用：小批更好，但最坏会使硬件利用不足、训练很长。展开显著稳定预测，但每次网络更新的计算图更长，资源开销更大。实验表明，在批大小与展开时域的 Pareto 前沿上，可通过训练时尽量长的展开，并采用足以利用 GPU 显存的批大小，找到合适折中。

学习任务　用 ACDM 基准的两类流动分析长展开的时间稳定性：较简单的不可压圆柱绕流 Inc，以及 Reynolds 数 10000 的复杂跨声速尾流 Tra。Inc 训练的 Reynolds 数为 200–900，推断外推至 960、980、1000（Inc-high）。Tra 训练的 Mach 数为 0.53–0.9，测试为 0.50、0.51、0.52（Tra-ext）；该范围含大量与流动相互作用的激波，很有挑战。每个数据集的每条序列上，对各结构的三次训练结果展开 200000 步。此长度不能证明无限长稳定，但作者据此认为发散概率极小。

[1]译注：本章保留作者的术语和经验概括；这与数值分析中“对所有步长稳定”的用法不同。下述 20 万步有限测试不能证明任意无限时间稳定，也不能由未观察到发散直接确定其概率。

30.2 结构比较

首先比较三个采用相同 U-Net 骨干、稳定化技术不同的网络，说明可通过不同方式实现“无条件稳定性”：

- 展开训练 U-Net-ut：训练时通过多个时间步反向传播梯度。
- 带训练噪声的单步预测 U-Net-tn：加入噪声模拟推断累积的误差，减少数据偏移，已知可改善稳定性。
- 自回归条件扩散模型 ACDM：以前一时间步为条件，迭代细化噪声，生成下一步，见基于扩散的时间预测一章。

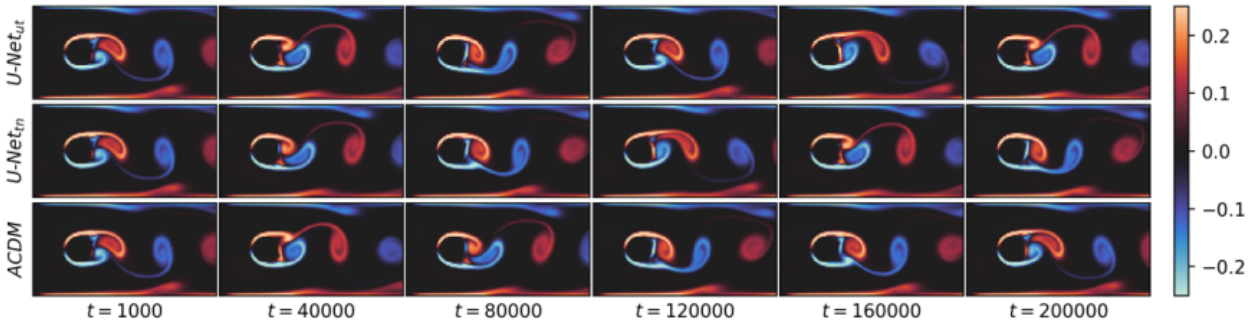


图 30.1: Reynolds 数 1000 的不可压流动在 200000 步中的涡量预测（Inc-high）。

图中所有方法及训练运行在整个 Inc-high 展开中均保持稳定。这种流动虽非定常，却完全周期，各网络都产生简单周期轨迹，避免误差累积。本例说明简单任务的长期稳定较少成为问题；网络较易将预测保持在解流形内。接着看更难的 Tra 激波流动。Tra-ext 测试序列上，三个 U-Net-tn 网络中有一个在最初几千步内出现稳定性问题，退化为无涡的简单平均流。展开训练 U-Net-ut 和 ACDM 在各序列及训练运行中均稳定，说明更能抵御通常导致不稳定的展开误差。自回归扩散模型在全部测试中均稳定（源文另链细节），以下不再评价它，转而关注较难稳定的 U-Net，作为卷积离散神经算子的代表。

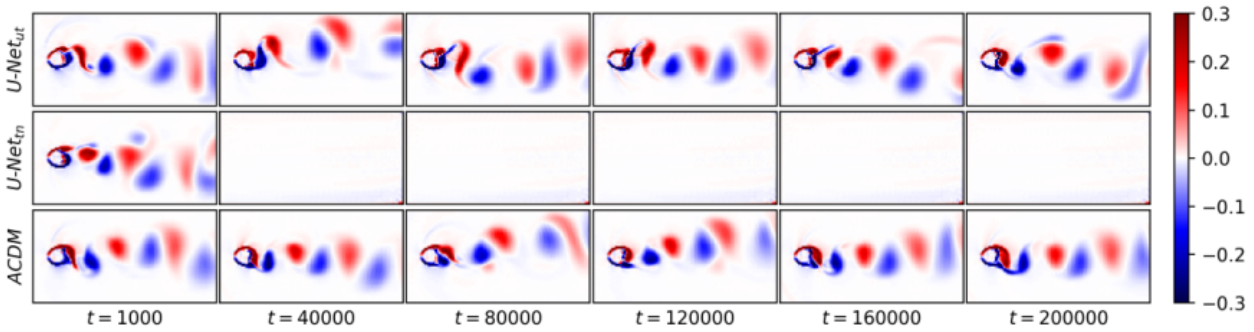


图 30.2: Mach 数 0.52 的跨声速流动 200000 步涡量预测；Tra-ext 位于训练范围之外。

30.3 稳定性判据

针对展开训练的 U-Net，每种设置训练三个模型，测量稳定运行的百分比，以获得比单个定性示例更充分的统计。先研究展开长度如何影响极长时域稳定性。图中列出 Tra-ext 上 200000 步展开的稳定比例，包括各 Mach 数及顶部平均值。源文此处称 “Figure 2”，所指实为下图。不同 Mach 数的泛化测

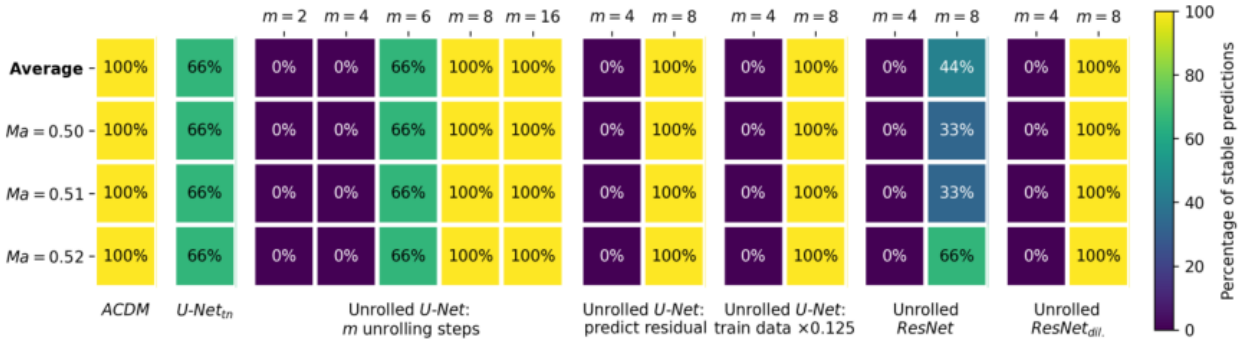


图 30.3: 展开训练不同消融设置在 Tra-ext 数据集上的稳定运行比例。

试不改变结果。最重要的稳定性因素是展开步数 m：$m \leq 4$ 的网络始终无法获得稳定展开，而 $m \geq 8$ 足以使不同 Mach 数上的结果稳定。

影响可忽略的方面 实验中预测策略、训练数据量和骨干结构三项未显著影响稳定性。第一，预测相对前一步的差值而非完整下一状态的残差预测，没有影响稳定性。第二，把每个 Mach 数的训练数据由 1000 步降至 125 步，即减少 8 倍，同时将 epoch 增至 8 倍以公平比较，也未影响稳定性；减少后的数据仍含完整涡脱落周期，保留了全部物理行为。第三，扩张 ResNet 等其他骨干也可经展开训练得到完全稳定结果。无扩张 ResNet 只有一个训练网络稳定，最可能因感受野较小；作者预期延长训练展开时域也可使其完全稳定。

30.4 批大小与展开

批大小会显著影响自回归网络稳定性。这类似图像领域中小批改善泛化的现象，也是使用小批而非全数据梯度的动机。下图展示它对两数据集稳定性及训练时间的影响。大批时接近理想展开长度的网络，可用小批稳定。但不能完全取代展开：不展开的网络在所有测试批大小下均不稳定。Inc 任务中，将 U-Net 每层宽度相对上述设置减小 8 倍，人为增加难度；否则所有参数设置均已稳定，就无法展示批大小的影响。[2] 两数据集上增大批都会因计算的内存效率较低而增加训练时间。延长训练展开不一定延长时间，因为每个 epoch 会相应减少更新次数：例如展开 4 步用 250 批，展开 8 步用 125 批，使训练所见仿真状态总量不变。但 Tra-ext 上较大 U-Net 实际仍有额外开销。因此核心问题是：哪种展开长度与批大小组合效率最高？下图给出二者的主要折中，只含稳定设置。要获得无条件稳定的网络和神经算子，始终有利的选择是长展开，配合足以利用 GPU 显存的批大小。即延长训练展开比减小批更高效地改善稳定性，如训练时间最短的绿点所示。

[2]译注：源文最后一句字面称 “would already be stable and show the effect”；按本段因果语境应为无法展示变化的效果，此处明确记录该措辞疑点。

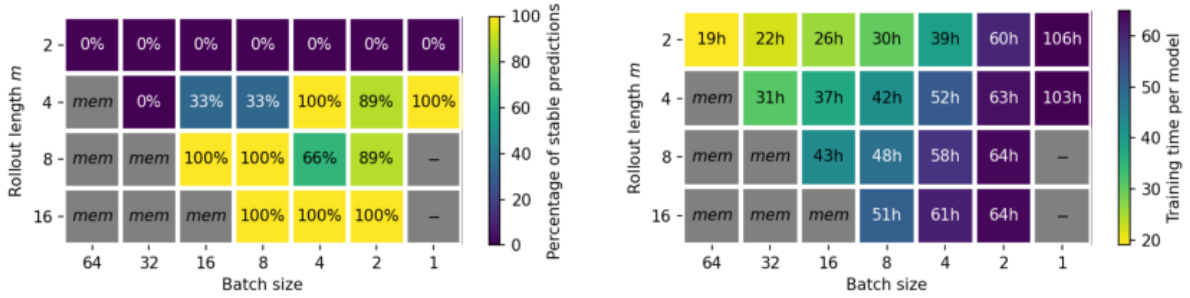


图 30.4: Tra-ext 中不同展开长度和批大小的稳定运行比例及训练时间。灰色为显存限制（mem）或计算开销过大（-）而未做的设置。

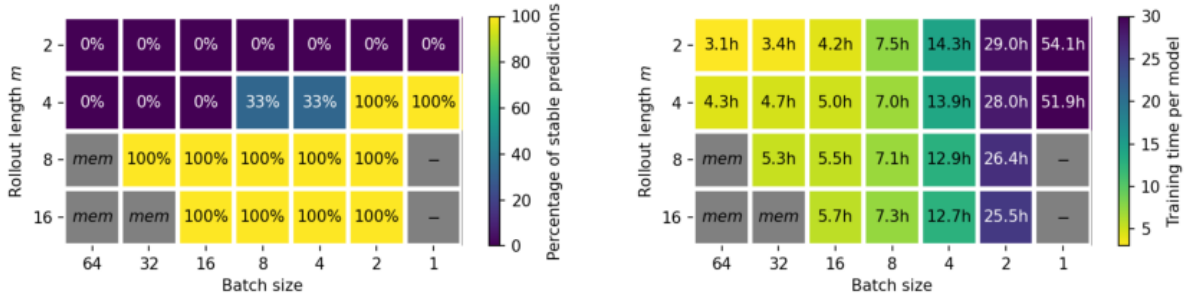


图 30.5: Inc-high 中不同展开长度和批大小的稳定运行比例及训练时间。灰色同样表示显存不足（mem）或计算开销过大（-）。

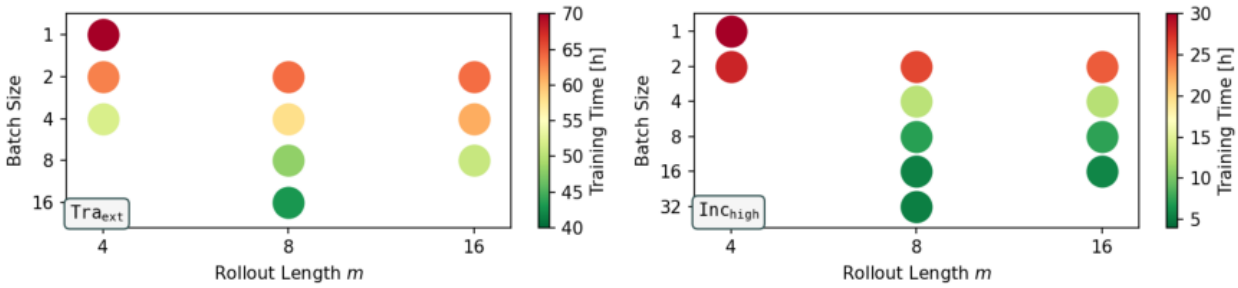


图 30.6: Tra-ext（左）与 Inc-high（右）不同展开长度和批大小的训练时间。仅含高度稳定的设置（稳定运行比例 $\geq 89\%$）。

30.5 小结

适当训练设置下，即使复杂流动也能获得极长展开的无条件稳定预测。根据实验，最重要的因素是是否采用基于扩散的训练。不使用扩散时需考虑：

- 训练时使用长展开；
- 小批大小；
- 比较二者，长展开更好，比减小批可获得更快训练；
- 同时网络需足够大，具体依学习任务复杂度而定。

未显著影响长期稳定性的因素为：

- 训练预测范式：残差与直接预测都可行；
- 不含新物理行为的额外训练数据；
- 不同网络结构，但各结构理想展开步数可能不同。

至此结束 “无条件稳定性” 讨论。实验更多细节见 ACDM 论文。

译注：独立核对的适用条件 正文称增大批会增加训练时间，但表中固定展开长度的趋势相反：Tra 的 $m = 2$ 从 batch64 到 1 为 19h 到 106h，Inc 为 3.1h 到 54.1h。原句与表照录，不把该句当成表中结论。$m \le 4$ 不稳定、$m \ge 8$ 稳定也只指前一基线消融；Tra 的 $m = 4$、batch4/1 已达 100%，而 $m = 8$、batch4/2 为 66%/89%；Inc 的 $m = 4$、batch2/1 也达 100%。稳定性与任务、批大小耦合，8 步不是普适必要或充分阈值。

第三十一章　基于图的扩散模型

与经典数值方法一样，规则网格对某些问题理想，对另一些却并非最优。扩散模型也如此；好在将规则网格换成图后，前章概念仍然适用。下面将展示，去噪和流匹配在非结构化 Euler 网格上也同样有效。本例还说明扩散模型擅长补全数据分布：即使单个实例（由物理域几何、边界条件和物理参数定义）的训练分布不完整，跨实例学习的“全局”视角，也能在观察许多实例的部分数据时补全后验分布。许多流动等仿真问题难以用单个平均解表示。例如湍流应用往往需要可能流态的完整分布，以导出均方根（RMS）与两点相关等统计量。这正是扩散模型的强项：无需先模拟漫长瞬态以趋近平衡，而可跳过预热，直接生成目标样本，从而比经典求解器更高效地计算相关统计。[1]

31.1　扩散图网络（DGN）

以下用扩散图网络 DGN [LPT25] 展示这些能力；源文另链完整源码。为学习由离散网格和物理参数确定的系统动态状态分布，可直接在网格节点上使用 DDPM 和流匹配。DGN 还引入第二种变体，不直接在物理空间，而在预训练的语义潜空间中运行，称为 LDGN。系统几何用节点集 $\mathcal{V}_M$、边集 $\mathcal{E}_M$ 的网格表示，节点 i 位于 x_i。时刻 t 的状态由网格节点上采样的 F 个连续场定义：

$$Y(t) := \{y_i(t) \in \mathbb{R}^F \mid i \in \mathcal{V}_M\}, \qquad\qquad y_i(t) \equiv y(x_i, t).$$

仿真器从初态 $Y(t_0)$ 出发，演化出状态序列 $\mathcal{Y} = \{Y(t_0), Y(t_1), \ldots, Y(t_n), \ldots\}$。假定系统在初始瞬态后达到统计平衡：足够长时间区间计算的 Y 的统计量不随时间变化，即使动力学振荡或混沌。平衡阶段状态 $\mathcal{Z} \subset \mathcal{Y}$ 只依赖几何和物理参数，不依赖初态，见下图。[2] 许多空气动力学和结构振动等工程应用主要关心系统动力学的统计，而非轨迹上的每个状态。为准确捕捉统计，模拟足够长的收敛状态轨迹 $\mathcal{Z}$ 可能很贵，尤其三维混沌系统。DGN 旨在不模拟初始瞬态而直接采样收敛态 $Z(t) \in \mathcal{Z}$，再用足够多样本分析动力学。给定 N 个系统的短轨迹集 $\mathfrak{Z} = \{\mathcal{Z}_1, \mathcal{Z}_2, \ldots, \mathcal{Z}_N\}$，目标是学习其概率模型，以网格、边界条件和物理参数为条件采样收敛态 $Z(t) \in \mathcal{Z}$。即使各训练轨迹短到不足以刻画自身统计，模型仍须捕捉底层分布。问题虽不适定，但训练轨迹充分时，图扩散模型能发现统计相关和共享模式，在条件空间中插值。

31.2　图上的扩散

用 DDPM（后用流匹配）生成 $Z(t)$：从各向同性 Gaussian 分布抽取 $Z^R \in \mathbb{R}^{|\mathcal{V}_M| \times F}$ 并去噪。条件信息编码在有向图 $\mathcal{G} := (\mathcal{V}, \mathcal{E})$ 中，$\mathcal{V} \equiv \mathcal{V}_M$，网格边 $\mathcal{E}_M$ 表为双向图边 $\mathcal{E}$。节点属性 $V_c = \{v_i^c \mid i \in \mathcal{V}\}$

[1] 译注：这些是本章方法的目标和所示实验表现；部分分布能否补全取决于跨系统结构及训练覆盖，不是任意缺失分布可唯一恢复的定理。

[2] 译注：初态无关性是此建模设置的假设；一般多吸引子或非遍历系统不能由“统计平衡”一词自动推出该结论。

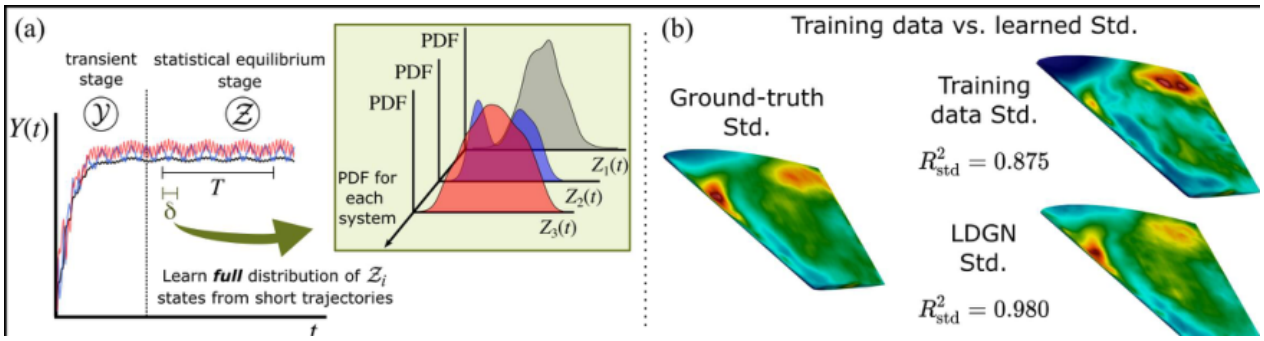


图 31.1: (a) 每系统仅提供长为 $\delta \ll T$ 的短轨迹，DGN 仍学习收敛态分布。(b) 湍流机翼示例：虽训练时每翼只见不完整分布（右上），LDGN 所学分布仍准确捕捉全部状态的方差（右下）。

与边属性 $E_c = \{e_{ij}^c \mid (i,j) \in \mathcal{E}\}$ 编码条件特征，包括相邻位置差 $x_j - x_i$。正向扩散逐次加入 Gaussian 噪声，由 $Z^1 \in \mathbb{R}^{|\mathcal{V}| \times F}$ 生成至 Z^R：

$$q(Z^r \mid Z^{r-1}) = \mathcal{N}\big(Z^r; \sqrt{1-\beta_r}\, Z^{r-1}, \beta_r I\big),$$

其中 $\beta_r \in (0,1)$，$Z^0 \equiv Z(t)$。任意 Z^r 可直接抽样：

$$Z^r = \sqrt{\bar{\alpha}_r}\, Z^0 + \sqrt{1-\bar{\alpha}_r}\, \epsilon, \qquad \alpha_r := 1-\beta_r, \qquad \bar{\alpha}_r := \prod_{s=1}^{r} \alpha_s, \qquad \epsilon \sim \mathcal{N}(0, I).$$

去噪通过学习的 Gaussian 转移移除噪声，$p_\theta(Z^{r-1} \mid Z^r) = \mathcal{N}(Z^{r-1}; \mu_\theta^r, \Sigma_\theta^r)$，其中

$$\begin{aligned}
\mu_\theta^r &= \frac{1}{\sqrt{\alpha_r}} \left(Z^r - \frac{\beta_r}{\sqrt{1-\bar{\alpha}_r}} \epsilon_\theta^r \right), \\
\Sigma_\theta^r &= \exp\left(\mathbf{v}_\theta^r \log \beta_r + (1 - \mathbf{v}_\theta^r) \log \tilde{\beta}_r \right), \\
\tilde{\beta}_r &:= \frac{1-\bar{\alpha}_{r-1}}{1-\bar{\alpha}_r} \beta_r.
\end{aligned}$$

这里 $\epsilon_\theta^r \in \mathbb{R}^{|\mathcal{V}| \times F}$ 预测上式的噪声 ϵ（源文称 “equation (1)”，本章原 PDF 未显示该式号），$\mathbf{v}_\theta^r \in \mathbb{R}^{|\mathcal{V}| \times F}$ 在过程熵的两个界 β_r 与 $\tilde{\beta}_r$ 之间插值。[3] DGN 用常规消息传递 GNN [SGGP+20] 预测 ϵ_θ^r 与 $\mathbf{v}_\theta^r$，输入为 Z^{r-1}，条件为图、节点/边特征与扩散步：

$$[\epsilon_\theta^r, \mathbf{v}_\theta^r] \leftarrow \mathrm{DGN}_\theta(Z^{r-1}, \mathcal{G}, V_c, E_c, r).$$

[4] 网络用 Nichol 等 “Improved Denoising Diffusion Probabilistic Models” 提出的混合损失训练。完整去噪需计算 DGN 共 R 次，将 Z^R 变为 Z^0。DGN 遵循编码器–处理器–解码器 GNN 结构。除节点、边编码器外，还含扩散步编码器，生成 $r_{\mathrm{emb}} \in \mathbb{R}^{F_{\mathrm{emb}}}$。节点编码器处理条件特征 v_i^c 和此嵌入：

$$\begin{aligned}
r_{\mathrm{emb}} &\leftarrow \phi \circ \mathrm{Linear} \circ \mathrm{SinEmb}(r), \\
v_i &\leftarrow \mathrm{Linear}\big([\phi \circ \mathrm{Linear}(v_i^c) \mid r_{\mathrm{emb}}]\big), \qquad i \in \mathcal{V}.
\end{aligned}$$

[3]译注：源式按元素指数给出方差参数，对应对角协方差的简写。$r=1$ 时 $\tilde{\beta}_1 = 0$，其对数需要实现中的端点处理；源文此处未说明，不能把显示式无条件用于该端点。若称 “插值”，权重还需相应范围条件，源文仅给出实值向量。

[4]译注：源文此处两次把输入写成 Z^{r-1}，照录；若用于上述 $p_\theta(Z^{r-1} \mid Z^r)$，通常应输入当前噪声态 Z^r。这是源索引冲突，不静默修改。

ϕ 为激活函数，SinEmb 为正弦嵌入。边编码器对 e_{ij}^c 用线性层。编码后的节点和边特征为 $\mathbb{R}^{F_h}$ 向量，$F_{\text{emb}} = 4F_h$。每层消息传递前，将 r_{emb} 投影到 F_h 维并加到节点特征，即 $v_i \leftarrow v_i + \text{Linear}(r_{\text{emb}})$。每层为

$$\begin{aligned}
\mathbf{e}_{ij} &\leftarrow W_e \mathbf{e}_{ij} + \text{MLP}_e\big(\text{LN}([\mathbf{e}_{ij} \mid \mathbf{v}_i \mid \mathbf{v}_j])\big), && (i,j) \in \mathcal{E}, \\
\bar{\mathbf{e}}_j &\leftarrow \sum_{i \in \mathcal{N}_j^-} \mathbf{e}_{ij}, && j \in \mathcal{V}, \\
\mathbf{v}_j &\leftarrow W_v \mathbf{v}_j + \text{MLP}_v\big(\text{LN}([\bar{\mathbf{e}}_j \mid \mathbf{v}_j])\big), && j \in \mathcal{V}.
\end{aligned}$$

此前图扩散工作用顺序消息传递传播节点特征，但对这里的流动等大尺度现象，全局去噪会受消息传递有限范围限制。因此处理器采用多尺度 GNN，以 U-Net 方式在 $\mathcal{G}$ 及其多个粗化图上传递消息，利用 U-Net 移除高、低频噪声的能力。每个低分辨率图用 Guillard 粗化算法从高分辨率图得到；该算法原为 CFD 快速网格粗化开发。与常规 U-Net 一样，用池化与反池化在不同分辨率间转换，这里二者也基于消息传递。

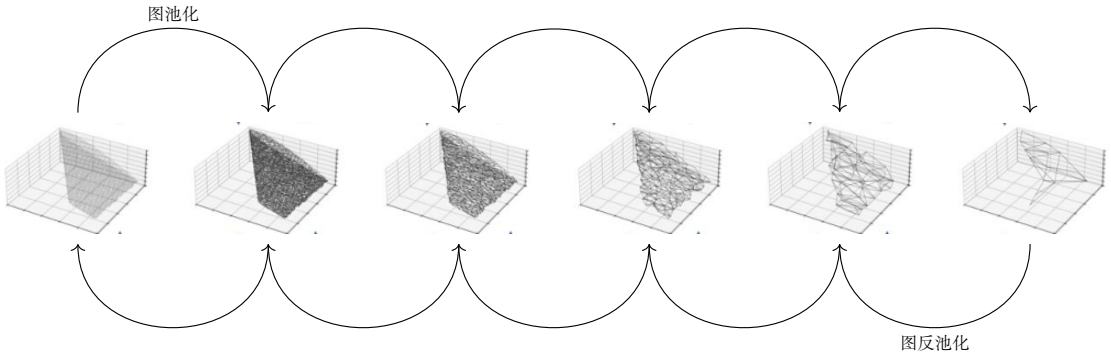


图 31.2: 以 U-Net 方式在 $\mathcal{G}$ 及多个粗化图上消息传递。低分辨率图由 CFD 中推广的网格粗化算法得到。

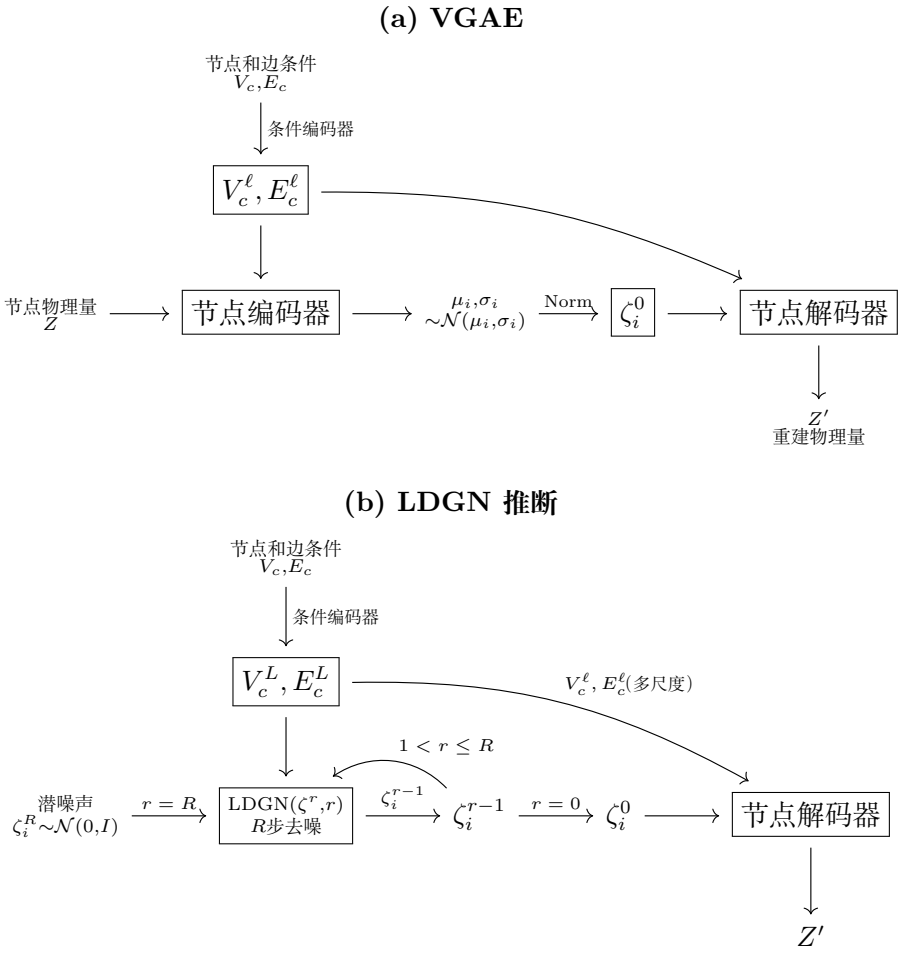
31.3 潜空间中的扩散

扩散模型也可作用在与 3 感知等价的低维图表示上。这是训练来重建 $Z(t)$ 的变分图自编码器 VGAE 的潜空间；在其中训练的 DGN 称 LDGN。VGAE 捕捉空间梯度和小涡等高频信息，LDGN 则关注尾流、涡街等中大尺度模式。解耦两任务可简化生成学习，使 LDGN 专注对小尺度涨落不敏感、更有意义的潜表示。推断时，VGAE 解码器还可移除 LDGN 样本的残余噪声。LDGN 作用于更小的图，显著降低采样开销。VGAE 采用编码器–解码器结构，另有条件编码器处理 V_c, E_c，在 L 个图 $\{\mathcal{G}_\ell := (\mathcal{V}_\ell, \mathcal{E}_\ell) \mid 1 \le \ell \le L\}$ 上编码为潜节点特征 V_c^ℓ 与边特征 E_c^ℓ。$\mathcal{G}_1 \equiv \mathcal{G}$，规模递减，$|\mathcal{V}_1| > \cdots > |\mathcal{V}_L|$。先将 V_c, E_c 线性投影到 F_{ae} 维，用两层消息传递得到 V_c^1, E_c^1；再顺序作用 $L-1$ 个编码块：

$$[V_c^{\ell+1}, E_c^{\ell+1}] \leftarrow \text{MP} \circ \text{MP} \circ \text{GraphPool}(V_c^\ell, E_c^\ell), \quad \ell = 1, \dots, L-1.$$

MP 为消息传递层，GraphPool 为图池化层。[5] 编码器为每个 $i \in \mathcal{V}_L$ 输出两个 F_L 维向量：Gaussian 潜分布的均值 μ_i 与标准差 σ_i。输入态 $Z(t)$ 先线性投影到 F_{ae} 维，再过 $L-1$ 个以下采样块（消息传递

[5]译注：源图集公式误含字面 “Id”，此处以其相邻明确的 $1 \le \ell \le L$ 范围排版；不赋予该残留新数学意义。



ζ_i^r：第 r 步的去噪潜变量。

图 31.3: (a)VGAE 含条件编码器、节点编码器和节点解码器；多尺度条件特征同时输入后两者。(b)LDGN 推断在 VGAE 潜空间采样 Gaussian 噪声，以低分辨率条件编码特征为条件多步去噪，再由解码器变换到物理空间。

加图池化），每块均以条件编码器输出为条件：

$$V \leftarrow \mathrm{GraphPool} \circ \mathrm{MP} \circ \mathrm{MP}\big(V + \mathrm{Linear}(V_c^{\ell}), \mathrm{Linear}(E_c^{\ell})\big), \quad \ell = 1, \ldots, L-1;$$

随后为瓶颈块：

$$V \leftarrow \mathrm{MP} \circ \mathrm{MP}\big(V + \mathrm{Linear}(V_c^{L}), \mathrm{Linear}(E_c^{L})\big).$$

输出特征经过逐节点 MLP，返回每个节点的 μ_i, σ_i。潜变量为 $\zeta_i = \mathrm{BatchNorm}(\mu_i + \sigma_i \epsilon_i), \epsilon_i \sim \mathcal{N}(0, I)$。解码器镜像编码器，以图反池化替代池化，上采样回原图 $\mathcal{G}$，各块同样以条件编码器输出为条件。VGAE 中的消息传递、图池化与反池化层都与 (L)DGN 相同。VGAE 训练重建 $Z(t) \in \mathfrak{Z}$，并用 KL 惩罚使潜空间接近标准正态。训练后按第 31.1 节的方法训练 LDGN（源文此处残留未解析的 `Section~\ref{sec:DGN}`）；目标改为学习粗图 $\mathcal{G}_L$ 上潜态 ζ 的分布，以条件编码器的 V_c^L, E_c^L 为条件。推断时条件编码器生成各层 V_c^{ℓ}, E_c^{ℓ}；LDGN 去噪后，解码器将 ζ^0 变回原图上的物理特征。与常规 VGAE 不同，此处必须有条件编码器：推断时 LDGN 在 $\mathcal{G}_L$ 上需要 V_c, E_c 的编码，不能直接用节点编码器生成，因为后者还需未知的 $Z(t)$。也可直接在粗表示 $\mathcal{G}_L$ 上定义条件，但它缺少细节，结果不理想。

31.4 三维机翼周围的湍流

直接用复杂案例说明 DGN 能力；下一页 Jupyter notebook 将研究更基础的案例。DGN 的 Wing 实验研究三维湍流中的机翼，其表面有细致涡旋形成和消散。湍流高维、混沌，且跨广泛尺度相互作用，任务尤其困难。机翼几何的相对厚度、渐缩比、后掠角和扭转角均变化。仿真昂贵，GNN 可将计算集中在翼表面，避免昂贵体积场。规则网格需超过 10^5 个单元，表面网格只约 7000 节点。表面压力可确定气动性能和结构需求，若能快速取得其概率分布，对气动建模很有价值。训练数据用 OpenFOAM 的 PISO 求解器进行脱体涡模拟（DES）生成，取仿真达到统计平衡后不久的连续 250 态；这仅为获得统计平稳方差所需状态量的约 10%，故每例训练视野都很不完整。

31.5 分布精度

每个样本准确不意味着模型学到真实分布；两者常冲突。例如 VGAE 可用 KL 惩罚控制优先提高样本质量还是覆盖模式。用 Wasserstein-2 距离评价状态分布：一是独立考察每个节点后平均，记 W_2^{node}；二是考察全图所有节点联合分布，记 W_2^{graph}。前者反映节点均值、标准差等逐点统计的精度，却不惩罚错误空间相关；后者则会惩罚。为使距离计算稳定，用连续 2500 态表示目标分布，用 3000 样本表示预测分布。训练轨迹足以捕捉平均流，但不足以捕捉标准差、空间相关或高阶统计。尽管如此，DGN 尤其 LDGN 仍能准确学习训练轨迹的完整概率分布，并在分布内外物理设置生成准确的新分布。下图给出定性和相关测量；两种 DGN 都明显优于 Gaussian 混合基线 GM-GNN。潜空间模型的 W_2^{graph} 也最好，为 1.95 ± 0.89；DGN 为 2.12 ± 0.90，Gaussian 混合模型为 4.32 ± 0.86。

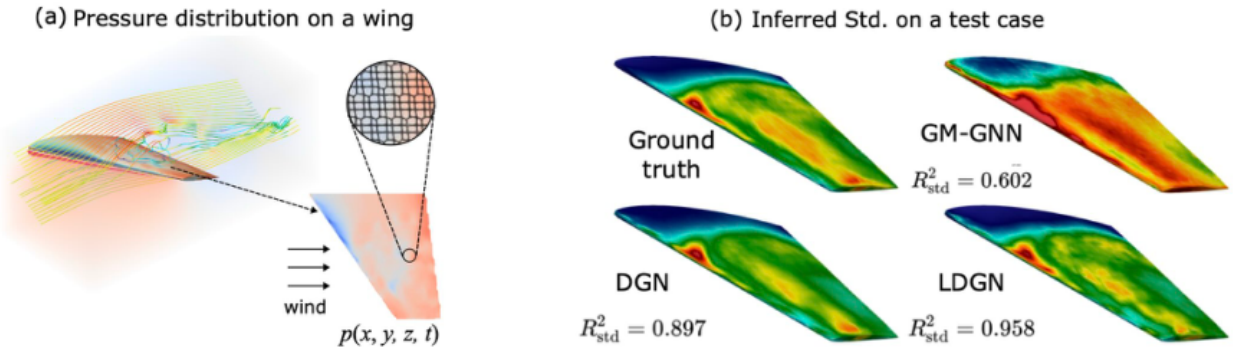


图 31.4: (a)Wing 任务研究三维湍流中机翼压力分布。(b)LDGN 生成分布的标准差最接近真值，此处以相关性展示。

31.6 计算性能

不同实现的运行时间比较总应谨慎。不过 Wing 实验中，真值仿真器以 8 个 CPU 线程运行，初始瞬态加 2500 个平衡态需 2989 分钟，刚够得到良好收敛的方差。LDGN 生成 3000 样本则只需 8 线程 CPU 上的 49 分钟，或单 GPU 上的 2.43 分钟。若只生成单个收敛态（如用作另一仿真器的初态），CPU 与 GPU 加速分别达四和五个数量级。得益于潜空间，LDGN 不仅更准确，采样还比 DGN 快 8 倍，而训练仅多约 55% 时间。这样的效率优势意味着图扩散模型可能特别适用于计算开销原本难以承担的情况。这些结果表明，非结构仿真中的扩散建模向真实工程应用的概率方法迈出重要一步。为具体说明 DGN 及实现，接下来研究可在 Jupyter 中详细分析的简单测试例。[6]

译注：独立核对的适用条件 图 31.3(b) 沿源将最终产生 ζ^0 的箭头标为 $r=0$，但若每步产生 ζ^{r-1}，最后应为 $r=1$；这是源图索引冲突。(a) 的 $\mathcal{N}(\mu,\sigma)$ 第二参数与正文将 σ 作为标准差的用法存在协方差记号歧义。$\beta_r, \tilde{\beta}_r$ 是方差界，显示式在对数方差之间插值，源称“熵的界”不宜按字面当熵值。2500 个连续参考态不等于 2500 个独立样本，3000 个生成样本与标准差 R^2 也不足以证明完整分布精确恢复；$1.95 \pm .89$ 与 $2.12 \pm .90$ 仅报告均值排序，误差范围本身不能建立显著优越性。

[6]译注：时间与加速比均为源实验报告，硬件、样本任务和训练成本口径不同，本译稿未复测；不据此声称普遍加速保证。

第三十二章　扩散图网络的分布精度

本 notebook 展示如何运行并评价训练好的扩散图网络（DGN），并与常用基线比较推断分布的精度。这里故意采用很短的训练轨迹，短到不足以覆盖一个完整振荡周期，因此没有显式提供系统的完整统计信息。源文附 “在 Colab 中运行” 链接。

用部分统计量学习　要展示的关键是：扩散训练能结合多个初始条件和几何的信息，学到完整统计。这种强大能力使其区别于其他概率学习方法。[1] Ellipse 任务是经典流体力学问题：预测椭圆柱周围的压力 p 场。仅关注浸入物体表面的图表示可降低维数；又因使用预训练模型，notebook 较易运行。必须记住，

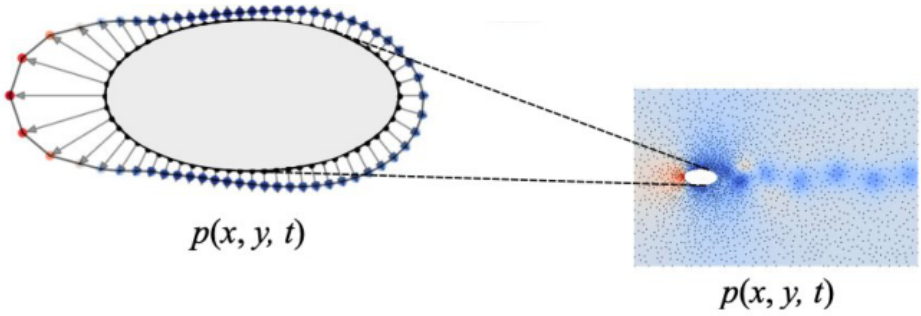


图 32.1: Ellipse 任务预测浸没椭圆障碍物表面的压力分布样本。模型应直接预测表面样本，无需解析远场或初始瞬态。

这里不解析完整时间演化（那是基于扩散的时间预测一章的任务），而是按训练数据高效采样仿真的平衡态。模型作用于描述椭圆几何的图，对时间本身无感，只产生状态在时间上分布的一个样本。这也使逐样本评价更难：不知道生成的是完整分布中的哪一个样本。因此测试需密集采样的真值分布，对每个网络推断样本，在给定几何的测试集中寻找最接近的状态。离散测试样本会留下一点 “离散化误差”，但双方样本足够多时仍可给出良好估计。[2]

32.1　实现

使用 dgn4cfd 代码，先导入基本库并克隆仓库：

```
import numpy as np
```

[1] 译注：此处为所示实验的能力陈述，不是仅凭任意部分统计都可识别完整分布的普遍保证。

[2] 译注：最近真值样本的匹配分数主要评价样本是否落在支持附近，单独不能评价模式概率或完整分布精度；后文另用分布统计与 Wasserstein 距离。

```
import torch, tqdm
from torchvision import transforms
import matplotlib.pyplot as plt
device = torch.device('cuda:0')

try:
    import google.colab # to ensure that we are inside colab
    !pip install --upgrade --quiet torch_geometric bayesian_torch pot
except ImportError:
    print('This notebook is running locally, please make sure the necessary pip
    ↪  packages are installed.')
    pass
```

原书输出：

```
?25l ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/897.5 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 890.9/897.5 kB 44.2 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 897.5/897.5 kB 25.2 MB/s eta 0:00:00
?25h
```

```
# clone respository, note - deactivate for local installs
!git clone https://github.com/tum-pbs/dgn4cfd
%cd dgn4cfd/
import dgn4cfd as dgn
```

原书输出：

```
/content/dgn4cfd
```

为控制运行时间，直接加载仓库中的预训练模型，比较：

- 扩散图网络 DGN；
- 潜扩散图网络 LDGN；
- 用流匹配训练的 DGN，即 FM-DGN；
- 用流匹配训练的 LDGN，即 FM-LDGN。

后两者展示更快的流匹配推断的优势；其他方面与采用 DDPM 的 DGN/LDGN 相同。还加载几种基线。常规图网络与 DGN 结构相同，已含层级等结构改进，可考察仅把监督训练改成扩散建模的得失。普通图网络完全确定，不能给出完整分布。基线包括普通图网络、Bayesian 图网络、Gaussian 混合图网络、变分图自编码器 VGAE。后三者是概率模型，更直接与 DGN 竞争，应也能从训练数据学到后验；下文会看到 DGN 明显更好。

```
# Diffusion Graph Net
DGN = dgn.nn.DiffusionGraphNet(
    checkpoint = "./examples/Ellipse/checkpoints/dgn-nt10.chk",
    device = device,
)
```

```
# Latent Diffusion Graph Net
LDGN = dgn.nn.LatentDiffusionGraphNet(
    autoencoder_checkpoint = "./examples/Ellipse/checkpoints/ae-nt10.chk",
    checkpoint = "./examples/Ellipse/checkpoints/ldgn-nt10.chk",
    device = device,
)

# Vanilla Graph Net
VanillaGN = dgn.nn.VanillaGnn(
    checkpoint = "./examples/Ellipse/checkpoints/vanilla-nt10.chk",
    device = device,
)

# Bayesian Graph Net
BayesianGN = dgn.nn.BayesianGnn(
    checkpoint = "./examples/Ellipse/checkpoints/bayesian-nt10.chk",
    device = device,
)

# Gassian Mixure Graph Net
GaussianMixGN = dgn.nn.GaussianMixtureGnn(
    checkpoint = "./examples/Ellipse/checkpoints/gaussian-nt10.chk",
    device = device,
)

# Variational Graph Autoencoder
VGAE = dgn.nn.VGAE(
    checkpoint = "./examples/Ellipse/checkpoints/vgae-nt10.chk",
    device = device,
)

# Flow-Matching Graph Net
FMGN = dgn.nn.FlowMatchingGraphNet(
    checkpoint = "./examples/Ellipse/checkpoints/fmgn-nt10.chk",
    device = device,
)

# Latent Flow-Matching Graph Net
LFMGN = dgn.nn.LatentFlowMatchingGraphNet(
    autoencoder_checkpoint = "./examples/Ellipse/checkpoints/ae-nt10.chk",
    checkpoint = "./examples/Ellipse/checkpoints/lfmgn-nt10.chk",
    device = device,
)
```

下一步加载测试集。dgn4cfd 含多个数据集；这里用 `pOnEllipseInDist`，其未见仿真参数仍在训练范围内。还加载 `TimeEllipseInDist`，记录每次仿真的长度（两或三个振荡周期）。`transform` 配置几何和网格，本例采用三个粗化层级。

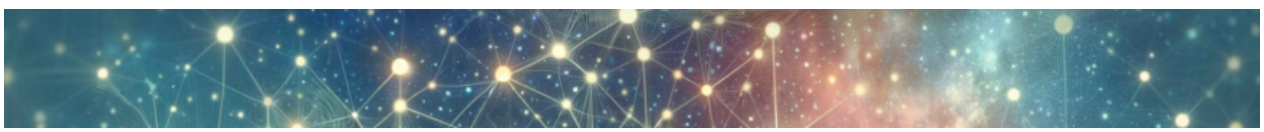
```
DATASET = dgn.datasets.DatasetUrl.pOnEllipseInDist
```

```
TIME_DATASET = dgn.datasets.DatasetUrl.TimeEllipseInDist
# Training dataset
transform = transforms.Compose([
    dgn.transforms.MeshEllipse(), # Create a mesh on the ellipse
    dgn.transforms.ScaleEdgeAttr(0.02), # Scale the relative position stored as
    ↪  `edge_attr`
    dgn.transforms.EdgeCondFreeStreamProjection(), # Add the projection of the free
    ↪  stream velocity along the edges as `edge_cond`
    dgn.transforms.ScaleAttr('target', vmin=-1.05, vmax=0.84), # Scale the target field
    ↪  (pressure)
    dgn.transforms.ScaleAttr('glob', vmin=500, vmax=1000), # Scale the global feature
    ↪  (Re)
    dgn.transforms.ScaleAttr('loc', vmin=2, vmax=3.5), # Scale the local feature
    ↪  (distances to the walls)
    dgn.transforms.MeshCoarsening( # Create 3 lower-resolution graphs and normalise the
    ↪  relative position betwen the inter-graph nodes.
        num_scales = 4,
        rel_pos_scaling = [0.02, 0.06, 0.15, 0.3],
        scalar_rel_pos = True,
    ),
])
dataset = dgn.datasets.pOnEllipse(
    path = dgn.datasets.DatasetDownloader(DATASET).file_path,
    T = 101,
    transform = transform,
)
print('Number of samples:', len(dataset))
# Load the length of each simulation to compute statistics
T = dgn.datasets.DatasetDownloader(TIME_DATASET).numpy()
```

原书输出：

```
Downloading dataset from https://huggingface.co/datasets/mariolinov/Ellipse/resolve/m⌋
↪  ain/pOnEllipseInDist.h5...
Dataset downloaded.
Number of samples: 50
Downloading dataset from https://huggingface.co/datasets/mariolinov/Ellipse/resolve/m⌋
↪  ain/TimeEllipseInDist.npy...
Dataset downloaded.
```

3



[3]译注：所有代码、安装/克隆/下载说明、输出及图均来自冻结源，未在翻译中执行。cuda:0 是原例固定设备设置，不代表任意本地环境都有 GPU。

32.2　逐样本精度

下一单元定义绘图函数，以黑色画出参考集中最近的真值压力分布，以浅红色画网络输出。SIM_IDX 选择测试流动条件，也可试其他值。

```
SIM_IDX = 25
def plot(ax, pos, target, pred, r2, title):
    pos = pos.cpu()
    target = target.cpu()
    pred = pred.cpu()
    # Plots
    top = pos[:, 1] >= 0.
    bottom = torch.logical_not(top)
    ax.plot(pos[top , 0].cpu(), target[top ].cpu(), 'k^', label='g.t. top')
    ax.plot(pos[bottom, 0].cpu(), target[bottom].cpu(), 'kv', label='g.t. bottom')
    ax.plot(pos[top, 0].cpu(), pred [top ].cpu(), '^', color="red", label='pred.top',
    ↪  alpha=0.6)
    ax.plot(pos[bottom, 0].cpu(), pred [bottom].cpu(), 'v', color="red",
    ↪  label='pred.bottom', alpha=0.6)
    ax.set_title(title+r' ($R^2$ = ' + f"{r2:.4f}" + r')', fontsize=16)
    ax.set_ylabel(r'$p$', fontsize=16)
    ax.set_xlabel(r'$x$', fontsize=16)
    ax.grid()
    ax.legend(fontsize=16)
```

接着用 plot() 组成 3×3 网格。第一行是 DGN 模型与普通 GN，第二行把流匹配与第一行 DDPM 直接比较，最后一行是余下三个基线。

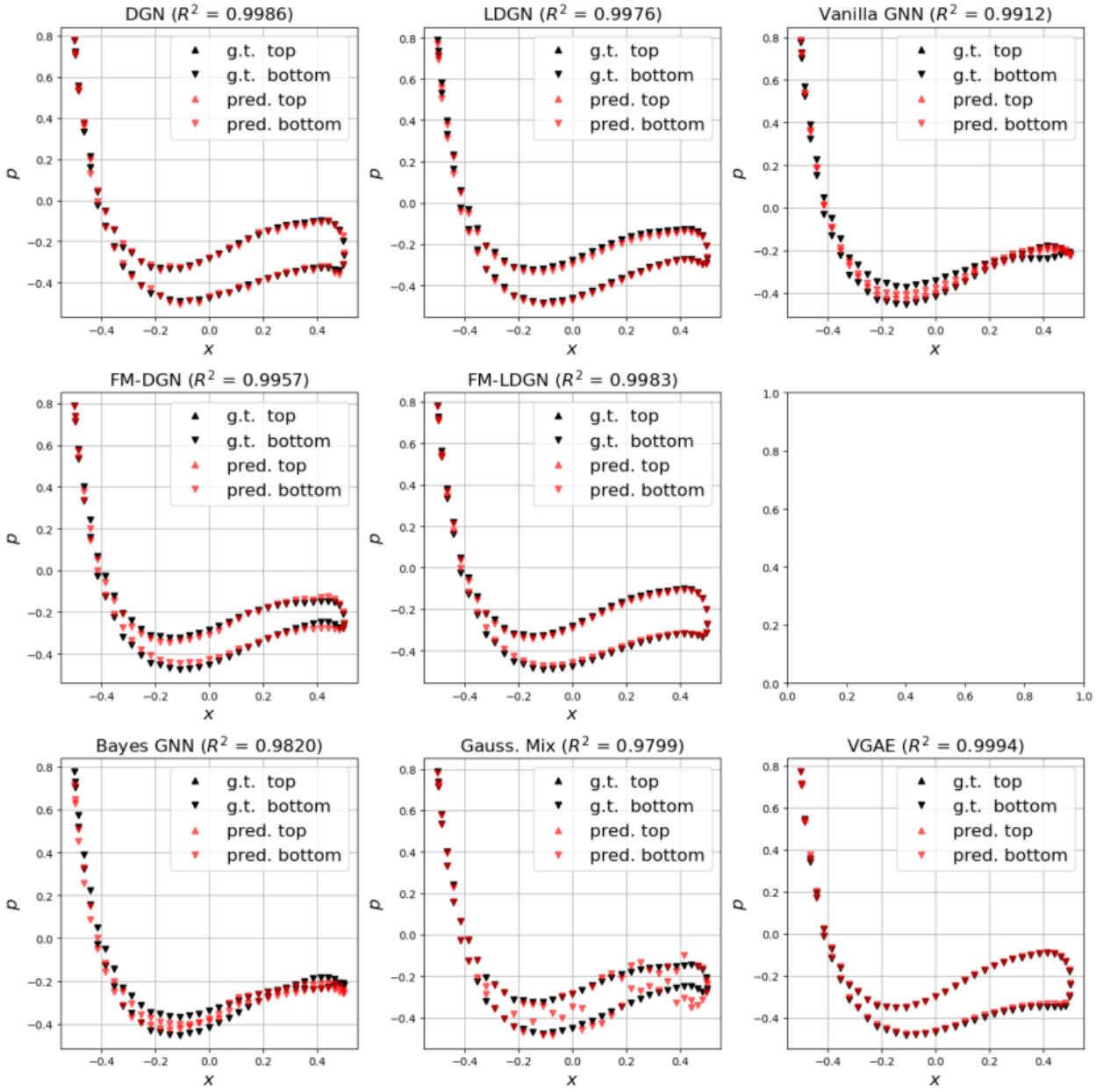
```
# denoising steps
NUM_DENOISING_STEPS = 50
# flow matching steps
NUM_FM_STEPS = int(NUM_DENOISING_STEPS/10)+1
fig, ax = plt.subplots(3, 3, figsize=(14, 14))
ax = ax.flatten()
curr_ax = 0
graph = dataset.get_sequence(SIM_IDX, n_in=T[SIM_IDX])
# DGN inference
steps = dgn.nn.diffusion.DiffusionStepsGenerator('linear',
↪  DGN.diffusion_process.num_steps)(NUM_DENOISING_STEPS)
pred = DGN.sample(graph, steps=steps)
# Compute the accuracy
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
r2, t = np.max(r2), np.argmax(r2)
print(f"DGN R2 = {r2:.4f} at t={t}")
# Plot the results
plot(ax[curr_ax],graph.pos, graph.target[:,t], pred, r2, "DGN")
curr_ax += 1
# LDGN inference
```

```
steps = dgn.nn.diffusion.DiffusionStepsGenerator('linear',
↪  DGN.diffusion_process.num_steps)(NUM_DENOISING_STEPS)
pred = LDGN.sample(graph, steps=steps)
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
r2, t = np.max(r2), np.argmax(r2)
print(f"LDGN R2 = {r2:.4f} at t={t}")
plot(ax[curr_ax],graph.pos, graph.target[:,t], pred, r2, "LDGN")
curr_ax += 1
# Vanilla Graph Net inference
with torch.no_grad():
    VanillaGN.eval()
    pred = VanillaGN(graph)
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
r2, t = np.max(r2), np.argmax(r2)
print(f"Vanilla Graph Net R2 = {r2:.4f} at t={t}")
plot(ax[curr_ax], graph.pos, graph.target[:,t], pred, r2, "Vanilla GNN")
curr_ax += 1
# Flow matching versions
# Flow-Matching Graph Net inference
steps = np.linspace(0, 1, NUM_FM_STEPS)
pred = FMGN.sample(graph, steps=steps)
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
r2, t = np.max(r2), np.argmax(r2)
print(f"Flow-Matching Graph Net R2 = {r2:.4f} at t={t}")
plot(ax[curr_ax],graph.pos, graph.target[:,t], pred, r2, "FM-DGN")
curr_ax += 1
# Latent Flow-Matching Graph Net inference
steps = np.linspace(0, 1, NUM_FM_STEPS)
pred = LFMGN.sample(graph, steps=steps)
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
r2, t = np.max(r2), np.argmax(r2)
print(f"Latent Flow-Matching Graph Net R2 = {r2:.4f} at t={t}")
plot(ax[curr_ax],graph.pos, graph.target[:,t], pred, r2, "FM-LDGN")
curr_ax += 1
# skip one cell
curr_ax += 1
# Other baselines following
# Bayesian Graph Net inference
pred = BayesianGN.sample(graph)
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
r2, t = np.max(r2), np.argmax(r2)
print(f"Bayesian Graph Net R2 = {r2:.4f} at t={t}")
plot(ax[curr_ax],graph.pos, graph.target[:,t], pred, r2, "Bayes GNN")
curr_ax += 1
# Gaussian Mixture Graph Net inference
pred = GaussianMixGN.sample(graph)
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
```

```
r2, t = np.max(r2), np.argmax(r2)
print(f"Gaussian Mixture Graph Net R2 = {r2:.4f} at t={t}")
# Plot the results
plot(ax[curr_ax],graph.pos, graph.target[:,t], pred, r2, "Gauss. Mix")
curr_ax += 1
# VGAE inference
pred = VGAE.sample(graph)
r2 = [dgn.metrics.r2_accuracy(pred, target) for target in graph.target.split(1, dim=1)]
r2, t = np.max(r2), np.argmax(r2)
print(f"VGAE R2 = {r2:.4f} at t={t}")
# Plot the results
plot(ax[curr_ax],graph.pos, graph.target[:,t], pred, r2, "VGAE")
curr_ax += 1
# Show the combined figure
plt.tight_layout()
plt.show()
```

原书输出：

```
DGN R2 = 0.9986 at t=74
LDGN R2 = 0.9976 at t=15
Vanilla Graph Net R2 = 0.9912 at t=6
Flow-Matching Graph Net R2 = 0.9957 at t=28
Latent Flow-Matching Graph Net R2 = 0.9983 at t=50
Bayesian Graph Net R2 = 0.9820 at t=67
Gaussian Mixture Graph Net R2 = 0.9799 at t=28
VGAE R2 = 0.9994 at t=36
```



图虽多，却说明了几点：

- DGN 及 FM 变体的压力分布很准确；默认设置下通常只有直接 DGN 略显噪声，潜空间 LDGN 已经更好。
- 流匹配采用十分之一步数（原文继称“即快 5 倍”），质量却与去噪相同。[4]
- Bayesian GNN 表现有问题，Gaussian 混合模型输出明显欠佳。作者对 GM 做了不少调参，仍无改善。

[4]译注：这两个倍率不是同一计数：代码 50 个去噪步，FM 取 6 个时间节点，即 5 个区间。实际速度还依每步网络评价数和实现，不能由节点比例单独推出 5 倍。

- VGAE 单个输出常好得惊人，但这只对单样本成立；跨许多样本时难以学习正确分布，混沌和高维任务更明显。

困难之一是目前只看一个输出。重跑会产生变化，自然还需充分评价分布精度，才能判断模型性能。

32.3 评价分布精度

评价大量样本并计算逐节点和全图 Wasserstein 距离，是量化的良好起点；但画出分布，仍有助直观理解方法优劣。因此叠画 Gaussian 核密度估计，展示沿椭圆长度各处的压力分布，可直接比较图中不同区域密度。下方第一格是真值分布。

```
from scipy.stats import gaussian_kde
def pdf(ax, pos, pred, title, vmin=None, vmax=None, w2_distance_1d=None,
↪  w2_distance_nd=None):
    pos = pos.cpu()
    ang = torch.atan2(pos[:,1], pos[:,0])
    idx = torch.argsort(ang)
    idx = idx[ang[idx] > 0]
    y = pred[idx].cpu().numpy()
    if vmin is None:
        vmin = y.min() - 0.1
    if vmax is None:
        vmax = y.max() + 0.1
    x = np.linspace(vmin, vmax, 1000)
    f = np.stack([gaussian_kde(y[i])(x) for i in range(y.shape[0])])
    f = f / f.max(axis=1)[:,None]
    ax.imshow(np.flip(f.T), aspect='auto', cmap='binary')
    ax.set_yticks(np.linspace(100, 900, 5), np.flip(np.round(x[np.linspace(100, 900,
    ↪  5).astype(int)], 2)), fontsize=16)
    ax.set_xticks([])
    #ax.set_yticks(fontsize=16)
    ax.set_title(title, fontsize=28)
    # Add a label with the Wasserstein-2 distance
    if w2_distance_1d is not None and w2_distance_nd is not None:
        ax.text(0.2, 0.76, r'$W_2^\mathregular{node}$' + f' = {w2_distance_1d:.4f} \n'
        ↪  + r'$W_2^\mathregular{graph}$' + f' = {w2_distance_nd:.4f}', fontsize=18,
        ↪  bbox=dict(facecolor='white', edgecolor='black', boxstyle='round,pad=0.2'),
        ↪  transform=ax.transAxes)
```

以下设 NUM_SAMPLES=200，画真值与八种模型的分布。

```
NUM_SAMPLES = 200
BATCH_SIZE = 200 # Number of samples generated in parallel. Reduce this number if you
↪  run out of memory.
```

```
fig, ax = plt.subplots(3, 3, figsize=(14, 14))
ax = ax.flatten()
curr_ax = 0
graph = dataset.get_sequence(SIM_IDX, n_in=T[SIM_IDX])
gt_mean = graph.target.mean(dim=1)
gt_std = graph.target.std (dim=1)
# Plot the ground-truth PDF on the upper half of the ellipse
vmin, vmax = graph.target.min().item() - 0.1, graph.target.max().item() + 0.1
pdf(ax[curr_ax], graph.pos, graph.target, '*Ground truth*', vmin=vmin, vmax=vmax)
curr_ax += 1
# DGN inference
steps = dgn.nn.diffusion.DiffusionStepsGenerator('linear',
↪  DGN.diffusion_process.num_steps)(NUM_DENOISING_STEPS)
pred = DGN.sample_n(NUM_SAMPLES, graph, steps=steps,
↪  batch_size=BATCH_SIZE).cpu().squeeze(-1)
mean = pred.mean(dim=1)
std = pred.std (dim=1)
# Compute the accuracy of the mean and std
mean_r2 = dgn.metrics.r2_accuracy(mean, gt_mean)
std_r2 = dgn.metrics.r2_accuracy(std , gt_std )
print('DGN')
print(f"R2 of mean: {mean_r2:.4f}", f"R2 of std: {std_r2:.4f}")
# Compute the Wasserstein-2 distance
w2_distance_1d = dgn.metrics.w2_distance_1d(pred, graph.target)
w2_distance_nd = dgn.metrics.w2_distance_nd(pred, graph.target)
print(f"Wasserstein-2 distance 1d: {w2_distance_1d:.4f}")
print(f"Wasserstein-2 distance nd: {w2_distance_nd:.4f}")
# Plot the PDF on the upper half of the ellipse
pdf(ax[curr_ax], graph.pos, pred, 'DGN', w2_distance_1d=w2_distance_1d,
↪  w2_distance_nd=w2_distance_nd, vmin=vmin, vmax=vmax)
curr_ax += 1
# LDGN inference
steps = dgn.nn.diffusion.DiffusionStepsGenerator('linear',
↪  DGN.diffusion_process.num_steps)(NUM_DENOISING_STEPS)
pred = LDGN.sample_n(NUM_SAMPLES, graph, steps=steps,
↪  batch_size=BATCH_SIZE).cpu().squeeze(-1)
mean = pred.mean(dim=1); std = pred.std (dim=1)
mean_r2 = dgn.metrics.r2_accuracy(mean, gt_mean)
std_r2 = dgn.metrics.r2_accuracy(std , gt_std )
print('LDGN'); print(f"R2 of mean: {mean_r2:.4f}", f"R2 of std: {std_r2:.4f}")
w2_distance_1d = dgn.metrics.w2_distance_1d(pred, graph.target)
w2_distance_nd = dgn.metrics.w2_distance_nd(pred, graph.target)
print(f"Wasserstein-2 distance 1d: {w2_distance_1d:.4f}")
print(f"Wasserstein-2 distance nd: {w2_distance_nd:.4f}")
pdf(ax[curr_ax], graph.pos, pred, 'LDGN', w2_distance_1d=w2_distance_1d,
↪  w2_distance_nd=w2_distance_nd, vmin=vmin, vmax=vmax)
curr_ax += 1
```

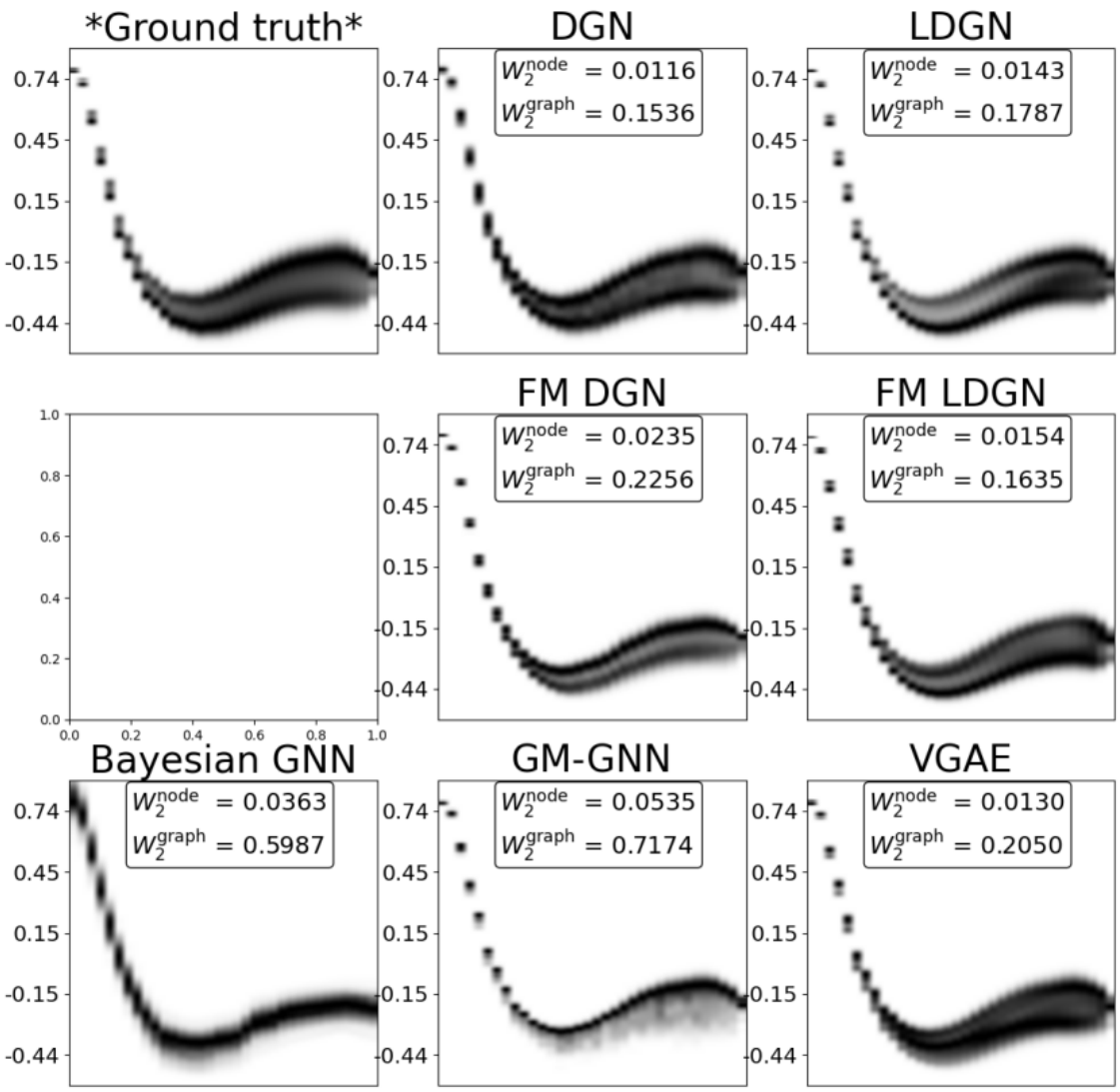
```
# skip one
curr_ax += 1
# FMGN inference
steps = np.linspace(0, 1, NUM_FM_STEPS )
pred = FMGN.sample_n(NUM_SAMPLES, graph, steps=steps,
↪  batch_size=BATCH_SIZE).cpu().squeeze(-1)
mean = pred.mean(dim=1); std = pred.std (dim=1)
mean_r2 = dgn.metrics.r2_accuracy(mean, gt_mean)
std_r2 = dgn.metrics.r2_accuracy(std , gt_std )
print('Flow-Matching DGN'); print(f"R2 of mean: {mean_r2:.4f}", f"R2 of std:
↪  {std_r2:.4f}")
w2_distance_1d = dgn.metrics.w2_distance_1d(pred, graph.target)
w2_distance_nd = dgn.metrics.w2_distance_nd(pred, graph.target)
print(f"Wasserstein-2 distance 1d: {w2_distance_1d:.4f}")
print(f"Wasserstein-2 distance nd: {w2_distance_nd:.4f}")
pdf(ax[curr_ax], graph.pos, pred, 'FM DGN', w2_distance_1d=w2_distance_1d,
↪  w2_distance_nd=w2_distance_nd, vmin=vmin, vmax=vmax)
curr_ax += 1
# LFMGN inference
steps = np.linspace(0, 1, NUM_FM_STEPS )
pred = LFMGN.sample_n(NUM_SAMPLES, graph, steps=steps,
↪  batch_size=BATCH_SIZE).cpu().squeeze(-1)
mean = pred.mean(dim=1); std = pred.std (dim=1)
mean_r2 = dgn.metrics.r2_accuracy(mean, gt_mean)
std_r2 = dgn.metrics.r2_accuracy(std , gt_std )
print('Latent Flow-Matching DGN'); print(f"R2 of mean: {mean_r2:.4f}", f"R2 of
↪  std:{std_r2:.4f}")
w2_distance_1d = dgn.metrics.w2_distance_1d(pred, graph.target)
w2_distance_nd = dgn.metrics.w2_distance_nd(pred, graph.target)
print(f"Wasserstein-2 distance 1d: {w2_distance_1d:.4f}")
print(f"Wasserstein-2 distance nd: {w2_distance_nd:.4f}")
pdf(ax[curr_ax], graph.pos, pred, 'FM LDGN', w2_distance_1d=w2_distance_1d,
↪  w2_distance_nd=w2_distance_nd, vmin=vmin, vmax=vmax)
curr_ax += 1
# Bayesian Graph Net inference
pred_list = []
for _ in tqdm.tqdm(range(NUM_SAMPLES)):
    pred_list.append(
        BayesianGN.sample(graph).cpu()
    )
pred = torch.concatenate(pred_list, dim=1)
mean = pred.mean(dim=1); std = pred.std (dim=1)
mean_r2 = dgn.metrics.r2_accuracy(mean, gt_mean)
std_r2 = dgn.metrics.r2_accuracy(std , gt_std )
print('Bayesian Graph Net'); print(f"R2 of mean: {mean_r2:.4f}", f"R2 of std:
↪  {std_r2:.4f}")
w2_distance_1d = dgn.metrics.w2_distance_1d(pred, graph.target)
```

```
w2_distance_nd = dgn.metrics.w2_distance_nd(pred, graph.target)
print(f"Wasserstein-2 distance 1d: {w2_distance_1d:.4f}")
print(f"Wasserstein-2 distance nd: {w2_distance_nd:.4f}")
pdf(ax[curr_ax], graph.pos, pred, 'Bayesian GNN', w2_distance_1d=w2_distance_1d,
↪  w2_distance_nd=w2_distance_nd, vmin=vmin, vmax=vmax)
curr_ax += 1
# Gaussian Mixture Graph Net inference
pred = GaussianMixGN.sample_n(NUM_SAMPLES, graph,
↪  batch_size=BATCH_SIZE).cpu().squeeze(-1)
mean = pred.mean(dim=1); std = pred.std (dim=1)
mean_r2 = dgn.metrics.r2_accuracy(mean, gt_mean)
std_r2 = dgn.metrics.r2_accuracy(std , gt_std )
print('Gaussian Mixture Graph Net'); print(f"R2 of mean: {mean_r2:.4f}", f"R2 of
↪  std:{std_r2:.4f}")
# Compute the Wasserstein-2 distance
w2_distance_1d = dgn.metrics.w2_distance_1d(pred, graph.target)
w2_distance_nd = dgn.metrics.w2_distance_nd(pred, graph.target)
print(f"Wasserstein-2 distance 1d: {w2_distance_1d:.4f}")
print(f"Wasserstein-2 distance nd: {w2_distance_nd:.4f}")
pdf(ax[curr_ax], graph.pos, pred, 'GM-GNN', w2_distance_1d=w2_distance_1d,
↪  w2_distance_nd=w2_distance_nd, vmin=vmin, vmax=vmax)
curr_ax += 1
# VGAE inference
pred = VGAE.sample_n(NUM_SAMPLES, graph, batch_size=BATCH_SIZE).cpu().squeeze(-1)
mean = pred.mean(dim=1); std = pred.std (dim=1)
mean_r2 = dgn.metrics.r2_accuracy(mean, gt_mean)
std_r2 = dgn.metrics.r2_accuracy(std , gt_std )
print('VGAE'); print(f"R2 of mean: {mean_r2:.4f}", f"R2 of std: {std_r2:.4f}")
w2_distance_1d = dgn.metrics.w2_distance_1d(pred, graph.target)
w2_distance_nd = dgn.metrics.w2_distance_nd(pred, graph.target)
print(f"Wasserstein-2 distance 1d: {w2_distance_1d:.4f}")
print(f"Wasserstein-2 distance nd: {w2_distance_nd:.4f}")
pdf(ax[curr_ax], graph.pos, pred, 'VGAE', w2_distance_1d=w2_distance_1d,
↪  w2_distance_nd=w2_distance_nd, vmin=vmin, vmax=vmax)
curr_ax += 1
```

原书输出：

```
DGN
R2 of mean: 0.9989 R2 of std: 0.9814
Wasserstein-2 distance 1d: 0.0116
Wasserstein-2 distance nd: 0.1536
LDGN
R2 of mean: 0.9986 R2 of std: 0.9614
Wasserstein-2 distance 1d: 0.0143
Wasserstein-2 distance nd: 0.1787
Flow-Matching DGN
R2 of mean: 0.9976 R2 of std: 0.4237
```

```
Wasserstein-2 distance 1d: 0.0235
Wasserstein-2 distance nd: 0.2256
Latent Flow-Matching DGN
R2 of mean: 0.9985 R2 of std: 0.8425
Wasserstein-2 distance 1d: 0.0154
Wasserstein-2 distance nd: 0.1635
100%|██████████| 200/200 [00:04<00:00, 43.76it/s]
Bayesian Graph Net
R2 of mean: 0.9980 R2 of std: -0.6514
Wasserstein-2 distance 1d: 0.0363
Wasserstein-2 distance nd: 0.5987
Gaussian Mixture Graph Net
R2 of mean: 0.9708 R2 of std: 0.1079
Wasserstein-2 distance 1d: 0.0535
Wasserstein-2 distance nd: 0.7174
VGAE
R2 of mean: 0.9994 R2 of std: 0.8818
Wasserstein-2 distance 1d: 0.0130
Wasserstein-2 distance nd: 0.2050
```



先仔细看真值：椭圆后周期性涡脱落使其成为双峰分布。模型既要捕捉密度较高的两个极端，也要捕捉较稀疏的内部。比较可见 DGN 方法忠实再现分布；直接 DGN 的噪声在此一定程度平均掉，甚至可优于 LDGN。与前面一样，流匹配只用一小部分开销即可生成这些分布，因此作者认为通常更值得采用。Bayesian 网络和 Gaussian 混合模型的问题在分布中十分明显。VGAE 的模式坍缩也更清楚，复杂场景中更严重。全图 Wasserstein 距离大致捕捉上述直觉，但难以说明外部和内部分别匹配得多好。不过，对上一章三维机翼等高维分布，它仍是关键工具。[5]

译注：独立核对的适用条件 此例用 SIM_IDX=25、每模型 200 样本，FM “同质量” 的措辞需按精度–成本权衡理解：FM-DGN 标准差 $R^2 = .4237$，DDPM-DGN 为 .9814；全图 W_2 分别 .2256 和 .1536。单样本 R^2 接近不表示分布精度相同。每个位置的 KDE 又单独除以自己的峰值，因此不同位置/面板灰度不再共享概率密度标尺；这些是逐节点边缘图，未展示空间联合相关。

[5]译注：绘图函数仅取椭圆上半部，并将每个位置的核密度除以其峰值；图像不是未经归一化的联合密度。源代码此处跳过普通确定性 GN，分布图实际为真值加七种概率模型，并留一个空格，正文 “八种模型” 沿源保留。

第三十三章　概率学习讨论

如前章所示，概率学习为基于物理的学习提供了广泛而令人兴奋的可能性。首先，这些方法有有趣且发展成熟的理论；有些部分竟比更简单学习方法的基本问题研究得还充分。同时，它们提供了与仿真交互的根本不同方式：可以简单地处理复杂解分布。这对逆问题意义重大，例如在基于仿真的推断中获得基于似然的估计。

不过，在确定性设置中，基于扩散的方法优势很少：并非更准确，计算开销通常还略大。长期稳定性是一个有趣例外，见无条件稳定性一章。[1] 概率深度学习方法的要点如下。

优点

- 可针对分布训练和推断；
- 理论成熟；
- 训练稳定。

缺点

- 推断开销略增；
- 在确定性设置中没有实质优势。

最后建议：若问题有模糊性，作者首选流匹配形式的扩散建模；若数据是可靠的输入输出对，则采用更简单的确定性训练。[2]

接着转向学习问题的新视角——强化学习。下一章会说明，尽管视角新，它与前面主题其实并非差别很大。

[1] 译注：源句写作 “will not show relatively few advantages”，有双重否定疑点；这里依紧接着的解释与下方利弊表呈现其语境含义，保留此记录。

[2] 译注：此为作者基于本书案例的选择建议，不是对任意任务、模型与成本的定理。

第六部分

强化学习

第三十四章　强化学习导言

深度强化学习（下简称强化学习，RL）是深度学习中一类不同于经典“用数据训练”视角的方法：让 AI 智能体从与环境的交互中学习。智能体执行动作，接收奖励，辨别哪些动作带来较高奖励，并调整行为。RL 在围棋等游戏中极为成功 [SSS+17]，也为机器人等工程应用带来希望。RL 一般由环境与智能体两部分组成。环境接收动作 a，给智能体状态 s 形式的观测以及奖励 r。观测只代表智能体能够感知的环境状态信息。奖励由预定、通常针对环境设计的函数给出，可含游戏得分、错误动作惩罚或任务完成奖金。最简单的 RL 学习目标可写为

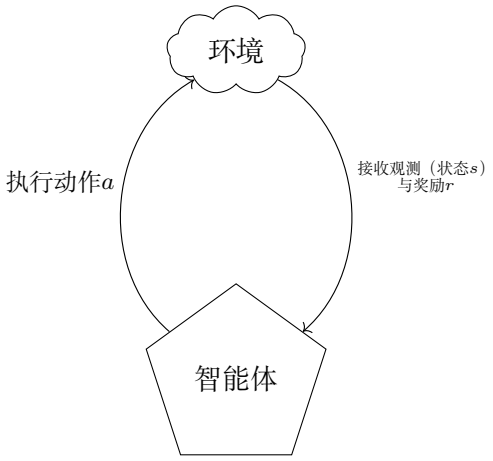


图 34.1: 环境以状态和奖励向智能体提供观测，智能体通过动作与环境交互。

$$\arg\max_{\theta} \mathbb{E}_{a\sim\pi(;s,\theta_p)} \left[\sum_t r_t \right]. \tag{34.1}$$

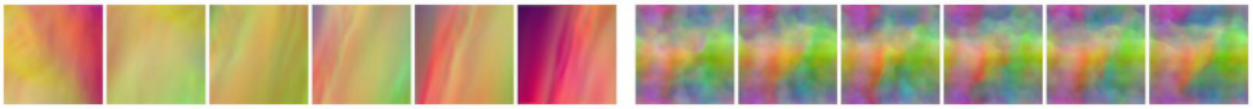
奖励 r_t 是智能体动作 a 的结果。智能体用神经网络策略，根据观测选动作。$\pi(a; s, \theta)$ 给出动作概率，以环境状态和权重为条件。[1] 训练旨在结合状态、动作及奖励信息，提高每条轨迹的累计奖励。算法大致分为策略梯度与基于价值两类 [SB18]。

34.1　算法

普通策略梯度中，网络直接由观测选择动作，学习动作概率：提高使轨迹后续回报较高的动作的概率，降低较小回报动作的概率。Q-learning 等价值方法则优化状态动作价值函数 Q。网络由 (s, a) 预测

[1] 译注：源式优化 θ，却在采样下标只写固定旧权重 θ_p，未显式给出当前参数如何进入轨迹分布或重要性权重；作为完整优化目标尚缺此依赖。原式保留，不自动解释为无偏梯度估计。连续动作时“概率”应理解为密度或相应分布。

轨迹余下部分的平均累计奖励 $Q(s,a)$，再在给定状态下选择使其最大的动作。演员–评论家方法结合二者。策略网络生成动作，另一网络用相应状态势的变化评价动作，并近似给定状态的预期累计奖励。近端策略优化 PPO [SWD+17] 属于这类，是本章 Burgers 方程控制例的选择。



34.2 近端策略优化

PPO 需训练相互依赖的演员与评论家网络。演员目标依赖评论家输出，得到哪些动作值得执行的反馈；评论家又依赖演员生成的动作，因为它们决定探索哪些状态。相互依赖可能导致不稳定。例如严重高估或低估状态价值会给出错误学习信号。高奖励动作也常有助于到达信息价值较高的状态。若单个样本可能错误的价值估计能不受限制地影响行为，训练进展可能崩溃。PPO 正为对抗此问题而提出，限制单个状态价值估计对演员学习变化的影响。它尤其常用于连续动作空间，因通常能较稳定地获得良好结果，实现也相对简单。

34.2.1 PPO-clip

具体采用 PPO-clip [SWD+17]。源文称其对单次更新引起的行为变化设置硬限制。算法保留旧网络状态（下标 p），限制每步学习变化。演员参数记 θ，评论家参数记 ϕ。[2]

34.2.2 演员

演员返回给定状态 s 与当前参数 θ 的动作分布，特定动作概率记 $\pi(a;s,\theta)$。训练用固定旧网络 $\pi(a;s,\theta_p)$ 采样并做若干次更新，定期以 θ 重置 θ_p。用 $\mathrm{clip}(a,b,c)$ 将 a 限在 $[b,c]$，ϵ 限定相对旧策略的偏离。演员目标为

$$\arg\max_{\theta} \mathbb{E}_{a\sim\pi(;s,\theta_p)}\left[\min\left(\frac{\pi(a;s,\theta)}{\pi(a;s,\theta_p)}A(s,a;\phi),\right.\right.$$
$$\left.\left.\mathrm{clip}\left(\frac{\pi(a;s,\theta)}{\pi(a;s,\theta_p)},1-\epsilon,1+\epsilon\right)A(s,a;\phi)\right)\right].$$

演员网络学习期望动作值；训练时加入额外标准差，在该均值周围 Gaussian 采样。标准差随训练下降，推断时只评价均值，即方差为零的分布。

34.2.3 评论家与优势

评论家价值函数 $V(s;\phi)$ 预测从状态 s 起的预期累计奖励，最小化优势 A 的平方：

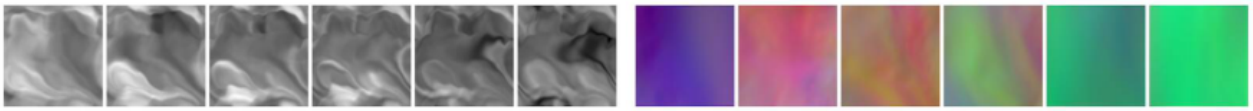
$$\arg\min_{\phi} \mathbb{E}_{a\sim\pi(;s,\theta_p)}[A(s,a;\phi)^2].$$

[2] 译注：截断的是目标中的概率比贡献，通常不构成所有状态动作上策略变化或 KL 距离的严格硬约束；保持原算法公式。

优势函数基于 V，评价相对平均累计奖励的偏离，即用固定旧策略所作决策相较随机决策改善多少。采用广义优势估计 GAE [SML+15]：

$$A(s_t, a_t; \phi) = \sum_{i=0}^{n-t-1} (\gamma\lambda)^i \delta_{t+i},$$
$$\delta_t = r_t + \gamma\big(V(s_{t+1};\phi) - V(s_t;\phi)\big).$$

r_t 为时间步奖励，n 为轨迹总长。通常小于 1 的超参数 γ, λ 控制远期奖励和状态价值对优势的影响。δ_t 是真实优势的有偏近似，GAE 可理解为从当前步到轨迹末端这些估计的折扣累积和。[3]



34.3 用于逆问题

RL 广用于多个相互依赖决策的轨迹优化。在物理系统和 PDE 中同样有吸引力：可像监督单次打靶一样，生成完整轨迹，再比较最终近似与目标来学习。二者优化方式不同。PPO 等算法训练时对演员动作加随机偏移以探索动作空间，发现比以往更精细的新行为。考虑力的长期影响也不同。使用可微物理 DP 损失的控制力估计器（如 Burgers 可微物理梯度优化章），将损失梯度穿过仿真步传回更早时间。RL 通常将环境视为无梯度黑箱；PPO 改用价值网络预测动作对系统未来的影响，追踪长期依赖。以 Burgers 方程为环境，轨迹生成中仿真与智能体交错：

$$\mathbf{u}^{t+1} = \mathcal{P}\big(\mathbf{u}^t + \pi(\mathbf{u}^t; \mathbf{u}^*, t, \theta)\Delta t\big).$$

星号仍表示参考或目标，$\mathbf{u}^*$ 为目标速度。PDE 连续动作空间中，π 直接算力形式的动作，而不是离散动作概率。奖励类似 DP 损失，含两部分：每步给施力平方范数的负值，轨迹结束再按终态与目标的 L2 距离惩罚：

$$r_t = r_t^f + r_t^o,$$
$$r_t^f = -\|\mathbf{a}_t\|_2^2,$$
$$r_t^o = \begin{cases} -\|\mathbf{u}^* - \mathbf{u}^t\|_2^2, & t = n-1, \\ 0, & \text{其他情况}. \end{cases}$$

[4]

34.4 实现

以下概述物理系统的 PPO 训练，也是下一章 Burgers 控制 notebook 的基础。下一章提供实际例子并与 DP 训练比较，这里先给通用思路。要训练智能体控制 PDE 系统，先将物理模型形式化为 RL

[3]译注：源 δ_t 把 γ 也乘到当前价值上，与常见 TD 残差 $r_t + \gamma V(s_{t+1}) - V(s_t)$ 不同，差为 $(1-\gamma)V(s_t)$。原式照录；此外优势的基线通常对应旧策略的价值而非均匀随机决策，终止态和截断 bootstrap 也须依实现区分。

[4]译注：原式终端索引是 u^t、$t = n-1$；若动作后才算奖励，代码可能使用更新后的状态，应以具体实现为核，不把此索引静默改成 u^n。

环境。采用 stable-baselines3 框架，它使用向量化 OpenAI gym 环境，能并行收集多条轨迹，提高资源利用与运行效率。向量化环境需定义观测和动作空间，即策略输入输出。观测将当前物理状态与目标状态（如速度场）沿通道堆叠，再加一个通道表示已过仿真时间除以总轨迹长。动作空间则是速度场每个单元一个力值。主要方法是 `reset`、`step_async`、`step_wait`、`render`。reset 计算初态目标，开始新轨迹，返回各实例首个观测。其他应用的实例不必同步终止，故环境内部到终态时也需调用 reset。step 的两个主体 async 和 wait 接收动作、施加到速度场并执行一次物理模型；分开可支持各实例分线程，但这里 phiflow 内部已处理批仿真，无需如此。render 实时显示重建轨迹或渲染到文件。演员与评论家输出空间差异很大，因此用不同结构。动作网络用 Holl 等 [HKT19] 结构的变体，与其执行动作的 CFE 函数一致。另一网络由核宽 3 卷积构成，各后接核宽 2、步长 2 的最大池化。特征图降至每通道一个值后，最终全连接层合并通道，预测状态价值。下一章的 `BurgersTraining` 类内部管理环境和智能体设置、训练模型及监控日志存盘。还含一个不随机生成轨迹而使用预定数据集的环境变体；训练中定期在此评价，以便更准确地与 DP 训练进展比较。接下来用该类运行完整 PPO 场景，评价性能，并与使用更多物理知识的 DP 梯度控制训练比较。

译注：独立核对的适用条件　若对 $\mathbb{E}[A(s,a;\phi)^2]$ 中含未来 V_ϕ 的整个 GAE 链对 ϕ 求导，并不自动等于通常固定回报目标的价值回归或半梯度更新。演员更新把优势视作固定量；评论家的目标、bootstrap 与 detach 边界须由实现定义。第 35 章仅调用外部 BurgersTraining/SB3，冻结 PDF 未给内部更新代码，不能由显示式独立核实这些边界。

第三十五章　用强化学习控制 Burgers 方程

以下以 Burgers 方程逆问题为强化学习（RL）试验场。设置类似先前可微物理（DP）训练的逆问题（见用神经网络求解逆问题），因此也直接比较两类方法。Burgers 方程简单但非线性，动力学有趣，是 RL 实验的良好起点。目标是训练控制力估计网络，预测两个给定状态间平滑过渡所需的力。源文附“在 Colab 中运行” 链接。

35.1　概览

强化学习的智能体感知环境并采取动作，以最大化环境回报的累计奖励，由经验学习不同情况下如何行动。近端策略优化 PPO 是常用算法，含选择动作的策略网络及评价状态奖励潜力的价值网络。所选动作引起的奖励潜力变化形成策略网络的损失。本 notebook 将 PPO 用于 Burgers 控制。RL 不取得可微物理求解器的梯度，因此是无模型方法。价值网络试图捕捉单个动作的长期影响，补偿缺少求解器模型的不足。有趣问题是：无模型 PPO 能否达到有模型 DP 训练的性能？以下比较学习速度与所需力的量。

35.2　软件安装

采用 stable-baselines3 的 PPO，物理仿真用可微 PDE 求解器 ΦFlow 1.5.1。RL 后还与用神经网络求逆章 [HKT19] 的控制力估计器 CFE 作 DP 比较。

```
!pip install stable-baselines3==1.1 phiflow==1.5.1
!git clone https://github.com/Sh0cktr4p/PDE-Control-RL.git
!git clone https://github.com/holl-/PDE-Control.git
```

例子涉及较广，需导入不少模块：

```
import sys; sys.path.append('PDE-Control/src'); sys.path.append('PDE-Control-RL/src')
import time, csv, os, shutil
from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
from phi.flow import *
import burgers_plots as bplt
import matplotlib.pyplot as plt
from envs.burgers_util import GaussianClash, GaussianForce
```

1

[1]译注：本章所有安装、克隆、数据生成、训练、推断、写盘代码均只按冻结 PDF 转写，未执行；日志和图均为源输出。版本依赖按源保留，不改成现代 API。

35.3 生成数据

先生成 DP 模型训练集，也用它评价两方法训练中及训练后的性能。模拟 1000 个案例（phiflow 场景），验证与测试各留 100，余 800 训练。

```
DOMAIN = Domain([32], box=box[0:1]) # Size and shape of the fields
VISCOSITY = 0.003
STEP_COUNT = 32 # Trajectory length
DT = 0.03
DIFFUSION_SUBSTEPS = 1
DATA_PATH = 'forced-burgers-clash'
SCENE_COUNT = 1000
BATCH_SIZE = 100
TRAIN_RANGE = range(200, 1000)
VAL_RANGE = range(100, 200)
TEST_RANGE = range(0, 100)
for batch_index in range(SCENE_COUNT // BATCH_SIZE):
    scene = Scene.create(DATA_PATH, count=BATCH_SIZE)
    print(scene)
    world = World()
    u0 = BurgersVelocity(
        DOMAIN,
        velocity=GaussianClash(BATCH_SIZE),
        viscosity=VISCOSITY,
        batch_size=BATCH_SIZE,
        name='burgers'
    )
    u = world.add(u0, physics=Burgers(diffusion_substeps=DIFFUSION_SUBSTEPS))
    force = world.add(FieldEffect(GaussianForce(BATCH_SIZE), ['velocity']))
    scene.write(world.state, frame=0)
    for frame in range(1, STEP_COUNT + 1):
        world.step(dt=DT)
        scene.write(world.state, frame=frame)
```

原书输出：

```
forced-burgers-clash/sim_000000
forced-burgers-clash/sim_000100
forced-burgers-clash/sim_000200
forced-burgers-clash/sim_000300
forced-burgers-clash/sim_000400
forced-burgers-clash/sim_000500
forced-burgers-clash/sim_000600
forced-burgers-clash/sim_000700
forced-burgers-clash/sim_000800
forced-burgers-clash/sim_000900
```

35.4 强化学习训练

设置 RL 环境。PPO 用专门价值网络（评论家）预测给定状态的累计奖励，再更新策略网络（演员），后者像 CFE 一样预测控制仿真的力。

```
from experiment import BurgersTraining
N_ENVS = 10 # On how many environments to train in parallel, load balancing
FINAL_REWARD_FACTOR = STEP_COUNT # Penalty for not reaching the goal state
STEPS_PER_ROLLOUT = STEP_COUNT * 10 # How many steps to collect per environment between
↪  agent updates
N_EPOCHS = 10 # How many epochs to perform during each agent update
RL_LEARNING_RATE = 1e-4 # Learning rate for agent updates
RL_BATCH_SIZE = 128 # Batch size for agent updates
RL_ROLLOUTS = 500 # Number of iterations for RL training
```

建立内部管理环境和智能体的 trainer，并创建保存模型、日志和超参数的目录，以便后续按相同配置续训。若 `exp_name` 指定的模型目录已存在便加载，否则新建。trainer 包装 stable-baselines3 的 PPO，内部建立 `BurgersEnv` gym 环境，随机生成初态、预计算相应真值仿真，并处理动作影响的演化。还定期加载使用验证集初态和目标的另一环境来评价性能。

35.4.1 Gym 环境

gym 接口规定智能体观测和动作空间，以及主要方法：

- `reset` 在轨迹结束后恢复初态并返回观测；
- `step` 接收动作、前进一步，返回观测、奖励、终态标志及调试日志字典；
- `render` 按环境作者规定的方式显示状态，可检查训练结果。

stable-baselines3 还提供向量化环境，可同时计算多轨迹前向传播，提高资源利用与时间效率。方法处理观测、动作、奖励、终态标志和信息字典的向量；step 拆成 `step_async` 与 `step_wait`，可使实例在不同线程运行。

35.4.2 物理仿真

环境含 phiflow 的 Burgers 物理对象，状态存为 `BurgersVelocity`。初态像上述数据生成一样批量随机产生。观测将当前和目标速度场沿通道堆叠，再加当前时间步通道。动作是一维数组，对应全部速度值。step 调用物理对象前进一时间步，并以 `FieldEffect` 施加动作。每步按施力平方范数惩罚；轨迹末端再减去目标场 L2 距离乘 `FINAL_REWARD_FACTOR`。奖励随后用运行中的均值和标准差估计归一化。[2]

[2]译注：正文在此称 L2 距离，前章显示式为平方范数，保留原措辞差别；具体归一化形式及终端奖励应以依赖实现为准，本地仅有源 PDF，未运行仓库。

35.4.3 神经网络设置

演员用 [HKT19] 的 U-Net 变体；评论家用一维卷积与池化将特征图降至一个值，再用核宽 1 卷积合并特征，得到单输出。`CustomActorCriticPolicy` 允许使用这两种不同结构。默认智能体存于 `PDE-Control-RL/networks/rl-models/bench`，若存在就加载。可换新路径从头训练。因训练很长，这里从已训练 3500 次迭代的模型开始，再微调 `RL_ROLLOUTS=500` 次，通常约 2 小时；总共近 18 小时不适合交互试验，不过代码也能从头训练。

```
rl_trainer = BurgersTraining(
    path='PDE-Control-RL/networks/rl-models/bench', # Replace path to train a new model
    domain=DOMAIN,
    viscosity=VISCOSITY,
    step_count=STEP_COUNT,
    dt=DT,
    diffusion_substeps=DIFFUSION_SUBSTEPS,
    n_envs=N_ENVS,
    final_reward_factor=FINAL_REWARD_FACTOR,
    steps_per_rollout=STEPS_PER_ROLLOUT,
    n_epochs=N_EPOCHS,
    learning_rate=RL_LEARNING_RATE,
    batch_size=RL_BATCH_SIZE,
    data_path=DATA_PATH,
    val_range=VAL_RANGE,
    test_range=TEST_RANGE,
)
```

原书输出：

```
Tensorboard log path: PDE-Control-RL/networks/rl-models/bench/tensorboard-log
Loading existing agent from PDE-Control-RL/networks/rl-models/bench/agent.zip
```

下单元可选，用 notebook 内的 tensorboard 显示进展，便于调试。若模型在其他位置，需改路径。续训预训练智能体时，新运行会分开显示，可在齿轮中启用 reload。“forces” 图显示网络施力总量的训练变化；“rew_unnormalized” 为未经上述归一化的奖励，相应归一化值见 “rollout/ep_rew_mean”；“val_set_forces” 显示验证集性能。

```
%load_ext tensorboard
%tensorboard --logdir PDE-Control-RL/networks/rl-models/bench/tensorboard-log
```

现在开始训练。RL 需多次迭代探索环境，下一单元通常需几小时，500 次展开约 2 小时。

```
rl_trainer.train(n_rollouts=RL_ROLLOUTS, save_freq=50)
```

原书输出：

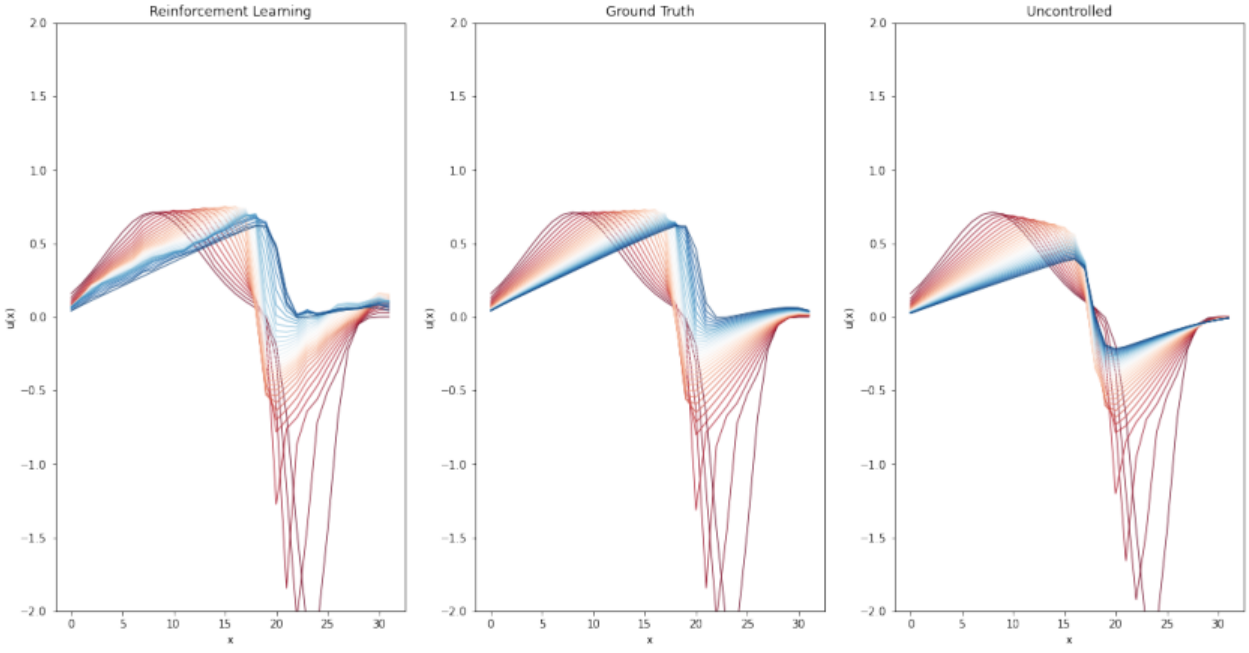
```
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
```

```
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
Storing agent and hyperparameters to disk...
```

35.5 RL 评价

看训练后的结果：最左为 RL 智能体重建，中间为应重建的真值轨迹，右侧是系统自然演化的无控制仿真。

```
TEST_SAMPLE = 0 # Change this to display a reconstruction of another scene
rl_frames, gt_frames, unc_frames = rl_trainer.infer_test_set_frames()
fig, axs = plt.subplots(1, 3, figsize=(18.9, 9.6))
axs[0].set_title("Reinforcement Learning"); axs[1].set_title("Ground Truth");
↪  axs[2].set_title("Uncontrolled")
for plot in axs:
    plot.set_ylim(-2, 2); plot.set_xlabel('x'); plot.set_ylabel('u(x)')
for frame in range(0, STEP_COUNT + 1):
    frame_color = bplt.gradient_color(frame, STEP_COUNT+1);
    axs[0].plot(rl_frames[frame][TEST_SAMPLE,:], color=frame_color, linewidth=0.8)
    axs[1].plot(gt_frames[frame][TEST_SAMPLE,:], color=frame_color, linewidth=0.8)
    axs[2].plot(unc_frames[frame][TEST_SAMPLE,:], color=frame_color, linewidth=0.8)
```



训练后的智能体能较好重建轨迹，但仍明显不如真值平滑。

35.6 可微物理训练

为定位 RL 效果，与 DP 训练比较。用神经网络求逆章的完整方法还含第二个 OP 物理预测网络；为与 RL 公平比较，这里去掉它，直接控制。DP 能取得求解器梯度，跨多步追踪损失，更好理解施力长期效果。RL 则在策略上生成新样本，不像此 DP 算法受固定训练集大小限制；但额外仿真开销也可能增加收敛时间。

```
from control.pde.burgers import BurgersPDE
from control.control_training import ControlTraining
from control.sequences import StaggeredSequence
```

原书输出：

```
Could not load resample cuda libraries: CUDA binaries not found at
↪  /usr/local/lib/python3.7/dist-packages/phi/tf/cuda/build/resample.so. Run "python
↪  setup.py cuda" to compile them
```

下单元设置用于训练或加载已有检查点的模型。

```
dp_app = ControlTraining(
    STEP_COUNT,
    BurgersPDE(DOMAIN, VISCOSITY, DT),
    datapath=DATA_PATH,
    val_range=VAL_RANGE,
    train_range=TRAIN_RANGE,
    trace_to_channel=lambda trace: 'burgers_velocity',
    obs_loss_frames=[],
    trainable_networks=['CFE'],
    sequence_class=StaggeredSequence,
    batch_size=100,
    view_size=20,
    learning_rate=1e-3,
    learning_rate_half_life=1000,
    dt=DT
).prepare()
```

原书输出：

```
App created. Scene directory is /root/phi/model/control-training/sim_000000
↪  (INFO),2021-08-04 10:11:58,466n
Sequence class: <class 'control.sequences.StaggeredSequence'> (INFO), 2021-08-04
↪  10:12:01,449n
Partition length 32 sequence (from 0 to 32) at frame 16
Partition length 16 sequence (from 0 to 16) at frame 8
Partition length 8 sequence (from 0 to 8) at frame 4
Partition length 4 sequence (from 0 to 4) at frame 2
Partition length 2 sequence (from 0 to 2) at frame 1
Execute -> 1
Execute -> 2
```

```
Partition length 2 sequence (from 2 to 4) at frame 3
Execute -> 3
Execute -> 4
Partition length 4 sequence (from 4 to 8) at frame 6
Partition length 2 sequence (from 4 to 6) at frame 5
Execute -> 5
Execute -> 6
Partition length 2 sequence (from 6 to 8) at frame 7
Execute -> 7
Execute -> 8
Partition length 8 sequence (from 8 to 16) at frame 12
Partition length 4 sequence (from 8 to 12) at frame 10
Partition length 2 sequence (from 8 to 10) at frame 9
Execute -> 9
Execute -> 10
Partition length 2 sequence (from 10 to 12) at frame 11
Execute -> 11
Execute -> 12
Partition length 4 sequence (from 12 to 16) at frame 14
Partition length 2 sequence (from 12 to 14) at frame 13
Execute -> 13
Execute -> 14
Partition length 2 sequence (from 14 to 16) at frame 15
Execute -> 15
Execute -> 16
Partition length 16 sequence (from 16 to 32) at frame 24
Partition length 8 sequence (from 16 to 24) at frame 20
Partition length 4 sequence (from 16 to 20) at frame 18
Partition length 2 sequence (from 16 to 18) at frame 17
Execute -> 17
Execute -> 18
Partition length 2 sequence (from 18 to 20) at frame 19
Execute -> 19
Execute -> 20
Partition length 4 sequence (from 20 to 24) at frame 22
Partition length 2 sequence (from 20 to 22) at frame 21
Execute -> 21
Execute -> 22
Partition length 2 sequence (from 22 to 24) at frame 23
Execute -> 23
Execute -> 24
Partition length 8 sequence (from 24 to 32) at frame 28
Partition length 4 sequence (from 24 to 28) at frame 26
Partition length 2 sequence (from 24 to 26) at frame 25
Execute -> 25
Execute -> 26
Partition length 2 sequence (from 26 to 28) at frame 27
```

```
Execute -> 27
Execute -> 28
Partition length 4 sequence (from 28 to 32) at frame 30
Partition length 2 sequence (from 28 to 30) at frame 29
Execute -> 29
Execute -> 30
Partition length 2 sequence (from 30 to 32) at frame 31
Execute -> 31
Execute -> 32
Target loss: Tensor("truediv_1:0", shape=(), dtype=float32) (INFO), 2021-08-04
↪  10:13:10,701n
Force loss: Tensor("Sum_97:0", shape=(), dtype=float32) (INFO), 2021-08-04
↪  10:13:14,221n
Setting up loss (INFO), 2021-08-04 10:13:14,223n
Preparing data (INFO), 2021-08-04 10:13:51,128n
INFO:tensorflow:Summary name Total Force is illegal; using Total_Force instead.
Initializing variables (INFO), 2021-08-04 10:13:51,156n
Model variables contain 0 total parameters. (INFO), 2021-08-04 10:13:55,961n
Validation (000000): Learning_Rate: 0.001, Loss_reg_unscaled: 205.98526,
↪  Loss_reg_scale: 1.0, Loss: 0.0, Total Force: 393.8109 (INFO), 2021-08-04
↪  10:14:32,455n
```

现在执行训练，通常也需较长时间，原文称 1000 次迭代约 2 小时。

```
DP_TRAINING_ITERATIONS = 10000 # Change this to change training duration
dp_training_eval_data = []
start_time = time.time()
for epoch in range(DP_TRAINING_ITERATIONS):
    dp_app.progress()
    # Evaluate validation set at regular intervals to track learning progress
    # Size of intervals determined by RL epoch count per iteration for accurate
    ↪  comparison
    if epoch % N_EPOCHS == 0:
        f = dp_app.infer_scalars(VAL_RANGE)['Total Force'] / DT
        dp_training_eval_data.append((time.time() - start_time, epoch, f))
```

[3] 将训练模型及按迭代和运行时间记录的验证表现 `val_forces.csv` 保存到磁盘：

```
DP_STORE_PATH = 'networks/dp-models/bench'
if not os.path.exists(DP_STORE_PATH):
    os.makedirs(DP_STORE_PATH)
# store training progress information
with open(os.path.join(DP_STORE_PATH, 'val_forces.csv'), 'at') as log_file:
    logger = csv.DictWriter(log_file, ('time', 'epoch', 'forces'))
    logger.writeheader()
    for (t, e, f) in dp_training_eval_data:
```

[3]译注：代码实际默认 `DP_TRAINING_ITERATIONS=10000`，不是正文例举的 1000。初始化日志 “Model variables contain 0 total parameters” 也按源保留，不能据该日志确认真正可训练参数总数。

```
        logger.writerow({'time': t, 'epoch': e, 'forces': f})
dp_checkpoint = dp_app.save_model()
shutil.move(dp_checkpoint, DP_STORE_PATH)
```

原书输出：

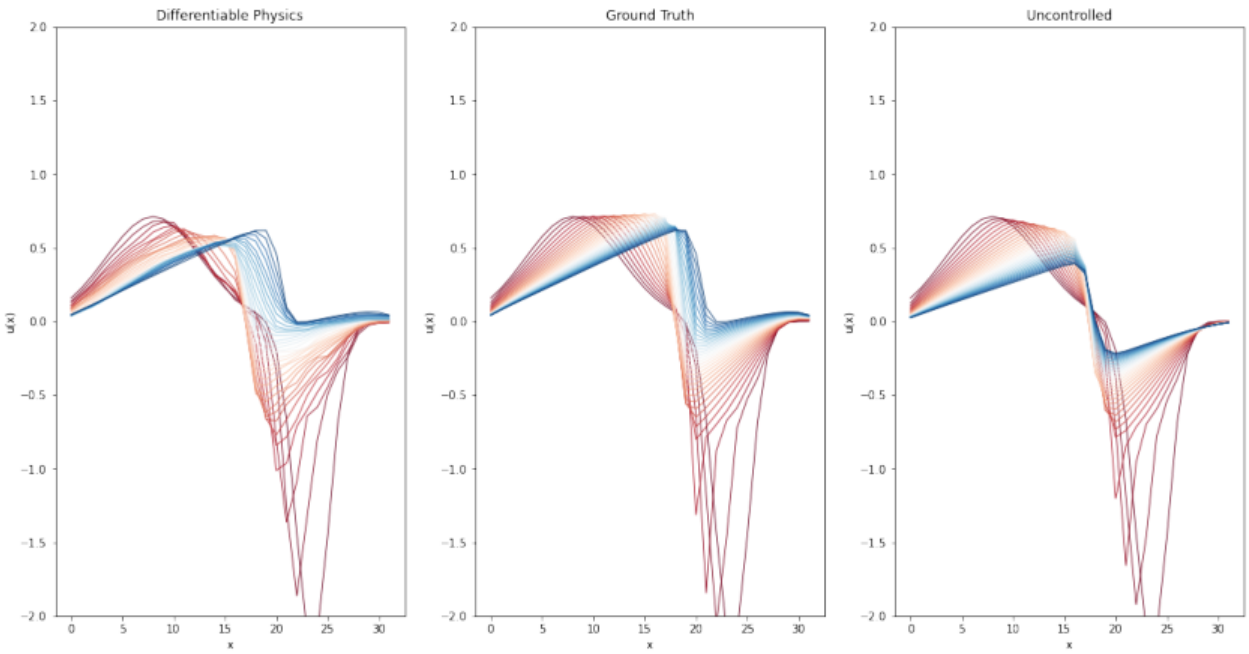
```
'networks/dp-models/bench/checkpoint_00010000'
```

也可取消以下注释以加载已有网络。

```
# dp_path = 'PDE-Control-RL/networks/dp-models/bench/checkpoint_00020000/'
# networks_to_load = ['OP2', 'OP4', 'OP8', 'OP16', 'OP32']
# dp_app.load_checkpoints({net: dp_path for net in networks_to_load})
```

类似 RL 版本，画一个 DP 结果，左为学习结果，中间真值，右为自然演化。

```
dp_frames = dp_app.infer_all_frames(TEST_RANGE)
dp_frames = [s.burgers.velocity.data for s in dp_frames]
_, gt_frames, unc_frames = rl_trainer.infer_test_set_frames()
TEST_SAMPLE = 0 # Change this to display a reconstruction of another scene
fig, axs = plt.subplots(1, 3, figsize=(18.9, 9.6))
axs[0].set_title("Differentiable Physics")
axs[1].set_title("Ground Truth")
axs[2].set_title("Uncontrolled")
for plot in axs:
    plot.set_ylim(-2, 2)
    plot.set_xlabel('x')
    plot.set_ylabel('u(x)')
for frame in range(0, STEP_COUNT + 1):
    frame_color = bplt.gradient_color(frame, STEP_COUNT+1)
    axs[0].plot(dp_frames[frame][TEST_SAMPLE,:], color=frame_color, linewidth=0.8)
    axs[1].plot(gt_frames[frame][TEST_SAMPLE,:], color=frame_color, linewidth=0.8)
    axs[2].plot(unc_frames[frame][TEST_SAMPLE,:], color=frame_color, linewidth=0.8)
```



DP 同样紧密重建原轨迹，且看起来比 RL 噪声更少。接着更细致比较。

35.7 RL 与 DP 比较

比较轨迹视觉质量及施力总量。二者训练都希望最小化施力，这也是重要指标：末步施加极大力即可平凡达到目标，理想解应利用 PDE 动力学尽量少施力。因此这一指标可衡量网络学到多少底层物理环境知识。

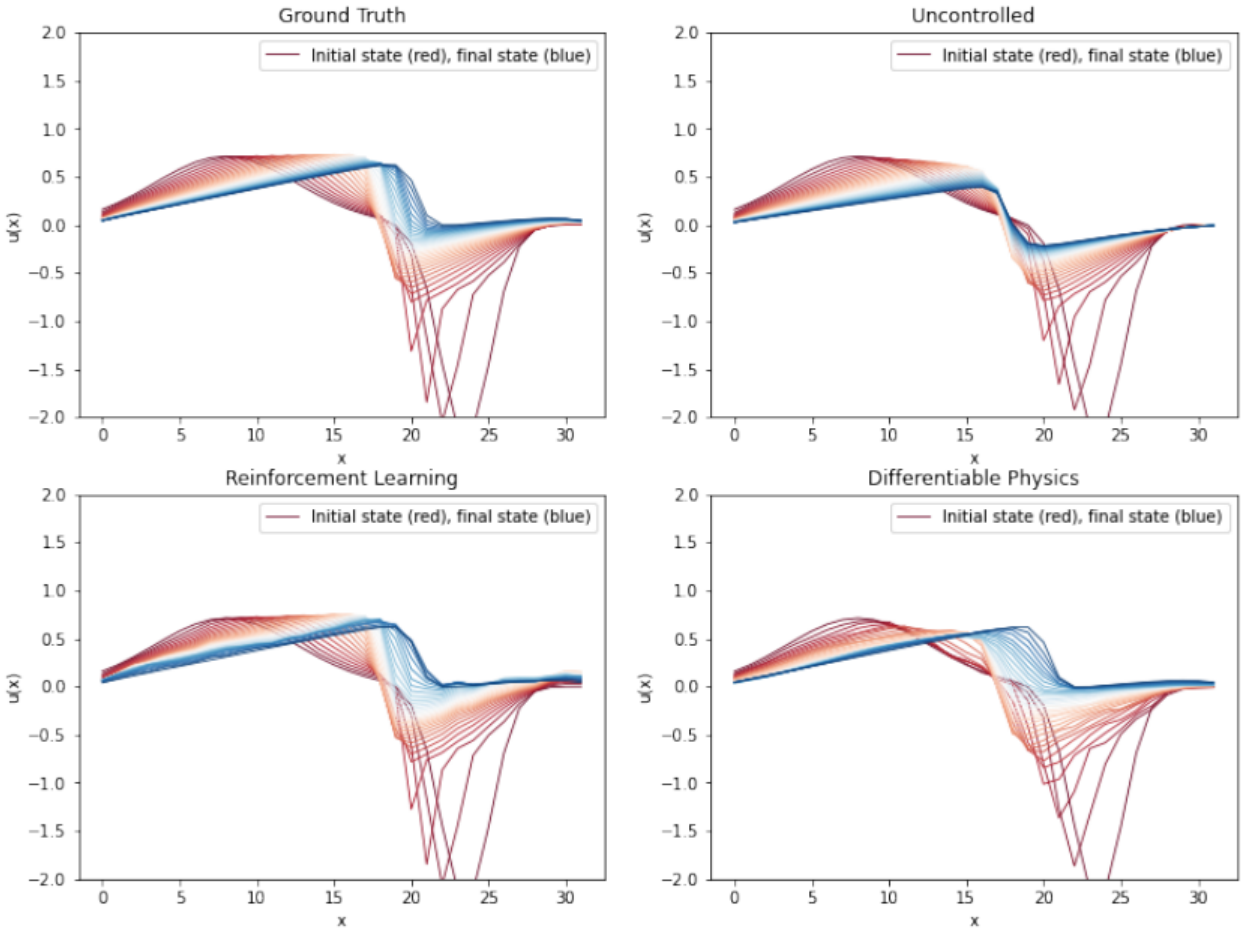
```
import utils
import pandas as pd
```

35.7.1 轨迹比较

分别生成测试轨迹，并收集真值及自然演化。

```
rl_frames, gt_frames, unc_frames = rl_trainer.infer_test_set_frames()
dp_frames = dp_app.infer_all_frames(TEST_RANGE)
dp_frames = [s.burgers.velocity.data for s in dp_frames]
frames = {
    (0, 0): ('Ground Truth', gt_frames),
    (0, 1): ('Uncontrolled', unc_frames),
    (1, 0): ('Reinforcement Learning', rl_frames),
    (1, 1): ('Differentiable Physics', dp_frames),
}
TEST_SAMPLE = 0 # Specifies which sample of the test set should be displayed
def plot(axs, xy, title, field):
    axs[xy].set_ylim(-2, 2); axs[xy].set_title(title)
    axs[xy].set_xlabel('x'); axs[xy].set_ylabel('u(x)')
```

```
    label = 'Initial state (red), final state (blue)'
    for step_idx in range(0, STEP_COUNT + 1):
        color = bplt.gradient_color(step_idx, STEP_COUNT+1)
        axs[xy].plot(
            field[step_idx][TEST_SAMPLE].squeeze(), color=color, linewidth=0.8,
            ↪ label=label)
        label = None
    axs[xy].legend()
fig, axs = plt.subplots(2, 2, figsize=(12.8, 9.6))
for xy in frames:
    plot(axs, xy, *frames[xy])
```



此图组合前面各训练后的图。DP 似乎噪声更少，而两者都能近似真值。

35.7.2 施力比较

计算两方法在测试轨迹中产生和施加的力：

```
gt_forces = utils.infer_forces_sum_from_frames(
    gt_frames, DOMAIN, DIFFUSION_SUBSTEPS, VISCOSITY, DT
)
```

```
dp_forces = utils.infer_forces_sum_from_frames(
    dp_frames, DOMAIN, DIFFUSION_SUBSTEPS, VISCOSITY, DT
)
rl_forces = rl_trainer.infer_test_set_forces()
```

原书输出：

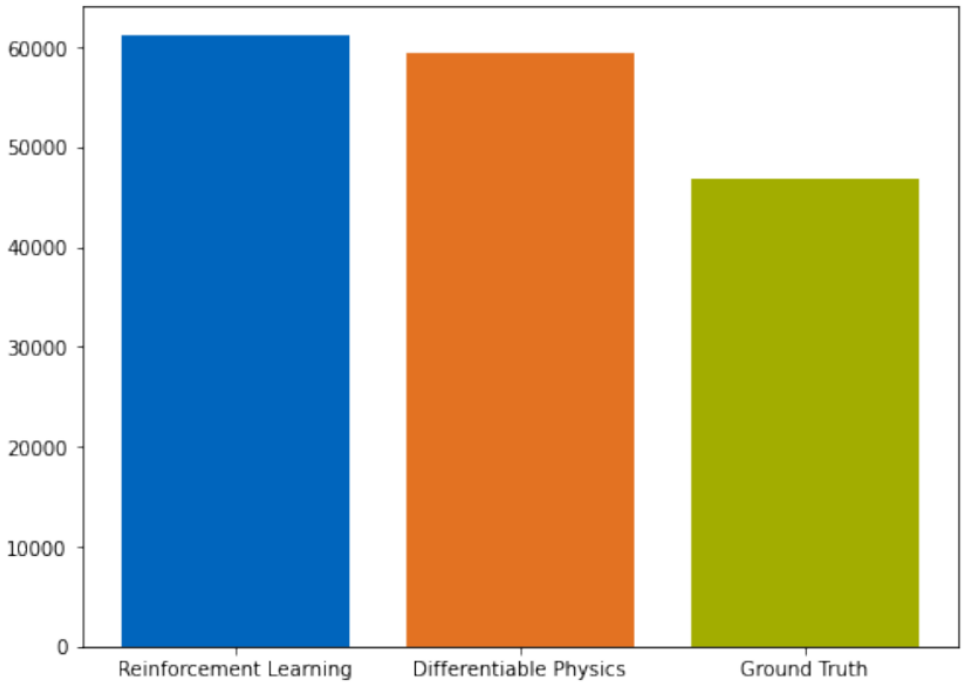
```
Sanity check - maximum deviation from target state: 0.000000
Sanity check - maximum deviation from target state: 0.000011
```

先把 RL、DP 的施力总和与真值比较。

```
plt.figure(figsize=(8, 6))
plt.bar(
    ["Reinforcement Learning", "Differentiable Physics", "Ground Truth"],
    [np.sum(rl_forces), np.sum(dp_forces), np.sum(gt_forces)],
    color = ["#0065bd", "#e37222", "#a2ad00"],
    align='center', label='Absolute forces comparison' )
```

原书输出：

```
<BarContainer object of 3 artists>
```



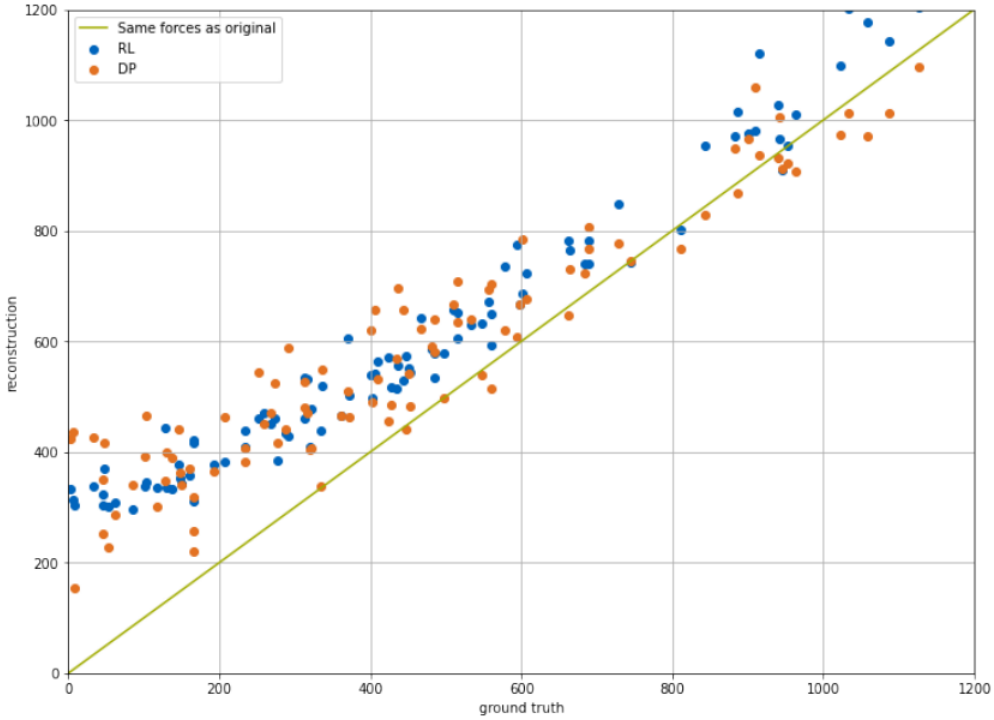
柱状图表明 DP 学习施加的力略小于 RL。两者到达目标的精度相当，故以此作为主要性能指标。再逐样本比较与真值的施力。源文称蓝线上方的点表示方法施力大于真值，反之亦然。

```
plt.figure(figsize=(12, 9))
plt.scatter(gt_forces, rl_forces, color="#0065bd", label='RL')
plt.scatter(gt_forces, dp_forces, color="#e37222", label='DP')
```

```
plt.plot([x * 100 for x in range(15)], [x * 100 for x in range(15)], color="#a2ad00",
↪  label='Same forces as original')
plt.xlabel('ground truth'); plt.ylabel('reconstruction')
plt.xlim(0, 1200); plt.ylim(0, 1200); plt.grid(); plt.legend()
```

原书输出：

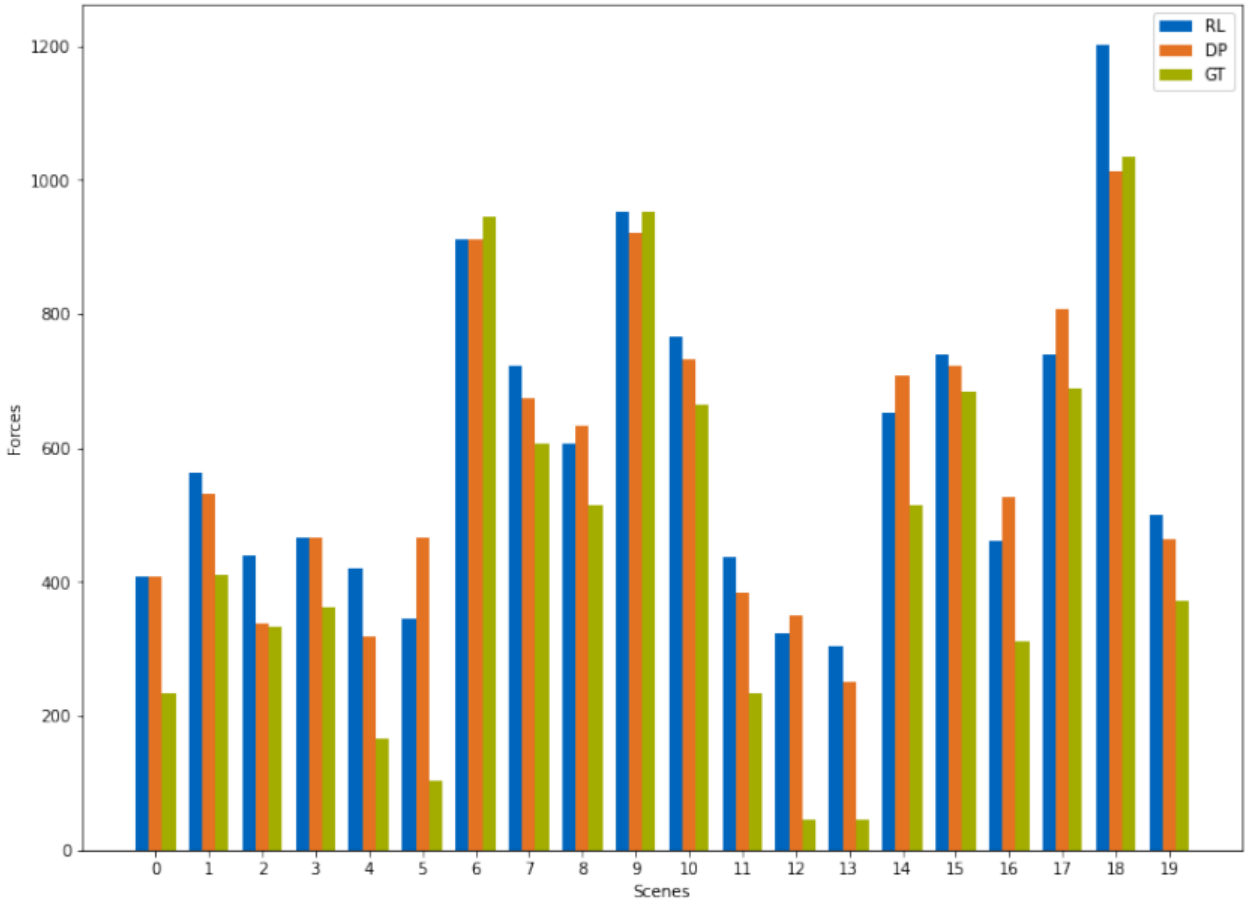
```
<matplotlib.legend.Legend at 0x7f4cbc6d5090>
```



[4] DP 橙点通常更接近对角线，即学到更接近真值的力。下图逐样本展示 RL、DP 与真值。

```
w=0.25; plot_count=20 # How many scenes to show
plt.figure(figsize=(12.8, 9.6))
plt.bar( [i - w for i in range(plot_count)], rl_forces[:plot_count], color="#0065bd",
↪  width=w, align='center', label='RL' )
plt.bar( [i for i in range(plot_count)], dp_forces[:plot_count], color="#e37222",
  width=w, align='center', label='DP' )
plt.bar( [i + w for i in range(plot_count)], gt_forces[:plot_count], color="#a2ad00",
↪  width=w, align='center', label='GT' )
plt.xlabel('Scenes'); plt.xticks(range(plot_count))
plt.ylabel('Forces'); plt.legend(); plt.show()
```

[4]译注：源绘图的等施力对角线实为绿色（#a2ad00），正文“蓝线”照录并说明；RL 点为蓝色，DP 为橙色。



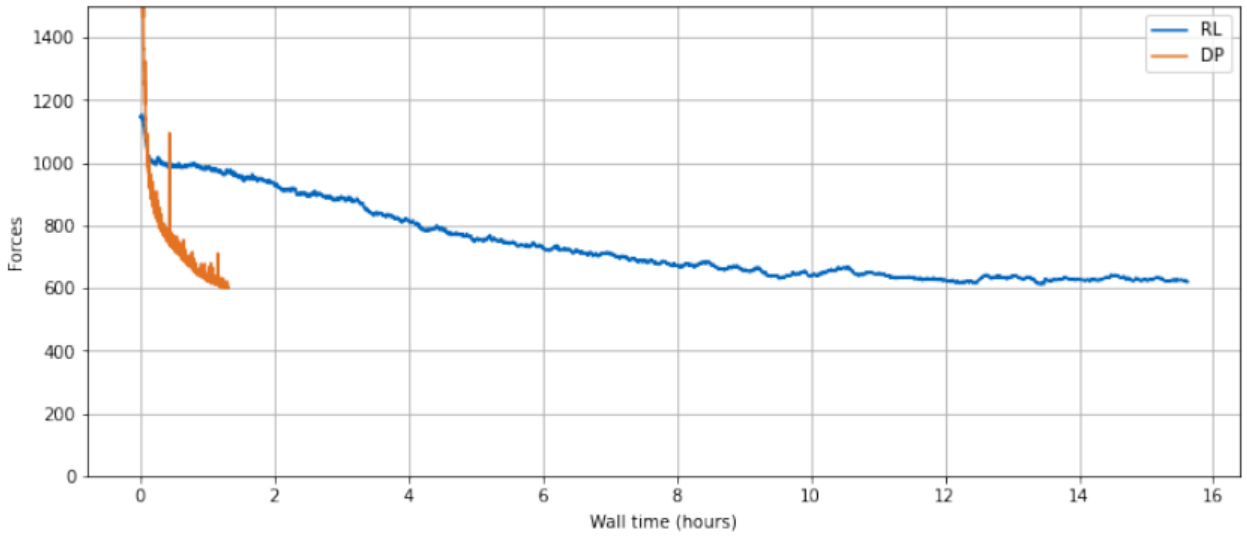
35.8 训练进展比较

虽然力大小所衡量的控制质量是主要目标，训练行为也有有趣差异。不了解物理的 RL 与紧耦合求解器的 DP 最主要区别，是后者收敛显著更快：求解器梯度提供的学习信号远好于 RL 无方向的探索。RL 行为部分来自在策略收集数据及“蛮力”探索。下一单元按实际运行时间画训练进展。

```
def get_dp_val_set_forces(experiment_path):
    path = os.path.join(experiment_path, 'val_forces.csv')
    table = pd.read_csv(path)
    return list(table['time']), list(table['epoch']), list(table['forces'])
rl_w_times, rl_step_nums, rl_val_forces = rl_trainer.get_val_set_forces_data()
dp_w_times, dp_epochs, dp_val_forces = get_dp_val_set_forces(DP_STORE_PATH)
plt.figure(figsize=(12, 5))
plt.plot(np.array(rl_w_times) / 3600, rl_val_forces, color="#0065bd", label='RL')
plt.plot(np.array(dp_w_times) / 3600, dp_val_forces, color="#e37222", label='DP')
plt.xlabel('Wall time (hours)'); plt.ylabel('Forces')
plt.ylim(0, 1500); plt.grid(); plt.legend()
```

原书输出：

```
<matplotlib.legend.Legend at 0x7f4cbc595c50>
```



总之，源实验中的 PPO 比 DP 施加更大力，学习解质量稍逊；所需收敛时间也显著更长，无论运行时间还是训练迭代数。[5]

35.9 下一步

- 改变学习率等超参数，考察其对训练的影响；
- 改变场分辨率，比较两方法；更高分辨率使动力学更复杂、更难控制；
- 在不同环境参数（如黏度、dt）下使用训练模型，测试泛化。

译注：独立核对的适用条件 正文建议取消注释加载已有训练网络，但本例 trainable_networks 只含 CFE，networks_to_load 却列 OP2/4/8/16/32，不能据此保证恢复 CFE。val_forces.csv 采用追加模式且每次 writeheader，重跑会插入重复表头；后续 read_csv 的 time/forces 可能含字符串，不能继续当纯数值除以 3600。time 和 epoch 也从零重新计数，这不是安全的续训日志合并。轨迹图只向 plot 传入 y，横轴 0–31 是数组索引，尽管标成 x，并非 Domain 的物理坐标 $[0, 1]$。

[5]译注：此结论限于原例给定结构、超参数、训练数据与硬件口径；未在本译稿复现实验，也不据此断言所有 RL 方法均较差。

第七部分

改进梯度

第三十六章　尺度不变性与求逆

以下质疑此前表述的一个基本方面：由梯度计算更新步。回顾前章，方法或纯监督训练，或把物理模型作为损失项，或把可微物理 DP 算子嵌入训练图。后两者与本书更相关，虽相似，但物理损失仅在训练时求值，DP 通常推断时也用求解器，从而端到端训练网络和数值求解器。三者均用一阶导数驱动优化，后两者也对物理项求导。这从深度学习角度很自然，但我们还没问它是否最好。本章核心认识是：对涉及物理量的学习，普通梯度可能并非最优。监督与 DP 梯度各有利弊，给了解物理算子的定制方法留下空间。特别是将依 [HKT22] 说明 DP 梯度尺度问题如何影响网络训练，重访多模态解，并介绍避免这些问题的替代方法。

本章预览

- 展示优化器性质及尺度问题如何负面影响网络训练；
- 找出此前 GD 或 Adam 的问题：缺少使训练尺度不变的求逆过程；
- 介绍两种缓解方案：解析的完全求逆与数值的半求逆。

36.1　问题的关键

先直观说明求逆为何重要。所有此前方法都用梯度，存在根本尺度问题：即使简单线性情形，梯度方向也可能被扭曲，妨碍有效走向极小值。在非线性情形，梯度长度还可能与距极小点的距离反相关，使收敛更难。[1] 一维可调学习率缓解，高维问题更明显。考虑二维玩具“物理”函数，仅把第二分量乘 α，再用 L2 损失：

$$\mathcal{P}(x_1, x_2) = \begin{bmatrix} x_1 \\ \alpha x_2 \end{bmatrix}, \qquad\qquad L(\mathcal{P}) = |\mathcal{P}|^2.$$

$\alpha = 1$ 时损失径向对称，两个分量行为相同。梯度 $\nabla_x = (\partial L/\partial x)^T$ 垂直等值线，$-\eta\nabla_x$ 直指零点极小值。这类似大多数监督或分类等经典深度学习情形，见下图左侧。本书针对物理学习，在学习中集成物理函数。这很不同：物理过程几乎总给各分量不同尺度，有些状态变化敏感、响应很大，有些影响极小。不同 α 模拟这一点，见图中、右。α 增大时，远离极小点的损失沿 x_2 更陡，与 x_1 尺度差增大，梯度也沿 x_2 增长。为避免优化发散，需减小 η，却削弱 x_1 的进展。梯度仍垂直等值线，故先沿 x_2 快速靠近横轴，随后很慢向左爬行。更糟的是，普通更新实际施加尺度的平方。多维时极难选好单一学习率，必

[1] 译注：后一句为源文动机性概括，并非任意非线性函数的性质。

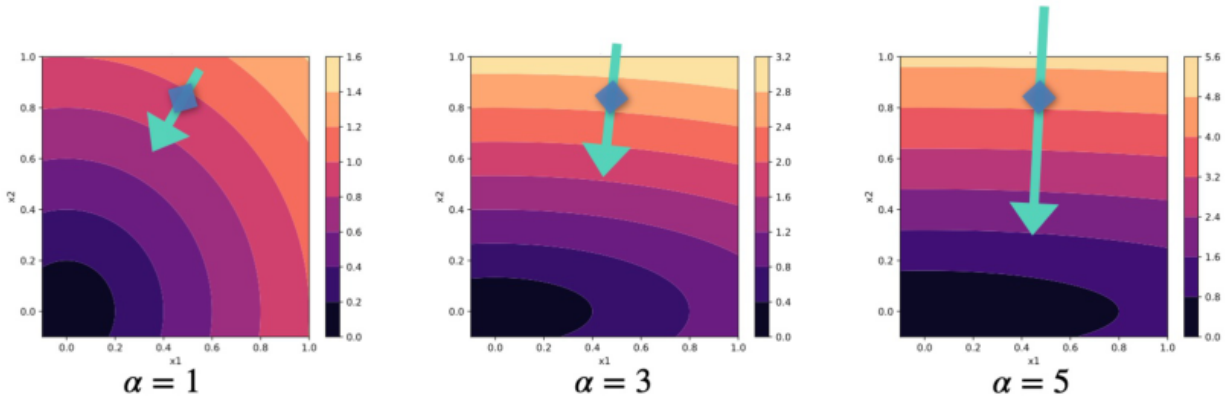
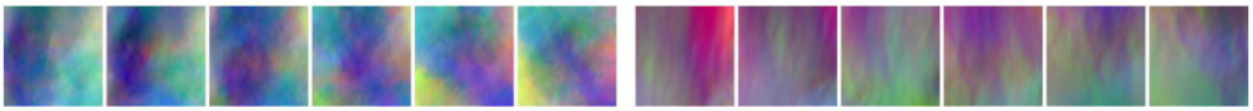


图 36.1: 二维示例中不同 α 的损失景观。绿箭头是示例更新 $-\nabla_x$，未严格按比例。

须通过某种求逆计及各维不同尺度。求逆不等于取负，$g^{-1} \neq -g$；把梯度本身取负会指向错误方向。需要仍降低损失、又计及各维尺度的更新，故核心目标是尺度不变性。[2]

尺度不变性定义 给定函数的尺度不变优化，在函数的不同参数化（即缩放）下产生相同结果。例如上述玩具问题，不论 α 如何选，优化轨迹均相同。[3]



36.2 传统优化方法

令标量损失 $L(x)$ 待最小化，目标是计算输入参数步 Δx，以下用下标区分。此前网络都以反向传播和梯度下降 GD 训练；对 PDE 求解器 $\mathcal{P}$ 同样求导就得到 DP 训练。暂去网络，问题为 $\arg\min_x L(x)$，$L(x) = \frac{1}{2}\|\mathcal{P}(x) - y^*\|_2^2$。核心复合梯度为

$$\left(\frac{\partial L}{\partial x}\right)^T = \left(\frac{\partial \mathcal{P}}{\partial x}\right)^T \left(\frac{\partial L}{\partial \mathcal{P}}\right)^T. \tag{36.1}$$

转置记号不便阅读，作者以下因主要处理转置 Jacobian 而省略 T。此前已展示 $\partial L/\partial x$ 有效，但经典优化更常用拟 Newton 等不同更新。因此重访 GD、拟 Newton，并以逆 Jacobian 为第三方案，从理论比较利弊，也讨论为何经典算法虽有优势却不普遍用于网络训练。仅考虑多变量函数，除说明外符号均为向量值。[4]

[2]译注：此句的“梯度取负”指在已定义更新中再反号，不能误解为反对常规负梯度下降。源文多处有提取为 “2/7” 的评价图标，原 PDF 显示不清，以下不将其当数值或公式分数。

[3]译注：比较轨迹须规定哪些坐标、目标、步长与参数化相应变换；此处依作者后续例子的含义理解，不扩为任意非线性重参数化不变性。

[4]译注：源后文有标量示意式，省略转置并不许可任意交换矩阵乘法顺序；多维应核对 Jacobian 方向和维数。

36.3 梯度下降

GD 更新随目标对输入的导数缩放：

$$\Delta x_{\mathrm{GD}} = -\eta \frac{\partial L}{\partial x}. \tag{36.2}$$

η 是标量学习率，Jacobian 描述损失对输入小变化的响应。此常用更新却有一些不理想性质。监督训练如翼型 RANS 章用了它，可微物理如神经算子减小数值误差章也用了流体求解器导数；后者虽只更新网络参数，求解器 Jacobian 仍通过式 (36.1) 进入更新。以下按单位、函数敏感度及最优点附近行为联合评价各方法，相关主题可揭示异同。

单位 GD 内在地错误表示量纲。若 x_1, x_2 单位不同，因它们在分母 ∂x，更新随其单位的倒数缩放。学习率可补偿，但一个 η 无法同时为两种不同单位给出正确更新。虽可说网络参数单位不重要，物理角度仍不自然，也暗示更基本的问题。

函数敏感度 未归一化函数也有问题。取简化例 $L(x) = \alpha x$，GD 步为 $-\eta\alpha$，而 $L(x + \Delta x_{\mathrm{GD}})$ 含 α^2 阶项。$\alpha = 1$ 正常，实际常有 $\alpha \ll 1$ 或更糟 $\alpha \gg 1$。展开为

$$L(x + \Delta x_{\mathrm{GD}}) = L(x) + \Delta x_{\mathrm{GD}} \frac{\partial L}{\partial x} + \cdots,$$
$$L(x) - L(x + \Delta x_{\mathrm{GD}}) = -\eta \left(\frac{\partial L}{\partial x}\right)^2 + \mathcal{O}(\Delta x^2) \quad \text{（源式）}.$$

损失变化含导数平方，因而出现 α^2。希望优化器在梯度缩放 α 时提供类似 $1/\alpha^2$ 的量，得到可靠更新。[5] 敏感函数中，小输入变化带来大输出，GD 却给大步，造成更大损失变化、梯度爆炸；不敏感函数中，输入大变输出仍小，GD 步却小，可能停滞，即梯度消失。深网络各层未充分归一化时易有此问题。归一化与适当学习率可部分缓解，但优化仿真时不能直接使用：若在最后求解步前任意归一化，会破坏物理状态。调学习率也困难，例如同时优化不同时间步参数，或输出量级依初态改变。

最优点附近收敛 源文称可微函数在最优点附近损失必变平，收敛时梯度趋零，所以 $\Delta x_{\mathrm{GD}} \to 0$，收敛慢。这有时能稳定训练，却也使学习过程难控制。[6]

36.4 拟 Newton 方法

Newton 更新用梯度与逆 Hessian：

$$\Delta x_{\mathrm{QN}} = -\eta \left(\frac{\partial^2 L}{\partial x^2}\right)^{-1} \frac{\partial L}{\partial x}. \tag{36.3}$$

实践更常用 BFGS 等拟 Newton 变体近似 Hessian，但仍采用这种更新结构。常以线搜索选步长，这里暂略。它解决 GD 一些问题，也有其他缺点。

[5] 译注：第二行源式符号有误：代入负梯度，左侧下降量的一阶项应为正的 $\eta(\partial L/\partial x)^2$；右侧负号对应 $L(x + \Delta x) - L(x)$。标量示意保留，向量推广应用梯度范数平方。

[6] 译注：这里需内部可微驻点等条件；步长趋零不自动等于不良收敛率，例如强凸二次问题 GD 可线性收敛。边界约束最优点也未必零梯度。

单位与敏感度 拟 Newton 较好处理物理单位，所有参数更新单位正确，η 可无量纲。考虑

$$L(x+\Delta x_{\mathrm{QN}}) = L(x) - \eta \left(\frac{\partial^2 L}{\partial x^2}\right)^{-1} \frac{\partial L}{\partial x}\frac{\partial L}{\partial x} + \cdots,$$

第二项正确消去 x 的量纲，留下 L 的标量变化。回到 GD 中的尺度 α，逆 Hessian 可提供 $1/\alpha^2$ 抵消不良尺度。[7]

最优点附近收敛 损失较平时拟 Newton 也可更快。即使固定 η，逆 Hessian 特征值与 Hessian 特征值成倒数，随平坦程度增大而放大步，避免 GD 式减速。

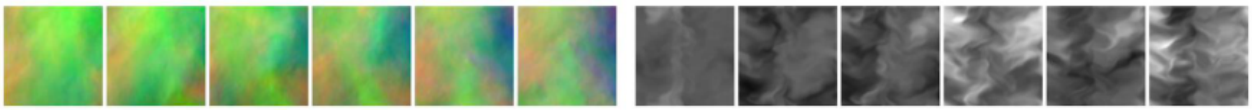
函数复合的一致性 但中间空间更新仍依赖此前全部函数，因为复合 Hessian 含梯度非线性项。对 $L(y(x))$，源标量示意为

$$\frac{d^2 L}{dx^2} = \frac{\partial^2 L}{\partial y^2}\left(\frac{\partial y}{\partial x}\right)^2 + \frac{\partial L}{\partial y}\frac{\partial^2 y}{\partial x^2}.$$

它依赖内层 Jacobian 平方，故若反向传播用此更新，Hessian 受反传链中后续函数影响。在计算全部梯度前，不知道各潜空间的更新。

依赖 Hessian 这也是根本缺点。精确 Hessian 虽只多一次求导传播，却涉及比梯度更高维的张量，存储规模随参数数平方增长，求值与求逆在大系统很贵。拟 Newton 花许多功夫以合理资源近似，问题仍核心。接近不可逆的 Hessian 常导致数值不稳，真正不可逆时需回退一阶 GD。目标还需二阶可微；许多网络用 ReLU，其二阶导为零。高阶导在参数空间移动时往往变化更快，更易受损失高频噪声影响。[8]

注 拟 Newton 仍是活跃研究领域，许多扩展可在特定设置缓解这些问题，例如只存低维向量近似 Hessian 以绕过内存要求。上述困难仍说明 BFGS 等常遇到的问题。



36.5 逆梯度

先考虑所谓逆梯度 IG。它实际用 Jacobian 的逆，因计算链末端为标量损失，结果是梯度向量。它也有自身问题，因此只是中间步骤，后面会用更实用的形式。暂不用标量 L，考虑可能非标量的 $y(x)$，后面通常是仿真器 $\mathcal{P}$，这里记 y。给目标 y^*，求 $y=\mathcal{P}(x)$ 的输入，是逆问题。定义

$$\Delta x_{\mathrm{IG}} = \frac{\partial x}{\partial y}\,\Delta y. \tag{36.4}$$

逆 Jacobian 一阶描述为使输出变化 Δy，输入应如何变化。关键是求逆，因此需 Jacobian 可逆，类似 Hessian 求逆问题；若可逆，性质很好。步长改由希望输出增加或减少的 Δy 决定，替代学习率但作用相同。最简单可用 $\Delta y = \eta(y^* - y)$ 向真值走一步。后续常出现它，使公式初看与前面不同。

[7] 译注：此为源标量式；多维项应为 $-\eta g^T H^{-1} g$。$1/\alpha^2$ 对二次复合损失的坐标缩放成立，不能用于前述线性 $L=\alpha x$ 的零 Hessian 直接求逆。

[8] 译注：源概括不表示所有拟 Newton 实现都显式算 Hessian 或必须回退 GD；阻尼、正则化和矩阵自由方法另有选择。ReLU 在折点不二阶可微，但网络损失在固定激活区的 Hessian 未必为零。

单位 $\partial x/\partial y$ 乘 Δy 自动给出 x 单位，无需任意有量纲学习率。

函数敏感度 前述 $L = \alpha x$ 的参数更新现在随 α^{-1} 缩放：敏感函数获小步，不敏感函数获大步，甚至爆炸。

最优点附近与函数复合 如 Newton，IG 在最优点附近仍产生推动优化的更新，通常改善收敛。还具有复合一致性，源式写为

$$\Delta x_{\mathrm{IG}} = \Delta L \, \frac{\partial x}{\partial y} \frac{\partial y}{\partial L},$$
$$\Delta y = \Delta L \, \frac{\partial y}{\partial x} \frac{\partial x}{\partial y} \frac{\partial y}{\partial L} = \Delta L \, \frac{\partial y}{\partial L}.$$

至少一阶上，中间空间的变化不依赖之前的函数，可在全部梯度算完前于反传过程中估计。即使逆 Hessian 的 Newton 也未完全做到；关键是 Jacobian 可逆时，直接得到该层正确尺度的方向，无需逆 Hessian 辅助。

依赖逆 Jacobian 逆只对方阵定义，即输入输出维数相同。但优化通常输入高维、输出标量，且方阵也未必可逆。因此接下来利用 IG 是逆函数线性化这一点，说明直接用逆函数可保留优点并获得更多收益。[9]

36.6 逆仿真器

Newton 与 IG 共同说明普通梯度不够，需某种求逆实现尺度不变。讨论网络和数值求逆前，先看特殊情形：若能使用逆仿真器，也可解决 GD 多项内在问题，是物理仿真部分的理想更新设置。令 $y = \mathcal{P}(x)$，逆为 $\mathcal{P}^{-1}(y) = x$，这是整个函数的逆，而非仅 Jacobian 或 Hessian 的逆。若只处理一个 (x, y^*)，逆求解器使最小化问题本身多余：算一次 $\mathcal{P}^{-1}(y^*)$ 就得到解。为引回网络和复杂场景，假定有多个目标，解不显然，例如希望同一 x 以最小 L2 失真满足多个 y^*。不再只求一次，而迭代更新当前近似 x_0，称步为 Δx_{PG}。最好用以当前猜测为条件的局部逆 $\mathcal{P}^{-1}(y^*; x)$：若多个相距很远的 x 产生近似目标，选择最靠近当前猜测的一个，避免多模态设置下解流形成为零散点集。令 $y_0 = \mathcal{P}(x_0)$，更新为

$$\Delta x_{\mathrm{PG}} = \frac{\mathcal{P}^{-1}(y_0 + \Delta y; x_0) - x_0}{\Delta y} \, \Delta y. \tag{36.5}$$

Δy 可为全距离 $y^* - y_0$ 或其中一部分。采用括号中的逆差时，即使 $\mathcal{P}$ 高度非线性，也会精确给出 $\mathcal{P}(x_0 + \Delta x) = y_0 + \Delta y$。式 (36.5) 中的 Δy 实际抵消，留下 x 步；这样写是为展示与 IG 式 (36.4) 的相似性。PG 利用逆函数全部信息，包括高阶项。[10]

[9] 译注：上述 $\partial y/\partial L$ 若 y 为多维而 L 标量，并非唯一逆；末节也明确要另选右逆或输出步。矩形 Jacobian 的广义逆不等于此处假定的双侧逆。

[10] 译注：向量除以向量没有一般定义；源分式应限于标量或选定路径的方向性差商理解，实际向量更新为分子之差。精确输出等式要求所选逆在更新点确为右逆，近似局部逆则只有相应误差保证。

36.7 小结

普通 GD 有惊人的尺度缺陷。IG 和 Newton 等基于求逆的经典方法消除部分问题；略理论化的逆仿真更新 PG 包含最多高阶信息，可视为改进更新的理想情形：正确处理单位、函数敏感度、复合及最优点附近收敛，并具有尺度不变性，代价是需要构造、离散化局部逆求解器。相比 Newton 二阶与 IG 一阶近似，PG 可计及高度非线性。构造逆仿真器困难，故后面要展示纳入高阶信息的收益。三方法都通过求逆重缩放搜索方向，与此前 GD 不同；它们仍是不同形式的可微物理。引入网络前，先讨论补充理论，再用实际例子说明差别。

注 以下深入讨论逆求解器优化；若更关心实际例子或网络联系，可分别跳读不同优化器简单示例或尺度不变物理训练章。

36.8 深入逆仿真器

物理过程可表为状态空间轨迹，每点是系统配置。仿真器通常把一个状态向量变为另一时刻的状态，故其 Jacobian 为方阵。只要过程不破坏信息，Jacobian 非奇异。源文进一步以宇宙信息不能被毁灭的观点，认为掌握完整状态时原则上可逆，所以许多设置能构造 $\mathcal{P}^{-1}$。上述更新还具良好理论性质，例如局部逆持续指向固定目标 x^* 时优化收敛，证明细节见 [HKT22]。[11]

36.8.1 微积分基本定理

用微积分基本定理重写比值：

$$\frac{\Delta x_{\text{PG}}}{\Delta y} = \frac{\displaystyle\int_{y_0}^{y_0+\Delta y} \frac{\partial x}{\partial y}\, dy}{\Delta y}.$$

积分内为局部梯度，假定在 y_0 与 $y_0 + \Delta y_0$ 间处处存在（源文此处多写下标 0）。沿更新前后连接路径平均局部梯度，因此源文称它为 $\mathcal{P}$ 在当前 x 和下一估计 $x_0 + \Delta x_{\text{PG}}$ 间的平均梯度。等效于以计及非线性的更新平滑优化景观。高维可把积分换为连接两端的可微路径积分，并将局部梯度换成沿路径方向的导数。[12]

36.8.2 全局与局部逆仿真器

全局逆只对双射 $\mathcal{P}$ 定义，可对任意值域中的 y 找到 x。实践常用以 x_0 及 $y_0 = \mathcal{P}(x_0)$ 为条件的局部逆，只需在附近存在准确，不需所有 y。若全局逆存在，局部逆近似它，并随 $y \to y_0$ 精确匹配，源

[11] 译注：相同状态维数、无信息损失和 Jacobian 非奇异是不同条件；双射可在某点导数为零，如 x^3。粗粒化、耗散离散化或噪声也不由微观信息观点保证可逆。收敛须依原文献具体假设，非任意近似逆均成立。

[12] 译注：显示积分实际平均的是逆映射 $x(y) = \mathcal{P}^{-1}(y)$ 的导数，而非正向 $\mathcal{P}$；高维须先指定输出路径并积分 $D\mathcal{P}^{-1}(y)\, dy$，不能直接保留一般向量分式。

文形式化为

$$\lim_{y\to y_0}\frac{\mathcal{P}^{-1}(y;x_0)-P^{-1}(y)}{|y-y_0|}=0.$$

即使全局逆不存在，也可有局部逆。例如非单射 $\mathcal{P}(x)=x^2$ 每个正输出有两解，选择离 x_0 最近的分支即可。对可微函数，只要 Jacobian 非奇异，逆函数定理保证局部逆；逆 Jacobian 给出一阶局部逆近似。即使 Jacobian 奇异（源文举非单射、混沌或噪声），通常仍可找良好的局部逆。[13]

36.8.3 时间反演

逆仿真器通常对应时间反演物理过程。某些情形把正向仿真 $t\mapsto -t$ 就得到合适全局逆；若数值误差累积或刚性系统未在实践中破坏信息，可据此开始逆仿真或构造局部逆。求解器还需足够准确；如多步流体等复杂问题，Navier–Stokes 正向仿真章的一、二阶格式并不足够。

36.8.4 引入损失函数

重新考虑标量 $L(y)$，$y=\mathcal{P}(x)$ 可逆。包含损失的 IG 为 $\Delta x=(\partial x/\partial L)\Delta L$，其中 ΔL 是损失步。用链式法则并以逆仿真替代 IG，一阶上为

$$\begin{aligned}\Delta x_{\mathrm{PG}}&=\frac{\partial x}{\partial L}\Delta L=\frac{\partial x}{\partial y}\left(\frac{\partial y}{\partial L}\Delta L\right)=\frac{\partial x}{\partial y}\Delta y\\&=\mathcal{P}^{-1}(y_0+\Delta y;x_0)-x_0+\mathcal{O}(\Delta y^2).\end{aligned}$$

所以 PG 与 IG 一阶相同，却含非线性项，源文继写 $\Delta x_{\mathrm{PG}}/\Delta y=\partial x/\partial y+\mathcal{O}(\Delta y^2)$，更新精度依逆函数保真度。可用局部导数为局部逆误差定上界，最坏回退普通梯度。[14] 损失步已转为输出步 Δy，但并不唯一：$\partial y/\partial L$ 作为行向量 $\partial L/\partial y$ 的右逆，对 Δy 限制很少。因此用 Newton 式 (36.3) 确定 Δy，由 η 控步长。在下一代码例后结合网络进一步说明。

译注：独立核对的适用条件 梯度趋零不意味着 Hessian 特征值趋零，Newton 步也通常趋零：$L=x^2$ 的 Hessian 恒为 2，更新 $-\eta x\to 0$；更快局部收敛还需正则性、非奇异性等条件。非凸 Newton 也未必下降，如 $L=-x^2$、$\eta=1$ 将 x 移至 0 却增加 L。标量 $\Delta y=0$ 时源 PG 分式为 $0/0$，应以逆映射差值或连续延拓定义零步。连续流反演、离散映射求逆和数值近似也须区分：Euler 求解 $x\prime=ax$，先步长 h 再步长 $-h$ 的因子是 $1-a^2h^2\neq 1$，即使没有浮点误差也不互逆。一致无量纲化并同步变换算子与损失可保持物理问题，例如 $\widetilde{\mathcal{P}}(\tilde{x})=S_y^{-1}\mathcal{P}(S_x\tilde{x})$；只在未同步变换时，归一化才会改变被求解的问题，不能概括为物理仿真没有归一化或预条件手段。

[13]译注：逆函数定理的常见版本要求邻域内连续可微；单个逆 Jacobian 是线性化而非完整逆函数。x^2 在零点无可微双侧局部逆，分支选择需范围条件。混沌、噪声和非单射也并不分别等价于 Jacobian 奇异，最后一句仅为实践性期望。

[14]译注：一般光滑标量逆函数的差商误差是 $\mathcal{O}(\Delta y)$，而非源写的平方阶；若非对称差商，无额外消去条件不能提高阶。前一行也混合线性 IG 与有限 PG 的命名，等价性仅按一阶理解。仅有一个局部梯度值不足以给有限邻域误差界，还需正则性及导数控制等条件。

第三十七章　比较不同优化器的简单示例

上一章讨论了优化方法的诸多利弊，下面用实际例子展示这些性质究竟造成多大差异。源文附 “在 Colab 中运行” 链接。

37.1　问题表述

取很简单的设置：二维输入 $\mathbf{x}\in\mathbb{R}^2$、二维模拟 “物理模型” $\mathbf{y}:\mathbb{R}^2\to\mathbb{R}^2$ 及标量损失 $L:\mathbb{R}^2\to\mathbb{R}$。向量分量记 x_i，与 Python 一致从 0 编号。

$$\mathbf{y}(\mathbf{x})=\mathbf{y}(x_0,x_1)=\begin{bmatrix}x_0\\ x_1^2\end{bmatrix},\qquad\qquad L(\mathbf{y})=|\mathbf{y}|^2=y_0^2+y_1^2.$$

即只平方第二输入分量，损失是平方 L2。以 $\mathbf{x}=[3\ 3]^T$ 为起点求 $\arg\min_{\mathbf{x}}L(\mathbf{x})$。人一眼就知道答案是 $[0\ 0]^T$，但要看算法多快找到。函数虽简单，x_1^2 仍使其非线性。

37.2　三个空间

务必记住输入 $\mathbf{x}$、物理 $\mathbf{y}$ 和损失 L 三空间。前向把 $\mathbf{x}$ 经 $\mathbf{y}$ 映到 L；优化则要把损失值及变化对应回输入位置，同时研究寻找输入解时物理空间的位置如何变化。

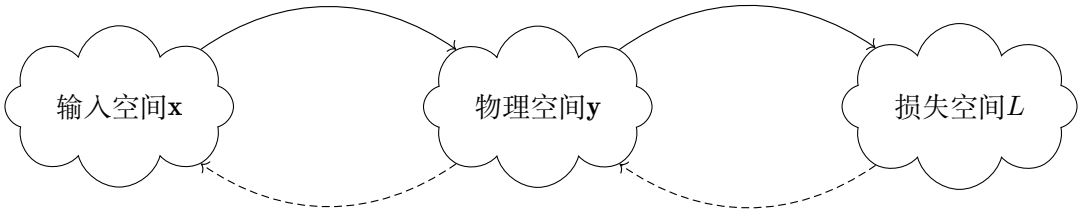


图 37.1: 由物理输出计算的损失求回输入，构成逆问题。前向依次经过 $\mathbf{x},\mathbf{y},L$，反向回到输入；输入精度最关键，也可追踪物理及损失空间的进展。

37.3　实现

采用便于高效操作可微函数的 JAX。其 numpy 包装实现大部分 numpy 函数，下记 `np`，原 numpy 记 `onp`。

```
import jax
import jax.numpy as np
import numpy as onp
```

定义物理、损失及调用二者的复合函数。许多 JAX 操作需要单个原生 Python 函数。源说明称此复合函数 “fun”，实际命名为 `loss_x`。

```
# "physics" function y
def physics_y(x):
    return np.array( [x[0], x[1]*x[1]] )
# simple L2 loss
def loss_y(y):
    #return y[0]*y[0] + y[1]*y[1] # "manual version"
    return np.sum( np.square(y) )
# composite function with L & y , evaluating the loss for x
def loss_x(x):
    return loss_y(physics_y(x))
```

```
x = np.asarray([3,3], dtype=np.float32)
print("Starting point x = "+format(x) +"\n")
print("Some test calls of the functions we defined so far, from top to bottom, y,
↪  manual L(y), L(y):")
physics_y(x) , loss_y( physics_y(x) ), loss_x(x)
```

原书输出：

```
Starting point x = [3. 3.]
Some test calls of the functions we defined so far, from top to bottom, y, manual L(y),
↪  L(y):
(DeviceArray([3., 9.], dtype=float32),
DeviceArray(90., dtype=float32),
DeviceArray(90., dtype=float32))
```

用 `jax.grad` 评价导数，例如 `jax.grad(loss_y)(physics_y(x))` 计算 $\partial L/\partial \mathbf{y}$。还计算几个变体并用逆 Jacobian 作一致性检查。

```
# this works:
print("Jacobian L(y): " + format(jax.grad(loss_y)(physics_y(x))) +"\n")
# the following would give an error as y (and hence physics_y) is not scalar
#jax.grad(physics_y)(x)
# computing the jacobian of y is a valid operation:
J = jax.jacobian(physics_y)(x)
print( "Jacobian y(x): \n" + format(J) )
# the code below also gives error, JAX grad needs a single function object
#jax.grad( loss_y(physics_y) )(x)
print( "\nSanity check with inverse Jacobian of y, this should give x again: " +
↪  format(np.linalg.solve(J, np.matmul(J,x) )) +"\n")
# instead use composite 'fun' from above
print("Gradient for full L(x): " + format( jax.grad(loss_x)(x) ) +"\n")
```

原书输出：

```
Jacobian L(y): [ 6. 18.]
Jacobian y(x):
[[1. 0.]
[0. 6.]]
Sanity check with inverse Jacobian of y, this should give x again: [3. 3.]
Gradient for full L(x): [ 6. 108.]
```

最后一行值得看：初态梯度为 $[6, 108]$。虽知应对角走向原点，梯度却沿 x_1 强烈占优。比较一阶普通非随机最陡下降 GD、二阶 Newton 及逆仿真器的尺度不变更新。[1]

37.4 梯度下降

式 (36.2) 在本例给出

$$\Delta \mathbf{x}_{\mathrm{GD}} = -\eta (J_L J_{\mathbf{y}})^T = -\eta \left(\frac{\partial L}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^T .$$

从 $[3, 3]$ 出发，以 $\eta = 0.01$ 更新十次：

```
x = np.asarray([3.,3.])
eta = 0.01
historyGD = [x]; updatesGD = []
for i in range(10):
    G = jax.grad(loss_x)(x)
    x += -eta * G
    historyGD.append(x); updatesGD.append(G)
    print( "GD iter %d: "%i + format(x) )
```

原书输出：

```
GD iter 0: [2.94 1.9200001]
GD iter 1: [2.8812 1.6368846]
GD iter 2: [2.823576 1.4614503]
GD iter 3: [2.7671044 1.3365935]
GD iter 4: [2.7117622 1.2410815]
GD iter 5: [2.657527 1.1646168]
GD iter 6: [2.6043763 1.1014326]
GD iter 7: [2.5522888 1.0479842]
GD iter 8: [2.501243 1.0019454]
GD iter 9: [2.4512184 0.96171147]
```

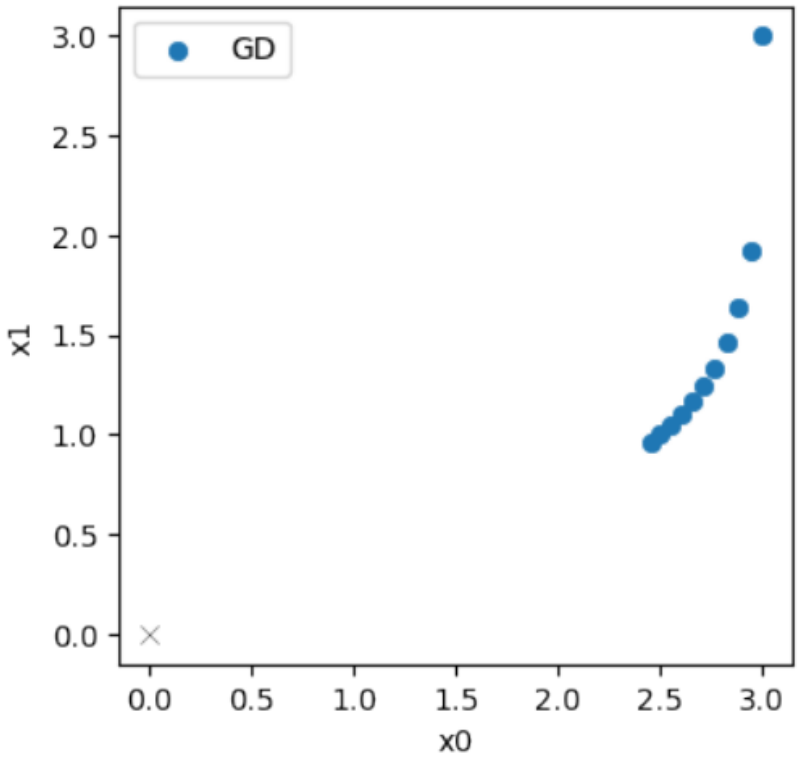
位置在下降，方向似乎正确；末点 $[2.451, 0.962]$ 距原点仍为 2.63。画出 `history` 中的过程，蓝点为 GD，细黑叉为目标。

[1] 译注：本章所有程序和输出仅从源转写、token/AST 核对，未执行优化或训练。保留原 JAX API 与浮点打印，不把源经验比较当普遍排名。

```
import matplotlib.pyplot as plt
axes = plt.figure(figsize=(4, 4), dpi=100).gca()
historyGD = onp.asarray(historyGD)
updatesGD = onp.asarray(updatesGD) # for later
axes.scatter(historyGD[:,0], historyGD[:,1], lw=0.5, color='#1F77B4', label='GD')
axes.scatter([0], [0], lw=0.25, color='black', marker='x') # target at 0,0
axes.set_xlabel('x0'); axes.set_ylabel('x1'); axes.legend()
```

原书输出：

```
<matplotlib.legend.Legend at 0x7fc742a69430>
```



果然初步在右上角主要沿 x_1 下移，之后弯向原点，却离左下的解仍远。

37.5　Newton 方法

更新为

$$\Delta\mathbf{x}_{\mathrm{N}} = -\eta\left(\frac{\partial^2 L}{\partial \mathbf{x}^2}\right)^{-1}\frac{\partial L}{\partial \mathbf{x}} = -\eta H^{-1}(J_L J_{\mathbf{y}})^T.$$

除同一梯度外，还要算并求逆 `loss_x` 的 Hessian。在 JAX 中连用两次 `jax.jacobian`，再用 `numpy.linalg.inv` 即可。取较大 $\eta = 1/3$。本例及下例步长选得首步长度与 GD 大致相同，便于相对比较轨迹；绝非比较稳定性或步长上界，这里聚焦收敛。仍从同一初态更新十次：

```
x = np.asarray([3.,3.])
eta = 1./3.
historyNt = [x]; updatesNt = []
Gx = jax.grad(loss_x)
Hx = jax.jacobian(jax.jacobian(loss_x))
for i in range(10):
    g = Gx(x)
    h = Hx(x)
    hinv = np.linalg.inv(h)
    x += -eta * np.matmul( hinv , g )
    historyNt.append(x); updatesNt.append( np.matmul( hinv , g) )
    print( "Newton iter %d: "%i + format(x) )
```

原书输出：

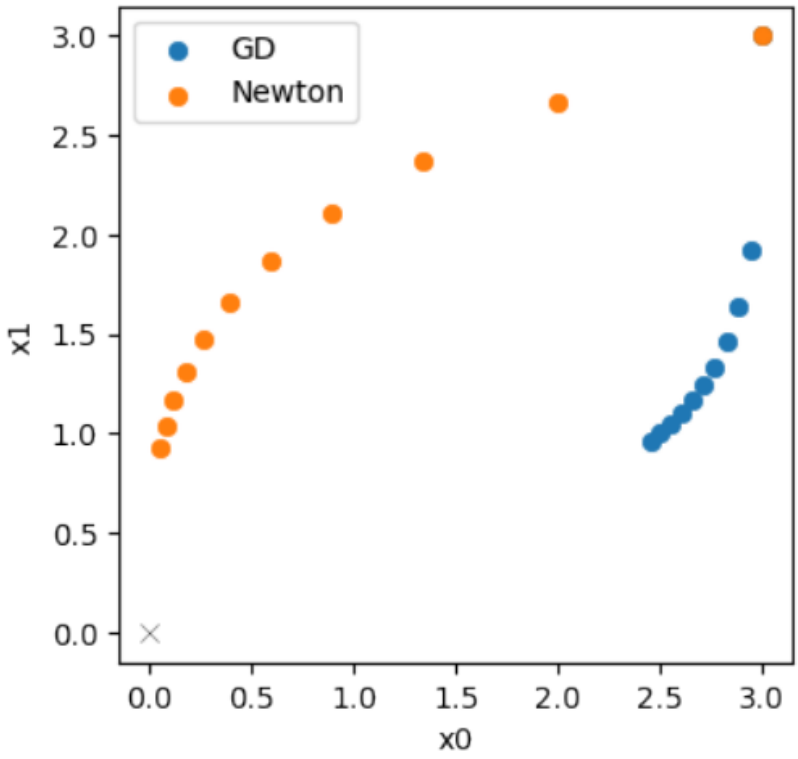
```
Newton iter 0: [2. 2.6666667]
Newton iter 1: [1.3333333 2.3703704]
Newton iter 2: [0.88888884 2.1069958 ]
Newton iter 3: [0.59259254 1.8728852 ]
Newton iter 4: [0.39506167 1.6647868 ]
Newton iter 5: [0.26337445 1.4798105 ]
Newton iter 6: [0.17558296 1.315387 ]
Newton iter 7: [0.1170553 1.1692328]
Newton iter 8: [0.07803687 1.0393181 ]
Newton iter 9: [0.05202458 0.92383826]
```

末点 $[0.052, 0.924]$ 距原点 0.925，明显优于 GD 的 2.63。把 Newton 橙色轨迹与 GD 蓝色同画。

```
axes = plt.figure(figsize=(4, 4), dpi=100).gca()
historyNt = onp.asarray(historyNt)
updatesNt = onp.asarray(updatesNt)
axes.scatter(historyGD[:,0], historyGD[:,1], lw=0.5, color='#1F77B4', label='GD')
axes.scatter(historyNt[:,0], historyNt[:,1], lw=0.5, color='#FF7F0E', label='Newton')
axes.scatter([0], [0], lw=0.25, color='black', marker='x') # target at 0,0
axes.set_xlabel('x0'); axes.set_ylabel('x1'); axes.legend()
```

原书输出：

```
<matplotlib.legend.Legend at 0x7fc7428c5bb0>
```



此例能可靠求 Hessian，Newton 得益于二阶信息，路径更接近理想的最短对角线，也不像 GD 那样减速。

37.6 逆仿真器

再用解析逆 $\mathbf{y}^{-1}(\mathbf{x}) = [x_0\ x_1^{1/2}]^T$，即前章 $\mathcal{P}^{-1}$，计算下称 PG 的尺度不变更新。预先采用损失空间 Newton 步，再与逆物理函数组合：

$$\Delta \mathbf{x}_{\mathrm{PG}} = \mathbf{y}^{-1}\left(\mathbf{y}(\mathbf{x}) - \eta \left(\frac{\partial^2 L}{\partial \mathbf{y}^2}\right)^{-1} \frac{\partial L}{\partial \mathbf{y}}\right) - \mathbf{x}.$$

源说明称逆函数 `physics_y_inv_analytic`，实际代码命名 `physics_y_inv`。定义后做十步：

```
x = np.asarray([3.,3.])
eta = 0.3
historyPG = [x]; historyPGy = []; updatesPG = []
def physics_y_inv(y):
    return np.array( [y[0], np.power(y[1],0.5)] )
Gy = jax.grad(loss_y)
Hy = jax.jacobian(jax.jacobian(loss_y))
for i in range(10):
    # Newton step for L(y)
    zForw = physics_y(x)
    g = Gy(zForw)
```

```
    h = Hy(zForw)
    hinv = np.linalg.inv(h)
    # step in y space
    zBack = zForw -eta * np.matmul( hinv , g)
    historyPGy.append(zBack)
    # "inverse physics" step via y-inverse
    x = physics_y_inv(zBack)
    historyPG.append(x)
    updatesPG.append( historyPG[-2] - historyPG[-1] )
    print( "PG iter %d: "%i + format(x) )
```

原书输出：

```
PG iter 0: [2.1 2.5099802]
PG iter 1: [1.4699999 2.1000001]
PG iter 2: [1.0289999 1.7569861]
PG iter 3: [0.72029996 1.47 ]
PG iter 4: [0.50421 1.2298902]
PG iter 5: [0.352947 1.029 ]
PG iter 6: [0.24706289 0.86092323]
PG iter 7: [0.17294402 0.7203 ]
PG iter 8: [0.12106082 0.60264623]
PG iter 9: [0.08474258 0.50421 ]
```

原书独立输入（历史数组转换）：

```
historyPG = onp.asarray(historyPG)
updatesPG = onp.asarray(updatesPG)
```

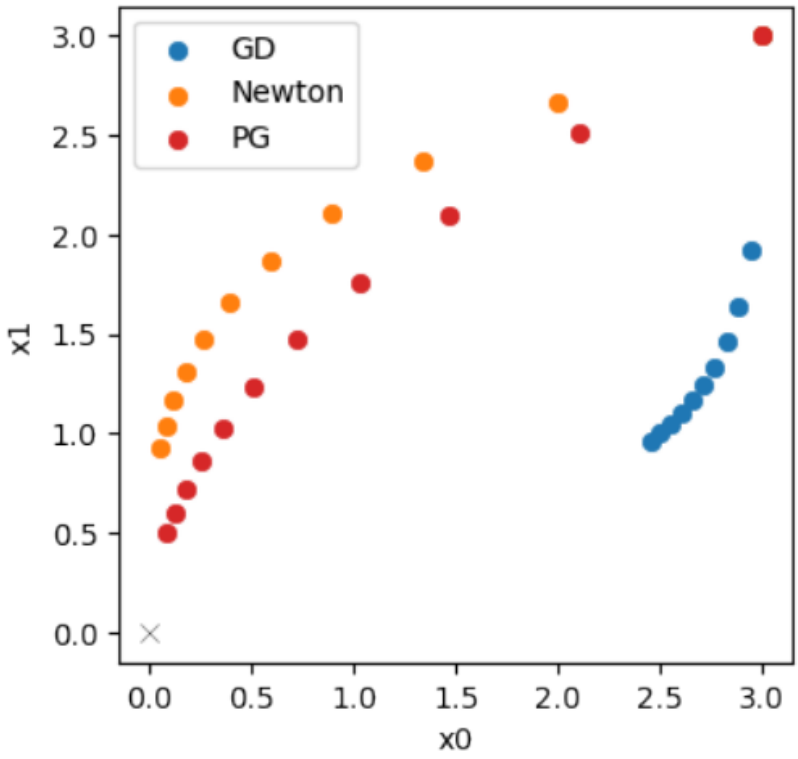
[2] 末位置约 $[0.084, 0.504]$，距原点仅 0.51，优于这里的 Newton 和 GD。将 PG 红色与二者比较。

```
axes = plt.figure(figsize=(4, 4), dpi=100).gca()
axes.scatter(historyGD[:,0], historyGD[:,1], lw=0.5, color='#1F77B4', label='GD')
axes.scatter(historyNt[:,0], historyNt[:,1], lw=0.5, color='#FF7F0E', label='Newton')
axes.scatter(historyPG[:,0], historyPG[:,1], lw=0.5, color='#D62728', label='PG')
axes.scatter([0], [0], lw=0.25, color='black', marker='x') # target at 0,0
axes.set_xlabel('x0'); axes.set_ylabel('x1'); axes.legend()
```

原书输出：

```
<matplotlib.legend.Legend at 0x7fc742ddba60>
```

[2]译注：平方映射无全局单值逆；这里取非负平方根分支，适用于本例正初态及非负输出更新，不能据此覆盖负支或负输出。



红色 PG 轨迹更接近理想对角线，终点更近。关键是逆函数提供超过 Newton 二阶近似的高阶信息，改善尺度不变性。即使简单例子，Newton 仍难找对搜索方向，逆仿真高阶信息则改善方向。再测各首步与对角方向的对齐：

```
def mag(x):
    return np.sqrt(np.sum(np.square(x)))
def one_len(x):
    return np.dot( x/mag(x), np.array([1,1]))
print("Diagonal lengths (larger is better): GD %f, Nt %f, PG %f " %
      (one_len(updatesGD[0]) , one_len(updatesNt[0]) , one_len(updatesPG[0])) )
```

原书输出：

```
Diagonal lengths (larger is better): GD 1.053930, Nt 1.264911, PG 1.356443
```

PG 最大值 1.356 支持图像观察：其首步最接近起点到原点的对角方向。[3]

37.7 物理空间

不能把输入到损失仅当黑箱，中间空间也重要。这里仅一个 $\mathbf{y}$，深网络可能有许多部分可控的潜空间。PG 显式从 L 映到 $\mathbf{y}$，再映到 $\mathbf{x}$，故已在 `historyPGy` 存下物理空间轨迹。

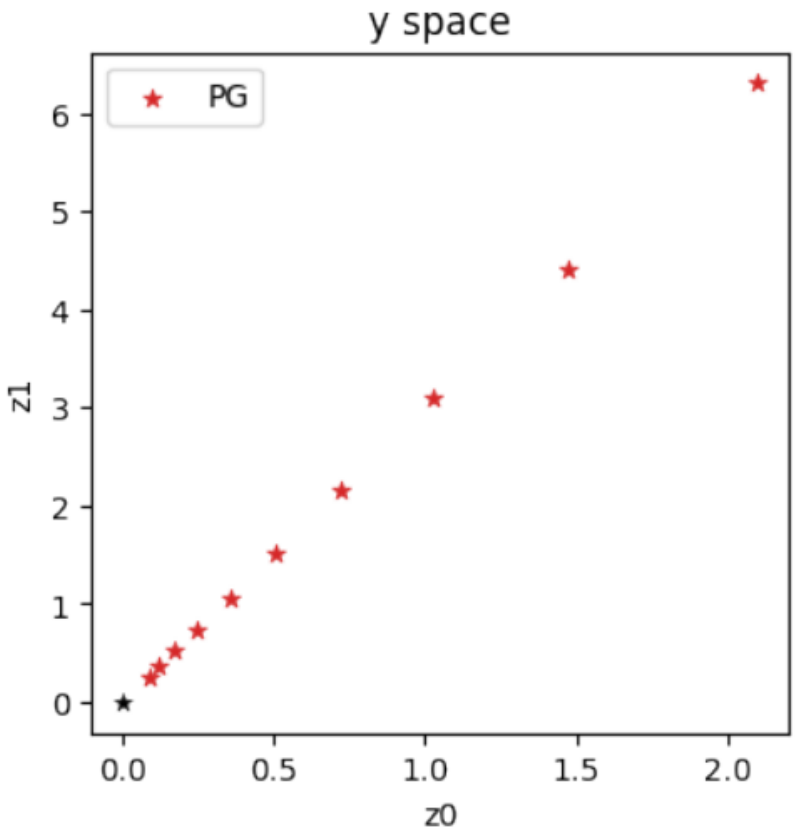
```
historyPGy = onp.asarray(historyPGy)
axes = plt.figure(figsize=(4, 4), dpi=100).gca()
```

[3]译注：代码将更新单位化后点乘未单位化的 $[1,1]$，故理想值为 $\sqrt{2}$，不是余弦值上界 1。三法步长不同，结论限于该比较设置。

```
axes.set_title('y space')
axes.scatter(historyPGy[:,0], historyPGy[:,1], lw=0.5, color='#D62728', marker='*',
↪  label='PG')
axes.scatter([0], [0], lw=0.25, color='black', marker='*')
axes.set_xlabel('z0'); axes.set_ylabel('z1'); axes.legend()
```

原书输出：

```
<matplotlib.legend.Legend at 0x7fc742e83a90>
```



它在物理空间作显式步，沿一条直线走向原点。[4] GD 和 Newton 则不直接给中间空间进展。GD 连接 Jacobian，局部方向应降损失，但物理位置受输入影响，直到确定更新输入才能知道终点。对网络，实际算出全部更新权重前，也不知道各潜空间终点。GD 步为 $-\eta\partial L/\partial\mathbf{x}$，所以物理空间到达 $\mathbf{y}(\mathbf{x}-\eta\partial L/\partial\mathbf{x})$。令 $h=\eta\partial L/\partial\mathbf{x}$，源文 Taylor 示意为

$$\begin{aligned}\mathbf{y}(\mathbf{x}-h) &= \mathbf{y}(\mathbf{x}) - h\frac{\partial\mathbf{y}}{\partial\mathbf{x}} + \mathcal{O}(h^2)\\ &= \mathbf{y}(x) - \eta\frac{\partial L}{\partial\mathbf{y}}\left(\frac{\partial\mathbf{y}}{\partial x}\right)^2 + \mathcal{O}(h^2).\end{aligned}$$

这显然不同于直接对物理空间做 GD 的 $\partial L/\partial\mathbf{y}$。[5] Newton 也只计算全链一、二阶近似，中间实际状态仍须整个更新链评价后才知道，前章复合一致性讨论中的 Jacobian 平方已说明此依赖。逆仿真器可直

[4] 译注：源称 “straight diagonal”；本例物理初态为 $[3,9]$，直线指该初态至原点，不是等分量的 45 度方向。

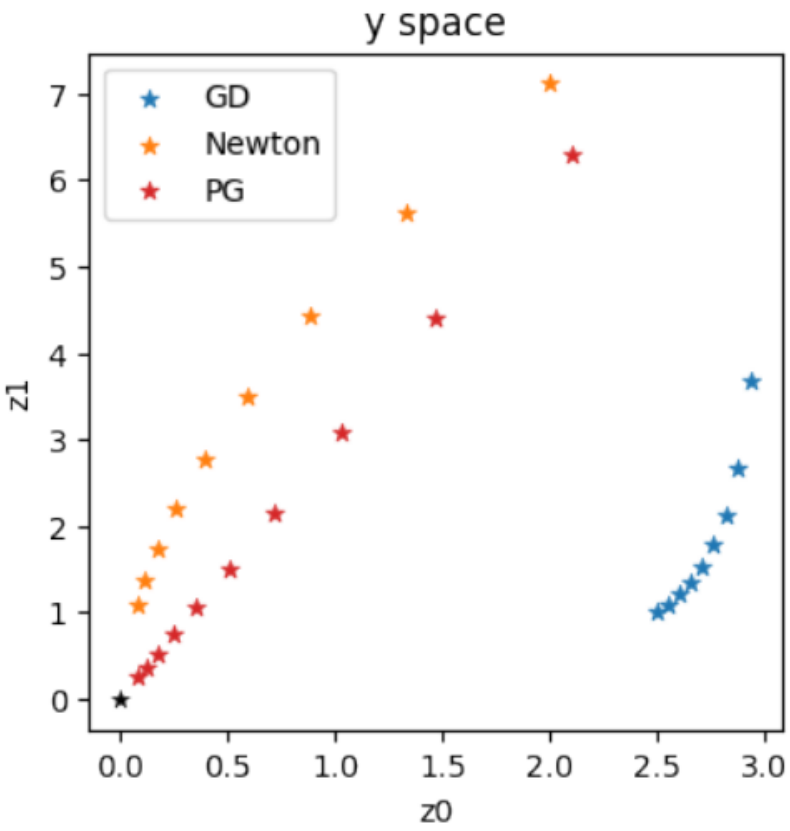
[5] 译注：源示意省略向量转置及矩阵方向；列梯度约定下物理变化为 $-\eta JJ^T\nabla_y L+\mathcal{O}(\|h\|^2)$，一般不是元素平方。原式保留供对照。

接把物理点映到输入，因此确切知道用于求逆的物理位置。这里已存输入历程，重新评价 physics_y 就得到 GD 和 Newton 的物理位置。

```
x = np.asarray([3.,3.])
eta = 0.01
historyGDy = []
historyNty = []
for i in range(1,10):
    historyGDy.append(physics_y(historyGD[i]))
    historyNty.append(physics_y(historyNt[i]))
historyGDy = onp.asarray(historyGDy)
historyNty = onp.asarray(historyNty)
axes = plt.figure(figsize=(4, 4), dpi=100).gca()
axes.set_title('y space')
axes.scatter(historyGDy[:,0], historyGDy[:,1], lw=0.5, marker='*', color='#1F77B4',
↪  label='GD')
axes.scatter(historyNty[:,0], historyNty[:,1], lw=0.5, marker='*', color='#FF7F0E',
↪  label='Newton')
axes.scatter(historyPGy[:,0], historyPGy[:,1], lw=0.5, marker='*', color='#D62728',
↪  label='PG')
axes.scatter([0], [0], lw=0.25, color='black', marker='*')
axes.set_xlabel('z0'); axes.set_ylabel('z1'); axes.legend()
```

原书输出：

```
<matplotlib.legend.Legend at 0x7fc7430c4b20>
```



蓝色 GD 路径很不理想，橙色 Newton 较好却仍弯曲，不足以近似此例高阶项；红色逆仿真路径是直线。中间空间若具有实际物理含义，而不只是抽象潜空间，这更重要。[6]

37.8 结论

简单例子已显示 GD、Newton 和逆仿真显著不同：

- GD 易产生“失衡”更新，进展停滞；
- Newton 较好，但仍非最优；
- 逆仿真高阶信息更好，即使只部分使用（损失端仍用 Newton）；
- 优化器选择还强烈影响物理等潜空间进展。

后章据此用可逆物理模型的 PG 训练网络。

37.9 近似求逆

若解析逆不可得，可用 Newton、BFGS 等数值优化求局部逆。这与外层优化器比较是另一个问题，但可在本例说明。采用 scipy 的 `fmin_l_bfgs_b`，内部切回普通 numpy 和 scipy，代码略显繁琐。

[6]译注：源 GD/Newton 物理重算循环为 `range(1,10)`，不含初态和第十次更新点；PG 列表存储口径不同，按源保留。

```
def physics_y_inv_opt(target_y, x_ini):
    # a bit ugly, we switch to pure scipy here inside each iteration for BFGS
    import numpy as np
    from scipy.optimize import fmin_l_bfgs_b
    target_y = onp.array(target_y)
    x_ini = onp.array(x_ini)
    def physics_y_opt(x,target_y=[2,2]):
        y = onp.array( [x[0], x[1]*x[1]] ) # we cant use physics_y from JAX here
        ret = onp.sum( onp.square(y-target_y) )
        return ret
    ret = fmin_l_bfgs_b(lambda x: physics_y_opt(x,target_y), x_ini, approx_grad=True )
    #print( ret ) # return full BFGS details
    return ret[0]
print("BFGS optimization test run, find x such that y=[2,2]:")
physics_y_inv_opt([2,2], [3,3])
```

原书输出：

```
BFGS optimization test run, find x such that y=[2,2]:
array([2.00000003, 1.41421353])
```

现在把数值逆用于 PG，除调用 `physics_y_inv_opt` 外其余相同。

```
x = np.asarray([3.,3.])
eta = 0.3
history = [x]; updates = []
Gy = jax.grad(loss_y)
Hy = jax.jacobian(jax.jacobian(loss_y))
for i in range(10):
    # same as before, Newton step for L(y)
    y = physics_y(x)
    g = Gy(y)
    y += -eta * np.matmul( np.linalg.inv( Hy(y) ) , g)
    # optimize for inverse physics, assuming we dont have access to an inverse for
    ↪ physics_y
    x = physics_y_inv_opt(y,x)
    history.append(x)
    updates.append( history[-2] - history[-1] )
    print( "PG iter %d: "%i + format(x) )
```

原书输出：

```
PG iter 0: [2.09999967 2.50998022]
PG iter 1: [1.46999859 2.10000011]
PG iter 2: [1.02899871 1.75698602]
PG iter 3: [0.72029824 1.4699998 ]
PG iter 4: [0.50420733 1.22988982]
PG iter 5: [0.35294448 1.02899957]
PG iter 6: [0.24705997 0.86092355]
```

```
PG iter 7: [0.17294205 0.72030026]
PG iter 8: [0.12106103 0.60264817]
PG iter 9: [0.08474171 0.50421247]
```

结果与解析 PG 相符，不必再画基本相同的轨迹。算二者 MAE，此例约为所用浮点精度量级。

```
historyPGa = onp.asarray(history)
updatesPGa = onp.asarray(updates)
print("MAE difference between analytic PG and approximate inversion: %f" %
↪  (np.average(np.abs(historyPGa-historyPG))) )
```

原书输出：

```
MAE difference between analytic PG and approximate inversion: 0.000001
```

37.10 下一步

- 用其他更复杂的 $L(\mathbf{y}(\mathbf{x}))$ 替换简单函数；
- 用 AdaGrad、RmsProp 或 Adam 等常用优化器替换普通 GD，与已有轨迹比较。

第三十八章　尺度不变的物理训练

前两章表明，梯度求逆是优化与学习的重要一步。现在把 Δx_{PG} 纳入神经网络训练，详细说明简单例子已采用的双向过程：逆仿真器与损失的 Newton 步。以下聚焦逆问题的网络解：$y = \mathcal{P}(x)$，训练表示 f 使 $f(y;\theta) = x$。这比此前可微物理（DP）训练的设置稍受约束。由于现在讨论优化算法，不再单独标出 DP 方法：下列变体都包含物理仿真器，GD 及 Adam 等变体仍使用 DP 训练。

注意：与此前章节及“模型与方程”不同，此处求逆问题，网络输入是 y，输出是 x。

以 i 为小批索引，最小化问题为

$$\arg\min_{\theta} \sum_i \frac{1}{2} \left| \mathcal{P}(f(y_i^*;\theta)) - y_i^* \right|_2^2 . \tag{38.1}$$

38.1　神经网络训练

把式 (36.5) 的更新纳入网络训练，需考虑网络、物理仿真器和损失函数三部分。用以下算法连接三

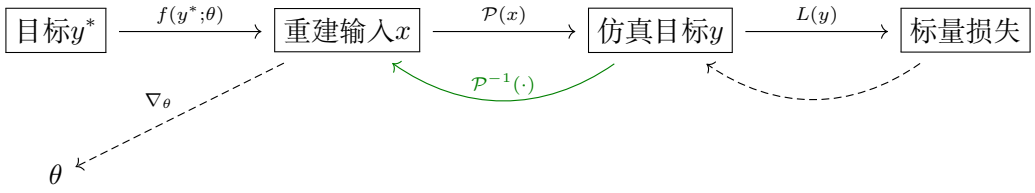


图 38.1: SIP 训练涉及的不同空间。原图 38.1 的节点和前向、逆向关系原生重建。

部分，依 Holl 等 [HKT22] 称为**尺度不变物理（SIP）训练**。

SIP 训练：更新网络 f 的权重 θ 时：

1. 给定输入集 y^*，前向计算网络预测 $x = f(y^*;\theta)$。
2. 前向仿真 $y = \mathcal{P}(x)$，调用局部逆仿真器 $P^{-1}(y;x)$，获得步 $\Delta x_{\mathrm{PG}} = \mathcal{P}^{-1}(y + \eta\Delta y; x)$，其中 $\Delta y = y^* - y$。[1]
3. 评价网络损失，例如 $L = \frac{1}{2}|x - \tilde{x}|_2^2$，其中 $\tilde{x} = x + \Delta x_{\mathrm{PG}}$；将 $\tilde{x}$ 视为常量，作 Newton 步。
4. 用 GD 或 Adam 等基于 GD 的优化器，把 x 的变化传播到网络权重 θ，学习率为 η_{NN}。

[1]译注：源算法此行把逆仿真所得状态直接称作步，未减去 x；与第 36 章的差值定义及下一行 $\tilde{x} = x + \Delta x_{\mathrm{PG}}$ 不一致。若逆返回状态，应取 $\mathcal{P}^{-1}(y + \eta\Delta y; x) - x$；若实现返回增量，须明确该约定。源式照录。

组合算法同时依赖网络学习率 η_{NN} 和决定 Δy 的步长 η。一阶上有效学习率为 $\eta_{\text{eff}} = \eta\eta_{\text{NN}}$。建议在逆仿真器精度允许时尽量增大 η；许多情形可取 1，否则相应调节 η_{NN}，以充分利用仿真器的非线性来调整优化方向。算法将逆仿真器的准确高阶更新与传统网络训练结合，因而保留了现有网络训练方法的用途。损失作为网络与物理部分之间的“胶水”，起核心作用。

38.2 损失函数

上述算法假设 L^2 损失，并引入 Newton 步把逆仿真器的步传播给网络。下面解释这一处理。引入 Newton 步的主要理由是提高损失导数的准确性。与式 (36.3) 的常规 Newton 或拟 Newton 不同，只需要 $L(y)$ 的 Hessian，无需整个系统的 Hessian。这再次使 Newton 方法具有吸引力，而且许多常用损失的 Newton 更新有解析式。例如最常见监督目标 $L(y) = \frac{1}{2}|y - y^*|_2^2$，$y$ 是预测，y^* 是目标，则 $\partial L/\partial y = y - y^*$、$\partial^2 L/\partial y^2 = 1$，代入式 (36.3) 直接得 $\Delta y = \eta(y^* - y)$，无需额外评价 Hessian 矩阵。[2] 确定 Δy 后，用逆仿真器 $\mathcal{P}^{-1}$ 将变化传回 x（例如更早时刻）。此前优化器简单比较已使用这一“损失 Newton 步 + PDE 逆仿真器”组合。这里的 x 空间损失只是把逆仿真更新嵌入网络训练的代理，不是传统的 x 空间监督损失。由于 $\mathcal{P}^{-1}$ 依赖预测 y，它不会对 x 中的多个解模式直接求平均。例如若以 GD 求逆仿真，总损失仍纯粹定义在 y 空间，退化为普通一阶优化。因此 SIP 对 y 空间损失采用平凡 Newton 步，以 x 空间代理 L^2 损失连接逆物理与网络的反向传播计算图。

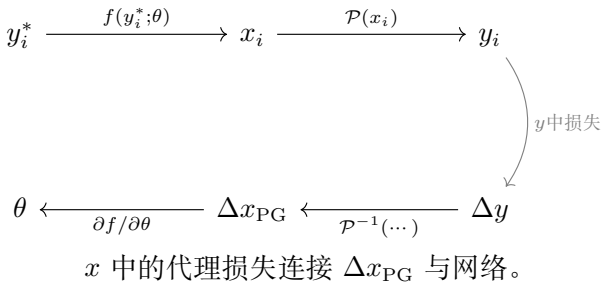


图 38.2: 小批中第 i 个条目的 SIP 训练，包括 y 中损失与 x 中代理损失。原图 38.2。

38.3 迭代与时间依赖

上述过程针对作一次预测的网络，适合从 $t_e > t_0$ 时刻状态重建 t_0 状态，或估计满足末端条件的最优初态。SIP 也可用于多个目标、多个时刻网络交互的复杂设置。例如控制任务中网络每步施小力以在 t_e 达到指定状态；或修正任务中每步修正仿真以提高质量。此时反复执行损失 Newton 步、物理逆仿真步和网络 GD 步，例如沿不同时间步重复，得到 L 中一系列相加项。反复通过物理求解器与网络反传通常使学习更难，但算法可像普通 GD 训练一样扩展。

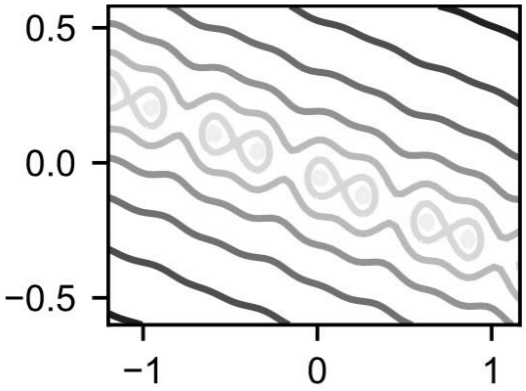
[2] 译注：多维欧氏平方损失的 Hessian 是单位矩阵 I，源式 1 为标量简写。

38.4 SIP 训练实例

用 [HKT22] 的例子说明 SIP 收敛与 $\mathcal{P}$ 特征的关系。考虑二维合成函数

$$\mathcal{P}(x) = \big(\sin(\hat{x}_1)/\xi,\ \hat{x}_2\xi\big), \qquad \hat{x} = R_\phi x, \quad R_\phi \in \mathrm{SO}(2).$$

ξ 与 ϕ 连续改变系统特征：大 ξ 表示病态问题，ϕ 描述 x_1, x_2 耦合。$\phi = 0$ 时 Hessian 非对角项消失，问题分为两个独立问题。以下损失景观取 $y^* = (.3, -.5)$、$\xi = 1$、$\phi = 15°$，展示 x_1 正弦与 x_2 线性变化的耦合。



接着按式 (38.1) 训练全连接网络求逆，将使用无鞍点 Newton 求解器的 SIP 与多种先进网络优化器比较。每种优化器独立选择最优学习率，以求公平。$\xi = 1$ 时源文称问题完全良态，各优化器都收敛，Adam 略占优势，见左图。[3] $\xi = 32$ 时已相当病态，右图仅 SIP 与 Adam 取得显著优化进展。两图横轴都是

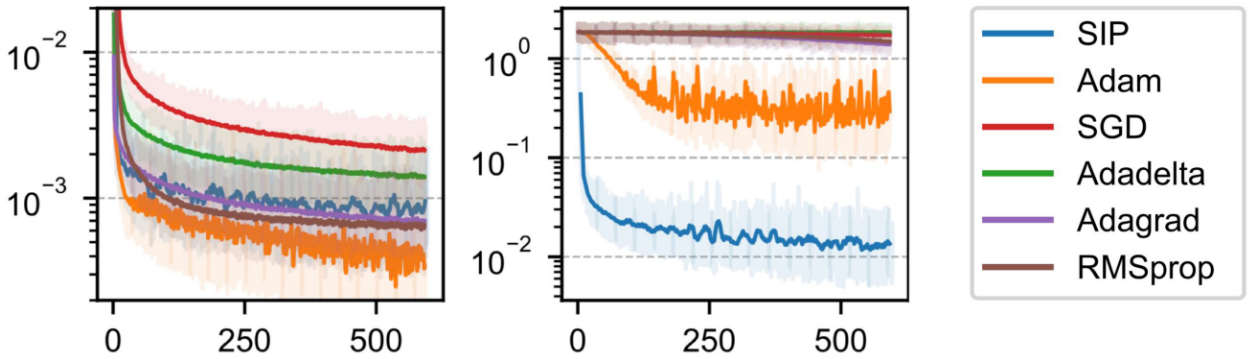


图 38.3: 良态（左）与病态（右）例子的损失随时间（秒）变化。原图 38.3。

时间；SIP 较慢主要因为每次迭代平均耗时为 Adam 的 3 倍。Hessian 评价本就需更多计算，但优化实现可能显著减少单步耗时。固定 $\phi = 0$、增大 ξ，可连续展示条件变化的影响；固定 $\xi = 32$、改变 ϕ，则展示维度耦合的作用。传统优化器精度下降，是因为 x 中梯度按 $(1/\xi, \xi)$ 缩放，在需要更精确取值的 x_2 方向更长。SIP 利用 Hessian 反转这种缩放，产生对齐 x 中平坦方向的更新，使相对精度在大范围 ξ 下保持；Adam 也随 ξ 增大而变差。改变 ϕ 还解释了 Adam 如何学习病态问题：当两参数尺度不同

[3] 译注：非线性正弦映射的局部 Jacobian 奇异值仍含 $|\cos \hat{x}_1|$；$\xi = 1$ 不使全域条件数恒为 1，“完全良态”按该实验区域的描述理解。

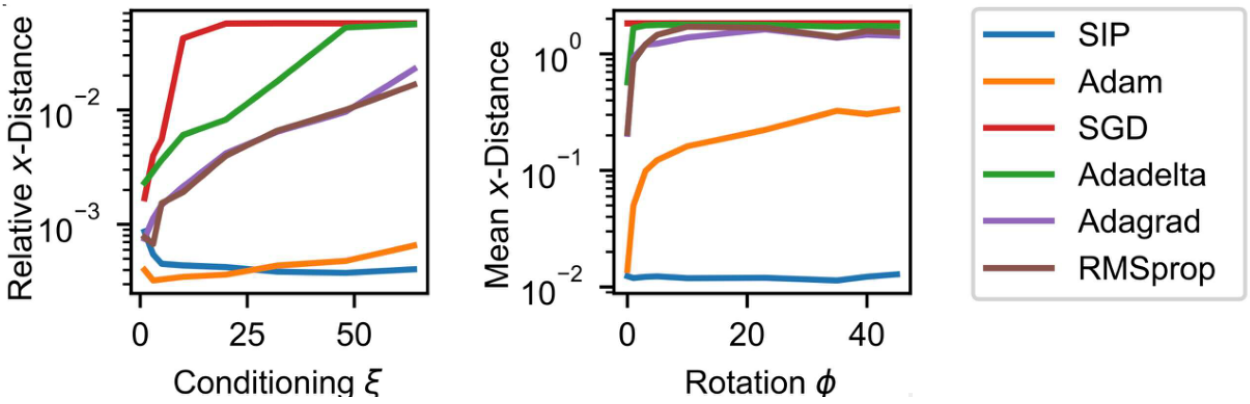


图 38.4: 改变条件参数（左）与旋转导致的维度耦合（右）时的性能。原图 38.4。

时，其 Hessian 对角近似减轻缩放效应；参数耦合后，缺失非对角项便妨碍这一点，性能下降超过一个数量级。SIP 优化步使用完整秩 Hessian $\partial^2 L/\partial x^2$，因而在此组问题中表现最好。[4]

38.5 SIP 训练的讨论

虽仅考察较小玩具问题，仍先讨论 SIP；下一章给出更复杂例子，随后转向新算法。总体而言，作者称 SIP 的尺度不变性使许多物理问题比其他学习方法快指数级，同时成本较低；每次求解器评价后执行足够多网络更新 $\Delta\theta$ 时可证明收敛，并可展示许多物理实验只需单次 $\Delta\theta$ 更新也收敛。[5]

38.5.1 局限

SIP 虽能找到精确得多的解，仍有几个限制。第一，需要近似尺度不变的物理求解器。低维 x 可用 Newton，高维则需其他求逆方法；有些方程可局部解析求逆，复杂问题可能需领域知识，或采用后续数值方法。第二，SIP 准确求逆物理部分，却仍用传统一阶优化器确定 $\Delta\theta$。这些方法在病态设置下表现差，网络输出尺度差别大时也会影响 SIP，所以网络求逆仍值得追求。第三，x 空间更精确不保证总损失 $L = \sum_i L_i$ 更小。SIP 不依赖损失值，对各样本等权；这有时有益，但若过小或过大曲率 $|\partial^2 L/\partial x^2|$ 扭曲样本重要性，就可能有问题。在这些情况或控制等不关心 x 精度的任务中，传统训练可能更好。

38.5.2 与监督训练的相似性

SIP 类似“监督训练”中的方法，但可靠更新在训练时即时计算。逆仿真器可用高阶方法给出所需逆，并避免对多模态解取平均（参见前面的预览例子）。预计算数据集不能计及多模态，因此输入到参考解映射不唯一时，作者称必然学到次优解。[6] 这也说明 DP 训练的一项困难：梯度没有恰当求逆，也难以用预处理可靠归一化，所以出现前章所述尺度问题，并在训练中导致消失或爆炸梯度。本章正针对这些问题。下一章展示逆仿真器 SIP 更新训练网络的更复杂例子，再介绍处理尺度问题的第二种方法。

[4] 译注：这是该实验对优化行为的解释，不把 Adam 二阶矩估计认定为一般 Hessian 恒等式，也不保证任意点的 Hessian 非奇异。

[5] 译注：指数级优势及收敛须依 [HKT22] 的假设、问题族和误差度量；本章有限图表本身不构成普遍复杂度或单步更新收敛定理。

[6] 译注：这里针对确定性网络配合逐点平方监督损失时的条件平均现象；多模态数据本身可被保存，条件生成模型或其他目标也不必输出模式平均，不能将该断言推广为所有预计算数据集或监督方法。

第三十九章　用尺度不变更新学习热传导求逆

现在用较复杂的实用例子考察尺度不变物理（SIP）更新。热方程有两个有趣挑战：可微物理（DP）梯度只会进一步扩散，而求逆在数值上困难。SIP 处理前者，专门的求解器处理后者。以下 notebook 用 phiflow 完整实现数据生成、DP 版本与改进 SIP 更新。源文另标“在 Colab 运行”，冻结 PDF 未给可恢复链接，故不猜测地址。

39.1　问题表述

考虑二维热方程 $\partial_t u = \nu \nabla^2 u$。从 t_0 的初态 $x = u(t_0)$ 出发，仿真器计算 $y = u(t^*) = \mathcal{P}(x)$。源文称只有 $t\nu = 0$ 才能精确求逆；$t\nu$ 增大时，原本不同的温度逐渐均匀，原信息淹没在噪声中，物理 Jacobian $\partial y / \partial x$ 接近奇异。[1] 采用周期边界，在频率空间解析计算

$$\hat{y} = \hat{x}\, e^{-k^2 (t^* - t_0)}, \qquad \hat{y}_k \equiv \mathcal{F}(y)_k.$$

正向仿真指数衰减高频，逆仿真将再次遇到这一点。逆问题可写为

$$L(x) = \| \mathcal{P}(x) - y^* \|_2^2 = \| \mathcal{F}^{-1}(\mathcal{F}(x) e^{-k^2 (t^* - t_0)}) - y^* \|_2^2.$$

[2]

39.2　实现

在 64×64 个单位边长单元的域上设置 $t\nu = 8$，大多数细节扩散后仅剩大尺度结构。默认用 phiflow 的 PyTorch 后端，也可按源改为 `phi.tf.flow` 使用 TensorFlow。

```
!pip install --upgrade --quiet phiflow==3.1
from phi.torch.flow import * # switch to TF with "phi.tf.flow"
```

原书输出：

```
?25l ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/182.2 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 182.2/182.2 kB 9.2 MB/s eta 0:00:00
?25h Preparing metadata (setup.py) ... ?25l?25hdone
```

[1] 译注：无噪声、有限频率离散系统在正时间仍可有代数逆，困难在逆算子的强烈病态及噪声放大；连续热半群也不能把“逆不稳定”直接等同“仅零时间才单射”。源句按数值动机保留。

[2] 译注：源频域显示式未显式写 ν 或频率定义的 2π 因子；应把它们理解为已吸收到时间/波数，不能与未缩放物理时间直接混用。代码明确采用扩散量 8 及 $-(2\pi)^2 k^2$。

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 306.1/306.1 kB 18.7 MB/s eta 0:00:00
?25h Preparing metadata (setup.py) ... ?25l?25hdone
Building wheel for phiflow (setup.py) ... ?25l?25hdone
Building wheel for phiml (setup.py) ... ?25l?25hdone
```

[3]

39.3 数据生成

随机放置 4 到 10 个、大小形状随机的“热”矩形生成 x^*。下列函数通过 `shape.batch` 生成整小批。它们是待网络重建的初态，不作为网络直接输入。[4]

```
def generate_heat_example(*shape, bounds: Box = None):
    shape = math.merge_shapes(*shape)
    heat_t0 = CenteredGrid(0, extrapolation.PERIODIC, bounds, resolution=shape.spatial)
    bounds = heat_t0.bounds
    component_counts = math.to_int32(4 + 7 * math.random_uniform(shape.batch))
    positions = (math.random_uniform(shape.batch, batch(components=10),
    ↪  channel(vector=shape.spatial.names)) - 0.5) * bounds.size * 0.8 + bounds.size *
    ↪  0.5
    for i in range(10):
        position = positions.components[i]
        half_size = math.random_uniform(shape.batch,
        ↪  channel(vector=shape.spatial.names)) * 10
        strength = math.random_uniform(shape.batch) * math.to_float(i <
        ↪  component_counts)
        position = math.clip(position, bounds.lower + half_size, bounds.upper -
        ↪  half_size)
        component_box = Cuboid(position, half_size)
        component_mask = SoftGeometryMask(component_box)
        component_mask = component_mask.at(heat_t0)
        heat_t0 += component_mask * strength
    return heat_t0
```

数据在训练时即时生成；先用 `vis.plot` 看两个初态例子。

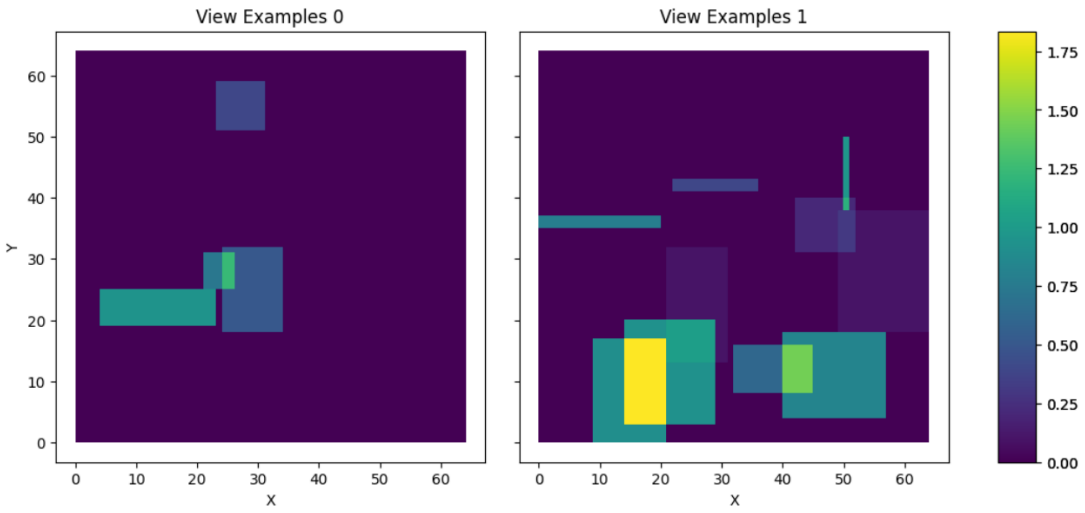
```
vis.plot(generate_heat_example(batch(view_examples=2), spatial(x=64, y=64)));
```

原书输出：

```
<ipython-input-2-350e9b9c59ed>:13: DeprecationWarning: HardGeometryMask and
↪  SoftGeometryMask are deprecated. Use field.mask or field.resample instead.
component_mask = SoftGeometryMask(component_box)
```

[3] 译注：本章安装、数据生成、训练和推断代码均只转写冻结源，未执行。所有图和日志均是源结果；安装进度中的控制字符可见文本按 PDF 保留。

[4] 译注：源称初态“从不传给求解器”，而下文损失函数实际将 `x_gt` 送入正向扩散以生成目标；不传给网络的未知重建目标，与生成观测时调用求解器应区分。



39.4 可微物理与梯度下降

这一设置可照常用 DP 训练。扩散例子的 GD 更新可解析写作

$$\Delta x_{\mathrm{GD}} = -\eta \mathcal{F}^{-1}\Big(e^{-k^2(t^*-t_0)}\mathcal{F}(y-y^*)\Big).$$

这意味着可微仿真器的梯度又把正向物理作用于梯度向量：前向已扩散，反向竟继续扩散，而非撤销它。这是 DP 固有且“正确”的行为，更新稳定，却缺少高频空间信息。因此拟合粗结构后，GD 收敛缓慢，难恢复高频细节；作者强调这不一定是信息根本缺失，而是 GD 处理细节不佳。[5] 实现前向用 `y = diffuse.fourier(x, 8., 1)`，比较两个 y 场的 L^2 损失，其反向也相应施加扩散。

39.5 稳定 SIP 梯度

更有趣的是逆仿真器给出的改进 SIP 步，仍记作 Δx_{PG}。热方程可在频域解析求逆：$\hat{x}_k = \hat{y}_k e^{k^2(t^*-t_0)}$，因此

$$\Delta x_{\mathrm{PG}} = -\eta \mathcal{F}^{-1}\Big(e^{k^2(t^*-t_0)}\mathcal{F}(y-y^*)\Big).$$

高频被指数大因子放大，直接使用会导致更新强烈振荡。实空间求梯度同样会有这种数值不稳，但频域更易量化。利用物理知识，可以从概率角度构造稳定逆：观测 y 含噪声 n，余下为信号 $s = y - n$。假设噪声 $n \sim \mathcal{N}(0, \epsilon y)$、$\epsilon > 0$；信号来自合理的 x，因而 $y \sim \mathcal{N}(0, \delta e^{-k^2})$、$\delta > 0$。用 Bayes 公式估计观测来自信号的概率：

$$p(s \mid v) = \frac{p(v \mid s)p(s)}{p(v \mid s)p(s) + p(v \mid n)p(n)}, \qquad p(s) = p(n) = \tfrac{1}{2}.$$

据此阻尼逆物理的放大，获得稳定逆。[6] 作者称这种梯度提取给定噪声下尽可能多的高频信息，相比通用优化器更快且更准。下列 `apply_damping` 以 `probability_signal` 决定阻尼，`inv_diffuse` 实现稳

[5] 译注：对有限带宽、适当信噪比的例子可作此解释；有噪声时无法恢复低于噪声的信息，不能普遍断言信息没有丢失。这里采用不含 1/2 的平方范数目标却省略梯度因子 2，可吸收到学习率；源式保留。

[6] 译注：源 $\mathcal{N}$ 的第二参数含 y，未说明方差/标准差及非负性，频域复幅度的概率模型也未完整定义。代码使用未含各尺度归一化因子的指数权重，故这里的 probability_signal 应按源构造的启发式权重理解，不能无附加假设认证为规范 Gaussian 似然的 Bayes 后验。

定逆扩散。[7]

```
def apply_damping(kernel, inv_kernel, amp, f_uncertainty, log_kernel):
    signal_prior = 0.5
    expected_amp = 1. * kernel.shape.get_size('x') * inv_kernel # This can be measured
    signal_likelihood = math.exp(-0.5 * (abs(amp) / expected_amp) ** 2) * signal_prior
    ↪  # this can be NaN
    signal_likelihood = math.where(math.isfinite(signal_likelihood), signal_likelihood,
    ↪  math.zeros_like(signal_likelihood))
    noise_likelihood = math.exp(-0.5 * (abs(amp) / f_uncertainty) ** 2) * (1 -
    ↪  signal_prior)
    probability_signal = math.divide_no_nan(signal_likelihood, (signal_likelihood +
    ↪  noise_likelihood))
    action = math.where((0.5 >= probability_signal) | (probability_signal >= 0.68), 2 *
    ↪  (probability_signal - 0.5), 0.) # 1 sigma required to take action
    prob_kernel = math.exp(log_kernel * action)
    return prob_kernel, probability_signal
def inv_diffuse(grid: Grid, amount: float, uncertainty: Grid):
    f_uncertainty: math.Tensor = math.sqrt(math.sum(uncertainty.values ** 2,
    ↪  dim='x,y')) # all frequencies have the same uncertainty, 1/N in iFFT
    k_squared: math.Tensor = math.sum(math.fftfreq(grid.shape, grid.dx) ** 2, 'vector')
    fft_laplace: math.Tensor = -(2 * np.pi) ** 2 * k_squared
    # --- Compute sharpening kernel with damping ---
    log_kernel = fft_laplace * -amount
    log_kernel_clamped = math.minimum(log_kernel,
    ↪  math.to_float(math.floor(math.log(math.wrap(np.finfo(np.float32).max)))) #
    ↪  avoid overflow
    raw_kernel = math.exp(log_kernel_clamped) # inverse diffusion FFT kernel, all
    ↪  values >= 1
    inv_kernel = math.exp(-log_kernel)
    amp = math.fft(grid.values)
    kernel, sig_prob = apply_damping(raw_kernel, inv_kernel, amp, f_uncertainty,
    ↪  log_kernel)
    # --- Apply and compute uncertainty ---
    data = math.real(math.ifft(amp * math.to_complex(kernel)))
    uncertainty = math.sqrt(math.sum(((f_uncertainty * kernel) ** 2))) /
    ↪  grid.shape.get_size('x') # 1/N normalization in iFFT
    uncertainty = grid * 0 + uncertainty
    return grid.with_values(data), uncertainty, abs(amp), raw_kernel, kernel, sig_prob
```

39.6 神经网络与损失函数

SIP 与普通 DP + Adam（沿前文简称 GD）均用简单 U-Net，小批 128、固定学习率 10^{-3}；物理用 64 位，网络用 32 位，权重更新由 TensorFlow/PyTorch 自动微分计算。

```
math.set_global_precision(64)
```

[7]译注：该优势限于所示物理先验和实验，未构成对任意通用优化器或所有噪声模型的最优性证明。

```
BATCH = batch(batch=128)
STEPS = 50
math.seed(0)
net = u_net(1, 1)
optimizer = adam(net, 0.001)
```

定义 phiflow 损失，权重梯度始终为 d(loss_function)/dθ。SIP 先求物理逆，以校正目标为常量定义代理平方损失；代码通过 `x = field.stop_gradient(prediction)` 实现。代理损失触发网络反传，`y_l2` 只供比较，不参与 SIP 训练。`sip=False` 则直接比较预测扩散场与目标，并通过这些运算反传。参考目标 $y = \mathcal{P}(x^*)$ 也在损失中即时生成。

```
# @math.jit_compile
def loss_function(net, x_gt: CenteredGrid, sip: bool):
    y_target = diffuse.fourier(x_gt, 8., 1)
    with math.precision(32):
        prediction = field.native_call(net, field.to_float(y_target)).vector[0]
        prediction += field.mean(x_gt) - field.mean(prediction)
    x = field.stop_gradient(prediction)
    if sip:
        y = diffuse.fourier(x, 8., 1)
        dx, _, amp, raw_kernel, kernel, sig_prob = inv_diffuse(y_target - y, 8.,
        ↪  uncertainty=abs(y_target - y) * 1e-6)
        correction = x + dx
        y_l2 = field.l2_loss(y - y_target) # not important, just for tracking
        loss = field.l2_loss(prediction - correction) # proxy L2 loss for network
    else:
        y = diffuse.fourier(prediction, 8., 1)
        loss = y_l2 = field.l2_loss(y - y_target) # for Adam we backprop through the
        ↪  loss and y
    return loss, x, y, field.l2_loss(x_gt - x), y_l2
```

39.7　训练

循环即时生成数据，用 `update_weights` 调用相应后端。为清楚仅打印前 5 步，全部历史存入 `loss_` 列表。先训练 SIP：

```
loss_sip_x=[]; loss_sip_y=[]
for training_step in range(STEPS):
    data = generate_heat_example(spatial(x=64, y=64), BATCH)
    loss_value, x_sip, y_sip, x_l2, y_l2 = update_weights(net, optimizer,
    ↪  loss_function, net, data, sip=True)
    loss_sip_x.append(float(x_l2.mean))
    loss_sip_y.append(float(y_l2.mean))
    if(training_step<5): print("SIP L2 loss x ; y: "+format(float(x_l2.mean))+"
    ↪  ;"+format(float(y_l2.mean)) )
```

原书输出：

```
<ipython-input-2-350e9b9c59ed>:13: DeprecationWarning: HardGeometryMask and
↪  SoftGeometryMask are deprecated. Use field.mask or field.resample instead.
component_mask = SoftGeometryMask(component_box)
SIP L2 loss x ; y: 185.01902418878524 ; 52.86203787432579
SIP L2 loss x ; y: 80.09229485521256 ; 13.743946340489366
SIP L2 loss x ; y: 51.198420978751095 ; 4.212126705173006
SIP L2 loss x ; y: 51.166816963598585 ; 3.871802844235444
SIP L2 loss x ; y: 41.97257479831386 ; 3.3333446973370604
```

再以 `sip=False` 关闭 SIP 更新，重复 DP + Adam 训练。

```
math.seed(0)
net_gd = u_net(1, 1)
optimizer_gd = adam(net_gd, 0.001)
loss_gd_x=[]; loss_gd_y=[]
for training_step in range(STEPS):
    data = generate_heat_example(spatial(x=64, y=64), BATCH)
    loss_value, x_gd, y_gd, x_l2, y_l2 = update_weights(net_gd, optimizer_gd,
    ↪  loss_function, net_gd, data, sip=False)
    loss_gd_x.append(float(x_l2.mean))
    loss_gd_y.append(float(y_l2.mean))
    if(training_step<5): print("GD L2 loss x ; y: "+format(float(x_l2.mean))+"
    ↪  ;"+format(float(y_l2.mean)) )
```

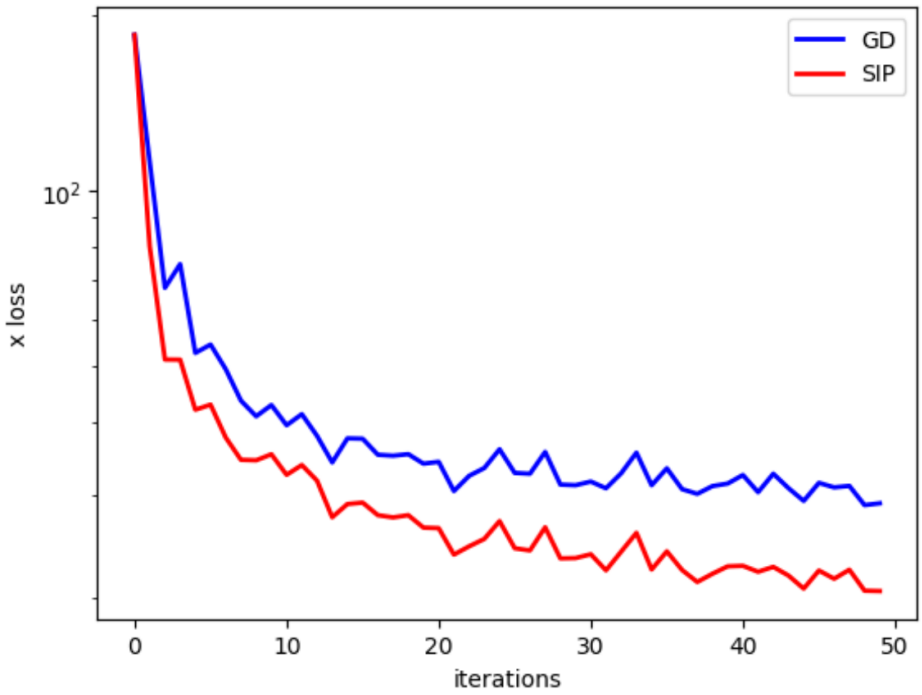
原书输出：

```
<ipython-input-2-350e9b9c59ed>:13: DeprecationWarning: HardGeometryMask and
↪  SoftGeometryMask are deprecated. Use field.mask or field.resample instead.
component_mask = SoftGeometryMask(component_box)
GD L2 loss x ; y: 185.01902418878524 ; 52.86203787432579
GD L2 loss x ; y: 111.90876514444714 ; 19.610744935770025
GD L2 loss x ; y: 67.98615290073411 ; 5.831835761821972
GD L2 loss x ; y: 74.69661928237372 ; 11.433268852185064
GD L2 loss x ; y: 52.55936273792499 ; 4.227996622501346
```

39.8 评估

随机数据即时生成，每次样本此前未见过，因此作者直接用训练曲线比较性能。下图是主要目标：重建输入 x 的 L^2 误差。

```
import pylab as plt
fig = plt.figure().gca()
pltx = range(len(loss_gd_x))
fig.plot(pltx, loss_gd_x , lw=2, color='blue', label="GD")
fig.plot(pltx, loss_sip_x , lw=2, color='red', label="SIP")
plt.xlabel('iterations'); plt.ylabel('x loss'); plt.legend(); plt.yscale("log")
```



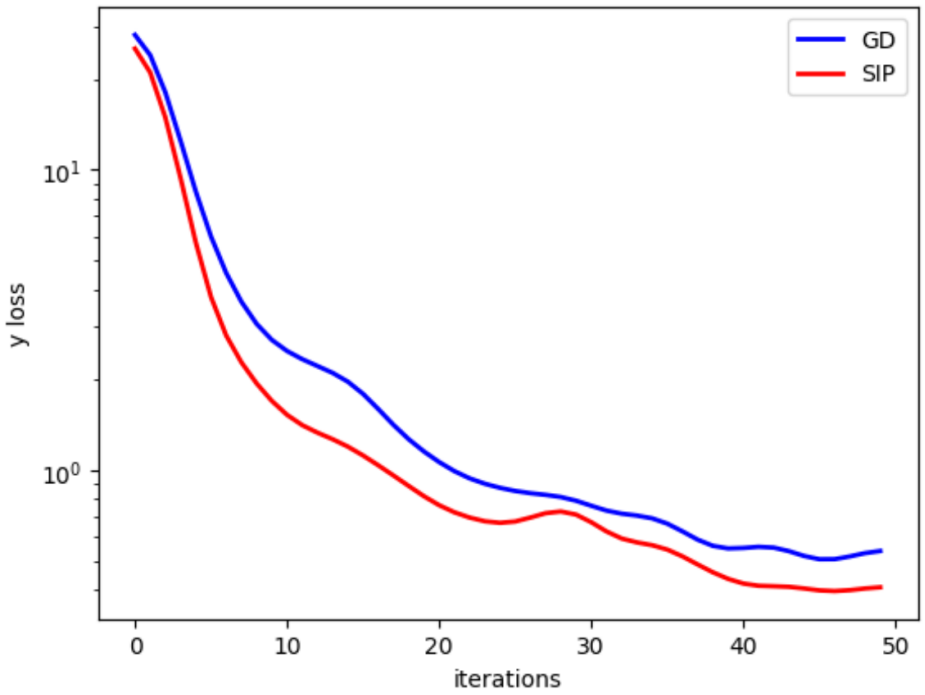
对数纵轴显示 SIP 的网络训练收敛显著改善，这来自代理损失提供的更好物理信号。两者仍用相同反传更新网络权重，差异来自逆仿真器计算的高阶物理步。[8] 也比较输出 y 空间误差：

```
fig = plt.figure().gca()
pltx = range(len(loss_gd_y))
import scipy # for filtering
fig.plot(pltx, scipy.ndimage.filters.gaussian_filter1d(loss_gd_y,sigma=2) , lw=2,
↪  color='blue', label="GD")
fig.plot(pltx, scipy.ndimage.filters.gaussian_filter1d(loss_sip_y ,sigma=2), lw=2,
  color='red', label="SIP")
plt.xlabel('iterations'); plt.ylabel('y loss'); plt.legend(); plt.yscale("log")
```

原书输出：

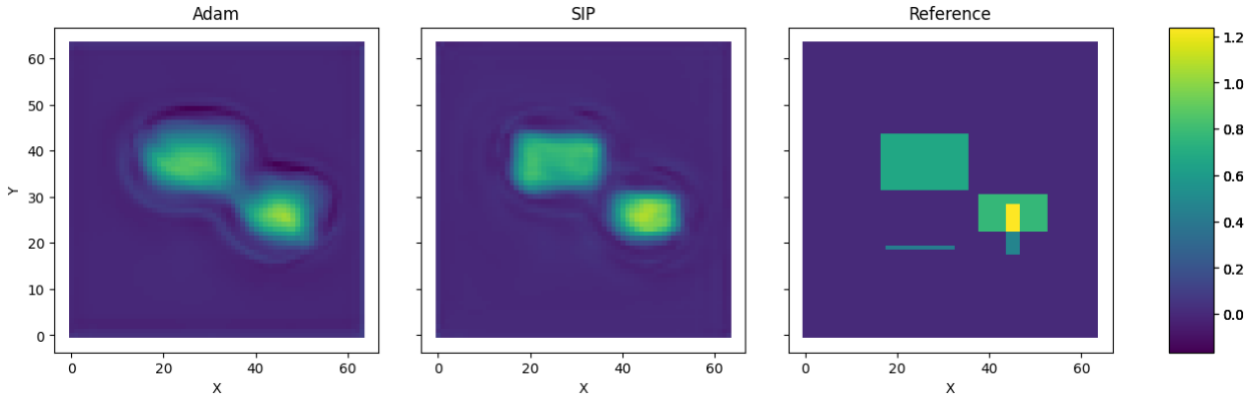
```
<ipython-input-10-63a3d36fc9b2>:4: DeprecationWarning: Please import
↪  `gaussian_filter1d` from the `scipy.ndimage` namespace; the `scipy.ndimage.filters`
↪  namespace is deprecated and will be removed in SciPy 2.0.0.
fig.plot(pltx, scipy.ndimage.filters.gaussian_filter1d(loss_gd_y,sigma=2) , lw=2,
↪  color='blue', label="GD")
<ipython-input-10-63a3d36fc9b2>:5: DeprecationWarning: Please import
↪  `gaussian_filter1d` from the `scipy.ndimage` namespace; the `scipy.ndimage.filters`
↪  namespace is deprecated and will be removed in SciPy 2.0.0.
fig.plot(pltx, scipy.ndimage.filters.gaussian_filter1d(loss_sip_y ,sigma=2), lw=2,
↪  color='red', label="SIP")
```

[8] 译注：图中是各次更新前、对新小批评价的损失，不是训练后固定测试集误差；随机小批与即时目标也应纳入比较口径。



SIP 同样改善，但不如 x 空间显著；本逆问题更关心初态重建。直接比较也很明显：下列单元绘出 GD/Adam、SIP 与右侧真值。SIP 更清晰、光晕更少，体现 GD 将反传梯度继续扩散而非对抗扩散的不足。

```
pltv = vis.plot(x_gd.values.batch[0], x_sip.values.batch[0], data.values.batch[0],
↪  size=(15,4) )
pltv.axes[0].set_title("Adam"); pltv.axes[1].set_title("SIP");
↪  pltv.axes[2].set_title("Reference");
```



9

[9]译注：图中 `x_gd/x_sip` 是最后一次训练调用返回的预测，通常在该次权重更新之前计算；不是显式训练完成后重新推断。两循环均重设 seed，但网络初始化与随机数消费能否保证完全相同小批，还需后端实现约定，未在此运行核查。

39.9　下一步

- 尝试不同训练参数：延长训练，改变学习率、网络大小，甚至网络架构。

第四十章　半逆梯度

此前尺度不变物理（SIP）更新说明，除利用高阶项外，对更新方向求逆也很重要。现在介绍另一种求逆方法：半逆梯度（HIG）[SHT22]。它各有利弊，为物理深度学习的改进更新提供另一选择。HIG 与 SIP 不同，不要求解析逆求解器，而是联合求逆网络与物理模型；代价是对大型 Jacobian 作奇异值分解（SVD）。

预览：HIG、SIP 与 Adam

- HIG 不需要解析逆求解器，且联合求逆网络与物理模型；
- 代价是大型 Jacobian 的 SVD，且只基于一阶信息，类似普通梯度；
- 但它使用完整 Jacobian，所以下文例子中通常显著优于 GD 与 Adam。

40.1　推导

式 (36.3) 的普通 Newton 更新用逆 Hessian。对 L^2 损失重写，可得到 Gauss–Newton（GN）方法：

$$\Delta\theta_{\mathrm{GN}} = -\eta \left[\left(\frac{\partial y}{\partial \theta} \right)^T \frac{\partial y}{\partial \theta} \right]^{-1} \left(\frac{\partial y}{\partial \theta} \right)^T \left(\frac{\partial L}{\partial y} \right)^T . \tag{40.1}$$

源文称满秩 Jacobian 可消去转置，简化为

$$\Delta\theta_{\mathrm{GN}} = -\eta \left(\frac{\partial y}{\partial \theta} \right)^{-1} \left(\frac{\partial L}{\partial y} \right)^T . \tag{40.2}$$

[1] 形式虽简单，仍需对通常非方、带小奇异值的 Jacobian 求逆。朴素 GN 可能快速爆炸。但训练循环中含物理求解器时，小奇异值往往仍承载重要物理信息，不希望简单丢弃，而想尽量保留学习信号。这引出部分、截断求逆的 HIG 更新：

$$\Delta\theta_{\mathrm{HIG}} = -\eta \left(\frac{\partial y}{\partial \theta} \right)^{-1/2} \left(\frac{\partial L}{\partial y} \right)^T . \tag{40.3}$$

这里 $-1/2$ 的平方根用 SVD 定义半逆：若 $A = U\Lambda V^T$，则 $A^{-1/2} = V\Lambda^{-1/2}U^T$。$\Lambda$ 含奇异值；小于阈值 τ 的条目置零，以处理数值噪声。[2]

[1] 译注：平方损失的 Gauss–Newton 忽略残差加权的模型二阶导项，不是完整 Newton 的恒等改写。若 $J = \partial y/\partial\theta$ 为方阵且非奇异，式 (40.2) 是普通逆；矩形满列秩时 $(J^TJ)^{-1}J^T$ 是左伪逆，不能写成一般双侧逆。网络参数常多于输出，J^TJ 还可能奇异，须明确伪逆/正则化。源式保留。

[2] 译注：截断应理解为谱滤波 $h(\sigma) = \sigma^{-1/2}$（$\sigma \geq \tau$）或 0（$\sigma < \tau$），不能先把奇异值设 0 再直接计算其负幂。此 SVD “半逆” 不是通常方阵幂的平方根逆，矩形时维数也互换。

截断与钳位：看似可将小奇异值钳至 τ 而不舍弃，但这些奇异值对应的方向可能不可靠；小 τ 又在求逆中产生大贡献，所以作者建议直接丢弃对应内容。

用 $-1/2$ 部分求逆而非 -1，可减轻小特征值造成的过大更新。源文以 Adam 为启发：它用 $J/\sqrt{\mathrm{diag}(J^TJ)}$ 而非 $J/(J^TJ)$ 归一化方向，并称 J 为 Jacobian 对角线。Adam 因粗略对角近似而需要折中；HIG 使用完整 Jacobian，可作恰当求逆，但 [SHT22] 指出半逆仍能正则化、改善学习并减小梯度爆炸风险。[3]

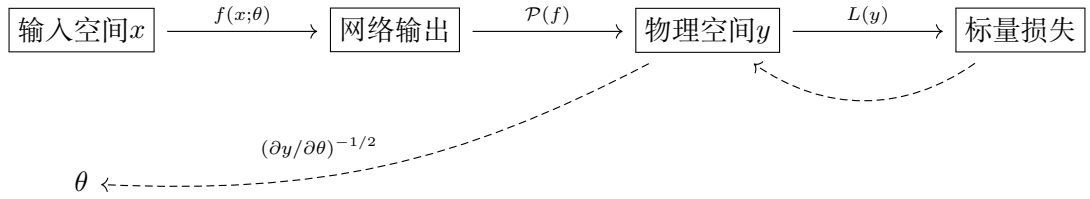


图 40.1: HIG 训练的空间关系：联合利用网络与物理的半逆 Jacobian。原图 40.1 的节点、前向链、损失反传与联合半逆关系原生重建。

40.2 构造 Jacobian

上述形式还隐藏一项重要性质：搜索方向不仅联合考虑网络与物理尺度，还可利用小批所有样本的信息。作者称这有利于寻找 L^2 意义下最小化损失的最优方向，而非像 GD/Adam 一样平均各方向。[4] 把小批各样本的 Jacobian 纵向拼接。令 x_i, y_i 为第 i 个样本输入输出，定义

$$\frac{\partial y}{\partial \theta} := \begin{pmatrix} \left.\dfrac{\partial y_1}{\partial \theta}\right|_{x_1} \\ \left.\dfrac{\partial y_2}{\partial \theta}\right|_{x_2} \\ \vdots \\ \left.\dfrac{\partial y_b}{\partial \theta}\right|_{x_b} \end{pmatrix}.$$

$|_{x_i}$ 表明各块在相应输入状态评价。普通优化因平均效应通常难从大批获益，HIG 对批大小依赖更强，常能从较大批获益。因此，计算 HIG 需逐样本评价 Jacobian，对拼接矩阵作 SVD，截断并半逆奇异值，再重组半逆 Jacobian 以算更新方向。

40.3 玩具例子展示性质

考虑二维学习目标

$$\hat{y}(x) = (\sin(6x), \cos(9x)), \qquad x \in [-1, 1],$$

和缩放损失

$$L(y, \hat{y}; \lambda) = \tfrac{1}{2}(y_1 - \hat{y}_1)^2 + \tfrac{1}{2}(\lambda y_2 - \hat{y}_2)^2.$$

[3]译注：源本段在奇异值/特征值及 J 的矩阵/对角向量记号间切换；实际 Adam 用梯度一、二阶矩估计，不能把该启发式比值当作一般 Adam 或 Hessian 恒等式。半逆减弱放大，不自动消除小奇异值风险，仍需阈值和步长。

[4]译注：严格的局部最小二乘解对应满足条件的 GN 伪逆步；截断半逆是正则化谱更新，一般不等同无正则最小二乘的精确最优方向。这里保留方法动机。

这里下标 1、2 表示输出分量，不是前述小批样本。故意只在第二分量的损失中施加 λ，模拟物理仿真常见的不均尺度，并以 λ 控制程度。使用仅一个 7 神经元隐藏层、tanh 激活的小网络学习 $\hat{y}$。[5]

40.4 良态情形

先取 $\lambda = 1$，比较常用 GD 代表 Adam、经典 GN 与 HIG。评价三方面：损失变化、网络状态的神经元激活分布，以及优化怎样影响网络产生的目标状态 y。下图 y 空间只展示一个有代表性的输入输出对，另两图用较大验证集的统计量。三法表现均尚可，损失下降至 10^{-2} 到 10^{-3}。激活的均值和标准差

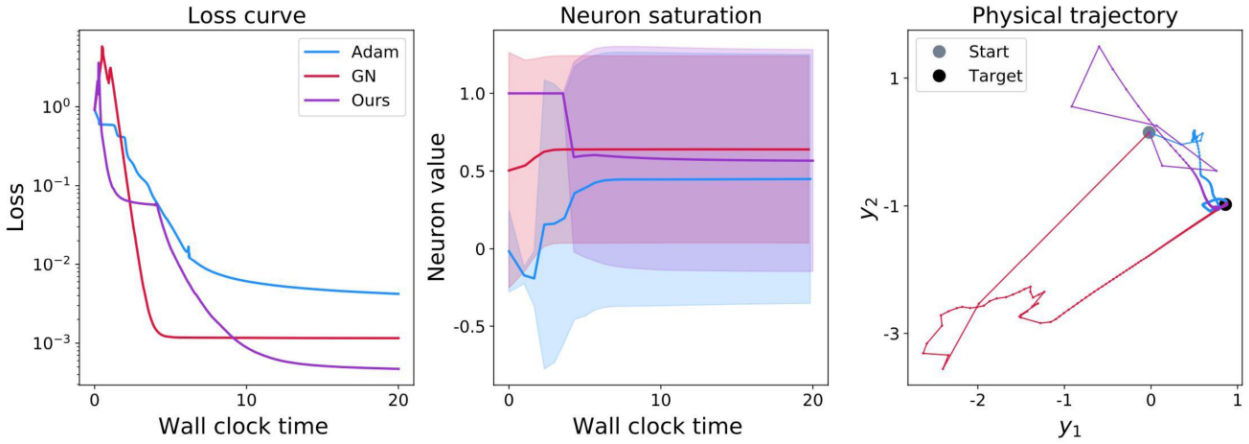


图 40.2: 良态玩具问题，比较 Adam、GN 与 HIG。原图 40.2。

（实色阴影）覆盖较宽数值范围，说明不同输入产生不同值，是好信号。若不同输入在目标明显不同时仍产生常值，就很不理想；这往往源于过大更新导致神经元完全饱和。但良态例子中未出现此情况。右图单个输入输出对的初始状态为浅灰点，真值目标为黑点，三法最终均到达黑点。虽是简单例子，GN 和 HIG 初期较大跳跃仍值得注意：较差初态与求逆使网络及输出大变。Adam 的动量则减轻跳跃，浅蓝线初段小于另两法。总体符合预期：损失仍可再降，y 中个别步似乎暂朝错误方向，但三法均应付良好。较小 λ 使 Jacobian 更病态时，情况将改变。

40.5 病态情形

令 $\lambda = .01$。真实 PDE 的条件可能更差，但此处 1/100 已足以展示实际问题。Adam 和 GN 的损失均难低于约.2，而此前能达到.01 或更低。Adam 难以处理 y_2 的不良尺度，无法正常收敛；GN 完全求逆则引起梯度爆炸，抵消求逆益处，甚至使网络停滞。中图更明显：GN 红线迅速饱和到 1 且无可见方差，作者将其解释为神经元全部饱和，不再产生有意义信号。目标拟合差，未来梯度也接近零，这些神经元于是无法继续学习，必须避免。这并非 GN 总会发生，但批中某些样本使 Jacobian 向量线性相关或近相关时常发生，故作者认为 GN 次优。[6] 右图也显示 Adam、GN 未到达黑点，尤其难沿故意缩

[5]译注：按显示损失的字面定义，第二分量的零损失目标为 $y_2 = \hat{y}_2/\lambda$，不等于未缩放的 $\hat{y}_2$（除 $\lambda = 1$）。因此须区分原始网络输出与在损失/图中比较的缩放物理输出；源式和“学习 $\hat{y}$”措辞照录。

[6]译注：这是所示有限精度训练的观察和解释，并非任意饱和网络永远不能恢复或所有 GN 必失败的定理；阻尼、信赖域、正则化与步长均可改变行为。

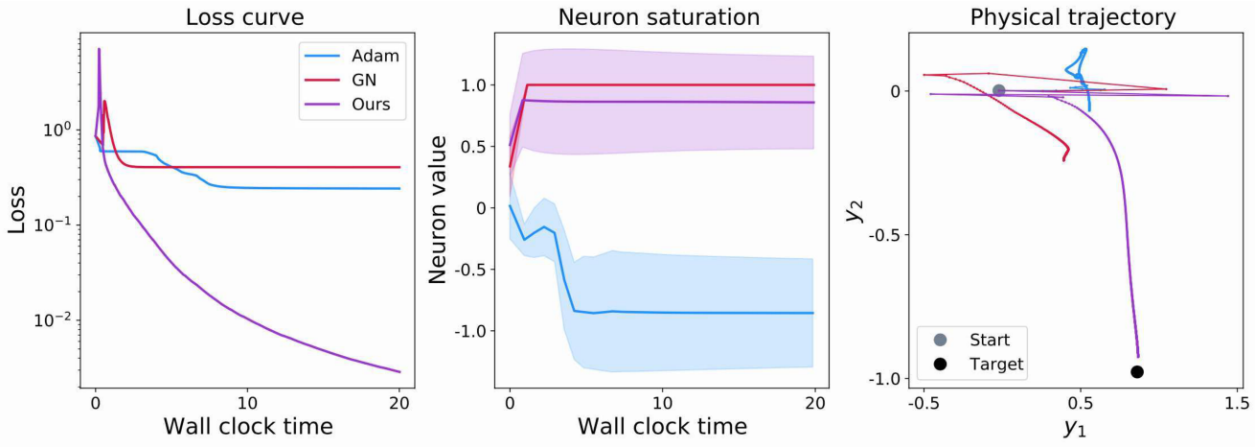


图 40.3: 病态玩具问题，比较 Adam、GN 与 HIG。原图 40.3。

放的 y_2 前进；两者沿横轴 y_1 有进展，却很少接近 y_2 目标。GN 因激活饱和停滞，Adam 难撤销 y_2 缩放，符合前述讨论。

40.6 半逆梯度小结

此前所有例子都在改进 DP 训练，即网络与 PDE 求解算子的组合；普通 GD 与 HIG 均要求物理算子可微。SIP 则还要求完整逆求解器，虽有优点，也与 DP/HIG 不同。因此 HIG 更像“用神经算子减少数值误差”和“用神经网络求逆问题”的设置，而不是上一章热传导逆问题。接下来给出用 HIG 训练物理网络的具体代码例子：经典耦合振子系统。

第四十一章　使用半逆梯度的耦合振子

本 notebook 用实际例子，比较以物理损失训练神经网络的三种方法：

1. Adam：标准梯度下降（GD）类网络优化器；
2. 尺度不变物理（SIP）：此前完整求逆物理的算法；
3. 半逆梯度（HIG）：局部联合求逆物理与网络。

41.1　逆问题设置

学习任务是寻找控制函数，操纵耦合振子的动力学。这是经典物理问题，规模较小，适合评价 HIG。用两个质点，其位置与速度共 4 个自由度；相比之下，仅 32×32 单元的小流体仿真也有 $32\times32\times2$ 个未知量。振子仍非平凡：希望控制使系统在指定时间后回到初态。用 24 步四阶 Runge–Kutta（RK4），网络需学习各时间步怎样“轻推”质点，使它们在正确时刻以正确位置、速度返回。源给出 N 个耦合振子的 Hamiltonian：

$$\mathcal{H}(x_i,p_i,t)=\sum_i\left(\frac{x_i^2}{2}+\frac{p_i^2}{2}+\alpha(x_i-x_{i+1})^4+u(t)x_ic_i\right),$$

作为下述 RK4 积分的基础。[1]

41.2　问题表述

给定不同物理输入 x_i，控制 u_i 影响物理系统的时间演化，产生输出

$$y_i=\mathcal{P}(x_i;u_i).$$

给定初态 x_i 和目标态 y_i^*，求控制的逆问题，以损失 L 衡量目标与所得状态差别：

$$\arg\min_{u_i} L(y_i^*,\mathcal{P}(x_i;u_i)).$$

若以权重 θ 的网络 f 在输入输出对 (x_i,y_i^*) 上学习控制，便转为学习问题：

$$\arg\min_{\theta}\sum_i L\big(y_i^*,\mathcal{P}(x_i;f(x_i,y_i;\theta))\big).$$

[1]译注：该显示式含四次耦合势，产生三次力；下文代码实际采用线性 Laplacian 耦合，二者不是同一系统。该 Hamiltonian 的控制势给 $\dot p_i=-uc_i$，代码却加正的控制力，也有符号差别。源式与代码均保留，不能将此代码称为该四次 Hamiltonian 的直接离散实现。

[2] 设置求解器前，导入库（本例 TensorFlow），以全局 MODE 切换 Adam（GD）、SIP 与 HIG。

```
import numpy as np
import tensorflow as tf
import time, os
# main switch for the three methods:
MODE = 'HIG' # HIG | SIP | GD
```

[3]

41.3 耦合线性振子仿真

求解带控制项的耦合线性振子 ODE，用 RK4 积分。全局量 Nx 为振子数，Nt 为时间步数，DT 为步长。先定义 Laplacian 模板，再给整小批的 ODE 与按给定控制信号执行所有时间步的求解器。

```
Nx = 2
Nt = 24
DT = 0.5
def build_laplace(n,boundary='0'):
    if n==1:
        return np.zeros((1,1),dtype=np.float32)
    d1 = -2 * np.ones((n,),dtype=np.float32)
    d2 = 1 * np.ones((n-1,),dtype=np.float32)
    lap = np.zeros((n,n),dtype=np.float32)
    lap[range(n),range(n)]=d1
    lap[range(1,n),range(n-1)]=d2
    lap[range(n-1),range(1,n)]=d2
    if boundary=='0':
        lap[0,0]=lap[n-1,n-1]=-1
    return lap
@tf.function
def coupled_oscillators_batch( x, control):
    '''
    ODE of type: x' = f(x)
    :param x_in: position and velocities, shape: (batch, 2 * number of osc) order
    ↪  second index: x_i , v_i
    :param control: control function, shape: (batch,)
    :return:
    '''
    #print('coupled_oscillators_batch')
    n_osc = x.shape[1]//2
    # natural time evo
    a1 = np.array([[0,1],[-1,0]],dtype=np.float32)
```

[2]译注：源此式网络参数写未加星的 y_i，与前述目标对 (x_i, y_i^*) 冲突，照录。本例实现只把 `y_batch` 输入网络，且训练特设 $x_i = y_i^*$，不实现任意初态–目标对联合条件化的通用控制器。

[3]译注：本章代码/输出均按冻结 PDF 转写和静态核对；未训练、未运行旧 TensorFlow 私有 API、未写示例路径。科学图为原书结果，非本地复现。

```
    a2 = np.eye(n_osc,dtype=np.float32)
    A = np.kron(a1,a2)
    x_dot1 = tf.tensordot(x,A,axes = (1,1))
    # interaction term
    interaction_strength = 0.2
    b1 = np.array([[0,0],[1,0]],dtype=np.float32)
    b2 = build_laplace(n_osc)
    B = interaction_strength * np.kron(b1,b2)
    x_dot2 = tf.tensordot(x,B, axes=(1, 1))
    # control term
    control_vector = np.zeros((n_osc,),dtype=np.float32)
    control_vector[-1] = 1.0
    c1 = np.array([0,1],dtype=np.float32)
    c2 = control_vector
    C = np.kron(c1,c2)
    x_dot3 = tf.tensordot(control,C, axes=0)
    #all terms
    x_dot = x_dot1 + x_dot2 +x_dot3
    return x_dot
@tf.function
def runge_kutta_4_batch(x_0, dt, control, ODE_f_batch):
    f_0_0 = ODE_f_batch(x_0, control)
    x_14 = x_0 + 0.5 * dt * f_0_0
    f_12_14 = ODE_f_batch(x_14, control)
    x_12 = x_0 + 0.5 * dt * f_12_14
    f_12_12 = ODE_f_batch(x_12, control)
    x_34 = x_0 + dt * f_12_12
    terms = f_0_0 + 2 * f_12_14 + 2 * f_12_12 + ODE_f_batch(x_34, control)
    x1 = x_0 + dt * terms / 6
    return x1
@tf.function
def solver(x0, control):
    x = x0
    for i in range(Nt):
        x = runge_kutta_4_batch(x, DT, control[:,i], coupled_oscillators_batch)
    return x
```

41.4 训练设置

网络很简单：4 个全连接层，中间 3 层各 20 个神经元、tanh 激活，末层给 Nt 个控制量。

```
act = tf.keras.activations.tanh
model = tf.keras.models.Sequential([
    tf.keras.layers.InputLayer(input_shape=(2*Nx)),
    tf.keras.layers.Dense(20, activation=act),
    tf.keras.layers.Dense(20, activation=act),
    tf.keras.layers.Dense(20, activation=act),
```

```
    tf.keras.layers.Dense(Nt, activation='linear')
    ])
```

[4] 采用 L^2 损失：

```
@tf.function
def loss_function(a,b):
    diff = a-b
    loss_batch = tf.reduce_sum(diff**2,axis=1)
    loss = tf.reduce_sum(loss_batch)
    return loss
```

训练集生成 4096 个随机起始状态，要求仿真末端回到初态，故 X_TRAIN=Y_TRAIN。

```
N = 2**12
X_TRAIN = np.random.rand(N, 2 * Nx).astype(np.float32)
Y_TRAIN = X_TRAIN # the target states are identical
```

[5]

41.5 训练

先设置按方法启发式调好的全局参数；若所有方法强用同样设置，会使部分比较不公平。

1. Adam 为 GD 家族代表，截断参数对它无意义。
2. SIP 中所选优化器负责网络；物理逆用 GN，作者称因物理优化景观为二次而为精确求逆，GN 的 Jacobian 逆还可指定截断参数。
3. HIG 需将优化器设为 SGD，可指定半逆截断；本例最佳批往往比另两法小，学习率 1 通常很好。

MAX_TIME 指定训练秒数。

```
if MODE=='HIG': # HIG training
  OPTIMIZER = tf.keras.optimizers.SGD
  BATCH_SIZE = 32 # larger batches make HIGs unnecessariy slow...
  LR = 1.0
  TRUNC = 10**-10
elif MODE=='SIP': # SIP training
  OPTIMIZER = tf.keras.optimizers.Adam
  BATCH_SIZE = 256
  LR = 0.001 # for the internal step with Adam
  TRUNC = 0 # not used
else: # Adam Training (as GD representative)
  MODE = 'GD'
  OPTIMIZER = tf.keras.optimizers.Adam
```

[4]译注：源 `input_shape=(2*Nx)` 的括号不创建元组，实际为整数；不同 Keras 版本可能要求显式一维 shape 元组。这里保留源，不据此保证当前版本可直接执行。

[5]译注：数组有 $2N_x$ 个分量，包含位置和速度，均采样于 $[0,1)$，不只是位置。源未固定随机 seed，也未在此划分独立测试集。

```
  BATCH_SIZE = 256
  LR = 0.001
  TRUNC = 0 # not used
# global parameters for all three methods
MAX_TIME = 100 # [s]
print("Running variant: "+MODE)
```

原书输出：

```
Running variant: HIG
```

[6] 下面关键函数 HIG_pinv 对给定矩阵作 SVD，对奇异值取平方根，再重组半逆。函数由 TensorFlow 的 SVD 相关实现改编，原代码保留 Apache-2.0 许可说明及链接。

```
from tensorflow.python.framework import ops
from tensorflow.python.framework import tensor_shape
from tensorflow.python.ops import array_ops
from tensorflow.python.ops import math_ops
from tensorflow.python.util import dispatch
from tensorflow.python.util.tf_export import tf_export
from tensorflow.python.ops.linalg.linalg_impl import _maybe_validate_matrix,svd
# partial inversion of the Jacobian via SVD:
# this function is adopted from tensorflow's SVD function, and published here under its
↪  license: Apache-2.0 License ,
↪  https://github.com/tensorflow/tensorflow/blob/master/LICENSE
@tf_export('linalg.HIG_pinv')
@dispatch.add_dispatch_support
def HIG_pinv(a, rcond=None,beta=0.5, validate_args=False, name=None):
  with ops.name_scope(name or 'pinv'):
    a = ops.convert_to_tensor(a, name='a')
    assertions = _maybe_validate_matrix(a, validate_args)
    if assertions:
      with ops.control_dependencies(assertions):
        a = array_ops.identity(a)
    dtype = a.dtype.as_numpy_dtype
    if rcond is None:
      def get_dim_size(dim):
        dim_val = tensor_shape.dimension_value(a.shape[dim])
        if dim_val is not None:
          return dim_val
        return array_ops.shape(a)[dim]
      num_rows = get_dim_size(-2)
      num_cols = get_dim_size(-1)
      if isinstance(num_rows, int) and isinstance(num_cols, int):
        max_rows_cols = float(max(num_rows, num_cols))
      else:
```

[6]译注：本实现 SIP 分支实际把 `rcond` 固定为 10^{-5}，未使用全局 TRUNC；HIG 使用 `self.trunc`。SIP 输出梯度含 $2/B$，随后直接作伪逆乘积，并未除去平方损失 Hessian/批归一化；故其更新一般不是单步达到零残差的完整 GN 校正，即使控制到末态的物理映射为仿射。

```
        max_rows_cols = math_ops.cast(
            math_ops.maximum(num_rows, num_cols), dtype)
      rcond = 10. * max_rows_cols * np.finfo(dtype).eps
    rcond = ops.convert_to_tensor(rcond, dtype=dtype, name='rcond')
    [ singular_values, left_singular_vectors, right_singular_vectors, ] = svd(
        a, full_matrices=False, compute_uv=True)
    cutoff = rcond * math_ops.reduce_max(singular_values, axis=-1)
    singular_values = array_ops.where_v2(
        singular_values > array_ops.expand_dims_v2(cutoff, -1), singular_values**beta,
        np.array(np.inf, dtype))
    a_pinv = math_ops.matmul(
        right_singular_vectors / array_ops.expand_dims_v2(singular_values, -2),
        left_singular_vectors,
        adjoint_b=True)
    if a.shape is not None and a.shape.rank is not None:
      a_pinv.set_shape(a.shape[:-2].concatenate([a.shape[-1], a.shape[-2]]))
    return a_pinv
```

现在用 Python 类组织训练，接收求解器、网络、损失和数据，在 MAX_TIME 内尽量多跑 epoch。各小批更新：

1. Adam：求损失梯度，应用 Adam；
2. PG（此处即 SIP）：求损失梯度与物理 Jacobian，逐数据点求逆，通过代理损失与 Adam 更新网络；
3. HIG：求损失梯度及网络–物理 Jacobian，联合半逆，用所得步更新网络。

`mini_batch_update` 实现三分支。

```
class Optimization():
    def __init__(self,model,solver,loss_function,x_train,y_train):
        self.model = model
        self.solver = solver
        self.loss_function = loss_function
        self.x_train = x_train
        self.y_train = y_train
        self.y_dim = y_train.shape[1]
        self.weight_shapes = [weight_tensor.shape for weight_tensor in
        ↪  self.model.trainable_weights]
    def set_params(self,batch_size,learning_rate,optimizer,max_time,mode,trunc):
        self.number_of_batches = N // batch_size
        self.max_time = max_time
        self.batch_size = batch_size
        self.learning_rate = learning_rate
        self.optimizer = optimizer(learning_rate)
        self.mode = mode
        self.trunc = trunc
```

```
    def computation(self,x_batch, y_batch):
        control_batch = self.model(y_batch)
        y_prediction_batch = self.solver(x_batch,control_batch)
        loss = self.loss_function(y_batch,y_prediction_batch)
        return loss
    @tf.function
    def gd_get_derivatives(self,x_batch, y_batch):
        with tf.GradientTape(persistent=True) as tape:
            tape.watch(self.model.trainable_variables)
            loss = self.computation(x_batch,y_batch)
            loss_per_dp = loss / self.batch_size
        grad = tape.gradient(loss_per_dp, self.model.trainable_variables)
        return grad
    @tf.function
    def pg_get_physics_derivatives(self,x_batch, y_batch): # physics gradient for SIP
        with tf.GradientTape(persistent=True) as tape:
            control_batch = self.model(y_batch)
            tape.watch(control_batch)
            y_prediction_batch = self.solver(x_batch,control_batch)
            loss = self.loss_function(y_batch,y_prediction_batch)
            loss_per_dp = loss / self.batch_size
        jacy = tape.batch_jacobian(y_prediction_batch,control_batch)
        grad = tape.gradient(loss_per_dp, y_prediction_batch)
        return jacy,grad,control_batch
    @tf.function
    def pg_get_network_derivatives(self,x_batch, y_batch,new_control_batch):#physical
    ↪  grads
        with tf.GradientTape(persistent=True) as tape:
            tape.watch(self.model.trainable_variables)
            control_batch = self.model(y_batch)
            loss = self.loss_function(new_control_batch,control_batch)
            #y_prediction_batch = self.solver(x_batch,control_batch)
            #loss = self.loss_function(y_batch,y_prediction_batch)
            loss_per_dp = loss / self.batch_size
        network_grad = tape.gradient(loss_per_dp, self.model.trainable_variables)
        return network_grad
    @tf.function
    def hig_get_derivatives(self,x_batch,y_batch):
        with tf.GradientTape(persistent=True) as tape:
            tape.watch(self.model.trainable_variables)
            control_batch = self.model(y_batch)
            y_prediction_batch = self.solver(x_batch,control_batch)
            loss = self.loss_function(y_batch,y_prediction_batch)
            loss_per_dp = loss / self.batch_size
        jacy = tape.jacobian(y_prediction_batch, self.model.trainable_variables,
  experimental_use_pfor=True)
        loss_grad = tape.gradient(loss_per_dp, y_prediction_batch)
```

```
        return jacy, loss_grad
    def mini_batch_update(self,x_batch, y_batch):
        if self.mode=="GD":
            grad = self.gd_get_derivatives(x_batch, y_batch)
            self.optimizer.apply_gradients(zip(grad, self.model.trainable_weights))
        elif self.mode=="SIP":
            jacy,grad,control_batch = self.pg_get_physics_derivatives(x_batch, y_batch)
            grad_e = tf.expand_dims(grad,-1)
            pinv = tf.linalg.pinv(jacy,rcond=10**-5)
            delta_control_label_batch = (pinv@grad_e)[:,:,0]
            new_control_batch = control_batch - delta_control_label_batch
            network_grad = self.pg_get_network_derivatives(x_batch,
            ↪  y_batch,new_control_batch)
            self.optimizer.apply_gradients(zip(network_grad,
            ↪  self.model.trainable_weights))
        elif self.mode =='HIG':
            jacy, grad = self.hig_get_derivatives(x_batch, y_batch)
            flat_jacy_list = [tf.reshape(jac, (self.batch_size * self.y_dim, -1)) for
            ↪  jac in jacy]
            flat_jacy = tf.concat(flat_jacy_list, axis=1)
            flat_grad = tf.reshape(grad, (-1,))
            inv = HIG_pinv(flat_jacy, rcond=self.trunc)
            processed_derivatives = tf.tensordot(inv, flat_grad, axes=(1, 0))
            #processed_derivatives = self.linear_solve(flat_jacy, flat_grad)
            update_list = []
            l1 = 0
            for k, shape in enumerate(self.weight_shapes):
                l2 = l1 + np.prod(shape)
                upd = processed_derivatives[l1:l2]
                upd = np.reshape(upd, shape)
                update_list.append(upd)
                l1 = l2
            self.optimizer.apply_gradients(zip(update_list,
            ↪  self.model.trainable_weights))
    def epoch_update(self):
        for batch_index in range(self.number_of_batches):
            position = batch_index * self.batch_size
            x_batch = self.x_train[position:position + self.batch_size]
            y_batch = self.y_train[position:position + self.batch_size]
            self.mini_batch_update(x_batch, y_batch)
    def eval(self,epoch,wc_time,ep_dur):
        train_loss = self.computation(self.x_train,self.y_train)
        train_loss_per_dp = train_loss / N
        if epoch<5 or epoch%20==0: print('Epoch: ', epoch,', wall clock time:
        ↪  ',wc_time,', loss: ', float(train_loss_per_dp) )
        #print('TrainLoss:', train_loss_per_dp)
        #print('Epoch: ', epoch,' WallClockTime: ',wc_time,' EpochDuration: ',ep_dur )
```

```
        return train_loss_per_dp
    def start_training(self):
        init_loss = self.eval(0,0,0)
        init_time = time.time()
        time_list = [init_time]
        loss_list = [init_loss]
        epoch=0
        wc_time = 0
        while wc_time<self.max_time:
            duration = time.time()
            self.epoch_update()
            duration = time.time()-duration
            epoch += 1
            wc_time += duration
            loss = self.eval(epoch,wc_time,duration)
            time_list.append(duration)
            loss_list.append(loss)
        time_list = np.array(time_list)
        loss_list = np.array(loss_list)
        time_list[0] = 0
        return time_list, loss_list
```

用全局参数启动训练，收集 time_list 与 loss_list。

```
opt = Optimization(model, solver, loss_function, X_TRAIN, Y_TRAIN)
opt.set_params(BATCH_SIZE, LR, OPTIMIZER, MAX_TIME, MODE, TRUNC)
time_list, loss_list = opt.start_training()
```

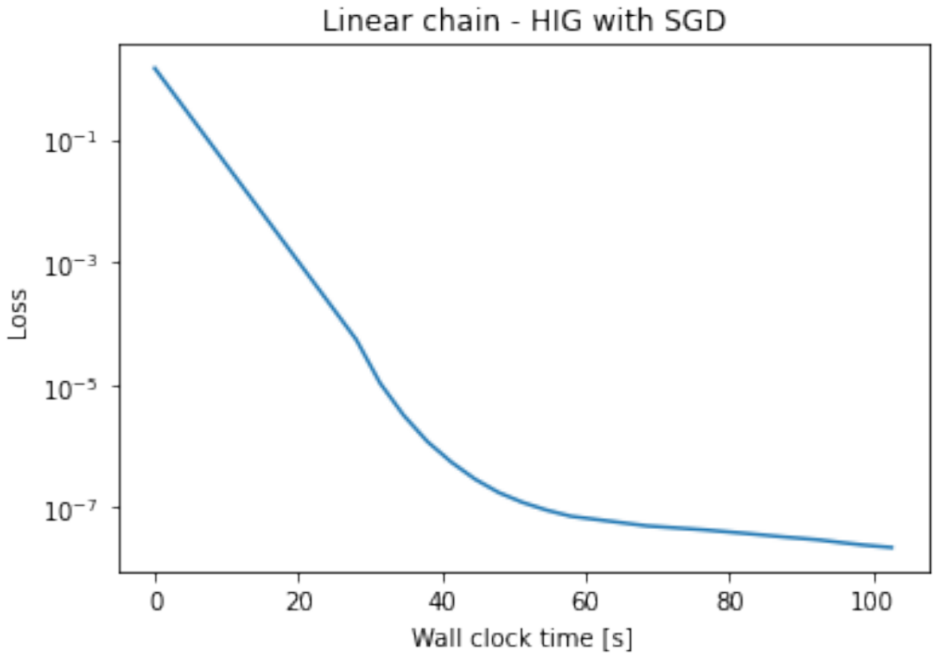
原书输出：

```
Epoch: 0 , wall clock time: 0 , loss: 1.4401766061782837
Epoch: 1 , wall clock time: 28.06755018234253 , loss: 5.44398972124327e-05
Epoch: 2 , wall clock time: 31.38792371749878 , loss: 1.064436037268024e-05
Epoch: 3 , wall clock time: 34.690271854400635 , loss: 3.163525434501935e-06
Epoch: 4 , wall clock time: 37.9914448261261 , loss: 1.1857609933940694e-06
```

41.6 评估

看以秒计的训练损失演化：

```
import matplotlib.pyplot as plt
plt.plot(np.cumsum(time_list),loss_list)
plt.yscale('log')
plt.xlabel('Wall clock time [s]'); plt.ylabel('Loss')
plt.title('Linear chain - '+MODE+' with '+str(OPTIMIZER.__name__))
plt.show()
```



三法起始都有较大的线性段。作者解释，公平起见计入 TensorFlow 初始化，HIG/SIP 初始化明显更复杂；Adam 初始化与每次训练迭代均快得多。[7] 三法单独看都能降损失，更值得看相对表现。下列单元保存训练历史，须分别以三种 MODE 运行，才能生成最终比较图。

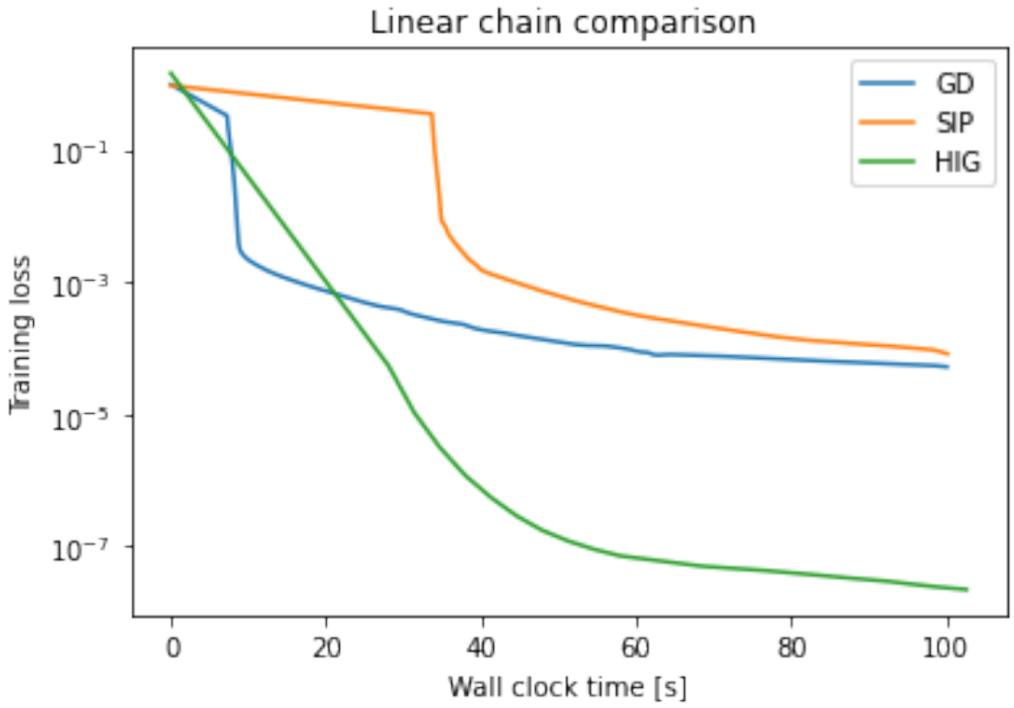
```
path = '/home/'
namet = 'time'
namel = 'loss'
np.savetxt(path+MODE+namet+'.txt',time_list)
np.savetxt(path+MODE+namel+'.txt',loss_list)
```

有三组记录后并排比较：

```
from os.path import exists
# if previous runs are available, compare all 3
if exists(path+'HIG'+namet+'.txt') and exists(path+'HIG'+namel+'.txt') and
↪  exists(path+'SIP'+namet+'.txt') and exists(path+'SIP'+namel+'.txt') and
↪  exists(path+'GD'+namet+'.txt') and exists(path+'GD'+namel+'.txt'):
  lt_hig = np.loadtxt(path+'HIG'+namet+'.txt')
  ll_hig = np.loadtxt(path+'HIG'+namel+'.txt')
  lt_sip = np.loadtxt(path+'SIP'+namet+'.txt')
  ll_sip = np.loadtxt(path+'SIP'+namel+'.txt')
  lt_gd = np.loadtxt(path+'GD'+namet+'.txt')
  ll_gd = np.loadtxt(path+'GD'+namel+'.txt')
  plt.plot(np.cumsum(lt_gd),ll_gd, label="GD")
  plt.plot(np.cumsum(lt_sip),ll_sip, label="SIP")
```

[7] 译注：计时器在初始全训练集 eval 之后启动，且每 epoch 仅计 `epoch_update` 的 duration，不含后续 eval、打印和外层开销。因此图轴是累计被计训练段耗时，不能按文字当成整个程序的完整墙钟时间；首 epoch 内部追踪/初始化会计入，初次 eval 中的工作不会。while 按已累计时长判断，最后一 epoch 还可超过 MAX_TIME。

```
  plt.plot(np.cumsum(lt_hig),ll_hig, label="HIG")
  plt.yscale('log')
  plt.xlabel('Wall clock time [s]'); plt.ylabel('Training loss'); plt.legend()
  plt.title('Linear chain comparison ')
  plt.show()
else:
  print("Run this notebook three times with MODE='HIG|SIP|GD' to produce the final
  ↪  graph")
```



图清楚显示收敛差别：蓝色 GD（Adam）虽作许多更新，作者将其停留在较高损失解释为粗略 Hessian 近似不足以取得高精度。SIP 在此未优于 Adam，原因是线性振子物理较简单、SIP 运行成本较高；延长训练会超过 Adam，但完整求逆又开始有数值问题。[8] HIG 虽每次迭代较慢，但半逆给出很好的更新，很快达到很低损失，精度比另两法高约 4 个数量级。

41.7 下一步

- 最重要的是，本例迄今只看训练表现！这使 notebook 较短，却并非良好实践。作者声称 HIG 对真实测试数据也有效，但建议明确另配测试样本，对三法在测试数据上重作评价。
- 改变物理行为：更多振子、更长或更短时间，或像 HIG 论文一样加入非线性力；最后一种情况需为 SIP 重新设计逆求解器。

[8] 译注：Adam 的梯度矩估计不是一般 Hessian 近似恒等式，沿用作者的启发式解释；延长训练的判断是源作者报告，本节所印有限曲线本身不证明任意延长后的排序。

第四十二章　改进梯度的讨论

现在再次回顾此前方法。深度学习应用大致可分三种：普通可微物理（DP）训练、半逆梯度（HIG）训练、尺度不变物理（SIP）更新。不能简单抛弃其中两种，今后只用一种；比较利弊却能揭示物理深度学习的基本方面。

42.1　处理尺度问题

改进更新的主要动机是处理学习问题的尺度。这并非全新问题，已有许多深度学习算法处理网络训练尺度。但网络与物理仿真结合，既带来新挑战，也提供新处理角度：负面是 PDE 增加高度非线性算子，正面是这些算子在学习中通常没有自由参数，可采用专门方法。HIG 与 SIP 正着眼于此：不将仿真与网络其余部分同等处理（DP），而用专用逆求解器（SIP）或专门数值逆（HIG）取得进展，二者都是尺度不变训练的重要一步。

42.2　计算资源

两者通常训练更复杂、更耗资源。不过，若训练后能反复使用模型，许多应用可很快收回投入：网络运行时与其他训练法所得相同，却常明显更准，普通 Adam/DP 可能难以达到类似精度。例如用网络代理求逆问题时，可能调用大量次；提高精度能在后续阶段节省很多资源。流体中物体减阻的形状优化 [CCHT21] 是潜在例子。

42.3　小结

本部分说明求逆的重要性。一个主要认识是：含 PDE 时，普通网络训练梯度未必最佳，除了局部一阶信息，还能得到更有助于指引优化的信息。即使只求逆物理部分（SIP），也可显著改善学习；专用逆求解器让训练利用高阶信息。

SIP 优点

- 物理仿真的“梯度”信息很准确；
- 收敛和模型表现常明显改善。

SIP 缺点

- 需要至少局部的逆仿真器；

- 只使物理部分具有尺度不变性。

HIG 则回到 Jacobian 的一阶信息，表明没有高阶项，求逆也很有用。它联合求逆网络与物理，考虑小批所有样本以计算最优一阶方向。

HIG 优点

- 同时处理物理模型与网络尺度问题，表现稳健；
- 改善收敛与模型表现。

HIG 缺点

- 需对可能很大的 Jacobian 作 SVD；
- 运行时间和内存开销可能很高。

两者都可能训练出仅靠简单 DP 或监督训练延长时间仍无法取得的性能。因此，若计划经常评价模型，例如随应用一起交付，一次性成本的增加可在长期收回。[1] 这结束物理网络改进学习方法的讨论。该领域仍活跃，有大量新方法空间，现有算法已显示为物理问题专门设计学习算法的潜力。也结束数值仿真作为深度学习组件的重点讨论。源文接称下一章转向统计观点、纳入不确定性。[2]

[1]译注：这些是原作者对适用实验和应用的总结，不是任意网络/PDE 上严格尺度不变、全局最优方向或成本回收的普遍保证；HIG 截断半逆与精确最小二乘方向的差别见第 40 章注。

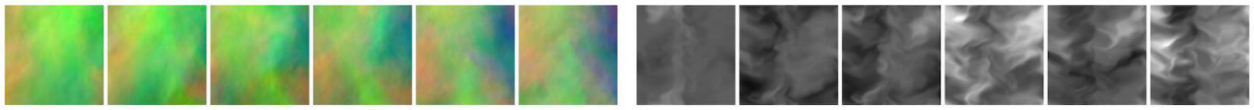
[2]译注：冻结生成式 AI 版已在第 21–33 章集中讨论概率与不确定性；此过渡句保留源版本的章节组织痕迹，下文实际为快速概览主题。

第八部分

快速概览主题

第四十三章　其他主题

接下来简要介绍物理深度学习中其他经典、有趣的主题。这些主题目前不配可执行 notebook，但会指出相关开源实现。



具体包括：

- 模型降阶与时间序列预测：用深度学习在潜空间预测物理系统演化，通常替代数值求解器，并利用专门处理时间序列的技术。
- 生成模型：这是深度学习的独立主题，尤其生成对抗网络已显示强大能力，以两个独立网络形成很有趣的训练方式。
- 无网格方法与非结构网格：它们对经典仿真很重要；下文考察一种在动态粒子表示中使用学习的 Lagrange 方法。

第四十四章　模型降阶与时间序列

许多实用 PDE 求解器的内在挑战是问题维数很高。三维模型 $\mathcal{P}$ 通常需 $\mathcal{O}(n^3)$ 个空间样本，n 为每轴采样数；时间依赖现象还需时间离散，通常随空间维数相应缩放，总样本数约 $\mathcal{O}(n^4)$。高保真应用希望 n 尽可能大，工作量遂迅速增长。[1] 常用降复杂度方法是把空间状态 $\mathbf{s}_t \in \mathbb{R}^{n^3}$ 映到低维潜状态 $\mathbf{c}_t \in \mathbb{R}^m$，$m \ll n^3$。在潜空间推断 $\mathbf{c}_{t+1}$，再解码得 $\mathbf{s}_{t+1}$。必须使 m 足够大以捕捉解流形的重要结构，潜空间预测足够高效，才能在计入编解码成本后仍获加速。常规仿真的未知量爆炸，加上直接计算新状态的超线性复杂度，令其昂贵；小 m 的潜空间运算则可能很快收回成本。[2] 关键是编解码要很好降维，这很适合深度学习。多数实用模型的潜空间演化又无解析闭式解，也适合学习。因此有两个挑战：学好空间编解码，以及准确时间演化。以下依据 Wiewel 等 [WBT19]、[WKA+20]，采用 Kim 等 [KAT+19] 的编解码器。

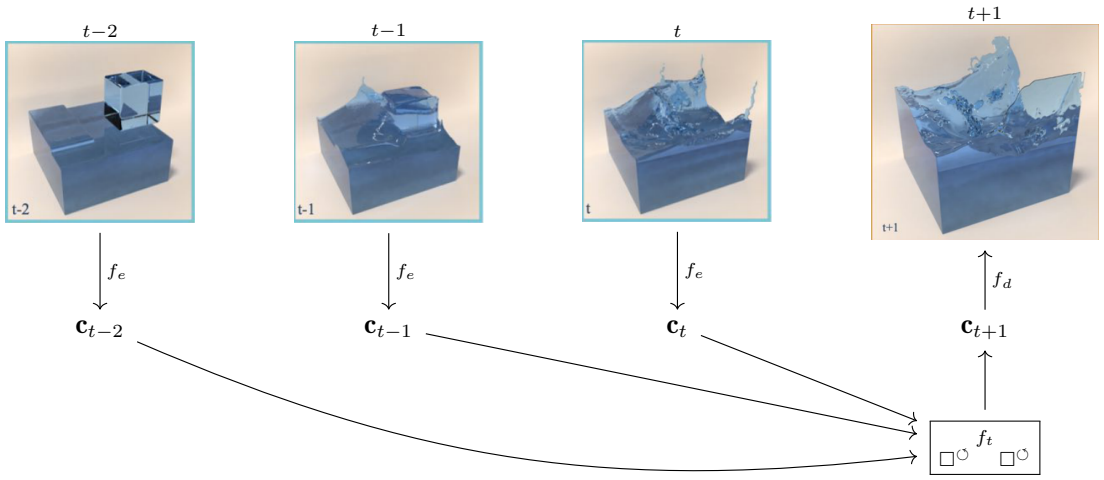


图 44.1: ROM 时间预测：编码器 f_e 编码状态，f_t 预测时间演化，解码器 f_d 恢复完整空间信息。原图 44.1，流体画面来自源图，全部潜变量与运算关系原生重建。

[1] 译注：n^4 假定固定物理时间区间的步数为 $\mathcal{O}(n)$，如某些显式对流 CFL 设置；扩散显式步数可能为 $\mathcal{O}(n^2)$，隐式/自适应方法也不同，不是所有 PDE 的通用复杂度。

[2] 译注：源将直接更新概括为超线性成本，但局部显式模板、适当多重网格等也可近线性；潜空间收益须按具体求解器、误差和编解码成本核算。

44.1 降阶模型

降低计算模型的维数与复杂度，称降阶建模（ROM）或模型降阶，是经典计算主题。传统方法常用主成分分析等构造解空间基；线性结构限制了对复杂非线性解流形的表示。实际有趣解往往高度非线性，因此深度学习的非线性表示备受关注，有望用比传统方法更少自由度达到高精度。典型网络是自编码器：将输入 x 经网络中部附近瓶颈再重建。瓶颈数据为压缩潜表示 $\mathbf{c}$；此前部分是编码器 f_e，此后为解码器 f_d。学习目标为

$$\arg\min_{\theta_e,\theta_d} |f_d(f_e(\mathbf{s};\theta_e);\theta_d) - \mathbf{s}|_2^2,$$

其中 $f_e : \mathbb{R}^{n^3} \to \mathbb{R}^m$、$f_d : \mathbb{R}^m \to \mathbb{R}^{n^3}$，权重分别为 θ_e, θ_d。只需状态 $\mathbf{s}$，兼作输入与参考输出。自编码器常为卷积层堆叠。层细节可灵活选，但编解码器必须能分开使用，不能有绕过瓶颈的连接；否则共享信息会妨碍独立使用，例如解码器必须仅凭 $\mathbf{c}$ 恢复完整状态。[3]

44.1.1 自编码器变体

值得一提的是变分自编码器（VAE）。它在上述结构上增加塑造潜空间分布的损失，使潜状态服从已知分布，从而直接采样，无需先投影到潜空间。通常取正态分布为目标，源文称这使潜空间成为 m 维单位立方体，各维零均值、单位标准差。[4] 这对作为生成模型的解码器很有用：直接采样 $\mathbf{c}$ 再解码成状态。对人脸等自然图像生成很有价值，但仿真更关注潜空间是否方便时间预测，而非是否容易采样。

44.2 时间序列

目标是由一个或多个过去潜状态计算 $t+1$ 的潜状态。最直接的目标是

$$\arg\min_{\theta_p} |f_p(\mathbf{c}_t;\theta_p) - \mathbf{c}_{t+1}|_2^2.$$

预测网络记作 f_p，区别于编解码器（源图 44.1 记 f_t）。任务本质循环：第 i 步是 f_p 评价 i 次的结果，$\mathbf{c}_{t+i} = f_p^{(i)}(\mathbf{c}_t;\theta_p)$。每次评价有误差，故通常需多步训练，使网络“看到”自身潜空间状态随时间漂移。

> **Koopman 算子：**经典动力系统文献常用 Koopman 算子表述数据驱动的未来状态预测，通常以矩阵作线性方法；传统工作寻找合适算子及张成解空间的基。源将上述 f_p 称为“非线性 Koopman 算子”。

[5] 要唯一确定正确下一状态，必须提供适当历史，或让网络内部存储所见历史。前者给 f_p 多于一个 $\mathbf{c}_t$，后者采用循环网络（RNN），如 LSTM、GRU，或源所称较新的注意力 Transformer。源称这些方法均用全连接层，因为潜向量没有空间结构，值看似随机；参数量很快增长，要求 m 较小，正好符合降阶目标。[6]

[3] 译注：这是此处要求独立解码的瓶颈架构约束，不是禁止所有称为自编码器的模型采用跳接。

[4] 译注：标准正态的支持是 $\mathbb{R}^m$，不是单位立方体；有限样本矩接近零均值、单位标准差也不独自保证完整标准正态分布。源几何说法保留并澄清。

[5] 译注：严格 Koopman 算子作用于观测函数空间且本身线性，往往无限维；有限矩阵是近似表示。一般非线性状态预测网络可与 Koopman 方法比较，但不是严格意义的“非线性 Koopman 算子”。

[6] 译注：这是此处非结构潜向量和所选架构的经验说明；时间模型不一概必须全连接，潜空间也可被设计为具有空间或其他结构。

44.3 端到端训练

前面明确分离编解码与时间预测，但实际常偏好联合端到端训练，使各网络适应其他组件。可在完整状态中定义时间预测目标：

$$\arg\min_{\theta_e,\theta_p,\theta_d} |f_d(f_p(f_e(\mathbf{s}_t;\theta_e);\theta_p);\theta_d) - \mathbf{s}_{t+1}|_2^2 .$$

理想上进一步沿时间展开以稳定演化。需同时训练更多权重、处理更多中间状态，训练明显更贵，但通常能降低推断误差。下图为 [WKA+20] 若干预测时刻，该方法还将学习的时间演化与数值平流步结合。因此深度学习可从线性子空间推进到非线性流形，并在所得潜空间中执行时间演化等复杂步骤。

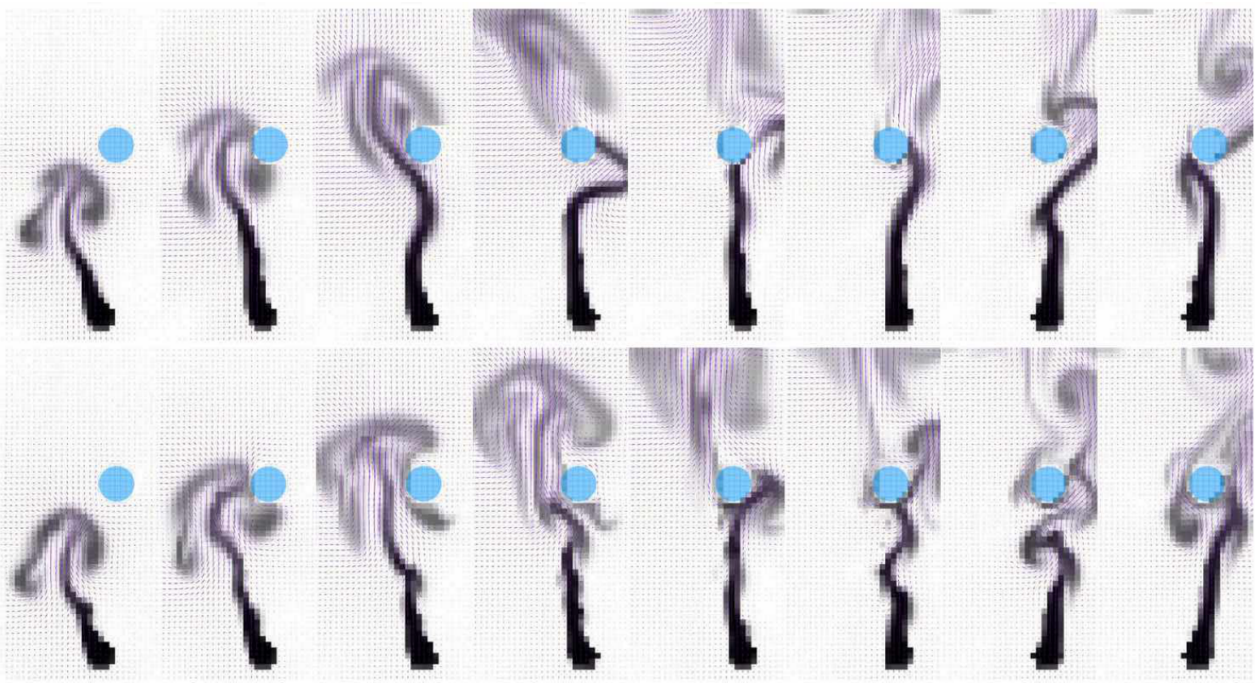


图 44.2: 上排为学习预测，下排为参考仿真。原图 44.2。

44.4 源代码

源推荐潜空间仿真代码，实现端到端编码与预测训练；另一套学习模型降阶代码侧重编解码。两者均为开源，以 TensorFlow 和 mantaflow 作深度学习/流体仿真框架。[7]

[7]译注：冻结 PDF 未给这两处可恢复的链接注释，不猜测项目地址；这里只保留来源描述，未下载或运行外部代码。

第四十五章　非结构网格与无网格方法

所有计算方法都需合适的离散表示。纯整数数据较直接，室温等连续变化量则更难。此前用 Cartesian 网格为占位，重点在离散化之外；本章讨论动态变化、自适应离散化中学习神经算子的益处。

45.1　计算网格类型

如“神经网络架构”所述，离散化常用三类网格：

- **结构网格**：采样点规则排列，连接隐式定义，最简单是稠密 Cartesian 网格；
- **非结构网格**：排列和连接可任意，灵活性通常增加计算成本；
- **无网格或粒子方法**：同样任意排列，但由邻域及距离度量隐式定义连接。

结构网格与图像相似，现有深度学习算法支持好，实现通常简单，可用成熟稳定组件，尤其常规卷积层。但目标函数平滑区与复杂区混合不均时，另两类可能有优势。

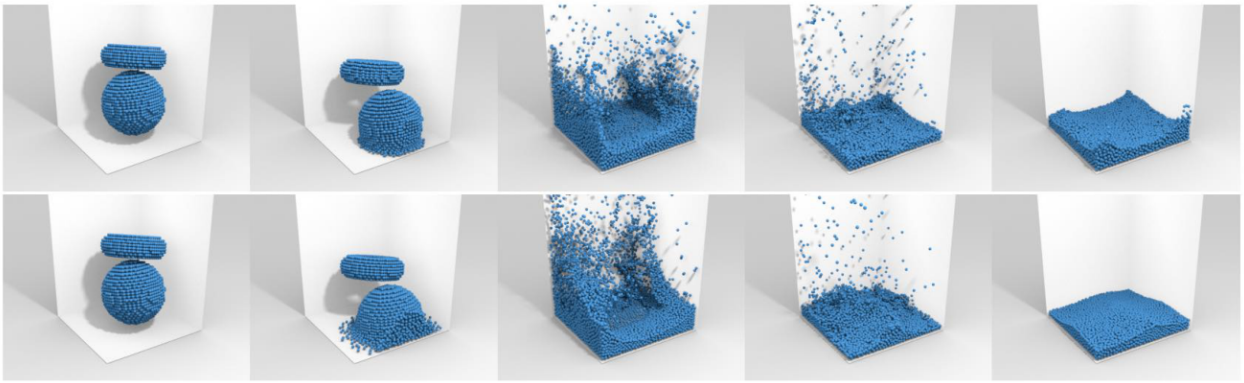


图 45.1: Lagrange 液体仿真：采样点随材料移动，大幅变化。上排学习仿真，下排传统 SPH 求解器。原图 45.1。

45.2　非结构网格与图神经网络

改进计算网格仍是有挑战的研究课题，翼型流动长期提出众多 H、C、O 型网格及变体就是例子。非结构网格提供最大灵活性，但仿真器必须支持。非结构网格与图神经网络（GNN）共享许多性质，后

者把 Cartesian 网格上的深度学习思想推广到图。虽支持渐多，实现仍复杂，任意连接要求图节点间消息传递，通常以全连接层而非卷积实现。以下重点考察粒子方法 [UPTK19]，空间自适应灵活性与 GNN 类似。[SGGP+20] 也曾用于相似目标，但下述方法可用真正卷积算子学习物理关系。

45.3 无网格与粒子方法

动态情况下显式组织连接很难，例如移动材料的 Lagrange 表示中，连接很快过时。适合用灵活、重新计算的连接，运算按采样点（粒子/点）周围空间邻域定义。因无显式网格结构，称无网格方法。不同非结构、图和无网格变体常可相互转换，但粗分仍提示其工作方式。以下以飞溅液体为困难例子，材料大幅运动，分布常很不均。学习粒子仿真的大致过程类似 Cartesian 网格方法：在位置存速度等数据，反复卷积，逐位置构造潜空间。每次卷积读取支撑域内潜内容，产生结果，再用 ReLU 等激活；并行多次得到潜向量，作为下一层输入。经过若干层先扩潜维再收缩，输出加速度等所需量。

45.4 连续卷积

一般离散卷积可写为

$$(f * g)(\mathbf{x}) = \sum_{\tau \in \Omega} f(\mathbf{x} + \tau) g(\tau),$$

其中 τ 为偏移向量，Ω 为滤波函数（通常 g）的支撑。[1] 将这一思想用于粒子/点云，在半径 R 的径向邻域 $\mathcal{N}(\mathbf{x}, R)$ 内位置 $\mathbf{x}_i$ 评价，按 [UPTK19] 定义连续版本：

$$(f * g)(\mathbf{x}) = \sum_{i} f(\mathbf{x}_i) g(\Lambda(\mathbf{x}_i - \mathbf{x})).$$

Λ 将单位球映到单位立方体，允许用简单网格表示卷积核未知量，简化构造与处理。物理应用如流体中，

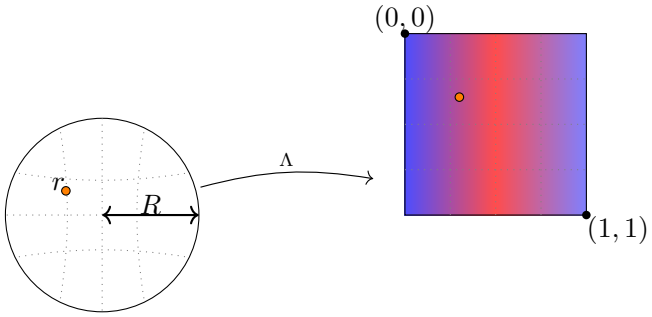


图 45.2: 连续卷积核使用的单位球到单位立方体映射。原图 45.2：r, R, Λ、方向及角坐标原生重建；色块和网格为示意，不补造源未给出的映射解析式。

可引入径向权重 a 使核平滑衰减：

$$(f * g)(\mathbf{x}) = \frac{1}{a_{\mathcal{N}}} \sum_{i} a(\mathbf{x}_i, \mathbf{x}) f(\mathbf{x}_i) g(\Lambda(\mathbf{x}_i - \mathbf{x})), \quad a_{\mathcal{N}} = \sum_{i \in \mathcal{N}(\mathbf{x}, R)} a(\mathbf{x}_i, \mathbf{x}).$$

[1] 译注：冻结 PDF 该式偏移处显示缺字方框，文本提取为 “2/7”；紧随正文明确将偏移定义为 τ，故此处按该定义恢复 τ，不是把 “2/7” 当数值。源采用加偏移约定，若与常见卷积减偏移定义比较，须相应反转核。

a 可灵活选，这里采用

$$a(\mathbf{x}_i, \mathbf{x}) = \begin{cases} \left(1 - \dfrac{\|\mathbf{x}_i - \mathbf{x}\|_2^2}{R^2}\right)^3, & \|\mathbf{x}_i - \mathbf{x}\|_2 < R, \\ 0, & \text{其他情况.} \end{cases}$$

于是单个卷积的学习影响平滑趋零。[2] 为精简架构，可为每个卷积加小全连接层，处理目标粒子自身内容，从而采用较小、偶数尺寸核，例如 4^3 [UPTK19]。

45.5 学习液体动力学

用粒子 Navier–Stokes 求解器生成随机参考数据训练上述架构，可用很小、高效模型取得好精度。作者报告连续卷积相比 GNN 方法权重更少、评价更快。[3] 难例之一是容器中液体最终静止。若不特意加入很多静止例子，训练中此类样本少；而仿真通常要很多步才静止，远多于训练展开步数，故网络没有被明确训练去再现长期静止行为。有趣的附带收益是，训练好的液体网络构成可微求解器，因而支持通过梯度下降优化粘度等输入参数。下图为 [UPTK19] 连续卷积预测例子。

图 45.3: 以学习的连续卷积仿真粒子液体在地形中铺展。原图 45.3。

45.6 源代码

实用连续卷积还需快速收集邻域粒子。源给高效实现与液体动力学训练代码：`https://github.com/intel-isl/DeepLagrangianFluids`，上图即例子。这里保留源地址，未下载或运行。

[2] 译注：求和范围按前文限于邻域；使用归一化式须有 $a_{\mathcal{N}} > 0$，空邻域/全零权重须实现另行处理。若 $R \neq 1$，映到单位球的半径缩放还需包含在 Λ 或其输入约定中，源显示式未明写。

[3] 译注：比较限于所引方法与实验设置，并非任意 GNN 的统一复杂度排序。

第四十六章　生成对抗网络

“概率学习导论”已详述生成式 AI 与扩散建模。完整表示变量 $\mathbf{x}$ 所有可能状态、即捕捉完整分布，是古老问题。DDPM 等之前，生成对抗网络（GAN）已是有力方法。源称研究中 GAN 很大程度被扩散取代，但其方式很有趣，以下介绍原理与用途。[1] 传统上，当解有歧义且无可微物理模型帮助消歧时，GAN 很有用；普通监督训练可能产生不理想平均，GAN 可避免。

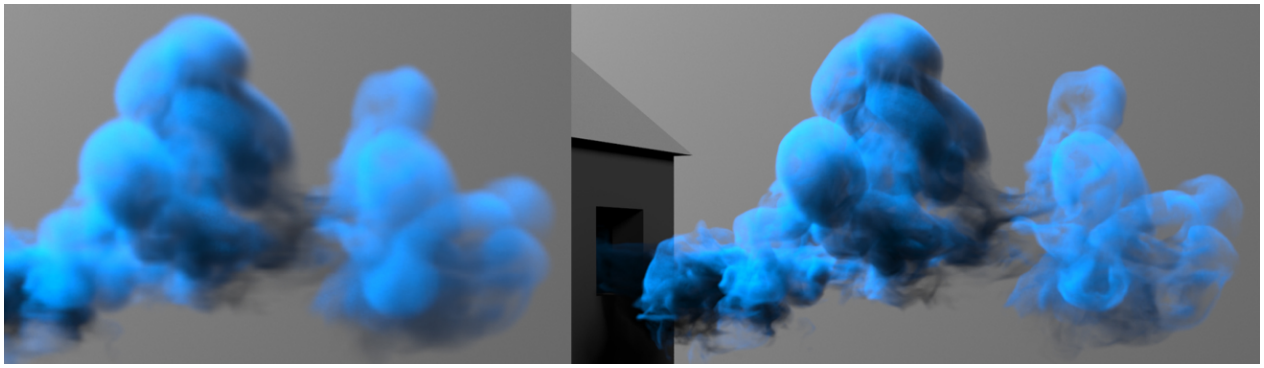


图 46.1: GAN 适用于超分辨率推断等任务，此时可能结果的范围很不确定。原图 46.1。

46.1　极大似然估计

先简述此前未讨论的分类。相较式 (3.2) 回归，分类希望权重 θ 的表示 f 为输入 $\mathbf{x}_i$ 分配概率，在选定类别集上极大化似然：

$$\arg\max_{\theta} \prod_i f(\mathbf{x}_i;\theta). \tag{46.1}$$

实际通常改为负对数似然和：

$$\arg\min_{\theta} -\sum_i \log f(\mathbf{x}_i;\theta).$$

它可理解为最小化数据经验分布与学习分布的 KL 散度，或极大化数据期望 $\mathbb{E}\log f(\mathbf{x}_i;\theta)$，也对应经典分类交叉熵，例如 sigmoid 分类器。广泛使用的交叉熵训练，实质是对式 (46.1) 中概率作极大似然估计。[2]

[1] 译注：这是冻结源版本对研究趋势的判断，非译稿对当前领域的重新调查。

[2] 译注：分类通常拟合 $p_\theta(\text{label} \mid \mathbf{x})$，$i$ 是样本索引，不能将其不加区分当作输入空间密度 $p_\theta(\mathbf{x})$；多类常用 softmax 而非单 sigmoid。经验分布 KL 等价也需相同离散/测度约定，连续经验点质量与连续密度不能直接作有限 KL。

46.2 对抗训练

GAN 中的生成器通常类似自编码器的后半解码器，接收随机潜向量 $\mathbf{z}$，产生期望输出。它不直接按固定目标训练，而用第二网络作损失。这个**判别器**负责分类：区分生成样本与训练集提供的真实样本（源先记 $\mathbf{x}$，以下公式改记 $\mathbf{y}$）。常规 GAN 的判别目标为

$$\arg\min_{\theta_d} -\tfrac{1}{2}\mathbb{E}\log D(\mathbf{y}) - \tfrac{1}{2}\mathbb{E}\log\big(1 - D(G(\mathbf{z}))\big).$$

即真实样本与生成样本的二元交叉熵，真实输出 1，生成输出 0。源称判别器这样“极大化损失”。[3] 生成器则希望样本被判为真实：

$$\arg\min_{\theta_g} -\tfrac{1}{2}\mathbb{E}\log D(G(\mathbf{z})).$$

通常交替一步 D、一步 G。更新 G 时固定 D 的权重，让其梯度指引 G 生成难以区分的样本。D 也是网络，可提供所需导数。[4]

46.3 正则化

GAN 交替耦合两个非线性优化问题，需维持脆弱平衡，训练常难调；否则可能模式坍塌，网络退化到平凡解，耦合训练失效。正则化常是稳定训练的关键。最简单可为生成器加入相对参考数据、带小系数的 L^1 项；也可先监督预训练 G，使其从稳定状态开始，但 D 通常也需一定预训练以维持平衡。[5]

46.4 条件 GAN

物理问题中，只从随机 $\mathbf{z}$ 生成解的普通 GAN 不一定实用，常需参数、边界条件或部分解作为输入。因此用条件 GAN，将 $G(\mathbf{z})$ 改为 $G(\mathbf{x})$，$\mathbf{x}$ 为输入数据。超分辨率就是例子：由稀疏或低分辨输入求高分辨输出。[6]

46.5 歧义解

GAN 可避免歧义数据的不理想平均。例如低分辨观测往往对应无穷多个高分辨解；若数据含多种可能，监督网络会学其均值，平均结果可能不像任何原始解。这是多模态问题，即同一问题有多个同样有效模式；流体分岔就是前述预览例子。[7]

---

[3] 译注：对显示的负交叉熵目标应为最小化；最大化的是相应对数分类似然，源此处措辞与 argmin 冲突。

[4] 译注：这里是常用非饱和生成器目标；固定 D 权重不等于截断其对输入的导数，更新 G 仍须经过 D 向生成样本反传。

[5] 译注：逐样本 L^1 参考项需配对或另定对应规则；无条件 GAN 的任意随机样本与任意真值之间不存在天然逐点配对。模式坍塌也并非总等于某个网络输出完全常值。

[6] 译注：若要表示给定条件下的多个可能解，通常仍需随机潜量，如 $G(\mathbf{x}, \mathbf{z})$；仅确定性的 $G(\mathbf{x})$ 每个条件只能输出一个结果，单靠对抗损失不保证恢复完整条件分布。

[7] 译注：条件均值结论针对确定性网络的平方误差风险最优，需相应数据/优化条件，并非所有监督目标；对抗训练避免模糊平均也不自动保证各模式及概率权重正确。

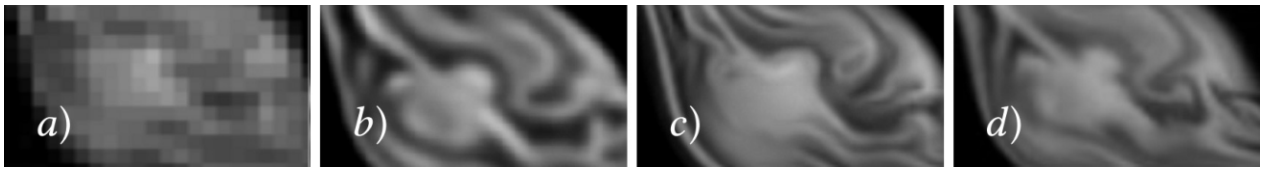


图 46.2: 超分辨率：a) 输入，b) 监督结果，c)GAN 结果，d) 高分辨参考。原图 46.2。

46.6 时空超分辨率

GAN 不只用于空间分辨率。已有工作展示学习自监督可推广到时空解，例如流体超分辨率 [XFCT18]。下图比较时间导数：倒数第二幅时空自监督 GAN 接近最右参考；判别器接收参考时间三

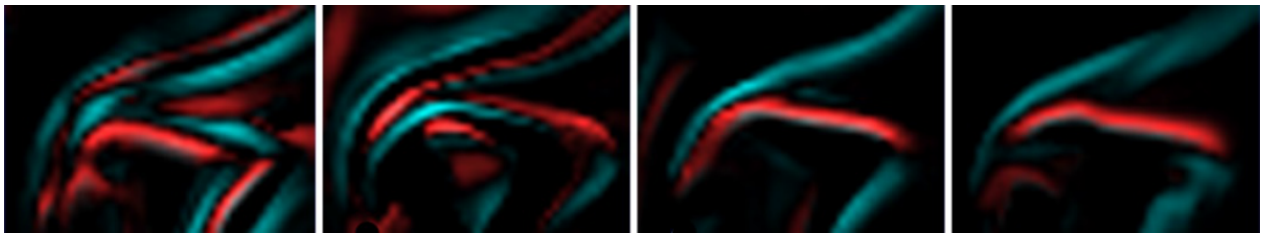


图 46.3: 从左到右：仅空间 GAN（不感知时间）、时间监督学习、时空 GAN、参考解的时间导数。原图 46.3。

元组，学习判断生成解时间演化是否匹配参考。

46.7 物理生成模型

GAN 也能准确捕捉由物理参数参数化的 PDE 解流形 [CTS+21]，例如改变浮力、涡量、边界条件和障碍几何的 Navier–Stokes 解。挑战很大，需扩展的“循环” GAN，让判别器考虑所有物理参数。生成器仅凭数据、无可微求解器显式帮助，仍学到逼真准确解。下图是物理参数 GAN 输出。

46.8 讨论

判别器本质是学习的损失，生成器训练好后推断时可丢弃，因而其资源需求相对不太关键。源文认为若有合理 PDE 模型，采用 GAN 是否合理值得怀疑：若能离散方程并纳入 DP 训练，往往优于用判别器近似 PDE。DP 通过离散仿真器给局部梯度，也可防止样本间不理想平均，所以作者进一步断言 DP 与 GAN 组合不会优于二者单独使用。[8] 不过，当求解器为无梯度黑箱、DP 不可行时，GAN 仍有吸引力。

[8]译注：此为原作者的强经验判断，不是数学推论。物理约束与数据分布判别可提供不同信息；有 DP 不自动保证多模态条件分布或排除组合收益。有限例子不足以证明混合方法必不优于任一单法。

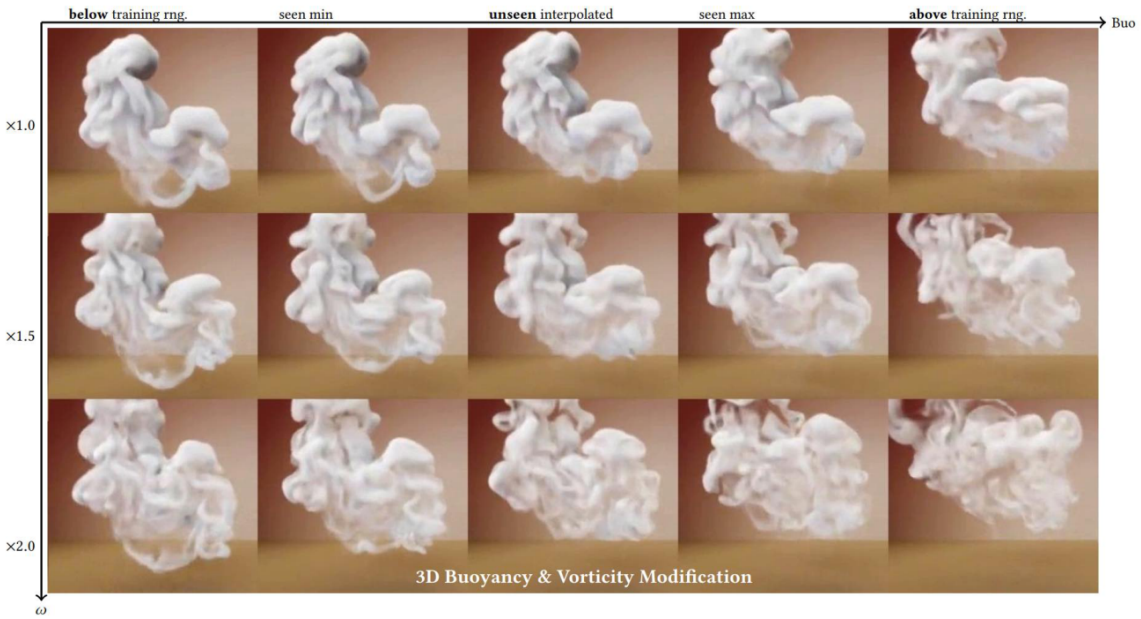


图 46.4: 网络可外推至训练取值范围以外的浮力设置。原图 46.4。

46.9 源代码

因训练设置复杂，只指向外部开源实现。时空 GAN[XFCT18] 在 `https://github.com/thunil/tempoGAN`，还包含 L^1 正则、生成器–判别器平衡等稳定化扩展。此处未下载或运行。

第九部分

书末资料

第四十七章　展望

尽管已深入讨论、展示多样例子，我们仍只刚开始探索物理深度学习的巨大潜力。此前技术不仅有用，作者相信它们有能力在未来数十年重塑计算方法。代码例子表明其中没有魔法：深度学习是处理复杂非线性函数的强大新工具。深度学习并不替代传统数值方法，而是增强它们。二者协作，为仿真与建模开拓新方向。更深层目标是增进人类对世界的理解。作者认为，不应再把网络简单视为无法理解的黑箱，而应视作另一种数值工具：正确使用时，可像传统仿真一样解释。[1] 未来令人期待的挑战之一是改进学习网络的分析，提炼其发现的模式与结构，从解流形获得基本、可由人理解的认识。可微仿真的未来不仅要更好预测，也希望以新方式揭示物理世界的隐藏秩序。

47.1　若干具体方向

除长期愿景，还有许多近期工作。Burgers 与 Navier–Stokes 虽非平凡，也只是可改进 PDE 模型与算子的一小部分。例如：

- **化学反应 PDE**：多物种相互作用产生复杂行为。可训练快速预测实验或工业过程、动态调整控制参数以稳定过程的模型，实现实时智能控制。
- **等离子体仿真**：与涡量流体有相似性，但电磁相互作用增加复杂度，适合探索深度学习，尤其聚变实验与能源装置中的可微物理。
- **天气与气候建模**：这是人类重大科学挑战，多尺度流动与大量环境因素交织。深度学习增强数值仿真，既有望改善预报，也可深化对地球动力学的认识。

47.2　结束语

这些仅是少数例子，却展示可微物理与深度学习的广泛机会。仍有大量令人期待的研究，未来几年、几十年不会无聊。[2]

[1] 译注：这里是作者的研究立场与愿景；可解释程度仍依模型、任务、分析方法和验证证据，并非所有训练网络均已具有传统模型的可解释保证。

[2] 译注：源末尾两处图标在冻结 PDF 中不能可靠辨认，提取为 “2/7 2/7”；不将其误作数值分数，也不自行补造具体表情。

第四十八章　记号与缩写

48.1　数学记号

符号	含义
A	矩阵
η	学习率或步长
Γ	计算域 Ω 的边界
f^*	待逼近的一般函数，通常未知
f	f^* 的近似版本
Ω	计算域
$\mathcal{P}^*$	连续/理想物理模型
$\mathcal{P}$	离散物理模型，PDE
θ	神经网络参数
t	时间维度
$\mathbf{u}$	向量速度
x	网络输入或空间坐标
y	网络输出
y^*	学习目标：真值、参考或观测数据

48.2 主要缩写汇总

缩写	含义
AI	近来各种场合冒出的神秘热词（原作者的玩笑说法）
BNN	Bayesian 神经网络
CNN	卷积神经网络，一种特定架构
DDPM	去噪扩散概率模型，一种扩散建模变体
DL	深度学习
FM	流匹配，一种扩散建模变体
FNO	Fourier 神经算子，一种特定架构
GD	（最速）梯度下降
MLP	多层感知机，具有全连接层的神经网络
NN	一般神经网络，区别于 CNN、MLP 等具体类型
PDE	偏微分方程
PBDL	基于物理的深度学习
SGD	随机梯度下降

译注：局部记号 表按原书汇总保留；第 36–39 章逆问题中网络输入为 y、输出为 x，与表中通用约定相反，应优先采用各章明确给出的局部定义。

参考文献

[AAC+19] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, and others. Solving rubik's cube with a robot hand. arXiv:1910.07113, 2019.

[CCHT21] Li-Wei Chen, Berkay A Cakal, Xiangyu Hu, and Nils Thuerey. Numerical investigation of minimum drag profiles in laminar flow using deep learning surrogates. Journal of Fluid Mechanics, 2021. URL: https://ge.in.tum.de/publications/2020-chen-dl-surrogates/.

[CT22] Li-Wei Chen and Nils Thuerey. Towards high-accuracy deep learning inference of compressible turbulent flows over aerofoils. In Computers and Fluids. 2022. URL: https://ge.in.tum.de/publications/.

[CRBD19] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. arXiv:1806.07366, 2019.

[CTS+21] Mengyu Chu, Nils Thuerey, Hans-Peter Seidel, Christian Theobalt, and Rhaleb Zayer. Learning Meaningful Controls for Fluids. ACM Trans. Graph., 2021. URL: https://people.mpi-inf.mpg.de/~mchu/gvv-den2vel/den2vel.html.

[CKM+23] Hyungjin Chung, Jeongsol Kim, Michael Mccann, Marc Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In International Conference on Learning Representations. 2023.

[Gol90] H Goldstine. A history of scientific computing. ACM, 1990.

[HKT19] Philipp Holl, Vladlen Koltun, and Nils Thuerey. Learning to control pdes with differentiable physics. In International Conference on Learning Representations. 2019. URL: https://ge.in.tum.de/publications/2020-iclr-holl/.

[HKT22] Philipp Holl, Vladlen Koltun, and Nils Thuerey. Scale-invariant physics for deep learning. Advances in Neural Information Processing Systems, 35:5390–5403, 2022. URL: https://arxiv.org/abs/2109.15048.

[HT23] Benjamin Holzschuh and Nils Thuerey. Solving inverse physics problems with score matching. Advances in Neural Information Processing Systems (NeurIPS), 2023.

[HVT23] Benjamin Holzschuh, Simona Vegetti, and Nils Thuerey. Solving inverse physics problems with score matching. Advances in Neural Information Processing Systems (NeurIPS), 2023.

[KAT+19] Byungsoo Kim, Vinicius C Azevedo, Nils Thuerey, Theodore Kim, Markus Gross, and Barbara Solenthaler. Deep Fluids: A Generative Network for Parameterized Fluid Simulations. Comp. Grap. Forum, 38(2):12, 2019. URL: `http://www.byungsoo.me/project/deep-fluids/`.

[KB14] Diederik P Kingma and Jimmy Ba. Adam: a method for stochastic optimization. arXiv:1412.6980, 2014.

[KPB20] Ivan Kobyzev, Simon JD Prince, and Marcus A Brubaker. Normalizing flows: an introduction and review of current methods. IEEE transactions on pattern analysis and machine intelligence, 2020.

[KSA+21] Dmitrii Kochkov, Jamie A Smith, Ayya Alieva, Qing Wang, Michael P Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. Proceedings of the National Academy of Sciences, 2021.

[KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems. 2012.

[LKA+21] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. M. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. ICLR, 2021.

[LPT25] Mario Lino, Tobias Pfaff, and Nils Thuerey. Learning distributions of complex fluid simulations with diffusion graph networks. In International Conference on Learning Representations. 2025.

[LCBH+22] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. arXiv:2210.02747, 2022.

[LCT22] Bjoern List, Liwei Chen, and Nils Thuerey. Learned turbulence modelling with differentiable fluid solvers. In Journal of Fluid Mechanics (929/25). 2022. URL: `https://ge.in.tum.de/publications/`.

[LGL22] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: learning to generate and transfer data with rectified flow. arXiv:2209.03003, 2022.

[MLA+19] Rajesh Maingi, Arnold Lumsdaine, Jean Paul Allain, Luis Chacon, SA Gourlay, and others. Summary of the fesac transformative enabling capabilities panel report. Fusion Science and Technology, 75(3):167–177, 2019.

[OMalleyBK+16] Peter JJ O' Malley, Ryan Babbush, Ian D Kivlichan, Jonathan Romero, Jarrod R McClean, Rami Barends, Julian Kelly, Pedram Roushan, Andrew Tranter, Nan Ding, and others. Scalable quantum simulation of molecular energies. Physical Review X, 6(3):031007, 2016.

[PUKT22] Lukas Prantl, Benjamin Ummenhofer, Vladlen Koltun, and Nils Thuerey. Guaranteed conservation of momentum for learning particle-based fluid dynamics. Advances in Neural Information Processing Systems, 2022.

[Qur19] Mohammed Al Quraishi. Alphafold at casp13. Bioinformatics, 35(22):4862–4865, 2019.

[RWC+19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI blog, 1(8):9, 2019.

[RPK19] Maziar Raissi, Paris Perdikaris, and George Karniadakis. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378:686–707, 2019.

[RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: convolutional networks for biomedical image segmentation. In International Conference on Medical Image Computing and Computer-Assisted Intervention. 2015.

[SGGP+20] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In International Conference on Machine Learning, 8459–8468. 2020.

[SHT22] Patrick Schnell, Philipp Holl, and Nils Thuerey. Half-inverse gradients for physical deep learning. In ICLR. 2022. URL: `https://github.com/tum-pbs/half-inverse-gradients`.

[SML+15] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438, 2015.

[SWD+17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[SSS+17] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, and others. Mastering the game of Go without human knowledge. Nature, 2017.

[Sto14] Thomas Stocker. Climate change 2013: the physical science basis: Working Group I contribution to the Fifth assessment report of the Intergovernmental Panel on Climate Change. Cambridge university press, 2014.

[SB18] Richard S Sutton and Andrew G Barto. Reinforcement learning: An introduction. MIT press, 2018.

[TWPH20] Nils Thuerey, Konstantin Weissenow, Lukas Prantl, and Xiangyu Hu. Deep learning methods for reynolds-averaged navier–stokes simulations of airfoil flows. AIAA Journal, 58(1):25–36, 2020. URL: https://ge.in.tum.de/publications/2018-deep-flow-pred/.

[TSSP17] Jonathan Tompson, Kristofer Schlachter, Pablo Sprechmann, and Ken Perlin. Accelerating eulerian fluid simulation with convolutional networks. In Proceedings of Machine Learning Research, 3424–3433. 2017.

[UBH+20] Kiwon Um, Robert Brand, Philipp Holl, Raymond Fei, and Nils Thuerey. Solver-in-the-loop: learning from differentiable physics to interact with iterative pde-solvers. Advances in Neural Information Processing Systems, 2020. URL: https://ge.in.tum.de/publications/2020-um-solver-in-the-loop/.

[UPTK19] Benjamin Ummenhofer, Lukas Prantl, Nils Thuerey, and Vladlen Koltun. Lagrangian fluid simulation with continuous convolutions. In International Conference on Learning Representations. 2019. URL: https://ge.in.tum.de/publications/2020-ummenhofer-iclr/.

[Vin11] Pascal Vincent. A connection between score matching and denoising autoencoders. Neural computation, 23(7):1661–1674, 2011.

[WBT19] Steffen Wiewel, Moritz Becher, and Nils Thuerey. Latent-space Physics: Towards Learning the Temporal Evolution of Fluid Flow. Comp. Grap. Forum, 38(2):12, 2019. URL: https://ge.in.tum.de/publications/latent-space-physics/.

[WKA+20] Steffen Wiewel, Byungsoo Kim, Vinicius C Azevedo, Barbara Solenthaler, and Nils Thuerey. Latent space subdivision: stable and controllable time predictions for fluid flow. Symposium on Computer Animation, 2020. URL: https://ge.in.tum.de/publications/2020-lssubdiv-wiewel/.

[XFCT18] You Xie, Erik Franz, Mengyu Chu, and Nils Thuerey. tempoGAN: A Temporally Coherent, Volumetric GAN for Super-resolution Fluid Flow. ACM Trans. Graph., 2018. URL: https://ge.in.tum.de/publications/tempogan/.

[YK15] Fisher Yu and Vladlen Koltun. Multi-scale context aggregation by dilated convolutions. arXiv preprint arXiv:1511.07122, 2015.

译注：书目范围 以上逐项保留冻结 PDF 书目。HT23 与 HVT23 标题相同但作者列表不同，HKT19、UPTK19 等条目的年份与所列 URL 路径年份也有差别；均未静默统一。原书内文个别引文缩写与此表不完全一致，按原样保留并以源文对应关系核对，不补造缺失文献。